World of Protocols

A comprehensive quick-reference for 274 protocols

From physical layer to application layer

Generated on March 22, 2026

https://github.com/willswire/world-of-protocols

Table of Contents

Automotive 16

DoIP / UDS -- Diagnostics over Internet Protocol / Unified Diagnostic Services 17

SAE J1939 22

OBD-II -- On-Board Diagnostics II 27

SOME/IP -- Scalable service-Oriented MiddlewarE over IP 31

Aviation 36

ACARS -- Aircraft Communications Addressing and Reporting System 37

ASTERIX -- All Purpose Structured Eurocontrol Surveillance Information Exchange 41

CPDLC -- Controller-Pilot Data Link Communications 46

METAR / TAF -- Aviation Weather Reports 50

Mode S / Mode A/C / TCAS -- Secondary Surveillance Radar 56

Bus & IC Protocols 62

ARINC 429 -- Aeronautical Radio, Incorporated 63

Art-Net 67

CAN -- Controller Area Network 71

DALI -- Digital Addressable Lighting Interface 75

DMX512 -- Digital Multiplex 80

HDMI CEC -- Consumer Electronics Control 83

I2C -- Inter-Integrated Circuit 88

I2S -- Inter-IC Sound 91

JTAG / SWD 94

MIDI -- Musical Instrument Digital Interface 100

MIL-STD-1553 -- Military Standard 1553B 104

1-Wire 108

PCIe -- PCI Express 112

sACN / E1.31 -- Streaming Architecture for Control Networks 117

SPI -- Serial Peripheral Interface 121

USB -- Universal Serial Bus 124

Data & Model Formats 128

GGUF -- GPT-Generated Unified Format 129

HDF5 -- Hierarchical Data Format 5 133

iCalendar -- Internet Calendaring and Scheduling 137

MCAP -- Multimodal Container for Autonomous Platforms 144

ONNX -- Open Neural Network Exchange 149

Apache Parquet 152

Protocol Buffers -- Protobuf 155

Safetensors 161

TFRecord 165

vCard -- Electronic Business Card 169

Database Wire Protocols 175

Memcached Protocol 176

MongoDB Wire Protocol 181

MySQL Protocol 185

PostgreSQL Wire Protocol 189

Redis Protocol -- RESP 194

TDS -- Tabular Data Stream 197

Email 203

DANE -- DNS-Based Authentication of Named Entities 204

DKIM -- DomainKeys Identified Mail 207

DMARC -- Domain-based Message Authentication, Reporting, and Conformance 210

IMAP -- Internet Message Access Protocol 213

JMAP -- JSON Meta Application Protocol 217

POP3 -- Post Office Protocol version 3 221

SMTP -- Simple Mail Transfer Protocol 223

SPF -- Sender Policy Framework 227

Encoding & Symbology 230

Code 128 231

Data Matrix 233

QR Code -- Quick Response Code 237

UPC / EAN -- Universal Product Code / European Article Number 241

File Sharing & Transfer 245

FTP -- File Transfer Protocol 246

NetBIOS -- Network Basic Input/Output System 249

NFS -- Network File System 252

SCP -- Secure Copy Protocol 255

SFTP -- SSH File Transfer Protocol 258

SMB (Server Message Block) / CIFS 263

TFTP -- Trivial File Transfer Protocol 267

Healthcare 271

DICOM -- Digital Imaging and Communications in Medicine 272

HL7 (Health Level Seven) -- v2 Messaging & FHIR 277

High-Performance Computing 284

MPI -- Message Passing Interface 285

NCCL -- NVIDIA Collective Communication Library 289

RDMA / RoCE / InfiniBand -- Remote Direct Memory Access 292

Industrial & SCADA 296

BACnet -- Building Automation and Control Networks 297

DNP3 -- Distributed Network Protocol 3 303

EtherNet/IP -- EtherNet/Industrial Protocol 309

IEC 60870-5-104 312

IEC 61850 318

ISO 8583 -- Financial Transaction Card Messaging 323

KNX 329

Modbus 334

OCPP -- Open Charge Point Protocol 337

OPC DA -- OPC Data Access / OPC Classic 341

OPC UA -- Open Platform Communications Unified Architecture 346

PROFIBUS -- Process Field Bus 351

PROFINET -- Process Field Network 354

Data Link Layer 359

ARP -- Address Resolution Protocol 360

ATM -- Asynchronous Transfer Mode 362

CDP -- Cisco Discovery Protocol 366

Ethernet -- IEEE 802.3 370

Frame Relay 373

HDLC -- High-Level Data Link Control 378

LACP -- Link Aggregation Control Protocol 382

LLDP -- Link Layer Discovery Protocol 384

PPP -- Point-to-Point Protocol 387

STP / RSTP / MSTP -- Spanning Tree Protocols 390

IEEE 802.1Q -- VLAN Tagging 394

Media & Broadcast 397

AES67 -- High-Performance Audio over IP 398

AVB / TSN -- Audio Video Bridging / Time-Sensitive Networking 403

Dante -- Digital Audio Network Through Ethernet 409

NDI -- Network Device Interface 415

ONVIF -- Open Network Video Interface Forum 418

SMPTE ST 2022 -- Professional Video over IP 425

SMPTE ST 2110 -- Professional Media over Managed IP Networks 429

UPnP / DLNA -- Universal Plug and Play / Digital Living Network Alliance 435

Messaging & Pub/Sub 443

AMQP -- Advanced Message Queuing Protocol 444

IRC -- Internet Relay Chat 447

Kafka Protocol 453

Matrix 455

MQTT -- Message Queuing Telemetry Transport 460

NATS 464

XMPP -- Extensible Messaging and Presence Protocol 466

ZeroMQ -- ØMQ / ZMTP 470

Mobile Sync 476

DeltaSync 477

Exchange ActiveSync -- EAS 479

MM5 -- MMS MMSC-to-HLR Interface 483

SMPP -- Short Message Peer-to-Peer 485

SyncML / OMA DS & DM -- Synchronization Markup Language 489

WBXML -- WAP Binary XML 494

Network Management & Monitoring 498

gNMI -- gRPC Network Management Interface 499

NETCONF -- Network Configuration Protocol 503

NetFlow / IPFIX -- IP Flow Information Export 509

OpenFlow 513

OTLP -- OpenTelemetry Protocol 520

PTP -- Precision Time Protocol 523

RESTCONF 528

sFlow 533

SMPTE Timecode -- SMPTE ST 12M / LTC / VITC 538

SNMP -- Simple Network Management Protocol 542

Syslog 545

TR-069 (CWMP) / TR-369 (USP) 549

Naming, Addressing & Discovery 556

BOOTP / PXE -- Bootstrap Protocol / Preboot Execution Environment 557
DHCP -- Dynamic Host Configuration Protocol 561
DHCP Options Reference 563
DHCPv6 -- Dynamic Host Configuration Protocol for IPv6 569
DNS -- Domain Name System 573
DNSSEC -- DNS Security Extensions 575
DoH -- DNS over HTTPS 580
DoT -- DNS over TLS 584
LDAP -- Lightweight Directory Access Protocol 587
mDNS / DNS-SD -- Multicast DNS / DNS Service Discovery 592
NTP -- Network Time Protocol 595
Wake-on-LAN -- WOL 598

Network Layer 602

GRE -- Generic Routing Encapsulation 603
ICMP -- Internet Control Message Protocol 605
ICMPv6 -- Internet Control Message Protocol for IPv6 607
IGMP -- Internet Group Management Protocol 612
IPv4 -- Internet Protocol version 4 614
IPsec -- Internet Protocol Security 617
IPv6 -- Internet Protocol version 6 621
IS-IS -- Intermediate System to Intermediate System 623
MPLS -- Multiprotocol Label Switching 626
OSPF -- Open Shortest Path First 629

VRRP -- Virtual Router Redundancy Protocol 632

Remote Access 635

SSH -- Secure Shell 636

Telnet 639

Robotics 643

CANopen 644

DDS / ROS 2 -- Data Distribution Service 648

EtherCAT -- Ethernet for Control Automation Technology 652

MAVLink -- Micro Air Vehicle Link 655

ROS 1 -- Robot Operating System -- TCPROS/UDPROS 659

Routing 664

BGP -- Border Gateway Protocol 665

EIGRP -- Enhanced Interior Gateway Routing Protocol 668

HSRP -- Hot Standby Router Protocol 672

LDP -- Label Distribution Protocol 676

PIM -- Protocol Independent Multicast 680

RIP -- Routing Information Protocol 683

RSVP -- Resource Reservation Protocol 687

SRv6 -- Segment Routing over IPv6 690

Security & Authentication 694

802.1X / EAP -- Port-Based Network Access Control 695

ACME -- Automatic Certificate Management Environment 698

FIDO2 / WebAuthn -- Fast Identity Online 2 / Web Authentication 701

Kerberos 706

NTLM -- NT LAN Manager 710

OAuth 2.0 / OpenID Connect 715

RADIUS -- Remote Authentication Dial-In User Service 722

SAML 2.0 -- Security Assertion Markup Language 726

SCIM -- System for Cross-domain Identity Management 732

Signal Protocol -- Double Ratchet 737

TACACS+ -- Terminal Access Controller Access-Control System Plus 741

TLS -- Transport Layer Security 746

WireGuard 749

WPA2 / WPA3 -- Wi-Fi Protected Access 752

X.509 Certificates 755

Serial Protocols 763

RS-232 -- EIA/TIA-232 764

RS-422 -- EIA/TIA-422 767

RS-485 -- EIA/TIA-485 769

UART -- Universal Asynchronous Receiver/Transmitter 772

XMODEM / YMODEM 775

ZMODEM 779

Space & Satellite 784

CCSDS -- Consultative Committee for Space Data Systems 785

GPS Navigation Message -- L1 C/A 789

LRIT / HRIT -- Weather Satellite Downlink 794

RTCM 3.x -- Differential GNSS Corrections 799

Storage Networking 804

Fibre Channel -- FC 805

iSCSI -- Internet Small Computer Systems Interface 809

NVMe-oF -- NVMe over Fabrics 813

Telecommunications 818

5G NR -- New Radio 819

Diameter 822

DOCSIS -- Data Over Cable Service Interface Specification 826

E1 830

EPON / 10G-EPON -- Ethernet Passive Optical Network 834

GPON / XGS-PON -- Gigabit Passive Optical Network 839

GSM -- Global System for Mobile Communications 845

ISDN -- Integrated Services Digital Network 848

LTE -- Long Term Evolution -- 4G 851

5G NAS -- Non-Access Stratum 855

NGAP -- Next Generation Application Protocol 860

PFCP -- Packet Forwarding Control Protocol 864

50G-PON / NG-PON2 -- Next-Generation PON 868

5G SBI -- Service-Based Interface 873

SMS / MMS -- Short Message Service / Multimedia Messaging Service 878

SS7 -- Signaling System No. 7 884

T1 -- DS1 887

USSD -- Unstructured Supplementary Service Data 890

WAP -- Wireless Application Protocol 894

xDSL -- Digital Subscriber Line 901

Transport Layer 906

DCCP -- Datagram Congestion Control Protocol 907

QUIC 911

SCTP -- Stream Control Transmission Protocol 915

TCP -- Transmission Control Protocol 917

UDP -- User Datagram Protocol 920

Tunneling & VPN 923

EVPN -- Ethernet VPN 924

Geneve -- Generic Network Virtualization Encapsulation 928

GTP -- GPRS Tunneling Protocol 930

L2TP -- Layer 2 Tunneling Protocol 933

OpenVPN 936

SOCKS5 -- Socket Secure Protocol Version 5 939

Tor -- The Onion Router 944

VXLAN -- Virtual Extensible LAN 948

VoIP & Real-Time 953

DTLS -- Datagram Transport Layer Security 954

H.323 -- Packet-Based Multimedia Communications 956

IAX -- Inter-Asterisk eXchange 959

ICE -- Interactive Connectivity Establishment 962

MGCP / H.248 -- Media Gateway Control 966

RTCP -- RTP Control Protocol 969

RTP -- Real-time Transport Protocol 973

RTSP -- Real Time Streaming Protocol 976

SDP -- Session Description Protocol 977

SIMPLE -- SIP for Instant Messaging and Presence Leveraging Extensions 981

SIP -- Session Initiation Protocol 986

Skinny -- SCCP -- Skinny Client Control Protocol 990

SRTP -- Secure Real-time Transport Protocol 993

STUN -- Session Traversal Utilities for NAT 995

TURN -- Traversal Using Relays around NAT 999

WebRTC -- Web Real-Time Communication 1002

Web & API 1006

BitTorrent 1007

CalDAV -- Calendaring Extensions to WebDAV 1011

CardDAV -- vCard Extensions to WebDAV 1016

CoAP -- Constrained Application Protocol 1021

GraphQL 1026

gRPC -- gRPC Remote Procedure Call 1030

HLS / MPEG-DASH -- Adaptive Bitrate Streaming 1033

HTTP -- Hypertext Transfer Protocol 1037

IPP -- Internet Printing Protocol 1040

LwM2M -- Lightweight Machine-to-Machine 1044

PROXY Protocol 1049

Raft Consensus Protocol 1054

S3 API -- Amazon Simple Storage Service 1057

Server-Sent Events -- SSE 1062

WebSocket 1065

WebTransport 1068

Wireless & Radio 1073

6LoWPAN -- IPv6 over Low-Power Wireless Personal Area Networks 1074

IEEE 802.11 -- Wi-Fi 1078

IEEE 802.11s -- Wi-Fi Mesh Networking 1081

ADS-B -- Automatic Dependent Surveillance-Broadcast 1085

ANT / ANT+ 1089

BLE -- Bluetooth Low Energy 1095

Bluetooth / BLE -- Bluetooth Low Energy 1104

DMR -- Digital Mobile Radio 1107

DVB-S / DVB-S2 / DVB-S2X -- Digital Video Broadcasting - Satellite 1111

EnOcean 1116

LoRa PHY -- Long Range Physical Layer 1122

LoRaWAN -- Long Range Wide Area Network 1127

Matter 1132

NFC -- Near Field Communication 1137

TETRA -- Terrestrial Trunked Radio 1140

Thread 1145

UWB -- Ultra-Wideband 1149

Zigbee 1154

Z-Wave 1158

Automotive

Protocols used in vehicle networks, from in-car buses to V2X communication.

4 protocols in this section

DoIP / UDS -- Diagnostics over Internet Protocol / Unified Diagnostic Services

Standard: ISO 13400 (DoIP) / ISO 14229 (UDS) | **Layer:** Application (over TCP/UDP) | **Wireshark filter:** `doip`

DoIP is the transport layer for carrying automotive diagnostic messages over Ethernet/IP, replacing the CAN-based diagnostic transport (ISO 15765). UDS defines the actual diagnostic services (read data, flash ECUs, clear faults, security access) that ride on top of DoIP. Together, DoIP+UDS enable high-speed diagnostics, ECU reprogramming, and remote vehicle access over standard IP infrastructure. A diagnostic tester discovers DoIP entities on the network via UDP broadcast, activates a routing path through the vehicle gateway, and then exchanges UDS request-response messages over a TCP connection. DoIP is mandatory for vehicles using automotive Ethernet and is central to over-the-air (OTA) update architectures.

DoIP Header

[Diagram: see online version]

Every DoIP message starts with an 8-byte header. The inverse version field is the bitwise NOT of the protocol version (e.g., version 0x02, inverse 0xFD) for integrity checking.

Key Fields

Field	Size	Description
Protocol Version	8 bits	DoIP protocol version (0x02 = ISO 13400-2:2019, 0xFF = default/discovery)
Inverse Version	8 bits	Bitwise NOT of protocol version (~version)
Payload Type	16 bits	Identifies the DoIP message type
Payload Length	32 bits	Length of payload following the header (bytes)

DoIP Payload Types

Type (hex)	Name	Transport	Description
0x0000	Generic Negative Ack	TCP/UDP	Protocol error (incorrect pattern, unknown type, etc.)
0x0001	Vehicle Identification Request	UDP	Tester broadcasts to discover DoIP entities
0x0002	Vehicle ID Request with EID	UDP	Request by Entity ID (MAC address)
0x0003	Vehicle ID Request with VIN	UDP	Request by VIN

0x0004	Vehicle Announcement / ID Response	UDP	DoIP entity responds with VIN, addresses, EID
0x0005	Routing Activation Request	TCP	Tester requests diagnostic access
0x0006	Routing Activation Response	TCP	Gateway confirms or denies access
0x0007	Alive Check Request	TCP	Gateway checks if tester is still connected
0x0008	Alive Check Response	TCP	Tester confirms it is alive
0x4001	DoIP Entity Status Request	UDP	Query entity capacity
0x4002	DoIP Entity Status Response	UDP	Entity reports max sockets, currently open sockets
0x4003	Diagnostic Power Mode Request	UDP	Query diagnostic power mode
0x4004	Diagnostic Power Mode Response	UDP	Entity reports power mode readiness
0x8001	Diagnostic Message	TCP	Carries UDS request/response payload
0x8002	Diagnostic Message Positive Ack	TCP	Gateway acknowledges receipt, routing to target ECU
0x8003	Diagnostic Message Negative Ack	TCP	Gateway cannot route message (unknown target, etc.)

Routing Activation Response Codes

Code	Name	Description
0x00	Denied unknown source	Source address not recognized
0x01	Denied all sockets active	No TCP sockets available
0x02	Denied source already active	Source address already registered on different socket
0x03	Denied source already registered	Source already has an active routing on this socket
0x04	Denied missing authentication	Authentication required but not provided
0x05	Denied rejected confirmation	Confirmation step was rejected
0x06	Denied unsupported activation type	Activation type not supported
0x10	Success	Routing activation accepted
0x11	Success, confirmation required	Routing activation requires additional confirmation

Vehicle Discovery

[Diagram: see online version]

Routing Activation

[Diagram: see online version]

UDS Over DoIP

Diagnostic Message Structure

[Diagram: see online version]

Field	Size	Description
Source Address	16 bits	Tester logical address (e.g., 0x0E80)
Target Address	16 bits	Target ECU logical address (e.g., 0x0010)
UDS Data	Variable	UDS service request or response (SID + parameters)

UDS Services (ISO 14229)

SID (hex)	Name	Description
0x10	DiagnosticSessionControl	Switch between diagnostic sessions
0x11	ECUReset	Reset the ECU (hard, key off/on, soft)
0x14	ClearDiagnosticInformation	Clear DTCs and status
0x19	ReadDTCInformation	Read DTC status, snapshot, extended data
0x22	ReadDataByIdentifier	Read data by 2-byte DID (e.g., VIN, serial number)
0x23	ReadMemoryByAddress	Read raw memory from ECU
0x27	SecurityAccess	Seed-key authentication for protected services
0x28	CommunicationControl	Enable/disable ECU communication (Tx/Rx)
0x2E	WriteDataByIdentifier	Write data by DID
0x2F	InputOutputControlByIdentifier	Actuator control (e.g., force output on/off)
0x31	RoutineControl	Start/stop/request results of routines (e.g., self-test)

0x34	RequestDownload	Initiate data transfer to ECU (flash programming)
0x35	RequestUpload	Initiate data transfer from ECU
0x36	TransferData	Transfer data blocks (follows 0x34 or 0x35)
0x37	TransferExit	Complete the transfer
0x3E	TesterPresent	Keepalive to prevent session timeout
0x85	ControlDTCSetting	Enable/disable DTC detection during programming

UDS Sessions

Session	Value	Description
Default (DS)	0x01	Limited services, normal operation
Programming (PS)	0x02	ECU flash programming (requires SecurityAccess)
Extended Diagnostic (EDS)	0x03	Full diagnostic access (I/O control, routines)
Safety System (SS)	0x04	Safety-critical diagnostics
0x40-0x5F	Vendor-specific	Manufacturer-defined sessions

Negative Response Codes (NRC)

UDS negative responses use SID 0x7F followed by the rejected SID and an NRC byte:

NRC (hex)	Name	Description
0x10	generalReject	General rejection
0x11	serviceNotSupported	SID not supported in active session
0x12	subFunctionNotSupported	Sub-function not supported
0x13	incorrectMessageLengthOrInvalidFormat	Wrong message length
0x14	responseTooLong	Response exceeds transport buffer
0x22	conditionsNotCorrect	Preconditions not met
0x24	requestSequenceError	Requests sent out of order
0x25	noResponseFromSubnetComponent	Target sub-ECU did not respond
0x31	requestOutOfRange	DID/address/parameter out of valid range
0x33	securityAccessDenied	SecurityAccess not granted
0x35	invalidKey	Wrong security key

0x36	exceededNumberOfAttempts	Too many failed security attempts
0x37	requiredTimeDelayNotExpired	Lockout timer active after failed security
0x70	uploadDownloadNotAccepted	Transfer setup rejected
0x71	transferDataSuspended	Transfer interrupted
0x72	generalProgrammingFailure	Flash programming error
0x73	wrongBlockSequenceCounter	Block sequence number mismatch
0x78	requestCorrectlyReceivedResponsePending	ECU busy, response coming later
0x7E	subFunctionNotSupportedInActiveSession	Sub-function not available in current session
0x7F	serviceNotSupportedInActiveSession	Service not available in current session

ECU Flash Programming Sequence

[Diagram: see online version]

Network Architecture

[Diagram: see online version]

The DoIP gateway bridges between Ethernet and CAN buses, routing UDS messages to the appropriate target ECU regardless of its physical bus.

Encapsulation

[Diagram: see online version]

For discovery: Ethernet --> IP --> UDP (port 13400) --> DoIP.

DoIP vs CAN-Based Diagnostics

Feature	DoIP (ISO 13400)	CAN Diagnostics (ISO 15765)
Transport	TCP/IP over Ethernet	CAN bus
Speed	100 Mbps+	500 kbps max
Flash programming	Minutes (large firmware)	Hours over CAN
Discovery	UDP broadcast	Fixed CAN IDs
Remote access	Yes (IP routable, OTA)	No (local bus only)
Addressing	16-bit logical addresses	11-bit CAN IDs
Concurrent sessions	Multiple TCP connections	Limited by bus bandwidth

Diagnostic services	UDS (ISO 14229)	UDS (ISO 14229) -- same services

Standards

Document	Title
ISO 13400-1	DoIP -- General information and use case definition
ISO 13400-2	DoIP -- Transport protocol and network layer services
ISO 13400-3	DoIP -- IEEE 802.3 wired vehicle interface
ISO 13400-4	DoIP -- Ethernet-based high-speed data link
ISO 14229-1	UDS -- Application layer services
ISO 14229-2	UDS -- Session layer services
ISO 14229-5	UDS on IP (UDSonIP)
ISO 15765-2	ISO-TP (CAN transport, for comparison)

See Also

- CAN -- physical/data link layer; DoIP replaces CAN as the diagnostic transport
- OBD-II -- legislated diagnostics (uses UDS services 01-0A over CAN)
- J1939 -- heavy-duty vehicle CAN protocol with its own diagnostic messages
- SOME/IP -- service-oriented automotive Ethernet middleware (coexists with DoIP)

SAE J1939

Standard: SAE J1939 | **Layer:** Application / Data Link (over CAN 2.0B) | **Wireshark filter:** `j1939`

SAE J1939 is the standard communication protocol for heavy-duty vehicles -- trucks, buses, agricultural equipment, marine engines, and construction machinery. Built on CAN 2.0B with 29-bit extended identifiers, J1939 defines a complete networking stack including addressing, transport, diagnostics, and a vast library of standardized parameters. Unlike OBD-II's simple request-response model, J1939 ECUs broadcast data continuously at defined rates, and any node on the bus can transmit. The protocol operates at 250 kbps (500 kbps for J1939-14 high-speed supplement).

29-Bit CAN ID Structure

[Diagram: see online version]

Field	Size	Description

Priority	3 bits	0 (highest) to 7 (lowest); default 6 for most data, 3 for control
R	1 bit	Reserved (0)
DP	1 bit	Data Page -- selects between two PGN pages
PDU Format (PF)	8 bits	Determines message format: PF < 240 = peer-to-peer, PF >= 240 = broadcast
PDU Specific (PS)	8 bits	If PF < 240: Destination Address. If PF >= 240: Group Extension (GE)
Source Address (SA)	8 bits	Address of the transmitting node (0-253; 254 = null, 255 = global)

PGN (Parameter Group Number)

The PGN is derived from the CAN ID: **PGN = (DP << 16) | (PF << 8) | PS** (for broadcast, PF >= 240) or **PGN = (DP << 16) | (PF << 8) | 0x00** (for peer-to-peer, PF < 240, PS = destination address).

PGNs are 18-bit identifiers that group related parameters. Each PGN has a defined transmission rate, priority, and data length.

Common PGNs

PGN	Acronym	Name	Key SPNs	Rate
61444 (0xF004)	EEC1	Electronic Engine Controller 1	Engine Speed (SPN 190), Engine Torque (SPN 513)	10 ms
65262 (0xFEEE)	ET1	Engine Temperature 1	Coolant Temp (SPN 110), Fuel Temp (SPN 174)	1 s
65263 (0xFEEF)	EFL/P	Engine Fluid Level/Pressure	Oil Pressure (SPN 100), Fuel Level (SPN 96)	500 ms
65265 (0xFEF1)	CCVS	Cruise Control / Vehicle Speed	Vehicle Speed (SPN 84), Brake Switch (SPN 597)	100 ms
65269 (0xFEF5)	AMB	Ambient Conditions	Barometric Pressure (SPN 108), Ambient Temp (SPN 171)	1 s
65270 (0xFEF6)	IC1	Inlet/Exhaust Conditions 1	Boost Pressure (SPN 102), Intake Temp (SPN 105)	500 ms
65279 (0xFEFF)	VEP1	Vehicle Electrical Power 1	Battery Voltage (SPN 168)	1 s
65272 (0xFEF8)	TF	Transmission Fluids	Trans Oil Temp (SPN 177), Trans Oil Pressure (SPN 127)	1 s

SPNs (Suspect Parameter Numbers)

SPNs identify individual data fields within a PGN. Each SPN has a defined bit position, length, resolution, and offset:

SPN	Name	PGN	Bits	Resolution	Offset	Range
190	Engine Speed	61444	24-39 (16 bits)	0.125 rpm/bit	0	0-8031.875 rpm
84	Vehicle Speed	65265	8-23 (16 bits)	1/256 km/h/bit	0	0-250.996 km/h
110	Coolant Temp	65262	0-7 (8 bits)	1 C/bit	-40	-40 to 210 C
100	Oil Pressure	65263	24-31 (8 bits)	4 kPa/bit	0	0-1000 kPa
96	Fuel Level	65263	8-15 (8 bits)	0.4 %/bit	0	0-100%
513	Actual Engine Torque	61444	16-23 (8 bits)	1 %/bit	-125	-125 to 125%

Data Broadcast

[Diagram: see online version]

Transport Protocol (TP)

J1939 messages larger than 8 bytes (the CAN frame limit) use the Transport Protocol defined in J1939-21. There are two modes:

BAM (Broadcast Announce Message)

For broadcast multi-packet messages (one-to-all):

[Diagram: see online version]

Connection Mode (CM / RTS-CTS)

For peer-to-peer multi-packet messages with flow control:

[Diagram: see online version]

TP PGNs

PGN	Name	Description
60416 (0xEC00)	TP.CM	Connection Management (BAM, RTS, CTS, EndOfMsgAck, Abort)
60160 (0xEB00)	TP.DT	Data Transfer (carries 7 data bytes per frame)

Address Claiming

J1939 nodes dynamically claim addresses using a priority-based arbitration mechanism based on the NAME (64-bit unique identifier):

[Diagram: see online version]

NAME Structure (64 bits)

Bits	Field	Description
63	Arbitrary Address Capable	1 = can negotiate addresses
62-59	Industry Group	0=Global, 1=Highway, 2=Agriculture, 3=Construction, 4=Marine
58-55	Vehicle System Instance	Instance of this vehicle system type
54-48	Vehicle System	Type of vehicle system
47-40	Function	ECU function (e.g., engine, transmission, brakes)
39-35	Function Instance	Instance of this function
34-32	ECU Instance	Instance of this ECU
31-21	Manufacturer Code	SAE-assigned manufacturer ID
20-0	Identity Number	Unique serial number

Diagnostic Messages

PGN	Name	Description
65226 (0xFECA)	DM1	Active Diagnostic Trouble Codes (broadcast continuously)
65227 (0xFECB)	DM2	Previously Active DTCs
65228 (0xFECC)	DM3	Diagnostic Data Clear / Reset
65229 (0xFECD)	DM4	Freeze Frame Parameters
65230 (0xFECE)	DM5	Diagnostic Readiness
65231 (0xFECF)	DM6	Pending DTCs
65232 (0xFED0)	DM7	Test Results Command
65233 (0xFED1)	DM8	Test Results Response
65235 (0xFED3)	DM11	Active DTC Clear
65236 (0xFED4)	DM12	Emissions-Related Active DTCs

DM1 DTC Structure (4 bytes per DTC)

Byte(s)	Field	Description
1-2	SPN	Suspect Parameter Number (19 bits: bytes 1-2 + upper 3 bits of byte 3)
3 (upper 3)	SPN (MSB)	Upper 3 bits of 19-bit SPN
3 (bits 4-0)	FMI	Failure Mode Identifier (0-31)
4 (bit 7)	CM	Conversion Method (0=J1939, 1=manufacturer)
4 (bits 6-0)	OC	Occurrence Count (0-126; 127=not available)

Common FMI Values

FMI	Meaning
0	Data valid but above normal range (most severe)
1	Data valid but below normal range (most severe)
2	Data erratic, intermittent, or incorrect
3	Voltage above normal or shorted to high source
4	Voltage below normal or shorted to low source
5	Current below normal or open circuit
6	Current above normal or grounded circuit
7	Mechanical system not responding or out of adjustment
11	Root cause not known
12	Bad intelligent device or component
31	Condition exists

J1939 vs OBD-II vs CANopen

Feature	J1939	OBD-II (CAN)	CANopen
CAN ID	29-bit extended	11-bit standard	11-bit standard
Speed	250 kbps	250 / 500 kbps	10 kbps - 1 Mbps
Addressing	254 node addresses (SA)	Fixed IDs (0x7DF, 0x7E0-7)	127 node IDs
Data model	PGN/SPN (broadcast)	Service/PID (request-response)	Object Dictionary (index/subindex)
Application	Heavy-duty vehicles	Passenger car diagnostics	Industrial automation
Message flow	Continuous broadcast	On-demand request/response	PDO (broadcast) + SDO (request/response)
Multi-frame	TP BAM / CM (J1939-21)	ISO-TP (ISO 15765-2)	SDO segmented / block

Diagnostics	DM1-DM31 (SPN+FMI)	DTCs (P/C/B/U codes)	Emergency (EMCY) objects

Encapsulation

[Diagram: see online version]

Standards

Document	Title
SAE J1939-21	Data Link Layer (TP, address claiming)
SAE J1939-71	Vehicle Application Layer (PGNs, SPNs)
SAE J1939-73	Application Layer -- Diagnostics (DM messages)
SAE J1939-81	Network Management (NAME, address claiming)
SAE J1939-13	Physical Layer (off-board connector, 250 kbps)
SAE J1939-14	Physical Layer (500 kbps high-speed supplement)
ISO 11898	CAN data link and physical layer (underlying bus)

See Also

- CAN -- physical and data link layer that J1939 runs on
- OBD-II -- passenger vehicle diagnostics (also CAN-based, but simpler)
- DoIP / UDS -- modern Ethernet-based diagnostics
- SOME/IP -- automotive Ethernet service-oriented middleware
- Modbus -- industrial protocol with a similar register-based data model

OBD-II -- On-Board Diagnostics II

Standard: SAE J1979 / ISO 15031 | **Layer:** Application (over CAN / K-Line / J1850) | **Wireshark filter:** `iso15765`

OBD-II is a mandatory diagnostic interface for all passenger vehicles sold in the US (since 1996) and Europe (EOBD, since 2001). It provides standardized access to engine parameters, emissions data, and diagnostic trouble codes (DTCs) through a 16-pin connector under the dashboard. The most common physical layer is CAN (ISO 15765), though older vehicles may use K-Line (ISO 14230) or J1850 (legacy US). OBD-II defines a request-response protocol where a diagnostic tool sends a Service ID (mode) + Parameter ID (PID), and the ECU responds with the requested data.

Physical Layers

Protocol	Standard	Speed	Notes
CAN (most common)	ISO 15765-4	250 / 500 kbps	Standard since ~2008, required in US since 2008
K-Line	ISO 14230 (KWP2000)	10.4 kbps	Single-wire bidirectional, older European vehicles
J1850 VPW	SAE J1850	10.4 kbps	Variable Pulse Width, legacy GM
J1850 PWM	SAE J1850	41.6 kbps	Pulse Width Modulation, legacy Ford
ISO 9141-2	ISO 9141	10.4 kbps	K-Line variant, older Asian/European

DLC (Data Link Connector) -- SAE J1962

[Diagram: see online version]

[Diagram: see online version]

Key pins: **6** (CAN-H), **14** (CAN-L), **4/5** (ground), **16** (+12V power), **7** (K-Line).

CAN-Based OBD Request/Response IDs

CAN ID	Direction	Description
0x7DF	Request (functional)	Broadcast to all ECUs ("anyone who supports this PID")
0x7E0-0x7E7	Request (physical)	Addressed to specific ECU (0x7E0 = engine ECU)
0x7E8-0x7EF	Response	Response from ECU (0x7E8 = engine ECU response)

CAN OBD Frame (ISO 15765 -- Single Frame)

[Diagram: see online version]

For CAN-based OBD, each request/response is wrapped in ISO-TP (ISO 15765-2) framing. Most PID responses fit in a single CAN frame.

ISO-TP Frame Types (ISO 15765-2)

PCI Type	Nibble	Description
Single Frame (SF)	0x0	Complete message in one frame, lower nibble = length

First Frame (FF)	0x1	First of multi-frame, followed by length (12 bits)
Consecutive Frame (CF)	0x2	Continuation, lower nibble = sequence number (0-F)
Flow Control (FC)	0x3	Receiver controls sender's pacing (BS, STmin)

Service IDs (Modes)

Service (hex)	Name	Description
0x01	Current Data	Read real-time sensor values (PIDs)
0x02	Freeze Frame	Snapshot of data when DTC was set
0x03	Read DTCs	Retrieve confirmed diagnostic trouble codes
0x04	Clear DTCs	Clear DTCs and reset MIL (check engine light)
0x05	O2 Sensor Monitoring	Oxygen sensor test results
0x06	On-Board Test Results	Non-continuous monitoring test results
0x07	Pending DTCs	DTCs detected during current drive cycle
0x08	Control On-Board System	Actuator tests (manufacturer-specific)
0x09	Vehicle Information	VIN, calibration IDs, ECU name
0x0A	Permanent DTCs	DTCs that persist even after clearing

Common PIDs (Service 0x01)

PID (hex)	Name	Formula	Unit
0x00	Supported PIDs [01-20]	Bit-encoded (each bit = one PID supported)	bitmask
0x04	Calculated Engine Load	A / 2.55	%
0x05	Engine Coolant Temp	A - 40	C
0x0C	Engine RPM	(256*A + B) / 4	rpm
0x0D	Vehicle Speed	A	km/h
0x0F	Intake Air Temp	A - 40	C
0x10	MAF Air Flow Rate	(256*A + B) / 100	g/s
0x11	Throttle Position	A / 2.55	%
0x1F	Run Time Since Start	256*A + B	sec
0x2F	Fuel Tank Level	A / 2.55	%

0x31	Distance Since Codes Cleared	256*A + B	km

A, B = response data bytes.

PID Request/Response

[Diagram: see online version]

[Diagram: see online version]

DTC Read / Clear

[Diagram: see online version]

DTC Format

DTCs are 2 bytes (16 bits) encoded as follows:

Bits	Field	Values
15-14	Category	00=P (Powertrain), 01=C (Chassis), 10=B (Body), 11=U (Network)
13-12	Sub-type	0=SAE standard, 1=Manufacturer-specific
11-8	System group	0-F
7-0	Fault number	00-FF

DTC Category Prefixes

Prefix	System	Examples
P0xxx	Powertrain (SAE-defined)	P0300 = Random misfire, P0420 = Catalyst efficiency
P1xxx	Powertrain (manufacturer)	Manufacturer-specific codes
C0xxx	Chassis	ABS, stability control, steering
B0xxx	Body	Airbags, HVAC, lighting
U0xxx	Network	CAN bus communication faults

VIN Read (Service 09, PID 02)

The Vehicle Identification Number requires multi-frame ISO-TP because it is 17 characters (> 8 bytes):

[Diagram: see online version]

Encapsulation (CAN-Based OBD)

[Diagram: see online version]

Standards

Document	Title
SAE J1979	OBD-II diagnostic services (modes/PIDs)
SAE J1962	OBD-II diagnostic connector (DLC)
ISO 15765-4	OBD on CAN (emissions-related)
ISO 15765-2	ISO-TP transport protocol for CAN
ISO 15031-5	Emissions-related diagnostic services
ISO 15031-6	DTC definitions
ISO 14230	KWP2000 (K-Line diagnostics)
SAE J1850	J1850 VPW/PWM physical layer

See Also

- CAN -- physical and data link layer for modern OBD-II
- J1939 -- heavy-duty vehicle protocol (also CAN-based)
- DoIP / UDS -- modern Ethernet-based diagnostics replacing OBD-II in newer vehicles
- SOME/IP -- automotive Ethernet middleware

SOME/IP -- Scalable service-Oriented MiddlewarE over IP

Standard: AUTOSAR SOME/IP Protocol Specification | **Layer:** Application (over UDP/TCP) | **Wireshark filter:** `someip`

SOME/IP is a service-oriented communication protocol designed for automotive Ethernet networks. It enables ECUs to offer and consume services (remote procedure calls and events) over standard IP networking, replacing the signal-oriented paradigm of CAN. SOME/IP is a key enabler for next-generation vehicle architectures (zonal, centralized compute) where high-bandwidth sensors (cameras, lidar, radar) and ADAS functions require Ethernet. It includes a built-in Service Discovery (SOME/IP-SD) mechanism for dynamic service registration and subscription. SOME/IP is specified by AUTOSAR and is used in both Classic and Adaptive AUTOSAR platforms.

SOME/IP Header

[Diagram: see online version]

Every SOME/IP message has a 16-byte header followed by the payload.

Key Fields

Field	Size	Description
Service ID	16 bits	Identifies the service (e.g., 0x0001 = Navigation Service)
Method ID	16 bits	Method, getter, setter, or event within the service; 0x8000+ = events/notifications
Length	32 bits	Length of payload + Client ID through Return Code (payload + 8 bytes)
Client ID	16 bits	Unique ID of the calling client (ECU ID + local client)
Session ID	16 bits	Sequence number for request-response matching
Protocol Version	8 bits	SOME/IP protocol version (currently 0x01)
Interface Version	8 bits	Service interface version
Message Type	8 bits	Type of message (request, response, notification, etc.)
Return Code	8 bits	Result status for responses

Message Types

Value	Name	Description
0x00	REQUEST	Client calls a method on a service (expects response)
0x01	REQUEST_NO_RETURN	Fire-and-forget method call (no response expected)
0x02	NOTIFICATION	Event/notification from server to subscribed clients
0x80	RESPONSE	Server response to a REQUEST
0x81	ERROR	Server response indicating an error
0x20	TP_REQUEST	Segmented request (SOME/IP-TP, large payloads)
0x21	TP_REQUEST_NO_RETURN	Segmented fire-and-forget
0x22	TP_NOTIFICATION	Segmented notification
0xA0	TP_RESPONSE	Segmented response

0xA1	TP_ERROR	Segmented error response

Return Codes

Value	Name	Description
0x00	E_OK	No error
0x01	E_NOT_OK	Unspecified error
0x02	E_UNKNOWN_SERVICE	Service ID not known
0x03	E_UNKNOWN_METHOD	Method ID not known
0x04	E_NOT_READY	Service not ready (still initializing)
0x05	E_NOT_REACHABLE	Service not reachable (network issue)
0x06	E_TIMEOUT	Response timeout
0x07	E_WRONG_PROTOCOL_VERSION	Protocol version mismatch
0x08	E_WRONG_INTERFACE_VERSION	Interface version mismatch
0x09	E_MALFORMED_MESSAGE	Deserialization error
0x0A	E_WRONG_MESSAGE_TYPE	Unexpected message type

Method Invocation (Request-Response)

[Diagram: see online version]

Event Notification (Publish-Subscribe)

[Diagram: see online version]

SOME/IP-SD (Service Discovery)

SOME/IP-SD allows ECUs to dynamically find and subscribe to services at runtime. SD messages are sent on UDP port 30490 (default), typically to multicast address 224.224.224.245.

SD Header

[Diagram: see online version]

SD messages contain a list of **entries** (service offers, finds, subscriptions) and **options** (endpoint addresses).

SD Entry Types

Type	Value	Description

FindService	0x00	Client looking for a service on the network
OfferService	0x01	Server advertising an available service
StopOfferService	0x01 (TTL=0)	Server withdrawing a service
SubscribeEventgroup	0x06	Client subscribing to events from a service
StopSubscribeEventgroup	0x06 (TTL=0)	Client unsubscribing
SubscribeEventgroupAck	0x07	Server confirming subscription
SubscribeEventgroupNack	0x07 (TTL=0)	Server rejecting subscription

SD Options

Type	Description
IPv4 Endpoint	Service endpoint: IPv4 address + port + protocol (UDP/TCP)
IPv6 Endpoint	Service endpoint: IPv6 address + port + protocol
IPv4 Multicast	Multicast group for event distribution
IPv6 Multicast	Multicast group (IPv6) for event distribution
Configuration	Key-value configuration strings

Service Discovery Flow

[Diagram: see online version]

SD State Machine (Server)

Phase	Behavior	Duration
Initial Wait	Wait random 0-100 ms before first offer	Startup
Repetition	Send OfferService at exponentially increasing intervals	Configurable (e.g., 3 rounds)
Main	Send OfferService at fixed interval (cyclic)	Ongoing

Transport Binding

Transport	Use Case	Notes
UDP	Events, notifications, fire-and-forget	Low latency, best for cyclic data
TCP	Reliable method calls, large payloads	Connection-oriented, guaranteed delivery

SOME/IP-TP	Large payloads over UDP	Segmentation for messages > UDP MTU

SOME/IP-TP (Segmentation)

For payloads exceeding UDP MTU, SOME/IP-TP adds a 4-byte TP header after the standard SOME/IP header:

Field	Size	Description
Offset	28 bits	Byte offset of this segment in the original message
More Segments	1 bit	1 = more segments follow, 0 = last segment
Reserved	3 bits	Reserved

AUTOSAR Integration

[Diagram: see online version]

Encapsulation

[Diagram: see online version]

Standards

Document	Title
AUTOSAR SOME/IP PRS	SOME/IP Protocol Specification
AUTOSAR SOME/IP-SD PRS	SOME/IP Service Discovery Protocol Specification
AUTOSAR SOME/IP-TP PRS	SOME/IP Transport Protocol Specification
AUTOSAR SWS SomeIPXf	SOME/IP Transformer (serialization)
AUTOSAR SWS SD	Service Discovery Module
IEEE 802.3	Ethernet (including automotive 100BASE-T1, 1000BASE-T1)

See Also

- CAN -- legacy automotive bus that SOME/IP is replacing for high-bandwidth use cases
- OBD-II -- automotive diagnostics (signal-oriented, CAN-based)
- J1939 -- heavy-duty vehicle CAN protocol
- DoIP / UDS -- diagnostics over IP (often coexists with SOME/IP on automotive Ethernet)
- MQTT -- lightweight pub-sub protocol (similar subscribe pattern, different domain)

Aviation

Communication and data-link protocols used in aviation and air traffic systems.

5 protocols in this section

ACARS -- Aircraft Communications Addressing and Reporting System

Standard: ARINC 618 | **Layer:** Application (VHF / SATCOM / HF data link) | **Wireshark filter:** `acars` (with VDL Mode 2 plugins)

ACARS is a character-oriented digital data link system for air-to-ground and ground-to-air messaging between aircraft and airline operations centers or ATC facilities. Introduced in 1978, it carries pre-departure clearances, weather requests, OOOI events (Out of gate, Off ground, On ground, Into gate), position reports, and free-text messages. ACARS operates over VHF (most common), HF data link, and SATCOM (Inmarsat, Iridium), providing global coverage. It is the transport backbone for higher-level protocols such as CPDLC (FANS-1/A) and ADS-C.

ACARS Message Frame

[Diagram: see online version]

Key Fields

Field	Size	Description
SOH	1 byte	Start of Header (0x01) -- marks beginning of message
Mode	1 char	Link mode character: `2` = VHF ACARS, `H` = HF data link, etc.
Aircraft Address	7 chars	Aircraft registration (e.g., `.N12345`) -- dot-padded
ACK/NAK	1 char	Acknowledgment character or NAK for negative acknowledgment
Label	2 chars	Message type identifier (determines content interpretation)
Block ID	1 char	Sequence identifier for multi-block messages (0-9, A-Z)
STX	1 byte	Start of Text (0x02) -- separates header from message body
Message Text	Variable	Free text or structured data, up to ~220 characters
Suffix	1 byte	ETX (0x03) for final block, ETB (0x17) for intermediate blocks
BCS	2 bytes	Block Check Sequence -- parity-based error detection
DEL	1 byte	Delete character (0x7F) -- end of transmission block

Field Details

Mode Characters

Mode	Description
`2`	VHF ACARS (standard)
`H`	HF data link
`C`	VHF ACARS (Canada)
`E`	VHF ACARS (Europe secondary)
`X`	Airline-defined

Common Labels

Label	Description
`_d`	OOOI event (Out/Off/On/Into gate)
`H1`	HF voice position report
`SA`	System status / test
`Q0`	Link test / request
`RA`	ETA update
`5Z`	Airline-defined free text
`B6`	Takeoff report
`B3`	Turbulence report
`_7`	Weather request
`AA`	ATN / CPDLC message (FANS)

OOOI Events

ACARS automatically reports four key flight events via sensors on the aircraft:

Event	Trigger	Label
Out of gate	Parking brake released / door closed	`_d`
Off ground	Weight off wheels (liftoff)	`_d`
On ground	Weight on wheels (touchdown)	`_d`
Into gate	Parking brake set / door opened	`_d`

These OOOI events are the original core function of ACARS, used by airlines for fleet tracking and on-time performance.

Message Types

Type	Direction	Description
AOC (Airline Operational Control)	Both	Operational messages -- OOOI, fuel, loadsheet, gate info, delays
ATC (Air Traffic Control)	Both	CPDLC clearances, ADS-C reports, DCL (pre-departure clearance)
AAC (Airline Administrative Control)	Both	Crew scheduling, maintenance, company messages

Direction

Direction	Description
Downlink	Aircraft to ground station
Uplink	Ground station to aircraft

Downlink Message Flow

[Diagram: see online version]

Uplink Message Flow

[Diagram: see online version]

Physical Layer

VHF (Primary)

Parameter	Value
Modulation	AM-MSK (Minimum Shift Keying on AM carrier)
Data rate	2400 bps
Frequencies	129.125, 130.025, 130.450, 131.125, 131.550 MHz (common)
Range	Line-of-sight, ~200 nm
Providers	ARINC (Americas), SITA (Europe, Asia)

VDL Mode 2

Parameter	Value
Data rate	31.5 kbps

Modulation	D8PSK
Link layer	AVLC (Aviation VHF Link Control) -- HDLC derivative
Frequency	136.725, 136.775, 136.875, 136.975 MHz
Protocol stack	VDL2 PHY -> AVLC -> ACARS / ATN

VDL Mode 2 is the next-generation VHF data link, offering higher throughput and supporting both ACARS (AOA -- ACARS over AVLC) and ATN-based applications.

SATCOM

System	Coverage	Data rate
Inmarsat (Classic Aero)	Global (excl. poles)	600-10,500 bps
Inmarsat SwiftBroadband	Global (excl. poles)	Up to 432 kbps
Iridium	Global (incl. poles)	2400 bps (short burst)

HF Data Link (HFDL)

Parameter	Value
Frequencies	2-22 MHz (HF band)
Data rate	300-1800 bps
Coverage	Global (especially oceanic/polar)
Ground stations	~15 worldwide

ARINC Specification Family

Specification	Description
ARINC 618	Character-oriented ACARS protocol (original)
ARINC 619	ACARS Protocols for Avionics End Systems
ARINC 620	FANS-1/A data link (CPDLC / ADS-C over ACARS)
ARINC 623	Binary-format ACARS messages (extended character set)
ARINC 631	VDL Mode 2 aviation VHF data link
ARINC 633	ACARS over IP (modern ground infrastructure)

Hobbyist Monitoring

ACARS messages can be received and decoded by radio enthusiasts using inexpensive SDR equipment:

Tool	Description

RTL-SDR (hardware)	USB software-defined radio dongle (~$25), tunes VHF frequencies
acarsdec	Open-source multi-channel ACARS decoder
dumpvdl2	VDL Mode 2 decoder for RTL-SDR
acarsdeco2	Multi-channel ACARS/VDL2 decoder with web interface
Jaero	Inmarsat ACARS decoder (L-band satellite)
airframes.io	Community aggregator for decoded ACARS messages

Encapsulation

[Diagram: see online version]

Standards

Document	Title
ARINC 618	Air/Ground Character-Oriented Protocol Specification
ARINC 620	Data Link Ground System Standard and Interface Specification (FANS)
ARINC 623	Character-Oriented Air Traffic Service (ATS) Applications
ARINC 631	Aviation VHF Digital Link (VDL) Mode 2
ARINC 633	ACARS over IP
ICAO Annex 10 Vol III	Aeronautical Telecommunications -- Communication Systems

See Also

- CPDLC -- controller-pilot data link communications carried over ACARS
- Mode S -- secondary surveillance radar transponder protocol
- ASTERIX -- ATC surveillance data exchange format

ASTERIX -- All Purpose Structured Eurocontrol Surveillance Information Exchange

Standard: Eurocontrol ASTERIX | **Layer:** Application (typically over UDP) | **Wireshark filter:** `asterix`

ASTERIX is a binary protocol developed by Eurocontrol for the exchange of ATC surveillance data. It carries radar plots, system tracks, ADS-B reports, MLAT data, weather information, and sensor status between surveillance sensors, tracker systems, and ATC displays. ASTERIX uses a compact, bit-oriented encoding with a flexible

field-presence mechanism (FSPEC) that allows each message to carry only the data items actually present. It is the standard surveillance data format across European airspace and is widely adopted worldwide, typically transported over UDP multicast or unicast.

Data Block Structure

[Diagram: see online version]

A single UDP datagram may contain one or more data blocks, and each data block may contain one or more records of the same category.

Record Structure

[Diagram: see online version]

Key Fields

Field	Size	Description
Category (CAT)	1 byte	Identifies the type of data (001 = radar, 021 = ADS-B, 048 = radar, etc.)
Length	2 bytes	Total length of the data block in bytes (including CAT and Length)
FSPEC	Variable	Field Specification -- bit mask indicating which data items are present
FX bit	1 bit	Field Extension -- if 1, another FSPEC byte follows
Data Items	Variable	Encoded data items in the order defined by the category specification

Field Details

FSPEC (Field Specification)

The FSPEC is a variable-length bit field at the start of each record. Each byte contains 7 data item presence bits (MSB first) and 1 FX (Field Extension) bit in the LSB:

Bit Position	Meaning
Bit 7 (MSB)	Data Item 1 present (1) or absent (0)
Bit 6	Data Item 2 present/absent
Bit 5	Data Item 3 present/absent
Bit 4	Data Item 4 present/absent
Bit 3	Data Item 5 present/absent

Bit 2	Data Item 6 present/absent
Bit 1	Data Item 7 present/absent
Bit 0 (LSB)	FX -- if 1, another FSPEC byte follows; if 0, FSPEC ends

This mechanism allows records to include only the data items that are actually populated, keeping messages compact.

Data Item Types

Type	Description
Fixed Length	Predefined size (e.g., 2 bytes for a position)
Variable Length	Length indicator followed by data
Repetitive	Repetition factor (1 byte) followed by N fixed-length sub-items
Compound	Sub-field presence indicator (like a nested FSPEC) followed by sub-items
Explicit	Length byte followed by variable data (for extensions and special data)

Common Encoding Conventions

Data Type	Encoding
Time of Day	24-bit unsigned, LSB = 1/128 second (midnight reference)
Position (lat/lon)	32-bit signed, LSB = 180/2^25 degrees (WGS-84)
Altitude	16-bit signed, LSB = 6.25 ft (Mode C) or 25 ft
Range	16-bit unsigned, LSB = 1/128 NM (category-dependent)
Azimuth	16-bit unsigned, LSB = 360/2^16 degrees
Groundspeed	16-bit unsigned, LSB = 2^-14 NM/s
Track angle	16-bit unsigned, LSB = 360/2^16 degrees

Key Categories

Category	Description	Typical Source
CAT 001	Monoradar target reports (plots and tracks)	Primary/secondary radar
CAT 002	Monoradar service messages	Radar sensor status
CAT 008	Monoradar weather data	Weather radar
CAT 010	Monosensor surface movement data	MLAT, SMR
CAT 020	Multilateration target reports	MLAT systems

CAT 021	ADS-B target reports	ADS-B ground stations
CAT 023	CNS/ATM ground station service messages	ADS-B sensor status
CAT 030	ARTAS tracks (deprecated, replaced by 062)	Multi-radar tracker
CAT 034	Monoradar transmission of service messages	Radar status/config
CAT 048	Monoradar target reports (enhanced)	Modern radar systems
CAT 062	System track data	Multi-sensor tracker (ARTAS, SASS-C)
CAT 063	Sensor status messages	System status
CAT 065	SDPS service status	Data processing system
CAT 240	Radar video (digitized)	Primary radar video
CAT 247	Version cross-reference	Version numbering

CAT 048 (Monoradar) -- Common Data Items

Data Item	FRN	Description
I048/010	1	Data Source Identifier (SAC/SIC)
I048/140	2	Time of Day
I048/020	3	Target Report Descriptor
I048/040	4	Measured Position (range/azimuth)
I048/070	5	Mode 3/A Code (squawk)
I048/090	6	Mode C Code (altitude)
I048/130	7	Radar Plot Characteristics
I048/220	8	Aircraft Address (24-bit ICAO)
I048/240	9	Aircraft Identification (callsign)
I048/250	10	Mode S BDS register data

CAT 062 (System Tracks) -- Common Data Items

Data Item	FRN	Description
I062/010	1	Data Source Identifier
I062/015	2	Service Identification
I062/070	3	Time of Track Information
I062/105	4	Calculated Track Position (WGS-84)
I062/100	5	Calculated Track Position (Cartesian)
I062/185	6	Calculated Track Velocity (Cartesian)

I062/210	7	Calculated Acceleration
I062/060	8	Track Mode 3/A Code
I062/245	9	Target Identification
I062/380	10	Aircraft Derived Data (compound)

Radar Data Distribution

[Diagram: see online version]

Transport

ASTERIX data blocks are typically transported over UDP:

Parameter	Typical Value
Transport	UDP (connectionless)
Addressing	Multicast (e.g., 232.x.x.x) or unicast
Port	Application-defined (no standard port)
MTU	Usually fits within single Ethernet frame
Reliability	Application-layer sequence numbers (no TCP retransmission)

Users

Organization	Region	Role
Eurocontrol	Pan-European	Standards body and ASTERIX maintainer
ENAV	Italy	Air navigation service provider
DFS	Germany	Air navigation service provider
NATS	United Kingdom	Air navigation service provider
DSNA	France	Air navigation service provider
FAA	United States	Uses ASTERIX adaptations for some surveillance feeds
NAV CANADA	Canada	ASTERIX adoption for ATC surveillance

Encapsulation

[Diagram: see online version]

Standards

Document	Title
Eurocontrol ASTERIX	ASTERIX specification overview and category documents
CAT 021 Ed 2.4	ADS-B Target Reports
CAT 048 Ed 1.31	Monoradar Target Reports
CAT 062 Ed 1.19	System Track Data
ICAO Annex 10 Vol IV	Surveillance and Collision Avoidance Systems

See Also

- Mode S -- secondary surveillance radar protocol (source of much ASTERIX data)
- ACARS -- aircraft data link messaging
- CPDLC -- controller-pilot data link communications

CPDLC -- Controller-Pilot Data Link Communications

Standard: ICAO Doc 4444 (PANS-ATM) / RTCA DO-258A | **Layer:** Application (over ACARS or ATN) | **Wireshark filter:** N/A (carried over ACARS or ATN stack)

CPDLC is a digital text-based communication system that supplements (and in some airspace replaces) voice radio between air traffic controllers and pilots. Instead of spoken clearances on congested VHF frequencies, CPDLC delivers standardized message elements as selectable text on cockpit displays, with defined response options (WILCO, UNABLE, STANDBY). Two implementations exist: FANS-1/A CPDLC, which runs over ACARS for oceanic and remote airspace (deployed since the mid-1990s across the Pacific, Atlantic, and Indian oceans), and ATN-based CPDLC (ATN B1), which uses the OSI-based Aeronautical Telecommunication Network for domestic and European airspace (Link 2000+).

CPDLC Message Structure

[Diagram: see online version]

Note: CPDLC messages are encoded as structured data elements within ACARS (FANS-1/A) or ATN ULCS/ACSE frames, not as a standalone binary frame. The diagram above represents the logical message structure.

Key Fields

Field	Description
Message ID (MIN)	Message Identification Number -- sequence number assigned by originator
Message Reference (MRN)	References the MIN of the message being responded to

Message Assurance	Indicates whether message delivery confirmation is required
Response Attribute	Specifies which responses are valid (W/U = WILCO/UNABLE, A/N = AFFIRM/NEGATIVE, R = ROGER, Y = YES/NO)
Message Element	Standardized uplink (UM) or downlink (DM) element with parameters
Free Text	Optional unformatted text (discouraged for operational use)

Message Types

Uplink Messages (UM) -- ATC to Pilot

Tag	Message	Response
UM19	MAINTAIN [level]	W/U
UM20	CLIMB TO [level]	W/U
UM23	DESCEND TO [level]	W/U
UM46	CROSS [position] AT [level]	W/U
UM47	CROSS [position] AT OR ABOVE [level]	W/U
UM74	PROCEED DIRECT TO [position]	W/U
UM77	AT [position] PROCEED DIRECT TO [position]	W/U
UM79	CLEARED TO [position] VIA [route]	W/U
UM80	CLEARED [route clearance]	W/U
UM94	TURN LEFT HEADING [degrees]	W/U
UM95	TURN RIGHT HEADING [degrees]	W/U
UM106	MAINTAIN [speed]	W/U
UM116	RESUME NORMAL SPEED	W/U
UM169	RADAR CONTACT	R
UM170	RADAR CONTACT LOST	R
UM171	RADAR SERVICE TERMINATED	R
UM179	SQUAWK [code]	W/U
UM183	[free text]	R/W/U
UM190	REPORT HEADING	Y
UM215	TURN LEFT/RIGHT [degrees] DEGREES	W/U
UM228	REPORT ETA [position]	Y

Downlink Messages (DM) -- Pilot to ATC

Tag	Message	Response
DM0	WILCO	N/A
DM1	UNABLE	N/A
DM2	STANDBY	N/A
DM3	ROGER	N/A
DM4	AFFIRM	N/A
DM5	NEGATIVE	N/A
DM6	REQUEST [level]	Y
DM9	REQUEST CLIMB TO [level]	Y
DM10	REQUEST DESCENT TO [level]	Y
DM15	REQUEST HEADING [degrees]	Y
DM18	REQUEST [speed]	Y
DM22	REQUEST DIRECT TO [position]	Y
DM25	REQUEST [route clearance]	Y
DM27	REQUEST WEATHER DEVIATION TO [position] VIA [route]	Y
DM48	POSITION REPORT	N/A
DM62	ERROR [error information]	N/A
DM63	NOT CURRENT DATA AUTHORITY	N/A
DM67	[free text]	Y
DM80	DEVIATING [direction] OF ROUTE [distance]	N/A

Response Types

Response	Code	Meaning
WILCO	DM0	Will comply -- pilot accepts and will execute the clearance
UNABLE	DM1	Unable to comply -- pilot cannot execute (with optional reason)
STANDBY	DM2	Need more time to evaluate -- must follow up with WILCO or UNABLE
ROGER	DM3	Message received and understood (no action required)
AFFIRM	DM4	Yes (in response to a question)
NEGATIVE	DM5	No (in response to a question)

CPDLC Logon and Connection

[Diagram: see online version]

Transfer of Data Authority

When an aircraft transitions between ATC sectors or centers, the CPDLC connection transfers:

[Diagram: see online version]

Clearance Exchange Example

[Diagram: see online version]

FANS-1/A vs ATN CPDLC

Feature	FANS-1/A CPDLC	ATN B1 CPDLC (Link 2000+)
Transport	ACARS (ARINC 622/620)	ATN (OSI stack, TP4/CLNP)
Airspace	Oceanic, remote	Continental (Europe, domestic)
Encoding	Character-oriented (ARINC 622)	ASN.1 BER/PER
Latency requirement	<4 minutes (oceanic)	<60 seconds (en-route)
Deployed regions	NAT, PAC, Indian Ocean, South Atlantic	EUROCONTROL LINK 2000+ airspace
Connection management	FANS logon/logoff	ATN CM (Context Management)
Voice backup	HF radio	VHF radio

Operational Parameters

Parameter	Value
Message delivery time (target)	<60 seconds (continental), <4 min (oceanic)
Pilot response time (expected)	<120 seconds for RA-required messages
CPDLC timeout (no response)	5 minutes (controller typically follows up via voice)
Max message elements per message	Multiple elements allowed, but typically 1-3
Emergency use	Not primary -- voice radio remains primary for emergencies

Deployment

Region / Program	Implementation	Status
North Atlantic (NAT)	FANS-1/A over SATCOM/HF	Operational since ~1995

Pacific (NOPAC, CENPAC)	FANS-1/A over SATCOM	Operational
Indian Ocean	FANS-1/A over SATCOM	Operational
South Atlantic (SAT)	FANS-1/A over SATCOM	Operational
Europe (Link 2000+)	ATN B1 over VDL Mode 2	Operational (Maastricht UAC, DSNA, etc.)
US domestic	FANS-1/A (en-route, limited)	Expanding (FAA DataComm program)
Australia	FANS-1/A over SATCOM	Operational (oceanic)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
ICAO Doc 4444 (PANS-ATM)	Procedures for Air Navigation Services -- Air Traffic Management
RTCA DO-258A	Interoperability Requirements for ATS Applications Using ARINC 622
EUROCAE ED-110B	Initial CPDLC MASPS (Minimum Aviation System Performance Standard)
ICAO Doc 9694	Manual of Air Traffic Services Data Link Applications
GOLD (Global Operational Data Link)	Guidance for operational implementation of data link
ARINC 622	ATS Data Link Application over ACARS Air-Ground Network
ARINC 620	Data Link Ground System Standard (FANS-1/A)
ICAO Annex 10 Vol III	Aeronautical Telecommunications -- Communication Systems

See Also

- ACARS -- transport layer for FANS-1/A CPDLC messages
- Mode S -- secondary surveillance radar (SSR) transponder protocol
- ASTERIX -- ATC surveillance data exchange format
- METAR / TAF -- aviation weather reports (weather drives CPDLC re-route requests)

METAR / TAF -- Aviation Weather Reports

Standard: ICAO Annex 3 / WMO Manual on Codes | **Layer:** Application (text format) | **Wireshark filter:** N/A (text format, distributed via AFTN or HTTP APIs)

METAR (Meteorological Aerodrome Report) and TAF (Terminal Aerodrome Forecast) are standardized text formats for aviation weather. METAR reports current observed conditions at an airport, issued every 30 or 60 minutes. TAF provides a forecast for the next 24-30 hours. Both use a compact, fixed-order encoding that pilots and dispatchers can read at a glance. SPECI is a special unscheduled METAR issued when conditions change significantly. These reports are distributed via AFTN (Aeronautical Fixed Telecommunication Network), AMHS, NOAA Aviation Weather Center, and numerous aviation weather APIs.

METAR Field Layout

[Diagram: see online version]

Note: METAR is a text format -- the packet diagram above represents the logical field order, not binary framing.

Example Decode

Raw METAR:

```
METAR KJFK 211756Z 31012G20KT 10SM FEW250 M02/M17 A3042 RMK AO2 SLP308
```

Field	Value	Meaning
Type	`METAR`	Routine weather observation
ICAO	`KJFK`	John F. Kennedy International Airport
Day/Time	`211756Z`	21st day of month, 17:56 UTC
Wind	`31012G20KT`	From 310 degrees, 12 knots gusting 20 knots
Visibility	`10SM`	10 statute miles
Clouds	`FEW250`	Few clouds at 25,000 ft AGL
Temp/Dew	`M02/M17`	Temperature -2C, dewpoint -17C
Altimeter	`A3042`	30.42 inHg altimeter setting
Remarks	`RMK AO2 SLP308`	Automated station type 2, sea-level pressure 1030.8 hPa

Key Fields

Field	Format	Description
Type	`METAR` or `SPECI`	METAR = routine, SPECI = special (unscheduled, significant change)
ICAO Identifier	4 letters	Airport code (e.g., KJFK, EGLL, LFPG)

Date/Time	`ddhhmmZ`	Day of month, hour, minute, always Zulu (UTC)
Wind	`dddssGggKT`	Direction (degrees true), speed, gusts, in knots
Visibility	`nnSM` or `nnnn`	Statute miles (US) or meters (ICAO)
RVR	`Rnn/nnnnFT`	Runway Visual Range -- specific runway, in feet or meters
Weather	`[+-]xxxx`	Present weather phenomena (intensity + descriptor + type)
Clouds	`xxxnnn[CB/TCU]`	Coverage + height in hundreds of feet AGL
Temperature	`TT/TD`	Temperature / dewpoint in Celsius, M prefix = negative
Altimeter	`Annnn` or `Qnnnn`	inHg (US) or hPa (ICAO)
Remarks	`RMK ...`	Supplementary data (automated station type, sea-level pressure, etc.)

Field Details

Wind

Format	Example	Meaning
`dddssKT`	`27015KT`	From 270 degrees at 15 knots
`dddssGggKT`	`31012G25KT`	From 310 at 12 gusting 25 knots
`VRBssKT`	`VRB03KT`	Variable direction at 3 knots
`dddssKT dddVddd`	`24010KT 210V270`	Variable between 210 and 270 degrees
`00000KT`	`00000KT`	Calm

Visibility

Format	Region	Example	Meaning
`nnSM`	US	`10SM`	10 statute miles
`n n/nSM`	US	`1 1/2SM`	1.5 statute miles
`nnnn`	ICAO	`9999`	10 km or greater
`nnnn`	ICAO	`0800`	800 meters
`CAVOK`	ICAO	`CAVOK`	Ceiling and Visibility OK (vis >10 km, no significant weather, no clouds below 5000 ft)

Weather Phenomena

Intensity

Prefix	Meaning
`-`	Light
(none)	Moderate
`+`	Heavy
`VC`	In vicinity (5-10 SM from airport)

Descriptor

Code	Meaning
`TS`	Thunderstorm
`SH`	Showers
`FZ`	Freezing
`DR`	Low drifting
`BL`	Blowing
`MI`	Shallow
`PR`	Partial
`BC`	Patches

Precipitation

Code	Meaning
`RA`	Rain
`SN`	Snow
`DZ`	Drizzle
`GR`	Hail (>5mm)
`GS`	Small hail/snow pellets
`PL`	Ice pellets
`SG`	Snow grains
`IC`	Ice crystals
`UP`	Unknown precipitation (automated)

Obscuration

Code	Meaning
`FG`	Fog (visibility <1 km)

BR	Mist (visibility 1-5 km)
HZ	Haze
FU	Smoke
SA	Sand
DU	Dust

Cloud Coverage

Code	Coverage	Oktas
SKC	Sky clear	0 (manual)
CLR	Clear	0 (automated, no clouds below 12,000 ft)
FEW	Few	1-2
SCT	Scattered	3-4
BKN	Broken	5-7
OVC	Overcast	8
VV	Vertical visibility (sky obscured)	N/A

Height is encoded in hundreds of feet AGL: SCT025 = scattered at 2,500 ft, OVC005 = overcast at 500 ft. Suffix CB indicates cumulonimbus, TCU indicates towering cumulus.

Temperature / Dewpoint

Format	Example	Meaning
TT/TD	22/15	22C temperature, 15C dewpoint
MTT/MTD	M02/M17	-2C temperature, -17C dewpoint
TT/	15/	Dewpoint missing

Altimeter Setting

Format	Region	Example	Meaning
Annnn	US, Canada	A3042	30.42 inches of mercury
Qnnnn	ICAO (rest of world)	Q1013	1013 hPa (hectopascals)

TAF (Terminal Aerodrome Forecast)

TAF follows a similar field structure but adds forecast change groups:

TAF Example

```
TAF KJFK 211730Z 2118/2224 31015G25KT P6SM FEW250
  TEMPO 2118/2122 4SM -SNBR BKN020
  FM220200 28008KT P6SM SCT040
  BECMG 2210/2212 18012KT
```

TAF Change Groups

Keyword	Format	Description
`FM`	`FMddhhmm`	From -- abrupt change at specified time
`BECMG`	`BECMG ddhh/ddhh`	Becoming -- gradual change over time period
`TEMPO`	`TEMPO ddhh/ddhh`	Temporary -- fluctuations expected during period
`PROB30`	`PROB30 ddhh/ddhh`	30% probability of conditions during period
`PROB40`	`PROB40 ddhh/ddhh`	40% probability of conditions during period

TAF Validity

Format	Example	Meaning
`ddhh/ddhh`	`2118/2224`	Valid from 21st 18:00Z to 22nd 24:00Z (30 hours)

SPECI (Special Observation)

SPECI is issued when significant changes occur between routine METAR reports:

Trigger	Condition
Visibility	Drops below or rises above 3, 2, 1.5, 1, 0.5 SM thresholds
Ceiling	Drops below or rises above 3000, 1500, 1000, 500, 200 ft thresholds
Tornado/Funnel	Observed
Thunderstorm	Begins or ends at the airport
Freezing precip	Begins or ends
Wind shift	Direction change of 45+ degrees in <15 minutes

Distribution

[Diagram: see online version]

Related Formats

Format	Description
METAR	Routine aviation weather observation
SPECI	Special (unscheduled) observation
TAF	Terminal area forecast (24-30 hours)
SIGMET	Significant meteorological information (en-route)
AIRMET	Airmen's meteorological information (less severe)
PIREP	Pilot report (in-flight weather conditions)
NOTAM	Notices to Air Missions (runway closures, hazards)

Standards

Document	Title
ICAO Annex 3	Meteorological Service for International Air Navigation
WMO Manual on Codes (WMO-306)	International Codes -- Vol I.1, Part A (FM 15 METAR, FM 51 TAF)
FAA Order 7900.5	Surface Weather Observing (ASOS/AWOS)
FAA AC 00-45	Aviation Weather Services

See Also

- ACARS -- data link that carries weather requests and METAR uplinks to aircraft
- ASTERIX -- ATC surveillance data (CAT 008 carries weather radar data)
- CPDLC -- digital ATC communications (weather may prompt re-routes)

Mode S / Mode A/C / TCAS -- Secondary Surveillance Radar

Standard: ICAO Annex 10 Vol IV / RTCA DO-260B | **Layer:** Physical / Data Link (1090 MHz RF) | **Wireshark filter:** N/A (radio protocol, decoded by dump1090 / readsb)

Mode S (Mode Select) is a secondary surveillance radar (SSR) interrogation-reply protocol used by ATC to identify and track aircraft. It extends the earlier Mode A (identity/squawk code) and Mode C (altitude) with 24-bit unique aircraft addressing, selective interrogation, and data link capabilities. Mode S is also the physical layer for ADS-B Extended Squitter (DF=17) and provides the surveillance foundation for TCAS (Traffic Alert and Collision Avoidance System). All interrogations transmit on 1030 MHz; all replies and squitters transmit on 1090 MHz.

Mode A/C (Legacy SSR)

Mode A and Mode C are the original SSR transponder modes, still in widespread use alongside Mode S.

Mode A (Identity)

Parameter	Value
Purpose	Aircraft identity (squawk code)
Encoding	12 pulse positions -> 4 octal digits (0000-7777)
Total codes	4096
Interrogation	1030 MHz, P1-P3 spacing = 8 us

Special Squawk Codes

Code	Meaning
7700	Emergency / distress
7600	Radio failure (communications lost)
7500	Hijack / unlawful interference
1200	VFR (US default)
2000	Entering controlled airspace (ICAO default)

Mode C (Altitude)

Parameter	Value
Purpose	Pressure altitude
Encoding	12-bit Gillham (Gray) code
Resolution	100 ft increments
Range	-1200 to 126,700 ft
Interrogation	1030 MHz, P1-P3 spacing = 21 us

Mode S Short Message (56 bits)

[Diagram: see online version]

Mode S Long Message (112 bits)

[Diagram: see online version]

Key Fields

Field	Size	Description
DF	5 bits	Downlink Format -- identifies the reply type
CA	3 bits	Capability -- transponder capability level (in DF=11/17)
AA	24 bits	Aircraft Address -- ICAO 24-bit unique identifier
AP	24 bits	Address/Parity -- CRC-24 remainder XORed with ICAO address
PI	24 bits	Parity/Interrogator ID -- for addressed replies
ME	56 bits	Extended Squitter message (in DF=17, contains ADS-B data)
MB	56 bits	Comm-B message field (in DF=20/21)

Downlink Formats (DF)

DF	Name	Length	Description
0	Short Air-Air Surveillance	56 bits	ACAS/TCAS short reply
4	Surveillance Altitude Reply	56 bits	Altitude in response to ground interrogation
5	Surveillance Identity Reply	56 bits	Squawk code in response to ground interrogation
11	All-Call Reply	56 bits	Acquisition squitter -- broadcast ICAO address and capability
16	Long Air-Air Surveillance	112 bits	ACAS/TCAS long reply (with resolution advisory data)
17	Extended Squitter (ADS-B)	112 bits	Unsolicited broadcast: position, velocity, ID
18	Extended Squitter (non-transponder)	112 bits	TIS-B and ADS-R messages
20	Comm-B with Altitude	112 bits	Ground interrogation response with BDS register data + altitude
21	Comm-B with Identity	112 bits	Ground interrogation response with BDS register data + squawk

Mode S Interrogation / Reply

[Diagram: see online version]

TCAS (Traffic Alert and Collision Avoidance System)

TCAS II interrogates nearby Mode S and Mode A/C transponders to build a picture of surrounding traffic and generate collision avoidance advisories independent of ATC.

TCAS Advisory Types

Advisory	Abbreviation	Description
Traffic Advisory	TA	Warning: traffic detected, pilot awareness
Resolution Advisory	RA	Command: climb, descend, or maintain vertical rate

TCAS Resolution Advisory (RA) Types

RA	Vertical Command
Climb	Climb at 1500-2500 fpm
Descend	Descend at 1500-2500 fpm
Adjust Vertical Speed	Reduce climb/descent rate
Maintain Vertical Speed	Continue current vertical rate
Monitor Vertical Speed	Do not exceed specified rate
Level Off	Return to level flight
Crossing	Climb/descend through intruder's altitude
Reversal	Reverse a previous RA

TCAS Interrogation / Reply Sequence

[Diagram: see online version]

TCAS Parameters

Parameter	Value
Surveillance range	~14 nm (Mode S), ~7 nm (Mode A/C)
TA threshold	~35 seconds to CPA (varies by altitude)
RA threshold	~25 seconds to CPA (varies by altitude)
Update rate	~1 second
Vertical resolution	25 ft or 100 ft

RF Parameters

Parameter	Value
Interrogation frequency	1030 MHz
Reply frequency	1090 MHz
Pulse modulation	PPM (Mode A/C), DPSK-on-pulse (Mode S)
Mode S short reply	56 bits, 8 us preamble + 56 us data
Mode S long reply	112 bits, 8 us preamble + 112 us data
Bit rate	1 Mbps
Power (transponder)	Typically 125-500 W peak

Hobbyist Decoding

Tool	Description
RTL-SDR	USB software-defined radio dongle, 1090 MHz reception
dump1090	Open-source Mode S / ADS-B decoder
readsb	Enhanced fork of dump1090 with additional features
tar1090	Web-based visualization for dump1090/readsb
FlightAware / Flightradar24	Community feed networks aggregating 1090 MHz data

Encapsulation

[Diagram: see online version]

Standards

Document	Title
ICAO Annex 10 Vol IV	Surveillance and Collision Avoidance Systems
RTCA DO-260B	Minimum Operational Performance Standards for 1090 MHz ADS-B
RTCA DO-181E	Minimum Operational Performance Standards for TCAS II
EUROCAE ED-73E	TCAS II MOPS (European equivalent of DO-181E)
ICAO Doc 9871	Technical Provisions for Mode S Services and Extended Squitter

See Also

- ACARS -- VHF data link messaging for aircraft
- ASTERIX -- ATC surveillance data exchange format (carries decoded Mode S data)
- CPDLC -- digital controller-pilot communications

Bus & IC Protocols

Low-level bus and inter-IC communication protocols for hardware and embedded systems.

16 protocols in this section

ARINC 429 -- Aeronautical Radio, Incorporated

Standard: ARINC Specification 429 | **Layer:** Physical / Data Link | **Wireshark filter:** N/A (avionics bus; dedicated analyzer equipment)

ARINC 429 is the most widely used avionics data bus in commercial aircraft. Defined in 1977 and still in use today, it provides a simple, reliable, simplex (one-way) broadcast link where a single transmitter sends 32-bit data words to up to 20 receivers over a twisted-pair cable. The protocol encodes avionics parameters -- airspeed, altitude, heading, navigation data, and system status -- in a well-defined word format with self-identifying labels. Its simplicity and fault tolerance make it the backbone of flight data exchange on aircraft from the Boeing 737 to the Airbus A320.

32-Bit Word Format

[Diagram: see online version]

Key Fields

Field	Bits	Size	Description
Label	0-7	8 bits	Parameter identifier (transmitted LSB first, encoded in octal)
SDI	8-9	2 bits	Source/Destination Identifier -- distinguishes multiple sources or receivers
Data	10-28	19 bits	Parameter value (BNR, BCD, or discrete)
SSM	29-30	2 bits	Sign/Status Matrix -- data validity and sign
Parity	31	1 bit	Odd parity over all 32 bits

Field Details

Label (Bits 0-7)

Labels are 8-bit identifiers transmitted LSB first and conventionally expressed in octal. The label identifies what parameter the word carries:

Label (Octal)	Parameter	Description
010	Discrete Data 1	System discretes
011	Computed Airspeed	Airspeed in knots
012	Mach Number	Mach speed

014	True Airspeed	TAS in knots
035	Altitude (baro)	Barometric altitude
036	Altitude Rate	Vertical speed
044	Heading (true)	True heading in degrees
045	Heading (mag)	Magnetic heading in degrees
101	ILS Localizer Deviation	Lateral guidance
102	ILS Glideslope Deviation	Vertical guidance
110	VOR Bearing	VOR station bearing
133	DME Distance	Distance to station
203	Latitude	Aircraft latitude
204	Longitude	Aircraft longitude
251	Time of Day	UTC time
270	Discrete Word	System status discretes
310	GPS Latitude	GPS position
311	GPS Longitude	GPS position
350	Radio Altitude	Radar altimeter

SDI (Bits 8-9)

SDI	Meaning
00	All receivers / no source ID
01	Source/receiver 1 (e.g., left system)
10	Source/receiver 2 (e.g., right system)
11	Source/receiver 3 (e.g., center system)

In some label definitions, SDI bits are incorporated into the data field, extending it to 21 bits.

Data Encoding Types

BNR (Binary)

Two's complement binary representation with defined resolution and range:

Example Label	Parameter	Resolution	Range
203 (Latitude)	Degrees	180/2^20	-180 to +180
035 (Altitude)	Feet	131072/2^19	-131072 to +131072
011 (Airspeed)	Knots	0.0625	0 to 1024

BCD (Binary Coded Decimal)

Each 4-bit nibble encodes one decimal digit (0-9):

Bit Range	Content
10-13	Ones digit
14-17	Tens digit
18-21	Hundreds digit
22-25	Thousands digit
26-28	Ten-thousands (3 bits)

Discrete

Individual bits represent ON/OFF states, flags, or enumerated selections:

Bit	Meaning (example)
10	System ready
11	Warning flag
12-13	Mode selection (00=off, 01=mode1, 10=mode2)

SSM (Bits 29-30)

The SSM interpretation depends on the data encoding type:

BNR Data SSM

SSM	Meaning
00	Failure Warning
01	No Computed Data
10	Functional Test
11	Normal Operation

BCD Data SSM

SSM	Meaning
00	Plus, North, East, Right, To, Above
01	No Computed Data
10	Functional Test
11	Minus, South, West, Left, From, Below

Discrete Data SSM

SSM	Meaning

00	Verified Data / Normal
01	No Computed Data
10	Functional Test
11	Failure Warning

Electrical Characteristics

Parameter	High Speed	Low Speed
Bit Rate	100 kbps	12.5 kbps
Bit Period	10 us	80 us
Voltage (differential)	+/-10V (+/-1V null)	+/-10V (+/-1V null)
Cable	Shielded twisted pair	Shielded twisted pair
Impedance	75 ohm	75 ohm
Encoding	Bipolar RZ (Return-to-Zero)	Bipolar RZ

Signaling

```
High:   +10V ■■■    ■■■ +10V
                ■    ■
Null:    0V  ■■■■    ■■■ 0V
                ■    ■
Low:   -10V ■■■■■    ■■■ -10V

         Bit 1    Bit 0    Gap
```

- Logic 1 = High-then-Low within the bit period (bipolar return-to-zero)
- Logic 0 = Low-then-High within the bit period
- Null (0V) = bit boundary / inter-word gap

Bus Topology (Simplex)

[Diagram: see online version]

A single transmitter broadcasts to up to 20 receivers. Each parameter stream requires a dedicated wire pair. Bidirectional communication requires two separate buses (one in each direction).

Typical Aircraft Wiring

[Diagram: see online version]

Word Transmission

Parameter	Value

Word length	32 bits
Inter-word gap	Minimum 4 bit times (40 us at high speed)
Transmission rate	Up to ~2500 words/sec (high speed)
Bit order	LSB first (label), MSB first (data)

Note: The label field is transmitted LSB first (bit 0 first), which is why octal label numbers appear bit-reversed relative to the wire order. The data field is transmitted MSB first.

Standards

Document	Title
ARINC 429	Mark 33 Digital Information Transfer System (DITS)
ARINC 429P1-18	Part 1 -- Functional Description, Electrical Interface, Label Assignments
ARINC 429P2-19	Part 2 -- Standard Data Formats
ARINC 429P3-17	Part 3 -- Data Standards

See Also

- MIL-STD-1553 -- military avionics bus (bidirectional, command/response)
- CAN -- automotive bus (similar simplicity, different domain)

Art-Net

Standard: Art-Net 4 (Artistic Licence) | **Layer:** Application (Layer 7) | **Wireshark filter:** `artnet`

Art-Net is an open protocol for transmitting DMX512 lighting data over Ethernet/IP networks. It encapsulates DMX universes in UDP packets (port 6454), allowing lighting consoles to control fixtures across standard IT infrastructure instead of dedicated DMX cabling. Art-Net 4 supports up to 32,768 universes (over 16 million channels), discovery of nodes, RDM tunneling, and synchronization. It is the most widely deployed DMX-over-Ethernet protocol, used in concerts, theatre, architectural lighting, and broadcast studios.

Packet Header

All Art-Net packets begin with a common header:

[Diagram: see online version]

The 8-byte ASCII string `Art-Net\0` (null-terminated) identifies the packet. The OpCode determines the packet type.

Key Packet Types

OpCode	Name	Description
0x2000	ArtPoll	Controller discovers nodes on the network
0x2100	ArtPollReply	Node responds with capabilities and status
0x5000	ArtDmx (ArtOutput)	**DMX data** -- carries one universe of 512 channels
0x5200	ArtSync	Synchronization -- nodes hold output until sync received
0x7000	ArtTodRequest	Request Table of Devices (RDM discovery)
0x7100	ArtTodData	Table of Devices response (RDM UIDs)
0x7200	ArtTodControl	RDM discovery control
0x7300	ArtRdm	Tunneled RDM message
0x6000	ArtAddress	Re-program node settings (universe, name, merge mode)
0xF000	ArtInput	Set input enable/disable on a node
0x9700	ArtTimeCode	Distribute SMPTE time code
0x9800	ArtTimeSync	NTP-based time synchronization

ArtDmx Packet (OpCode 0x5000)

The most common packet -- carries one universe of DMX data:

[Diagram: see online version]

Field	Size	Description
ID	8 bytes	`Art-Net\0` identifier
OpCode	16 bits	0x5000 (little-endian)
ProtVer	16 bits	Protocol version (14)
Sequence	8 bits	Sequence counter (1-255, 0 = disable reordering)
Physical	8 bits	Physical DMX port this data came from
SubUni	8 bits	Low byte of the 15-bit universe address
Net	8 bits	High 7 bits of the 15-bit universe address
Length	16 bits	DMX data length (2-512, must be even, big-endian)

Data	2-512 bytes	DMX channel values (channel 1 at offset 0)

Universe Addressing

Art-Net 4 uses a 15-bit universe address:

[Diagram: see online version]

Field	Range	Calculation
Universe (low 4 bits)	0-15	
Sub-Net (next 4 bits)	0-15	
Net (upper 7 bits)	0-127	
Total address	0-32,767	Net x 256 + Sub-Net x 16 + Universe
Total channels	16,711,680	32,768 universes x 512 channels

ArtPoll / ArtPollReply (Discovery)

ArtPoll (broadcast to 2.255.255.255 or 10.255.255.255) discovers all Art-Net nodes:

[Diagram: see online version]

ArtPollReply Key Fields

Field	Description
IP Address	Node's IP
Port	UDP port (6454)
Short Name	18-char node name
Long Name	64-char node description
Num Ports	Number of DMX ports (1-4 typical)
Port Types	Input/output capability per port
Good Input/Output	Port status flags
SwIn/SwOut	Universe assignments per port
Style	Node type (controller, node, media server, etc.)

ArtSync (Synchronization)

Without sync, nodes update their DMX output as each ArtDmx packet arrives, causing visible tearing across universes. ArtSync solves this:

[Diagram: see online version]

Merge Modes

When multiple controllers send to the same universe, nodes merge using:

Mode	Description
HTP (Highest Takes Precedence)	Per-channel maximum value wins (default)
LTP (Latest Takes Precedence)	Most recent packet wins

Art-Net IP Addressing

Art-Net traditionally uses the `2.x.x.x` or `10.x.x.x` private subnet:

Network	Range	Usage
2.x.x.x	2.0.0.1 - 2.255.255.254	Art-Net primary (class A private)
10.x.x.x	10.0.0.1 - 10.255.255.254	Art-Net alternative
Any	DHCP-assigned	Modern Art-Net 4 supports any subnet

Broadcast address for discovery: `2.255.255.255` or `10.255.255.255` (or subnet broadcast).

Art-Net vs sACN

Feature	Art-Net	sACN (E1.31)
Standard body	Artistic Licence (proprietary, free)	ESTA/ANSI (open standard)
Transport	UDP unicast + broadcast	UDP **multicast** (or unicast)
Port	6454	5568
Discovery	ArtPoll/ArtPollReply	E1.31 Universe Discovery
Max universes	32,768	63,999
Sync	ArtSync	E1.31 Synchronization (Universe 63999)
RDM support	ArtRdm (tunneled)	Draft (E1.33 RDMnet)
Priority	HTP/LTP merge	Per-source priority (0-200)
Multicast	No (unicast + broadcast)	Yes (efficient for many receivers)
Adoption	Dominant in live entertainment	Growing, preferred for install

Encapsulation

[Diagram: see online version]

Standards

Document	Title
Art-Net 4 Specification	Art-Net 4 Protocol (Artistic Licence, freely available)
Art-Net 4 OEM Codes	Manufacturer OEM code registration

See Also

- sACN (E1.31) -- standards-based alternative (multicast)
- DMX512 -- the lighting data Art-Net carries
- Ethernet -- physical transport
- UDP -- Art-Net transport

CAN -- Controller Area Network

Standard: ISO 11898 | **Layer:** Data Link / Physical | **Wireshark filter:** `can` (with SocketCAN capture)

CAN is a robust, multi-master serial bus developed by Bosch in 1986 for automotive applications. It uses differential signaling, priority-based bitwise arbitration, and built-in error detection to provide reliable communication in electrically noisy environments. CAN is the backbone of automotive networking (OBD-II, J1939), industrial automation (CANopen, DeviceNet), and medical devices. CAN FD (Flexible Data-Rate) extends the original standard with larger payloads and higher data rates.

Frame (CAN 2.0A -- Standard Frame)

[Diagram: see online version]

Frame (CAN 2.0B -- Extended Frame)

[Diagram: see online version]

Key Fields

Field	Size	Description
SOF	1 bit	Start of Frame -- dominant (0), synchronizes all nodes
Identifier	11 or 29 bits	Message priority and ID (lower value = higher priority)

RTR	1 bit	Remote Transmission Request -- 0 = data, 1 = remote request
IDE	1 bit	Identifier Extension -- 0 = standard (11-bit), 1 = extended (29-bit)
DLC	4 bits	Data Length Code -- number of data bytes (0-8)
Data	0-64 bits	Payload (0-8 bytes)
CRC	15 bits	Cyclic Redundancy Check
ACK	1 bit	Acknowledgment -- receiver drives dominant to acknowledge
EOF	7 bits	End of Frame -- 7 recessive (1) bits
IFS	3+ bits	Inter-Frame Space -- minimum gap between frames

Field Details

Bus States

State	CAN_H - CAN_L	Logic
Recessive (idle)	~0V differential	1
Dominant	~2V differential	0

A dominant bit always overwrites a recessive bit -- this is how arbitration works.

Arbitration

When multiple nodes transmit simultaneously, they perform bitwise arbitration on the identifier field. Each transmitter monitors the bus -- if it sends recessive (1) but reads dominant (0), another node with higher priority (lower ID) is transmitting and it backs off:

[Diagram: see online version]

Data Length Code (DLC)

CAN 2.0:

DLC	Data Bytes
0-8	0-8 bytes (1:1 mapping)

CAN FD extends this:

DLC	Data Bytes
0-8	0-8

9	12
10	16
11	20
12	24
13	32
14	48
15	64

Error Detection

CAN has five mechanisms for error detection:

Mechanism	Description
CRC	15-bit CRC over frame contents
ACK check	Transmitter checks for dominant ACK bit
Bit monitoring	Transmitter compares sent vs. read values
Bit stuffing	After 5 consecutive same-polarity bits, a stuff bit is inserted
Form check	Fixed-format fields (EOF, delimiters) verified

Error States

Each node maintains transmit and receive error counters:

State	Condition	Behavior
Error Active	TEC < 128 and REC < 128	Normal operation, sends active error flags
Error Passive	TEC >= 128 or REC >= 128	Can transmit, but with restrictions
Bus Off	TEC >= 256	Node disconnects from bus, requires recovery

CAN FD (Flexible Data-Rate)

Feature	CAN 2.0	CAN FD
Max data bytes	8	64
Arbitration rate	Up to 1 Mbps	Up to 1 Mbps
Data phase rate	Same as arbitration	Up to 8 Mbps
CRC	15-bit	17-bit (<=16 bytes) or 21-bit (>16 bytes)

CAN FD switches to a higher bit rate after arbitration for the data and CRC fields, then switches back for the ACK and EOF.

Bit Rates and Cable Length

Bit Rate	Approximate Max Length
1 Mbps	25 m
500 kbps	100 m
250 kbps	250 m
125 kbps	500 m
50 kbps	1000 m

Common Higher-Layer Protocols

Protocol	Application	Standard
OBD-II	Automotive diagnostics	SAE J1979, ISO 15765
SAE J1939	Heavy-duty vehicles, trucks	SAE J1939
CANopen	Industrial automation	CiA 301
DeviceNet	Factory automation	IEC 62026-3
NMEA 2000	Marine electronics	NMEA 2000 / IEC 61162-3
ISO-TP	Transport protocol (multi-frame)	ISO 15765-2

Bus Topology

[Diagram: see online version]

Linear bus with 120Ω termination at each end. Stubs should be kept short.

Standards

Document	Title
ISO 11898-1	CAN data link layer and physical signaling
ISO 11898-2	CAN high-speed physical layer
ISO 11898-1:2015	Includes CAN FD
Bosch CAN 2.0 Specification	Original CAN specification
CiA 301	CANopen application layer
SAE J1939	CAN-based protocol for commercial vehicles

See Also

- RS-485 -- another differential bus used in industrial settings
- I2C -- simpler bus for on-board IC communication

DALI -- Digital Addressable Lighting Interface

Standard: IEC 62386 | **Layer:** Physical / Data Link / Application | **Wireshark filter:** N/A (physical bus protocol, not IP-based)

DALI is a two-wire digital bus protocol for intelligent lighting control in commercial and industrial buildings. It provides individual addressability of up to 64 luminaires per bus, enabling dimming, color control, scene management, and diagnostics. DALI uses Manchester encoding on a low-voltage, polarity-independent bus at 1200 bps. Originally defined for fluorescent ballasts, DALI has expanded through device type extensions (LED, color, emergency) and DALI-2, which adds input devices like sensors and push buttons.

Forward Frame (Controller to Device)

The forward frame is sent from a controller (application controller or bus power supply with control) to gear (drivers, ballasts) or input devices:

[Diagram: see online version]

Forward Frame Fields

Field	Size	Description
Start bit	1 bit	Always 1 (transition from idle)
Address byte	8 bits	Target address and addressing mode
Data byte	8 bits	Command or direct arc power level
Stop bits	2 bits (idle)	Idle bus state (settling time)

Total forward frame: 19 bits at 1200 bps = ~16.7 ms

Backward Frame (Device to Controller)

The backward frame is a reply from a device, sent within a defined window after a query command:

[Diagram: see online version]

Backward Frame Fields

Field	Size	Description
Start bit	1 bit	Always 1
Data byte	8 bits	Response value (0-255 or status)
Stop bits	2 bits (idle)	Idle bus state

Total backward frame: 11 bits at 1200 bps = ~9.2 ms

Address Byte Format

Bit 7	Bits 6-1	Bit 0	Addressing Mode
0	AAAAAA	S	Short address (0-63), S = selector bit
1	0GGGG	S	Group address (0-15), S = selector bit
1	1111111	S	Broadcast, S = selector bit
1	1111110	S	Broadcast unaddressed

The selector bit (bit 0) determines interpretation of the data byte:

- **S = 0**: Data byte is a direct arc power command (0-254 = dimming level, 255 = MASK)
- **S = 1**: Data byte is a DALI command code

Direct Arc Power

When the selector bit is 0, the data byte sets the output level directly:

Value	Meaning
0	OFF (via fade)
1-254	Logarithmic dimming levels (0.1% to 100%)
255	MASK (do not change)

DALI uses a logarithmic dimming curve following the human eye's perception. The actual light output for level N is approximately: output = 10^((N-1)/(253/3)) percent.

DALI Commands

When the selector bit is 1, the data byte encodes a command:

Arc Power Commands

Code	Command	Description
0	OFF	Immediate off (no fade)

1	UP	Increase light level by one step
2	DOWN	Decrease light level by one step
3	STEP UP	Step up and switch on
4	STEP DOWN	Step down and switch off if minimum
5	RECALL MAX LEVEL	Go to configured maximum level
6	RECALL MIN LEVEL	Go to configured minimum level
7	STEP DOWN AND OFF	Step down, switch off at minimum
8	ON AND STEP UP	Switch on and step up
16-31	GO TO SCENE 0-15	Recall scene 0 through scene 15

Configuration Commands (Sent Twice)

Code	Command	Description
32	RESET	Reset all parameters to defaults
33	STORE ACTUAL DIM LEVEL AS DTR	Store current level into DTR
42	STORE DTR AS MAX LEVEL	Set maximum output level
43	STORE DTR AS MIN LEVEL	Set minimum output level
44	STORE DTR AS SYSTEM FAILURE LEVEL	Level on bus failure
45	STORE DTR AS POWER ON LEVEL	Level on power up
46	STORE DTR AS FADE TIME	Set fade time
47	STORE DTR AS FADE RATE	Set fade rate
64-79	STORE DTR AS SCENE 0-15	Store DTR as scene level
80-95	REMOVE FROM SCENE 0-15	Remove device from scene
96-111	ADD TO GROUP 0-15	Add device to group
112-127	REMOVE FROM GROUP 0-15	Remove device from group

Query Commands

Code	Command	Response
144	QUERY STATUS	Status byte (bits: controlGear failure, lamp failure, limitError, fadeRunning, resetState, missingShort, powerOn)
145	QUERY CONTROL GEAR PRESENT	YES (0xFF) if present
146	QUERY LAMP FAILURE	YES if lamp has failed
148	QUERY LAMP POWER ON	YES if lamp is on
154	QUERY ACTUAL LEVEL	Current light level (0-254)

155	QUERY MAX LEVEL	Configured maximum level
156	QUERY MIN LEVEL	Configured minimum level
160	QUERY POWER ON LEVEL	Level after power cycle
161	QUERY SYSTEM FAILURE LEVEL	Level during bus failure
165	QUERY FADE TIME / FADE RATE	Encoded fade parameters
176-191	QUERY SCENE LEVEL 0-15	Stored scene level
192	QUERY GROUPS 0-7	Group membership bitmask (low)
193	QUERY GROUPS 8-15	Group membership bitmask (high)
197	QUERY DEVICE TYPE	Device type number (0, 6, 8, etc.)

Device Types

DT	Device Type	Description
0	Fluorescent lamp	Original DALI device type
1	Emergency lighting	Self-contained emergency gear
2	HID (discharge lamp)	High-intensity discharge
3	Low-voltage halogen	Halogen transformers
4	Incandescent dimmer	Phase-cut dimmers
5	DC converter	Constant voltage supplies
6	LED driver	LED gear (most common today)
7	Switching function	Relay modules
8	Color control	Tunable white, RGB, RGBW, xy color

DT8 (Color) Extensions

DT8 adds color control commands for tunable white and full-color luminaires:

Feature	Description
Tc (color temperature)	1000-10000 Kelvin (Mirek: 100-1000)
RGBWAF	Individual channel control (Red, Green, Blue, White, Amber, Freecolor)
xy coordinates	CIE 1931 xy chromaticity
Primary N	Direct control of individual LED primaries

Electrical Characteristics

Parameter	Specification

Bus voltage	9.5-22.5 V (typical 16 V from bus power supply)
Maximum current	250 mA per bus
Data rate	1200 bps (Manchester encoded)
Manchester encoding	Bit period = 833.3 us (2 x half-bit of 416.7 us)
Logic high	Bus voltage > 9.5 V
Logic low	Bus voltage < 6.5 V
Polarity	Independent (either wire can be DATA+ or DATA-)
Max bus length	~300 m (depending on cable and load)
Max devices	64 per bus (short addresses 0-63)
Max groups	16 per bus
Max scenes	16 per device
Topology	Bus, star, tree, or mixed (no ring)

DALI-2 Enhancements

DALI-2 (IEC 62386 Parts 103 and 3xx) significantly extends the original DALI specification:

Feature	DALI (Edition 1)	DALI-2
Control gear	Yes	Yes (expanded)
Input devices	No	Yes (push buttons, sliders, occupancy sensors, light sensors)
Application controllers	Single master only	Multi-master support
Event-based messaging	No	Yes (instance-based events from input devices)
Bus power supply	Unspecified	Standardized (Part 101)
Interoperability testing	Limited	Mandatory certification program
Device types	DT0-DT8	DT0-DT8 + new device types
24-bit frames	No	Yes (extended frame for device addressing)

DALI-2 Input Device Types (Part 3xx)

Part	Device Type	Examples
301	Push button	Wall switches, scene panels
302	Absolute input	Sliders, rotary encoders
303	Occupancy sensor	PIR, ultrasonic, microwave sensors
304	Light sensor	Daylight harvesting
305	Color sensor	Measurement of illuminance and color

Bus Topology

[Diagram: see online version]

Addressing and Commissioning

[Diagram: see online version]

Standards

Document	Title
IEC 62386-101	General requirements -- System components
IEC 62386-102	General requirements -- Control gear
IEC 62386-103	General requirements -- Control devices (DALI-2)
IEC 62386-201	Control gear -- Fluorescent lamps (DT0)
IEC 62386-207	Control gear -- LED gear (DT6)
IEC 62386-209	Control gear -- Color control (DT8)
IEC 62386-301	Input devices -- Push buttons
IEC 62386-303	Input devices -- Occupancy sensors
IEC 62386-304	Input devices -- Light sensors

See Also

- DMX512 -- entertainment lighting control protocol
- I2C -- another two-wire serial bus (different domain)

DMX512 -- Digital Multiplex

Standard: ANSI E1.11 (USITT DMX512-A) | **Layer:** Data Link / Physical | **Wireshark filter:** `dmx` (via Art-Net or sACN capture)

DMX512 is the standard protocol for controlling stage lighting, moving heads, fog machines, LED fixtures, and other entertainment equipment. It transmits up to 512 channels of 8-bit data per universe over a single RS-485 bus at 250 kbps. Each channel carries a value from 0 to 255, and fixtures are assigned a starting address to determine which channels they respond to. DMX512 is unidirectional -- the controller broadcasts and fixtures listen.

Packet

A DMX512 packet consists of a Break, Mark After Break, Start Code, and up to 512 data slots:

[Diagram: see online version]

Each slot is framed as a standard UART byte: 1 start bit + 8 data bits + 2 stop bits = 11 bits at 250 kbps.

Key Fields

Field	Duration / Size	Description
Break	>= 88 µs (low)	Signals start of a new packet; resets all receivers
MAB (Mark After Break)	>= 8 µs (high)	Gap between Break and first byte
Start Code	1 byte (8 bits)	Identifies the packet type (0x00 = dimmer data)
Slot 1-512	1 byte each	Channel data (0-255 per channel)
Inter-Slot Time	>= 0 µs	Optional gap between slots
Mark Before Break	>= 0 µs	Optional gap before next packet

Field Details

Timing

Parameter	Minimum	Typical	Maximum
Break	88 µs	88-176 µs	--
MAB	8 µs	8-12 µs	< 1 sec
Slot time	44 µs	44 µs	--
Packet (512 ch)	--	~23 ms	--
Refresh rate (512 ch)	--	~44 Hz	--
Refresh rate (24 ch)	--	~830 Hz	--

Fewer channels = faster refresh. A controller can send fewer than 512 slots per packet.

Start Codes

Code	Purpose
0x00	Dimmer/intensity data (standard lighting)
0x17	Text packet (ASCII)
0x55	Test packet

0x91	SIP (System Information Packet)
0xCC	RDM (Remote Device Management, ANSI E1.20)
0xCF	Sub-device discovery

Channel Assignment Example

A typical RGB LED fixture uses 3 channels:

Channel	Function	Value Range
Start + 0	Red	0-255
Start + 1	Green	0-255
Start + 2	Blue	0-255

A moving head might use 16+ channels (pan, tilt, color wheel, gobo, intensity, zoom, etc.).

Universe Addressing

One DMX512 link = one universe = 512 channels. Large installations use multiple universes:

Method	Description
Multiple RS-485 links	One physical cable per universe
Art-Net	DMX over Ethernet/IP (UDP port 6454)
sACN (E1.31)	Streaming ACN -- DMX over Ethernet/IP (multicast UDP)

Physical Layer

Parameter	Specification
Electrical	RS-485 differential
Data rate	250 kbps
Connector	5-pin XLR (ANSI standard) or 3-pin XLR (common but non-standard)
Cable	Shielded twisted pair, 120Ω characteristic impedance
Termination	120Ω at the last fixture in the daisy chain
Max cable length	300 m (1000 ft) per run
Max devices	32 unit loads per universe
Topology	Daisy chain (bus)

XLR Pinout (5-pin)

Pin	Signal
1	Signal Ground
2	Data – (cold)
3	Data + (hot)
4	Optional second data pair –
5	Optional second data pair +

Bus Topology

[Diagram: see online version]

RDM (Remote Device Management)

ANSI E1.20 extends DMX512 with bidirectional communication on the same RS-485 bus, allowing:

- Device discovery and addressing
- Remote configuration (mode, personality, label)
- Sensor readback (temperature, lamp hours, status)

RDM uses start code 0xCC and interleaves with normal DMX packets.

Standards

Document	Title
ANSI E1.11	USITT DMX512-A -- Entertainment Technology
ANSI E1.20	RDM -- Remote Device Management
ANSI E1.31	sACN -- Streaming Architecture for Control Networks (DMX over IP)
Art-Net 4	Art-Net -- DMX over Ethernet (Artistic Licence)

See Also

- RS-485 -- physical layer DMX512 runs on
- UART -- framing used for each DMX slot
- MIDI -- another entertainment control protocol (music)

HDMI CEC -- Consumer Electronics Control

Standard: HDMI 1.4+ Specification (CEC Supplement) | **Layer:** Application (over HDMI physical) | **Wireshark filter:** N/A (hardware bus; protocol analyzer / logic analyzer)

HDMI CEC is a single-wire, bidirectional serial bus that allows HDMI-connected devices to control each other. It runs on pin 13 of the HDMI connector at 400 bps, enabling features like one-touch play (turn on a TV by pressing play on a Blu-ray player), system standby (one remote powers off all devices), and automatic input switching. CEC is implemented under various vendor brand names: Samsung Anynet+, LG SimpLink, Sony Bravia Sync, and Roku 1-Touch Play, among others.

Bus Characteristics

Parameter	Value
Signal	Single-wire, open-drain with pull-up
HDMI Pin	Pin 13 (CEC)
Bit Rate	~400 bps
Voltage	3.3V logic levels
Max Devices	15 (logical addresses 0-14; 15 = broadcast)
Protocol	CSMA/CA-like with bit-level arbitration

CEC Frame

Start Bit

[Diagram: see online version]

Start bit: 3.7 ms low, 0.8 ms high.

Header Block (Initiator + Destination)

[Diagram: see online version]

Data Block

[Diagram: see online version]

Complete CEC Frame

[Diagram: see online version]

Key Fields

Field	Size	Description
Start Bit	1 bit time	3.7 ms low + 0.8 ms high -- marks frame start
Initiator	4 bits	Logical address of the sender
Destination	4 bits	Logical address of the target (0xF = broadcast)
EOM	1 bit	End of Message -- 1 = last block, 0 = more blocks follow
ACK	1 bit	Acknowledgment -- receiver pulls low to acknowledge
Opcode	8 bits	Command code (see opcode table)
Operands	0-14 bytes	Opcode-specific parameters

Logical Addresses

Address	Device Type	Description
0	TV	Television display
1	Recording Device 1	DVR, PVR
2	Recording Device 2	Second recording device
3	Tuner 1	Set-top box, satellite receiver
4	Playback Device 1	Blu-ray, DVD, streaming box
5	Audio System	AV receiver, soundbar
6	Tuner 2	Second tuner
7	Tuner 3	Third tuner
8	Playback Device 2	Second playback device
9	Recording Device 3	Third recording device
10	Tuner 4	Fourth tuner
11	Playback Device 3	Third playback device
12-13	Reserved	Future use
14	Specific Use	Free use / backup
15	Broadcast	Unregistered / broadcast

Key Opcodes

Opcode	Name	Direction	Description
0x04	Image View On	Source -> TV	Turn on the TV
0x0D	Text View On	Source -> TV	Turn on TV, display text

0x36	Standby	Any -> Any	Put the destination device in standby
0x44	User Control Pressed	Any -> Any	Remote control key press (play, pause, etc.)
0x45	User Control Released	Any -> Any	Remote control key release
0x46	Give OSD Name	Any -> Any	Request the device's OSD name
0x47	Set OSD Name	Any -> Any	Response with device's display name
0x70	System Audio Mode Request	Source -> Audio	Request audio system to handle audio
0x72	Set System Audio Mode	Audio -> All	Enable/disable system audio mode
0x80	Routing Change	Switch -> All	Active route has changed
0x82	Active Source	Source -> Broadcast	Declare this device as the active source
0x83	Give Physical Address	Any -> Any	Request physical address
0x84	Report Physical Address	Any -> Broadcast	Report physical address + device type
0x85	Request Active Source	TV -> Broadcast	Ask which device is the active source
0x86	Set Stream Path	TV -> Broadcast	Direct all switches to route to this path
0x8F	Give Power Status	Any -> Any	Request power status
0x90	Report Power Status	Any -> Any	Reply with power status (on/standby/transition)
0x9E	CEC Version	Any -> Any	Report supported CEC version
0xFF	Abort	Any -> Any	Feature not supported or refused

Physical Address

The CEC physical address encodes the device's position in the HDMI topology as a 4-level address (a.b.c.d):

Address	Meaning
0.0.0.0	TV (root)
1.0.0.0	Device on HDMI port 1 of the TV
2.0.0.0	Device on HDMI port 2 of the TV
2.1.0.0	Device on port 1 of a switch on TV's HDMI port 2

One-Touch Play

[Diagram: see online version]

System Standby

[Diagram: see online version]

Routing and Input Switching

[Diagram: see online version]

Bit Timing

Timing	Low Period	High Period	Total
Start Bit	3.7 ms	0.8 ms	4.5 ms
Logic 0	1.5 ms	0.9 ms	2.4 ms
Logic 1	0.6 ms	1.8 ms	2.4 ms

Each data bit has a fixed total period of 2.4 ms, with the duration of the low period distinguishing 0 from 1. The initiator drives the line low; receivers sample mid-bit.

Arbitration

CEC uses a mechanism similar to CAN arbitration. During the header block, if two devices transmit simultaneously, the one sending a 0 (longer low pulse) wins over the one sending 1 -- the losing device detects the bus mismatch and backs off.

Standards

Document	Title
HDMI 1.4 Specification	HDMI Specification (includes CEC supplement)
HDMI 2.1 Specification	HDMI 2.1 (enhanced CEC features)
CEC-O-Matic	Community CEC frame decoder and reference

See Also

- USB -- another device interconnect with control capabilities
- I2C -- CEC's signaling shares similarities with I2C (open-drain, arbitration)

I2C -- Inter-Integrated Circuit

Standard: NXP I2C-bus specification | **Layer:** Data Link / Physical | **Wireshark filter:** N/A (sub-packet-capture; logic analyzer protocols)

I2C (pronounced "I-squared-C" or "I-two-C") is a synchronous, multi-master, multi-slave serial bus invented by Philips (now NXP) in 1982. It uses just two bidirectional lines -- SDA (data) and SCL (clock) -- with open-drain drivers and pull-up resistors. I2C is ubiquitous for connecting low-speed peripherals on a PCB: sensors, EEPROMs, RTCs, ADCs, DACs, display controllers, and I/O expanders.

Bus Signals

Signal	Description
SDA	Serial Data -- bidirectional, open-drain
SCL	Serial Clock -- driven by master, open-drain
GND	Common ground

Both lines idle high (pulled up by resistors, typically 4.7kΩ for 100 kHz).

Frame

Write Transaction

[Diagram: see online version]

Read Transaction

[Diagram: see online version]

Key Fields

Field	Size	Description
S (Start)	1 condition	SDA falls while SCL is high
Address	7 or 10 bits	Slave device address
R/W	1 bit	0 = Write (master to slave), 1 = Read (slave to master)
ACK/NACK	1 bit	0 = ACK (acknowledged), 1 = NACK (not acknowledged)
Data	8 bits	Data byte, MSB first
P (Stop)	1 condition	SDA rises while SCL is high

Sr (Repeated Start)	1 condition	Start without preceding Stop (for combined transactions)

Field Details

Start and Stop Conditions

```
SDA:  ■■■\___    ___/■■■
SCL:  ■■■■■■■    ■■■■■■■
      START       STOP
```

- **Start**: SDA transitions high->low while SCL is high
- **Stop**: SDA transitions low->high while SCL is high
- During normal data transfer, SDA only changes while SCL is low

Addressing

Mode	Address Bits	Addresses Available
7-bit	7	112 (16 reserved)
10-bit	10	1024

Reserved 7-bit addresses:

Address	Purpose
0x00	General Call
0x01	CBUS compatibility
0x02	Reserved for different bus formats
0x03	Reserved for future use
0x04-0x07	High-speed master code
0x78-0x7B	10-bit addressing prefix (11110xx)
0x7C-0x7F	Reserved for future use

10-bit Addressing

Uses a two-byte address with the first byte containing the prefix `11110` + upper 2 address bits + R/W:

[Diagram: see online version]

ACK/NACK

- The **receiver** pulls SDA low during the 9th clock cycle to acknowledge
- **ACK** (SDA low): Byte received successfully
- **NACK** (SDA high): No device at address, or receiver signals end of read

Clock Stretching

A slave can hold SCL low to pause the master when it needs more time to process data. The master must check SCL before proceeding.

Speed Modes

Mode	Max Clock	Max Capacitance
Standard	100 kHz	400 pF
Fast	400 kHz	400 pF
Fast-plus	1 MHz	550 pF
High-speed	3.4 MHz	400 pF
Ultra-fast	5 MHz	-- (push-pull, unidirectional)

Common I2C Device Addresses

Address (7-bit)	Device Type
0x20-0x27	PCF8574 I/O expander
0x48-0x4F	TMP102/LM75 temperature sensor
0x50-0x57	AT24C EEPROM
0x68	DS1307/DS3231 RTC
0x68/0x69	MPU-6050 IMU
0x76/0x77	BME280/BMP280 pressure/temperature
0x3C/0x3D	SSD1306 OLED display
0x27	HD44780 LCD (via PCF8574)

Bus Topology

[Diagram: see online version]

All devices share SDA and SCL with pull-up resistors.

Standards

Document	Title
NXP UM10204	I2C-bus specification and user manual (Rev. 7.0)
SMBus 3.2	System Management Bus specification (I2C variant)

See Also

- SPI -- faster alternative for point-to-point IC communication
- I2S -- related Philips/NXP bus for digital audio
- 1-Wire -- simpler single-wire bus for sensors

I2S -- Inter-IC Sound

Standard: NXP I2S bus specification | **Layer:** Data Link / Physical | **Wireshark filter:** N/A (sub-packet-capture; logic analyzer protocols)

I2S (pronounced "I-squared-S") is a synchronous serial bus designed specifically for digital audio data transfer between integrated circuits. Developed by Philips (now NXP) in 1986 and revised in 1996, it provides a simple, standardized way to connect audio codecs, DACs, ADCs, DSPs, and microcontrollers. I2S carries PCM audio data in a time-division multiplexed format with separate clock and word-select lines.

Bus Signals

Signal	Description
SCK (BCLK)	Serial Clock / Bit Clock -- one pulse per audio bit
WS (LRCLK)	Word Select / Left-Right Clock -- selects the audio channel
SD	Serial Data -- audio sample bits, MSB first

Some configurations add:

Signal	Description
MCLK	Master Clock -- typically 256x or 384x the sample rate

Frame

I2S uses a continuous stream. The Word Select signal toggles to indicate left and right channels:

[Diagram: see online version]

Timing

```
WS:    ■■■■■■■■■■\_________________________/■■■■■■■■■■■■■■■■■■■■■■■■■
SCK:   _/■\_/■\_/■\_/■\_/■\_/■\_/■\_/■\_/■\_/■\_/■\_/■\_/■\_/■\_/■\_
SD:    ==X MSB X b1 X b2 X ...                X MSB X b1 X b2 X ...
              LEFT CHANNEL                        RIGHT CHANNEL
```

Key timing detail: In standard I2S, the data transitions one clock cycle **before** the WS transition (the MSB is clocked in on the clock after WS changes). This one-cycle offset distinguishes I2S from other formats.

Key Fields

Field	Description
Word Select = 0	Left channel (or Channel 1)
Word Select = 1	Right channel (or Channel 2)
Data bits	Audio sample, MSB first, two's complement
Bit depth	Commonly 16, 24, or 32 bits per channel

Field Details

Word Select Timing

Standard	WS Transition Relative to MSB
I2S (Philips)	WS changes 1 BCLK before MSB (data delayed by 1 clock)
Left-Justified	WS changes aligned with MSB (no delay)
Right-Justified	MSB aligned to end of WS period
DSP/TDM	Short WS pulse, data starts immediately

Clock Relationships

Parameter	Formula	Example (44.1 kHz, 16-bit stereo)
Sample Rate (Fs)	--	44,100 Hz
Bit Clock (BCLK)	Fs x bits x channels	44,100 x 16 x 2 = 1.4112 MHz
Master Clock (MCLK)	Fs x 256 (typical)	44,100 x 256 = 11.2896 MHz

Common Sample Rates

Rate	Application
8 kHz	Telephony
16 kHz	Wideband voice
44.1 kHz	CD audio
48 kHz	Professional audio, video soundtracks
96 kHz	High-resolution audio
192 kHz	Studio recording

Common Bit Depths

Bits	Dynamic Range	Usage
16	96 dB	CD quality, most consumer audio
24	144 dB	Professional audio, modern DACs
32	192 dB	Internal processing, some high-end DACs

TDM (Time Division Multiplexing)

TDM extends I2S to more than 2 channels by dividing each WS period into time slots:

[Diagram: see online version]

TDM is used for multi-channel audio (5.1/7.1 surround, microphone arrays, multi-channel ADC/DAC).

Bus Topology

[Diagram: see online version]

I2S has one master that generates SCK and WS. The master can be the transmitter, the receiver, or a separate controller.

I2S vs SPI vs I2C

Feature	I2S	SPI	I2C
Purpose	Audio data	General-purpose	General-purpose
Wires	3 (+ optional MCLK)	4 + CS	2
Clock	Continuous	Burst	Burst
Data format	MSB first, two's complement	Configurable	MSB first
Duplex	Simplex per data line	Full	Half
Addressing	None (dedicated connections)	CS lines	7/10-bit address

Standards

Document	Title
NXP I2S bus specification (1996)	Original I2S standard
NXP UM11732	I2S bus specification Rev. 2

See Also

- I2C -- related NXP bus for control/configuration (often used alongside I2S)
- SPI -- general-purpose serial bus with similar signal count

JTAG / SWD

Standard: IEEE 1149.1 (JTAG), ARM Debug Interface Architecture (ADIv5) | **Layer:** Physical / Debug | **Wireshark filter:** N/A (hardware debug interface)

JTAG (Joint Test Action Group) and SWD (Serial Wire Debug) are the two dominant interfaces for debugging, programming, and testing integrated circuits. JTAG was originally designed for boundary scan testing of solder joints on PCBs (IEEE 1149.1, 1990) and was later extended for on-chip debug access. SWD is ARM's 2-wire alternative to JTAG, offering the same debug capabilities with fewer pins. Together they are used for flashing firmware, setting breakpoints, reading/writing memory, and production testing on virtually every modern microcontroller and processor.

JTAG (IEEE 1149.1)

Bus Signals

Signal	Direction	Description
TCK	Input	Test Clock -- clocks the TAP state machine
TMS	Input	Test Mode Select -- controls state transitions on TCK rising edge
TDI	Input	Test Data In -- serial data into the device
TDO	Output	Test Data Out -- serial data out of the device
TRST	Input (optional)	Test Reset -- active-low asynchronous reset of TAP controller

IDCODE Register (32 bits)

[Diagram: see online version]

The IDCODE register identifies the device. Bit 0 is always 1 (distinguishes IDCODE from BYPASS). The 11-bit manufacturer ID follows the JEDEC JEP106 standard. The part number and version are vendor-defined.

Key Fields

Field	Size	Description
Instruction Register (IR)	Device-specific (2+ bits)	Selects which data register is active
BYPASS Register	1 bit	Passes data through with 1-clock delay (all 1s instruction)
Boundary Scan Register (BSR)	1 bit per pin	Captures/drives every I/O pin on the device
IDCODE Register	32 bits	Device identification (version + part + manufacturer + 1)

Mandatory Instructions

Instruction	IR Code	Description
BYPASS	All 1s	Selects 1-bit bypass register (shortest path through device)
EXTEST	Implementation-defined	Drives BSR values onto pins for testing interconnects
SAMPLE/PRELOAD	Implementation-defined	Captures pin states into BSR without affecting device

Common Debug Instructions (ARM)

Instruction	Description
IDCODE	Selects the 32-bit IDCODE register
DPACC	Access to Debug Port registers (ARM CoreSight)
APACC	Access to Access Port registers (ARM CoreSight)
ABORT	Abort current AP transaction

TAP State Machine

The JTAG TAP (Test Access Port) controller is a 16-state finite state machine. TMS is sampled on the rising edge of TCK to determine the next state.

[Diagram: see online version]

TAP State Summary

State	Purpose
Test-Logic-Reset	All test logic disabled, device operates normally
Run-Test/Idle	Idle state between operations
Select-DR-Scan	Entry point for data register operations
Capture-DR	Load parallel data into the selected data register

Shift-DR	Serially shift data through TDI -> Data Register -> TDO
Update-DR	Latch shifted data into the data register output
Select-IR-Scan	Entry point for instruction register operations
Capture-IR	Load status into instruction register
Shift-IR	Serially shift new instruction in via TDI
Update-IR	Latch new instruction -- selects the active data register
Exit1/Exit2, Pause	Intermediate states for multi-step operations

Daisy-Chaining

Multiple JTAG devices share a single bus. TDO of one device connects to TDI of the next, forming a serial chain. TCK and TMS are shared. The debugger shifts instructions for all devices in the chain simultaneously, using BYPASS for devices it does not need to access.

[Diagram: see online version]

SWD (Serial Wire Debug)

Bus Signals

Signal	Direction	Description
SWCLK	Input	Serial Wire Clock -- driven by debug probe
SWDIO	Bidirectional	Serial Wire Data I/O -- driven by probe or target (turnaround)

SWD uses just two wires to provide the same debug access as JTAG's four. SWDIO is bidirectional -- the probe and target take turns driving it, with a turnaround period in between.

SWD Packet (Read)

[Diagram: see online version]

SWD Packet (Write)

[Diagram: see online version]

Key Fields

Field	Size	Description
Start	1 bit	Always 1 -- marks packet start

APnDP	1 bit	0 = Debug Port access, 1 = Access Port access
RnW	1 bit	0 = Write, 1 = Read
A[2:3]	2 bits	Register address within the selected port
Parity (request)	1 bit	Even parity over APnDP + RnW + A[2:3]
Stop	1 bit	Always 0
Park	1 bit	Always 1 (drive high before turnaround)
Turnaround (Trn)	1 bit	Bus direction change -- neither side drives
ACK	3 bits	Target response: OK, WAIT, or FAULT
Data	32 bits	Read or write data
Parity (data)	1 bit	Even parity over the 32 data bits

ACK Responses

ACK Value	Bits	Meaning
OK	001	Transaction completed successfully
WAIT	010	Target busy, retry later
FAULT	100	Error -- check CTRL/STAT for details

Debug Port (DP) Registers

Register	Address	Description
DPIDR	0x00 (read)	DP Identification Register
CTRL/STAT	0x04	Control and status (power-up, error flags)
SELECT	0x08	Select AP and register bank
RDBUFF	0x0C (read)	Read buffer for previous AP read result

Access Port (AP) Registers (MEM-AP)

Register	Description
CSW	Control/Status Word -- configures transfer size, auto-increment
TAR	Transfer Address Register -- target memory address
DRW	Data Read/Write -- reads or writes data at TAR address

Through MEM-AP, the debugger can read and write any memory-mapped address on the target, enabling flash programming, register inspection, and breakpoint control.

SWD Read/Write Sequence

[Diagram: see online version]

SWD Multidrop (ADIv5.2+)

SWD multidrop allows multiple targets on a single SWCLK/SWDIO bus. The debug probe sends a TARGETSEL command (a write to DP address 0x0C with specific target ID) to select which device responds. Unselected devices ignore subsequent transactions. This is ARM's answer to JTAG daisy-chaining with fewer wires.

JTAG vs SWD

Feature	JTAG	SWD
Wires	4 (+ optional TRST)	2
Direction	Unidirectional TDI/TDO	Bidirectional SWDIO
Multi-device	Daisy-chain (serial)	Multidrop (target select, ADIv5.2+)
Debug capability	Full (CoreSight, RISC-V DMI)	Full (ARM CoreSight only)
Boundary scan	Yes (IEEE 1149.1)	No
Speed	Up to ~50 MHz typical	Up to ~50 MHz typical
Architectures	ARM, RISC-V, MIPS, x86, FPGA, any	ARM only
Pin sharing	Fixed function	SWDIO shared with TMS, SWCLK shared with TCK
Production test	Yes (boundary scan, BIST)	No (debug only)

SWD pins are typically shared with JTAG: SWCLK maps to TCK, and SWDIO maps to TMS. A debug probe can switch between JTAG and SWD mode using a special bit sequence on the shared lines.

Common Debug Probes

Probe	Interfaces	Description
SEGGER J-Link	JTAG, SWD	Industry-standard commercial probe, wide device support
ST-Link	JTAG, SWD	Bundled with STMicroelectronics development boards
CMSIS-DAP	JTAG, SWD	ARM's open-source debug probe standard (DAPLink)
OpenOCD	JTAG, SWD	Open-source debug software, supports many probe hardware
Black Magic Probe	JTAG, SWD	Open-source probe with built-in GDB server

Lauterbach TRACE32	JTAG, SWD, trace	High-end commercial probe with trace capture

Common Use Cases

Use Case	Interface	Description
Flash programming	JTAG or SWD	Write firmware to on-chip or external flash memory
Breakpoint debugging	JTAG or SWD	Set hardware breakpoints, single-step, inspect registers
Memory read/write	JTAG or SWD	Direct access to any memory-mapped address
Boundary scan test	JTAG only	Test solder joints and PCB interconnects in production
FPGA programming	JTAG	Load bitstreams into FPGAs (Xilinx, Intel/Altera)
Chip identification	JTAG (IDCODE)	Read device ID to verify correct part
Trace capture	JTAG + SWO/ETM	Capture instruction trace, ITM printf, profiling data

Standards

Document	Title
IEEE 1149.1-2013	Standard Test Access Port and Boundary-Scan Architecture
IEEE 1149.7	Reduced-pin JTAG (cJTAG, 2-wire JTAG)
ARM ADIv5	ARM Debug Interface Architecture Specification v5
ARM ADIv6	ARM Debug Interface Architecture Specification v6
RISC-V Debug Spec	RISC-V External Debug Support (uses JTAG DMI)
CMSIS-DAP	ARM Cortex Microcontroller Software Interface Standard -- Debug Access Port

See Also

- SPI -- synchronous serial bus (similar clock + data topology)
- I2C -- 2-wire bus with addressing (compare with SWD's 2-wire approach)
- USB -- debug probes typically connect to host via USB
- UART -- serial debug output (SWO/ITM often uses UART framing)
- ARINC 429 -- avionics data bus

MIDI -- Musical Instrument Digital Interface

Standard: MIDI 1.0 / MIDI 2.0 (MMA/AMEI) | **Layer:** Data Link / Physical / Application | **Wireshark filter:** N/A (sub-packet-capture; USB-MIDI via `usbmidi`)

MIDI is a protocol for communicating musical performance data -- notes, control changes, program changes, and timing -- between electronic instruments, computers, and controllers. Defined in 1983, it became the universal language for electronic music. The original MIDI 1.0 uses a 31.25 kbps serial link with 5-pin DIN connectors. Modern MIDI travels over USB, Bluetooth LE, and Ethernet (RTP-MIDI). MIDI 2.0 (2020) adds higher resolution, bidirectional negotiation, and per-note control.

MIDI 1.0 Message

MIDI messages consist of a Status byte followed by one or two Data bytes:

[Diagram: see online version]

Field	Size	Description
Status Byte	8 bits	High bit always 1; identifies message type and channel
Data Byte 1	7 bits	First parameter (high bit always 0)
Data Byte 2	7 bits	Second parameter (some messages have only one)

The high bit distinguishes Status (1) from Data (0), enabling receivers to resynchronize mid-stream.

Channel Messages

The Status byte encodes the message type (upper nibble) and channel (lower nibble, 0-15):

[Diagram: see online version]

Channel Voice Messages

Status	Type	Data 1	Data 2	Description
0x8n	Note Off	Note (0-127)	Velocity (0-127)	Release a note
0x9n	Note On	Note (0-127)	Velocity (0-127)	Play a note (velocity 0 = Note Off)
0xAn	Poly Aftertouch	Note (0-127)	Pressure (0-127)	Per-note pressure
0xBn	Control Change	Controller (0-127)	Value (0-127)	Knobs, pedals, switches
0xCn	Program Change	Program (0-127)	--	Change patch/preset

0xDn	Channel Aftertouch	Pressure (0-127)	--	Channel-wide pressure
0xEn	Pitch Bend	LSB (0-127)	MSB (0-127)	14-bit pitch bend (center = 0x2000)

Common Control Change Numbers

CC	Name	Description
0	Bank Select (MSB)	Selects bank of programs
1	Modulation Wheel	Vibrato/modulation amount
7	Channel Volume	Main volume
10	Pan	Left-right positioning
11	Expression	Sub-volume (performance dynamics)
64	Sustain Pedal	On/Off (0-63 = off, 64-127 = on)
120	All Sound Off	Silence all voices
121	Reset All Controllers	Reset CCs to defaults
123	All Notes Off	Release all held notes

Note Numbers

MIDI Note	Name	Octave
0	C-1	-1
21	A0	0 (lowest piano key)
60	C4	4 (Middle C)
69	A4	4 (440 Hz tuning reference)
108	C8	8 (highest piano key)
127	G9	9

System Messages

System messages have no channel number:

System Common

Status	Name	Description
0xF1	MTC Quarter Frame	MIDI Time Code
0xF2	Song Position	14-bit position in song (MIDI beats)
0xF3	Song Select	Select a song (0-127)

0xF6	Tune Request	Request oscillator tuning

System Real-Time

Single-byte messages that can be inserted between any other bytes:

Status	Name	Description
0xF8	Timing Clock	24 pulses per quarter note (PPQN)
0xFA	Start	Start playback from beginning
0xFB	Continue	Resume playback
0xFC	Stop	Stop playback
0xFE	Active Sensing	Keepalive (optional, every 300ms)
0xFF	System Reset	Reset all devices

System Exclusive (SysEx)

Variable-length manufacturer-specific messages:

[Diagram: see online version]

Manufacturer IDs are assigned by the MMA. Three-byte IDs use 0x00 as the first byte.

Physical Layer (MIDI 1.0)

Parameter	Specification
Data rate	31.25 kbps (±1%)
Encoding	Asynchronous serial, 8N1 (start + 8 data + stop)
Connector	5-pin DIN 180°
Signal	Current loop (5 mA) via opto-isolator
Direction	Unidirectional per cable
Cable	Shielded twisted pair, max 15 m

DIN Connector Pinout

Pin	Function
1	Not connected (shield on some cables)
2	Shield / Ground
3	Not connected
4	Current source (+5V via 220Ω)

5	Current sink (data, via 220Ω)

The receiver uses an opto-isolator to electrically isolate the two devices, preventing ground loops.

MIDI Ports

Port	Direction	Description
MIDI OUT	Transmit	Sends messages from this device
MIDI IN	Receive	Receives messages (opto-isolated)
MIDI THRU	Forward	Copies MIDI IN to pass along the chain

Modern Transports

Transport	Standard	Description
USB-MIDI	USB Device Class	Most common modern connection
BLE-MIDI	Apple/MMA spec	Bluetooth Low Energy MIDI
RTP-MIDI	RFC 6295	MIDI over IP (network sessions)
MIDI 2.0	MMA/AMEI	32-bit resolution, bidirectional, profiles

MIDI 2.0

Feature	MIDI 1.0	MIDI 2.0
Velocity resolution	7-bit (128 values)	16-bit (65,536 values)
Controller resolution	7-bit (or 14-bit paired)	32-bit
Per-note controllers	No (Poly Aftertouch only)	Yes (pitch, CC, management)
Bidirectional	No	Yes (capability inquiry)
Profiles	No	Yes (standardized device behaviors)
Property Exchange	No	Yes (JSON-based configuration)

Standards

Document	Title
MIDI 1.0 Specification	MIDI 1.0 Detailed Specification (MMA/AMEI)
MIDI 2.0 Specification	MIDI 2.0 (MMA/AMEI, 2020)
General MIDI (GM)	Standard instrument/drum map
RFC 6295	RTP Payload Format for MIDI

USB MIDI Class	USB Device Class Definition for MIDI Devices

See Also

- DMX512 -- entertainment control for lighting (similar ecosystem)
- UART -- MIDI 1.0 uses asynchronous serial framing
- RS-485 -- sometimes used for long-distance MIDI runs

MIL-STD-1553 -- Military Standard 1553B

Standard: MIL-STD-1553B | **Layer:** Physical / Data Link | **Wireshark filter:** N/A (military bus; dedicated analyzer equipment)

MIL-STD-1553 is a military standard that defines a serial data bus for interconnecting avionics subsystems in aircraft, spacecraft, and ground vehicles. First published in 1973 (current revision 1553B from 1978), it uses a dual-redundant, transformer-coupled, differential bus operating at 1 Mbps with Manchester II bi-phase encoding. The bus follows a strict command/response architecture: a single Bus Controller (BC) orchestrates all communication by issuing commands to up to 31 Remote Terminals (RTs), while optional Bus Monitors (BMs) passively record traffic. MIL-STD-1553 remains in use on platforms ranging from the F-16 to the International Space Station.

Word Format (20 bits)

Every MIL-STD-1553 transmission consists of 20-bit words: 3 sync bits, 16 data/command bits, and 1 odd parity bit.

Command Word

[Diagram: see online version]

Data Word

[Diagram: see online version]

Status Word

[Diagram: see online version]

Key Fields

Field	Size	Description
Sync	3 bit times	Synchronization pattern -- distinguishes word types

RT Address	5 bits	Remote Terminal address (0-30 valid; 31 = broadcast)
T/R	1 bit	Transmit/Receive -- 1 = RT transmits, 0 = RT receives
Subaddress	5 bits	Sub-unit or data type identifier within the RT
Word Count	5 bits	Number of data words (0 = 32 words)
Data	16 bits	Payload data
Parity	1 bit	Odd parity over bits 3-18

Sync Patterns

Word Type	Sync Pattern	Description
Command Word	Positive-going	+V then -V (3 bit times)
Status Word	Positive-going	+V then -V (same as command)
Data Word	Negative-going	-V then +V (3 bit times)

The sync pattern is invalid Manchester encoding, making it unambiguously distinguishable from data.

Status Word Flags

Bit	Name	Description
8	Message Error (ME)	Error in the received command/data
9	Instrumentation (Inst)	RT is in instrumentation mode
10	Service Request (SR)	RT needs servicing by the BC
11	Reserved	Set to 0
12	Broadcast Command (BC)	RT received a broadcast command
13	Busy	RT cannot process the command
14	Subsystem Flag (SF)	Subsystem-defined condition
15	Dynamic Bus Accept (DBA)	RT accepts dynamic bus control
16	Terminal Flag (TF)	RT has a fault condition
17-18	Reserved	Set to 0

Bus Architecture

[Diagram: see online version]

Component Roles

Component	Count	Role
Bus Controller (BC)	1 active	Initiates all transfers; manages bus schedule
Remote Terminal (RT)	Up to 31	Responds to BC commands; interfaces to subsystems
Bus Monitor (BM)	Optional	Passively records all bus traffic (flight recorder)
Backup BC	Optional	Standby controller; can take over if primary BC fails

Message Formats

BC-to-RT Transfer (Receive)

[Diagram: see online version]

RT-to-BC Transfer (Transmit)

[Diagram: see online version]

RT-to-RT Transfer

[Diagram: see online version]

Broadcast Transfer

[Diagram: see online version]

Mode Codes (Subaddress 0x00 or 0x1F)

When the subaddress is 00000 or 11111, the word count field becomes a mode code for bus management:

Mode Code	Name	T/R	Description
00001	Dynamic Bus Control	T	Offer bus control to an RT
00010	Synchronize	R	Synchronize RT with BC timing
00011	Transmit Status Word	T	RT sends its status word
00100	Initiate Self-Test	R	RT runs built-in test
00101	Transmitter Shutdown	R	Disable RT transmitter
00110	Override Transmitter	R	Override transmitter shutdown
01000	Reset Remote Terminal	R	Reset the RT

10000	Transmit Vector Word	T	RT sends an interrupt vector
10001	Synchronize (with data)	R	Synchronize with data word
10010	Transmit Last Command	T	RT echoes last received command
10011	Transmit BIT Word	T	RT sends built-in test results

Electrical Characteristics

Parameter	Value
Data Rate	1 Mbps
Encoding	Manchester II bi-phase
Cable	Shielded twisted pair (dual redundant)
Impedance	70-85 ohm
Voltage	6-9V peak-to-peak (across transformer)
Coupling	Transformer-coupled (direct or stub)
Bus Length	Up to ~100 m (depending on stub count)
Stub Length	Up to 6 m (direct coupled) or 0.3 m (transformer coupled)
Termination	Matched impedance at each end

Manchester II Encoding

```
Clock:  ■ ■■■ ■■■ ■■■ ■■■ ■
        ■■■ ■■■ ■■■ ■■■ ■■■

Bit 1:  ■■■    Transition high-to-low at mid-bit
          ■■■

Bit 0:  ■■■    Transition low-to-high at mid-bit
          ■■■
```

Each bit always has a transition at its center: logic 1 transitions from high to low, logic 0 from low to high. This guarantees clock recovery and DC balance.

Timing

Parameter	Value
Bit period	1.0 us
Word (20 bits)	20 us
Inter-message gap	4-12 us minimum
RT response time	4-12 us (after command received)

Max message length	20 us + (32 x 20 us) = 660 us
Bus utilization	Typically scheduled at 80% or less

Dual Redundancy

MIL-STD-1553 mandates two physically separate buses (Bus A and Bus B):

Feature	Description
Normal operation	BC uses Bus A for all transfers
Fault detected	BC retries on Bus B if Bus A transfer fails
RT requirement	Every RT must connect to both buses
Active bus	Only one bus is active for any given transfer
Failover	BC decides which bus to use per message

Standards

Document	Title
MIL-STD-1553B	Digital Time Division Command/Response Multiplex Data Bus
MIL-HDBK-1553A	Multiplex Applications Handbook
AS15531	SAE commercial equivalent of MIL-STD-1553
STANAG 3838	NATO standardization agreement for 1553
DEF STAN 00-18	UK Defence Standard (1553 equivalent)

See Also

- ARINC 429 -- commercial avionics bus (simpler, unidirectional)
- CAN -- automotive/industrial bus (multi-master, similar robustness goals)

1-Wire

Standard: Maxim/Dallas proprietary (public datasheets) | **Layer:** Data Link / Physical | **Wireshark filter:** N/A (sub-packet-capture; logic analyzer protocols)

1-Wire is a serial bus developed by Dallas Semiconductor (now Maxim/Analog Devices) that uses a single data wire plus ground for both communication and parasitic power. Each 1-Wire device has a globally unique 64-bit ROM code burned in at the factory, enabling multiple devices on a single wire without addressing conflicts. 1-Wire is commonly used for temperature sensors (DS18B20), iButtons, battery monitors, and secure authentication ICs.

Bus Signals

Signal	Description
DQ	Data line -- bidirectional, open-drain with pull-up (typically 4.7kΩ)
GND	Ground
VDD	Power supply (optional -- devices can run parasitically from DQ)

The bus idles high. All communication is initiated by the master pulling the line low.

ROM Code (64-bit)

Every 1-Wire device has a unique 64-bit identifier:

[Diagram: see online version]

Field	Size	Description
Family Code	8 bits	Identifies the device type
Serial Number	48 bits	Globally unique serial number
CRC-8	8 bits	CRC of the preceding 56 bits (polynomial: x^8 + x^5 + x^4 + 1)

Common Family Codes

Code	Device
0x10	DS18S20 temperature sensor
0x22	DS1822 temperature sensor
0x28	DS18B20 temperature sensor
0x01	DS1990A iButton (serial number)
0x05	DS2405 addressable switch
0x26	DS2438 battery monitor
0x29	DS2408 8-channel I/O
0x3A	DS2413 2-channel I/O

Transaction Sequence

Every 1-Wire transaction follows the same pattern:

[Diagram: see online version]

Communication Primitives

Reset / Presence

```
DQ:  ■■■■\_________________/■■■■■\____/■■■■■■■■
          |&lt;- 480µs min -&gt;|  |&lt;- 60-240µs -&gt;|
          Reset pulse       Presence pulse
          (master pulls     (slave pulls
           low)              low)
```

Phase	Duration	Description
Reset pulse	480 µs min	Master pulls DQ low
Release	15-60 µs	Master releases, bus floats high
Presence pulse	60-240 µs	Slave pulls DQ low to indicate presence
Recovery	--	Bus returns high

Write Bit

Bit	Timing
Write 0	Master pulls low for 60-120 µs
Write 1	Master pulls low for 1-15 µs, then releases (bus floats high)

Read Bit

Phase	Timing
Master initiates	Pulls low for 1-15 µs
Slave drives	Slave holds low (for 0) or releases (for 1)
Master samples	At 15 µs after falling edge
Slot duration	60-120 µs total

All data is transmitted **LSB first**.

ROM Commands

Command	Code	Description
Search ROM	0xF0	Enumerate all devices on the bus
Read ROM	0x33	Read the 64-bit ROM code (single device only)

Match ROM	0x55	Address a specific device by its 64-bit code
Skip ROM	0xCC	Address all devices (single device or broadcast)
Alarm Search	0xEC	Find devices with alarm flag set

Search ROM Algorithm

The master discovers devices by walking the 64-bit address space bit by bit. At each bit position, devices respond with their bit value, and the master selects a branch. This binary tree search finds all devices in O(N x 64) time slots, where N is the number of devices.

Example: DS18B20 Temperature Reading

[Diagram: see online version]

Parasitic Power

1-Wire devices can draw power from the DQ line through an internal diode and capacitor, eliminating the need for a separate VDD wire. During high-current operations (e.g., temperature conversion), the master must provide a strong pull-up (typically via a MOSFET) on DQ.

Bus Topology

[Diagram: see online version]

All devices share the single DQ line. Each is individually addressable by its unique 64-bit ROM code.

Standards

1-Wire is proprietary to Maxim/Analog Devices but is fully documented in public datasheets:

Document	Title
Maxim AN126	1-Wire Communication Through Software
Maxim AN187	1-Wire Search Algorithm
DS18B20 Datasheet	Programmable Resolution 1-Wire Digital Thermometer

See Also

- I2C -- two-wire bus with more overhead but more features
- SPI -- faster multi-wire alternative for IC communication
- UART -- point-to-point serial alternative

PCIe -- PCI Express

Standard: PCI-SIG PCIe 6.0 | **Layer:** Full stack (Physical, Data Link, Transaction) | **Wireshark filter:** N/A (hardware bus; logic analyzer / protocol analyzer)

PCI Express is the dominant high-speed serial interconnect for connecting processors to GPUs, NVMe SSDs, network adapters, and virtually all expansion cards in modern computers. Unlike its parallel PCI predecessor, PCIe uses point-to-point serial links organized into lanes (x1, x4, x8, x16), with each lane consisting of a differential transmit and receive pair. PCIe defines a three-layer architecture -- Transaction, Data Link, and Physical -- with Transaction Layer Packets (TLPs) carrying the actual read/write/completion operations.

Protocol Stack

[Diagram: see online version]

Transaction Layer Packet (TLP)

3-DW Header (32-bit address)

[Diagram: see online version]

4-DW Header (64-bit address)

[Diagram: see online version]

Full TLP Structure

[Diagram: see online version]

Key Fields

Field	Size	Description
Fmt	3 bits	Format -- 3DW or 4DW header, with/without data
Type	5 bits	Transaction type (memory, I/O, config, completion, message)
TC	3 bits	Traffic Class -- maps to virtual channels for QoS
TD	1 bit	TLP Digest -- ECRC present

EP	1 bit	Poisoned -- data is known bad (error forwarding)
Length	10 bits	Payload length in DWs (0 = 1024 DWs = 4096 bytes)
Requester ID	16 bits	Bus:Device:Function of the requester
Tag	8 bits (10 with extended)	Transaction tag for matching completions
Byte Enables	4+4 bits	First and last DW byte enables
Address	30 or 62 bits	Target memory address (DW-aligned)

TLP Types

Fmt	Type	Name	Has Data	Description
000	00000	MRd (3DW)	No	Memory Read (32-bit address)
001	00000	MRd (4DW)	No	Memory Read (64-bit address)
010	00000	MWr (3DW)	Yes	Memory Write (32-bit address)
011	00000	MWr (4DW)	Yes	Memory Write (64-bit address)
000	00010	IORd	No	I/O Read (legacy)
010	00010	IOWr	Yes	I/O Write (legacy)
000	00100	CfgRd0	No	Configuration Read Type 0
010	00100	CfgWr0	Yes	Configuration Write Type 0
000	00101	CfgRd1	No	Configuration Read Type 1
010	00101	CfgWr1	Yes	Configuration Write Type 1
000	01010	Cpl	No	Completion without data
010	01010	CplD	Yes	Completion with data
010	10000	Msg	Varies	Message (interrupts, PM, errors)

Memory Read/Write Flow

[Diagram: see online version]

Data Link Layer Packet (DLLP)

DLLPs manage the link-level reliability and flow control between adjacent PCIe ports:

[Diagram: see online version]

DLLP Types

Type	Name	Description
0x00	Ack	Acknowledge successful TLP receipt (by sequence number)
0x10	Nak	Negative acknowledge -- request TLP retransmission
0x40-0x4B	InitFC1	Initialize Flow Control (credit advertisement, round 1)
0x50-0x5B	InitFC2	Initialize Flow Control (round 2)
0x60-0x6B	UpdateFC	Update Flow Control credits
0x20	PM_Enter_L1	Power management -- enter L1
0x21	PM_Enter_L23	Power management -- enter L23
0x23	PM_Active_St_Req_L1	Request L1 entry
0x24	PM_Request_Ack	Power management request acknowledged

ACK/NAK Protocol

[Diagram: see online version]

Flow Control

PCIe uses a credit-based flow control system. The receiver advertises available buffer credits for each TLP category:

Credit Type	Description
Posted Header (PH)	Credits for posted request headers (e.g., MWr)
Posted Data (PD)	Credits for posted request data
Non-Posted Header (NPH)	Credits for non-posted request headers (e.g., MRd)
Non-Posted Data (NPD)	Credits for non-posted request data
Completion Header (CplH)	Credits for completion headers
Completion Data (CplD)	Credits for completion data

Physical Layer

Encoding Schemes

Generation	Encoding	Overhead	Description
Gen 1, Gen 2	8b/10b	20%	8 data bits encoded as 10 symbols
Gen 3, Gen 4, Gen 5	128b/130b	1.5%	128 data bits + 2-bit sync header
Gen 6	PAM4 + 242B/256B FEC	~5.5% (net)	4-level signaling with forward error correction

Speed Generations

Gen	Year	Rate (per lane)	Encoding	x16 Throughput
Gen 1	2003	2.5 GT/s	8b/10b	4 GB/s
Gen 2	2007	5 GT/s	8b/10b	8 GB/s
Gen 3	2010	8 GT/s	128b/130b	15.75 GB/s
Gen 4	2017	16 GT/s	128b/130b	31.5 GB/s
Gen 5	2019	32 GT/s	128b/130b	63 GB/s
Gen 6	2022	64 GT/s	PAM4 + FEC	121 GB/s

Lane Widths

Width	Common Use
x1	NVMe SSDs, Wi-Fi cards, low-bandwidth devices
x4	NVMe SSDs, USB controllers, most add-in cards
x8	Network adapters (25/100GbE), RAID controllers
x16	GPUs, high-end network adapters, accelerators

Configuration Space

Every PCIe device exposes a configuration space for enumeration and setup:

Region	Offset	Size	Description
Header (Type 0)	0x00-0x3F	64 bytes	Vendor ID, Device ID, BARs, interrupt info
PCI Capabilities	0x40-0xFF	Variable	MSI, MSI-X, Power Management, PCIe Capability
Extended Config	0x100-0xFFF	3840 bytes	AER, ARI, SR-IOV, LTR, and other extended caps

Key Configuration Registers

Register	Offset	Description
Vendor ID	0x00	Manufacturer (e.g., 0x8086 = Intel, 0x10DE = NVIDIA)
Device ID	0x02	Specific device model
Command	0x04	Enable memory space, bus master, interrupt disable
Status	0x06	Capability list, error indicators
BAR 0-5	0x10-0x24	Base Address Registers (memory / I/O regions)
Interrupt Line/Pin	0x3C-0x3D	Legacy interrupt routing

Power States

State	Description	Exit Latency
D0	Fully active	N/A
D1	Light sleep (context preserved)	Microseconds
D2	Deeper sleep	Milliseconds
D3hot	Software-off (context preserved, aux power)	~10 ms
D3cold	Power removed (full re-initialization needed)	~100 ms
L0	Link active	N/A
L0s	Link standby (fast re-entry)	< 1 us
L1	Link low-power	2-4 us
L2/L3	Link off / auxiliary power	~100 ms

Standards

Document	Title
PCI Express Base Spec 6.0	PCI Express Base Specification Revision 6.0
PCI Express Base Spec 5.0	PCI Express Base Specification Revision 5.0
PCI Express CEM 5.0	Card Electromechanical Specification
PCI Express M.2 Spec	M.2 form factor for NVMe SSDs
Single Root I/O Virtualization (SR-IOV)	PCIe device virtualization for VMs

See Also

- USB -- peripheral interconnect (USB4 tunnels PCIe)
- NVMe-oF -- extends NVMe (which runs on PCIe) over networks

sACN / E1.31 -- Streaming Architecture for Control Networks

Standard: ANSI E1.31-2018 | **Layer:** Application (Layer 7) | **Wireshark filter:** `acn` or `e131`

sACN (Streaming ACN, formally ANSI E1.31) is a standard for transmitting DMX512 lighting data over Ethernet/IP using UDP **multicast**. Each DMX universe is mapped to a multicast group, allowing multiple receivers to efficiently receive the same data without the sender needing to know about them. sACN is an ANSI/ESTA standard (unlike the proprietary Art-Net) and is increasingly preferred for permanent installations, architectural lighting, and large-scale entertainment systems.

Packet Structure

sACN is built on top of ACN (Architecture for Control Networks, E1.17). Each sACN packet has three layers of PDU (Protocol Data Unit):

Root Layer PDU

[Diagram: see online version]

Framing Layer PDU

[Diagram: see online version]

DMP Layer PDU (Device Management Protocol)

[Diagram: see online version]

Key Fields

Field	Size	Description
CID	16 bytes	Component Identifier -- UUID uniquely identifying the sender
Source Name	64 bytes	Human-readable source name (UTF-8, null-padded)
Priority	8 bits	Source priority (0-200, default 100, higher wins)

Sync Address	16 bits	Universe number used for synchronization (0 = no sync)
Sequence	8 bits	Sequence counter (0-255, wrapping)
Options	8 bits	Bit 6 = Force Sync, Bit 5 = Stream Terminated, Bit 7 = Preview
Universe	16 bits	DMX universe number (1-63999)
Start Code	8 bits	DMX start code (0x00 = dimmer data, 0xCC = RDM)
DMX Data	1-512 bytes	Channel values for this universe

Multicast Addressing

Each universe maps to a specific IPv4 multicast address:

```
239.255.{universe_high}.{universe_low}
```

Universe	Multicast Address
1	239.255.0.1
2	239.255.0.2
256	239.255.1.0
1000	239.255.3.232
63999	239.255.249.255

Formula: `239.255.(universe >> 8).(universe & 0xFF)`

Receivers join the multicast group for the universes they need -- no configuration on the sender side.

Priority and Merging

sACN's priority system is its key advantage over Art-Net:

Priority	Usage
0	Lowest (background)
100	Default
200	Highest (emergency override)

Per-Source Priority

When multiple sources send to the same universe:

1. **Highest priority wins** (entire universe from that source)
2. **Equal priority** -- receiver decides (typically HTP merge or reject)

Per-Channel Priority (E1.31-2018, Appendix)

Using start code 0xDD, a source can assign different priorities per channel, allowing fine-grained merging:

```
Source A: Priority 100, channels 1-100 (stage left)
Source B: Priority 100, channels 101-200 (stage right)
Source C: Priority 200, channels 1-200 (emergency override)
```

Synchronization

sACN supports synchronization so multiple universes update simultaneously:

[Diagram: see online version]

The sync universe is a separate multicast group that receivers subscribe to.

Universe Discovery

E1.31 includes a discovery mechanism (added in 2018 revision):

Packet	Description
Universe Discovery	Source periodically multicasts a list of universes it's transmitting

Multicast address for discovery: `239.255.250.0`

Receivers can use this to automatically subscribe to relevant universes.

Options Flags

Bit	Name	Description
5	Stream Terminated	Source is stopping -- receivers should time out gracefully
6	Force Synchronization	Data must not be output until sync packet received
7	Preview Data	Data is for visualization only, not output (lighting previz)

Timing

Parameter	Value
Transmit rate	Up to 44 packets/sec per universe (same as DMX512)
Timeout	2.5 seconds (no data = source lost)

Keep-alive	Source should send at least 1 packet/sec even with no changes
Terminated flag	Send 3 packets with Stream Terminated before stopping

sACN vs Art-Net

Feature	sACN (E1.31)	Art-Net 4
Standard	ANSI/ESTA (open)	Artistic Licence (proprietary, free)
Transport	**Multicast** (+ unicast option)	Unicast + broadcast
Port	5568	6454
Priority	0-200 per source (+ per channel)	HTP/LTP merge only
Discovery	Universe Discovery	ArtPoll/ArtPollReply
Max universes	63,999	32,768
Sync	Sync universe	ArtSync
Multicast efficiency	Excellent (IGMP snooping)	Poor (broadcast or unicast per node)
RDM	Via E1.33 (RDMnet)	ArtRdm (tunneled)
Adoption	Installations, architectural	Live entertainment, concerts
Network impact	Low (IGMP + multicast)	Higher (broadcast/unicast per node)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
ANSI E1.31-2018	Entertainment Technology -- Lightweight streaming protocol for transport of DMX512 using ACN
ANSI E1.17	Architecture for Control Networks (ACN) -- base protocol
ANSI E1.33	RDMnet -- RDM over ACN/sACN networks

See Also

- Art-Net -- alternative DMX-over-Ethernet protocol
- DMX512 -- the lighting data sACN carries
- IGMP -- multicast group management (critical for sACN)
- Ethernet -- physical transport
- UDP -- sACN transport

SPI -- Serial Peripheral Interface

Standard: No formal standard (Motorola de facto) | **Layer:** Data Link / Physical | **Wireshark filter:** N/A (sub-packet-capture; logic analyzer protocols)

SPI is a synchronous, full-duplex serial bus originally developed by Motorola in the 1980s. It uses a master-slave architecture with separate clock, data-in, data-out, and chip-select lines. SPI is faster than I2C (commonly 1-50 MHz, sometimes 100+ MHz) at the cost of more wires. It is widely used for flash memory, SD cards, display controllers, ADCs/DACs, radio transceivers, and any peripheral requiring high throughput.

Bus Signals

Signal	Alternative Names	Description
SCLK	SCK, CLK	Serial Clock -- driven by master
MOSI	SDO, COPI, DI	Master Out Slave In -- data from master to slave
MISO	SDI, CIPO, DO	Master In Slave Out -- data from slave to master
CS	SS, NSS, CSn	Chip Select -- active low, one per slave

Note: The MOSI/MISO terminology is being replaced by COPI (Controller Out Peripheral In) / CIPO (Controller In Peripheral Out) in some specifications.

Frame

SPI transmits data simultaneously in both directions. A typical 8-bit exchange:

[Diagram: see online version]

Each clock cycle shifts one bit out on MOSI and one bit in on MISO simultaneously. There is no addressing, start/stop conditions, or acknowledgment -- the CS line selects the target device.

Key Fields

Element	Description
CS assertion	Master pulls CS low to select a slave
Data bits	Shifted MSB-first (most common) or LSB-first
Clock edge	Data sampled/shifted on configurable edges (see clock modes)

CS deassertion	Master pulls CS high to end the transaction

There is no fixed frame format -- the number of bits, byte order, and meaning are defined by the peripheral's datasheet.

Clock Modes (CPOL / CPHA)

SPI has four clock modes defined by Clock Polarity (CPOL) and Clock Phase (CPHA):

Mode	CPOL	CPHA	Idle State	Sample Edge	Shift Edge
0	0	0	Low	Rising	Falling
1	0	1	Low	Falling	Rising
2	1	0	High	Falling	Rising
3	1	1	High	Rising	Falling

Mode 0 (CPOL=0, CPHA=0) is the most common default.

Timing Diagram (Mode 0)

```
CS:    ■■■\_________________________________/■■■
SCLK:  ____/■\_/■\_/■\_/■\_/■\_/■\_/■\_/■\____
MOSI:  ====X D7 X D6 X D5 X D4 X D3 X D2 X D1 X D0 X====
MISO:  ====X D7 X D6 X D5 X D4 X D3 X D2 X D1 X D0 X====
            ↑    ↑    ↑    ↑    ↑    ↑    ↑    ↑
          Sample on rising edge
```

Bus Topology

Independent Slave (standard)

Each slave has its own CS line:

[Diagram: see online version]

Daisy Chain

Slaves are chained -- MISO of one feeds MOSI of the next. Data shifts through all devices:

[Diagram: see online version]

Used by LED drivers (WS2801), shift registers, and some sensor chains.

Common SPI Speeds

Device Type	Typical Clock	Notes

SD Card (SPI mode)	400 kHz init, 25 MHz data	SPI is the slow/compatible mode
SPI Flash (W25Q)	50-133 MHz	Quad-SPI uses 4 data lines
ADC (MCP3008)	1.35 MHz	10-bit, 200 ksps
Display (ILI9341)	10-40 MHz	SPI TFT LCD
Radio (nRF24L01)	8 MHz	2.4 GHz transceiver
IMU (BMI160)	10 MHz	Accelerometer/gyroscope

SPI vs I2C

Feature	SPI	I2C
Wires	4 + 1 CS per slave	2 (shared)
Speed	1-100+ MHz	100 kHz - 5 MHz
Duplex	Full	Half
Addressing	CS line per device	7/10-bit address
Multi-master	Rare	Supported
Acknowledgment	None	ACK/NACK
Max devices	Limited by CS pins	112+ on shared bus
Complexity	Simple hardware	More complex (start/stop, ACK, arbitration)

Variants

Variant	Data Lines	Description
Standard SPI	1 in, 1 out	Full duplex, single data line each way
Dual SPI	2 bidirectional	Half duplex, 2x throughput
Quad SPI (QSPI)	4 bidirectional	Half duplex, 4x throughput (common for flash)
Octal SPI (OSPI)	8 bidirectional	Half duplex, 8x throughput

Standards

SPI has no formal standard body specification. It is a de facto standard originating from Motorola:

Document	Title
Motorola M68HC11 Reference	Original SPI definition
JEDEC xSPI	JESD251 -- eXpanded SPI standard for flash memory

See Also

- I2C -- lower-speed alternative with fewer wires
- I2S -- audio-specific serial bus (different from SPI despite similar signals)

USB -- Universal Serial Bus

Standard: USB Specification (usb.org) | **Layer:** Full stack (Physical through Application) | **Wireshark filter:** usb

USB is the dominant wired peripheral interconnect, connecting keyboards, mice, storage, cameras, phones, audio devices, and virtually every computer peripheral manufactured since the late 1990s. It uses a host-controlled, polled bus architecture where the host initiates all data transfers. USB has evolved through multiple generations -- from 1.5 Mbps (USB 1.0 Low Speed) to 120 Gbps (USB4). USB Power Delivery enables up to 240W charging over the same cable.

Protocol Stack

[Diagram: see online version]

Token Packet (USB 2.0)

[Diagram: see online version]

Data Packet

[Diagram: see online version]

Packet IDs (PIDs)

Category	PID	Name	Description
Token	0x01	OUT	Host to device transfer
Token	0x09	IN	Device to host transfer
Token	0x05	SOF	Start of Frame (1 ms marker)
Token	0x0D	SETUP	Control transfer setup stage
Data	0x03	DATA0	Data packet (even)
Data	0x0B	DATA1	Data packet (odd -- for toggle)
Data	0x07	DATA2	High-speed isochronous

Data	0x0F	MDATA	High-speed split
Handshake	0x02	ACK	Transfer accepted
Handshake	0x0A	NAK	Device busy, retry later
Handshake	0x0E	STALL	Error -- endpoint halted
Handshake	0x06	NYET	Not yet (high-speed flow control)

Transfer Types

Type	Use Case	Max Size (HS)	Guaranteed Bandwidth	Error Recovery
Control	Device setup, configuration	64 bytes	No	Yes (retry)
Bulk	Large data (storage, printing)	512 bytes	No	Yes (retry)
Interrupt	Periodic input (keyboard, mouse)	1024 bytes	Yes (polling interval)	Yes (retry)
Isochronous	Streaming (audio, video)	1024 bytes	Yes (reserved)	No (no retry)

Control Transfer

[Diagram: see online version]

Setup Packet (Control Transfers)

[Diagram: see online version]

Standard Requests

bRequest	Name	Description
0	GET_STATUS	Read device/endpoint status
1	CLEAR_FEATURE	Clear a feature (e.g., endpoint halt)
5	SET_ADDRESS	Assign a device address (1-127)
6	GET_DESCRIPTOR	Read device, configuration, or string descriptor
9	SET_CONFIGURATION	Activate a device configuration
11	SET_INTERFACE	Select an alternate interface setting

Descriptor Hierarchy

[Diagram: see online version]

Common Device Classes

Class	Code	Description	Examples
HID	0x03	Human Interface Device	Keyboard, mouse, gamepad
Mass Storage	0x08	Block storage (SCSI over USB)	Flash drives, external HDDs
CDC	0x02	Communications Device	USB serial (ACM), Ethernet (ECM/NCM)
Audio	0x01	Audio streaming and control	USB microphones, DACs, headsets
Video	0x0E	Video streaming (UVC)	Webcams
Printer	0x07	Printing	USB printers
Wireless	0xE0	Wireless controllers	Bluetooth adapters
Vendor Specific	0xFF	Custom protocols	Proprietary devices

USB Generations

Version	Year	Speed	Marketing Name	Connector
1.0	1996	1.5 Mbps (Low) / 12 Mbps (Full)	USB	A, B
2.0	2000	480 Mbps	Hi-Speed	A, B, Mini, Micro
3.0	2008	5 Gbps	SuperSpeed	A, B, Micro-B (blue)
3.1	2013	10 Gbps	SuperSpeed+	A, C
3.2	2017	20 Gbps	SuperSpeed 20Gbps	C
USB4	2019	40 Gbps	USB4 40Gbps	C
USB4 v2	2022	120 Gbps	USB4 120Gbps	C

USB Type-C

Feature	Description
Connector	Reversible, 24-pin
USB PD	Up to 240W (48V x 5A) power delivery
Alternate Modes	DisplayPort, Thunderbolt, HDMI over USB-C
USB4	Tunnels USB 3.x, DisplayPort, and PCIe over USB-C

Standards

Document	Title
USB 2.0 Spec	USB 2.0 Specification
USB 3.2 Spec	USB 3.2 Specification
USB4 Spec	USB4 Specification
USB PD Spec	USB Power Delivery Specification
USB Type-C Spec	USB Type-C Cable and Connector Specification

See Also

- I2C -- low-speed IC bus (some USB devices bridge to I2C)
- SPI -- IC bus (some USB devices bridge to SPI)
- Bluetooth -- wireless alternative for peripherals

Data & Model Formats

Serialization, data modeling, and interchange format protocols.

10 protocols in this section

GGUF -- GPT-Generated Unified Format

Standard: GGUF Specification (github.com/ggerganov/ggml) | **Category:** LLM Model Storage Format

GGUF is the model file format for llama.cpp and the broader local LLM inference ecosystem. It stores quantized model weights with rich metadata (architecture, tokenizer, quantization type) in a single self-contained file. GGUF replaced the earlier GGML format and is designed for fast memory-mapped loading -- the entire file can be `mmap()`-ed directly, enabling instant model loading even for large models. GGUF is the format behind Ollama, LM Studio, GPT4All, and most local LLM tools.

File Structure

[Diagram: see online version]

Header

[Diagram: see online version]

Field	Size	Description
Magic	4 bytes	`0x46475547` ("GGUF" in little-endian)
Version	4 bytes	Format version (currently 3)
Tensor Count	8 bytes	Number of tensors in the file
Metadata KV Count	8 bytes	Number of metadata key-value pairs

Metadata

Metadata is stored as key-value pairs with typed values:

Value Types

Type	ID	Description
UINT8	0	Unsigned 8-bit
INT8	1	Signed 8-bit
UINT16	2	Unsigned 16-bit
UINT32	4	Unsigned 32-bit
INT32	5	Signed 32-bit
FLOAT32	6	32-bit float
UINT64	8	Unsigned 64-bit
INT64	9	Signed 64-bit

FLOAT64	10	64-bit float
BOOL	7	Boolean
STRING	3	Length-prefixed UTF-8 string
ARRAY	11	Typed array

Standard Metadata Keys

Key	Type	Description
`general.architecture`	string	Model architecture (llama, mistral, phi, gemma, etc.)
`general.name`	string	Model name
`general.quantization_version`	uint32	Quantization format version
`general.file_type`	uint32	Quantization type (see table below)
`llama.context_length`	uint32	Maximum context length
`llama.embedding_length`	uint32	Hidden size / embedding dimension
`llama.block_count`	uint32	Number of transformer layers
`llama.feed_forward_length`	uint32	FFN intermediate size
`llama.attention.head_count`	uint32	Number of attention heads
`llama.attention.head_count_kv`	uint32	Number of KV heads (GQA)
`llama.rope.freq_base`	float32	RoPE theta base frequency
`llama.vocab_size`	uint32	Vocabulary size
`tokenizer.ggml.model`	string	Tokenizer type (llama, gpt2, etc.)
`tokenizer.ggml.tokens`	string[]	Token vocabulary
`tokenizer.ggml.scores`	float[]	Token scores/priorities
`tokenizer.ggml.token_type`	int[]	Token type (normal, control, byte, etc.)
`tokenizer.ggml.bos_token_id`	uint32	Beginning of sequence token
`tokenizer.ggml.eos_token_id`	uint32	End of sequence token

Quantization Types

File Type	Name	Bits/Weight	Description
0	F32	32	Full precision (huge)
1	F16	16	Half precision
2	Q4_0	4.5	4-bit quantization (basic, fast)
3	Q4_1	5.0	4-bit with offset (slightly better quality)

7	Q8_0	8.5	8-bit quantization (near-lossless)
8	Q5_0	5.5	5-bit quantization
9	Q5_1	6.0	5-bit with offset
10	Q2_K	3.35	2-bit K-quantization (smallest, lower quality)
11	Q3_K_S	3.4375	3-bit K-quant small
12	Q3_K_M	3.9	3-bit K-quant medium
13	Q3_K_L	4.3	3-bit K-quant large
14	Q4_K_S	4.5	4-bit K-quant small
15	Q4_K_M	4.8	4-bit K-quant medium (best quality/size balance)
16	Q5_K_S	5.5	5-bit K-quant small
17	Q5_K_M	5.7	5-bit K-quant medium
18	Q6_K	6.5	6-bit K-quant (high quality)
30	IQ2_XXS	2.06	Importance-weighted 2-bit (cutting edge)
31	IQ2_XS	2.31	Importance-weighted 2-bit
36	IQ4_NL	4.5	Importance-weighted 4-bit non-linear

Quantization Size Comparison (7B parameter model)

Quantization	File Size	RAM Required	Quality
F16	~14 GB	~14 GB	Reference
Q8_0	~7.5 GB	~8 GB	Near-lossless
Q5_K_M	~5.0 GB	~5.5 GB	Excellent
Q4_K_M	~4.2 GB	~4.7 GB	Very good (most popular)
Q3_K_M	~3.3 GB	~3.8 GB	Good
Q2_K	~2.8 GB	~3.3 GB	Acceptable
IQ2_XXS	~1.9 GB	~2.4 GB	Usable (aggressive)

Tensor Info

Each tensor has metadata followed by the raw data:

Field	Type	Description
Name	string	Tensor name (e.g., `blk.0.attn_q.weight`)

N Dimensions	uint32	Number of dimensions
Dimensions	uint64[]	Shape (e.g., [4096, 4096])
Type	uint32	Quantization type for this tensor
Offset	uint64	Byte offset from start of tensor data section

Common Tensor Names (LLaMA)

Pattern	Description
`token_embd.weight`	Token embedding matrix
`blk.{n}.attn_q.weight`	Query weight for layer n
`blk.{n}.attn_k.weight`	Key weight for layer n
`blk.{n}.attn_v.weight`	Value weight for layer n
`blk.{n}.attn_output.weight`	Attention output projection
`blk.{n}.ffn_gate.weight`	FFN gate (SwiGLU)
`blk.{n}.ffn_up.weight`	FFN up projection
`blk.{n}.ffn_down.weight`	FFN down projection
`blk.{n}.attn_norm.weight`	Pre-attention RMSNorm
`blk.{n}.ffn_norm.weight`	Pre-FFN RMSNorm
`output.weight`	Language model head
`output_norm.weight`	Final RMSNorm

GGUF Ecosystem

Tool	Description
llama.cpp	Reference inference engine
Ollama	Desktop LLM manager (uses GGUF)
LM Studio	GUI for local LLM inference
GPT4All	Desktop LLM app
vLLM	High-performance serving (GGUF support added)
text-generation-webui	Web UI for local LLMs
convert-hf-to-gguf.py	Hugging Face -> GGUF converter (in llama.cpp)
llama-quantize	Quantize GGUF models to different types

GGUF vs Other Model Formats

Feature	GGUF	Safetensors	PyTorch (.pt)	ONNX
Primary use	Local LLM inference	Model weights (HF)	Training + inference	Cross-framework
Quantization	Built-in (Q2-Q8, IQ)	External	External	Built-in ops
Tokenizer	Embedded	Separate file	Separate	N/A
Architecture info	Embedded metadata	Separate config.json	Separate	In graph
Memory mapping	Designed for mmap	mmap-safe	Not mmap-safe	Not mmap-safe
Security	Safe (no executable code)	Safe (no pickle)	**Unsafe** (pickle)	Safe (protobuf)
Loading speed	Instant (mmap)	Fast	Slow (deserialize)	Medium

Standards

Resource	Description
GGUF Spec	Format specification
llama.cpp	Reference implementation
Hugging Face GGUF	GGUF support on Hugging Face Hub

See Also

- ONNX -- cross-framework model interchange
- Safetensors -- secure weight storage
- Parquet -- training data format

HDF5 -- Hierarchical Data Format 5

Standard: HDF5 Specification (hdfgroup.org) | **Category:** Scientific / ML Data Storage Format

HDF5 is a binary file format and library for storing large, heterogeneous, multi-dimensional datasets. It organizes data in a filesystem-like hierarchy of groups (directories) and datasets (arrays), with arbitrary metadata attributes on any object. HDF5 is the standard format for scientific computing, satellite imagery, genomics, physics simulations, and was the default model format for Keras/TensorFlow 1.x (`.h5` files). It supports chunking, compression, and partial I/O for datasets too large to fit in memory.

File Structure

[Diagram: see online version]

Key Concepts

Concept	Description
Group	Container (like a directory) -- can contain datasets and other groups
Dataset	N-dimensional array of a single type -- the actual data
Attribute	Key-value metadata attached to any group or dataset
Datatype	Element type (int8, float32, compound, string, etc.)
Dataspace	Shape and dimensionality of a dataset
Chunk	Sub-block of a dataset stored independently (enables compression, partial I/O)
Filter	Compression or transformation pipeline (gzip, SZIP, LZF, shuffle, checksum)

Datatypes

Category	Types
Integer	int8, int16, int32, int64, uint8, uint16, uint32, uint64
Float	float16, float32, float64
String	Fixed-length or variable-length (ASCII or UTF-8)
Compound	Struct-like (named fields of mixed types)
Array	Fixed-size array within each element
Enum	Named integer values
Opaque	Raw bytes (uninterpreted)
Reference	Pointer to another object or region in the file
Variable-length	Variable-size arrays per element

Chunking

Datasets can be stored contiguously or in chunks:

```
Dataset shape: [1000, 1000, 3]  (1M pixels x RGB)
Chunk shape:   [100, 100, 3]    (100x100 pixel tiles)

Reading pixel [500, 500]:
  -&gt; Only chunk [5, 5] (covering [500-599, 500-599]) is read and decompressed
  -&gt; Other 99 chunks are not touched
```

Storage Layout	Description
Contiguous	Single block (fast sequential read, no compression)

Chunked	Independent blocks (enables compression, partial I/O, extensible)
Compact	Stored in metadata (tiny datasets only)

Chunk Size Selection

Data Pattern	Recommended Chunk Shape	Why
Time-series	[N, full_width]	Read chunks of consecutive timesteps
Images	[1, H, W, C]	One image per chunk
3D volume	[32, 32, 32]	Cube-shaped for spatial queries
Training batches	[batch_size, features]	Align with training read pattern

Compression Filters

Filter	Description	Speed	Ratio
gzip	General-purpose (zlib)	Slow	High
LZF	Fast compression	Fast	Medium
SZIP	NASA-developed for scientific data	Medium	High (floating point)
Blosc	Meta-compressor (LZ4, Zstd, etc.)	Very fast	Variable
Shuffle	Reorder bytes for better compression	--	Used with gzip/LZF
Fletcher32	Checksum (error detection, not compression)	--	--

Filters can be chained: `shuffle -> gzip` typically gives 2-5x better compression on numeric data.

ML Usage

Keras Model Format (.h5)

```
model.h5:
  /model_weights/
    /dense_1/
      /dense_1/kernel:0  [784, 128] float32
      /dense_1/bias:0    [128] float32
    /dense_2/
      /dense_2/kernel:0  [128, 10] float32
      /dense_2/bias:0    [10] float32
  /model_config (attribute): JSON string of architecture
  /training_config (attribute): optimizer, loss, metrics
```

Training Data

```
import h5py

with h5py.File('dataset.h5', 'r') as f:
    # Lazy loading -- only reads requested slice from disk
    batch = f['train/images'][1000:1032]  # shape [32, 28, 28]
    labels = f['train/labels'][1000:1032]  # shape [32]
```

HDF5 vs Other Formats

Feature	HDF5	Parquet	TFRecord	NumPy (.npy)	Safetensors
Layout	Hierarchical (groups/datasets)	Columnar	Sequential records	Single array	Flat tensor map
Dimensions	N-dimensional	Tabular (2D)	Serialized protobufs	N-dimensional	N-dimensional
Partial I/O	Yes (chunked)	Yes (row groups, columns)	No (sequential)	No	Yes (offsets)
Compression	Per-chunk (gzip, LZF, etc.)	Per-column (Snappy, ZSTD)	Per-record (gzip)	No	No
Schema	Self-describing (groups + attrs)	Self-describing (footer)	Protobuf schema	dtype + shape only	JSON header
Random access	Yes (by index/slice)	By row group	No (sequential only)	Yes	By tensor name
File size limit	Exabytes (theoretical)	No limit (row groups)	No limit	~2 GB (single array)	No limit
Concurrent write	Limited (SWMR mode)	No (immutable)	Yes (append)	No	No
ML use	Keras weights, scientific data	Training data (tabular)	TensorFlow training	NumPy/Scikit-learn	Model weights (HF)

Common Tools

Tool	Language	Usage
h5py	Python	Primary Python interface
HDFView	Java (GUI)	Visual file explorer
h5dump	C (CLI)	Command-line inspection
PyTables	Python	High-level table interface
netCDF4	Python/C	Climate/weather data (built on HDF5)
Keras	Python	`model.save('model.h5')`

Standards

Document	Title
HDF5 File Format Spec	HDF5 File Format Specification
HDF5 Library	Reference implementation
h5py Documentation	Python HDF5 interface

See Also

- Parquet -- columnar format for tabular ML data
- Safetensors -- modern replacement for HDF5 model storage
- ONNX -- model interchange format
- GGUF -- LLM model format

iCalendar -- Internet Calendaring and Scheduling

Standard: RFC 5545 | **Category:** Data Format | **Wireshark filter:** N/A (data format, not wire protocol)

iCalendar is a text-based data format for representing calendar events, to-do items, journal entries, and free/busy information. It is the universal interchange format for calendaring: when you receive a meeting invitation by email, that is an `.ics` file containing iCalendar data. When your calendar app syncs via CalDAV, the events are stored as iCalendar objects. Every major calendar system -- Google Calendar, Apple Calendar, Outlook, Thunderbird -- reads and writes iCalendar. It is to scheduling what vCard is to contacts.

Format

An iCalendar object is plain text wrapped in `BEGIN:VCALENDAR` / `END:VCALENDAR` delimiters, containing one or more components (events, to-dos, etc.), each with properties in the form `PROPERTY;PARAMETER=VALUE:property-value`:

```
BEGIN:VCALENDAR
VERSION:2.0
PRODID:-//Example Corp//Calendar App//EN
CALSCALE:GREGORIAN
METHOD:PUBLISH
BEGIN:VEVENT
DTSTART:20240315T143000Z
DTEND:20240315T153000Z
SUMMARY:Project Review
DESCRIPTION:Quarterly project status review with the engineering team.
LOCATION:Conference Room B
ORGANIZER;CN=Alice:mailto:alice@example.com
ATTENDEE;PARTSTAT=ACCEPTED;CN=Bob:mailto:bob@example.com
ATTENDEE;PARTSTAT=NEEDS-ACTION;CN=Carol:mailto:carol@example.com
STATUS:CONFIRMED
UID:event-20240315-review@example.com
DTSTAMP:20240301T120000Z
CREATED:20240301T120000Z
LAST-MODIFIED:20240310T090000Z
SEQUENCE:0
TRANSP:OPAQUE
BEGIN:VALARM
ACTION:DISPLAY
TRIGGER:-PT15M
DESCRIPTION:Meeting in 15 minutes
END:VALARM
END:VEVENT
END:VCALENDAR
```

Components

Component	Description
VCALENDAR	Top-level wrapper (required) -- contains metadata and components
VEVENT	A calendar event (meeting, appointment, all-day event)
VTODO	A task or to-do item with optional due date and completion status
VJOURNAL	A journal entry or note associated with a date
VFREEBUSY	Free/busy time information for scheduling
VALARM	A reminder/alarm attached to a VEVENT or VTODO
VTIMEZONE	Time zone definition with daylight/standard rules

Calendar-Level Properties

Property	Required	Description
VERSION	Yes	iCalendar version (always `2.0`)
PRODID	Yes	Product identifier of the generating application
CALSCALE	No	Calendar scale (`GREGORIAN` is default and only defined value)

METHOD	No	iTIP method when used for scheduling (REQUEST, REPLY, CANCEL, etc.)

VEVENT Key Properties

Property	Required	Description
UID	Yes	Globally unique identifier for this event
DTSTAMP	Yes	Timestamp when the iCalendar object was created
DTSTART	Yes*	Start date/time of the event
DTEND	No	End date/time (mutually exclusive with DURATION)
DURATION	No	Duration of the event (e.g., `PT1H30M` = 1 hour 30 min)
SUMMARY	No	Short title / subject of the event
DESCRIPTION	No	Detailed description (plain text)
LOCATION	No	Location of the event (free text)
GEO	No	Geographic coordinates (`lat;lon`)
URL	No	Associated URL
ORGANIZER	No	Calendar address of the organizer (`mailto:...`)
ATTENDEE	No	Calendar address of an attendee (repeatable)
STATUS	No	Event status: `TENTATIVE`, `CONFIRMED`, or `CANCELLED`
CATEGORIES	No	Comma-separated category tags
PRIORITY	No	Priority (0 = undefined, 1 = highest, 9 = lowest)
SEQUENCE	No	Revision sequence number (incremented on updates)
RRULE	No	Recurrence rule
EXDATE	No	Exception dates excluded from recurrence
RDATE	No	Additional dates added to recurrence
TRANSP	No	Time transparency: `OPAQUE` (blocks time) or `TRANSPARENT` (does not)
CREATED	No	Date the event was first created
LAST-MODIFIED	No	Date the event was last modified
CLASS	No	Access classification: `PUBLIC`, `PRIVATE`, `CONFIDENTIAL`

Date-Time Formats

iCalendar supports several date-time representations:

Format	Example	Description
DATE	`20240315`	Date only (all-day event)
DATE-TIME (local)	`20240315T143000`	Local time (no time zone -- "floating")
DATE-TIME (UTC)	`20240315T143000Z`	UTC time (Z suffix)
DATE-TIME (TZID)	`DTSTART;TZID=America/New_York:20240315T103000`	Time in a specific zone
DURATION	`PT1H30M`	Period: 1 hour 30 minutes
DURATION	`-PT15M`	Negative period: 15 minutes before
PERIOD	`20240315T143000Z/PT1H`	Start time + duration

Recurrence Rules (RRULE)

The RRULE property defines repeating events with a powerful rule language:

```
RRULE:FREQ=WEEKLY;BYDAY=MO,WE,FR;UNTIL=20241231T235959Z
```

Parameter	Values	Description
FREQ	SECONDLY, MINUTELY, HOURLY, DAILY, WEEKLY, MONTHLY, YEARLY	Recurrence frequency (required)
INTERVAL	Integer	How often the rule repeats (default 1)
COUNT	Integer	Number of occurrences
UNTIL	Date or date-time	Recurrence end date (mutually exclusive with COUNT)
BYDAY	MO, TU, WE, TH, FR, SA, SU	Day(s) of the week (prefix with +1, -1 etc. for nth)
BYMONTH	1-12	Month(s) of the year
BYMONTHDAY	1-31, -1 to -31	Day(s) of the month
BYHOUR	0-23	Hour(s) of the day
BYMINUTE	0-59	Minute(s) of the hour
BYSETPOS	Integer	Position within the set of instances
WKST	MO-SU	Week start day (default MO)

Recurrence Examples

Rule	Meaning

`FREQ=DAILY;COUNT=10`	Every day for 10 days
`FREQ=WEEKLY;BYDAY=TU,TH`	Every Tuesday and Thursday
`FREQ=MONTHLY;BYDAY=2FR`	Second Friday of every month
`FREQ=MONTHLY;BYMONTHDAY=-1`	Last day of every month
`FREQ=YEARLY;BYMONTH=3;BYMONTHDAY=15`	Every March 15
`FREQ=WEEKLY;INTERVAL=2;BYDAY=MO`	Every other Monday

ATTENDEE Parameters

Parameter	Values	Description
PARTSTAT	NEEDS-ACTION, ACCEPTED, DECLINED, TENTATIVE, DELEGATED	Participation status
ROLE	CHAIR, REQ-PARTICIPANT, OPT-PARTICIPANT, NON-PARTICIPANT	Attendee role
RSVP	TRUE / FALSE	Whether a reply is requested
CN	Display name	Common name (display name)
CUTYPE	INDIVIDUAL, GROUP, ROOM, RESOURCE, UNKNOWN	Calendar user type
DELEGATED-TO	mailto: URI	Delegated to another attendee
DELEGATED-FROM	mailto: URI	Delegated from another attendee
SENT-BY	mailto: URI	Acting on behalf of

VALARM (Reminders)

Alarms are nested inside VEVENT or VTODO components:

```
BEGIN:VALARM
ACTION:DISPLAY
TRIGGER:-PT15M
DESCRIPTION:Meeting starts in 15 minutes
END:VALARM
```

Property	Description
ACTION	Alarm type: `DISPLAY` (pop-up), `AUDIO` (sound), `EMAIL` (send email)
TRIGGER	When to fire: `-PT15M` (15 min before), `-PT1H` (1 hour before), `-P1D` (1 day before)
REPEAT	Number of additional repetitions after initial trigger
DURATION	Interval between repetitions
DESCRIPTION	Alert text (for DISPLAY)

SUMMARY	Email subject (for EMAIL action)
ATTENDEE	Email recipient (for EMAIL action)

iTIP -- Scheduling Protocol (RFC 5546)

iTIP defines how iCalendar objects are used for scheduling between organizers and attendees. The METHOD property specifies the intent:

[Diagram: see online version]

iTIP Methods

Method	Direction	Description
PUBLISH	Organizer to anyone	Publish an event (no scheduling, information only)
REQUEST	Organizer to attendees	Invite attendees to an event or request update
REPLY	Attendee to organizer	Accept, decline, or tentatively accept
CANCEL	Organizer to attendees	Cancel a previously sent invitation
ADD	Organizer to attendees	Add instances to a recurring event
REFRESH	Attendee to organizer	Request an updated copy of the event
COUNTER	Attendee to organizer	Propose a different time or details
DECLINECOUNTER	Organizer to attendee	Decline a counter-proposal

iMIP -- Email Transport (RFC 6047)

iMIP defines how to carry iTIP messages over email. The iCalendar object is sent as a MIME attachment with `Content-Type: text/calendar; method=REQUEST`. This is how meeting invitations work in every email client:

MIME Header	Value
Content-Type	`text/calendar; method=REQUEST; charset=UTF-8`
Content-Disposition	`attachment; filename="invite.ics"`
Content-Transfer-Encoding	`base64` or `7bit`

VTODO (Tasks)

Property	Description
SUMMARY	Task title
DUE	Due date/time

DTSTART	Start date/time
COMPLETED	Date/time the task was completed
PERCENT-COMPLETE	0-100 completion percentage
STATUS	NEEDS-ACTION, IN-PROCESS, COMPLETED, CANCELLED
PRIORITY	1 (highest) to 9 (lowest), 0 = undefined

Line Folding

Identical to vCard: lines longer than 75 octets are folded by inserting a CRLF followed by a single space or tab. Parsers reassemble by removing CRLF + leading whitespace:

```
DESCRIPTION:This is a long description that needs to be folded because
  it exceeds the 75-octet line length limit specified by the RFC.
```

jCal (JSON Representation)

jCal (RFC 7265) maps iCalendar to JSON, mirroring jCard's approach:

```
["vcalendar", [
  ["version", {}, "text", "2.0"],
  ["prodid", {}, "text", "-//Example//App//EN"]
], [
  ["vevent", [
    ["dtstart", {}, "date-time", "2024-03-15T14:30:00Z"],
    ["dtend", {}, "date-time", "2024-03-15T15:30:00Z"],
    ["summary", {}, "text", "Project Review"],
    ["uid", {}, "text", "event-20240315@example.com"]
  ], []]
]]
```

File Format

Property	Value
MIME type	`text/calendar`
File extension	`.ics`
Character encoding	UTF-8
Line ending	CRLF (`\r\n`)
Line length limit	75 octets (fold longer lines)
Multiple events	Multiple VEVENTs in one VCALENDAR

Standards

Document	Title
RFC 5545	Internet Calendaring and Scheduling Core Object Specification (iCalendar)
RFC 5546	iCalendar Transport-Independent Interoperability Protocol (iTIP)
RFC 6047	iCalendar Message-Based Interoperability Protocol (iMIP)
RFC 7265	jCal: The JSON Format for iCalendar
RFC 7986	New Properties for iCalendar (NAME, COLOR, IMAGE, etc.)
RFC 7529	Non-Gregorian Recurrence Rules in iCalendar
RFC 9073	Event Publishing Extensions to iCalendar

See Also

- CalDAV -- sync protocol that uses iCalendar as its data format
- vCard -- sister format for contact data
- SMTP -- carries iCalendar meeting invitations via iMIP
- HTTP -- jCal is used in REST APIs
- JMAP -- modern API protocol with calendar extensions

MCAP -- Multimodal Container for Autonomous Platforms

Standard: MCAP Specification (mcap.dev) | **Category:** Multimodal Log Data Container Format

MCAP (pronounced "em-cap") is an open-source container file format for storing timestamped, multimodal log data. It is designed for robotics, autonomous vehicles, and any system that produces multiple concurrent data streams -- camera images, LiDAR point clouds, IMU readings, GPS, CAN bus, control commands, and diagnostics -- all in a single file with fast random access by time. MCAP is the default recording format for Foxglove and is widely used with ROS 2, replacing the older ROS 1 bag format.

File Structure

[Diagram: see online version]

Record Format

Every record in an MCAP file uses a uniform TLV (Type-Length-Value) structure:

[Diagram: see online version]

Field	Size	Description

Opcode	1 byte	Record type identifier
Record Length	8 bytes (uint64 LE)	Length of the data that follows
Record Data	Variable	Record-specific content

Record Types (Opcodes)

Opcode	Name	Description
0x01	Header	File metadata (profile, library)
0x02	Footer	Pointers to summary section
0x03	Schema	Message type definition (name + encoding + data)
0x04	Channel	Named data stream (topic + schema + metadata)
0x05	Message	Single timestamped data record
0x06	Chunk	Compressed block of messages
0x07	MessageIndex	Time-based index for messages within a chunk
0x08	ChunkIndex	Summary of each chunk (offset, time range, size)
0x09	Attachment	Arbitrary file attachment (calibration, config, etc.)
0x0A	AttachmentIndex	Index of attachments
0x0B	Statistics	File-level statistics (message counts, time range)
0x0C	Metadata	Arbitrary key-value metadata
0x0D	MetadataIndex	Index of metadata records
0x0E	SummaryOffset	Offset to groups of summary records
0x0F	DataEnd	Marks end of data section (CRC-32 of data)

Key Records

Header

Field	Type	Description
profile	string	Content profile (e.g., `ros2`, `foxglove`, empty for generic)
library	string	Writer library and version

Schema

Field	Type	Description
id	uint16	Unique schema ID within the file
name	string	Schema name (e.g., `sensor_msgs/Image`, `foxglove.PointCloud`)
encoding	string	Schema encoding (`ros2msg`, `protobuf`, `jsonschema`, `flatbuffer`, `omgidl`)
data	bytes	The schema definition itself

Channel

Field	Type	Description
id	uint16	Unique channel ID
schema_id	uint16	References a Schema record
topic	string	Channel name (e.g., `/camera/image_raw`, `/imu/data`)
message_encoding	string	How messages are serialized (`cdr`, `protobuf`, `json`, `ros1msg`, `cbor`)
metadata	map	Key-value metadata (e.g., `{"offered_qos_profiles": "..."}`)

Message

[Diagram: see online version]

Field	Type	Description
channel_id	uint16	Which channel this message belongs to
sequence	uint32	Publisher's sequence number
log_time	uint64	When the message was recorded (nanoseconds since epoch)
publish_time	uint64	When the message was published (nanoseconds since epoch)
data	bytes	Serialized message payload

Chunk (Compression)

Messages are grouped into chunks for compression:

Field	Type	Description

message_start_time	uint64	Earliest message timestamp in this chunk
message_end_time	uint64	Latest message timestamp in this chunk
uncompressed_size	uint64	Size before compression
uncompressed_crc	uint32	CRC-32 of uncompressed data
compression	string	`""` (none), `lz4`, `zstd`
records	bytes	Compressed block of Message records

Random Access

MCAP is designed for fast time-based seeking -- critical for replaying robot logs:

[Diagram: see online version]

The Summary Section at the end of the file contains:

- **ChunkIndex** records -- which time ranges each chunk covers
- **Statistics** -- total message count, channel counts, time bounds
- **Attachment/Metadata indexes**

This means a reader can seek to any timestamp in O(log N) time without scanning the entire file.

MCAP vs ROS 1 Bag vs Other Formats

Feature	MCAP	ROS 1 Bag	SQLite (ROS 2 default)	HDF5
Random access	Yes (chunk index)	Limited (connections + index)	Yes (SQL queries)	Yes (chunked datasets)
Compression	Per-chunk (LZ4, Zstd)	Per-chunk (LZ4, BZ2)	None (or whole-file)	Per-chunk (gzip, LZF)
Multi-encoding	Yes (CDR, Protobuf, JSON, FlatBuffers)	ROS 1 only	ROS 2 CDR only	Any (opaque datasets)
Streaming write	Yes (append-only)	Yes	Yes	Limited
File integrity	CRC-32 per chunk + data end	None	SQLite WAL	Optional checksums
Schema storage	Embedded in file	Embedded (connection records)	Not in file (type hash)	Not standardized
Recovery from corruption	Chunk-level (skip bad chunks)	Difficult	SQLite recovery	Difficult
Typical use	ROS 2, Foxglove, autonomous vehicles	ROS 1	ROS 2 (Humble default)	Scientific computing
File extension	`.mcap`	`.bag`	`.db3`	`.h5`, `.hdf5`

Ecosystem

Tool	Description
Foxglove	Web-based visualization and playback of MCAP files
ros2 bag	`ros2 bag record/play` supports MCAP storage plugin
mcap CLI	Command-line tool for inspecting, converting, and merging MCAP files
mcap libraries	Python, C++, Go, Rust, TypeScript, Swift readers/writers
MCAP -> Foxglove Studio	Drag-and-drop visualization of robot logs

ROS 2 Integration

```
# Record to MCAP format
ros2 bag record -s mcap /camera/image_raw /imu/data /tf

# Play back
ros2 bag play recording.mcap

# Inspect
mcap info recording.mcap
```

Message Encoding Support

Encoding	Schema Encoding	Description
`cdr`	`ros2msg` or `omgidl`	ROS 2 native (CDR serialization)
`ros1msg`	`ros1msg`	ROS 1 message format
`protobuf`	`protobuf`	Protocol Buffers
`json`	`jsonschema`	JSON with JSON Schema
`flatbuffer`	`flatbuffer`	FlatBuffers
`cbor`	--	Concise Binary Object Representation

The self-describing nature (schema + encoding embedded in the file) means MCAP files can be read without access to the original message definitions.

Standards

Resource	Description
MCAP Specification	Format specification
MCAP GitHub	Reference implementations (multi-language)
Foxglove	Primary visualization tool for MCAP

See Also

- DDS / ROS 2 -- the pub/sub middleware MCAP records
- ROS 1 (TCPROS) -- MCAP replaces ROS 1's bag format
- Parquet -- columnar data format (tabular, not timestamped multimodal)
- HDF5 -- hierarchical scientific data format

ONNX -- Open Neural Network Exchange

Standard: ONNX Specification (onnx.ai) | **Category:** ML Model Interchange Format

ONNX is an open format for representing machine learning models, enabling interoperability between training frameworks (PyTorch, TensorFlow, JAX) and inference engines (ONNX Runtime, TensorRT, OpenVINO, CoreML). An ONNX model is a directed acyclic graph of operators (Add, MatMul, Conv, Relu, etc.) with typed inputs, outputs, and weights. ONNX defines both the graph format (Protocol Buffers) and a standard set of operators (opset), ensuring models are portable.

File Structure

ONNX models are serialized as Protocol Buffers (`.onnx` files):

[Diagram: see online version]

Key Components

Component	Protobuf Type	Description
ModelProto	Top-level	Contains metadata, opset, and the graph
GraphProto	Model.graph	The computation graph (nodes, inputs, outputs, initializers)
NodeProto	Graph.node[]	Single operation (op_type, inputs, outputs, attributes)
TensorProto	Graph.initializer[]	Weight tensor (data_type, dims, raw_data)
ValueInfoProto	Graph.input/output	Typed tensor description (name, elem_type, shape)
AttributeProto	Node.attribute[]	Operator parameters (kernel_size, padding, etc.)

Data Types

Type	Enum	Description

FLOAT	1	32-bit floating point
UINT8	2	8-bit unsigned integer
INT8	3	8-bit signed integer
UINT16	4	16-bit unsigned
INT16	5	16-bit signed
INT32	6	32-bit signed
INT64	7	64-bit signed
STRING	8	String
BOOL	9	Boolean
FLOAT16	10	16-bit floating point
DOUBLE	11	64-bit floating point
BFLOAT16	16	Brain floating point (16-bit, 8-bit exponent)
FLOAT8E4M3FN	17	8-bit float (FP8, for transformer training)

Common Operators (Opset 20)

Category	Operators
Math	Add, Sub, Mul, Div, Pow, Sqrt, Exp, Log, MatMul, Gemm
Activation	Relu, Sigmoid, Tanh, Softmax, LeakyRelu, Gelu
Convolution	Conv, ConvTranspose, MaxPool, AveragePool, GlobalAveragePool
Normalization	BatchNormalization, LayerNormalization, InstanceNormalization
Attention	(composed from MatMul, Softmax, etc. -- or custom ops)
Recurrent	LSTM, GRU, RNN
Tensor	Reshape, Transpose, Concat, Split, Squeeze, Unsqueeze, Gather, Scatter
Reduction	ReduceSum, ReduceMean, ReduceMax, ReduceMin
Comparison	Equal, Greater, Less, Where
Quantization	QuantizeLinear, DequantizeLinear

Model Example (Simple MLP)

```
Input: X [batch, 784]
Node: MatMul(X, W1) -&gt; hidden        # W1 [784, 256]
Node: Add(hidden, B1) -&gt; hidden_b     # B1 [256]
Node: Relu(hidden_b) -&gt; activated
Node: MatMul(activated, W2) -&gt; logits # W2 [256, 10]
Node: Softmax(logits) -&gt; probs
Output: probs [batch, 10]
```

ONNX Workflow

[Diagram: see online version]

ONNX Runtime

Feature	Description
Execution Providers	CPU, CUDA, TensorRT, DirectML, OpenVINO, CoreML, NNAPI
Optimization	Graph optimization, operator fusion, constant folding
Quantization	INT8/UINT8 quantization for faster inference
Multi-threading	Intra-op and inter-op parallelism
Languages	C, C++, Python, Java, C#, JavaScript, Rust

ONNX vs Other Model Formats

Feature	ONNX	PyTorch (.pt)	TF SavedModel	GGUF	Safetensors
Cross-framework	Yes	PyTorch only	TF only	llama.cpp	Any (weights only)
Computation graph	Yes	Optional	Yes	No (weights + metadata)	No (weights only)
Inference engines	Many	TorchScript/libtorch	TF Serving	llama.cpp, ollama	Any (HF, vLLM)
Quantization	Built-in ops	Separate tool	TF Lite	Built-in types	No (separate)
Optimization	Graph-level (ORT)	torch.compile	XLA	At load time	N/A
Opset versioning	Yes (backward compat)	No formal versioning	SavedModel versions	Format version	Format version

Standards

Document	Title
ONNX IR Spec	ONNX Intermediate Representation
ONNX Operators	Operator definitions and schemas

ONNX GitHub	Specification and reference implementation
ONNX Runtime	Microsoft's high-performance inference engine

See Also

- GGUF -- LLM-specific format for local inference
- Safetensors -- secure weight storage format
- Parquet -- training data storage
- gRPC -- ML serving transport (Triton, TF Serving)

Apache Parquet

Standard: Apache Parquet Format (parquet.apache.org) | **Category:** Columnar Data Storage Format

Parquet is a columnar storage format designed for efficient analytics on large datasets. Unlike row-oriented formats (CSV, JSON), Parquet stores data column by column, enabling efficient compression (similar values together), column pruning (read only needed columns), and predicate pushdown (skip irrelevant row groups). It is the de facto standard for data lakes (S3, GCS, ADLS), ML training datasets, and analytics engines (Spark, DuckDB, Polars, BigQuery).

File Structure

[Diagram: see online version]

Component	Description
Magic bytes	PAR1 at start and end of file
Row Group	Horizontal partition (~128 MB default); unit of parallel reading
Column Chunk	All data for one column within a row group
Data Page	Compressed unit within a column chunk (~1 MB)
Dictionary Page	Optional -- stores unique values for dictionary encoding
Footer	Schema, row group metadata, column statistics (min/max/null count)

Key Properties

Property	Value
Format	Columnar (column-major storage)
Encoding	Multiple per column (dictionary, RLE, delta, etc.)

Compression	Snappy (default), ZSTD, Gzip, LZ4, Brotli, or none
Schema	Self-describing (schema in footer)
Types	Boolean, int32, int64, float, double, byte_array, fixed_len_byte_array
Logical types	String, decimal, date, timestamp, UUID, JSON, BSON, list, map, struct
Nested data	Full support (Dremel encoding -- definition/repetition levels)
Max file size	No hard limit (row groups are independently readable)

Encodings

Encoding	Best For	How It Works
PLAIN	Any	Raw values, no compression
DICTIONARY (PLAIN_DICTIONARY)	Low-cardinality columns	Map values to integer IDs
RLE (Run-Length Encoding)	Repeated values, booleans	Count consecutive identical values
DELTA_BINARY_PACKED	Sorted integers, timestamps	Store differences between consecutive values
DELTA_LENGTH_BYTE_ARRAY	Variable-length strings	Delta-encode string lengths
DELTA_BYTE_ARRAY	Strings with common prefixes	Delta-encode shared prefixes
BYTE_STREAM_SPLIT	Floating point	Split bytes across values (better compression)

Why Columnar Matters for ML

```
Query: SELECT age, score FROM users WHERE age &gt; 25

Row-oriented (CSV): Must read ALL columns for every row
  [name, age, score, email, address, ...]  &lt;- reads everything

Columnar (Parquet): Reads only age + score columns, skips row groups where max(age) &lt;= 25
  [age column]   &lt;- reads this
  [score column] &lt;- reads this
  [name, email, address, ...] &lt;- SKIPPED
```

Footer Metadata

The footer contains schema and statistics enabling query engines to skip data:

Metadata	Description
Schema	Column names, types, nesting
Row group metadata	Number of rows, total byte size

Column chunk metadata	File offset, compressed/uncompressed size, encoding, statistics
Column statistics	Min value, max value, null count, distinct count
Key-value metadata	User-defined (pandas metadata, Arrow schema, etc.)

Predicate Pushdown Example

```
File footer says Row Group 3 has: age min=18, max=22
Query filter: age &gt; 25
-&gt; Skip Row Group 3 entirely (never decompressed or read)
```

Parquet vs Other Formats

Feature	Parquet	CSV	JSON	ORC	Avro
Layout	Columnar	Row	Row	Columnar	Row
Schema	Self-describing	None	Implicit	Self-describing	Self-describing
Compression	Excellent (columnar)	Poor	Poor	Excellent	Good
Column pruning	Yes	No	No	Yes	No
Nested data	Yes (Dremel)	No	Yes	Yes (limited)	Yes
Streaming write	No (needs footer)	Yes	Yes	No	Yes
Ecosystem	Universal	Universal	Universal	Hive/Hadoop	Kafka/Hadoop
ML training	Excellent	Slow	Slow	Good	Good

Common Tools

Tool	Usage
Apache Arrow	In-memory columnar + Parquet reader/writer
pandas	`pd.read_parquet()` / `df.to_parquet()`
Polars	Native Parquet support (faster than pandas)
DuckDB	SQL queries directly on Parquet files
Apache Spark	Distributed Parquet processing
PyArrow	Python Parquet library (Arrow-based)
Hugging Face Datasets	ML datasets stored as Parquet

Standards

Document	Title
Apache Parquet Format	Format specification
Parquet Thrift Definition	Thrift schema for metadata
Apache Arrow	In-memory format + Parquet integration

See Also

- ONNX -- model interchange format
- HDF5 -- hierarchical data format for scientific computing
- TFRecord -- TensorFlow's training data format

Protocol Buffers -- Protobuf

Standard: Protocol Buffers Language Guide | **Layer:** Data Format (Serialization) | **Wireshark filter:** `protobuf`

Protocol Buffers is Google's language-neutral, platform-neutral binary serialization format. This document covers the **wire format** -- the binary encoding that goes on the network or into files -- not the `.proto` IDL used to define messages. A protobuf message is simply a concatenation of key-value pairs where each pair is a field tag (field number + wire type) followed by the encoded value. The format is self-delimiting, compact (typically 3-10x smaller than JSON), and designed for both forward and backward compatibility.

Message Structure

A serialized protobuf message is a sequence of tagged fields with no framing header or length prefix:

[Diagram: see online version]

Each field is encoded as:

```
[field_tag] [field_value]
```

The **field tag** is a single varint that encodes both the field number and the wire type:

```
tag = (field_number &lt;&lt; 3) | wire_type
```

Tag Encoding Example

For field number 2, wire type 0 (varint): `tag = (2 << 3) | 0 = 16 = 0x10`

For field number 1, wire type 2 (length-delimited): `tag = (1 << 3) | 2 = 10 = 0x0A`

Wire Types

Wire Type	Name	Size	Used For
0	Varint	Variable (1-10 bytes)	int32, int64, uint32, uint64, sint32, sint64, bool, enum
1	64-bit fixed	8 bytes	fixed64, sfixed64, double
2	Length-delimited	varint length + N bytes	string, bytes, embedded messages, packed repeated fields
3	Start group	N/A	Group start marker (deprecated)
4	End group	N/A	Group end marker (deprecated)
5	32-bit fixed	4 bytes	fixed32, sfixed32, float

Varint Encoding

Varints use Base-128 encoding with little-endian byte order. The most significant bit (MSB) of each byte is a continuation bit -- 1 means more bytes follow, 0 means this is the last byte.

[Diagram: see online version]

Varint Example: Encoding 300

```
300 = 0b100101100

Split into 7-bit groups (little-endian):
  Group 0: 0101100 (low 7 bits)
  Group 1: 0000010 (next 7 bits)

Add continuation bits:
  Byte 0: 1_0101100 = 0xAC (MSB=1, more bytes follow)
  Byte 1: 0_0000010 = 0x02 (MSB=0, last byte)

Wire bytes: 0xAC 0x02
```

Common Varint Values

Value	Encoded Bytes	Size
0	`0x00`	1 byte
1	`0x01`	1 byte
127	`0x7F`	1 byte
128	`0x80 0x01`	2 bytes
300	`0xAC 0x02`	2 bytes
16383	`0xFF 0x7F`	2 bytes

16384	`0x80 0x80 0x01`	3 bytes

Field numbers 1-15 encode in a single-byte tag (1 byte for tag + wire type), so assign frequently-used fields to numbers 1-15 for optimal size.

ZigZag Encoding

Standard varint encoding is inefficient for negative numbers because the sign bit sits in the most significant position, producing 10-byte encodings for small negative values. The `sint32` and `sint64` types use **ZigZag encoding** to map signed integers to unsigned integers so that small-magnitude values (positive or negative) use fewer bytes.

Signed Value	ZigZag Encoded
0	0
-1	1
1	2
-2	3
2	4
2147483647	4294967294
-2147483648	4294967295

Formula: `zigzag(n) = (n << 1) ^ (n >> 31)` for sint32, `(n << 1) ^ (n >> 63)` for sint64.

Length-Delimited Fields

Strings, bytes, embedded messages, and packed repeated fields all use wire type 2. The value is a varint-encoded length followed by that many bytes of content.

[Diagram: see online version]

String Example

Field 2, value `"testing"`:

```
Tag:    0x12  (field 2, wire type 2)
Length: 0x07  (7 bytes)
Data:   0x74 0x65 0x73 0x74 0x69 0x6E 0x67  ("testing" in UTF-8)
```

Embedded Messages

An embedded message is encoded as a length-delimited field whose payload is itself a complete serialized protobuf message.

[Diagram: see online version]

For example, given:

```
message Inner { int32 a = 1; }
message Outer { Inner inner = 3; }
```

Encoding `Outer { inner: Inner { a: 150 } }`:

```
0x1A       (field 3, wire type 2 = length-delimited)
0x03       (3 bytes follow)
  0x08     (field 1, wire type 0 = varint)
  0x96     (varint: 150, byte 1)
  0x01     (varint: 150, byte 2)
```

Packed Repeated Fields

In proto3, repeated fields of scalar numeric types are **packed** by default. Instead of repeating the tag for each element, all values are concatenated into a single length-delimited field.

[Diagram: see online version]

Packed Example

Field 4 (repeated int32), values `[3, 270, 86942]`:

```
0x22       (field 4, wire type 2)
0x06       (6 bytes of packed data)
  0x03             (varint: 3)
  0x8E 0x02        (varint: 270)
  0x9E 0xA7 0x05   (varint: 86942)
```

Without packing, this would require a separate tag before each value (3 extra bytes).

Encoding Flow

[Diagram: see online version]

[Diagram: see online version]

Field Ordering and Default Values

Behavior	Description
Field order	Fields can appear in any order on the wire; decoders must not assume ordering
Unknown fields	Preserved by default in proto3 (since 3.5); decoders keep unrecognized tags
Default values (proto3)	Zero-value fields (0, false, "", empty bytes) are **not serialized**
Default values (proto2)	Explicit defaults supported; unset fields distinguishable from default

Repeated fields	May appear multiple times; decoder concatenates values
Oneof fields	Only the last-written field in a oneof group is serialized
Map fields	Serialized as repeated messages with key (field 1) and value (field 2)
Field merging	Duplicate fields: last varint wins, embedded messages are merged, repeated fields concatenated

Proto2 vs Proto3

Feature	Proto2	Proto3
Required fields	Supported (`required`)	Removed (all fields optional)
Optional fields	Explicit (`optional`)	All fields optional by default
Default values	Custom defaults allowed	Always zero/empty
Field presence	Tracked for optional fields	Not tracked unless `optional` keyword used
Unknown fields	Preserved	Preserved (since 3.5; discarded in earlier versions)
Enums	Open (unknown values kept)	Open (unknown values kept in proto3, closed in proto2)
Maps	Supported	Supported
Packed repeated	Opt-in (`[packed=true]`)	Default for scalar numerics
Extensions	Supported	Removed (use `Any` type instead)
Groups	Supported (wire types 3/4)	Removed

Text Format and JSON Mapping

While the wire format is binary, protobuf also defines canonical text and JSON representations:

Format	Description	Use Case
Binary (wire format)	Compact, this document's focus	Network transport, storage
Text format	Human-readable, field names + values	Debugging, config files (`.textproto`)
JSON mapping	Standard JSON with field name conversion	REST gateways, browser clients
JSON field names	`camelCase` by default (from `snake_case` proto fields)	Interoperability with JSON APIs

JSON mapping: int64/uint64 encode as strings (JavaScript safety), bytes encode as Base64, enums encode as strings, `google.protobuf.Any` wraps typed JSON.

Size Comparison

Format	Payload Example (typical API response)	Relative Size
XML	Verbose tags, attributes, closing tags	10-20x
JSON	Field names repeated, text encoding	5-10x
JSON (minified)	Compact but still text-based	3-7x
Protobuf	**Binary, field numbers, varints**	**1x (baseline)**
MessagePack	Binary JSON, field names as strings	2-3x
CBOR	Binary, similar to MessagePack	2-3x

Protobuf vs Other Serialization Formats

Feature	Protobuf	JSON	MessagePack	CBOR	FlatBuffers	Cap'n Proto
Encoding	Binary (tags + values)	Text	Binary (schemaless)	Binary (schemaless)	Binary (zero-copy)	Binary (zero-copy)
Schema	Required (.proto)	None (optional JSON Schema)	None	None (optional CDDL)	Required (.fbs)	Required (.capnp)
Parsing	Deserialize to objects	Parse text	Deserialize to objects	Deserialize to objects	Zero-copy (no parsing)	Zero-copy (no parsing)
Code generation	Yes (protoc)	No	No	No	Yes (flatc)	Yes (capnpc)
Size	Compact	Large	Moderate	Moderate	Compact (+ alignment)	Moderate (+ alignment)
Human readable	No	Yes	No	No	No	No
Streaming	Delimited messages	NDJSON	Yes	Yes	No (needs full buffer)	Yes
Schema evolution	Excellent (field numbers)	N/A	N/A	N/A	Good (field IDs)	Good (field IDs)
Language support	12+ (official)	Universal	Many	Many	10+	7+
Random access	No (sequential)	No	No	No	Yes	Yes

Key Users

System	How Protobuf Is Used
gRPC	Default serialization for all RPC calls
Google internal (Stubby)	Predecessor to gRPC, all Google services
Kubernetes API	etcd stores resources as protobuf

OTLP (OpenTelemetry)	Traces, metrics, and logs encoded as protobuf
TensorFlow	TFRecord, SavedModel, graph definitions
Envoy Proxy	xDS configuration protocol
Android (AIDL)	Inter-process communication
Buf	Modern protobuf tooling (linting, breaking change detection)

Standards

Document	Title
Protocol Buffers Language Guide	Proto3 language specification
Protocol Buffers Encoding	Wire format encoding specification
Protocol Buffers GitHub	Reference implementation (C++, Java, Python, and more)
Buf	Modern protobuf toolchain (lint, format, breaking changes)
gRPC Protocol	gRPC over HTTP/2 (primary transport for protobuf)

See Also

- gRPC -- RPC framework built on protobuf serialization
- OTLP -- OpenTelemetry protocol uses protobuf encoding
- ONNX -- ML model format serialized as protobuf
- TFRecord -- TensorFlow training data stored as protobuf records
- MQTT -- IoT messaging (often carries protobuf payloads)

Safetensors

Standard: Safetensors (huggingface.co) | **Category:** ML Model Weight Storage Format

Safetensors is a secure, fast tensor serialization format developed by Hugging Face as a safe replacement for Python's pickle-based model files (.pt, .pkl). Unlike pickle (which can execute arbitrary code on load), safetensors stores only tensor data and metadata -- no executable code, making it immune to deserialization attacks. It is the default format for Hugging Face model weights and is supported by PyTorch, TensorFlow, JAX, and most ML frameworks.

File Structure

[Diagram: see online version]

Layout

[Diagram: see online version]

Component	Description
Header Size	8 bytes, uint64 little-endian -- size of the JSON header
Header	JSON object mapping tensor names to metadata (dtype, shape, offsets)
Tensor Data	Contiguous block of raw tensor bytes (no per-tensor headers)

Header Format (JSON)

```
{
  "__metadata__": {
    "format": "pt"
  },
  "model.embed_tokens.weight": {
    "dtype": "F16",
    "shape": [32000, 4096],
    "data_offsets": [0, 262144000]
  },
  "model.layers.0.self_attn.q_proj.weight": {
    "dtype": "F16",
    "shape": [4096, 4096],
    "data_offsets": [262144000, 295698432]
  }
}
```

Tensor Entry Fields

Field	Type	Description
`dtype`	string	Data type of the tensor
`shape`	int[]	Tensor dimensions
`data_offsets`	[int, int]	[start_byte, end_byte] in the data section

Supported Data Types

dtype	Description	Bytes
F64	64-bit float	8
F32	32-bit float	4
F16	16-bit float (IEEE 754)	2
BF16	Brain float 16	2
F8_E4M3	8-bit float (4-bit exponent, 3-bit mantissa)	1

F8_E5M2	8-bit float (5-bit exponent, 2-bit mantissa)	1
I64	64-bit signed integer	8
I32	32-bit signed integer	4
I16	16-bit signed integer	2
I8	8-bit signed integer	1
U8	8-bit unsigned integer	1
BOOL	Boolean	1

Metadata

The optional `metadata` key stores arbitrary key-value string pairs:

Key	Description
`format`	Source framework (`pt`, `tf`, `flax`, `np`)
Any custom key	User-defined string metadata

Design Principles

Principle	Implementation
Security	No executable code -- only JSON header + raw bytes
Speed	Memory-mappable -- `mmap()` the file, zero-copy tensor access
Simplicity	Single JSON header + contiguous data block
Validation	Header is validated before any data is read
Lazy loading	Individual tensors can be loaded by offset without reading the full file

Security Comparison

Format	Arbitrary Code Execution	Attack Vector
pickle (.pt, .pkl)	**YES** -- executes Python code on load	Malicious model files
safetensors	**NO** -- only data + JSON metadata	None
ONNX (.onnx)	No -- Protocol Buffers	None
GGUF (.gguf)	No -- binary data + metadata	None
NumPy (.npy)	Low risk -- but allow_pickle=True is dangerous	pickle in .npz

Loading Performance

Operation	pickle (.pt)	safetensors
Load mechanism	Deserialize Python objects	`mmap()` + offset lookup
Zero-copy	No (full deserialization)	Yes (tensors point into mmap)
Lazy loading	No (must load everything)	Yes (load individual tensors by name)
Loading 7B F16 model	~5-10 seconds	~0.5-1 second
Memory overhead	2x (load + copy)	1x (mmap shares with page cache)

Sharded Files

Large models are split across multiple safetensors files:

```
model-00001-of-00004.safetensors
model-00002-of-00004.safetensors
model-00003-of-00004.safetensors
model-00004-of-00004.safetensors
model.safetensors.index.json         &lt;- maps tensor names to files
```

Index File

```
{
  "metadata": { "total_size": 14483456000 },
  "weight_map": {
    "model.embed_tokens.weight": "model-00001-of-00004.safetensors",
    "model.layers.0.self_attn.q_proj.weight": "model-00001-of-00004.safetensors",
    "model.layers.31.mlp.down_proj.weight": "model-00004-of-00004.safetensors",
    "lm_head.weight": "model-00004-of-00004.safetensors"
  }
}
```

Language Support

Language	Library	Usage
Python	`safetensors`	`save_file()` / `load_file()`
Rust	`safetensors` (native)	Core implementation (Python bindings via PyO3)
JavaScript	`@huggingface/safetensors`	Browser/Node.js
C++	Via Rust FFI	Inference engines
PyTorch	`safetensors.torch`	`save_file()` / `load_file()` with torch tensors
TensorFlow	`safetensors.tensorflow`	TF tensor support
JAX/Flax	`safetensors.flax`	Flax model support

Standards

Resource	Description
Safetensors Docs	Official documentation
Safetensors GitHub	Source code and specification
Format Spec	Binary format specification

See Also

- GGUF -- LLM format with quantization and tokenizer
- ONNX -- model interchange with computation graph
- Parquet -- training data storage
- HDF5 -- hierarchical scientific data format

TFRecord

Standard: TensorFlow TFRecord (tensorflow.org) | **Category:** ML Training Data Storage Format

TFRecord is TensorFlow's native binary format for storing training data as a sequence of serialized Protocol Buffer records. It is designed for high-throughput sequential reading during model training -- data is stored as a stream of length-prefixed, CRC-checked binary records that can be read efficiently from disk, cloud storage (GCS), or over the network. TFRecord files are the standard input format for TensorFlow's `tf.data` pipeline and Google Cloud AI Platform.

File Structure

A TFRecord file is a sequence of records with no global header or index:

[Diagram: see online version]

Record Format

Each record contains:

[Diagram: see online version]

Field	Size	Description
Length	8 bytes	Length of the data field
Length CRC	4 bytes	CRC-32C of the length field (masked)

Data	Variable	Serialized `tf.train.Example` or `tf.train.SequenceExample`
Data CRC	4 bytes	CRC-32C of the data field (masked)

CRC masking: `masked_crc = ((crc >> 15) | (crc << 17)) + 0xa282ead8`

tf.train.Example (Protobuf)

Each record typically contains a `tf.train.Example` -- a map of feature names to values:

```
message Example {
  Features features = 1;
}

message Features {
  map&lt;string, Feature&gt; feature = 1;
}

message Feature {
  oneof kind {
    BytesList bytes_list = 1;
    FloatList float_list = 2;
    Int64List int64_list = 3;
  }
}
```

Feature Types

Type	Protobuf	Python Type	Use Case
BytesList	bytes_list	`bytes`, `string`	Images (JPEG/PNG bytes), text, raw data
FloatList	float_list	`float`	Features, embeddings, coordinates
Int64List	int64_list	`int`	Labels, indices, categorical features

Example: Image Classification

```
tf.train.Example(features=tf.train.Features(feature={
    'image': tf.train.Feature(bytes_list=tf.train.BytesList(value=[jpeg_bytes])),
    'label': tf.train.Feature(int64_list=tf.train.Int64List(value=[3])),
    'height': tf.train.Feature(int64_list=tf.train.Int64List(value=[224])),
    'width': tf.train.Feature(int64_list=tf.train.Int64List(value=[224])),
    'filename': tf.train.Feature(bytes_list=tf.train.BytesList(value=[b'cat_001.jpg'])),
}))
```

tf.train.SequenceExample

For sequential/temporal data (time series, video, NLP):

```
message SequenceExample {
  Features context = 1;              // Fixed-length context features
  FeatureLists feature_lists = 2;    // Variable-length sequences
}

message FeatureLists {
  map&lt;string, FeatureList&gt; feature_list = 1;
}

message FeatureList {
  repeated Feature feature = 1;      // One Feature per timestep
}
```

Example: Video Classification

```
tf.train.SequenceExample(
    context=tf.train.Features(feature={
        'video_id': bytes_feature(b'video_001'),
        'label': int64_feature(5),
    }),
    feature_lists=tf.train.FeatureLists(feature_list={
        'frames': tf.train.FeatureList(feature=[
            bytes_feature(frame1_jpeg),
            bytes_feature(frame2_jpeg),
            bytes_feature(frame3_jpeg),
        ]),
        'timestamps': tf.train.FeatureList(feature=[
            float_feature(0.0),
            float_feature(0.033),
            float_feature(0.066),
        ]),
    })
)
```

Sharding

Large datasets are split into multiple TFRecord files (shards) for parallel reading:

```
train-00000-of-01024.tfrecord
train-00001-of-01024.tfrecord
...
train-01023-of-01024.tfrecord
```

Benefits:

- Parallel reading across workers
- Shuffle at the file level + within-file shuffle
- Distribute across storage systems
- Typical shard size: 100-200 MB

Compression

Method	Extension	Trade-off
None	`.tfrecord`	Fastest I/O, largest files

GZIP	`.tfrecord.gz`	~3-5x smaller, slower read
ZLIB	`.tfrecord.zlib`	Similar to GZIP

For image datasets, data is usually pre-compressed (JPEG/PNG bytes inside the record), so file-level compression adds little benefit.

tf.data Pipeline

```
dataset = tf.data.TFRecordDataset(
    filenames=['train-00000.tfrecord', 'train-00001.tfrecord'],
    compression_type='GZIP',
    num_parallel_reads=8
)

def parse_fn(serialized):
    features = tf.io.parse_single_example(serialized, {
        'image': tf.io.FixedLenFeature([], tf.string),
        'label': tf.io.FixedLenFeature([], tf.int64),
    })
    image = tf.io.decode_jpeg(features['image'], channels=3)
    return image, features['label']

dataset = dataset.map(parse_fn, num_parallel_calls=tf.data.AUTOTUNE)
dataset = dataset.shuffle(10000).batch(32).prefetch(tf.data.AUTOTUNE)
```

TFRecord vs Other Training Data Formats

Feature	TFRecord	Parquet	HDF5	WebDataset (TAR)	CSV
Optimized for	Sequential training reads	Analytics + ML	Random access	PyTorch streaming	Simplicity
Random access	No (sequential only)	Yes (row groups)	Yes (chunked)	No (sequential)	No
Compression	Per-file (gzip)	Per-column	Per-chunk	Per-file	None
Schema	Protobuf (flexible)	Self-describing	Self-describing	File-based	None
Nested data	SequenceExample	Dremel encoding	Groups/datasets	Files in TAR	No
Framework	TensorFlow	Any	Any (Keras legacy)	PyTorch	Any
Sharding	Standard pattern	Row groups	Single file	Multiple TARs	Multiple files
Ecosystem	TF, Google Cloud	Universal	Scientific computing	PyTorch	Universal

Standards

Resource	Description

TFRecord Tutorial	Official TensorFlow guide
tf.train.Example	Protobuf message definition
tf.data	Data pipeline API

See Also

- Parquet -- columnar format (better for tabular ML data)
- HDF5 -- hierarchical format with random access
- ONNX -- model format (TFRecord is for training data)
- gRPC -- TF Serving uses gRPC for inference

vCard -- Electronic Business Card

Standard: RFC 6350 | **Category:** Data Format | **Wireshark filter:** N/A (data format, not wire protocol)

vCard is a text-based data format for representing contact information -- names, phone numbers, email addresses, physical addresses, photos, and more. It is the universal interchange format for contacts across platforms: phones share contacts as `.vcf` files, email clients attach vCards to signatures, QR codes encode them for business cards, and sync protocols like CardDAV use vCard as their native data format. vCard is to contacts what iCalendar is to calendar events.

Format

A vCard is plain text wrapped in `BEGIN:VCARD` / `END:VCARD` delimiters, with one property per line in the form `PROPERTY;PARAMETER=VALUE:property-value`:

```
BEGIN:VCARD
VERSION:4.0
FN:Grace Hopper
N:Hopper;Grace;Murray;Rear Admiral;
TEL;TYPE=work;VALUE=uri:tel:+1-202-555-0143
TEL;TYPE=cell;VALUE=uri:tel:+1-202-555-0187
EMAIL;TYPE=work:grace.hopper@navy.mil
ADR;TYPE=work:;;1200 Navy Pentagon;Washington;DC;20350;USA
ORG:United States Navy
TITLE:Computer Scientist
PHOTO;MEDIATYPE=image/jpeg:https://example.com/hopper.jpg
URL:https://en.wikipedia.org/wiki/Grace_Hopper
BDAY:19061209
NOTE:Pioneer of computer programming. Invented the first compiler.
REV:20240315T120000Z
UID:urn:uuid:f81d4fae-7dec-11d0-a765-00a0c91e6bf6
END:VCARD
```

Example Annotated

```
BEGIN:VCARD                          &lt;- Start delimiter
VERSION:4.0                          &lt;- vCard version (required, must be first after BEGIN)
FN:Grace Hopper                      &lt;- Formatted name (display name, required in v4.0)
N:Hopper;Grace;Murray;Rear Admiral;  &lt;- Structured name: last;first;middle;prefix;suffix
TEL;TYPE=work;VALUE=uri:tel:+1-...   &lt;- Telephone with type parameter
EMAIL;TYPE=work:grace.hopper@...     &lt;- Email with type parameter
ADR;TYPE=work:;;1200 Navy...         &lt;- Structured address: PO;ext;street;city;region;postal;country
ORG:United States Navy               &lt;- Organization name
TITLE:Computer Scientist             &lt;- Job title
PHOTO;MEDIATYPE=image/jpeg:https://  &lt;- Photo as URI (can also be base64-embedded)
URL:https://en.wikipedia.org/...     &lt;- Associated URL
BDAY:19061209                        &lt;- Birthday (YYYYMMDD)
NOTE:Pioneer of computer...          &lt;- Free-text notes
REV:20240315T120000Z                 &lt;- Last revision timestamp
UID:urn:uuid:f81d4fae-...            &lt;- Globally unique identifier
END:VCARD                            &lt;- End delimiter
```

Key Properties

Property	Required	Description
VERSION	Yes	vCard version (2.1, 3.0, or 4.0)
FN	Yes (v3+)	Formatted name -- the display name as a single string
N	Yes (v3), Optional (v4)	Structured name: last;first;middle;prefix;suffix
TEL	No	Telephone number (with TYPE=work, cell, home, fax, voice, etc.)
EMAIL	No	Email address
ADR	No	Structured address: PO box;extended;street;city;region;postal code;country
ORG	No	Organization name (can be multi-level: `ORG:Company;Division;Team`)
TITLE	No	Job title or position
PHOTO	No	Photo -- base64-encoded data or URI reference
URL	No	Associated web URL
NOTE	No	Free-text notes
BDAY	No	Birthday (date or date-time)
GEO	No	Geographic coordinates (v4: `geo:37.386,-122.083`; v3: `37.386;-122.083`)
TZ	No	Time zone
REV	No	Last revision timestamp
UID	No	Globally unique identifier for this contact

CATEGORIES	No	Comma-separated category tags
IMPP	No	Instant messaging handle (e.g., `xmpp:user@example.com`)
LOGO	No	Organization logo (base64 or URI)
KEY	No	Public encryption key (base64 or URI)
SOUND	No	Name pronunciation audio
ROLE	No	Role within organization
NICKNAME	No	Nickname(s)

v4.0 Additions

Property	Description
KIND	Contact type: `individual` (default), `group`, `org`, `location`
GENDER	Gender identity (e.g., `M`, `F`, `O`, `N`, `U`)
ANNIVERSARY	Wedding or other anniversary date
RELATED	Relationship to another contact (with TYPE=spouse, child, parent, etc.)
MEMBER	Group membership (used when KIND:group) -- references other vCard UIDs
LANG	Language preference(s)

Parameters

Parameters modify properties, appearing between the property name and the colon:

Parameter	Description	Example
TYPE	Categorizes the value (work, home, cell, fax, voice)	`TEL;TYPE=cell:+1-555-0123`
VALUE	Specifies the value data type (text, uri, date, etc.)	`TEL;VALUE=uri:tel:+1-555-0123`
PREF	Preference ranking (1 = highest)	`EMAIL;PREF=1:primary@example.com`
MEDIATYPE	MIME type for URI values	`PHOTO;MEDIATYPE=image/png:https://...`
LANGUAGE	Language of the property value	`NOTE;LANGUAGE=en:Some note`
ENCODING	Encoding method (b = base64 in v3.0)	`PHOTO;ENCODING=b;TYPE=JPEG:[base64 data]`
CHARSET	Character set (v2.1 only; v4.0 mandates UTF-8)	`N;CHARSET=UTF-8:...`

Structured Address Format (ADR)

The ADR property uses semicolons to separate seven ordered fields:

```
ADR;TYPE=work:PO Box;Extended Address;Street;City;Region;Postal Code;Country
```

Position	Field	Example
1	PO Box	`PO Box 42`
2	Extended address	`Suite 300`
3	Street address	`1600 Pennsylvania Ave`
4	City (locality)	`Washington`
5	Region (state/province)	`DC`
6	Postal code	`20500`
7	Country	`USA`

Empty fields are represented by consecutive semicolons: `ADR:;;123 Main St;Springfield;IL;62704;USA`

Line Folding

Lines longer than 75 octets must be folded by inserting a CRLF followed by a single space or tab character. The receiving parser reassembles the original line by stripping CRLF + whitespace sequences:

```
NOTE:This is a very long note that exceeds the 75-octet line length lim
 it and must be folded onto the next line with a leading space character
 to indicate continuation.
```

Version Differences

Feature	v2.1	v3.0 (RFC 2426)	v4.0 (RFC 6350)
Encoding	Quoted-Printable, Base64	B (base64)	UTF-8 only; binary via URI
Character set	CHARSET parameter	UTF-8 default	UTF-8 mandatory
PHOTO format	Inline base64 with TYPE	Inline base64 with ENCODING=b	URI preferred; data: URI for inline
FN property	Optional	Required	Required
N property	Required	Required	Optional (FN sufficient)
GEO format	`GEO:lat;lon`	`GEO:lat;lon`	`GEO:geo:lat,lon` (RFC 5870 URI)
TEL format	Plain number	Plain number	`VALUE=uri:tel:+1-555-0123` preferred
KIND property	N/A	N/A	Supported (individual, group, org, location)

GENDER	N/A	N/A	Supported
IMPP	N/A	N/A	Supported
RELATED	N/A	N/A	Supported
MIME type	text/x-vcard	text/vcard	text/vcard
Line ending	CRLF	CRLF	CRLF

File Format

Property	Value
MIME type	`text/vcard` (v3+), `text/x-vcard` (v2.1)
File extension	`.vcf` or `.vcard`
Character encoding	UTF-8 (mandatory in v4.0)
Line ending	CRLF (`\r\n`)
Line length limit	75 octets (fold longer lines)
Multiple contacts	Multiple vCards concatenated in one `.vcf` file

jCard (JSON Representation)

jCard (RFC 7095) maps vCard to a JSON array format, used by RDAP and some REST APIs:

```
["vcard", [
  ["version", {}, "text", "4.0"],
  ["fn", {}, "text", "Grace Hopper"],
  ["n", {}, "text", ["Hopper", "Grace", "Murray", "Rear Admiral", ""]],
  ["tel", {"type": "work"}, "uri", "tel:+1-202-555-0143"],
  ["email", {"type": "work"}, "text", "grace.hopper@navy.mil"],
  ["org", {}, "text", "United States Navy"]
]]
```

Each property is a JSON array: `[name, parameters-object, value-type, value]`.

Common Uses

Use Case	How vCard Is Used
CardDAV sync	Native data format for contact synchronization
Email signatures	`.vcf` attachment for easy contact import
QR codes	vCard or MECARD encoding on printed business cards
NFC tags	vCard payload for tap-to-share contacts
Phone contacts	Export/import format for contact lists
CRM systems	Contact import/export interchange format

LDAP directories	vCard mapping for directory entries

Standards

Document	Title
RFC 6350	vCard Format Specification (v4.0)
RFC 2426	vCard MIME Directory Profile (v3.0)
RFC 2425	MIME Content-Type for Directory Information
RFC 7095	jCard: The JSON Format for vCard
RFC 6868	Parameter Value Encoding in iCalendar and vCard
RFC 6474	vCard Format Extensions: Place of Birth, Death

See Also

- CardDAV -- sync protocol that uses vCard as its data format
- iCalendar -- sister format for calendar/event data
- LDAP -- directory protocol often mapped to/from vCard
- SMTP -- vCards are commonly sent as email attachments
- HTTP -- jCard is used in REST APIs

Database Wire Protocols

Wire protocols used by databases for client-server communication.

6 protocols in this section

Memcached Protocol

Standard: Memcached Protocol Documentation | **Layer:** Application (Layer 7) | **Wireshark filter:** `memcache`

Memcached is a high-performance, distributed in-memory key-value cache used to reduce database load and speed up dynamic web applications. Originally written by Brad Fitzpatrick for LiveJournal in 2003, it is deployed at massive scale across the internet. Memcached supports three wire protocols: the original text protocol (human-readable, line-based), a binary protocol (fixed-header, more efficient), and the newer meta commands protocol (text-based but with richer semantics). All protocols run over TCP on port 11211 by default. Keys are limited to 250 bytes and values to 1 MB.

Text Protocol

The text protocol is line-oriented. Commands and responses are terminated by `\r\n`. Data blocks follow the command line and are terminated by `\r\n`.

Storage Commands

Command	Format	Description
`set`	`set <key> <flags> <exptime> <bytes> [noreply]\r\n<data>\r\n`	Store unconditionally
`add`	`add <key> <flags> <exptime> <bytes> [noreply]\r\n<data>\r\n`	Store only if key does not exist
`replace`	`replace <key> <flags> <exptime> <bytes> [noreply]\r\n<data>\r\n`	Store only if key already exists
`append`	`append <key> <flags> <exptime> <bytes> [noreply]\r\n<data>\r\n`	Append data to existing value
`prepend`	`prepend <key> <flags> <exptime> <bytes> [noreply]\r\n<data>\r\n`	Prepend data to existing value
`cas`	`cas <key> <flags> <exptime> <bytes> <cas_unique> [noreply]\r\n<data>\r\n`	Compare-and-swap (optimistic locking)

Retrieval Commands

Command	Format	Description
`get`	`get <key> [<key> ...]\r\n`	Retrieve one or more items
`gets`	`gets <key> [<key> ...]\r\n`	Retrieve with CAS token

gat	gat <exptime> <key> [<key> ...]\r\n	Get and touch (reset expiration)

Other Commands

Command	Format	Description
delete	delete <key> [noreply]\r\n	Delete an item
incr	incr <key> <value> [noreply]\r\n	Increment a numeric value
decr	decr <key> <value> [noreply]\r\n	Decrement a numeric value
touch	touch <key> <exptime> [noreply]\r\n	Update expiration without fetching
stats	stats [<args>]\r\n	Return server statistics
flush_all	flush_all [<delay>] [noreply]\r\n	Invalidate all items
version	version\r\n	Return server version string
quit	quit\r\n	Close connection

Text Protocol Responses

Response	Description
STORED\r\n	Storage command succeeded
NOT_STORED\r\n	Condition not met (add/replace/cas)
EXISTS\r\n	CAS value has been modified since last fetch
NOT_FOUND\r\n	Key does not exist (for cas, delete, incr, decr)
VALUE <key> <flags> <bytes> [<cas>]\r\n<data>\r\n	Retrieved item
END\r\n	End of retrieval response
DELETED\r\n	Delete succeeded
ERROR\r\n	Unknown command
CLIENT_ERROR <msg>\r\n	Client sent invalid input
SERVER_ERROR <msg>\r\n	Server-side error

Text Protocol Flow

[Diagram: see online version]

Binary Protocol

The binary protocol uses a fixed 24-byte header for both requests and responses, enabling faster parsing and reduced bandwidth:

Request Header

[Diagram: see online version]

Response Header

[Diagram: see online version]

Header Fields

Field	Size	Description
Magic	8 bits	`0x80` = request, `0x81` = response
Opcode	8 bits	Operation code (see opcodes table)
Key Length	16 bits	Length of the key in bytes
Extras Length	8 bits	Length of the extras (command-specific parameters)
Data Type	8 bits	Reserved (0x00 for raw bytes)
vbucket ID / Status	16 bits	Request: virtual bucket ID; Response: status code
Total Body Length	32 bits	Length of extras + key + value
Opaque	32 bits	Copied back in response -- client uses for request matching
CAS	64 bits	Compare-and-swap value (0 = not used)

Binary Opcodes

Opcode	Name	Description
0x00	Get	Retrieve a key
0x01	Set	Store a key unconditionally
0x02	Add	Store only if key does not exist
0x03	Replace	Store only if key exists
0x04	Delete	Remove a key
0x05	Increment	Increment a numeric counter
0x06	Decrement	Decrement a numeric counter
0x07	Quit	Close connection
0x08	Flush	Invalidate all items

0x09	GetQ	Quiet Get (no response on miss)
0x0A	Noop	No operation (used to end a GetQ pipeline)
0x0B	Version	Return server version
0x0E	Append	Append to existing value
0x0F	Prepend	Prepend to existing value
0x10	Stat	Return statistics
0x21	SASL List Mechs	List SASL authentication mechanisms
0x22	SASL Auth	Begin SASL authentication
0x23	SASL Step	Continue SASL authentication

Binary Status Codes

Code	Name	Description
0x0000	No error	Operation succeeded
0x0001	Key not found	Key does not exist
0x0002	Key exists	CAS conflict or key already present (for Add)
0x0003	Value too large	Value exceeds maximum size
0x0004	Invalid arguments	Malformed request
0x0005	Item not stored	Condition not met
0x0006	Non-numeric value	Incr/decr on non-numeric value
0x0020	Auth error	Authentication failed
0x0081	Unknown command	Opcode not recognized
0x0082	Out of memory	Server cannot allocate memory

Binary Protocol Flow

[Diagram: see online version]

Meta Commands Protocol

The meta commands protocol is a modern text-based alternative that combines the readability of the text protocol with richer semantics. It uses single-letter commands with flag arguments:

Command	Description
`mg <key> <flags>`	Meta Get -- retrieve with optional flags (v=return value, t=return TTL, c=return CAS, etc.)

`ms <key> <datalen> <flags>`	Meta Set -- store with flags (T=TTL, C=CAS compare, F=client flags, etc.)
`md <key> <flags>`	Meta Delete -- delete with optional flags
`ma <key> <flags>`	Meta Arithmetic -- increment/decrement with flags
`mn`	Meta Noop -- pipeline terminator
`me <key> <flags>`	Meta Debug -- return internal item metadata

Memcached vs Redis

Feature	Memcached	Redis
Data structures	Key-value only (strings)	Strings, lists, sets, sorted sets, hashes, streams, etc.
Persistence	None (pure cache)	RDB snapshots, AOF log, or both
Replication	None (client-side sharding)	Built-in master-replica replication
Clustering	Client-side consistent hashing	Redis Cluster (server-side sharding)
Memory efficiency	Slab allocator, very efficient	More overhead per key (metadata, pointers)
Max value size	1 MB (configurable)	512 MB
Threading	Multi-threaded	Single-threaded event loop (I/O threads in Redis 6+)
Eviction	LRU per slab class	Multiple policies (LRU, LFU, random, TTL-based)
Pub/Sub	No	Yes
Scripting	No	Lua scripting, Redis Functions
Transactions	CAS (single-key optimistic locking)	MULTI/EXEC with WATCH
Authentication	SASL (binary protocol)	Password, ACL (Redis 6+)
Use case	Simple caching, session storage	Cache, database, message broker, real-time analytics

Encapsulation

[Diagram: see online version]

Standards

Document	Title
Memcached Protocol Wiki	Protocol overview and links
Text Protocol	Text protocol specification

Binary Protocol	Binary protocol specification
Meta Commands	Meta commands protocol specification

See Also

- Redis -- in-memory data structure store with richer features
- MySQL -- relational database often fronted by Memcached
- PostgreSQL -- relational database often fronted by Memcached

MongoDB Wire Protocol

Standard: MongoDB Wire Protocol | **Layer:** Application (Layer 7) | **Wireshark filter:** `mongo`

The MongoDB wire protocol is a binary, little-endian, request-response protocol for communication between clients (drivers) and MongoDB servers (mongod/mongos). Every message begins with a standard 16-byte header containing the message length, request ID, response-to ID, and opcode. Modern MongoDB (3.6+) uses OP_MSG (opcode 2013) as the universal message format, replacing all legacy opcodes. OP_MSG carries BSON command documents in typed sections and supports checksums, exhaust cursors, and streaming. MongoDB also supports wire compression via OP_COMPRESSED. The protocol runs over TCP on port 27017 by default.

Message Header (MsgHeader)

Every MongoDB wire protocol message starts with this 16-byte header:

[Diagram: see online version]

Key Fields

Field	Size	Description
messageLength	4 bytes	Total size of the message in bytes, including the header
requestID	4 bytes	Client-generated identifier for this message
responseTo	4 bytes	requestID of the message this is a response to (0 for client requests)
opCode	4 bytes	Type of message

Opcodes

Opcode	Value	Status	Description
OP_MSG	2013	Current	Universal message format (MongoDB 3.6+)
OP_COMPRESSED	2012	Current	Compressed wrapper around another message
OP_REPLY	1	Deprecated	Legacy reply to client request
OP_QUERY	2004	Deprecated	Legacy query message
OP_GET_MORE	2005	Deprecated	Legacy cursor iteration
OP_INSERT	2002	Removed	Legacy insert (removed in 5.1)
OP_UPDATE	2001	Removed	Legacy update (removed in 5.1)
OP_DELETE	2006	Removed	Legacy delete (removed in 5.1)
OP_KILL_CURSORS	2007	Deprecated	Close server-side cursors

OP_MSG (Opcode 2013)

The modern universal message format used for all operations:

[Diagram: see online version]

OP_MSG Flag Bits

Bit	Name	Description
0	checksumPresent	Message ends with a CRC-32C checksum (4 bytes)
1	moreToCome	Sender will send another message without waiting for a response
16	exhaustAllowed	Client permits the server to stream multiple replies (exhaust cursor)

OP_MSG Sections

Kind	Name	Description
0	Body	Single BSON document -- the command and its parameters
1	Document Sequence	Named sequence of BSON documents (identifier + documents) -- used for batch writes

Section Kind 1 (Document Sequence)

[Diagram: see online version]

OP_COMPRESSED (Opcode 2012)

Wraps another message with compression:

[Diagram: see online version]

Compressor IDs

ID	Name	Description
0	noop	No compression (testing only)
1	snappy	Snappy compression
2	zlib	Zlib compression
3	zstd	Zstandard compression (MongoDB 4.2+)

Query Flow (Modern OP_MSG)

All database operations are expressed as command documents sent via OP_MSG:

[Diagram: see online version]

Common Commands (via OP_MSG Body)

Command	Description
find	Query documents from a collection
insert	Insert one or more documents
update	Update documents matching a filter
delete	Delete documents matching a filter
aggregate	Run an aggregation pipeline
getMore	Retrieve next batch from a cursor
killCursors	Close open cursors
createIndexes	Create indexes on a collection
isMaster / hello	Server handshake and topology discovery
ping	Check server availability
saslStart / saslContinue	SCRAM-SHA-1 or SCRAM-SHA-256 authentication

BSON Document Format

MongoDB messages carry data as BSON (Binary JSON) documents:

[Diagram: see online version]

BSON Element Types

Type	ID	Description
Double	0x01	64-bit IEEE 754 floating point
String	0x02	UTF-8 string (length-prefixed)
Document	0x03	Embedded BSON document
Array	0x04	BSON array (document with "0", "1", ... keys)
Binary	0x05	Binary data with subtype
ObjectId	0x07	12-byte unique identifier
Boolean	0x08	true (0x01) or false (0x00)
DateTime	0x09	UTC milliseconds since epoch (int64)
Null	0x0A	Null value
Int32	0x10	32-bit signed integer
Timestamp	0x11	MongoDB internal timestamp (ordinal + seconds)
Int64	0x12	64-bit signed integer
Decimal128	0x13	128-bit IEEE 754 decimal

Replica Set Connection

[Diagram: see online version]

Authentication

MongoDB uses SASL for authentication, sent as commands via OP_MSG:

Mechanism	Description
SCRAM-SHA-1	Default through MongoDB 3.x
SCRAM-SHA-256	Default from MongoDB 4.0+
MONGODB-X509	TLS client certificate authentication
MONGODB-AWS	AWS IAM authentication
PLAIN	LDAP proxy authentication (Enterprise)

Server Response Fields

Field	Description
ok	1 for success, 0 for failure
errmsg	Error message (when ok=0)
code	Numeric error code
codeName	String name for the error code
n	Number of documents affected
cursor	Cursor object with id, firstBatch/nextBatch, ns
writeConcernError	Write concern was not satisfied
writeErrors	Array of per-document write errors

Encapsulation

[Diagram: see online version]

Standards

Document	Title
MongoDB Wire Protocol	Wire protocol specification
OP_MSG	OP_MSG message format
BSON Specification	Binary JSON format specification
Server Selection	Read preference and server selection
MongoDB Driver Specifications	Official driver protocol specs

See Also

- MySQL -- relational database wire protocol
- PostgreSQL -- relational database wire protocol
- Redis -- in-memory data store protocol
- TCP
- TLS -- encrypts MongoDB connections

MySQL Protocol

Standard: MySQL Client/Server Protocol | **Layer:** Application (Layer 7) | **Wireshark filter:** `mysql`

The MySQL protocol is a binary, client/server wire protocol used to communicate with MySQL and MariaDB database servers. It operates in two phases: a connection phase (handshake and authentication) followed by a command phase where the client sends queries and receives result sets. The protocol is length-prefixed and uses little-endian byte ordering. It runs over TCP on port 3306 by default and optionally over TLS for encrypted connections.

Packet Header

Every MySQL packet begins with a 4-byte header:

[Diagram: see online version]

Key Fields

Field	Size	Description
Payload Length	3 bytes	Length of the payload (max 16 MB per packet; larger results split across multiple packets)
Sequence ID	1 byte	Sequence counter, incremented with each packet in a command exchange; resets to 0 for each new command
Payload	Variable	The actual protocol message data

Connection Phase

Handshake Flow

[Diagram: see online version]

Initial Handshake Packet (Server to Client)

[Diagram: see online version]

Capability Flags

Flag	Hex	Description
CLIENT_LONG_PASSWORD	0x00000001	Use improved password hashing
CLIENT_FOUND_ROWS	0x00000002	Return found rows instead of affected rows
CLIENT_LONG_FLAG	0x00000004	Longer column flags in column definitions

CLIENT_CONNECT_WITH_DB	0x00000008	Connect to database directly
CLIENT_PROTOCOL_41	0x00000200	New 4.1 protocol (always set)
CLIENT_SSL	0x00000800	Switch to TLS after handshake
CLIENT_TRANSACTIONS	0x00002000	Client knows about transactions
CLIENT_SECURE_CONNECTION	0x00008000	Secure authentication (4.1 password)
CLIENT_MULTI_STATEMENTS	0x00010000	Multiple statements per COM_QUERY
CLIENT_MULTI_RESULTS	0x00020000	Multiple result sets
CLIENT_PS_MULTI_RESULTS	0x00040000	Multiple result sets for prepared statements
CLIENT_PLUGIN_AUTH	0x00080000	Pluggable authentication
CLIENT_DEPRECATE_EOF	0x01000000	OK packet replaces EOF in result sets

Authentication Plugins

Plugin	Description
mysql_native_password	SHA1-based challenge-response (default through MySQL 5.7)
caching_sha2_password	SHA-256 with server-side cache (default from MySQL 8.0)
sha256_password	SHA-256 without caching (requires TLS or RSA exchange)
auth_socket	Unix socket peer credential authentication
mysql_clear_password	Plaintext password (requires TLS)

Command Phase

After authentication, the client sends command packets. The first byte of the payload identifies the command.

Command Types

Code	Command	Description
0x00	COM_SLEEP	Internal (not used by clients)
0x01	COM_QUIT	Close the connection
0x02	COM_INIT_DB	Switch to a database (USE statement)
0x03	COM_QUERY	Execute a SQL statement (text protocol)
0x04	COM_FIELD_LIST	List columns in a table (deprecated)
0x08	COM_STATISTICS	Get server statistics string
0x0E	COM_PING	Check if server is alive
0x16	COM_STMT_PREPARE	Prepare a statement (binary protocol)
0x17	COM_STMT_EXECUTE	Execute a prepared statement

0x18	COM_STMT_SEND_LONG_DATA	Send blob data for a prepared statement parameter
0x19	COM_STMT_CLOSE	Deallocate a prepared statement
0x1A	COM_STMT_RESET	Reset a prepared statement
0x1F	COM_RESET_CONNECTION	Reset session state without re-authenticating

Query Flow (Text Protocol)

[Diagram: see online version]

Prepared Statement Flow (Binary Protocol)

[Diagram: see online version]

Result Set Structure

Text Protocol Result Set

Packet	Description
Column Count	Length-encoded integer indicating number of columns
Column Definition	One per column: catalog, schema, table, name, charset, length, type, flags
EOF / OK marker	Separates column definitions from rows (omitted if CLIENT_DEPRECATE_EOF)
Row	One per row: length-encoded strings for each column value (NULL = 0xFB)
EOF / OK	End of result set

Response Packets

First Byte	Packet Type	Description
0x00	OK	Success with affected rows, insert ID, status, warnings
0xFF	ERR	Error with code, SQL state, message
0xFE	EOF	End marker (deprecated in favor of OK with CLIENT_DEPRECATE_EOF)

Status Flags

Flag	Hex	Description

SERVER_STATUS_IN_TRANS	0x0001	A transaction is active
SERVER_STATUS_AUTOCOMMIT	0x0002	Autocommit mode is enabled
SERVER_MORE_RESULTS_EXISTS	0x0008	More result sets follow (multi-statement)
SERVER_STATUS_CURSOR_EXISTS	0x0040	A read-only cursor exists
SERVER_STATUS_LAST_ROW_SENT	0x0080	Last row has been sent
SERVER_SESSION_STATE_CHANGED	0x4000	Session state was changed

Encapsulation

[Diagram: see online version]

Standards

Document	Title
MySQL Client/Server Protocol	MySQL Internals -- Client/Server Protocol
MySQL COM Commands	Command Phase documentation
MariaDB Protocol	MariaDB Client/Server Protocol (compatible)

See Also

- PostgreSQL -- frontend/backend message-based database protocol
- TDS -- Microsoft SQL Server and Sybase wire protocol
- Redis -- in-memory data store protocol
- TCP
- TLS -- encrypts MySQL connections

PostgreSQL Wire Protocol

Standard: PostgreSQL Frontend/Backend Protocol | **Layer:** Application (Layer 7) | **Wireshark filter:** `pgsql`

The PostgreSQL wire protocol is a message-based, binary protocol for communication between clients (frontends) and the PostgreSQL database server (backend). Messages are typed and length-prefixed. The protocol supports two query sub-protocols: the Simple Query protocol (send SQL text, get results) and the Extended Query protocol (parse, bind, execute as separate steps, enabling prepared statements and portals). PostgreSQL also defines a COPY sub-protocol for bulk data transfer and an asynchronous notification mechanism (LISTEN/NOTIFY). It runs on TCP port 5432 by default.

Message Format

Frontend (Client) and Backend (Server) Messages

All messages after the startup phase share this format:

[Diagram: see online version]

Startup Message (no type byte)

The initial startup message omits the type byte:

[Diagram: see online version]

Key Fields

Field	Size	Description
Type	1 byte	Single ASCII character identifying the message type (omitted in startup)
Length	4 bytes	Total length of message including the length field itself but not the type byte
Payload	Variable	Message-specific data

Connection Phase

Startup and Authentication

[Diagram: see online version]

SSL Negotiation

[Diagram: see online version]

Authentication Types

Code	Type	Description
0	AuthenticationOk	Authentication successful
2	AuthenticationKerberosV5	Kerberos V5 required (deprecated)
3	AuthenticationCleartextPassword	Cleartext password required
5	AuthenticationMD5Password	MD5 password hash required (with 4-byte salt)
7	AuthenticationGSS	GSSAPI authentication

9	AuthenticationSSPI	SSPI authentication (Windows)
10	AuthenticationSASL	SASL mechanism negotiation (SCRAM-SHA-256)
11	AuthenticationSASLContinue	SASL challenge
12	AuthenticationSASLFinal	SASL completion

Frontend Messages (Client to Server)

Type	Name	Description
Q	Query	Execute a simple SQL query (text)
P	Parse	Parse a SQL statement into a prepared statement
B	Bind	Bind parameters to a prepared statement, creating a portal
D	Describe	Describe a prepared statement (S) or portal (P)
E	Execute	Execute a portal
H	Flush	Force server to deliver pending output
S	Sync	Synchronization point (ends extended query, triggers ReadyForQuery)
C	Close	Close a prepared statement or portal
X	Terminate	Disconnect cleanly
d	CopyData	Data chunk during COPY operation
c	CopyDone	COPY operation complete
f	CopyFail	COPY operation failed
p	PasswordMessage / SASLResponse	Authentication data
F	FunctionCall	Call a server function by OID (legacy)

Backend Messages (Server to Client)

Type	Name	Description
R	Authentication*	Authentication request or result
K	BackendKeyData	Process ID and secret key for cancel requests
S	ParameterStatus	Runtime parameter value (e.g., server_encoding)
Z	ReadyForQuery	Server is ready; includes transaction status (I/T/E)

T	RowDescription	Column metadata for upcoming rows
D	DataRow	One row of result data
C	CommandComplete	SQL command completed (with row count)
I	EmptyQueryResponse	Response to an empty query string
E	ErrorResponse	Error with severity, code, message, detail, hint
N	NoticeResponse	Non-fatal warning or notice
1	ParseComplete	Parse step succeeded
2	BindComplete	Bind step succeeded
3	CloseComplete	Close step succeeded
n	NoData	Describe returned no row info (e.g., for INSERT)
t	ParameterDescription	Parameter types for a prepared statement
G	CopyInResponse	Server is ready to receive COPY data
H	CopyOutResponse	Server is sending COPY data
d	CopyData	Data chunk during COPY OUT
c	CopyDone	COPY OUT complete
A	NotificationResponse	Asynchronous NOTIFY payload

Simple Query Protocol

[Diagram: see online version]

Extended Query Protocol

The extended protocol separates parsing, binding, and execution for prepared statements and parameterized queries:

[Diagram: see online version]

COPY Protocol

Bulk data transfer for imports and exports:

[Diagram: see online version]

LISTEN/NOTIFY (Async Notifications)

[Diagram: see online version]

ReadyForQuery Transaction Status

Status	Meaning
I	Idle (not in a transaction)
T	In a transaction block
E	In a failed transaction block (queries will be rejected until ROLLBACK)

ErrorResponse Fields

Code	Field	Description
S	Severity	ERROR, FATAL, PANIC, WARNING, NOTICE, DEBUG, INFO, LOG
V	Severity (non-localized)	Always English severity
C	Code	SQLSTATE error code (5 characters, e.g., 42P01)
M	Message	Primary human-readable error message
D	Detail	Optional detailed error explanation
H	Hint	Optional suggestion for fixing the problem
P	Position	Cursor position in the query string
W	Where	Call stack context (PL/pgSQL)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
Frontend/Backend Protocol	PostgreSQL Wire Protocol specification
Message Flow	Protocol message flow documentation
Message Formats	Detailed message format reference
Error Codes	PostgreSQL SQLSTATE error codes
SCRAM Authentication	SASL/SCRAM-SHA-256 authentication

See Also

- MySQL -- client/server database wire protocol
- TDS -- Microsoft SQL Server and Sybase wire protocol
- Redis -- in-memory data store protocol
- TCP
- TLS -- encrypts PostgreSQL connections

Redis Protocol -- RESP

Standard: RESP Protocol Specification | **Layer:** Application (Layer 7) | **Wireshark filter:** `redis`

RESP (REdis Serialization Protocol) is the wire protocol used by Redis, an in-memory data structure store used as a database, cache, message broker, and streaming engine. RESP is a text-oriented, request-response protocol where clients send commands as arrays of bulk strings and the server replies with typed responses. The protocol is designed for simplicity and efficient parsing. RESP2 is the baseline version; RESP3 (introduced in Redis 6.0) adds richer data types including maps, sets, booleans, and client-side caching support. Redis runs on TCP port 6379 by default.

RESP2 Data Types

Each RESP value begins with a single-byte type prefix, followed by data and a CRLF (`\r\n`) terminator:

Prefix	Type	Format	Example
`+`	Simple String	`+OK\r\n`	Success responses
`-`	Error	`-ERR unknown command\r\n`	Error messages
`:`	Integer	`:1000\r\n`	Numeric responses (counters, booleans)
`$`	Bulk String	`$5\r\nhello\r\n`	Binary-safe strings (length-prefixed)
`*`	Array	`*2\r\n$3\r\nfoo\r\n$3\r\nbar\r\n`	Ordered collections of RESP values

Null Values

RESP2 Representation	Meaning
`$-1\r\n`	Null Bulk String (key does not exist)
`*-1\r\n`	Null Array

RESP3 Additional Types

RESP3 introduces new type prefixes for richer semantics:

Prefix	Type	Format	Description
`_`	Null	`_\r\n`	Unified null type
`#`	Boolean	`#t\r\n` or `#f\r\n`	True/false
`,`	Double	`,3.14\r\n`	Floating-point number
`(`	Big Number	`(3492890328409238509324850943850943825024385\r\n`	Arbitrary precision integer
`!`	Bulk Error	`!21\r\nSYNTAX invalid syntax\r\n`	Binary-safe error (length-prefixed)
`=`	Verbatim String	`=15\r\ntxt:Some text\r\n`	String with encoding hint (txt, mkd)
`%`	Map	`%2\r\n+key1\r\n:1\r\n+key2\r\n:2\r\n`	Key-value pairs
`~`	Set	`~3\r\n+a\r\n+b\r\n+c\r\n`	Unordered unique elements
`\`	`	Attribute	`\
`>`	Push	`>3\r\n+message\r\n+channel\r\n$5\r\nhello\r\n`	Out-of-band push data (pub/sub, invalidation)

Command Format

Clients send commands as RESP arrays of bulk strings:

```
*3\r\n
$3\r\n
SET\r\n
$5\r\n
mykey\r\n
$7\r\n
myvalue\r\n
```

Redis also supports inline commands (space-separated, no RESP framing) for human interaction:

```
SET mykey myvalue\r\n
```

Command/Response Flow

[Diagram: see online version]

Pipelining

Clients can send multiple commands without waiting for each response. The server buffers responses and sends them in order:

[Diagram: see online version]

Pub/Sub

When a client enters subscription mode, it only receives push messages:

[Diagram: see online version]

Transactions (MULTI/EXEC)

Command	Description
MULTI	Begin a transaction (commands are queued)
EXEC	Execute all queued commands atomically
DISCARD	Discard the transaction queue
WATCH key	Optimistic locking -- abort EXEC if key changed

[Diagram: see online version]

Authentication and ACL

Command	Description
AUTH password	Authenticate with a single password (legacy)
AUTH username password	Authenticate with username and password (Redis 6+ ACL)
ACL SETUSER	Create or modify a user with permissions
ACL LIST	List all configured ACL rules
HELLO 3	Switch to RESP3 protocol (optionally with AUTH)

Cluster Protocol

Redis Cluster distributes data across multiple nodes using hash slots (0-16383):

Response	Description
-MOVED 3999 127.0.0.1:6380	Slot permanently moved to another node; client should update slot map
-ASK 3999 127.0.0.1:6380	Slot temporarily migrating; send ASKING then retry to that node
-CLUSTERDOWN	Cluster is not available

Cluster Redirect Flow

[Diagram: see online version]

Sentinel Protocol

Redis Sentinel provides high availability. Sentinels use the same RESP protocol on port 26379:

Command	Description
SENTINEL get-master-addr-by-name mymaster	Get current master address
SENTINEL sentinels mymaster	List known sentinels
SENTINEL replicas mymaster	List known replicas
+switch-master	Pub/Sub notification when master changes

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RESP Protocol Spec	Redis Serialization Protocol specification
RESP3 Specification	RESP3 protocol specification
Redis Commands	Complete command reference
Redis Cluster Spec	Redis Cluster protocol and design
Redis Sentinel	Sentinel high availability documentation

See Also

- MySQL -- relational database wire protocol
- PostgreSQL -- relational database wire protocol
- MongoDB -- document database wire protocol
- MQTT -- lightweight pub/sub messaging (similar pub/sub concept)
- TCP

TDS -- Tabular Data Stream

Standard: MS-TDS (Microsoft Open Specifications) | **Layer:** Application (Layer 7) | **Wireshark filter:** `tds`

TDS (Tabular Data Stream) is the binary wire protocol used by Microsoft SQL Server and Sybase ASE for client-server communication. It is a message-oriented protocol layered on top of TCP, where each message consists of one or more packets with an 8-byte header. The protocol defines a connection establishment phase (pre-login negotiation and Login7 authentication), followed by request/response exchanges where clients send SQL batches or RPC calls and servers return results as token streams. TDS has evolved through multiple versions, with TDS 7.x/8.0 covering modern SQL Server releases. SQL Server listens on TCP port 1433 by default; Sybase uses port 2638.

Packet Header

Every TDS packet begins with an 8-byte header:

[Diagram: see online version]

Key Fields

Field	Size	Description
Type	1 byte	Message type identifier
Status	1 byte	Status flags (bit 0: last packet in message; bit 1: ignore event; bit 3: reset connection)
Length	2 bytes	Total packet length including the header (big-endian, max typically 4096 or 32767)
SPID	2 bytes	Server Process ID (used by the server for session tracking)
Packet ID	1 byte	Incremented per packet within a message (wraps at 255)
Window	1 byte	Reserved, always 0x00

Message Types

Type	Hex	Name	Direction	Description
1	0x01	SQL Batch	Client to Server	Raw SQL text to execute
2	0x02	Pre-TDS7 Login	Client to Server	Legacy login (TDS 4.2/5.0)
3	0x03	RPC	Client to Server	Remote procedure call (sp_executesql, stored procs)
4	0x04	Tabular Result	Server to Client	Token stream with results, errors, info

6	0x06	Attention	Client to Server	Cancel the current request
7	0x07	Bulk Load	Client to Server	Bulk insert data stream
14	0x0E	Transaction Manager	Client to Server	Distributed transaction coordination
16	0x10	Login7	Client to Server	TDS 7.x login message
17	0x11	SSPI	Client to Server	NTLM/Kerberos authentication token
18	0x12	Pre-Login	Both	Pre-login negotiation (version, encryption, instance)

TDS Versions

Version	Value	Description
TDS 4.2	0x04000000	Sybase, SQL Server 6.5 and earlier
TDS 5.0	0x05000000	Sybase ASE only
TDS 7.0	0x00000070	SQL Server 7.0
TDS 7.1	0x01000071	SQL Server 2000
TDS 7.2	0x02000972	SQL Server 2005
TDS 7.3	0x03000A73	SQL Server 2008
TDS 7.4	0x04000B74	SQL Server 2012+

Login Flow

Pre-Login and Authentication

[Diagram: see online version]

Pre-Login Message Options

Token	Description
VERSION	Client/server TDS version
ENCRYPTION	Encryption negotiation (off, on, required, not supported)
INSTOPT	SQL Server instance name
THREADID	Client thread ID
MARS	Multiple Active Result Sets support
TRACEID	Activity tracing ID

FEDAUTHREQUIRED	Federated authentication (Azure AD)

Login7 Packet

[Diagram: see online version]

Token Stream (Tabular Result)

Server responses are encoded as a stream of typed tokens. Each token begins with a 1-byte type identifier:

Token Types

Token	Hex	Name	Description
0x81	COLMETADATA	Column Metadata	Column names, types, and flags for the result set
0xD1	ROW	Row	One row of data in the result set
0xD2	NBCROW	Null Bitmap Compressed Row	Row with null bitmap optimization
0xFD	DONE	Done	Completion of a SQL statement (with row count)
0xFE	DONEPROC	Done Procedure	Completion of a stored procedure
0xFF	DONEINPROC	Done In Procedure	Completion of a statement within a stored procedure
0xAA	ERROR	Error	Error message with number, state, severity, text
0xAB	INFO	Info	Informational message
0xE3	ENVCHANGE	Environment Change	Database, language, packet size, transaction, or collation change
0xAD	LOGINACK	Login Acknowledgement	Confirms successful login with TDS version and server name
0xA9	ORDER	Order	Column ordering information
0xE5	RETURNSTATUS	Return Status	Return value from a stored procedure
0xAC	RETURNVALUE	Return Value	Output parameter value
0x79	COLINFO	Column Info	Additional column metadata
0xA4	NBCROW	Null Bitmap	Bitmap of NULL columns in a row
0xED	SSPI	SSPI Token	Security token for NTLM/Kerberos

Query Flow

SQL Batch Request and Response

[Diagram: see online version]

RPC (Stored Procedure / Parameterized Query)

[Diagram: see online version]

Attention (Cancel)

[Diagram: see online version]

Data Types

Type ID	Name	Description
0x30	INTN	Nullable integer (1, 2, 4, or 8 bytes)
0x3E	FLTN	Nullable float (4 or 8 bytes)
0xA7	NVARCHAR	Variable-length Unicode string
0xA5	BIGVARBINARY	Variable-length binary
0x6D	DATETIMN	Nullable datetime
0x6A	DECIMALN	Nullable decimal
0x6C	NUMERICN	Nullable numeric
0x24	GUIDTYPE	Uniqueidentifier (16 bytes)
0xF1	XMLTYPE	XML data
0x62	BIGVARCHARTYPE	Variable-length non-Unicode string

ENVCHANGE Types

Type	Description
1	Database changed
2	Language changed
3	Character set changed
4	Packet size changed
8	Begin transaction (returns transaction descriptor)
9	Commit transaction

10	Rollback transaction
17	Transaction manager address
20	Routing information (AlwaysOn redirect)

DONE Status Flags

Flag	Hex	Description
DONE_FINAL	0x0000	Final DONE in the request
DONE_MORE	0x0001	More results follow
DONE_ERROR	0x0002	Error occurred
DONE_INXACT	0x0004	Transaction is in progress
DONE_COUNT	0x0010	Row count is valid
DONE_ATTN	0x0020	Response to an Attention request
DONE_SRVERROR	0x0100	Severe error, connection may be dead

Encapsulation

[Diagram: see online version]

Standards

Document	Title
MS-TDS	Tabular Data Stream Protocol (Microsoft Open Specifications)
MS-SSRP	SQL Server Resolution Protocol (UDP instance discovery)
MS-LOGIN7	Login7 message specification
jTDS	Open-source TDS implementation reference
FreeTDS	FreeTDS protocol documentation

See Also

- MySQL -- MySQL client/server database wire protocol
- PostgreSQL -- PostgreSQL frontend/backend wire protocol
- Redis -- in-memory data store protocol
- SMB -- often co-located in Windows environments
- TCP
- TLS -- encrypts TDS connections

Email

Protocols for sending, receiving, and accessing electronic mail.

8 protocols in this section

DANE -- DNS-Based Authentication of Named Entities

Standard: RFC 6698 / RFC 7671 | **Layer:** Application (Layer 7) | **Wireshark filter:** `dns` (DANE uses TLSA DNS records)

DANE uses DNSSEC-signed DNS records to pin TLS certificates or public keys to domain names, eliminating the need to trust certificate authorities (CAs). Instead of relying on any of hundreds of CAs to vouch for a server's identity, DANE publishes the expected certificate or key directly in DNS -- secured by DNSSEC's chain of trust from the DNS root. DANE is primarily used for SMTP (securing email server-to-server TLS) and is seeing growing adoption for HTTPS, XMPP, and other TLS-protected services.

How DANE Works

[Diagram: see online version]

TLSA Record

DANE publishes a **TLSA** DNS record at `<port>.<protocol>.<hostname>`:

```
_25._tcp.mail.example.com. IN TLSA 3 1 1 e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
```

TLSA Fields

[Diagram: see online version]

Field	Size	Description
Usage	8 bits	What the TLSA record asserts (see below)
Selector	8 bits	What part of the certificate to match
Matching Type	8 bits	How to match (exact, SHA-256, SHA-512)
Certificate Association Data	Variable	The certificate data or hash to match against

Usage

Value	Name	Description
0	PKIX-TA	CA constraint -- must chain to this CA AND pass normal PKIX validation
1	PKIX-EE	Service certificate constraint -- this exact cert AND pass PKIX validation

2	DANE-TA	Trust anchor -- chain to this CA (no PKIX CAs needed)
3	DANE-EE	Domain-issued certificate -- match this cert exactly (no CA validation at all)

Usage 3 (DANE-EE) is the most common -- the domain publishes the hash of its own certificate's public key. No CA is involved.

Selector

Value	Name	Matches Against
0	Full certificate	The entire DER-encoded certificate
1	SubjectPublicKeyInfo	Just the public key (SPKI)

Selector 1 is preferred -- it survives certificate renewal as long as the key doesn't change.

Matching Type

Value	Name	Description
0	Exact	Full data (no hashing)
1	SHA-256	SHA-256 hash of the selected data
2	SHA-512	SHA-512 hash of the selected data

Common Combination

`3 1 1` -- DANE-EE, SubjectPublicKeyInfo, SHA-256 -- is by far the most common:

```
_25._tcp.mail.example.com. IN TLSA 3 1 1 &lt;SHA-256 hash of the server's public key&gt;
```

This says: "The server at mail.example.com on port 25 will present a certificate with this public key hash. Trust it directly -- no CA validation needed."

DANE for SMTP (MTA-STS Alternative)

DANE is the primary mechanism for authenticating SMTP server-to-server TLS:

[Diagram: see online version]

Without DANE (or MTA-STS), SMTP STARTTLS is opportunistic -- a MITM can strip the TLS upgrade and the sending server won't know. DANE provides **authenticated encryption** for email in transit.

DANE vs MTA-STS

Feature	DANE	MTA-STS

Dependency	DNSSEC (signed DNS zones)	HTTPS (well-known URL)
Deployment barrier	DNSSEC required (not universal)	Simpler (just HTTPS + DNS TXT)
Certificate pinning	Yes (TLSA records)	No (standard CA validation)
First-use protection	Yes (DNSSEC prevents tampering)	TOFU (Trust On First Use)
Standard	RFC 6698 / 7672	RFC 8461
Adoption	Strong in Europe (NL, DE), growing	Gmail, Microsoft support

DANE for HTTPS

DANE can also pin HTTPS certificates (less common than SMTP DANE):

```
_443._tcp.www.example.com. IN TLSA 3 1 1 &lt;hash&gt;
```

Browsers have not widely adopted DANE for HTTPS (Chrome removed DNSSEC support), but it is used in some high-security environments.

DNSSEC Requirement

DANE **requires DNSSEC** -- without it, an attacker could forge TLSA records. The trust chain:

[Diagram: see online version]

Each level is signed with DNSKEY/DS records, creating a chain of trust from the root.

DANE Adoption

Country/Provider	DANE SMTP Support
Netherlands (.nl)	~50% of mail domains
Germany (.de)	~30% of mail domains
Postfix	Native DANE verification
Exim	Native DANE verification
Gmail	Sends DANE-validated when available
Microsoft 365	DANE support added 2022

Standards

Document	Title
RFC 6698	DANE TLSA Record
RFC 7671	DANE Updates and Operational Guidance

RFC 7672	SMTP Security via Opportunistic DANE TLS
RFC 8461	MTA-STS (alternative to DANE)
RFC 4033	DNSSEC Introduction

See Also

- DNS -- TLSA records are DNS resource records
- TLS -- the certificates DANE pins
- SMTP -- primary use case (authenticated email encryption)
- SPF -- email sender authorization (complementary)
- DKIM -- email message signing (complementary)
- DMARC -- email authentication policy (complementary)

DKIM -- DomainKeys Identified Mail

Standard: RFC 6376 | **Layer:** Application (Layer 7) | **Wireshark filter:** N/A (DKIM is in email headers, verified by receiving server)

DKIM allows the sending domain to cryptographically sign email messages, proving that the message was authorized by the domain owner and has not been tampered with in transit. The sending server signs selected headers and the body with a private key; the receiving server retrieves the corresponding public key from DNS (a TXT record) and verifies the signature. DKIM survives forwarding (unlike SPF) because the signature travels with the message.

How DKIM Works

[Diagram: see online version]

DKIM-Signature Header

```
DKIM-Signature: v=1; a=rsa-sha256; c=relaxed/relaxed;
  d=example.com; s=selector1;
  h=from:to:subject:date:message-id;
  bh=2jUSOH9NhtVGCQWNr9BrIAPreKQjO6Sn7XIkfJVOzv8=;
  b=AuUoFEfDxTDkHlLXSZEpZj79LICEps6eda7W3deTVFOk2P...
```

Key Fields

Tag	Required	Description
v	Yes	Version (always 1)

a	Yes	Signing algorithm (rsa-sha256 or ed25519-sha256)
d	Yes	Signing domain (the domain claiming responsibility)
s	Yes	Selector (identifies which key in DNS)
h	Yes	Signed header fields (colon-separated list)
b	Yes	Signature data (Base64-encoded)
bh	Yes	Body hash (Base64-encoded hash of the canonicalized body)
c	No	Canonicalization method for header/body (relaxed/relaxed, simple/simple)
l	No	Body length limit (how many bytes of body were signed)
t	No	Signature timestamp (Unix epoch)
x	No	Signature expiration timestamp
i	No	Agent or user identifier (defaults to @d)
q	No	Query method for key retrieval (default dns/txt)

DNS Public Key Record

Published at <selector>._domainkey.<domain>:

```
selector1._domainkey.example.com. IN TXT "v=DKIM1; k=rsa; p=MIGfMA0GCSqGSIb3DQEBAQUAA4..."
```

Tag	Description
v	Version (DKIM1)
k	Key type (rsa or ed25519)
p	Public key (Base64-encoded; empty = key revoked)
t	Flags (y = testing mode, s = strict alignment)
h	Acceptable hash algorithms
n	Notes (human-readable)
s	Service type (* = all, email = email only)

Canonicalization

Canonicalization normalizes the message before signing to survive minor transit modifications:

Method	Headers	Body
`simple`	No modifications	No modifications (except trailing empty lines)
`relaxed`	Lowercase names, unfold lines, compress whitespace	Compress whitespace, remove trailing whitespace, remove trailing empty lines

`relaxed/relaxed` is the most common choice -- it tolerates minor reformatting by intermediate servers.

Signing Process

[Diagram: see online version]

Verification Process

1. Extract `d=` (domain) and `s=` (selector) from DKIM-Signature
2. Fetch public key from DNS: `<s>._domainkey.<d>`
3. Canonicalize the body and compute hash; compare with `bh=`
4. Canonicalize the signed headers + DKIM-Signature (without `b=` value)
5. Verify the `b=` signature with the public key
6. Result: **pass**, **fail**, **temperror**, or **permerror**

Key Rotation

Selectors enable key rotation without downtime:

Step	Action
1	Generate new key pair, publish as a new selector in DNS
2	Configure sending server to sign with the new selector
3	Wait for DNS propagation + message delivery time
4	Remove old selector from DNS

Multiple selectors can coexist -- receiving servers use the selector from each message's DKIM-Signature.

DKIM vs SPF

Feature	SPF	DKIM
What it checks	Sending server IP	Message content signature
Survives forwarding	No (different IP)	Yes (signature travels with message)
Detects tampering	No	Yes (body and headers are signed)
Where it's checked	MAIL FROM domain	DKIM-Signature `d=` domain
DNS record	TXT on domain	TXT on selector._domainkey.domain

Standards

Document	Title
RFC 6376	DomainKeys Identified Mail (DKIM) Signatures
RFC 6377	DKIM and Mailing Lists
RFC 8463	Ed25519 Signatures for DKIM
RFC 8301	DKIM Cryptographic Algorithm Updates

See Also

- SPF -- IP-based sender authorization
- DMARC -- policy framework combining SPF and DKIM
- DANE -- certificate pinning via DNSSEC
- SMTP -- email transport
- DNS -- public key published as DNS TXT record

DMARC -- Domain-based Message Authentication, Reporting, and Conformance

Standard: RFC 7489 | **Layer:** Application (Layer 7) | **Wireshark filter:** `dns` (DMARC is a DNS TXT record + email policy)

DMARC ties SPF and DKIM together into a policy framework, telling receiving servers what to do when email fails authentication and providing a reporting mechanism so domain owners can see who is sending email using their domain. DMARC solves the critical gap that SPF and DKIM leave open: neither alone verifies that the From: header address (what the user sees) matches the authenticated domain. DMARC adds **identifier alignment** -- the From: domain must match the SPF or DKIM authenticated domain.

How DMARC Works

[Diagram: see online version]

DMARC Record

Published as a DNS TXT record at `_dmarc.<domain>`:

```
_dmarc.example.com. IN TXT "v=DMARC1; p=reject; sp=quarantine; rua=mailto:dmarc-reports@example.com; ruf=mailto:
```

Key Tags

Tag	Required	Description
`v`	Yes	Version (always `DMARC1`)
`p`	Yes	Policy for the domain
`sp`	No	Policy for subdomains (defaults to `p`)
`rua`	No	Aggregate report destination (mailto: URI)
`ruf`	No	Forensic/failure report destination (mailto: URI)
`adkim`	No	DKIM alignment mode: `r` = relaxed (default), `s` = strict
`aspf`	No	SPF alignment mode: `r` = relaxed (default), `s` = strict
`pct`	No	Percentage of messages to apply policy to (0-100, default 100)
`fo`	No	Failure reporting options
`rf`	No	Forensic report format (default `afrf`)
`ri`	No	Aggregate report interval in seconds (default 86400 = 24h)

Policy Values

Policy	Action
`none`	Monitor only -- deliver and report (deployment phase)
`quarantine`	Treat as suspicious -- deliver to spam/junk
`reject`	Reject the message outright (strongest)

Recommended Deployment Path

[Diagram: see online version]

1. Start with `p=none` and collect reports to identify all legitimate sending sources
2. Add SPF and DKIM for all legitimate sources
3. Move to `p=quarantine` with low `pct` and gradually increase
4. Move to `p=reject` once confident all legitimate mail passes

Identifier Alignment

DMARC requires that the From: header domain aligns with at least one authenticated domain:

Check	Alignment	Passes When
SPF alignment	From: domain matches MAIL FROM domain	`relaxed`: organizational domain match; `strict`: exact match
DKIM alignment	From: domain matches DKIM `d=` domain	`relaxed`: organizational domain match; `strict`: exact match

Relaxed alignment: `mail.example.com` aligns with `example.com` **Strict** alignment: `mail.example.com` does NOT align with `example.com`

DMARC **passes** if either SPF or DKIM passes with alignment.

DMARC Evaluation

[Diagram: see online version]

Aggregate Reports (rua)

Receiving servers send daily XML reports (gzipped) to the `rua` address:

Report Field	Description
Report metadata	Reporter, date range, policy published
Policy published	The DMARC record at time of evaluation
Records	Per-source-IP results: count, SPF result, DKIM result, disposition

Example Report Record

```
&lt;record&gt;
  &lt;row&gt;
    &lt;source_ip&gt;203.0.113.10&lt;/source_ip&gt;
    &lt;count&gt;1542&lt;/count&gt;
    &lt;policy_evaluated&gt;
      &lt;disposition&gt;none&lt;/disposition&gt;
      &lt;dkim&gt;pass&lt;/dkim&gt;
      &lt;spf&gt;pass&lt;/spf&gt;
    &lt;/policy_evaluated&gt;
  &lt;/row&gt;
  &lt;identifiers&gt;
    &lt;header_from&gt;example.com&lt;/header_from&gt;
  &lt;/identifiers&gt;
  &lt;auth_results&gt;
    &lt;dkim&gt;&lt;domain&gt;example.com&lt;/domain&gt;&lt;result&gt;pass&lt;/result&gt;&lt;/dkim&gt;
    &lt;spf&gt;&lt;domain&gt;example.com&lt;/domain&gt;&lt;result&gt;pass&lt;/result&gt;&lt;/spf&gt;
  &lt;/auth_results&gt;
&lt;/record&gt;
```

Authentication-Results Header

Receiving servers add this header for downstream filters:

```
Authentication-Results: mx.receiver.com;
  dmarc=pass (p=reject dis=none) header.from=example.com;
  dkim=pass header.d=example.com header.s=selector1;
  spf=pass smtp.mailfrom=example.com
```

Standards

Document	Title
RFC 7489	DMARC
RFC 7208	SPF (dependency)
RFC 6376	DKIM (dependency)
RFC 7601	Message Header Field for Authentication Results

See Also

- SPF -- IP-based sender authorization
- DKIM -- cryptographic message signing
- DANE -- TLS certificate pinning via DNSSEC
- SMTP -- email transport
- DNS -- DMARC records are DNS TXT records

IMAP -- Internet Message Access Protocol

Standard: RFC 9051 | **Layer:** Application (Layer 7) | **Wireshark filter:** `imap`

IMAP is the standard protocol for accessing email stored on a mail server. Unlike POP3 (which downloads and typically deletes messages), IMAP keeps mail on the server and synchronizes state across multiple clients -- read/unread flags, folders, and search all work consistently whether you check email from your phone, laptop, or webmail. IMAP supports multiple mailboxes (folders), server-side search, partial message fetch, and concurrent access. It is the protocol behind virtually every modern email client.

Commands

IMAP commands are prefixed with a client-generated tag (e.g., `a001`) that matches responses to requests:

Command	Arguments	Description
LOGIN	user password	Authenticate with plaintext credentials
AUTHENTICATE	mechanism	SASL authentication (PLAIN, XOAUTH2, etc.)

STARTTLS	--	Upgrade to TLS encryption
SELECT	mailbox	Open a mailbox (read-write)
EXAMINE	mailbox	Open a mailbox (read-only)
LIST	reference pattern	List available mailboxes
CREATE	mailbox	Create a new mailbox
DELETE	mailbox	Delete a mailbox
RENAME	old new	Rename a mailbox
SUBSCRIBE	mailbox	Add to subscribed list
STATUS	mailbox (items)	Get mailbox status (MESSAGES, UNSEEN, etc.)
FETCH	sequence items	Retrieve message data
STORE	sequence flags	Set/clear message flags
SEARCH	criteria	Search for messages matching criteria
COPY	sequence mailbox	Copy messages to another mailbox
MOVE	sequence mailbox	Move messages to another mailbox
EXPUNGE	--	Permanently remove messages flagged \Deleted
CLOSE	--	Close mailbox and expunge
IDLE	--	Wait for server-push notifications (RFC 2177)
NOOP	--	Keepalive / check for new mail
LOGOUT	--	End the session
CAPABILITY	--	List supported extensions
APPEND	mailbox message	Upload a message to a mailbox
UID	command args	Use UIDs instead of sequence numbers

Response Types

Prefix	Meaning
`*`	Untagged response (server-initiated data)
`+`	Command continuation (server needs more data)
`tag OK`	Command succeeded
`tag NO`	Command failed (operational error)
`tag BAD`	Command error (protocol/syntax error)

Session Flow

```
S: * OK [CAPABILITY IMAP4rev2 STARTTLS AUTH=PLAIN] IMAP ready
C: a001 STARTTLS
S: a001 OK Begin TLS negotiation
  [TLS handshake]
C: a002 LOGIN alice secret
S: a002 OK LOGIN completed
C: a003 SELECT INBOX
S: * 172 EXISTS
S: * 1 RECENT
S: * FLAGS (\Answered \Flagged \Deleted \Seen \Draft)
S: * OK [UIDVALIDITY 1234567890]
S: a003 OK [READ-WRITE] SELECT completed
C: a004 FETCH 170:172 (FLAGS ENVELOPE)
S: * 170 FETCH (FLAGS (\Seen) ENVELOPE ("Mon, 17 Mar 2026..." ...))
S: * 171 FETCH (FLAGS () ENVELOPE ("Tue, 18 Mar 2026..." ...))
S: * 172 FETCH (FLAGS (\Recent) ENVELOPE ("Wed, 19 Mar 2026..." ...))
S: a004 OK FETCH completed
C: a005 STORE 171 +FLAGS (\Seen)
S: * 171 FETCH (FLAGS (\Seen))
S: a005 OK STORE completed
C: a006 LOGOUT
S: * BYE IMAP server closing connection
S: a006 OK LOGOUT completed
```

Message Flags

Flag	Description
\Seen	Message has been read
\Answered	Message has been replied to
\Flagged	Message is flagged/starred
\Deleted	Message marked for deletion (expunged later)
\Draft	Message is a draft
\Recent	Message arrived since last session (read-only)

FETCH Data Items

Item	Description
BODY[]	Full message (headers + body)
BODY[HEADER]	Headers only
BODY[TEXT]	Body only
BODY[1]	First MIME part
BODYSTRUCTURE	MIME structure without downloading content
ENVELOPE	Parsed header fields (From, To, Subject, Date, etc.)
FLAGS	Message flags
RFC822.SIZE	Message size in bytes

UID	Unique Identifier (persistent across sessions)
INTERNALDATE	Date the server received the message

IMAP vs POP3

Feature	IMAP	POP3
Mail storage	Server (synced)	Downloaded to client
Folders	Full server-side folder support	Inbox only
Multi-client	Yes -- all clients see same state	Conflicts between clients
Server-side search	Yes	No
Partial fetch	Yes (MIME parts, ranges)	Full message only
Flags	Synced across clients	Local only
Offline	Cached locally	Full local copy
Bandwidth	Efficient (fetch only what's needed)	Downloads everything
Port	143 (STARTTLS), 993 (implicit TLS)	110 (STARTTLS), 995 (implicit TLS)

Common Extensions

Extension	RFC	Description
IDLE	RFC 2177	Push notifications (no polling)
CONDSTORE	RFC 7162	Conditional store with mod-sequences
QRESYNC	RFC 7162	Quick resynchronization after reconnect
COMPRESS	RFC 4978	DEFLATE compression
MOVE	RFC 6851	Atomic move operation
SPECIAL-USE	RFC 6154	Standard folder roles (Sent, Drafts, Trash, Junk)
LITERAL+	RFC 7888	Non-synchronizing literals
NOTIFY	RFC 5465	Advanced event notifications

Encapsulation

[Diagram: see online version]

Standards

Document	Title

RFC 9051	Internet Message Access Protocol (IMAP) Version 4rev2
RFC 3501	IMAP Version 4rev1 (widely deployed)
RFC 2177	IMAP IDLE Command
RFC 7162	CONDSTORE and QRESYNC Extensions
RFC 6154	IMAP LIST Extension for Special-Use Mailboxes

See Also

- SMTP -- sends email (IMAP retrieves it)
- TCP
- TLS -- encrypts IMAP connections
- DNS -- SRV records for IMAP server auto-discovery

JMAP -- JSON Meta Application Protocol

Standard: RFC 8620 / RFC 8621 | **Layer:** Application (Layer 7) | **Wireshark filter:** `http` or `http2` (JSON over HTTPS)

JMAP is a modern, HTTP-based protocol designed as a replacement for IMAP. It uses JSON over HTTPS instead of a custom text protocol, making it simpler to implement in web and mobile clients. JMAP supports efficient synchronization (delta updates via /changes), batched method calls in a single HTTP request, push notifications via Server-Sent Events, and binary upload/download for attachments. It eliminates the statefulness, complexity, and bandwidth inefficiency of IMAP while providing equivalent functionality.

Request Structure

All JMAP operations are sent as a single HTTP POST to the API endpoint:

```
POST /jmap/ HTTP/1.1
Content-Type: application/json
Authorization: Bearer &lt;token&gt;
```

[Diagram: see online version]

Request Body

```
{
  "using": [
    "urn:ietf:params:jmap:core",
    "urn:ietf:params:jmap:mail"
  ],
  "methodCalls": [
    ["Mailbox/get", {"accountId": "abc", "ids": null}, "call-0"],
    ["Email/query", {"accountId": "abc", "filter": {"inMailbox": "inbox-id"}}, "call-1"],
    ["Email/get", {"accountId": "abc", "#ids": {"resultOf": "call-1", "name": "Email/query", "path": "/ids"}}, "
  ]
}
```

Response Body

```
{
  "methodResponses": [
    ["Mailbox/get", {"accountId": "abc", "list": [...], "state": "m42"}, "call-0"],
    ["Email/query", {"accountId": "abc", "ids": ["e1","e2"], "queryState": "q7"}, "call-1"],
    ["Email/get", {"accountId": "abc", "list": [...], "state": "e15"}, "call-2"]
  ],
  "sessionState": "s3"
}
```

Method Calls

Method	Description
Mailbox/get	Retrieve mailbox properties (name, role, counts)
Mailbox/query	Search/filter mailboxes
Mailbox/set	Create, update, or destroy mailboxes
Mailbox/changes	Get mailbox changes since a given state
Email/get	Retrieve email properties (subject, from, body, etc.)
Email/query	Search/filter emails (by mailbox, date, keyword, text)
Email/set	Create, update (move, flag), or destroy emails
Email/changes	Get email changes since a given state
Email/queryChanges	Get changes to a previous query result
Email/copy	Copy emails between accounts
EmailSubmission/set	Submit an email for sending via SMTP
Thread/get	Retrieve conversation threads
Thread/changes	Get thread changes since a given state
Identity/get	Retrieve sender identities
SearchSnippet/get	Get highlighted search result snippets

Session Resource

The JMAP session is discovered via a well-known URL (`/.well-known/jmap`) and provides:

Field	Description
username	Authenticated user's username
accounts	Map of account IDs to account objects (name, capabilities)
capabilities	Server capabilities and limits
apiUrl	URL for JMAP API requests
uploadUrl	URL template for binary uploads
downloadUrl	URL template for binary downloads
eventSourceUrl	URL for push notifications (Server-Sent Events)
state	Session state string (changes when session config changes)

Key Fields

Field	Description
using	Array of capability URIs the request requires
methodCalls	Array of [methodName, arguments, callId] triples
methodResponses	Array of [methodName, response, callId] triples
accountId	Target account for the method call
ids	Array of object IDs to retrieve (null = all)
state	Opaque state string for synchronization
#ids (back-reference)	Reference the result of a previous method call in the same request
filter	Filter conditions for /query methods
sort	Sort criteria for /query methods
position / anchor	Pagination for /query results

Email Fetch Flow

[Diagram: see online version]

Email Send Flow

[Diagram: see online version]

Push Notifications

[Diagram: see online version]

Push mechanisms:

Method	Description
EventSource (SSE)	Long-lived HTTP connection; server sends state change events
Web Push (RFC 8030)	Push notifications via push service (for mobile/background)

Efficient Sync

JMAP eliminates IMAP's resync overhead with state-based delta updates:

Method	Purpose
Foo/changes	Returns created, updated, destroyed IDs since a given state
Foo/queryChanges	Returns additions/removals from a previous query result

The client stores the state string from each response and provides it as `sinceState` on the next sync. The server returns only what changed.

JMAP vs IMAP

Feature	JMAP	IMAP
Transport	HTTPS (standard port 443)	Custom TCP (port 143/993)
Data format	JSON	Custom text protocol
Connection model	Stateless HTTP requests	Stateful persistent connection
Batching	Multiple method calls per request	One command at a time
Sync	/changes and /queryChanges (delta)	Full re-fetch or CONDSTORE/QRESYNC
Push	EventSource (SSE) or Web Push	IDLE (holds connection open)
Binary data	Upload/download URLs	LITERAL inline or BINARY extension
Firewall friendly	Standard HTTPS (port 443)	Requires port 143/993
Back-references	Yes (chain method calls in one request)	No
Implementation	Simple (HTTP + JSON libraries)	Complex (custom parser, state machine)
Mobile battery	Efficient (stateless, push)	Poor (persistent connection, IDLE)
Partial fetch	Property selection in /get	BODY[section] and PARTIAL

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 8620	The JSON Meta Application Protocol (JMAP)
RFC 8621	JMAP for Mail
RFC 8887	JMAP Subprotocol for WebSocket
RFC 9007	Handling Message Disposition Notification with JMAP
jmap.io	JMAP community resources and specifications

See Also

- IMAP -- the protocol JMAP is designed to replace
- SMTP -- JMAP uses EmailSubmission to trigger SMTP delivery
- HTTP -- JMAP transport
- SSE -- Server-Sent Events for JMAP push notifications

POP3 -- Post Office Protocol version 3

Standard: RFC 1939 | **Layer:** Application (Layer 7) | **Wireshark filter:** pop

POP3 is a simple protocol for retrieving email from a mail server. It follows a download-and-delete model -- the client connects, authenticates, downloads messages, and optionally deletes them from the server. Unlike IMAP, POP3 has no concept of folders, server-side search, or multi-client synchronization. It remains in use for simple single-device email setups and legacy systems.

Commands

Command	Arguments	Description
USER	username	Identify the user
PASS	password	Authenticate
STAT	--	Get mailbox stats (count, total size)
LIST	[msg#]	List message sizes
RETR	msg#	Retrieve a message
DELE	msg#	Mark message for deletion
NOOP	--	No operation (keepalive)
RSET	--	Undelete all marked messages

QUIT	--	Commit deletions and close
TOP	msg# lines	Retrieve headers + first N lines
UIDL	[msg#]	Unique ID listing (for tracking seen messages)
APOP	name digest	MD5 challenge-response auth

Responses

Response	Meaning
`+OK`	Command succeeded (followed by data or info)
`-ERR`	Command failed (followed by error message)

Session Example

```
S: +OK POP3 server ready &lt;1896.697170952@example.com&gt;
C: USER alice
S: +OK
C: PASS secret
S: +OK alice's maildrop has 2 messages (3200 octets)
C: STAT
S: +OK 2 3200
C: LIST
S: +OK 2 messages (3200 octets)
S: 1 1600
S: 2 1600
S: .
C: RETR 1
S: +OK 1600 octets
S: [message content]
S: .
C: DELE 1
S: +OK message 1 deleted
C: QUIT
S: +OK dewstrstrr POP3 server signing off (1 message left)
```

Session States

[Diagram: see online version]

POP3 vs IMAP

Feature	POP3	IMAP
Model	Download and delete	Server-side sync
Folders	Inbox only	Full folder support
Multi-device	Conflicts	Synchronized state

Server search	No	Yes
Partial fetch	TOP only	Fine-grained (MIME parts)
Port	110 / 995 (TLS)	143 / 993 (TLS)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 1939	Post Office Protocol -- Version 3
RFC 2595	Using TLS with IMAP, POP3 and ACAP

See Also

- IMAP -- modern alternative with server-side sync
- SMTP -- sends email (POP3 retrieves it)
- TCP

SMTP -- Simple Mail Transfer Protocol

Standard: RFC 5321 | **Layer:** Application (Layer 7) | **Wireshark filter:** `smtp`

SMTP is the standard protocol for sending email across the Internet. It handles the transfer of mail from a sender's mail client (MUA) to their outgoing mail server, and between mail servers (MTAs) en route to the recipient. SMTP is a text-based, command-response protocol operating over TCP. It handles only sending -- retrieval is handled by IMAP or POP3. Modern SMTP universally uses TLS for encryption (via STARTTLS or implicit TLS on port 465).

Message Flow

[Diagram: see online version]

Commands

SMTP uses a command-response model. The client sends commands; the server responds with status codes.

Command	Arguments	Description

EHLO	domain	Extended greeting -- identifies client and requests extensions
HELO	domain	Basic greeting (legacy; EHLO preferred)
MAIL FROM	\<sender@domain\>	Specify the envelope sender (return path)
RCPT TO	\<recipient@domain\>	Specify an envelope recipient (repeat for multiple)
DATA	--	Begin message content (terminated by `.` on a line alone)
RSET	--	Reset the session (cancel current transaction)
VRFY	user	Verify a mailbox exists (often disabled)
EXPN	list	Expand a mailing list (often disabled)
NOOP	--	No operation (keepalive)
QUIT	--	Close the connection
STARTTLS	--	Upgrade to TLS encryption (RFC 3207)
AUTH	mechanism	Authenticate the client (RFC 4954)

Response Codes

Code	Category	Examples
2xx	Success	220 Service ready, 250 OK, 235 Authentication successful
3xx	Intermediate	334 Auth challenge, 354 Start mail input
4xx	Temporary failure	421 Service not available, 450 Mailbox unavailable, 451 Local error
5xx	Permanent failure	500 Syntax error, 550 Mailbox not found, 553 Invalid address, 554 Transaction failed

Common Response Codes

Code	Meaning
220	Server ready
250	Requested action completed
354	Start mail input, end with `<CRLF>.<CRLF>`
421	Service not available, closing connection
450	Mailbox temporarily unavailable
500	Syntax error, command unrecognized

550	Mailbox not found / access denied
552	Message too large
554	Transaction failed

Session Example

```
S: 220 mail.example.com ESMTP Postfix
C: EHLO client.example.com
S: 250-mail.example.com
S: 250-STARTTLS
S: 250-AUTH PLAIN LOGIN
S: 250-SIZE 52428800
S: 250 8BITMIME
C: STARTTLS
S: 220 Ready to start TLS
  [TLS handshake]
C: EHLO client.example.com
C: AUTH PLAIN AGFsaWNlAHNlY3JldA==
S: 235 Authentication successful
C: MAIL FROM:&lt;alice@example.com&gt;
S: 250 OK
C: RCPT TO:&lt;bob@example.com&gt;
S: 250 OK
C: DATA
S: 354 End data with &lt;CR&gt;&lt;LF&gt;.&lt;CR&gt;&lt;LF&gt;
C: From: alice@example.com
C: To: bob@example.com
C: Subject: Hello
C:
C: Hi Bob!
C: .
S: 250 OK: queued as ABC123
C: QUIT
S: 221 Bye
```

SMTP Extensions (ESMTP)

Extensions are advertised in the EHLO response:

Extension	RFC	Description
STARTTLS	RFC 3207	Upgrade to TLS
AUTH	RFC 4954	Client authentication (PLAIN, LOGIN, XOAUTH2)
SIZE	RFC 1870	Declare maximum message size
8BITMIME	RFC 6152	Allow 8-bit content in message body
PIPELINING	RFC 2920	Send multiple commands without waiting
CHUNKING	RFC 3030	Transfer large messages in chunks (BDAT)
SMTPUTF8	RFC 6531	Internationalized email addresses

DSN	RFC 3461	Delivery Status Notifications
BINARYMIME	RFC 3030	Binary content transfer

Authentication Methods

Method	Description
PLAIN	Base64-encoded username + password (requires TLS)
LOGIN	Legacy Base64 username/password exchange
CRAM-MD5	Challenge-response (avoids sending password)
XOAUTH2	OAuth 2.0 bearer token (Gmail, Microsoft)

Ports

Port	Service	Encryption
25	MTA-to-MTA relay	Opportunistic STARTTLS
465	Submission (implicit TLS)	Mandatory TLS from connection start
587	Submission (STARTTLS)	STARTTLS required before authentication

Port 25 is for server-to-server relay. Ports 587 and 465 are for client submission (MUA to MSA).

Email Authentication

Modern SMTP relies on DNS-based authentication to combat spoofing:

Mechanism	DNS Record	Purpose
SPF	TXT	Lists authorized sending IPs for a domain
DKIM	TXT	Cryptographic signature on message headers/body
DMARC	TXT	Policy for handling SPF/DKIM failures
MTA-STS	TXT + HTTPS	Enforce TLS for inbound mail
DANE/TLSA	TLSA	Certificate pinning via DNSSEC

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 5321	Simple Mail Transfer Protocol
RFC 5322	Internet Message Format
RFC 3207	SMTP Service Extension for Secure SMTP over TLS (STARTTLS)
RFC 4954	SMTP Service Extension for Authentication
RFC 6409	Message Submission for Mail (port 587)
RFC 8314	Cleartext Considered Obsolete -- Use of TLS for Email
RFC 7208	Sender Policy Framework (SPF)
RFC 6376	DomainKeys Identified Mail (DKIM)
RFC 7489	DMARC -- Domain-based Message Authentication

See Also

- TCP
- TLS -- encrypts SMTP connections
- DNS -- MX record lookup for mail routing; SPF/DKIM/DMARC
- SMPP -- SMS messaging protocol (sometimes bridged with email)

SPF -- Sender Policy Framework

Standard: RFC 7208 | **Layer:** Application (Layer 7) | **Wireshark filter:** `dns` (SPF is published as DNS TXT records)

SPF is an email authentication mechanism that allows domain owners to specify which mail servers are authorized to send email on behalf of their domain. It works by publishing a DNS TXT record listing the permitted sending IP addresses. When a receiving mail server gets an email, it checks the SMTP envelope sender (MAIL FROM) domain's SPF record against the connecting server's IP address. SPF is one of the three pillars of email authentication alongside DKIM and DMARC.

How SPF Works

[Diagram: see online version]

SPF Record Syntax

An SPF record is a DNS TXT record beginning with `v=spf1`:

```
v=spf1 ip4:203.0.113.0/24 ip6:2001:db8::/32 include:_spf.google.com -all
```

Mechanisms

Mechanism	Description	Example
`ip4`	IPv4 address or CIDR range	`ip4:203.0.113.0/24`
`ip6`	IPv6 address or CIDR range	`ip6:2001:db8::/32`
`a`	Domain's A/AAAA records	`a:mail.example.com`
`mx`	Domain's MX records	`mx`
`include`	Include another domain's SPF policy	`include:_spf.google.com`
`exists`	Check if a DNS A record exists	`exists:%{i}.spf.example.com`
`all`	Match everything (usually at the end)	`-all`
`redirect`	Use another domain's SPF entirely	`redirect=_spf.example.com`

Qualifiers

Qualifier	Result	Meaning
`+` (default)	Pass	Authorized sender
`-`	Fail (hard fail)	Unauthorized -- reject
`~`	SoftFail	Probably unauthorized -- accept but mark
`?`	Neutral	No assertion

Results

Result	Action
Pass	IP is authorized -- deliver normally
Fail	IP is not authorized -- reject or quarantine
SoftFail	IP is probably not authorized -- accept but flag
Neutral	Domain makes no assertion
None	No SPF record found
TempError	DNS lookup failed -- try later
PermError	SPF record is malformed

Common SPF Records

Provider	SPF Include
Google Workspace	`include:_spf.google.com`
Microsoft 365	`include:spf.protection.outlook.com`
Amazon SES	`include:amazonses.com`
Mailchimp	`include:servers.mcsv.net`
SendGrid	`include:sendgrid.net`

Limitations

Limitation	Description
DNS lookup limit	Max 10 DNS lookups (includes, a, mx, exists, redirect)
Forwarding breaks SPF	Forwarded email comes from a different IP than the original sender
Only checks envelope sender	Does not verify the From: header (that's DMARC's job)
No content integrity	Does not sign or verify the message body (that's DKIM)

Standards

Document	Title
RFC 7208	Sender Policy Framework (SPF)
RFC 7372	Email Authentication Status Codes

See Also

- DKIM -- cryptographic email signing
- DMARC -- policy framework that ties SPF and DKIM together
- DANE -- certificate pinning via DNS
- SMTP -- the email transport SPF protects
- DNS -- SPF records are DNS TXT records

Encoding & Symbology

Character encoding, barcode symbology, and data encoding schemes.

4 protocols in this section

Code 128

Standard: ISO/IEC 15417:2007 | **Category:** 1D Linear Barcode / Symbology

Code 128 is a high-density alphanumeric barcode capable of encoding all 128 ASCII characters. It is widely used in shipping labels (GS1-128), logistics, inventory management, and any application requiring compact alphanumeric or numeric barcodes. Code 128 uses three character sets (A, B, C) and can switch between them mid-barcode for optimal density -- Code Set C encodes digit pairs in a single symbol, making it extremely compact for numeric data.

Structure

[Diagram: see online version]

Each character is encoded as 3 bars and 3 spaces within 11 modules (the stop pattern uses 13 modules including the final bar).

Character Sets

Set	Start Code	Characters	Best For
Code A	103	ASCII 0-95 (uppercase, control chars, digits)	Uppercase + control characters
Code B	104	ASCII 32-127 (upper + lowercase, digits)	Mixed case text
Code C	105	Digit pairs 00-99	Dense numeric data (5.5 digits/cm)

Switching Between Sets

Code	From	To	Description
98	A or B	A or B	Shift (next character only from the other set)
99	A or B	C	Switch to Code C
100	A or C	B	Switch to Code B
101	B or C	A	Switch to Code A

Special Codes

Value	Name	Description
102	FNC1	GS1-128 application identifier prefix
96	FNC2	Message append

97	FNC3	Reader initialization
103	START A	Begin with Code Set A
104	START B	Begin with Code Set B
105	START C	Begin with Code Set C
106	STOP	End of barcode

Check Digit Calculation

Weighted modulo 103 checksum:

1. Start with the start code value
2. Add each data character value x its position (1-indexed)
3. Result mod 103 = check character value

Example: "Code128"

```
START B (104) x 1 = 104 (position weight for start is just its value)
'C' (35) x 1 = 35
'o' (79) x 2 = 158
'd' (68) x 3 = 204
'e' (69) x 4 = 276
'1' (17) x 5 = 85
'2' (18) x 6 = 108
'8' (24) x 7 = 168

Sum = 104 + 35 + 158 + 204 + 276 + 85 + 108 + 168 = 1138
Check = 1138 mod 103 = 5 (character '%' in Code B)
```

GS1-128 (formerly UCC/EAN-128)

GS1-128 uses Code 128 with FNC1 as the first character to signal GS1 Application Identifiers:

```
[FNC1] (01) 00012345678905 (17) 260401 (10) ABC123
```

Common Application Identifiers

AI	Name	Format	Example
(01)	GTIN	14 digits	Product identifier
(10)	Batch/Lot	Variable (up to 20)	Production lot
(17)	Expiration date	YYMMDD	260401 = April 1, 2026
(21)	Serial number	Variable (up to 20)	Unique item serial
(310n)	Net weight (kg)	6 digits	310**3** 001500 = 1.500 kg
(37)	Count	Variable (up to 8)	Quantity in shipment
(400)	Purchase order	Variable (up to 30)	Customer PO number

GS1-128 is the standard barcode on shipping labels, pallet labels, and pharmaceutical packaging.

Code 128 vs Other 1D Barcodes

Feature	Code 128	Code 39	UPC-A	Interleaved 2 of 5
Character set	Full ASCII	A-Z, 0-9, 7 symbols	Digits only	Digits only
Density	High	Low	Medium	Medium
Self-checking	No (check digit needed)	Yes	Yes	No
Variable length	Yes	Yes	Fixed (12)	Yes (even count)
GS1 use	GS1-128	No	Retail POS	ITF-14 (cartons)

Standards

Document	Title
ISO/IEC 15417:2007	Code 128 bar code symbology specification
GS1 General Specifications	GS1-128 (Application Identifiers)

See Also

- UPC / EAN -- retail product barcodes
- QR Code -- 2D barcode with much higher capacity
- Data Matrix -- 2D barcode for compact industrial marking

Data Matrix

Standard: ISO/IEC 16022:2006 | **Category:** 2D Matrix Barcode / Symbology

Data Matrix is a two-dimensional barcode consisting of black and white cells arranged in a square or rectangular pattern. It is widely used in industrial applications -- electronics component marking (PCBs, ICs), pharmaceutical packaging (serialization), aerospace part tracking, and postal services (USPS, Royal Mail). Data Matrix can encode data in very small sizes (as small as 2-3mm) and is readable even when printed directly on parts via laser etching or ink-jet. The ECC200 variant (current standard) uses Reed-Solomon error correction.

Structure

[Diagram: see online version]

Borders

Border	Pattern	Purpose
Left side	Solid dark modules	Finder pattern (orientation)
Bottom	Solid dark modules	Finder pattern (orientation)
Top	Alternating dark/light	Clock track (module counting)
Right side	Alternating dark/light	Clock track (module counting)

The L-shaped finder pattern (solid left + bottom borders) allows scanners to determine symbol orientation. The alternating clock patterns (top + right borders) help determine the number of rows and columns.

Symbol Sizes (ECC200)

Square Symbols

Size (modules)	Data Capacity (bytes)	ECC Codewords	Regions
10x10	1	5	1
12x12	3	5	1
14x14	6	7	1
16x16	10	10	1
18x18	16	12	1
20x20	22	14	1
22x22	30	18	1
24x24	36	20	1
26x26	44	24	1
32x32	72	36	4
44x44	142	72	4
64x64	300	140	16
88x88	576	280	16
120x120	1,050	510	36
144x144	1,558	620	36

Rectangular Symbols

Size (modules)	Data Capacity (bytes)
8x18	5
8x32	10

12x26	16
12x36	22
16x36	32
16x48	49

Data Capacity

Encoding	Maximum Capacity (144x144)
Numeric	3,116 digits
Alphanumeric	2,335 characters
Byte (ASCII)	1,558 bytes

Encoding Modes

Mode	Name	Efficiency	Description
ASCII	ASCII Encode	1 byte per character	Default; digits encoded in pairs (5.5 bits/digit)
C40	C40 Encode	3 chars per 2 bytes	Uppercase + digits (common in industrial)
Text	Text Encode	3 chars per 2 bytes	Lowercase + digits
X12	ANSI X12 Encode	3 chars per 2 bytes	EDI character set
EDIFACT	EDIFACT Encode	4 chars per 3 bytes	UN/EDIFACT character set
Base 256	Base 256 Encode	1 byte per byte	Binary data

The encoder automatically selects the most efficient mode and can switch modes mid-symbol.

Error Correction (ECC200)

Data Matrix ECC200 uses **Reed-Solomon** error correction:

- Error correction capacity varies by symbol size (see table above)
- Can correct errors equal to half the number of ECC codewords
- Provides both error **detection** and **correction**
- ECC200 replaced older ECC000-140 (convolutional codes, now obsolete)

GS1 Data Matrix

When used with GS1 Application Identifiers, Data Matrix carries a FNC1 character at the start, followed by AI-encoded data:

```
]d2 (01) 00012345678905 (17) 260401 (10) ABC123
```

The `]d2` symbology identifier indicates GS1 Data Matrix.

Common Applications

Application	AI Content	Example
Pharmaceutical serialization	(01) GTIN + (21) Serial + (17) Expiry + (10) Lot	Drug package tracking
Electronics (PCB marking)	Part number, lot, date	Component traceability
Postal	Tracking number, routing	USPS Intelligent Mail barcode
Aerospace	Part number, serial, manufacturer	AS9132 part marking
Medical devices	(01) GTIN + (11) Production date + (21) Serial	UDI (Unique Device Identification)

Direct Part Marking (DPM)

Data Matrix is the preferred symbology for DPM -- permanently marking parts:

Method	Surface	Description
Laser etching	Metal, plastic	Ablates material to create contrast
Dot peen	Metal	Mechanical indentation (rounded modules)
Ink-jet	Any	High-resolution printed code
Chemical etch	Metal	Acid/electrochemical marking

DPM codes often require specialized readers that handle low-contrast, textured, or curved surfaces.

Data Matrix vs QR Code

Feature	Data Matrix	QR Code
Finder pattern	L-shaped border	Three corner squares
Shape	Square or rectangular	Square only
Minimum size	10x10 (1 byte)	21x21 (17 bytes)
Maximum capacity	1,558 bytes	2,953 bytes
Small symbol performance	Better (smaller minimum)	Larger minimum
Industrial DPM	Dominant	Less common
Consumer use	Rare	Dominant
Licensing	Open standard	Open (originally patented, now free)

Standards

Document	Title
ISO/IEC 16022:2006	Data Matrix bar code symbology specification
ISO/IEC 15415	2D print quality (verification)
GS1 DataMatrix Guideline	GS1 Application Identifiers in Data Matrix
AIM DPM	Direct Part Marking quality guideline

See Also

- QR Code -- alternative 2D barcode (larger capacity, consumer-oriented)
- UPC / EAN -- 1D retail barcodes
- Code 128 -- 1D alphanumeric barcode (GS1-128)

QR Code -- Quick Response Code

Standard: ISO/IEC 18004:2015 | **Category:** 2D Matrix Barcode / Symbology

QR Code is a two-dimensional matrix barcode developed by Denso Wave (Toyota subsidiary) in 1994 for tracking automotive parts. It encodes data in a grid of black and white modules (squares) that can be read by cameras from any orientation. QR codes are ubiquitous -- used for URLs, Wi-Fi credentials, payment (Alipay, WeChat Pay, UPI), boarding passes, COVID certificates, contact cards (vCard), and authentication (TOTP setup). They support four encoding modes, four error correction levels, and can store up to 7,089 numeric characters or 4,296 alphanumeric characters.

Structure

[Diagram: see online version]

Module Layout

Component	Location	Purpose
Finder Patterns	Top-left, top-right, bottom-left corners	Detect position and orientation (3 nested squares)
Alignment Patterns	Grid positions (version 2+)	Correct for perspective distortion
Timing Patterns	Row 6 and column 6	Alternating black/white for module coordinate calibration
Format Information	Adjacent to finder patterns	Error correction level + mask pattern (15 bits, BCH-encoded)

Version Information	Near top-right and bottom-left finders (v7+)	Version number (18 bits, Golay-encoded)
Data Region	Remaining modules	Encoded data + Reed-Solomon error correction
Quiet Zone	4-module border around the symbol	Separation from surrounding content

Finder Pattern

Each finder pattern is a 7x7 module structure:

```
■■■■■■■
■     ■
■ ■■■ ■
■ ■■■ ■
■ ■■■ ■
■     ■
■■■■■■■
```

The 1:1:3:1:1 ratio of dark-light-dark-light-dark is unique and detectable from any angle at any rotation.

Encoding Modes

Mode	Mode Indicator	Characters	Bits per Character	Character Set
Numeric	0001	0-9	3.33 (10 bits per 3 digits)	Digits only
Alphanumeric	0010	0-9, A-Z, space, $%*+-./:	5.5 (11 bits per 2 chars)	45 characters
Byte	0100	Any	8	ISO 8859-1 or UTF-8
Kanji	1000	Kanji/Kana	13 (per character)	Shift JIS
ECI	0111	Any	Variable	Extended Channel Interpretation (specify charset)

Data Encoding Structure

[Diagram: see online version]

Multiple mode segments can be concatenated in a single QR code for optimal encoding.

Error Correction Levels

QR codes use **Reed-Solomon** error correction with four levels:

Level	Recovery Capacity	Typical Use
L (Low)	~7% of codewords	Maximum data capacity

M (Medium)	~15% of codewords	General purpose (default for most)
Q (Quartile)	~25% of codewords	Industrial, moderate damage risk
H (High)	~30% of codewords	Harsh environments, design QR codes with logos

Higher error correction allows more damage/obstruction but reduces data capacity.

Versions and Capacity

QR codes come in 40 versions (sizes):

Version	Modules	Numeric (L)	Alphanumeric (L)	Byte (L)	Numeric (H)	Byte (H)
1	21x21	41	25	17	17	7
2	25x25	77	47	32	34	14
5	37x37	202	122	84	88	36
10	57x57	652	395	271	281	119
15	77x77	1,250	758	520	511	217
20	97x97	2,061	1,249	858	858	367
25	117x117	3,057	1,853	1,273	1,273	535
30	137x137	4,158	2,520	1,732	1,732	742
40	177x177	7,089	4,296	2,953	3,057	1,273

Size formula: **4 x version + 17** modules per side.

Masking

After data placement, one of 8 mask patterns is applied (XORed with the data region) to avoid large areas of uniform color that confuse scanners. The pattern that produces the most balanced result is selected:

Mask	Condition (module inverted when true)
0	(row + col) mod 2 == 0
1	row mod 2 == 0
2	col mod 3 == 0
3	(row + col) mod 3 == 0
4	(row/2 + col/3) mod 2 == 0
5	(rowxcol) mod 2 + (rowxcol) mod 3 == 0
6	((rowxcol) mod 2 + (rowxcol) mod 3) mod 2 == 0
7	((row+col) mod 2 + (rowxcol) mod 3) mod 2 == 0

Common Data Types

Prefix	Data Type	Example
`http://` or `https://`	URL	`https://example.com`
`WIFI:`	Wi-Fi credentials	`WIFI:T:WPA;S:MyNetwork;P:password;;`
`BEGIN:VCARD`	Contact card	vCard format
`BEGIN:VEVENT`	Calendar event	iCalendar format
`MATMSG:`	Email	`MATMSG:TO:user@example.com;SUB:Hello;;`
`tel:`	Phone number	`tel:+15551234567`
`smsto:`	SMS	`smsto:+15551234567:Hello`
`otpauth://`	TOTP/HOTP setup	`otpauth://totp/user@example.com?secret=BASE32&issuer=Example`
`geo:`	Location	`geo:37.7749,-122.4194`

QR Code Variants

Variant	Size	Modules	Use Case
QR Code (standard)	Version 1-40	21-177	General purpose
Micro QR	M1-M4	11-17	Small labels (less data, one finder pattern)
iQR	--	Variable	Rectangular shapes, larger capacity
SQRC	--	Standard	Encrypted data (private + public areas)
Frame QR	--	Standard	Canvas area in center for images

Standards

Document	Title
ISO/IEC 18004:2015	QR Code bar code symbology specification
AIM ITS/04-001	International symbology specification -- QR Code

See Also

- UPC / EAN -- 1D barcodes for retail products
- Data Matrix -- alternative 2D barcode (smaller, industrial)

- Code 128 -- high-density 1D barcode

UPC / EAN -- Universal Product Code / European Article Number

Standard: ISO/IEC 15420 (EAN/UPC) / GS1 General Specifications | **Category:** 1D Linear Barcode / Symbology

UPC and EAN are the barcode symbologies printed on virtually every retail product worldwide. UPC-A (12 digits, North America) and EAN-13 (13 digits, international) are the dominant formats. A scanner reads the pattern of bars and spaces to decode a numeric product identifier (GTIN), which is looked up in a database for price and product information. The system was introduced in 1974 and is managed by GS1.

UPC-A Structure

[Diagram: see online version]

Bar Encoding

Each digit is encoded as a pattern of 2 bars and 2 spaces within 7 modules:

Component	Bars/Spaces	Modules
Start guard	bar-space-bar	3
Left digits (6)	7 modules each	42
Middle guard	space-bar-space-bar-space	5
Right digits (6)	7 modules each	42
End guard	bar-space-bar	3
Total		**95 modules**

Left-Side Encoding (Odd Parity)

Digit	Pattern (S=space, B=bar)	Binary
0	SSSBBSB	0001101
1	SSBBSSB	0011001
2	SSBSSBB	0010011
3	SBBBBSB	0111101
4	SBSSSBB	0100011
5	SBBSSAB	0110001

6	SBSBBBB	0101111
7	SBBBSBB	0111011
8	SBBSBBB	0110111
9	SSSBSBB	0001011

Right-side digits use the complement (inverted) patterns.

EAN-13 Structure

EAN-13 is a superset of UPC-A. The 13th digit (first digit) is encoded implicitly by varying the parity patterns of the left-side digits:

[Diagram: see online version]

The first digit (country/region) determines which parity pattern (L or G) each left-side digit uses:

First Digit	Left-Side Pattern (digits 2-7)
0	LLLLLL (= UPC-A)
1	LLGLGG
2	LLGGLG
3	LLGGGL
4	LGLLGG
5	LGGLLG
6	LGGGLL
7	LGLGLG
8	LGLGGL
9	LGGLGL

This encodes the first digit without adding extra bars -- EAN-13 and UPC-A have the same physical width.

Check Digit Calculation

The check digit ensures the barcode was scanned correctly:

1. From right to left (excluding check digit), alternate multiplying by 3 and 1
2. Sum all products
3. Check digit = (10 - (sum mod 10)) mod 10

Example: UPC-A `036000291452`

```
Digits:  0  3  6  0  0  0  2  9  1  4  5  ?
Weights: 1  3  1  3  1  3  1  3  1  3  1
Products: 0  9  6  0  0  0  2  27 1  12 5  = 62
Check: (10 - (62 mod 10)) mod 10 = (10 - 2) = 8
```

But actual check digit for this UPC is 2. (The algorithm applies from the rightmost digit with weight 3 first.)

GTIN (Global Trade Item Number)

The number encoded in a UPC/EAN barcode is a GTIN:

Format	Digits	Usage
GTIN-8	8	EAN-8 (small products)
GTIN-12	12	UPC-A (North America)
GTIN-13	13	EAN-13 (international)
GTIN-14	14	Shipping containers (ITF-14)

GTIN-13 Structure

[Diagram: see online version]

Field	Description
GS1 Prefix	Country/region or special use (00-09 = USA/Canada, 30-37 = France, 45/49 = Japan, 50 = UK, 471 = Taiwan, etc.)
Company Prefix	Assigned to the manufacturer by the local GS1 organization
Item Reference	Assigned by the manufacturer to the specific product
Check Digit	Calculated using the mod-10 algorithm

Special Prefixes

Prefix	Use
00-09	UPC-A (US/Canada)
20-29	Internal/store use (not globally unique)
978/979	ISBN (books)
977	ISSN (periodicals)
471	Taiwan
489	Hong Kong

Barcode Variants

Symbology	Digits	Usage
UPC-A	12	Retail products (North America)
UPC-E	6 (compressed)	Small packages (zero-suppressed UPC-A)
EAN-13	13	Retail products (international)
EAN-8	8	Small products with limited space
UPC-A + 2	12 + 2	Periodicals (issue number)
UPC-A + 5	12 + 5	Books (suggested price)
ITF-14	14	Shipping cartons (Interleaved 2 of 5)

Scanner Operation

Step	Description
1. Illumination	Scanner projects a light line (laser or LED)
2. Reflection	Dark bars absorb light; light spaces reflect
3. Decoding	Photodetector measures transitions; decoder determines digit patterns
4. Validation	Check digit verified
5. Lookup	GTIN sent to POS database for price/description

Scanners can read barcodes **in either direction** -- the guard patterns and parity indicate the orientation.

Standards

Document	Title
ISO/IEC 15420	EAN/UPC bar code symbology specification
GS1 General Specifications	GTIN allocation, barcode standards
GS1 DataBar	Compact barcode for produce, coupons

See Also

- QR Code -- 2D barcode with much higher capacity
- Code 128 -- alphanumeric 1D barcode
- Data Matrix -- 2D barcode for industrial marking

File Sharing & Transfer

Protocols for transferring and sharing files across networks.

7 protocols in this section

FTP -- File Transfer Protocol

Standard: RFC 959 | **Layer:** Application (Layer 7) | **Wireshark filter:** `ftp` or `ftp-data`

FTP is one of the oldest Internet protocols, providing file transfer between a client and server. It is distinctive in using two separate TCP connections -- a control connection for commands and a data connection for file transfers. FTP supports authentication, directory navigation, and both ASCII and binary transfer modes. While largely replaced by SFTP (SSH) and HTTPS for new deployments, FTP remains prevalent in legacy systems, firmware updates, and hosting environments. FTPS adds TLS encryption.

Two-Connection Model

[Diagram: see online version]

Commands

Command	Arguments	Description
USER	username	Specify username
PASS	password	Specify password
CWD	path	Change working directory
CDUP	--	Change to parent directory
PWD	--	Print working directory
LIST	[path]	List directory contents (over data connection)
NLST	[path]	Name list only
RETR	filename	Retrieve (download) a file
STOR	filename	Store (upload) a file
DELE	filename	Delete a file
MKD	dirname	Create a directory
RMD	dirname	Remove a directory
RNFR	oldname	Rename from
RNTO	newname	Rename to
TYPE	A/I	Set transfer type: A = ASCII, I = Image (binary)
PASV	--	Enter passive mode (server opens data port)
EPSV	--	Extended passive mode (IPv6 compatible)

PORT	h1,h2,h3,h4,p1,p2	Active mode (client opens data port)
EPRT	--	Extended active mode (IPv6 compatible)
SIZE	filename	Get file size
MDTM	filename	Get file modification time
REST	offset	Restart transfer at byte offset
ABOR	--	Abort current transfer
QUIT	--	Close the session
AUTH	TLS	Upgrade to TLS (FTPS explicit)
PBSZ	0	Protection buffer size (required for TLS)
PROT	P/C	Data channel protection: P = Private (TLS), C = Clear
FEAT	--	List supported extensions

Response Codes

Range	Category	Examples
1xx	Positive Preliminary	150 Opening data connection
2xx	Positive Completion	200 OK, 220 Service ready, 226 Transfer complete, 230 User logged in, 250 Action OK
3xx	Positive Intermediate	331 Username OK, need password; 350 Pending further info
4xx	Transient Negative	421 Service not available, 425 Can't open data connection, 450 File unavailable
5xx	Permanent Negative	500 Syntax error, 530 Not logged in, 550 File not found

Session Flow

[Diagram: see online version]

Data Connection Modes

Active Mode (PORT)

The client tells the server which port to connect to:

[Diagram: see online version]

Active mode often fails with NATs and firewalls because the server initiates the data connection inbound to the client.

Passive Mode (PASV)

The server opens a port and the client connects to it:

[Diagram: see online version]

Passive mode works through NATs and is the default in modern clients.

FTPS (FTP over TLS)

Mode	Description
Explicit FTPS	Client sends `AUTH TLS` to upgrade (port 21)
Implicit FTPS	TLS from connection start (port 990) -- deprecated

FTP vs Alternatives

Feature	FTP	SFTP (SSH)	HTTPS
Port	21 + data ports	22	443
Connections	2 (control + data)	1	1
Encryption	None (or FTPS)	Always (SSH)	Always (TLS)
NAT traversal	Problematic	Simple	Simple
Resume	REST command	Yes	Range headers
Directory listing	LIST (varies)	Standardized	No (application-dependent)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 959	File Transfer Protocol
RFC 4217	Securing FTP with TLS (FTPS)
RFC 2228	FTP Security Extensions
RFC 2428	FTP Extensions for IPv6 and NATs (EPSV, EPRT)
RFC 3659	Extensions to FTP (SIZE, MDTM, REST, MLST)

See Also

- TCP
- TLS -- used by FTPS
- SSH -- SFTP is the modern encrypted alternative (different protocol)

NetBIOS -- Network Basic Input/Output System

Standard: RFC 1001 / RFC 1002 | **Layer:** Session (Layer 5) | **Wireshark filter:** `nbns` or `nbss` or `nbds`

NetBIOS is a session-layer API and protocol suite originally developed by IBM in 1983 for LAN communication. It provides name registration/resolution, session establishment, and datagram services for applications on a local network. While the original NetBIOS ran over NetBEUI (a non-routable protocol), the dominant implementation today is **NetBIOS over TCP/IP (NBT)**, which encapsulates NetBIOS services in TCP and UDP. NetBIOS is the foundation of Windows file and printer sharing -- SMB historically depended on NetBIOS for name resolution and session transport, though modern SMB can run directly over TCP without it.

Three Services

Service	Transport	Port	Description
Name Service (NBNS)	UDP	137	Name registration, resolution, and release
Datagram Service (NBDS)	UDP	138	Connectionless messaging
Session Service (NBSS)	TCP	139	Connection-oriented data transfer

Name Service (NBNS -- Port 137)

NetBIOS names are 16 bytes: 15 characters of name + 1 byte suffix indicating the service type.

Name Header

[Diagram: see online version]

The format is based on DNS (RFC 1002 specifies NetBIOS name encoding within DNS-format messages).

Name Operations

Opcode	Operation	Description
0	Query	Resolve a NetBIOS name to an IP address

5	Registration	Register a name on the network
6	Release	Release a registered name
7	WACK	Wait for Acknowledgment (name registration pending)
8	Refresh	Refresh a name registration

Name Suffixes (16th byte)

Suffix	Type	Description
0x00	Unique	Workstation service
0x03	Unique	Messenger service (Windows messaging)
0x20	Unique	File server service (SMB)
0x1B	Unique	Domain Master Browser
0x1C	Group	Domain Controllers
0x1D	Unique	Master Browser
0x1E	Group	Browser Service Elections

Name Resolution Methods

Method	Description
B-node (Broadcast)	Broadcast query on local subnet
P-node (Point-to-point)	Query WINS server directly
M-node (Mixed)	Broadcast first, then WINS
H-node (Hybrid)	WINS first, then broadcast (Windows default)

Session Service (NBSS -- Port 139)

Provides reliable, connection-oriented transport for SMB and other NetBIOS applications:

Session Packet

[Diagram: see online version]

Session Message Types

Type	Name	Description
0x00	Session Message	User data (typically SMB)
0x81	Session Request	Initiate a session (called/calling names)

0x82	Positive Response	Session request accepted
0x83	Negative Response	Session request rejected
0x84	Retarget Response	Redirect to different IP/port
0x85	Keep Alive	Session keepalive

Session Establishment

[Diagram: see online version]

Datagram Service (NBDS -- Port 138)

Connectionless messaging for broadcasts and group communications:

Datagram Header

[Diagram: see online version]

Type	Name	Description
0x10	Direct Unique	Datagram to a specific name
0x11	Direct Group	Datagram to a group name
0x12	Broadcast	Datagram to all nodes

NetBIOS vs Direct SMB

Feature	NetBIOS over TCP (NBT)	Direct SMB
Port	TCP 139 / UDP 137-138	TCP 445
Name resolution	NBNS (broadcast or WINS)	DNS
Required for SMB	Windows 9x/NT era	Windows 2000+ can use direct
Overhead	NetBIOS session framing	None (SMB directly over TCP)
Modern status	Legacy, often disabled	Preferred

Encapsulation

[Diagram: see online version]

Standards

Document	Title

RFC 1001	Protocol Standard for a NetBIOS Service on a TCP/UDP Transport: Concepts and Methods
RFC 1002	Protocol Standard for a NetBIOS Service on a TCP/UDP Transport: Detailed Specifications

See Also

- SMB -- file sharing protocol that historically runs over NetBIOS
- DNS -- modern replacement for NetBIOS name resolution
- TCP
- UDP

NFS -- Network File System

Standard: RFC 7530 (NFSv4) / RFC 1813 (NFSv3) | **Layer:** Application (Layer 7) | **Wireshark filter:** `nfs`

NFS is a distributed file system protocol that allows a client to access files over a network as if they were on local storage. Developed by Sun Microsystems in 1984, NFS is the standard file sharing protocol in Unix/Linux environments -- used for home directories, shared storage, data centers, and HPC clusters. NFSv4 (current) consolidated the protocol into a single TCP port (2049) with built-in security, eliminating the auxiliary protocols (mountd, portmapper) that earlier versions required.

NFSv4 Compound Request

NFSv4 uses a single RPC program on TCP port 2049. Requests are **compound** -- multiple operations in a single RPC call:

[Diagram: see online version]

This is equivalent to `open("/home/alice/report.txt")` + `read()` in a single round trip.

NFSv4 Operations

Op	Name	Description
0	ACCESS	Check access permissions
1	CLOSE	Close a file
3	COMMIT	Commit cached writes to stable storage
4	CREATE	Create a non-regular file (directory, symlink, etc.)
6	GETATTR	Get file attributes

7	GETFH	Get current filehandle
9	LINK	Create a hard link
10	LOCK	Byte-range lock
11	LOCKT	Test a byte-range lock
12	LOCKU	Unlock a byte-range lock
15	LOOKUP	Look up a filename in a directory
18	OPEN	Open a file (state-based, with share modes)
22	PUTROOTFH	Set current FH to root of export
23	READ	Read file data
25	READDIR	Read directory entries
26	READLINK	Read a symbolic link
27	REMOVE	Remove a file or directory
28	RENAME	Rename a file
32	SETATTR	Set file attributes
38	WRITE	Write file data
40	DELEGRETURN	Return a delegation

File Access Flow

[Diagram: see online version]

Delegations

NFSv4 delegations allow the server to grant a client exclusive caching rights, dramatically improving performance for files accessed by a single client:

Delegation	Description
READ	Client can cache reads without revalidating; server recalls if another client reads/writes
WRITE	Client can cache reads AND writes; server recalls if another client accesses the file

Delegation Recall

[Diagram: see online version]

NFSv3 vs NFSv4

Feature	NFSv3	NFSv4
Transport	UDP or TCP	TCP only
Port	2049 + mountd + portmapper	2049 only
File path	Opaque filehandles	Named path navigation (LOOKUP)
Locking	Separate NLM protocol	Integrated (LOCK/LOCKU)
Mounting	Separate MOUNT protocol	Virtual filesystem + pseudo-root
State	Stateless	Stateful (open, lock, delegation state)
Security	AUTH_SYS (UID/GID), Kerberos optional	Kerberos mandatory support (RPCSEC_GSS)
ACLs	POSIX only	NFSv4 ACLs (richer, Windows-compatible)
Compound ops	No (one op per RPC)	Yes (multiple ops per RPC)
Replication	No	Optional (referrals, migration)

Security (RPCSEC_GSS)

Security Flavor	Name	Description
AUTH_NONE	None	No authentication
AUTH_SYS	System	UID/GID trust (legacy, no real security)
RPCSEC_GSS krb5	Kerberos auth	Authentication only
RPCSEC_GSS krb5i	Kerberos integrity	Auth + integrity (HMAC on every RPC)
RPCSEC_GSS krb5p	Kerberos privacy	Auth + integrity + encryption

pNFS (Parallel NFS -- NFSv4.1+)

pNFS separates metadata from data access, allowing clients to read/write directly to storage devices for massive parallelism:

[Diagram: see online version]

Common Mount Options

Option	Description
`vers=4`	NFSv4
`sec=krb5p`	Kerberos with encryption
`rsize=1048576`	Read block size (1MB)
`wsize=1048576`	Write block size (1MB)
`hard`	Retry indefinitely on timeout

`soft`	Return error after retries
`intr`	Allow signals to interrupt
`noatime`	Don't update access times

Encapsulation

[Diagram: see online version]

NFSv3 also uses: portmapper (111), mountd (variable), nlockmgr (variable), statd (variable).

Standards

Document	Title
RFC 7530	NFS Version 4 Protocol
RFC 8881	NFS Version 4.1 (sessions, pNFS)
RFC 7862	NFS Version 4.2 (server-side copy, space reservation)
RFC 1813	NFS Version 3 Protocol
RFC 7861	RPCSEC_GSS Version 2

See Also

- SMB -- Windows file sharing (alternative to NFS)
- Kerberos -- authentication for NFSv4 security
- TCP

SCP -- Secure Copy Protocol

Standard: No formal RFC (historically based on BSD rcp over SSH) | **Layer:** Application (Layer 7) | **Wireshark filter:** `ssh` (SCP runs inside an encrypted SSH channel)

SCP is a file transfer protocol that copies files between hosts over an SSH connection. It uses the SSH transport for encryption, authentication, and integrity -- providing the same security as an interactive SSH session. SCP was the original SSH-based file transfer mechanism, derived from the Berkeley `rcp` command. It is simple and fast for straightforward file copies but lacks features like directory listings, resume, and remote file management. OpenSSH deprecated SCP's legacy protocol in favor of SFTP internally, though the `scp` command now uses SFTP under the hood by default (since OpenSSH 9.0).

How SCP Works

SCP operates by executing a remote `scp` process via SSH and exchanging files over the SSH channel using a simple text-based control protocol:

[Diagram: see online version]

Control Messages

SCP uses single-character control codes and text metadata:

Source to Sink (sending)

Message	Format	Description
File header	`C<mode> <size> <filename>\n`	Regular file with permissions, size in bytes, and name
Directory start	`D<mode> 0 <dirname>\n`	Begin a directory (recursive copy)
Directory end	`E\n`	End of directory
Timestamp	`T<mtime> 0 <atime> 0\n`	Preserve modification/access times (-p flag)

Response Codes

Code	Meaning
0x00	OK (success, continue)
0x01	Warning (non-fatal error, message follows)
0x02	Fatal error (message follows, abort)

Example: Copy a File

```
C: C0644 1048576 backup.tar.gz\n      (file: mode 644, 1MB, name)
S: \0                                   (OK)
C: [1048576 bytes of file data]
C: \0                                   (done)
S: \0                                   (OK)
```

Example: Copy a Directory (-r)

```
C: D0755 0 mydir\n                    (enter directory)
S: \0
C: C0644 100 file1.txt\n             (file in directory)
S: \0
C: [100 bytes]
C: \0
S: \0
C: C0644 200 file2.txt\n             (another file)
S: \0
C: [200 bytes]
C: \0
S: \0
C: E\n                                (leave directory)
S: \0
```

SCP Modes

Flag	Mode	Description
`-t`	Sink (to)	Receive files -- server-side when copying TO remote
`-f`	Source (from)	Send files -- server-side when copying FROM remote

The local `scp` client executes the remote `scp` with `-t` or `-f` via SSH exec.

SCP vs SFTP

Feature	SCP	SFTP
Protocol	Text-based control + raw data over SSH exec	Binary protocol over SSH subsystem
Resume	No	Yes
Directory listing	No	Yes
Rename/delete	No	Yes
Symbolic links	Limited	Full support
Speed	Fast (minimal overhead)	Slightly slower (more protocol framing)
Firewall traversal	SSH only	SSH only
Modern OpenSSH	Uses SFTP internally (since 9.0)	Native
Formal standard	No	RFC 4253 (SSH) + draft-ietf-secsh-filexfer

Security

SCP inherits all of SSH's security properties:

Feature	Provided By

Encryption	SSH transport (AES-GCM, ChaCha20-Poly1305)
Authentication	SSH (public key, password, Kerberos)
Integrity	SSH MAC (or AEAD)
Host verification	SSH host key fingerprint

Encapsulation

[Diagram: see online version]

SCP is not a separate network protocol -- it is a program that runs inside an SSH session. The wire format is entirely SSH.

Standards

SCP has no formal standard. Its behavior is defined by the OpenSSH implementation:

Document	Title
OpenSSH source	Reference implementation
RFC 4251-4254	SSH Protocol Suite (transport for SCP)

See Also

- SSH -- the transport protocol SCP runs over
- FTP -- legacy file transfer (unencrypted)
- TCP

SFTP -- SSH File Transfer Protocol

Standard: draft-ietf-secsh-filexfer | **Layer:** Application (Layer 7) | **Wireshark filter:** `sftp`

SFTP is a binary file transfer protocol that runs as a subsystem of SSH. Despite the name, it is not FTP over SSH or FTP over SSL -- it is an entirely different protocol designed from the ground up for secure file operations. SFTP provides file upload, download, directory listing, rename, delete, and attribute management, all within a single encrypted SSH channel on port 22. No additional ports are needed. Version 3 of the protocol (from OpenSSH) is the most widely deployed, though the specification was never finalized as an RFC -- it remained an IETF draft through version 13.

Packet Structure

Every SFTP packet has a simple framing format inside the SSH channel:

[Diagram: see online version]

The `length` field covers everything after itself (type + request-id + data). The `request-id` is present in all packets except SSH_FXP_INIT and SSH_FXP_VERSION, and is used to match requests with responses asynchronously.

Key Fields

Field	Size	Description
length	4 bytes	Total length of the packet payload (excluding the length field itself)
type	1 byte	Packet type identifier
request-id	4 bytes	Client-chosen ID to correlate requests and responses
data	variable	Type-specific fields and payload

Packet Types

Client Requests

Type	Value	Description
SSH_FXP_INIT	1	Initialize protocol, send client version
SSH_FXP_OPEN	3	Open or create a file (returns handle)
SSH_FXP_CLOSE	4	Close a file or directory handle
SSH_FXP_READ	5	Read data from an open file
SSH_FXP_WRITE	6	Write data to an open file
SSH_FXP_LSTAT	7	Get attributes for a path (don't follow symlinks)
SSH_FXP_FSTAT	8	Get attributes for an open handle
SSH_FXP_SETSTAT	9	Set attributes on a path
SSH_FXP_FSETSTAT	10	Set attributes on an open handle
SSH_FXP_OPENDIR	11	Open a directory for reading (returns handle)
SSH_FXP_READDIR	12	Read directory entries from a handle
SSH_FXP_REMOVE	13	Delete a file
SSH_FXP_MKDIR	14	Create a directory
SSH_FXP_RMDIR	15	Remove a directory
SSH_FXP_REALPATH	16	Canonicalize a path (resolve `.`, `..`, symlinks)

SSH_FXP_STAT	17	Get attributes for a path (follow symlinks)
SSH_FXP_RENAME	18	Rename a file or directory
SSH_FXP_READLINK	19	Read the target of a symbolic link
SSH_FXP_SYMLINK	20	Create a symbolic link

Server Responses

Type	Value	Description
SSH_FXP_VERSION	2	Server version and supported extensions
SSH_FXP_STATUS	101	Status/error code response
SSH_FXP_HANDLE	102	File or directory handle (response to OPEN/OPENDIR)
SSH_FXP_DATA	103	File data (response to READ)
SSH_FXP_NAME	104	File name(s) and attributes (response to READDIR/REALPATH)
SSH_FXP_ATTRS	105	File attributes (response to STAT/FSTAT)

Extended

Type	Value	Description
SSH_FXP_EXTENDED	200	Vendor-defined extended request
SSH_FXP_EXTENDED_REPLY	201	Response to extended request

Status Codes

The SSH_FXP_STATUS response carries a status code:

Code	Name	Description
0	SSH_FX_OK	Success
1	SSH_FX_EOF	End of file reached
2	SSH_FX_NO_SUCH_FILE	File does not exist
3	SSH_FX_PERMISSION_DENIED	Insufficient permissions
4	SSH_FX_FAILURE	Generic failure
5	SSH_FX_BAD_MESSAGE	Malformed packet
6	SSH_FX_NO_CONNECTION	No connection (not used in practice)
7	SSH_FX_CONNECTION_LOST	Connection lost

8	SSH_FX_OP_UNSUPPORTED	Operation not supported by server

File Attributes

SFTP transfers file attributes as a structure with a flags bitmask indicating which fields are present:

Flag	Value	Fields Present
SSH_FILEXFER_ATTR_SIZE	0x00000001	size (uint64)
SSH_FILEXFER_ATTR_UIDGID	0x00000002	uid, gid (uint32 each)
SSH_FILEXFER_ATTR_PERMISSIONS	0x00000004	permissions (uint32, POSIX mode bits)
SSH_FILEXFER_ATTR_ACMODTIME	0x00000008	atime, mtime (uint32 each, Unix epoch)
SSH_FILEXFER_ATTR_EXTENDED	0x80000000	extended_count + extended name-value pairs

ATTRS Structure

[Diagram: see online version]

File Download Flow

[Diagram: see online version]

Clients typically send multiple READ requests in parallel (pipelining) to maximize throughput over high-latency links.

File Upload Flow

[Diagram: see online version]

Open Flags (pflags)

Used in SSH_FXP_OPEN to specify how a file should be opened:

Flag	Value	Description
SSH_FXF_READ	0x00000001	Open for reading
SSH_FXF_WRITE	0x00000002	Open for writing
SSH_FXF_APPEND	0x00000004	Append to end of file
SSH_FXF_CREAT	0x00000008	Create file if it does not exist
SSH_FXF_TRUNC	0x00000010	Truncate file to zero length if it exists
SSH_FXF_EXCL	0x00000020	Fail if file already exists (with CREAT)

SFTP vs SCP vs FTP vs FTPS

Feature	SFTP	SCP	FTP	FTPS
Transport	SSH subsystem (port 22)	SSH exec (port 22)	TCP ports 21 + data	TCP ports 21 + data (TLS)
Connections	1	1	2 (control + data)	2 (control + data)
Encryption	Always (SSH)	Always (SSH)	None	TLS
Resume/seek	Yes	No	REST command	REST command
Directory listing	Yes (READDIR)	No	LIST (inconsistent format)	LIST (inconsistent format)
Rename/delete	Yes	No	Yes	Yes
Random access	Yes (offset-based READ/WRITE)	No	No	No
NAT traversal	Simple (single port)	Simple (single port)	Problematic (multiple ports)	Problematic (multiple ports)
Formal standard	IETF draft (never RFC)	None	RFC 959	RFC 4217
Protocol type	Binary	Text control + raw data	Text	Text + TLS

Security

SFTP inherits all security from SSH:

Feature	Provided By
Encryption	SSH transport (AES-GCM, ChaCha20-Poly1305)
Authentication	SSH (public key, password, Kerberos, certificate)
Integrity	SSH MAC or AEAD
Host verification	SSH host key fingerprint
Key exchange	SSH (Diffie-Hellman, ECDH)

Encapsulation

[Diagram: see online version]

SFTP is not a separate network protocol visible on the wire -- all traffic appears as SSH. The SFTP binary framing exists entirely within an SSH channel. The subsystem is started with `ssh -s sftp` or by requesting the `sftp` subsystem on an existing SSH session.

Standards

Document	Title

draft-ietf-secsh-filexfer-02	SSH File Transfer Protocol (version 3 -- most widely deployed)
draft-ietf-secsh-filexfer-13	SSH File Transfer Protocol (version 6 -- final draft, limited adoption)
RFC 4253	SSH Transport Layer Protocol
RFC 4254	SSH Connection Protocol (subsystem channels)
OpenSSH source	Reference implementation (sftp-server, sftp-client)

See Also

- SCP -- simpler SSH-based file copy (deprecated in favor of SFTP)
- FTP -- legacy unencrypted file transfer
- SSH -- the transport protocol SFTP runs over

SMB (Server Message Block) / CIFS

Standard: MS-SMB2 (Microsoft) | **Layer:** Application (Layer 7) | **Wireshark filter:** `smb` or `smb2`

SMB is the network file sharing protocol used by Windows for accessing files, printers, and named pipes over a network. It enables "Network Drives," "\\server\share" paths, and Windows file/printer sharing. Originally developed by IBM and extended by Microsoft, SMB has evolved through multiple versions -- from SMB1 (1983) through SMB 3.1.1 (Windows 10+) with encryption and multi-channel support. Samba is the open-source implementation that brings SMB to Linux, macOS, and other Unix systems. The older name "CIFS" (Common Internet File System) refers to the SMB1 dialect.

SMB2/3 Header

[Diagram: see online version]

The SMB2/3 header is 64 bytes.

Key Fields

Field	Size	Description
Protocol ID	4 bytes	`0xFE534D42` (0xFE 'S' 'M' 'B') for SMB2/3
Header Length	16 bits	Always 64 for SMB2/3
Command	16 bits	Operation code
Credits	16 bits	Flow control (credit-based)

Flags	32 bits	Response flag, async, signed, encrypted, etc.
Message ID	64 bits	Unique request identifier
Tree ID	32 bits	Identifies the share connection
Session ID	64 bits	Identifies the authenticated session
Signature	128 bits	AES-CMAC or AES-GMAC integrity signature

Commands (SMB2/3)

Code	Command	Description
0x0000	NEGOTIATE	Protocol version and capability negotiation
0x0001	SESSION_SETUP	Authenticate (NTLM or Kerberos via SPNEGO)
0x0002	LOGOFF	End the session
0x0003	TREE_CONNECT	Connect to a share (\\server\share)
0x0004	TREE_DISCONNECT	Disconnect from a share
0x0005	CREATE	Open or create a file/directory/pipe
0x0006	CLOSE	Close a file handle
0x0007	FLUSH	Flush cached writes to disk
0x0008	READ	Read data from a file
0x0009	WRITE	Write data to a file
0x000A	LOCK	Lock/unlock byte ranges
0x000B	IOCTL	Device control / FSCTL operations
0x000C	CANCEL	Cancel a pending request
0x000D	ECHO	Keepalive
0x000E	QUERY_DIRECTORY	List directory contents
0x000F	CHANGE_NOTIFY	Watch for filesystem changes
0x0010	QUERY_INFO	Get file/FS/security attributes
0x0011	SET_INFO	Set file/FS/security attributes
0x0012	OPLOCK_BREAK	Opportunistic lock break notification

Session Establishment

[Diagram: see online version]

SMB Versions

Version	Windows	Year	Key Features
SMB1 / CIFS	Windows NT	1996	Original; insecure, slow -- **deprecated**
SMB 2.0	Vista	2006	Reduced chattiness, compound requests, large reads
SMB 2.1	Windows 7	2009	Oplock leasing, large MTU support
SMB 3.0	Windows 8	2012	Encryption, multichannel, RDMA, transparent failover
SMB 3.0.2	Windows 8.1	2013	Disable SMB1 support
SMB 3.1.1	Windows 10	2015	Pre-auth integrity (SHA-512), AES-128-GCM encryption, cluster reconnect

Security

Feature	SMB1	SMB 2.x	SMB 3.0+
Authentication	NTLM (weak)	NTLM or Kerberos	NTLM or Kerberos
Signing	Optional (MD5-based)	Optional (HMAC-SHA-256)	Required in AD by default (AES-CMAC)
Encryption	None	None	AES-128-CCM or AES-128-GCM (per-share or global)
Pre-auth integrity	None	None	SHA-512 hash of negotiate/session (SMB 3.1.1)

SMB Signing

Message signing prevents tampering and MITM. SMB 3.1.1 uses AES-CMAC or AES-GMAC computed over the entire message with a session-derived key.

SMB Encryption

SMB 3.0+ can encrypt all traffic on a per-share or per-server basis:

- **AES-128-CCM** (SMB 3.0)
- **AES-128-GCM** (SMB 3.1.1, preferred)
- **AES-256-CCM / AES-256-GCM** (SMB 3.1.1 with negotiation)

Opportunistic Locks (Oplocks)

Oplocks allow clients to cache file data and metadata locally for performance:

Lock Type	Description
Exclusive	No other opens; client can read/write/cache freely
Batch	Like exclusive but allows close/reopen caching
Level II	Shared read caching (multiple readers)
Lease (SMB 2.1+)	Read, Write, Handle leases -- more granular

Samba

Samba is the open-source SMB implementation for Unix/Linux systems:

- File and print serving to Windows clients
- Active Directory Domain Controller
- AD domain member (authentication via winbind)
- SMB client tools (`smbclient`, mount.cifs)

Encapsulation

[Diagram: see online version]

Modern Windows uses TCP 445 exclusively. Port 139 (NetBIOS) is legacy. SMB over QUIC (Windows 11 / Server 2022) enables secure access without VPN.

Standards

Document	Title
MS-SMB2	SMB Protocol Versions 2 and 3
MS-SMB	SMB Protocol Version 1 (legacy)
MS-SPNG	SPNEGO (authentication negotiation)
Samba	Open-source SMB implementation

See Also

- NetBIOS -- legacy name service and session transport for SMB
- TCP
- QUIC -- SMB over QUIC in modern Windows
- LDAP -- Active Directory (often used alongside SMB)
- DNS -- modern name resolution for SMB shares

TFTP -- Trivial File Transfer Protocol

Standard: RFC 1350 | **Layer:** Application (Layer 7) | **Wireshark filter:** `tftp`

TFTP is a minimal file transfer protocol that uses UDP with no authentication, no directory listing, and no encryption. Its simplicity makes it ideal for bootstrapping -- PXE network booting, firmware updates on embedded devices, and provisioning IP phones and network equipment. TFTP fits in a very small code footprint, making it implementable on devices with minimal resources. It uses a simple stop-and-wait acknowledgment scheme for reliability over UDP.

Packet Types

Opcode	Name	Direction	Description
1	RRQ	Client -> Server	Read Request (download a file)
2	WRQ	Client -> Server	Write Request (upload a file)
3	DATA	Either	Data block (512 bytes, or less for last block)
4	ACK	Either	Acknowledge a data block
5	ERROR	Either	Error message
6	OACK	Server -> Client	Option Acknowledgment (RFC 2347)

RRQ / WRQ Format

[Diagram: see online version]

DATA Format

[Diagram: see online version]

ACK Format

[Diagram: see online version]

ERROR Format

[Diagram: see online version]

Read Transfer (Download)

[Diagram: see online version]

The last DATA packet has fewer than 512 bytes (or exactly 0), signaling end of file.

Error Codes

Code	Name	Description
0	Not defined	See error message
1	File not found	Requested file does not exist
2	Access violation	Permission denied
3	Disk full	No space for write
4	Illegal operation	Unknown opcode or bad request
5	Unknown transfer ID	Wrong source port (session mismatch)
6	File already exists	File exists (for write)
7	No such user	User authentication failed (rarely used)
8	Option negotiation failed	Requested option rejected

Transfer Modes

Mode	Description
`netascii`	ASCII text with CR/LF conversion
`octet`	Raw binary (most common)
`mail`	Email delivery (obsolete)

Options (RFC 2347)

Option	RFC	Description
blksize	RFC 2348	Block size (8-65464 bytes, default 512)
tsize	RFC 2349	Transfer size (total file size in bytes)
timeout	RFC 2349	Retransmission timeout in seconds (1-255)
windowsize	RFC 7440	Number of blocks sent before waiting for ACK

Option Negotiation

[Diagram: see online version]

Common Uses

Use Case	Description
PXE Boot	BIOS/UEFI downloads bootloader via TFTP after DHCP
Cisco IOS	Firmware upload/download on routers/switches
VoIP Phones	Configuration file provisioning
Firmware Updates	Embedded device firmware loading
Network Equipment	Configuration backup/restore

PXE Boot Flow

[Diagram: see online version]

TFTP vs FTP vs HTTP

Feature	TFTP	FTP	HTTP
Transport	UDP	TCP	TCP
Port	69	21 + data	80/443
Authentication	None	Username/password	Optional
Directory listing	No	Yes	No (application-dependent)
File size limit	~32MB (16-bit block#) or larger with options	None	None
Encryption	None	FTPS	HTTPS
Code footprint	Tiny (~1KB)	Large	Large
Use case	Bootstrapping, firmware	General file transfer	Web, APIs

Encapsulation

[Diagram: see online version]

The initial request goes to port 69. The server responds from a random ephemeral port, and the rest of the transfer uses that port.

Standards

Document	Title
RFC 1350	The TFTP Protocol (Revision 2)
RFC 2347	TFTP Option Extension

RFC 2348	TFTP Blocksize Option
RFC 2349	TFTP Timeout and Transfer Size Options
RFC 7440	TFTP Windowsize Option

See Also

- FTP -- full-featured file transfer alternative
- DHCP -- works with TFTP for PXE boot (next-server option)
- UDP -- TFTP transport

Healthcare

Communication protocols used in medical and healthcare IT systems.

2 protocols in this section

DICOM -- Digital Imaging and Communications in Medicine

Standard: NEMA PS3 / ISO 12052 | **Layer:** Application (Layer 7) | **Wireshark filter:** `dicom`

DICOM is the universal standard for storing, transmitting, and displaying medical images and associated data. It defines both a file format and a network protocol that enables interoperability between imaging devices (CT, MRI, X-ray, ultrasound), PACS servers, workstations, and clinical systems. The DICOM network protocol uses an Upper Layer Protocol based on ISO 8822 (OSI Association) over TCP, typically on port 104 or 11112. Communication follows a Service Class User (SCU) / Service Class Provider (SCP) model where the SCU initiates requests and the SCP fulfills them.

PDU (Protocol Data Unit) Structure

Every DICOM network message is wrapped in a PDU:

[Diagram: see online version]

The PDU header is 6 bytes: 1 byte type, 1 byte reserved, and 4 bytes for the length of the remaining data (big-endian).

Key Fields

Field	Size	Description
PDU Type	1 byte	Identifies the PDU type (0x01-0x07)
Reserved	1 byte	Always 0x00
PDU Length	4 bytes	Length of the PDU data field (excludes the 6-byte header)
PDU Data	Variable	Content depends on PDU type

PDU Types

Type	Hex	Name	Direction	Description
0x01	01	A-ASSOCIATE-RQ	SCU -> SCP	Association request with proposed presentation contexts
0x02	02	A-ASSOCIATE-AC	SCP -> SCU	Association accepted with negotiated contexts
0x03	03	A-ASSOCIATE-RJ	SCP -> SCU	Association rejected with reason code

0x04	04	P-DATA-TF	Either	Transfer of command and data fragments
0x05	05	A-RELEASE-RQ	Either	Request graceful release of association
0x06	06	A-RELEASE-RP	Either	Confirm release of association
0x07	07	A-ABORT	Either	Abrupt termination of association

Association Negotiation

Before any image data is exchanged, the SCU and SCP must establish an Association. The SCU proposes one or more Presentation Contexts, each pairing an Abstract Syntax (what service/data) with one or more Transfer Syntaxes (how the data is encoded).

[Diagram: see online version]

A-ASSOCIATE-RQ Contents

Item	Description
Application Context	Always "1.2.840.10008.3.1.1.1" (DICOM Application Context)
Called AE Title	Application Entity title of the SCP (up to 16 chars)
Calling AE Title	Application Entity title of the SCU (up to 16 chars)
Presentation Contexts	List of proposed Abstract Syntax + Transfer Syntax pairs
User Information	Max PDU length, implementation class UID, implementation version

Common Transfer Syntaxes

UID	Name	Description
1.2.840.10008.1.2	Implicit VR Little Endian	Default transfer syntax (required)
1.2.840.10008.1.2.1	Explicit VR Little Endian	Most commonly used
1.2.840.10008.1.2.2	Explicit VR Big Endian	Retired in 2024
1.2.840.10008.1.2.4.50	JPEG Baseline	Lossy JPEG compression
1.2.840.10008.1.2.4.70	JPEG Lossless	Lossless JPEG compression
1.2.840.10008.1.2.4.80	JPEG-LS Lossless	JPEG-LS lossless compression
1.2.840.10008.1.2.4.90	JPEG 2000 Lossless	JPEG 2000 lossless compression
1.2.840.10008.1.2.4.91	JPEG 2000	JPEG 2000 lossy compression

P-DATA-TF and Presentation Data Values

The P-DATA-TF PDU carries the actual DIMSE commands and datasets, wrapped in Presentation Data Value (PDV) items:

[Diagram: see online version]

Field	Size	Description
PDV Item Length	4 bytes	Length of this PDV item (includes context ID and message header)
Presentation Context ID	1 byte	Odd number (1, 3, 5...) referencing a negotiated context
Message Control Header	1 byte	Bit 0: Command (1) or Dataset (0); Bit 1: Last fragment (1) or not (0)
Fragment	Variable	DIMSE command elements or dataset fragment

DIMSE Services

DICOM Message Service Element (DIMSE) defines the operations available over an association. They are split into composite (C-) and normalized (N-) services:

Composite Services (C-DIMSE)

Service	Description	SCU Action	SCP Action
C-STORE	Store an image/object	Sends an instance	Stores the instance
C-FIND	Query for matching objects	Sends query keys	Returns matching results
C-MOVE	Retrieve objects (via sub-association)	Requests retrieval	Sends via C-STORE to destination
C-GET	Retrieve objects (on same association)	Requests retrieval	Sends via C-STORE back to SCU
C-ECHO	Verify connectivity	Sends echo request	Returns echo response

Normalized Services (N-DIMSE)

Service	Description
N-CREATE	Create a managed SOP instance
N-SET	Modify attributes of a managed instance
N-GET	Retrieve attributes of a managed instance
N-ACTION	Request an action on a managed instance
N-EVENT-REPORT	Report an event from SCP to SCU
N-DELETE	Delete a managed SOP instance

C-STORE Flow

The most common DICOM operation is C-STORE, used to send images from a modality to a PACS:

[Diagram: see online version]

Common SOP Classes

SOP Class	UID	Description
CT Image Storage	1.2.840.10008.5.1.4.1.1.2	Computed Tomography images
MR Image Storage	1.2.840.10008.5.1.4.1.1.4	Magnetic Resonance images
CR Image Storage	1.2.840.10008.5.1.4.1.1.1	Computed Radiography images
US Image Storage	1.2.840.10008.5.1.4.1.1.6.1	Ultrasound images
DX Image Storage	1.2.840.10008.5.1.4.1.1.1.1	Digital X-Ray images
SC Image Storage	1.2.840.10008.5.1.4.1.1.7	Secondary Capture (screenshots, scanned docs)
RT Structure Set	1.2.840.10008.5.1.4.1.1.481.3	Radiation therapy structure sets
Verification	1.2.840.10008.1.1	C-ECHO verification SOP class
Patient Root Q/R Find	1.2.840.10008.5.1.4.1.2.1.1	C-FIND at patient root level
Study Root Q/R Find	1.2.840.10008.5.1.4.1.2.2.1	C-FIND at study root level

Data Element Format

DICOM datasets (both in files and network transfers) consist of Data Elements:

[Diagram: see online version]

Field	Size	Description
Group Number	2 bytes	Tag group (e.g., 0010 = Patient, 0020 = Study)
Element Number	2 bytes	Tag element within the group
VR	2 bytes	Value Representation -- data type (Explicit VR only)
Value Length	2 or 4 bytes	Length of the value field
Value	Variable	The actual data

Common Tags

Tag	Name	VR	Description
(0008,0060)	Modality	CS	CT, MR, CR, US, DX, etc.

(0010,0010)	Patient's Name	PN	Patient name
(0010,0020)	Patient ID	LO	Patient identifier
(0020,000D)	Study Instance UID	UI	Unique study identifier
(0020,000E)	Series Instance UID	UI	Unique series identifier
(0008,0018)	SOP Instance UID	UI	Unique image/object identifier
(7FE0,0010)	Pixel Data	OW/OB	The image pixel data

Value Representations (VR)

VR	Name	Description
AE	Application Entity	AE title (max 16 chars)
CS	Code String	Enumerated values (max 16 chars)
DA	Date	YYYYMMDD format
DS	Decimal String	Numeric string
IS	Integer String	Integer as string
LO	Long String	Character string (max 64 chars)
OB	Other Byte	Byte stream
OW	Other Word	16-bit word stream
PN	Person Name	Name in family^given^middle^prefix^suffix format
SQ	Sequence	Nested sequence of datasets
UI	Unique Identifier	UID string (max 64 chars)
UL	Unsigned Long	32-bit unsigned integer
US	Unsigned Short	16-bit unsigned integer

Encapsulation

[Diagram: see online version]

DICOM runs directly over TCP. Port 104 is the well-known port; 11112 is commonly used as an unprivileged alternative. TLS support was added in the DICOM 2017 edition for secure transport.

Standards

Document	Title
NEMA PS3.1-PS3.22	DICOM Standard (22 parts)
PS3.7	Part 7: Message Exchange (DIMSE, PDU definitions)

PS3.8	Part 8: Network Communication Support for Message Exchange (Upper Layer)
PS3.4	Part 4: Service Class Specifications (C-STORE, C-FIND, etc.)
PS3.5	Part 5: Data Structures and Encoding (Data Elements, VRs)
PS3.6	Part 6: Data Dictionary (tag definitions)
ISO 12052	Health informatics -- Digital imaging and communication in medicine

See Also

- HL7 -- healthcare messaging (often paired with DICOM in clinical workflows)
- TCP
- TLS -- optional encryption for DICOM connections

HL7 (Health Level Seven) -- v2 Messaging & FHIR

Standard: HL7 v2.x / HL7 FHIR R4 | **Layer:** Application (Layer 7) | **Wireshark filter:** `hl7`

HL7 is the most widely adopted family of standards for exchanging clinical and administrative healthcare data between systems. HL7 v2 is a pipe-delimited messaging protocol transported over TCP using MLLP (Minimum Lower Layer Protocol), and remains the workhorse of hospital integration engines worldwide. HL7 FHIR (Fast Healthcare Interoperability Resources) is the modern successor, using RESTful HTTP/HTTPS with JSON or XML resources. Both standards are maintained by Health Level Seven International.

Part 1: HL7 v2 Messaging

MLLP Framing

HL7 v2 messages are transmitted over TCP using MLLP (Minimum Lower Layer Protocol), which wraps each message with start and end markers:

[Diagram: see online version]

Field	Hex	Description
Start Block (SB)	0x0B	Vertical Tab -- marks beginning of HL7 message
HL7 Message	--	The complete HL7 v2 message (segments separated by `\r`)

End Block (EB)	0x1C	File Separator -- marks end of HL7 message
Carriage Return (CR)	0x0D	Follows EB to complete the MLLP frame

MLLP typically runs on TCP port 2575, though any port may be configured.

Message Structure

An HL7 v2 message is a sequence of segments, each on its own line (separated by `\r`). Fields within a segment are separated by `|`, components by `^`, sub-components by `&`, and repetitions by `~`.

```
MSH|^~\&amp;|SendingApp|SendingFac|ReceivingApp|ReceivingFac|20260321120000||ADT^A01^ADT_A01|MSG00001|P|2.5.1
EVN|A01|20260321120000
PID|1||MRN12345^^^Hospital^MR||Doe^John^A||19800115|M|||123 Main St^^Springfield^IL^62701
PV1|1|I|ICU^201^A||||1234^Smith^Jane^Dr|||MED||||||||V00001||||||||||||||||||||20260321120000
```

MSH (Message Header) Segment

The MSH segment is always the first segment and defines the message metadata:

[Diagram: see online version]

MSH Key Fields

Position	Field	Description
MSH-1	Field Separator	Always `\
MSH-2	Encoding Characters	Component `^`, repetition `~`, escape `\`, sub-component `&`
MSH-3	Sending Application	Name of the sending system
MSH-4	Sending Facility	Organization/facility of sender
MSH-5	Receiving Application	Name of the receiving system
MSH-6	Receiving Facility	Organization/facility of receiver
MSH-7	Date/Time of Message	YYYYMMDDHHMMSS format
MSH-9	Message Type	Type^Trigger Event^Structure (e.g., ADT^A01^ADT_A01)
MSH-10	Message Control ID	Unique identifier for this message
MSH-11	Processing ID	P = Production, D = Debugging, T = Training
MSH-12	Version ID	HL7 version (e.g., 2.5.1, 2.7)

Common Segment Types

Segment	Name	Description

MSH	Message Header	Message metadata, type, version, encoding
EVN	Event Type	Trigger event details and timestamps
PID	Patient Identification	Demographics, MRN, name, DOB, gender, address
PV1	Patient Visit	Encounter/visit details, location, attending physician
NK1	Next of Kin	Emergency contacts and relationships
OBR	Observation Request	Order details (lab, radiology, procedure)
OBX	Observation Result	Individual result values with units and status
ORC	Common Order	Order control information (new, cancel, replace)
AL1	Allergy Information	Patient allergies
DG1	Diagnosis	Diagnosis codes (ICD-10)
IN1	Insurance	Insurance coverage information
MSA	Message Acknowledgment	ACK/NAK response to received message

Common Message Types

Type	Trigger	Name	Description
ADT	A01	Admit	Patient admitted to inpatient
ADT	A02	Transfer	Patient transferred between units
ADT	A03	Discharge	Patient discharged
ADT	A04	Register	Outpatient registration
ADT	A08	Update Patient	Demographics update
ORM	O01	Order Message	New order (lab, radiology, pharmacy)
ORU	R01	Observation Result	Lab/diagnostic results
SIU	S12	Schedule Notification	Appointment scheduling
MDM	T02	Document Notification	Clinical document available
ACK	--	Acknowledgment	Confirms receipt and processing of a message

HL7 v2 Message Exchange

[Diagram: see online version]

ACK Codes (MSA-1)

Code	Name	Description
AA	Application Accept	Message received and processed successfully
AE	Application Error	Message received but processing error occurred
AR	Application Reject	Message rejected (invalid format, unknown type)
CA	Commit Accept	Enhanced mode -- committed to safe storage
CE	Commit Error	Enhanced mode -- commit failed
CR	Commit Reject	Enhanced mode -- commit rejected

Part 2: HL7 FHIR (Fast Healthcare Interoperability Resources)

FHIR is a modern, web-native standard for healthcare data exchange. It uses RESTful HTTP/HTTPS with resources represented in JSON or XML. FHIR is the foundation of national interoperability mandates (US ONC, EU EHDS) and is supported by all major EHR vendors.

FHIR REST Operations

HTTP Method	Operation	URL Pattern	Description
GET	Read	`/Patient/123`	Read a specific resource by ID
GET	Search	`/Patient?name=Smith`	Search for matching resources
POST	Create	`/Patient`	Create a new resource (server assigns ID)
PUT	Update	`/Patient/123`	Update a resource (client provides full resource)
DELETE	Delete	`/Patient/123`	Delete a resource
GET	vRead	`/Patient/123/_history/2`	Read a specific version
GET	History	`/Patient/123/_history`	Retrieve version history
POST	Operation	`/Patient/$everything`	Invoke a named operation

Core Resources

Resource	Description

Patient	Demographics, identifiers, contact information
Practitioner	Healthcare provider details
Encounter	A patient visit or interaction
Observation	Lab results, vital signs, clinical measurements
Condition	Diagnoses, problems, health concerns
MedicationRequest	Prescription / medication order
DiagnosticReport	Report grouping observations (lab panel, radiology)
AllergyIntolerance	Allergy and intolerance records
Procedure	Clinical procedures performed
Immunization	Vaccination records
DocumentReference	Reference to a clinical document (CDA, PDF)
Bundle	Collection of resources (transaction, batch, searchset)

FHIR REST Flow

[Diagram: see online version]

Search Parameters and Modifiers

Parameter	Example	Description
Simple	`?name=Smith`	Search by name
Token	`?identifier=MRN\	12345`
Date	`?birthdate=gt1990-01-01`	Date comparison (gt, lt, ge, le, eq)
Reference	`?subject=Patient/123`	Search by referenced resource
Chained	`?subject:Patient.name=Smith`	Search through references
Include	`?_include=Observation:subject`	Include referenced resources in results
Modifier	`?name:exact=Smith`	Exact match modifier
Reverse include	`?_revinclude=Observation:subject`	Include resources that reference this

Bundle Types

Type	Description
searchset	Results of a search operation
transaction	Atomic set of operations (all succeed or all fail)
batch	Independent set of operations (each processed individually)
document	Clinical document (Composition + referenced resources)

message	FHIR messaging (MessageHeader + payload resources)
collection	Arbitrary collection of resources

SMART on FHIR Authorization

SMART on FHIR defines an OAuth 2.0-based authorization framework for FHIR. It supports EHR-launched and standalone application authorization flows with clinical scopes.

Scope	Example	Description
patient/	`patient/Patient.read`	Access data for the current patient context
user/	`user/Observation.write`	Access data the user is authorized to see
launch	`launch`	Request EHR launch context
openid fhirUser	`openid fhirUser`	Get the identity of the logged-in user

HL7 v2 vs FHIR Comparison

Aspect	HL7 v2	FHIR
Format	Pipe-delimited text segments	JSON / XML resources
Transport	MLLP over TCP	HTTP/HTTPS (REST)
Data Model	Segments and fields (positional)	Resources with named elements
Query	Limited (QBP messages)	Rich RESTful search with parameters
Security	TLS (transport only)	OAuth 2.0, SMART on FHIR
Adoption	~95% of US hospitals (legacy)	Growing rapidly; mandated for new APIs
Versioning	v2.1 through v2.9	DSTU2, STU3, R4, R4B, R5
Default Port	2575 (MLLP)	443 (HTTPS)
Interoperability	Point-to-point, high variability	Standardized REST API, better consistency
Tooling	Integration engines (Mirth, Rhapsody)	Standard HTTP libraries, FHIR SDKs

Encapsulation

[Diagram: see online version]

Standards

Document	Title
HL7 v2.5.1	HL7 Version 2.5.1 Messaging Standard
HL7 v2.7	HL7 Version 2.7 Messaging Standard
HL7 FHIR R4	FHIR Release 4 (normative, most widely deployed)
HL7 FHIR R5	FHIR Release 5
SMART on FHIR	SMART App Launch Framework (OAuth 2.0 for FHIR)
IHE Profiles	Integration profiles using HL7 v2 and FHIR
US Core IG	US Core FHIR Implementation Guide (ONC/CMS mandated)

See Also

- DICOM -- medical imaging protocol (often integrated with HL7 in clinical workflows)
- HTTP -- transport for FHIR REST API
- TLS -- encrypts both MLLP and HTTPS connections
- TCP -- transport layer for MLLP

High-Performance Computing

Protocols optimized for high-performance and cluster computing.

3 protocols in this section

MPI -- Message Passing Interface

Standard: MPI Forum (mpi-forum.org) | **Layer:** Application / Middleware | **Wireshark filter:** Depends on transport (TCP, InfiniBand, or RDMA)

MPI is the standard API and wire protocol for parallel computing across distributed systems. It defines how processes exchange messages, perform collective operations, and synchronize in high-performance computing (HPC) clusters. MPI has been the backbone of scientific computing, weather simulation, molecular dynamics, and computational fluid dynamics for 30 years. With GPU-aware MPI implementations, it's also used in AI/ML training (alongside or as an alternative to NCCL).

Core Concepts

Concept	Description
Rank	Unique process ID within a communicator (0 to N-1)
Communicator	Group of processes that can communicate (MPI_COMM_WORLD = all)
Tag	Integer label to distinguish messages between the same rank pair
Datatype	MPI_INT, MPI_FLOAT, MPI_DOUBLE, or user-defined
Buffer	Application memory containing message data

Point-to-Point Operations

Function	Description
MPI_Send	Blocking send -- returns after buffer is safe to reuse
MPI_Recv	Blocking receive -- returns after message is in buffer
MPI_Isend	Non-blocking send -- returns immediately (check with MPI_Wait)
MPI_Irecv	Non-blocking receive -- returns immediately
MPI_Sendrecv	Simultaneous send and receive (avoids deadlock)
MPI_Probe	Check if a message is available without receiving it

Send/Receive Flow

[Diagram: see online version]

Collective Operations

Operation	Description	Communication Pattern
MPI_Bcast	One rank sends same data to all	One -> all
MPI_Scatter	One rank sends different data to each	One -> all (partitioned)
MPI_Gather	All ranks send data to one	All -> one
MPI_Allgather	Gather from all, result on all	All -> all
MPI_Reduce	Combine from all to one (sum, max, min)	All -> one
MPI_Allreduce	Reduce + broadcast (result on all)	All -> all
MPI_Alltoall	Each rank sends different data to each other	All -> all (transposed)
MPI_Barrier	Synchronize all ranks (block until everyone arrives)	Synchronization
MPI_Scan	Prefix reduction (each rank gets partial result)	All -> all (ordered)
MPI_Reduce_scatter	Reduce + scatter results	All -> all (partitioned)

MPI_Allreduce

[Diagram: see online version]

Reduction Operations

Operation	Description
MPI_SUM	Sum of all values
MPI_PROD	Product of all values
MPI_MAX	Maximum value
MPI_MIN	Minimum value
MPI_MAXLOC	Maximum value + location (rank)
MPI_MINLOC	Minimum value + location (rank)
MPI_LAND	Logical AND
MPI_LOR	Logical OR
MPI_BAND	Bitwise AND
MPI_BOR	Bitwise OR

Communication Modes

Mode	Function	Behavior
Standard	MPI_Send	Implementation decides buffering

Buffered	MPI_Bsend	User-provided buffer, always returns immediately
Synchronous	MPI_Ssend	Returns only after matching receive has started
Ready	MPI_Rsend	Assumes receive already posted (undefined if not)

Wire Protocol

MPI itself is an API specification -- implementations choose the wire protocol based on available hardware:

Transport	Implementation	When Used
Shared memory	All	Same-node communication (fastest)
TCP/IP	OpenMPI (tcp BTL), MPICH (tcp)	Between nodes on Ethernet (fallback)
InfiniBand Verbs	OpenMPI (openib), MVAPICH	Between nodes on IB fabric
UCX (Unified Communication X)	OpenMPI, MPICH	Auto-selects best transport (IB, RoCE, shared mem)
libfabric (OFI)	MPICH, Intel MPI	Abstract transport interface
RDMA (RoCE v2)	UCX, libfabric	Ethernet with RDMA capability

UCX Transport Selection

[Diagram: see online version]

MPI for AI/ML

Use Case	MPI Role
Data Parallel Training	MPI_Allreduce for gradient sync (or NCCL)
Pipeline Parallelism	MPI_Send/Recv between pipeline stages
Model Parallelism	MPI_Allgather for parameter sharing
Horovod	MPI + NCCL hybrid for distributed training
DeepSpeed	MPI for process management, NCCL for GPU communication

GPU-Aware MPI

Modern MPI implementations can send/receive directly from GPU memory:

```
// GPU buffer -- no need to copy to host first
float *gpu_gradients;  // CUDA device pointer
MPI_Allreduce(gpu_gradients, gpu_result, count, MPI_FLOAT, MPI_SUM, MPI_COMM_WORLD);
// Data moved directly between GPUs via GPUDirect RDMA
```

Job Launch

Launcher	Description
`mpirun -np 8 ./app`	OpenMPI/MPICH process launcher
`srun --ntasks=8 ./app`	SLURM workload manager
`torchrun --nproc_per_node=4`	PyTorch distributed launcher (uses MPI or Gloo)

MPI vs NCCL

Feature	MPI	NCCL
Scope	General HPC	GPU deep learning only
CPU support	Yes (primary)	No (GPU only)
GPU support	Via GPU-aware MPI	Native
NVLink optimization	Manual (UCX)	Automatic
Collectives	Full set + custom	Focused set (AllReduce, AllGather, etc.)
Point-to-point	Full support	Send/Recv only
Topology awareness	Limited (UCX helps)	Automatic (NVLink, PCIe, IB detection)
Use in training	Horovod, some DeepSpeed	PyTorch DDP, FSDP, Megatron

Implementations

Implementation	Vendor	Notes
OpenMPI	Open source	Most portable, UCX support
MPICH	ANL (open source)	Reference implementation
MVAPICH	Ohio State	Optimized for InfiniBand and GPU
Intel MPI	Intel	Based on MPICH, optimized for Intel hardware
NVIDIA HPC-X	NVIDIA	OpenMPI + UCX, optimized for NVIDIA
Microsoft MPI	Microsoft	Windows HPC

Standards

Document	Title
MPI 4.1 Standard	MPI: A Message-Passing Interface Standard Version 4.1
MPI Forum	MPI Forum (standards body)

See Also

- RDMA / RoCE -- high-performance transport MPI uses
- NCCL -- GPU-specific collective library (often used alongside MPI)
- TCP -- fallback MPI transport
- SCTP -- multi-stream transport (some MPI implementations)

NCCL -- NVIDIA Collective Communication Library

Standard: NVIDIA NCCL (developer.nvidia.com) | **Layer:** Application / Transport | **Wireshark filter:** `roce` or `tcp` (NCCL uses RDMA or TCP as transport)

NCCL (pronounced "nickel") is NVIDIA's library for multi-GPU and multi-node collective communication, optimized for deep learning training. It implements collective operations (AllReduce, AllGather, Broadcast, etc.) that are the core of distributed training -- when hundreds of GPUs need to synchronize gradients after each training step. NCCL automatically selects the optimal communication path: NVLink (intra-node GPU-to-GPU), PCIe, InfiniBand RDMA, RoCE, or TCP.

Collective Operations

Operation	Description	AI/ML Use
AllReduce	Reduce (sum/max/min) + broadcast to all	**Gradient aggregation** (most critical)
AllGather	Gather data from all, distribute to all	Model parallelism, embedding tables
ReduceScatter	Reduce + scatter results across ranks	ZeRO optimizer (DeepSpeed)
Broadcast	One rank sends to all others	Model initialization, checkpoint loading
Reduce	Combine from all to one rank	Loss aggregation
AllToAll	Each rank sends different data to each other rank	Expert routing (Mixture of Experts)
Send/Recv	Point-to-point between two ranks	Pipeline parallelism

AllReduce (the critical operation)

[Diagram: see online version]

Every GPU ends up with the same summed gradients -- ready to update the model.

Communication Topology

NCCL automatically builds optimal communication topologies:

Ring AllReduce

[Diagram: see online version]

Data is split into chunks and pipelined around a ring. After 2x(N-1) steps, all GPUs have the full result. Bandwidth-optimal for large messages.

Tree AllReduce

[Diagram: see online version]

Lower latency for small messages (log N steps vs 2N for ring).

NCCL Auto-Selection

NCCL detects the hardware topology and selects the best algorithm:

Message Size	Algorithm	Why
Small (<256KB)	Tree	Lower latency (fewer rounds)
Medium	Ring	Good balance
Large (>4MB)	Ring	Bandwidth-optimal
Very large	Pipeline/chunked ring	Overlaps computation and communication

Transport Hierarchy

NCCL selects the fastest available transport for each GPU pair:

Transport	Bandwidth	Latency	When Used
NVLink (intra-node)	600-900 GB/s (NVLink 4.0)	< 1 µs	GPUs on same node with NVLink
PCIe (intra-node)	32-64 GB/s (Gen4/5)	~1 µs	GPUs on same node without NVLink
InfiniBand RDMA	25-50 GB/s (per port)	1-2 µs	GPUs on different nodes (IB fabric)
RoCE v2	12.5-50 GB/s	2-5 µs	GPUs on different nodes (Ethernet)
TCP/IP	1-12.5 GB/s	20-50 µs	Fallback (no RDMA available)
GPUDirect RDMA	InfiniBand speed	~2 µs	GPU memory -> RNIC -> network (bypasses host)
NVLink + NVSwitch	900 GB/s all-to-all	< 1 µs	DGX/HGX systems (full bisection)

GPUDirect RDMA

[Diagram: see online version]

No host CPU or system memory involved -- tensor data moves directly from GPU to GPU across the network.

NCCL in Training Frameworks

Framework	How NCCL is Used
PyTorch (DDP)	`torch.distributed` backend="nccl" -- AllReduce gradients
PyTorch (FSDP)	Sharded AllGather/ReduceScatter for model parallelism
DeepSpeed (ZeRO)	ReduceScatter/AllGather for optimizer state sharding
Megatron-LM	Tensor/Pipeline parallelism, AllReduce within TP groups
TensorFlow	`tf.distribute.Strategy` with NCCL
JAX	`jax.distributed` with NCCL
Horovod	AllReduce across GPUs/nodes via NCCL

Data Parallelism (AllReduce)

[Diagram: see online version]

Environment Variables

Variable	Description
`NCCL_DEBUG=INFO`	Enable NCCL logging (topology, transports, performance)
`NCCL_IB_DISABLE=1`	Disable InfiniBand (force TCP fallback)
`NCCL_SOCKET_IFNAME=eth0`	Select network interface
`NCCL_P2P_LEVEL=NVL`	NVLink peer-to-peer policy
`NCCL_NET_GDR_LEVEL=5`	GPUDirect RDMA level
`NCCL_ALGO=Ring`	Force specific algorithm
`NCCL_PROTO=Simple`	Protocol: Simple, LL (low-latency), or LL128
`NCCL_CROSS_NIC=1`	Allow cross-NIC communication (multi-NIC nodes)

NCCL vs Other Collective Libraries

Feature	NCCL	MPI (OpenMPI/MVAPICH)	Gloo (PyTorch)
GPU-aware	Native (GPU memory)	Via GPU-aware MPI	CPU only (+ GPU staging)
NVLink optimization	Automatic	Manual	No

GPUDirect RDMA	Automatic	Supported (config required)	No
Topology detection	Automatic (NVLink, PCIe, IB)	Manual	Limited
Use case	GPU training	General HPC	CPU training, fallback
Vendor	NVIDIA	Open source	Meta (PyTorch)

Standards

NCCL is not a standards-body protocol -- it's a proprietary NVIDIA library with public API:

Resource	Description
NCCL Documentation	Official API and usage guide
NCCL GitHub	Open-source implementation
NCCL Tests	Performance benchmarking tools

See Also

- RDMA / RoCE -- network transport NCCL uses for inter-node communication
- MPI -- general HPC collective communication
- DDS / ROS 2 -- pub/sub middleware (different domain)
- gRPC -- used by ML serving (Triton), not training

RDMA / RoCE / InfiniBand -- Remote Direct Memory Access

Standard: InfiniBand Architecture Specification / RoCE v2 (Annex A17) | **Layer:** Transport / Network (Layers 3-4) | **Wireshark filter:** `infiniband` or `roce`

RDMA (Remote Direct Memory Access) allows one computer to read from or write to another computer's memory directly, bypassing both operating systems' CPU, caches, and kernel network stacks. This achieves microsecond latencies and near-line-rate throughput -- critical for AI/ML training clusters, HPC, and high-frequency trading. RDMA operates over three physical transports: InfiniBand (native), RoCE (RDMA over Converged Ethernet), and iWARP (RDMA over TCP).

How RDMA Works

[Diagram: see online version]

[Diagram: see online version]

The application registers memory regions with the RNIC. Data moves directly between application memory and the network -- **zero kernel involvement**, zero CPU copies.

RDMA Operations (Verbs)

Operation	Type	Description
Send/Receive	Two-sided	Sender sends, receiver must post a receive buffer
RDMA Write	One-sided	Write directly to remote memory (remote CPU unaware)
RDMA Read	One-sided	Read directly from remote memory (remote CPU unaware)
Atomic Compare-and-Swap	One-sided	Atomic CAS on remote memory
Atomic Fetch-and-Add	One-sided	Atomic add on remote memory

One-Sided vs Two-Sided

[Diagram: see online version]

Transport Types

InfiniBand

Parameter	Value
Physical	Dedicated InfiniBand fabric (not Ethernet)
Speeds	SDR (10G), DDR (20G), QDR (40G), FDR (56G), EDR (100G), HDR (200G), NDR (400G), XDR (800G)
Latency	< 1 µs
Topology	Fat-tree, Dragonfly (with InfiniBand switches)
Connector	QSFP, QSFP56, OSFP
Vendor	NVIDIA/Mellanox (dominant)

RoCE v2 (RDMA over Converged Ethernet)

[Diagram: see online version]

Parameter	Value
Physical	Standard Ethernet (25G, 50G, 100G, 200G, 400G)
Encapsulation	UDP/IP + InfiniBand transport headers
UDP port	4791

Latency	2-5 µs (slightly more than native IB)
Requires	Lossless Ethernet (PFC/ECN -- DCB) or congestion-tolerant RoCE
Advantage	Uses existing Ethernet switches and cabling

iWARP (Internet Wide Area RDMA Protocol)

Parameter	Value
Physical	Standard Ethernet
Encapsulation	TCP/IP (no lossless Ethernet required)
Latency	5-10 µs (TCP overhead)
Advantage	Works over any TCP/IP network (even WAN)
Disadvantage	Higher latency, lower throughput than RoCE

InfiniBand Headers

Local Route Header (LRH)

[Diagram: see online version]

Base Transport Header (BTH)

[Diagram: see online version]

Key Concepts

Concept	Description
Queue Pair (QP)	A send queue + receive queue -- the fundamental communication endpoint
Completion Queue (CQ)	Notifications when operations complete
Memory Region (MR)	Registered memory that the RNIC can access directly
Protection Domain (PD)	Isolates QPs and MRs for security
R-Key / L-Key	Remote/local keys authorizing memory access
Partition Key (P-Key)	Network isolation (like VLANs for InfiniBand)

RDMA Connection Setup

[Diagram: see online version]

AI/ML Use Case

GPU clusters use RDMA (InfiniBand or RoCE) for:

Operation	Description
All-Reduce	Aggregate gradients across all GPUs (training)
All-Gather	Collect model parameters from all GPUs
Point-to-point	Transfer tensors between specific GPUs
GPUDirect RDMA	RDMA directly from GPU memory (bypasses host CPU entirely)

[Diagram: see online version]

Performance Comparison

Transport	Latency	Throughput (per port)	CPU Overhead
TCP/IP	20-50 µs	10-100 Gbps	High (kernel copies)
RoCE v2	2-5 µs	25-400 Gbps	Near zero
InfiniBand	< 1 µs	100-400 Gbps (NDR)	Near zero
iWARP	5-10 µs	10-100 Gbps	Low

Standards

Document	Title
InfiniBand Architecture Spec	InfiniBand Architecture Specification
RoCE v2 (Annex A17)	RDMA over Converged Ethernet v2
RFC 5040	RDMA Protocol Verbs (iWARP)
RFC 5041	DDP -- Direct Data Placement (iWARP)
RFC 5042	MPA -- Marker PDU Aligned (iWARP)

See Also

- Ethernet -- RoCE runs over Ethernet
- UDP -- RoCE v2 uses UDP port 4791
- NCCL -- NVIDIA's collective communication library (uses RDMA)
- MPI -- HPC message passing (uses RDMA transports)

Industrial & SCADA

Industrial control, SCADA, and building automation protocols.

14 protocols in this section

BACnet -- Building Automation and Control Networks

Standard: ASHRAE 135 / ISO 16484-5 | **Layer:** Application (Layer 7) | **Wireshark filter:** `bacnet` or `bacapp`

BACnet is the dominant communication protocol for building automation systems (BAS), connecting HVAC controllers, lighting systems, fire/life safety panels, access control, elevators, and energy metering equipment. Developed by ASHRAE (American Society of Heating, Refrigerating and Air-Conditioning Engineers), BACnet provides a vendor-neutral way for building devices to share data and coordinate control. It defines an object-oriented data model, a set of application services, and multiple network transport options: BACnet/IP (UDP port 47808), BACnet MS/TP (master-slave/token-passing over RS-485), BACnet Ethernet (native 802.3), and BACnet/SC (Secure Connect, TLS-based). BACnet is required by many building codes and green building standards.

Data Link Options

Variant	Transport	Speed	Description
BACnet/IP	UDP port 47808 (0xBAC0)	10/100/1000 Mbps	IP-based, most common in modern buildings
BACnet MS/TP	RS-485	9.6-115.2 kbps	Master-slave/token-passing, field-level wiring
BACnet Ethernet	Ethernet 802.3 (EtherType 0x82BB)	10/100 Mbps	Native Ethernet (no IP), legacy
BACnet ARCnet	ARCnet	2.5 Mbps	Legacy token-passing (rare)
BACnet/SC	TLS over WebSocket	10/100/1000 Mbps	Secure Connect (modern, encrypted)

BACnet/IP (BVLL Header)

BACnet/IP wraps BACnet messages in a BACnet Virtual Link Layer (BVLL) header:

[Diagram: see online version]

BVLL Functions

Code	Name	Description
0x00	BVLC-Result	Acknowledgment
0x01	Write-Broadcast-Distribution-Table	Configure BBMD table
0x04	Forwarded-NPDU	BBMD-forwarded broadcast
0x09	Original-Broadcast-NPDU	Broadcast on local subnet
0x0A	Original-Unicast-NPDU	Unicast message
0x0B	Distribute-Broadcast-To-Network	Request BBMD to distribute

NPDU (Network Protocol Data Unit)

The NPDU provides network-layer routing between BACnet networks:

[Diagram: see online version]

NPDU Key Fields

Field	Size	Description
Version	8 bits	Always 0x01
Control	8 bits	Flags: DNET present, SNET present, expecting reply, network message, priority
DNET	16 bits	Destination network number (0xFFFF = global broadcast)
DLEN	8 bits	Destination MAC address length (0 = broadcast on DNET)
DADR	Variable	Destination MAC address
SNET	16 bits	Source network number
SLEN	8 bits	Source MAC address length
SADR	Variable	Source MAC address
Hop Count	8 bits	Decremented by each router (prevents loops)

NPDU Control Bits

Bit	Name	Description
7	Network Layer Message	1 = network layer message, 0 = APDU follows
5	Destination Specifier	1 = DNET/DLEN/DADR present
3	Source Specifier	1 = SNET/SLEN/SADR present
2	Expecting Reply	1 = reply expected
1-0	Priority	00 = normal, 01 = urgent, 10 = critical, 11 = life safety

APDU (Application Protocol Data Unit)

APDU Types

Type	Value	Name	Description

0	0x0	Confirmed-REQ	Request that expects a response
1	0x1	Unconfirmed-REQ	Request with no response expected
2	0x2	SimpleACK	Acknowledgment (no data)
3	0x3	ComplexACK	Acknowledgment with return data
4	0x4	SegmentACK	Acknowledge a segment of a segmented message
5	0x5	Error	Service error response
6	0x6	Reject	Message rejected (protocol error)
7	0x7	Abort	Abort a transaction

Confirmed Request APDU

[Diagram: see online version]

Unconfirmed Request APDU

[Diagram: see online version]

Application Services

Object Access Services

Code	Name	Type	Description
12	ReadProperty	Confirmed	Read a single property of an object
14	ReadPropertyMultiple	Confirmed	Read multiple properties in one request
15	WriteProperty	Confirmed	Write a single property
16	WritePropertyMultiple	Confirmed	Write multiple properties in one request
6	CreateObject	Confirmed	Dynamically create an object
7	DeleteObject	Confirmed	Dynamically delete an object

Alarm and Event Services

Code	Name	Type	Description

5	SubscribeCOV	Confirmed	Subscribe to Change of Value notifications
2	ConfirmedCOVNotification	Confirmed	COV notification (expects ACK)
3	UnconfirmedCOVNotification	Unconfirmed	COV notification (fire and forget)
0	AcknowledgeAlarm	Confirmed	Acknowledge an alarm condition

Device and Network Services

Code	Name	Type	Description
8	WhoIs	Unconfirmed	Discover devices (broadcast)
0	IAm	Unconfirmed	Device announcement (response to WhoIs)
7	WhoHas	Unconfirmed	Find which device has a specific object
1	IHave	Unconfirmed	Response to WhoHas
18	ReinitializeDevice	Confirmed	Warm/cold start a device
20	TimeSynchronization	Unconfirmed	Synchronize device clock

File Services

Code	Name	Type	Description
6	AtomicReadFile	Confirmed	Read data from a file object
7	AtomicWriteFile	Confirmed	Write data to a file object

Object Model

Every BACnet device exposes its data as a collection of objects, each with a set of standard properties:

Common Object Types

Type ID	Name	Description
0	Analog Input	Sensor reading (temperature, pressure, humidity)
1	Analog Output	Control output (valve position, fan speed)
2	Analog Value	Internal setpoint or calculated value
3	Binary Input	Digital sensor (switch, occupancy, status)

4	Binary Output	Digital control (relay, on/off)
5	Binary Value	Internal boolean (mode, enable/disable)
8	Device	The device itself (required, one per device)
10	File	File access (firmware, logs)
13	Multi-state Input	Enumerated input (operating mode)
14	Multi-state Output	Enumerated output (fan speed setting)
17	Schedule	Weekly schedule with exceptions
6	Calendar	Date-based schedule (holidays, special events)
20	Trend Log	Historical data logging
15	Notification Class	Alarm routing configuration
19	Program	Programmable logic within the device
12	Loop	PID control loop

Standard Properties (all objects)

Property	Description
Object_Identifier	Unique ID: (Object_Type, Instance_Number)
Object_Name	Human-readable unique name
Object_Type	Type of object (Analog Input, Binary Output, etc.)
Present_Value	Current value of the object
Status_Flags	In_Alarm, Fault, Overridden, Out_of_Service
Description	Optional text description
Units	Engineering units (degrees-C, percent, etc.)
Reliability	Sensor/communication reliability indication

Typical Flow

ReadProperty

[Diagram: see online version]

WhoIs / IAm Discovery

[Diagram: see online version]

SubscribeCOV (Change of Value)

[Diagram: see online version]

MS/TP (Master-Slave / Token-Passing)

BACnet MS/TP runs over RS-485 using a token-passing scheme among masters with polling of slaves:

Feature	Description
Physical	RS-485, half-duplex, up to 4000 ft (1200 m)
Baud	9600, 19200, 38400, 57600, 76800, 115200
Addressing	0-127 master, 128-254 slave, 255 broadcast
Token	Passed among masters in address order
Max Masters	128 (configurable)
Max Info Frames	Configurable limit per token hold

BACnet Network Architecture

[Diagram: see online version]

Encapsulation

[Diagram: see online version]

Standards

Document	Title
ASHRAE 135	BACnet -- A Data Communication Protocol for Building Automation and Control Networks
ISO 16484-5	Building automation and control systems -- Data communication protocol
ISO 16484-6	Data communication conformance testing
ASHRAE 135.1	Method of Test for Conformance to BACnet
Addendum BJ (BACnet/SC)	BACnet Secure Connect (TLS-based transport)

See Also

- Modbus -- simpler industrial protocol, also used in buildings
- OPC UA -- platform-independent industrial interoperability
- RS-485 -- physical layer for BACnet MS/TP
- UDP -- transport for BACnet/IP

DNP3 -- Distributed Network Protocol 3

Standard: IEEE 1815 | **Layer:** Data Link / Transport / Application | **Wireshark filter:** dnp3

DNP3 is a telemetry protocol designed for communication between SCADA masters, RTUs, and IEDs in electric utility, water/wastewater, and oil/gas systems. Developed by Westronic (now GE) in the early 1990s based on early IEC 60870-5 drafts, DNP3 provides reliable data acquisition and control over noisy, low-bandwidth serial and IP links. It features a three-layer architecture (Data Link, Transport, Application), time-stamped event reporting, unsolicited responses, and Secure Authentication for critical infrastructure protection. DNP3 is the dominant SCADA protocol in North America.

Protocol Stack

[Diagram: see online version]

Data Link Layer Frame

All DNP3 communication is built on Data Link frames. Each frame begins with a fixed 10-byte header followed by optional data blocks, each with its own CRC:

Frame Header (10 bytes)

[Diagram: see online version]

Field	Size	Description
Start Bytes	16 bits	Always 0x0564 -- identifies a DNP3 frame
Length	8 bits	Number of octets following (excluding CRCs) -- minimum 5
Control	8 bits	Direction, flow control, function code
Destination	16 bits	Destination address (0-65519; 65520-65535 reserved)
Source	16 bits	Source address
Header CRC	16 bits	CRC-16 over the first 8 header bytes

Data Blocks

After the header, data is sent in blocks of up to 16 bytes, each followed by a 2-byte CRC:

[Diagram: see online version]

This per-block CRC scheme enables detection and localization of errors on noisy serial links.

Data Link Control Byte

Bit	Name	Description
7	DIR	Direction: 1 = master-to-outstation, 0 = outstation-to-master
6	PRM	Primary: 1 = from initiator, 0 = from responder
5	FCB	Frame Count Bit -- alternates for duplicate detection
4	FCV / DFC	FCV (primary): FCB valid; DFC (secondary): Data Flow Control
3-0	Function	Data link function code

Data Link Function Codes

Primary (PRM=1):

Code	Name	Description
0	RESET_LINK	Reset link state
1	RESET_USER	Reset user process
2	TEST_LINK	Test link connectivity
3	USER_DATA_CONFIRM	Confirmed user data
4	USER_DATA_NO_CONFIRM	Unconfirmed user data
9	REQUEST_LINK_STATUS	Query link status

Secondary (PRM=0):

Code	Name	Description
0	ACK	Positive acknowledgment
1	NACK	Negative acknowledgment
11	LINK_STATUS	Link status response
15	NOT_SUPPORTED	Function not supported

Transport Layer

The Transport Layer segments application messages into Data Link frames and reassembles them at the receiver. Each transport segment has a 1-byte header:

[Diagram: see online version]

Field	Size	Description

FIN	1 bit	1 = final segment of the message
FIR	1 bit	1 = first segment of the message
Sequence	6 bits	Sequence number (0-63, wraps around)
Fragment Data	Variable	Portion of the Application Layer message

A single-fragment message has both FIR=1 and FIN=1. Multi-fragment messages use FIR=1 on the first, FIN=1 on the last, and incrementing sequence numbers.

Application Layer

Application Request Header

[Diagram: see online version]

Application Response Header

[Diagram: see online version]

Field	Size	Description
App Control	8 bits	FIR, FIN, CON (confirm), UNS (unsolicited), sequence
Function Code	8 bits	Operation requested or performed
IIN	16 bits	Internal Indications -- response only (device status flags)
Object Data	Variable	Data objects with group, variation, qualifier, and values

Application Control Byte

Bit	Name	Description
7	FIR	First fragment of application message
6	FIN	Final fragment of application message
5	CON	Confirm requested -- receiver must send Application Confirm
4	UNS	Unsolicited response (outstation-initiated)
3-0	Sequence	Application-layer sequence number (0-15)

Function Codes

Code	Name	Direction	Description
0x00	Confirm	Both	Application-layer acknowledgment
0x01	Read	Request	Read data objects
0x02	Write	Request	Write data objects
0x03	Select	Request	Select a control point (first step of SBO)
0x04	Operate	Request	Operate a selected control point
0x05	Direct Operate	Request	Operate immediately (no select)
0x06	Direct Operate No Ack	Request	Operate without application confirm
0x07	Immediate Freeze	Request	Freeze counter values
0x08	Immediate Freeze No Ack	Request	Freeze counters, no confirm
0x0D	Cold Restart	Request	Full device restart
0x0E	Warm Restart	Request	Partial restart (reload configuration)
0x14	Enable Unsolicited	Request	Enable unsolicited responses for specified classes
0x15	Disable Unsolicited	Request	Disable unsolicited responses
0x16	Assign Class	Request	Assign objects to event classes
0x81	Response	Response	Normal response to a request
0x82	Unsolicited Response	Response	Outstation-initiated event report

Data Objects (Groups and Variations)

DNP3 data is organized by group (type of data) and variation (encoding format):

Group	Name	Common Variations
1	Binary Input	1 = packed, 2 = with flags
2	Binary Input Event	1 = without time, 2 = with absolute time
3	Double-Bit Binary Input	1 = packed, 2 = with flags
10	Binary Output	1 = packed, 2 = with flags
12	Binary Output Command (CROB)	1 = control relay output block
20	Counter	1 = 32-bit with flags, 5 = 32-bit no flags
22	Counter Event	1 = 32-bit with flags and time

30	Analog Input	1 = 32-bit, 3 = 32-bit no flags, 5 = float
32	Analog Input Event	1 = 32-bit with time, 7 = float with time
40	Analog Output Status	1 = 32-bit, 3 = float
41	Analog Output Command	1 = 32-bit, 3 = float
50	Time and Date	1 = absolute time (48-bit ms since epoch)
60	Class Data	1 = Class 0 (static), 2/3/4 = Class 1/2/3 (events)
80	Internal Indications	1 = packed format
120	Authentication	Secure Authentication objects

Internal Indications (IIN) Bits

Byte	Bit	Name	Description
1	7	BROADCAST	Message was broadcast
1	6	CLASS_1	Class 1 events available
1	5	CLASS_2	Class 2 events available
1	4	CLASS_3	Class 3 events available
1	3	NEED_TIME	Outstation needs time sync
1	2	LOCAL_CONTROL	Some outputs in local override
1	1	DEVICE_TROUBLE	Abnormal condition
1	0	DEVICE_RESTART	Outstation restarted
2	7	NO_FUNC_CODE	Function code not supported
2	6	OBJECT_UNKNOWN	Requested object unknown
2	5	PARAM_ERROR	Parameter error in request
2	4	EVENT_OVERFLOW	Event buffer overflow
2	1	CONFIG_CORRUPT	Configuration corrupt

Communication Flow

Integrity Poll (Class 0 + Events)

[Diagram: see online version]

Select-Before-Operate (SBO)

[Diagram: see online version]

Unsolicited Response

[Diagram: see online version]

DNP3 over TCP/IP

When carried over TCP (port 20000), the Data Link frames are sent directly inside TCP segments -- no additional encapsulation header is needed. The start bytes (0x0564) serve as frame synchronization.

[Diagram: see online version]

Secure Authentication (SA)

DNP3 Secure Authentication (SAv5, IEEE 1815) adds challenge-response authentication to prevent spoofing and replay attacks on critical control operations:

Feature	Description
HMAC-SHA-256	Cryptographic authentication of critical messages
Challenge-Response	Master challenges outstation (or vice versa) before critical ops
Aggressive Mode	HMAC sent with request (reduces round trips)
Key Update	Periodic update of session keys via Authority
User Management	Multiple users with individual credentials

Standards

Document	Title
IEEE 1815-2012	IEEE Standard for Electric Power Systems Communications -- DNP3
IEEE 1815.1	DNP3 -- Mapping to IEC 61850
DNP3 Specification	DNP Users Group technical documents
NERC CIP	Critical Infrastructure Protection standards (US power grid)

See Also

- Modbus -- simpler SCADA protocol, also widely used in utilities
- PROFIBUS -- European industrial fieldbus standard
- RS-232 -- common serial transport for DNP3
- RS-485 -- multi-drop serial transport for DNP3
- TCP -- transport for DNP3/IP (port 20000)

EtherNet/IP -- EtherNet/Industrial Protocol

Standard: IEC 61158 / ODVA | **Layer:** Application (Layer 7) | **Wireshark filter:** `enip`

EtherNet/IP is an industrial network protocol that runs the Common Industrial Protocol (CIP) over standard Ethernet, TCP/IP, and UDP/IP. Developed by Rockwell Automation and managed by ODVA, it is one of the most widely deployed industrial Ethernet protocols, particularly in North American manufacturing. EtherNet/IP uses TCP port 44818 for explicit messaging (configuration, diagnostics, non-time-critical data) and UDP port 2222 for implicit messaging (real-time I/O data exchange for control loops). It leverages standard Ethernet infrastructure -- no special hardware required -- making it easy to integrate with enterprise IT networks.

Architecture

EtherNet/IP layers CIP on top of standard TCP/IP and UDP/IP:

[Diagram: see online version]

Encapsulation Header

Every EtherNet/IP message begins with a 24-byte encapsulation header:

[Diagram: see online version]

Key Fields

Field	Size	Description
Command	16 bits	Encapsulation command code
Length	16 bits	Length of command-specific data (bytes following this header)
Session Handle	32 bits	Session identifier (assigned by target on RegisterSession)
Status	32 bits	Status code (0 = success)
Sender Context	64 bits	Originator context (echoed back in response)
Options	32 bits	Reserved (must be 0)

Encapsulation Commands

Code	Name	Transport	Description
0x0001	ListTargets	TCP	List accessible targets (legacy)

0x0004	ListServices	TCP	Query supported services (CIP encapsulation)
0x0063	ListIdentity	TCP/UDP	Discover devices on the network
0x0064	ListInterfaces	TCP	List interface information
0x0065	RegisterSession	TCP	Open a session (get session handle)
0x0066	UnRegisterSession	TCP	Close a session
0x006F	SendRRData	TCP	Send explicit CIP request/response (Request-Reply)
0x0070	SendUnitData	TCP	Send connected explicit message

CIP (Common Industrial Protocol)

CIP is the application-layer protocol shared by EtherNet/IP, DeviceNet, and ControlNet. It uses an object-oriented model where every device is a collection of objects, each with attributes and services.

CIP Message Structure

[Diagram: see online version]

Common CIP Services

Code	Name	Description
0x01	Get_Attribute_All	Read all attributes of an object instance
0x0E	Get_Attribute_Single	Read a single attribute
0x02	Set_Attribute_All	Write all attributes
0x10	Set_Attribute_Single	Write a single attribute
0x03	Get_Attribute_List	Read a list of attributes
0x04	Set_Attribute_List	Write a list of attributes
0x05	Reset	Reset the device or object
0x08	Create	Create an object instance
0x09	Delete	Delete an object instance
0x4B	Read_Tag	Read a tag value (Logix-specific)
0x4C	Read_Tag_Fragmented	Read a large tag value in fragments
0x4D	Write_Tag	Write a tag value (Logix-specific)
0x4E	Write_Tag_Fragmented	Write a large tag value in fragments
0x52	Forward_Open	Establish a CIP connection

0x4E	Forward_Close	Tear down a CIP connection

Common CIP Objects

Class ID	Name	Description
0x01	Identity	Device name, vendor, serial number
0x02	Message Router	Routes CIP messages to target objects
0x04	Assembly	Groups I/O data into assemblies
0x06	Connection Manager	Manages CIP connections (Forward_Open/Close)
0xF5	TCP/IP Interface	IP address, gateway, DNS configuration
0xF6	Ethernet Link	MAC address, link speed, counters

Explicit Messaging (TCP)

Explicit messaging is used for configuration, diagnostics, and non-time-critical operations. It follows a request-response pattern over a TCP session:

[Diagram: see online version]

Implicit Messaging (UDP -- I/O Connections)

Implicit messaging is for real-time cyclic I/O data exchange. A CIP connection is first established via Forward_Open over TCP, then I/O data flows over UDP:

[Diagram: see online version]

Implicit Message (Common Packet Format)

[Diagram: see online version]

Connection Parameters

Parameter	Description
RPI (Requested Packet Interval)	Cyclic update rate in microseconds (typical: 2-100 ms)
O->T Size	Originator-to-Target data size (outputs)
T->O Size	Target-to-Originator data size (inputs)
Connection Type	Point-to-point or multicast
Transport Class	Class 0 (UDP, no sequence), Class 1 (UDP, sequenced), Class 3 (TCP)
Priority	Scheduled, high, low, urgent

Device Profiles

EtherNet/IP devices conform to CIP device profiles:

Device Type	Code	Description
AC Drive	0x02	Variable frequency drives
Motor Overload	0x03	Motor protection
Discrete I/O	0x07	Digital input/output modules
Analog I/O	0x0A	Analog input/output modules
PLC	0x0E	Programmable logic controllers
HMI	0x18	Human-machine interfaces
Managed Switch	0x2C	Managed Ethernet switches

Encapsulation

[Diagram: see online version]

Standards

Document	Title
IEC 61158	Industrial communication networks -- Fieldbus specifications
The EtherNet/IP Specification	ODVA EtherNet/IP Adaptation of CIP
The CIP Specification	Common Industrial Protocol (CIP)
IEEE 802.3	Standard Ethernet (physical/data link layer)

See Also

- Modbus -- simpler register-based industrial protocol
- PROFINET -- Siemens industrial Ethernet (comparable to EtherNet/IP)
- OPC UA -- platform-independent industrial interoperability
- Ethernet -- underlying data link layer

IEC 60870-5-104

Standard: IEC 60870-5-104 | **Layer:** Application (Layer 7) | **Wireshark filter:** `iec60870_104`

IEC 60870-5-104 (commonly called IEC 104) is a SCADA telecontrol protocol used by power utilities, water systems, and other critical infrastructure for communication between control centers and remote terminal units (RTUs) or substations. It adapts the serial-based IEC 60870-5-101 protocol for TCP/IP networks, preserving the same application-layer ASDU (Application Service Data Unit) format while adding a TCP transport layer. IEC 104 runs on TCP port 2404 and provides balanced (bidirectional) communication with built-in link supervision.

APDU Structure

The Application Protocol Data Unit (APDU) consists of an APCI (Application Protocol Control Information) header followed by an optional ASDU. There are three APDU formats: I-format (information transfer), S-format (supervisory), and U-format (unnumbered control).

I-Format APDU (Information Transfer)

[Diagram: see online version]

S-Format APDU (Supervisory)

[Diagram: see online version]

U-Format APDU (Unnumbered Control)

[Diagram: see online version]

APCI Key Fields

Field	Size	Description
Start byte	8 bits	Always 0x68
APDU Length	8 bits	Length of remaining bytes (max 253)
Control Field 1	8 bits	Determines format: bit 0 = 0 for I-format, bits 0-1 = 01 for S-format, bits 0-1 = 11 for U-format
Control Field 2	8 bits	Continuation of control
Control Field 3	8 bits	Receive sequence number (low byte)
Control Field 4	8 bits	Receive sequence number (high byte)

APDU Format Identification

Bit Pattern (Control Field 1)	Format	Purpose
xxxxxxx0	I-format	Information transfer (carries ASDU)
000000x1	S-format	Supervisory acknowledgment

000000x1 (specific patterns)	U-format	Link control (STARTDT, STOPDT, TESTFR)

U-Format Commands

Control Byte	Command	Direction	Description
0x07	STARTDT act	Client -> Server	Activate data transfer
0x0B	STARTDT con	Server -> Client	Confirm data transfer activation
0x13	STOPDT act	Client -> Server	Stop data transfer
0x23	STOPDT con	Server -> Client	Confirm data transfer stop
0x43	TESTFR act	Either	Test frame (keepalive)
0x83	TESTFR con	Either	Test frame confirmation

ASDU Structure

The ASDU is carried inside I-format APDUs and contains the actual telecontrol data:

[Diagram: see online version]

ASDU Key Fields

Field	Size	Description
Type Identification	8 bits	Defines the type and format of information objects
Variable Structure Qualifier	8 bits	Bit 7: SQ flag (0 = each object has IOA, 1 = sequence); bits 0-6: number of objects
Cause of Transmission	8 bits	Why this ASDU was sent
Originator Address	8 bits	Source identifier (0 if unused)
Common Address (CASDU)	16 bits	Station/RTU address (1-65534; 0 = not used, 65535 = broadcast)
Information Object Address (IOA)	24 bits	Specific point address within the station
Information Element(s)	Variable	Data values depending on Type ID

Common Type Identifications

Monitor Direction (Server to Client)

Type ID	Name	Description

1	M_SP_NA_1	Single-point information (1 bit status)
3	M_DP_NA_1	Double-point information (2 bit status)
5	M_ST_NA_1	Step position information (transformer tap)
7	M_BO_NA_1	Bitstring of 32 bits
9	M_ME_NA_1	Measured value, normalized (-1.0 to +1.0)
11	M_ME_NB_1	Measured value, scaled (integer)
13	M_ME_NC_1	Measured value, short floating point (IEEE 754)
15	M_IT_NA_1	Integrated totals (counter values)
30	M_SP_TB_1	Single-point with CP56Time2a timestamp
31	M_DP_TB_1	Double-point with CP56Time2a timestamp
34	M_ME_TD_1	Measured normalized with CP56Time2a
36	M_ME_TF_1	Measured float with CP56Time2a

Control Direction (Client to Server)

Type ID	Name	Description
45	C_SC_NA_1	Single command (ON/OFF)
46	C_DC_NA_1	Double command (ON/OFF with transient states)
47	C_RC_NA_1	Regulating step command (HIGHER/LOWER)
48	C_SE_NA_1	Set-point command, normalized value
50	C_SE_NC_1	Set-point command, short floating point
58	C_SC_TA_1	Single command with CP56Time2a
59	C_DC_TA_1	Double command with CP56Time2a

System Commands

Type ID	Name	Description
100	C_IC_NA_1	Interrogation command (general or group)
101	C_CI_NA_1	Counter interrogation command
102	C_RD_NA_1	Read command
103	C_CS_NA_1	Clock synchronization command

104	C_TS_NA_1	Test command
105	C_RP_NA_1	Reset process command
107	C_TS_TA_1	Test command with CP56Time2a

Cause of Transmission (CoT)

Code	Name	Description
1	Periodic/cyclic	Periodic transmission
2	Background	Background scan
3	Spontaneous	Spontaneous value change
4	Initialized	After station initialization
5	Request	Response to a request or interrogation
6	Activation	Command activation
7	Activation confirm	Confirmation of activation
8	Deactivation	Command deactivation
9	Deactivation confirm	Confirmation of deactivation
10	Activation termination	End of command execution
20	Interrogated by station	Response to general interrogation
37	Interrogated by group 1	Response to group 1 interrogation
44	Unknown type ID	Negative: type not supported
45	Unknown CoT	Negative: cause not supported
46	Unknown CASDU	Negative: address not known
47	Unknown IOA	Negative: object address not known

Connection Startup

[Diagram: see online version]

Connection Parameters

Parameter	Default	Description
t0	30 s	TCP connection establishment timeout
t1	15 s	Timeout for send or test APDUs
t2	10 s	Timeout for acknowledging received I-format APDUs (t2 < t1)
t3	20 s	Timeout for sending TESTFR if no data activity

k	12	Max outstanding unacknowledged I-format APDUs sent
w	8	Max I-format APDUs received before sending S-format ACK (w < k)

Quality Descriptor

Each information object includes quality bits:

Bit	Name	Description
0	OV	Overflow
4	BL	Blocked
5	SB	Substituted
6	NT	Not topical (value not current)
7	IV	Invalid (value is not usable)

IEC 101 vs IEC 104

Feature	IEC 60870-5-101	IEC 60870-5-104
Transport	Serial (RS-232/485)	TCP/IP (port 2404)
Link layer	FT1.2 framing	APCI (start 0x68)
Addressing	Link address + CASDU	IP address + CASDU
Mode	Unbalanced or balanced	Balanced only
ASDU format	Same	Same
Type IDs	Same set	Same set
Flow control	Link-level	Sequence numbers (N(S)/N(R))

Encapsulation

[Diagram: see online version]

Standards

Document	Title
IEC 60870-5-104	Telecontrol equipment and systems -- Network access for IEC 60870-5-101
IEC 60870-5-101	Companion standard for basic telecontrol tasks
IEC 60870-5-5	Basic application functions

IEC 62351-3	Security: TLS profiles for TCP/IP-based protocols

See Also

- Modbus -- simpler industrial protocol, often complementary
- IEC 61850 -- modern substation automation standard

IEC 61850

Standard: IEC 61850 | **Layer:** Application / Data Link (varies by service) | **Wireshark filter:** `goose` or `sv` or `mms`

IEC 61850 is the international standard for communication in electrical substations and smart grids. It defines a unified data model, configuration language, and multiple communication services mapped to different protocols: MMS (Manufacturing Message Specification) over TCP/IP for client-server SCADA, GOOSE (Generic Object Oriented Substation Event) over Ethernet for fast peer-to-peer event distribution, and Sampled Values (SV) over Ethernet for streaming measurement data. IEC 61850 replaces legacy protocols with a single, object-oriented approach to power system automation.

Protocol Stack

[Diagram: see online version]

GOOSE Frame

GOOSE is a Layer 2 multicast protocol (EtherType 0x88B8) for fast event distribution with less than 4 ms delivery. It is used for tripping signals, interlocking, and protection coordination.

[Diagram: see online version]

GOOSE Key Fields

Field	Size	Description
Destination MAC	48 bits	Multicast address (01-0C-CD-01-xx-xx)
Source MAC	48 bits	Sending IED MAC address
VLAN Tag	32 bits	Optional 802.1Q priority tagging (typically priority 4)
EtherType	16 bits	0x88B8 identifies GOOSE
APPID	16 bits	Application identifier (0x0000-0x3FFF)
Length	16 bits	Total length from APPID to end of PDU

Reserved 1	16 bits	Reserved (simulation bit in Edition 2)
Reserved 2	16 bits	Reserved
GOOSE PDU	Variable	ASN.1 BER encoded payload

GOOSE PDU Fields (ASN.1 BER)

Field	Description
gocbRef	GOOSE Control Block reference (e.g., `IED1LD0/LLN0$GO$gcb01`)
timeAllowedtoLive	Milliseconds before GOOSE is considered stale
datSet	Dataset reference containing the values
goID	GOOSE identifier string
t	Timestamp (UTC) of the event
stNum	State number (increments on value change)
sqNum	Sequence number (increments with each retransmission, resets to 0 on stNum change)
test	Test flag (TRUE = test mode)
confRev	Configuration revision
ndsCom	Needs commissioning flag
numDatSetEntries	Number of data entries
allData	Ordered list of data values

GOOSE Retransmission

GOOSE uses a retransmission scheme with exponential backoff to ensure reliability without acknowledgments:

[Diagram: see online version]

Sampled Values (SV) Frame

SV (EtherType 0x88BA) streams digitized current and voltage samples from merging units to protection relays at rates of 4000 or 4800 samples/second per channel.

[Diagram: see online version]

SV Key Fields

Field	Size	Description
EtherType	16 bits	0x88BA identifies Sampled Values
APPID	16 bits	Application identifier (0x4000-0x7FFF)

Length	16 bits	Total length from APPID to end of PDU
savPdu	Variable	Contains one or more ASDU sequences
svID	String	Sampled Value stream identifier
smpCnt	16 bits	Sample counter (0-4799 for 4800 smp/s)
confRev	32 bits	Configuration revision
smpSynch	8 bits	Synchronization quality (0=none, 1=local, 2=global)
seqData	Variable	Encoded measurement samples (I_a, I_b, I_c, I_n, V_a, V_b, V_c, V_n)

Sampling Rates

Standard	Rate	Application
IEC 61850-9-2	4000 samples/s	Protection
IEC 61850-9-2	4800 samples/s	Power quality / metering
IEC 61850-9-2LE	80 samples/cycle	Simplified profile (4000 or 4800 smp/s)

MMS (Manufacturing Message Specification)

MMS (ISO 9506) provides client-server access to the data model over TCP/IP (port 102 via TPKT/COTP). SCADA systems use MMS to read values, write setpoints, and manage devices.

MMS Services

Service	Description
GetNameList	Browse available objects (Logical Devices, Logical Nodes, Data Objects)
Read	Read one or more data attribute values
Write	Write a value to a data attribute
GetVariableAccessAttributes	Query the type and structure of a data object
DefineNamedVariableList	Create a named list of variables for grouped access
GetNamedVariableListAttributes	Query the contents of a named variable list
Report (BRCB/URCB)	Subscribe to data change reports (buffered or unbuffered)
Control	Operate / Select-Before-Operate / Cancel commands
GetServerDirectory	Discover the data model of an IED
FileOpen / FileRead / FileClose	Access disturbance recordings and configuration files

MMS Client-Server Flow

[Diagram: see online version]

Data Model

IEC 61850 organizes all substation data in a hierarchical object model:

[Diagram: see online version]

Common Logical Node Classes

Prefix	Category	Examples
L	System LNs	LLN0 (logical node zero), LPHD (physical device)
A	Automatic Control	ATCC (tap changer controller), ARCO (reactive power)
C	Supervisory Control	CSWI (switch controller), CILO (interlocking)
G	Generic References	GGIO (generic I/O)
I	Interfacing	IHMI (human-machine interface)
M	Metering/Measurement	MMXU (measurement), MMTR (metering), MSQI (sequence/imbalance)
P	Protection	PTOC (overcurrent), PDIF (differential), PTOV (overvoltage), PTUV (undervoltage)
R	Protection-Related	RREC (auto-reclosing), RBRF (breaker failure)
S	Sensors/Monitoring	SIML (insulation monitor), SARC (arc detection)
T	Instrument Transformers	TCTR (current transformer), TVTR (voltage transformer)
X	Switchgear	XCBR (circuit breaker), XSWI (disconnect switch)
Y	Power Transformers	YPTR (power transformer), YLTC (load tap changer)

Functional Constraints (FC)

FC	Category	Description
ST	Status	Runtime status values (read-only)
MX	Measured Values	Analog measurements
CO	Control	Operable points (commands)
SP	Setting	Configurable parameters

SG	Setting Group	Active setting group
CF	Configuration	Fixed configuration data
DC	Description	Textual descriptions and naming
EX	Extended	Extended definition

SCL (Substation Configuration Language)

SCL is the XML-based configuration language defined in IEC 61850-6. It describes the complete substation model, communication configuration, and IED capabilities.

SCL File Types

Extension	Name	Description
.icd	IED Capability Description	IED template from manufacturer
.ssd	System Specification Description	Substation single-line diagram and LN allocation
.scd	Substation Configuration Description	Complete configured system (all IEDs + communication)
.cid	Configured IED Description	Final per-IED configuration for download
.sed	System Exchange Description	Information exchange between projects

Edition 1 vs Edition 2

Feature	Edition 1 (2003)	Edition 2 (2010+)
Data model	Basic LN classes	Extended LN classes, new domains (hydro, DER, wind)
Services	Core services	Enhanced security, time sync, service tracking
GOOSE simulation	Not supported	Simulation bit in Reserved 1 field
Redundancy	Not standardized	PRP / HSR (IEC 62439-3)
Security	Not addressed	Role-based access, TLS, GDSP (IEC 62351)
SCL	Version 1.0	Version 2007, extended schema
Testing	Basic conformance	Edition 2 conformance with PIXIT/PICS

Encapsulation

[Diagram: see online version]

Standards

Document	Title
IEC 61850-5	Communication requirements for functions and device models
IEC 61850-6	Configuration description language (SCL)
IEC 61850-7-1	Principles and models
IEC 61850-7-2	Abstract communication service interface (ACSI)
IEC 61850-7-3	Common data classes
IEC 61850-7-4	Compatible logical node classes and data object classes
IEC 61850-8-1	Mapping to MMS and Ethernet
IEC 61850-9-2	Sampled Values over Ethernet
IEC 62351	Security for IEC 61850 and other power system protocols
IEC 62439-3	PRP and HSR network redundancy

See Also

- Modbus -- widely used industrial protocol often found alongside IEC 61850
- PROFIBUS -- alternative industrial fieldbus
- Ethernet -- transport for GOOSE and SV

ISO 8583 -- Financial Transaction Card Messaging

Standard: ISO 8583:2003 | **Layer:** Application (Layer 7) | **Wireshark filter:** Limited (typically over proprietary transports)

ISO 8583 is the international standard for financial transaction card-originated messages. It defines the message format used between card-accepting devices (ATMs, POS terminals) and card issuers for authorization, clearing, and settlement of payment card transactions. Virtually every swipe, dip, or tap of a credit or debit card worldwide generates ISO 8583 messages. The protocol is implemented by payment networks (Visa, Mastercard, UnionPay), acquirer processors, and issuing banks, typically carried over proprietary TCP-based transports with TLS encryption.

Message Structure

Every ISO 8583 message consists of three parts: a Message Type Indicator, one or two Bitmaps, and the Data Elements indicated by the bitmap.

[Diagram: see online version]

Field	Size	Description

MTI	4 bytes	Message type: version, class, function, origin
Primary Bitmap	8 bytes (64 bits)	Indicates presence of data elements 1-64
Secondary Bitmap	8 bytes (64 bits)	Indicates presence of data elements 65-128 (present if bit 1 is set)
Data Elements	Variable	Transaction data fields as indicated by bitmap

Message Type Indicator (MTI)

The MTI is a 4-digit code where each digit has a specific meaning:

[Diagram: see online version]

Version (Digit 1)

Value	Meaning
0	ISO 8583:1987
1	ISO 8583:1993
2	ISO 8583:2003

Message Class (Digit 2)

Value	Meaning
1	Authorization
2	Financial (presentment/clearing)
3	File actions
4	Reversal / chargeback
5	Reconciliation
6	Administrative
8	Network management

Function (Digit 3)

Value	Meaning
0	Request
1	Request response
2	Advice

3	Advice response
4	Notification
5	Notification acknowledgment

Origin (Digit 4)

Value	Meaning
0	Acquirer
1	Acquirer repeat
2	Issuer
3	Issuer repeat
4	Other
5	Other repeat

Common MTI Values

MTI	Description
0100	Authorization request
0110	Authorization response
0200	Financial request (purchase)
0210	Financial response
0400	Reversal request
0410	Reversal response
0420	Reversal advice
0430	Reversal advice response
0800	Network management request
0810	Network management response

Bitmap

Each bit in the bitmap corresponds to a data element number. If the bit is set (1), that data element is present in the message. Bit 1 of the primary bitmap indicates whether a secondary bitmap follows.

```
Primary Bitmap:   bits 1-64  (8 bytes)
Secondary Bitmap: bits 65-128 (8 bytes, present if bit 1 = 1)
```

Example: A bitmap of `F2 3C 44 81 08 E0 80 00` indicates that data elements 1, 2, 3, 4, 7, 11, 12, 13, 14, 22, 25, 32, 35, 37, 38, 39, 41, 42, 43, 49, and 52 are present.

Key Data Elements

DE	Name	Format	Description
2	Primary Account Number (PAN)	n..19	Card number (up to 19 digits)
3	Processing Code	n 6	Transaction type + account types (e.g., 00 = purchase, 01 = cash advance, 20 = refund)
4	Amount, Transaction	n 12	Transaction amount in minor units (cents)
7	Transmission Date & Time	n 10	MMDDhhmmss
11	System Trace Audit Number (STAN)	n 6	Unique transaction identifier from terminal
12	Local Transaction Time	n 6	hhmmss
13	Local Transaction Date	n 4	MMDD
14	Expiration Date	n 4	YYMM
22	POS Entry Mode	n 3	How card data was read (e.g., 05 = chip, 07 = contactless, 90 = magstripe)
23	Card Sequence Number	n 3	Distinguishes cards with same PAN
25	POS Condition Code	n 2	Transaction environment (e.g., 00 = normal, 08 = mail/phone order, 59 = e-commerce)
32	Acquiring Institution ID	n..11	Acquirer identifier
35	Track 2 Data	z..37	Magnetic stripe track 2 equivalent data
37	Retrieval Reference Number (RRN)	an 12	Unique reference for reconciliation
38	Authorization ID Response	an 6	Auth code returned by issuer
39	Response Code	an 2	Action code (00 = approved, 05 = declined, etc.)
41	Card Acceptor Terminal ID	ans 8	Terminal identifier
42	Card Acceptor ID Code	ans 15	Merchant identifier
43	Card Acceptor Name/Location	ans 40	Merchant name and location
49	Currency Code, Transaction	n 3	ISO 4217 currency code (840 = USD, 978 = EUR)
52	PIN Block	b 8	Encrypted PIN data (8 bytes binary)

54	Additional Amounts	ans..120	Balances, surcharges, cashback
55	ICC/EMV Data	ans..999	Chip card data (TLV-encoded EMV tags)

Response Codes (DE39)

Code	Meaning
00	Approved
01	Refer to card issuer
03	Invalid merchant
05	Do not honor (declined)
12	Invalid transaction
13	Invalid amount
14	Invalid card number
30	Format error
41	Lost card -- pick up
43	Stolen card -- pick up
51	Insufficient funds
54	Expired card
55	Incorrect PIN
57	Transaction not permitted
61	Exceeds withdrawal limit
65	Exceeds frequency limit
75	PIN tries exceeded
91	Issuer unavailable
96	System malfunction

Authorization Flow

[Diagram: see online version]

Reversal Flow

[Diagram: see online version]

POS Entry Mode (DE22)

Code	Description
01	Manual key entry
02	Magnetic stripe read
05	Chip card read (ICC)
07	Contactless chip (NFC/EMV)
10	Credential on file
79	Chip fallback to magnetic stripe
80	Chip fallback to manual entry
90	Magnetic stripe -- full track data
91	Contactless magnetic stripe

Processing Code (DE3)

The 6-digit processing code encodes the transaction type and the accounts affected:

Digits	Meaning
1-2	Transaction type: 00 = purchase, 01 = cash advance, 09 = purchase with cashback, 20 = refund, 30 = balance inquiry
3-4	From account: 00 = default, 10 = savings, 20 = checking, 30 = credit
5-6	To account: same as above

Encoding

ISO 8583 does not mandate a single encoding; implementations vary:

Format	Description
Binary	Fields packed as raw binary values
BCD (packed)	Digits packed two per byte (nibble-encoded)
ASCII	Digits as ASCII characters (one byte per digit)
EBCDIC	Used on IBM mainframes

Variable-length fields include a length prefix (LLVAR = 2-digit length, LLLVAR = 3-digit length) before the data.

Standards

Document	Title

ISO 8583:2003	Financial transaction card originated messages -- Interchange message specifications
ISO 8583-1:2003	Part 1: Messages, data elements and code values
ISO 8583-2:1998	Part 2: Application and registration procedures for ICCs
ISO 8583-3:2003	Part 3: Maintenance procedures
EMV Specifications	Chip card data carried in DE55

See Also

- TLS -- transport encryption for ISO 8583 connections
- TCP -- underlying transport for online authorization
- NFC -- contactless card interface (EMV Contactless)

KNX

Standard: ISO/IEC 14543-3 / EN 50090 | **Layer:** Full stack (Physical through Application) | **Wireshark filter:** `knxnetip` or `knx`

KNX is the leading open standard for building and home automation in Europe and worldwide. It connects lighting, blinds, HVAC, security, energy management, and audio/video systems. KNX emerged from the merger of three earlier standards (EIB, BatiBUS, EHS) and is managed by the KNX Association. Devices communicate via a common bus using a producer-consumer model with group addresses, enabling multi-vendor interoperability through standardized Datapoint Types (DPTs).

Media

KNX supports multiple physical media, all carrying the same application-layer telegrams:

Medium	Designation	Speed	Topology	Notes
Twisted Pair	KNX TP	9600 bps	Free (bus/tree/star)	Most common; bus-powered (29V DC)
Powerline	KNX PL	1200 bps	Existing mains wiring	Spread-frequency shift keying
Radio Frequency	KNX RF	16384 bps	Mesh / star	868 MHz (Europe)
IP	KNXnet/IP	Ethernet speed	LAN / WAN	Tunnelling and routing over UDP

KNXnet/IP Header (UDP port 3671)

KNXnet/IP encapsulates KNX telegrams over IP networks. All KNXnet/IP packets share a common header:

[Diagram: see online version]

Field	Size	Description
Header Length	8 bits	Always 0x06 (6 bytes)
Protocol Version	8 bits	Always 0x10 (v1.0)
Service Type ID	16 bits	Identifies the KNXnet/IP service
Total Length	16 bits	Total frame length including header (bytes)
Body	Variable	Service-specific data

Service Type IDs

Code	Service	Description
0x0201	SEARCH_REQUEST	Discover KNXnet/IP servers on LAN
0x0202	SEARCH_RESPONSE	Server responds with description
0x0205	CONNECT_REQUEST	Open a tunnelling or routing connection
0x0206	CONNECT_RESPONSE	Server confirms connection
0x0207	CONNECTIONSTATE_REQUEST	Heartbeat check
0x0208	CONNECTIONSTATE_RESPONSE	Heartbeat reply
0x0209	DISCONNECT_REQUEST	Tear down connection
0x020A	DISCONNECT_RESPONSE	Confirm disconnect
0x0420	TUNNELLING_REQUEST	Tunnelled cEMI frame (point-to-point)
0x0421	TUNNELLING_ACK	Acknowledge tunnelled frame
0x0530	ROUTING_INDICATION	Multicast cEMI frame (routing)
0x0531	ROUTING_LOST_MESSAGE	Notification of lost multicast frames

cEMI Frame (Common External Message Interface)

The cEMI frame is the standard format for KNX telegrams exchanged over KNXnet/IP:

[Diagram: see online version]

Field	Size	Description
Message Code	8 bits	0x11 = L_Data.req, 0x29 = L_Data.ind, 0x2E = L_Data.con
Additional Info Length	8 bits	Length of optional additional info (typically 0x00)
Ctrl1	8 bits	Frame type, repeat, broadcast, priority, ack request

Ctrl2	8 bits	Address type (individual/group), hop count, extended frame format
Source Address	16 bits	Individual address of sender (Area.Line.Device)
Destination Address	16 bits	Individual or group address of target
Data Length	8 bits	Length of APDU in bytes
APDU	Variable	Application Protocol Data Unit (APCI + data)

Ctrl1 Byte

Bit	Name	Description
7	Frame Type	1 = standard frame, 0 = extended frame
6	--	Reserved
5	Repeat	0 = repeat on error, 1 = do not repeat
4	System Broadcast	1 = domain broadcast, 0 = system broadcast
3-2	Priority	00 = system, 01 = alarm, 10 = high, 11 = low
1	Ack Request	0 = no L2 ack, 1 = request ack
0	Confirm	0 = no error (positive confirm)

Ctrl2 Byte

Bit	Name	Description
7	Address Type	0 = individual (unicast), 1 = group (multicast)
6-4	Hop Count	Remaining hops (decremented by routers, default 6)
3-0	Extended Frame Format	0x0 = standard frame

Addressing

Individual Address (Area.Line.Device)

Each KNX device has a unique individual address encoded in 16 bits:

[Diagram: see online version]

Field	Bits	Range	Example
Area	4	1-15	Building floor

Line	4	0-15	Room / segment
Device	8	0-255	Device on the line

Example: `1.2.3` = Area 1, Line 2, Device 3

Group Address

Group addresses connect function points (e.g., a switch to a light). They can be 2-level or 3-level:

3-Level (most common):

[Diagram: see online version]

Notation	Meaning	Example
Main/Middle/Sub	3-level group	1/2/3 = Lighting / Floor 2 / Lamp 3
Main/Sub	2-level group	1/3 = Lighting / Lamp 3
Free	16-bit value	0-65535

APCI Types (Application Protocol Control Information)

The APCI field identifies the service requested in the telegram:

APCI Code	Name	Description
0x000	GroupValueRead	Request the current value of a group object
0x040	GroupValueResponse	Reply with the current value
0x080	GroupValueWrite	Write a new value to a group object
0x0C0	IndividualAddrWrite	Set a device's individual address (during commissioning)
0x100	IndividualAddrRead	Query a device's individual address
0x140	IndividualAddrResponse	Reply with individual address
0x300	ADCRead	Read ADC channel
0x3C0	MemoryRead	Read device memory
0x3C8	MemoryResponse	Reply with memory contents
0x3D0	MemoryWrite	Write to device memory
0x3E0	DeviceDescrRead	Read device descriptor

Datapoint Types (DPTs)

DPTs standardize how values are encoded in KNX telegrams, ensuring interoperability between different manufacturers:

DPT	Name	Size	Encoding	Example
1.xxx	Boolean	1 bit	0/1	Switch On/Off, True/False
2.xxx	Control + Boolean	2 bits	Control + value	Dimming step
3.xxx	3-bit Control	4 bits	Control + 3-bit step	Dimmer (brighter/darker)
5.xxx	8-bit Unsigned	1 byte	0-255	Scaling (0-100%), angle (0-360)
7.xxx	16-bit Unsigned	2 bytes	0-65535	Pulse counter
9.xxx	16-bit Float	2 bytes	0.01 resolution float	Temperature (-273..+670760)
10.xxx	Time	3 bytes	Day + hour:min:sec	Time of day
11.xxx	Date	3 bytes	Day/month/year	Date
12.xxx	32-bit Unsigned	4 bytes	0-4294967295	Counter
13.xxx	32-bit Signed	4 bytes	Signed integer	Energy (Wh)
14.xxx	32-bit Float	4 bytes	IEEE 754	Electrical current (A)
16.xxx	String	14 bytes	ASCII / ISO 8859-1	Text display

Communication Flow

GroupValueWrite (Switching a Light)

[Diagram: see online version]

GroupValueRead (Status Request)

[Diagram: see online version]

KNXnet/IP Tunnelling Connection

[Diagram: see online version]

Topology

[Diagram: see online version]

- **Backbone:** connects areas (typically KNXnet/IP or TP)
- **Area:** up to 15 lines, connected via area router
- **Line:** up to 256 devices, connected via line router
- **Maximum:** 15 areas x 15 lines x 256 devices = 57,600 devices

Standards

Document	Title
ISO/IEC 14543-3	Home Electronic Systems (HES) -- KNX
EN 50090	Home and Building Electronic Systems (HBES)
KNX Standard v2.1	Full KNX system specification
KNXnet/IP	KNX communication over IP networks

See Also

- Modbus -- industrial automation protocol, simpler register-based model
- PROFIBUS -- European fieldbus for factory and process automation
- Zigbee -- wireless mesh protocol also used in home automation
- BLE -- Bluetooth Low Energy, another smart-home transport

Modbus

Standard: Modbus Protocol Specification (Modbus.org) | **Layer:** Application (Layer 7) | **Wireshark filter:** `modbus` or `mbtcp`

Modbus is the most widely used industrial communication protocol, connecting PLCs, sensors, actuators, HMIs, and SCADA systems. Developed by Modicon (now Schneider Electric) in 1979, its simplicity and open specification led to near-universal adoption. Modbus defines a master-slave request-response model with a simple register-based data model. It runs over serial (RTU/ASCII on RS-485) and TCP/IP (Modbus TCP).

Variants

Variant	Transport	Framing	Error Check
Modbus RTU	RS-485 / RS-232	Binary, timing-based	CRC-16
Modbus ASCII	RS-485 / RS-232	ASCII hex, `:` delimited	LRC
Modbus TCP	TCP/IP (port 502)	MBAP header	TCP checksum

Modbus RTU Frame

[Diagram: see online version]

Field	Size	Description
Device Address	8 bits	Slave address (1-247); 0 = broadcast

Function Code	8 bits	Operation to perform
Data	Variable	Request/response parameters
CRC-16	16 bits	Cyclic redundancy check (LSB first)

RTU frames are delimited by silence gaps of >= 3.5 character times (~3.6 ms at 9600 baud).

Modbus TCP Frame (MBAP Header)

[Diagram: see online version]

Field	Size	Description
Transaction ID	16 bits	Matches requests to responses
Protocol ID	16 bits	Always 0x0000 for Modbus
Length	16 bits	Bytes following (Unit ID + PDU)
Unit ID	8 bits	Slave address (for serial gateway bridging)
Function Code	8 bits	Operation to perform
Data	Variable	Request/response parameters

No CRC -- TCP provides error detection.

Function Codes

Code	Name	Description
0x01	Read Coils	Read 1-2000 discrete outputs (bits)
0x02	Read Discrete Inputs	Read 1-2000 discrete inputs (bits)
0x03	Read Holding Registers	Read 1-125 registers (16-bit words)
0x04	Read Input Registers	Read 1-125 input registers (16-bit words)
0x05	Write Single Coil	Write one discrete output (0x0000 or 0xFF00)
0x06	Write Single Register	Write one 16-bit register
0x0F	Write Multiple Coils	Write 1-1968 discrete outputs
0x10	Write Multiple Registers	Write 1-123 registers
0x17	Read/Write Multiple Registers	Atomic read + write in one transaction
0x2B	Read Device Identification	Query device model, vendor, etc.

Exception Responses

If a request fails, the slave returns the function code with bit 7 set (0x80 + function code) and an exception code:

Code	Name	Description
0x01	Illegal Function	Function code not supported
0x02	Illegal Data Address	Register address out of range
0x03	Illegal Data Value	Value out of range
0x04	Server Device Failure	Unrecoverable error
0x06	Server Device Busy	Slave is processing a long operation

Data Model

Type	Address Range	Access	Size	Description
Coils	00001-09999	Read/Write	1 bit	Discrete outputs (relays, LEDs)
Discrete Inputs	10001-19999	Read only	1 bit	Discrete inputs (switches, sensors)
Input Registers	30001-39999	Read only	16 bits	Analog inputs (ADC values)
Holding Registers	40001-49999	Read/Write	16 bits	Configuration, setpoints, outputs

Note: Addresses in the PDU are 0-based (0x0000-0xFFFF). The 5-digit convention above is for documentation only.

Request/Response Example

Read Holding Registers (0x03)

[Diagram: see online version]

Communication

RTU Master-Slave

[Diagram: see online version]

TCP Client-Server

[Diagram: see online version]

Encapsulation

[Diagram: see online version]

Standards

Document	Title
Modbus Application Protocol	Modbus Application Protocol Specification V1.1b3
Modbus over TCP/IP	Modbus Messaging on TCP/IP Implementation Guide V1.0b
Modbus over Serial	Modbus over Serial Line Specification V1.02
Modbus/TCP Security	Modbus/TCP Security Protocol Specification

See Also

- RS-485 -- physical layer for Modbus RTU
- TCP -- transport for Modbus TCP
- PROFIBUS -- alternative industrial fieldbus
- UART -- framing for Modbus RTU/ASCII

[Error processing nmea.md: paragraph text '<para><font face="Courier" color="#c62828"></font> followed by 2-char hex XOR of all bytes between <font face="Courier" color="#c62828">$</font> and <font face="Courier" color="#c62828"></font></para>' caused exception Parse error: saw </font> instead of expected]

OCPP -- Open Charge Point Protocol

Standard: Open Charge Alliance OCPP 2.0.1 | **Layer:** Application (Layer 7) | **Wireshark filter:** `websocket`

OCPP is the dominant open protocol for communication between electric vehicle (EV) charging stations and central management systems (CSMS). Developed by the Open Charge Alliance, it enables operators to remotely monitor, configure, and control charge points -- starting/stopping sessions, collecting meter data, managing firmware, and authorizing users. OCPP 1.6 (the most widely deployed version) supports SOAP or JSON over WebSocket. OCPP 2.0.1 uses JSON over WebSocket exclusively and adds ISO 15118 Plug & Charge, improved security profiles, and a comprehensive device model.

Architecture

[Diagram: see online version]

The Charge Point initiates a persistent WebSocket connection to the Central System. The Charge Point identity is encoded in the WebSocket URI path (e.g., `wss://csms.example.com/ocpp/CP001`).

Message Structure (OCPP-J / JSON)

OCPP-J defines three message types as JSON arrays:

CALL (Request)

[Diagram: see online version]

```
[2, "19223201", "BootNotification", {"chargePointVendor":"Vendor","chargePointModel":"Model"}]
```

CALLRESULT (Response)

[Diagram: see online version]

```
[3, "19223201", {"status":"Accepted","currentTime":"2025-01-15T10:00:00Z","interval":300}]
```

CALLERROR (Error Response)

[Diagram: see online version]

```
[4, "19223201", "InternalError", "An unexpected error occurred", {}]
```

Message Types

MessageTypeId	Name	Description
2	CALL	Request from either party
3	CALLRESULT	Successful response matching a CALL's UniqueId
4	CALLERROR	Error response matching a CALL's UniqueId

Key Actions (OCPP 1.6)

Charge Point -> Central System

Action	Description
BootNotification	Charge point registers on startup; receives heartbeat interval
Heartbeat	Periodic keepalive; Central System returns current time
Authorize	Validate an RFID tag or ID token before charging
StartTransaction	Report that a charging session has begun (includes meter start)
StopTransaction	Report that a charging session has ended (includes meter stop, reason)
MeterValues	Periodic or clock-aligned energy/power meter readings

StatusNotification	Report connector status changes (Available, Occupied, Faulted, etc.)
DiagnosticsStatusNotification	Report diagnostics upload progress
FirmwareStatusNotification	Report firmware update progress

Central System -> Charge Point

Action	Description
RemoteStartTransaction	Remotely start a charging session
RemoteStopTransaction	Remotely stop a charging session
Reset	Soft or hard reset of the charge point
ChangeConfiguration	Change a configuration key (e.g., heartbeat interval)
GetConfiguration	Read configuration keys
UnlockConnector	Remotely unlock a connector (e.g., stuck cable)
UpdateFirmware	Instruct charge point to download and install firmware
GetDiagnostics	Request upload of diagnostic logs
ChangeAvailability	Set connector to operative or inoperative
SetChargingProfile	Apply a charging schedule (power limits over time)
ClearChargingProfile	Remove a charging profile
TriggerMessage	Request charge point to send a specific message immediately

Charging Session Flow

[Diagram: see online version]

Boot Notification Flow

[Diagram: see online version]

Connector Statuses

Status	Description
Available	Connector is free and operative
Preparing	Cable plugged in, awaiting authorization
Charging	Actively delivering energy
SuspendedEVSE	Charging suspended by charge point (e.g., load balancing)
SuspendedEV	Charging suspended by vehicle (battery management)

Finishing	Transaction stopping, cable still connected
Reserved	Connector reserved for a specific user
Unavailable	Connector is inoperative
Faulted	Hardware or communication error

OCPP 2.0.1 Enhancements

Feature	Description
ISO 15118 (Plug & Charge)	Certificate-based EV authentication -- no RFID needed
Device Model	Structured component/variable model replaces flat configuration keys
Security Profiles	Four defined security levels (see below)
Display Messages	Central system can push messages to charge point display
Transaction Events	Richer transaction lifecycle (Started, Updated, Ended)
Cost Updates	Real-time running cost display to EV driver
Local Authorization Cache	Synchronized authorization list for offline operation
Reservations	Reserve a specific EVSE or connector

Security Profiles

Profile	Name	Description
0	Unsecured	No transport security (ws://) -- not recommended
1	Basic Authentication	HTTP Basic Auth over TLS (wss://)
2	TLS with Server Certificate	Charge point validates CSMS certificate
3	TLS with Client Certificate	Mutual TLS -- both sides present X.509 certificates

Error Codes

Code	Description
NotImplemented	Requested action is not implemented
NotSupported	Requested action is recognized but not supported
InternalError	Internal error in the receiving party
ProtocolError	Payload does not conform to the schema
SecurityError	Security policy violation

FormationViolation	Malformed message (not valid JSON array)
PropertyConstraintViolation	Property value violates a constraint
OccurrenceConstraintViolation	Required field missing or extra field present
TypeConstraintViolation	Field type does not match schema
GenericError	Any other error

OCPP 1.6 vs 2.0.1

Feature	OCPP 1.6	OCPP 2.0.1
Transport	SOAP or JSON/WebSocket	JSON/WebSocket only
Configuration	Flat key-value pairs	Structured device model (Component/Variable)
ISO 15118	Not supported	Native Plug & Charge support
Security	Basic Auth or TLS	Four defined security profiles
Transactions	Start/Stop model	Event-based (Started/Updated/Ended)
Display messages	Not supported	Push messages to EVSE screen
Smart charging	Basic profiles	Enhanced with cost updates

Encapsulation

[Diagram: see online version]

Standards

Document	Title
OCPP 2.0.1	Open Charge Point Protocol 2.0.1 Specification
OCPP 1.6	OCPP 1.6 (JSON & SOAP editions) -- most widely deployed
ISO 15118	Vehicle-to-Grid Communication Interface
OCPP Security Whitepaper	Security recommendations for OCPP implementations

See Also

- Modbus -- alternative industrial protocol (some older EVSE use Modbus)
- PROFIBUS -- industrial fieldbus

OPC DA -- OPC Data Access / OPC Classic

Standard: OPC DA 3.0 Specification (OPC Foundation) | **Layer:** Application (Layer 7) | **Wireshark filter:** `dcerpc`

OPC Data Access (OPC DA) is a set of COM/DCOM-based specifications for real-time data exchange between industrial automation devices and software applications. Developed by the OPC Foundation starting in 1996, OPC DA became the de facto standard for connecting SCADA, DCS, HMI, and historian systems to PLCs, RTUs, and field devices on Windows platforms. "OPC" originally stood for "OLE for Process Control" (OLE = Object Linking and Embedding). Despite being superseded by OPC UA (Unified Architecture), OPC DA remains widely deployed in legacy industrial environments. OPC DA relies on Microsoft DCOM (Distributed Component Object Model) for network communication, inheriting both its capabilities and its configuration complexity.

Architecture

OPC DA uses a three-tier object model built on COM:

[Diagram: see online version]

Object	Description
Server	Represents a connection to an OPC DA server process; manages groups
Group	A logical collection of items with shared update rate and deadband; manages items
Item	A single data point (tag) -- maps to a sensor, register, or variable in the device

OPC DA Interfaces (COM)

The OPC DA specification defines these key COM interfaces:

Interface	Description
`IOPCServer`	Create/remove groups, get server status, browse address space
`IOPCItemMgt`	Add, remove, and configure items within a group
`IOPCGroupStateMgt`	Get/set group properties (update rate, active state, deadband)
`IOPCSyncIO`	Synchronous read/write -- blocks until complete
`IOPCAsyncIO2`	Asynchronous read/write -- results delivered via callback
`IOPCDataCallback`	Client-side callback interface for async results and subscription data
`IOPCBrowseServerAddressSpace`	Browse available tags in the server's address space
`IConnectionPointContainer`	Standard COM interface for event subscription (advise/unadvise)

Item Properties

Each OPC item has the following properties:

Property	Description
Item ID	String identifier (tag name, e.g., `PLC1.Temperature.PV`)
Access Path	Optional hint for how to reach the item in the device
Value	Current data value (stored as a COM VARIANT)
Quality	Quality of the value (Good, Bad, Uncertain -- with substatus)
Timestamp	UTC time when the value was last updated
Data Type	Requested/canonical VARIANT type (VT_R4, VT_I2, VT_BOOL, VT_BSTR, etc.)
Access Rights	Readable, writable, or both

Quality Codes

Quality is a 16-bit field following the OPC quality model:

[Diagram: see online version]

Quality Bits

Value	Quality	Description
00	Bad	Value is not usable
01	Uncertain	Value is questionable
10	N/A	Reserved
11	Good	Value is reliable

Common Substatus Values

Quality	Substatus	Code	Description
Bad	Non-Specific	0x00	General bad quality
Bad	Config Error	0x04	Server configuration error
Bad	Not Connected	0x08	Input source not connected
Bad	Device Failure	0x0C	Hardware failure detected
Bad	Sensor Failure	0x10	Sensor calibration failure
Bad	Last Known	0x14	Communication lost; showing last good value
Bad	Comm Failure	0x18	Communication failure with device

Bad	Out of Service	0x1C	Item is not in service
Uncertain	Non-Specific	0x40	General uncertain quality
Uncertain	Last Usable	0x44	Value is old but may still be valid
Uncertain	Sensor Not Accurate	0x50	Sensor reading outside normal range
Uncertain	Engineering Units Exceeded	0x54	Value outside configured limits
Uncertain	Sub-Normal	0x58	Multiple sources, fewer than required are good
Good	Non-Specific	0xC0	Value is good
Good	Local Override	0xD8	Value set locally (manual mode)

Data Types (VARIANT Mapping)

OPC DA uses COM VARIANT types for data exchange:

VARIANT Type	Code	Industrial Use
VT_BOOL	0x000B	Digital I/O, alarms
VT_I1 / VT_UI1	0x0010 / 0x0011	8-bit integer values
VT_I2 / VT_UI2	0x0002 / 0x0012	16-bit registers (Modbus-style)
VT_I4 / VT_UI4	0x0003 / 0x0013	32-bit counters, accumulators
VT_R4	0x0004	32-bit float (temperature, pressure)
VT_R8	0x0005	64-bit float (high-precision measurements)
VT_BSTR	0x0008	String tags, device descriptions
VT_DATE	0x0007	Timestamps, schedule data
VT_ARRAY \	VT_*	0x2000+

Connect, Add Group, Read Flow

[Diagram: see online version]

OPC DA Specifications

The OPC Classic family includes several specifications:

Specification	Description
OPC DA 1.0	Original Data Access specification (1996)

OPC DA 2.05a	Added async I/O, connection points, browse
OPC DA 3.0	Added item deadband, browse filters, keep-alive
OPC HDA	Historical Data Access -- retrieve time-series history
OPC A&E	Alarms and Events -- subscribe to alarms, events, conditions
OPC DX	Data Exchange -- server-to-server data transfer
OPC XML-DA	XML/SOAP-based Data Access (web services variant)
OPC Commands	Execute commands on devices

DCOM Security Configuration

OPC DA's reliance on DCOM requires careful security configuration:

Setting	Description
Launch Permissions	Which users/groups can start the OPC server process
Access Permissions	Which users/groups can connect to a running OPC server
Authentication Level	None, Connect, Call, Packet, Packet Integrity, Packet Privacy
Identity	Account the OPC server runs as (Interactive User, Local System, custom)
DCOM Port Range	Configurable dynamic RPC port range (default: 1024-65535)
Firewall	Requires TCP 135 (RPC Endpoint Mapper) + dynamic ports

OPC DA vs OPC UA

Feature	OPC DA (Classic)	OPC UA
Transport	DCOM/DCE-RPC (Windows only)	TCP binary, HTTPS, WebSocket (cross-platform)
Platform	Windows only	Windows, Linux, macOS, embedded
Security	DCOM security (NTLM/Kerberos)	X.509 certificates, user tokens, encryption
Data model	Flat tag namespace (Server/Group/Item)	Rich information model (nodes, references, types)
Discovery	DCOM OPCEnum service	Local Discovery Server (LDS), Global Discovery Server (GDS)
Historical data	Separate OPC HDA spec	Integrated into UA (Historical Access)
Alarms	Separate OPC A&E spec	Integrated into UA (Alarms and Conditions)
Firewall traversal	Difficult (dynamic RPC ports)	Single port (4840 default)
Configuration	DCOM settings (dcomcnfg), error-prone	Certificate exchange, more consistent

Performance	Fast on local network	Comparable; pub/sub mode for high-throughput
Standard	OPC Foundation proprietary	IEC 62541 (international standard)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
OPC DA 3.0	OPC Data Access Custom Interface Specification 3.00
OPC DA 2.05a	OPC Data Access Custom Interface Specification 2.05a
OPC Common	OPC Common Definitions and Interfaces
MS-DCOM	Microsoft Distributed Component Object Model Protocol

See Also

- Modbus -- simpler industrial protocol often accessed via OPC DA servers
- PROFIBUS -- fieldbus protocol commonly wrapped by OPC DA

OPC UA -- Open Platform Communications Unified Architecture

Standard: IEC 62541 | **Layer:** Application (Layer 7) | **Wireshark filter:** `opcua`

OPC UA is a platform-independent, service-oriented architecture for industrial communication. It provides secure, reliable transport of both real-time data and historical data between sensors, PLCs, SCADA systems, MES, ERP, and cloud platforms. Unlike its predecessor OPC Classic (which was tightly coupled to Windows COM/DCOM), OPC UA runs on any operating system and supports two transport bindings: a high-performance binary protocol (`opc.tcp://`) and HTTPS for firewall-friendly deployments. OPC UA includes a rich information model, built-in security (X.509 certificates, encryption, signing), and is widely adopted as the interoperability standard for Industry 4.0 and IIoT.

Transport Bindings

Binding	URL Scheme	Port	Description

UA TCP Binary	`opc.tcp://`	4840 (default)	Native binary encoding, highest performance
HTTPS	`https://`	443	JSON or binary over HTTPS, firewall-friendly
WebSocket	`opc.wss://`	443	Binary over WebSockets (added in v1.04)

Message Header (UA Secure Conversation)

[Diagram: see online version]

Key Fields

Field	Size	Description
Message Type	3 bytes	Three ASCII characters identifying the message (HEL, ACK, OPN, CLO, MSG)
Chunk Type	1 byte	'F' = final, 'C' = intermediate chunk, 'A' = abort
Message Size	4 bytes	Total size of the message including header
Secure Channel ID	4 bytes	Identifies the secure channel (assigned by server)
Security Token ID	4 bytes	Identifies the security token for this message
Security Seq Number	4 bytes	Monotonically increasing sequence number (anti-replay)
Security Request ID	4 bytes	Matches requests to responses

Message Types

Type	Name	Description
HEL	Hello	Client initiates connection, declares buffer sizes
ACK	Acknowledge	Server acknowledges Hello, confirms buffer sizes
OPN	OpenSecureChannel	Establish or renew a secure channel (key exchange)
CLO	CloseSecureChannel	Tear down the secure channel
MSG	Message	Application-layer service request or response
ERR	Error	Transport-level error

Services

OPC UA defines service sets grouped by function:

Discovery

Service	Description
FindServers	Locate OPC UA servers on the network
GetEndpoints	Query a server's supported endpoints and security policies

SecureChannel

Service	Description
OpenSecureChannel	Create or renew a secure channel (nonces, security mode)
CloseSecureChannel	Release the secure channel

Session

Service	Description
CreateSession	Establish an application session (returns server nonce)
ActivateSession	Authenticate the user (username/password, X.509, token)
CloseSession	End the session

Node Management

Service	Description
Browse	Navigate the address space (discover nodes and references)
BrowseNext	Continue a Browse with continuation point
TranslateBrowsePathsToNodeIds	Resolve a path (e.g., "Objects/MyDevice/Temperature") to a NodeId

Attribute

Service	Description
Read	Read one or more attribute values from nodes
Write	Write one or more attribute values to nodes
HistoryRead	Read historical data or events
HistoryUpdate	Insert, replace, or delete historical data

Subscription / Monitored Items

Service	Description
CreateSubscription	Set up a subscription with publishing interval
CreateMonitoredItems	Attach items to a subscription (with sampling interval, filter)
Publish	Client requests queued notifications from the server
Republish	Re-request a missed notification
ModifySubscription	Change subscription parameters
DeleteSubscription	Remove a subscription

Method

Service	Description
Call	Invoke a method on a node (RPC-style)

Session Establishment

[Diagram: see online version]

Information Model

OPC UA organizes data in an address space of typed nodes connected by references:

Node Classes

Node Class	Description
Object	Container or physical/logical thing (e.g., a device, folder)
Variable	Holds a data value (e.g., temperature, setpoint)
Method	Callable function on an object
ObjectType	Defines the structure and behavior of objects
VariableType	Defines the type of variable values
ReferenceType	Defines the semantics of a reference
DataType	Defines data types (Int32, String, custom structures)
View	A subset of the address space

Node Attributes

Attribute	Description

NodeId	Unique identifier (namespace index + identifier)
BrowseName	Non-localized name for browsing
DisplayName	Localized human-readable name
NodeClass	Object, Variable, Method, etc.
Value	Current data value (Variables only)
DataType	Data type of the value
AccessLevel	Read, Write, HistoryRead, etc.

Address Space Structure

[Diagram: see online version]

Security Model

OPC UA security operates at two layers:

Transport Security

Security Mode	Signing	Encryption	Description
None	No	No	No security (testing only)
Sign	Yes	No	Messages are signed but not encrypted
SignAndEncrypt	Yes	Yes	Full protection (production default)

Security Policies

Policy URI	Algorithm	Key Length
None	--	--
Basic256Sha256	AES-256-CBC + RSA-SHA256	2048+ bit
Aes128_Sha256_RsaOaep	AES-128-CBC + RSA-SHA256	2048+ bit
Aes256_Sha256_RsaPss	AES-256-CBC + RSA-PSS-SHA256	2048+ bit

User Authentication

Token Type	Description
Anonymous	No user identity
UserName	Username and password
X509	Client X.509 certificate

IssuedToken	External token (e.g., Kerberos, JWT)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
IEC 62541	OPC Unified Architecture (multi-part)
IEC 62541-3	Address Space Model
IEC 62541-4	Services
IEC 62541-6	Mappings (Binary, XML, HTTPS)
IEC 62541-7	Profiles
OPC 10000 Series	OPC Foundation Specifications

See Also

- Modbus -- simple register-based industrial protocol
- PROFINET -- Siemens real-time industrial Ethernet
- EtherNet/IP -- Rockwell/ODVA industrial Ethernet
- MQTT -- lightweight pub/sub often paired with OPC UA in IIoT

PROFIBUS -- Process Field Bus

Standard: IEC 61158 / IEC 61784 | **Layer:** Physical / Data Link / Application | **Wireshark filter:** `profibus`

PROFIBUS is a fieldbus standard for industrial automation, connecting PLCs, sensors, actuators, drives, and remote I/O. It was developed in Germany in the late 1980s and became one of the most widely deployed industrial networks. PROFIBUS comes in two main variants: PROFIBUS DP (Decentralized Peripherals) for fast I/O communication, and PROFIBUS PA (Process Automation) for process instrumentation in hazardous areas. It uses a hybrid token-passing and master-slave access scheme over RS-485.

Variants

Variant	Physical Layer	Speed	Application
PROFIBUS DP	RS-485	9.6 kbps - 12 Mbps	Factory automation, fast I/O

PROFIBUS PA	MBP (Manchester, IEC 61158-2)	31.25 kbps	Process automation, intrinsically safe
PROFIBUS FMS	RS-485	9.6 kbps - 1.5 Mbps	Cell-level communication (legacy)

Frame (Telegram) Types

PROFIBUS defines four frame formats:

SD1 -- Fixed Length, No Data

[Diagram: see online version]

SD2 -- Variable Length, With Data

[Diagram: see online version]

SD3 -- Fixed Length, 8 Data Bytes

[Diagram: see online version]

SD4 -- Token

[Diagram: see online version]

Key Fields

Field	Size	Description
SD	8 bits	Start Delimiter (identifies frame type)
DA	8 bits	Destination Address (0-126, 127 = broadcast)
SA	8 bits	Source Address (0-125)
FC	8 bits	Frame Control (frame type, FCV, FCB)
LE / LEr	8 bits each	Length and repeated length (SD2 -- for validation)
PDU	Variable	Protocol Data Unit (user data + DSAP/SSAP)
FCS	8 bits	Frame Check Sequence (sum of DA+SA+FC+PDU mod 256)
ED	8 bits	End Delimiter (always 0x16)

Field Details

Frame Control (FC)

Bit	Name	Description
7-6	Frame Type	00=Request, 01=Ack/Response, 10=Reserved, 11=Reserved
5	FCV	Frame Count Bit Valid
4	FCB/ACD	Frame Count Bit (request) / Access Delay (response)
3-0	Function Code	SAP, data exchange type

Addresses

Address	Purpose
0-125	Station addresses (masters and slaves)
126	Default address (unconfigured devices)
127	Broadcast / global

Bus Access -- Token Passing + Master-Slave

[Diagram: see online version]

- **Masters** (Class 1/2) pass a token ring among themselves
- When a master holds the token, it polls its **slaves**
- Slaves only respond when polled -- they never initiate

PROFIBUS DP Cycle

[Diagram: see online version]

Speed vs Distance

Bit Rate	Max Segment Length
9.6 kbps	1200 m
93.75 kbps	1200 m
187.5 kbps	1000 m
500 kbps	400 m
1.5 Mbps	200 m
3 Mbps	100 m
6 Mbps	100 m
12 Mbps	100 m

Up to 3 repeaters can extend the total network to 4 segments.

GSD Files

Each PROFIBUS device provides a GSD (General Station Description) file that describes its configuration parameters, supported data lengths, and diagnostic capabilities. The master's configuration tool imports GSD files to set up the network.

Standards

Document	Title
IEC 61158	Industrial communication networks -- Fieldbus specifications
IEC 61784-1	Communication profile family 3 (PROFIBUS)
PROFIBUS Specification	PROFIBUS International technical documentation

See Also

- RS-485 -- physical layer for PROFIBUS DP
- CAN -- alternative industrial bus
- Ethernet -- PROFINET is the Ethernet-based successor

PROFINET -- Process Field Network

Standard: IEC 61158 / IEC 61784 | **Layer:** Application (Layer 7) / Data Link (Layer 2) | **Wireshark filter:** `pn_rt` or `pn_io`

PROFINET is an industrial Ethernet standard developed by Siemens and PROFIBUS International (PI) for real-time automation. It is the Ethernet-based successor to PROFIBUS and the most widely deployed industrial Ethernet protocol in Europe. PROFINET defines three performance classes: NRT (non-real-time, standard TCP/IP), RT (real-time, prioritized Ethernet frames), and IRT (isochronous real-time, hardware-synchronized for motion control with jitter under 1 microsecond). PROFINET uses standard Ethernet hardware for NRT and RT, while IRT requires special ASICs. Device descriptions are provided through GSD (General Station Description) files in GSDML (XML) format.

Performance Classes

Class	Name	Cycle Time	Jitter	Transport	Use Case

NRT	Non-Real-Time	100+ ms	Unbounded	Standard TCP/IP	Diagnostics, configuration, parameterization
RT (Class 1)	Real-Time	1-10 ms	~1 ms	VLAN-prioritized Ethernet frames	Standard I/O, typical factory automation
IRT (Class 3)	Isochronous Real-Time	250 us - 1 ms	< 1 us	Time-slotted Ethernet (hardware)	Motion control, synchronized drives

RT Frame Structure

PROFINET RT frames are standard Ethernet frames (EtherType 0x8892) with VLAN priority tagging:

[Diagram: see online version]

The full Ethernet frame wrapping:

[Diagram: see online version]

Key Fields

Field	Size	Description
EtherType	16 bits	0x8892 (PROFINET)
Frame ID	16 bits	Identifies the data relationship (I/O CR, alarm, etc.)
I/O Data	Variable	Process data (inputs or outputs)
Cycle Counter	16 bits	Incrementing counter for freshness detection
Data Status	8 bits	Validity and state of the I/O data
Transfer Status	8 bits	Provider/consumer transfer status

Field Details

Frame ID Ranges

Range	Name	Description
0x0000-0x00FF	Reserved	Not used
0x0100-0x7FFF	RT Class 3 (IRT)	Isochronous real-time frames
0x8000-0xBFFF	RT Class 1 (RT)	Real-time cyclic I/O
0xC000-0xFBFF	RT Class 1 (RT)	Real-time cyclic I/O (multicast)
0xFC01	Alarm High	High-priority alarms

0xFE01	Alarm Low	Low-priority alarms
0xFEFC	DCP Hello	Discovery and Configuration Protocol
0xFEFD	DCP Get/Set	DCP requests and responses
0xFEFE	DCP Identify	DCP device identification
0xFEFF	DCP Identify Response	DCP identification response

Data Status

Bit	Name	Description
0	State	0 = backup, 1 = primary
1	Redundancy	0 = non-redundant, 1 = redundant
2	Data Valid	0 = invalid, 1 = valid
4	Provider State	0 = stop, 1 = run
5	Station Problem Indicator	0 = problem detected, 1 = OK

Communication Relationships (CR)

PROFINET uses Application Relationships (AR) and Communication Relationships (CR) to manage connections:

CR Type	Description
IO CR	Cyclic I/O data exchange (RT or IRT)
Record Data CR	Acyclic read/write of parameters (via RPC)
Alarm CR	Alarm notifications (process, diagnostic, pull/plug)

Connection Establishment (Context Management)

PROFINET uses DCE/RPC (Distributed Computing Environment / Remote Procedure Call) over UDP for connection management:

[Diagram: see online version]

Discovery and Configuration Protocol (DCP)

DCP operates at Layer 2 (no IP required) for device discovery and IP assignment:

Service	Description
Identify	Multicast query to find devices by name or parameters
Get	Read device parameters (name, IP, vendor, device type)
Set	Assign IP address, device name, or reset to factory

Hello	Announcement sent by a device at startup

Device Model

Concept	Description
IO Controller	Typically a PLC; initiates connections and sends outputs
IO Device	Field device (I/O module, drive, sensor) providing inputs
IO Supervisor	Engineering station for diagnostics and commissioning
Slot / Subslot	Modular addressing (slot = module, subslot = channel)
Station Name	Unique DNS-style name assigned to each device

GSD / GSDML Files

Each PROFINET device provides a GSDML file (GSD Markup Language, XML-based) describing:

- Device identity (vendor ID, device ID, hardware/firmware versions)
- Available modules and submodules
- Supported I/O data sizes and cycle times
- Diagnostic capabilities and alarms
- IRT support and port configuration

The IO Controller's engineering tool imports GSDML files to configure the network.

IRT (Isochronous Real-Time)

IRT reserves time slots in the Ethernet frame cycle for deterministic delivery:

Phase	Description
IRT Phase (red)	Reserved, time-slotted PROFINET IRT frames (hardware-forwarded)
RT Phase (orange)	Standard PROFINET RT frames (prioritized)
NRT Phase (green)	Standard TCP/IP traffic (best effort)

IRT requires PROFINET-capable switches with hardware support for time-slot synchronization (PTCP -- Precision Transparent Clock Protocol).

Encapsulation

[Diagram: see online version]

Standards

Document	Title
IEC 61158	Industrial communication networks -- Fieldbus specifications
IEC 61784-2	Communication profile family 3 (PROFINET)
PROFINET Specification	PI (PROFIBUS & PROFINET International) technical documentation
IEC 61158-5-10	PROFINET IO application layer
IEC 61158-6-10	PROFINET IO application layer protocol specification

See Also

- EtherNet/IP -- ODVA/Rockwell industrial Ethernet (comparable to PROFINET)
- PROFIBUS -- predecessor fieldbus (RS-485 based)
- OPC UA -- platform-independent industrial interoperability
- Ethernet -- underlying data link layer
- EtherCAT -- Beckhoff real-time Ethernet (processing on the fly)

Data Link Layer

Data link layer (Layer 2) protocols for local network communication.

11 protocols in this section

ARP -- Address Resolution Protocol

Standard: RFC 826 | **Layer:** Data Link (Layer 2) | **Wireshark filter:** `arp`

ARP maps a network-layer address (IPv4) to a link-layer address (MAC). When a host needs to send a packet to an IP address on the local network, it broadcasts an ARP request asking "who has this IP?" The owner replies with its MAC address, which the sender caches for future use. ARP is essential for IPv4 over Ethernet but is not used with IPv6, which replaces it with Neighbor Discovery Protocol (NDP via ICMPv6).

Packet

[Diagram: see online version]

The diagram above shows ARP for IPv4 over Ethernet (the most common case). The address fields are variable-length in the general specification, sized by the HW Addr Len and Proto Addr Len fields.

Key Fields

Field	Size	Description
Hardware Type	16 bits	Link-layer type (1 = Ethernet)
Protocol Type	16 bits	Network-layer protocol (0x0800 = IPv4)
HW Addr Len	8 bits	Length of hardware address in bytes (6 for Ethernet)
Proto Addr Len	8 bits	Length of protocol address in bytes (4 for IPv4)
Operation	16 bits	ARP operation code
Sender HW Address	Variable	MAC address of the sender
Sender Proto Address	Variable	IP address of the sender
Target HW Address	Variable	MAC address of the target (zero in requests)
Target Proto Address	Variable	IP address of the target

Field Details

Operation

Value	Meaning
1	ARP Request
2	ARP Reply

3	RARP Request
4	RARP Reply
5	DRARP Request
6	DRARP Reply
8	InARP Request
9	InARP Reply

Hardware Type

Value	Type
1	Ethernet
6	IEEE 802
15	Frame Relay
20	Serial Line

Gratuitous ARP

A host may send an ARP reply for its own IP address without being asked. This is used to:

- Announce a new IP-to-MAC mapping (e.g., after a failover)
- Detect IP address conflicts
- Update stale ARP caches on the network

Encapsulation

[Diagram: see online version]

ARP packets are carried directly in Ethernet frames with EtherType `0x0806`. ARP is not carried over IP -- it operates below IP to resolve the addresses IP needs.

Standards

Document	Title
RFC 826	An Ethernet Address Resolution Protocol
RFC 5227	IPv4 Address Conflict Detection
RFC 5494	IANA Allocation Guidelines for ARP

See Also

- Ethernet
- IPv4

- IPv6 -- uses NDP instead of ARP

ATM -- Asynchronous Transfer Mode

Standard: ITU-T I.361 | **Layer:** Data Link (Layer 2) | **Wireshark filter:** `atm`

ATM is a cell-based switching protocol that transports data in fixed-size 53-byte cells (5-byte header + 48-byte payload). Developed in the late 1980s as the backbone of Broadband ISDN (B-ISDN), ATM was designed to carry voice, video, and data over a single network using hardware-based switching with deterministic latency. ATM dominated telco backbone networks and DSL access in the 1990s but was ultimately displaced by MPLS in the core and Ethernet in the LAN. Its legacy persists in DSL (ADSL uses ATM cells) and in concepts it pioneered such as QoS traffic classes and virtual circuit switching.

UNI Cell (User-Network Interface)

[Diagram: see online version]

NNI Cell (Network-Network Interface)

[Diagram: see online version]

The key difference: UNI cells have a 4-bit GFC field and 8-bit VPI. NNI cells eliminate GFC and expand VPI to 12 bits, providing a larger switching space for inter-switch trunks.

Key Fields

Field	Size	Description
GFC	4 bits	Generic Flow Control (UNI only) -- local flow control, rarely used
VPI	8 bits (UNI) / 12 bits (NNI)	Virtual Path Identifier -- first level of the switching hierarchy
VCI	16 bits	Virtual Channel Identifier -- second level, identifies an individual connection within a VP
PT	3 bits	Payload Type -- distinguishes user data from OAM cells
CLP	1 bit	Cell Loss Priority -- 1 = cell may be dropped during congestion
HEC	8 bits	Header Error Control -- CRC-8 over the header, can correct single-bit errors

Payload Type (PT)

PT Value	Meaning
000	User data, no congestion, SDU type 0
001	User data, no congestion, SDU type 1
010	User data, congestion experienced, SDU type 0
011	User data, congestion experienced, SDU type 1
100	OAM F5 segment (VC-level management, this segment only)
101	OAM F5 end-to-end (VC-level management, full path)
110	Resource management cell
111	Reserved

Reserved VPI/VCI Values

VPI	VCI	Purpose
0	0	Idle / unassigned cell
0	5	Signaling (Q.2931)
0	16	ILMI (Integrated Local Management Interface)
X	3	OAM F4 segment (VP-level)
X	4	OAM F4 end-to-end (VP-level)

VPI/VCI Switching Hierarchy

ATM uses a two-level switching hierarchy. A Virtual Path (VP) is a bundle of Virtual Channels (VCs), allowing network operators to switch entire bundles at VP cross-connects without examining individual VCI values:

[Diagram: see online version]

AAL (ATM Adaptation Layers)

AAL layers segment upper-layer data into 48-byte cell payloads and reassemble on the receiving side:

AAL	Traffic Type	Use Case	Payload Details
AAL1	CBR (constant bit rate)	Circuit emulation (T1/E1 over ATM)	47 bytes data + 1 byte SAR header (SN + SNP)
AAL2	VBR-rt (variable bit rate, real-time)	Low-rate voice, multiplexes multiple short packets per cell	CPS packets with CID, LI, UUI, HEC fields
AAL3/4	VBR (originally separate, merged)	Variable-length data with MID multiplexing	MID field allows interleaved sessions on one VCI

AAL5	Best-effort data	IP over ATM, DSL (most common)	CPCS-PDU with pad, UU, CPI, length, CRC-32

AAL5 CPCS-PDU

AAL5 is by far the most widely used adaptation layer. The user payload is padded to a 48-byte boundary and a trailer is appended:

[Diagram: see online version]

Field	Size	Description
User Payload	0-65,535 bytes	Upper-layer PDU (e.g., IP packet)
Pad	0-47 bytes	Padding to align trailer on 48-byte cell boundary
UU	1 byte	CPCS User-to-User indication
CPI	1 byte	Common Part Indicator (alignment, usually 0x00)
Length	2 bytes	Length of the user payload (not including pad or trailer)
CRC-32	4 bytes	CRC over payload + pad + UU + CPI + Length

The entire CPCS-PDU is segmented into 48-byte chunks, each placed in an ATM cell. The last cell is marked with PT bit 0 set to 1 (SDU type 1) to signal end of PDU.

Traffic Classes

Class	Name	Description
CBR	Constant Bit Rate	Fixed bandwidth, low latency (voice, video, circuit emulation)
rt-VBR	Real-Time Variable Bit Rate	Bursty real-time traffic with peak/sustained rate guarantees
nrt-VBR	Non-Real-Time VBR	Bursty data with bandwidth guarantee but relaxed delay
ABR	Available Bit Rate	Flow-controlled best-effort with minimum rate guarantee
UBR	Unspecified Bit Rate	Pure best-effort, no guarantees (IP traffic)

SVC Setup (Switched Virtual Circuit)

[Diagram: see online version]

PVCs (Permanent Virtual Circuits) skip this signaling -- VPI/VCI values are provisioned by the network operator.

OAM (Operations, Administration, and Maintenance)

ATM OAM operates at two hierarchical levels:

OAM Level	Scope	Cell Identification
F4 (VP level)	Monitors an entire Virtual Path	VCI=3 (segment), VCI=4 (end-to-end) on the target VPI
F5 (VC level)	Monitors a single Virtual Channel	PT=100 (segment), PT=101 (end-to-end)

OAM cell functions include: loopback (connectivity verification), continuity check, alarm indication signal (AIS), remote defect indication (RDI), and performance monitoring (block error counts, cell loss/misinsert).

ATM over Physical Layers

Physical Layer	Speed	Description
SONET STS-3c / SDH STM-1	155.52 Mbps	Most common ATM trunk interface
SONET STS-12c / SDH STM-4	622.08 Mbps	Higher-speed trunk
SONET STS-48c / SDH STM-16	2.488 Gbps	Backbone links
DS-3 (T3)	44.736 Mbps	ATM over T3 trunks
ADSL	0.5-8 Mbps down	ATM cells as DSL transport (before PTM in VDSL2)
25.6 Mbps UTP	25.6 Mbps	ATM to the desktop (ATM Forum, never widely adopted)

IP over ATM

Approach	Description
Classical IP (RFC 2225)	IP hosts on a single ATM LIS (Logical IP Subnet), uses ATMARP to resolve IP to ATM addresses
LANE (LAN Emulation)	Emulates Ethernet/Token Ring over ATM -- BUS, LES, LECS components
MPOA (Multi-Protocol Over ATM)	Combines LANE with NHRP to create direct ATM shortcuts across subnet boundaries
PPPoA (RFC 2364)	PPP over ATM (AAL5) -- used for DSL broadband access

ATM vs MPLS

Feature	ATM	MPLS

Switching unit	Fixed 53-byte cell	Variable-length packet
Header overhead	5 bytes per 48-byte payload (9.4% tax)	4-byte label on existing packets
QoS model	Native traffic classes (CBR, VBR, ABR, UBR)	DiffServ/IntServ, Traffic Engineering
Hardware	Dedicated ATM switches	Runs on existing IP routers
Scalability	N-squared VC mesh problem	Label-switched paths, better aggregation
Complexity	High (signaling, AAL, ILMI, PNNI)	Lower (LDP/RSVP-TE, leverages IP control plane)

ATM lost because its fixed cell size imposed a constant overhead tax, its complexity was enormous, and MPLS delivered equivalent traffic engineering on top of the existing IP infrastructure without requiring a forklift upgrade.

Encapsulation

[Diagram: see online version]

Standards

Document	Title
ITU-T I.361	B-ISDN ATM Layer Specification
ITU-T I.363.5	B-ISDN ATM Adaptation Layer -- AAL5
ITU-T I.363.1	B-ISDN ATM Adaptation Layer -- AAL1
ITU-T Q.2931	B-ISDN DSS2 -- UNI Layer 3 Signaling
ATM Forum UNI 4.0	User-Network Interface Specification v4.0
RFC 2225	Classical IP and ARP over ATM
RFC 2684	Multiprotocol Encapsulation over AAL5
RFC 2364	PPP over AAL5 (PPPoA)

See Also

- PPP -- PPPoA carries PPP over ATM AAL5
- Ethernet -- replaced ATM in LANs; LANE emulated Ethernet over ATM
- HDLC -- bit-oriented framing that influenced ATM's predecessors
- Frame Relay -- contemporary WAN protocol, also displaced by MPLS
- STP -- Layer 2 loop prevention (Ethernet world ATM sought to replace)

CDP -- Cisco Discovery Protocol

Standard: Cisco proprietary (no RFC) | **Layer:** Data Link (Layer 2) | **Wireshark filter:** `cdp`

CDP is a Cisco proprietary Layer 2 protocol that enables directly connected Cisco devices to discover each other and share information about their hardware, software, and network configuration. Each device periodically sends CDP frames containing TLV (Type-Length-Value) data to a well-known multicast address. CDP operates independently of Layer 3 protocols, making it useful for discovering neighbors even before IP addressing is configured. It is enabled by default on most Cisco devices.

Frame

CDP is encapsulated using SNAP (Subnetwork Access Protocol) on Ethernet:

[Diagram: see online version]

CDP Header

[Diagram: see online version]

Key Fields

Field	Size	Description
Version	1 byte	CDP version (1 or 2; version 2 is current)
TTL	1 byte	Time to live in seconds (default 180)
Checksum	2 bytes	Checksum over the entire CDP payload
TLVs	Variable	One or more Type-Length-Value fields

TLV Format

Each TLV consists of a 4-byte header followed by variable-length data:

[Diagram: see online version]

Field	Size	Description
Type	2 bytes	TLV type code
Length	2 bytes	Total TLV length including type and length fields
Value	Variable	TLV-specific data

TLV Types

Type	Name	Description

0x0001	Device-ID	Hostname of the sending device
0x0002	Addresses	Layer 3 addresses (IP, etc.) of the device
0x0003	Port-ID	Interface name the CDP frame was sent from
0x0004	Capabilities	Device type flags (router, switch, host, etc.)
0x0005	Version	Software version string (IOS version, etc.)
0x0006	Platform	Hardware platform (e.g., WS-C3560-48PS)
0x0007	IP Prefix	DEPRECATED: IP network prefixes
0x0008	Protocol Hello	Protocol-specific hello (e.g., Cluster Management)
0x0009	VTP Management Domain	VTP domain name configured on the device
0x000A	Native VLAN	Native VLAN ID of the sending interface
0x000B	Duplex	Duplex setting (0 = half, 1 = full)
0x000E	VoIP VLAN Reply	Voice VLAN ID for VoIP phone provisioning
0x000F	VoIP VLAN Query	Request for voice VLAN information
0x0010	Power	Power consumption of the device (in milliwatts)
0x0011	MTU	MTU of the sending interface
0x0012	Trust Bitmap	Extended trust QoS bitmap
0x0013	Untrusted Port CoS	CoS value for untrusted ports
0x0014	System Name	System name (different from Device-ID)
0x0015	System OID	SNMP system object identifier
0x0016	Management Addresses	Addresses for SNMP and other management
0x0017	Location	Physical location string
0x0019	Power Available	PoE power available on the port (in milliwatts)

Capabilities Bitmap

Bit	Capability
0x01	Router
0x02	Transparent Bridge
0x04	Source-Route Bridge

0x08	Switch (Layer 2)
0x10	Host
0x20	IGMP capable
0x40	Repeater

Timing

Parameter	Default
Advertisement interval	60 seconds
Holdtime (TTL)	180 seconds (3x advertisement interval)
CDPv2 enabled	Yes (default on Cisco devices)

Neighbor Discovery

[Diagram: see online version]

CDP vs LLDP

Feature	CDP	LLDP
Standard	Cisco proprietary	IEEE 802.1AB (vendor-neutral)
Multicast address	01:00:0C:CC:CC:CC	01:80:C2:00:00:0E
Encapsulation	SNAP (EtherType 0x2000)	Ethernet (EtherType 0x88CC)
Default interval	60 seconds	30 seconds
Default holdtime	180 seconds	120 seconds
Data format	TLV (2-byte type)	TLV (7-bit type, 9-bit length)
Multi-vendor	Cisco only	All vendors
VoIP VLAN	Native support	LLDP-MED extension
PoE negotiation	Power TLVs	IEEE 802.3 TLVs
IPv6 support	Limited	Full support

Security Considerations

CDP is enabled by default on Cisco devices and discloses significant information about the network:

Risk	Detail
Information disclosure	Exposes hostname, IOS version, IP addresses, platform, VLAN info

Reconnaissance	Attackers on the LAN can passively collect CDP frames
Version fingerprinting	Software version reveals known vulnerabilities
Topology mapping	Port-ID and Device-ID reveal network topology
DoS (CDP flooding)	Sending large numbers of bogus CDP frames can exhaust the neighbor table

Best practices:

- Disable CDP on ports facing untrusted networks (e.g., user access ports, internet-facing interfaces)
- Use `no cdp enable` on a per-interface basis or `no cdp run` globally
- Consider migrating to LLDP for multi-vendor environments
- Use CDP only on inter-switch and inter-router links where discovery is needed

Encapsulation

[Diagram: see online version]

CDP frames are **not forwarded** by switches -- they are consumed by the directly connected neighbor only, identical to LLDP behavior.

Standards

Document	Title
Cisco proprietary	No IETF RFC or IEEE standard
Wireshark CDP	Wireshark protocol reference for CDP

See Also

- LLDP -- vendor-neutral alternative to CDP
- Ethernet -- CDP runs on Ethernet LANs
- STP -- another Layer 2 protocol on Cisco switches
- 802.1Q -- Native VLAN advertised via CDP

Ethernet -- IEEE 802.3

Standard: IEEE 802.3 | **Layer:** Data Link (Layer 2) | **Wireshark filter:** `eth`

Ethernet is the dominant wired LAN technology, defining framing and physical-layer signaling for local network communication. Developed by Robert Metcalfe and David Boggs at Xerox PARC in the 1970s, it originally used CSMA/CD for media access. Modern switched Ethernet operates in full-duplex mode, making CSMA/CD largely historical.

The terms "Ethernet" and IEEE 802.3 are often used interchangeably, though they differ slightly: Ethernet II uses a Type field while 802.3 uses a Length field, distinguished by the field's value.

Frame

[Diagram: see online version]

Note: The Preamble (7 bytes) and Start Frame Delimiter (1 byte) precede the frame but are typically stripped by hardware before the packet reaches software.

Key Fields

Field	Size	Description
Destination MAC	6 bytes	MAC address of the receiving device
Source MAC	6 bytes	MAC address of the sending device
Type / Length	2 bytes	EtherType (`>= 0x0600`) or payload length (`<= 1500`)
Payload	46-1500 bytes	Upper-layer protocol data (LLC/SNAP or direct)
FCS	4 bytes	CRC-32 error check over the entire frame

Field Details

Destination MAC Address

[Diagram: see online version]

Bit	Field	Values
0	I/G (Individual/Group)	0 = Unicast, 1 = Multicast/Broadcast
1	U/L (Universal/Local)	0 = Universally administered (OUI), 1 = Locally administered

Broadcast address: `FF:FF:FF:FF:FF:FF`

Source MAC Address

[Diagram: see online version]

The first bit is always 0 (source is always unicast).

Type / Length

The interpretation depends on the value:

Value	Interpretation
`<= 1500` (0x05DC)	IEEE 802.3 Length field -- payload size in bytes
`>= 1536` (0x0600)	Ethernet II Type field -- identifies upper-layer protocol

Common EtherType values:

EtherType	Protocol
`0x0800`	IPv4
`0x0806`	ARP
`0x8100`	IEEE 802.1Q (VLAN tag)
`0x86DD`	IPv6
`0x8847`	MPLS (unicast)
`0x88CC`	LLDP

IEEE 802.1Q VLAN Tag (optional)

When present, a 4-byte VLAN tag is inserted between the Source MAC and the original Type/Length field:

[Diagram: see online version]

Field	Size	Description
TPID	16 bits	Tag Protocol Identifier, set to `0x8100`
PCP	3 bits	Priority Code Point (802.1p QoS)
DEI	1 bit	Drop Eligible Indicator
VID	12 bits	VLAN Identifier (0-4095)

Ethernet Standards

Standard	Speed	Medium
10BASE-T	10 Mbps	Twisted pair
100BASE-TX	100 Mbps	Twisted pair
1000BASE-T	1 Gbps	Twisted pair
10GBASE-T	10 Gbps	Twisted pair
25GBASE-CR	25 Gbps	Twinaxial copper
40GBASE-SR4	40 Gbps	Multi-mode fiber
100GBASE-SR4	100 Gbps	Multi-mode fiber
400GBASE-SR8	400 Gbps	Multi-mode fiber

Encapsulation

[Diagram: see online version]

Standards

Document	Title
IEEE 802.3-2022	IEEE Standard for Ethernet
IEEE 802.1Q-2022	IEEE Standard for VLANs
RFC 894	A Standard for the Transmission of IP Datagrams over Ethernet Networks
RFC 1042	A Standard for the Transmission of IP Datagrams over IEEE 802 Networks

See Also

- IPv4
- IPv6
- ARP

Frame Relay

Standard: ITU-T Q.922 | **Layer:** Data Link (Layer 2) | **Wireshark filter:** `fr`

Frame Relay is a packet-switched WAN protocol that provides connection-oriented, statistically multiplexed data transport over virtual circuits. Developed in the early 1990s as a faster, simpler successor to X.25, Frame Relay strips away X.25's hop-by-hop error recovery and flow control, relying instead on upper-layer protocols (TCP) to handle retransmission. This made Frame Relay switches far simpler and faster. Frame Relay was the dominant WAN technology for enterprise site-to-site connectivity throughout the 1990s and early 2000s, eventually displaced by MPLS VPNs and Ethernet WAN services.

Frame

[Diagram: see online version]

Header (2-Byte Standard)

[Diagram: see online version]

Key Fields

Field	Size	Description
Flag	8 bits	HDLC frame delimiter `0x7E`
DLCI	10 bits (standard 2-byte header)	Data Link Connection Identifier -- identifies the PVC or SVC
C/R	1 bit	Command/Response -- application-specific, passed transparently
EA	1 bit per byte	Extended Address -- 0 = more address bytes follow, 1 = last byte
FECN	1 bit	Forward Explicit Congestion Notification -- set by network toward receiver
BECN	1 bit	Backward ECN -- set by network toward sender
DE	1 bit	Discard Eligible -- marks frames that may be dropped during congestion
Information	Variable (up to 4096 bytes typical)	Upper-layer payload
FCS	16 bits	Frame Check Sequence (CRC-16-CCITT)

The 2-byte header yields a 10-bit DLCI (values 0-1023). Extended headers of 3 or 4 bytes expand the DLCI to 16 or 23 bits using additional EA bytes, though the 2-byte format is overwhelmingly common.

Reserved DLCIs

DLCI	Purpose
0	Signaling (Q.933, ANSI T1.617 Annex D)
1-15	Reserved
16-1007	User data circuits (PVCs and SVCs)
1008-1022	Reserved
1023	LMI (Cisco proprietary, "Gang of Four" LMI)

Congestion Management

Frame Relay uses three mechanisms to signal congestion:

Mechanism	Direction	Description
FECN	Toward receiver	Switch sets FECN=1 in frames traveling through a congested node

BECN	Toward sender	Switch sets BECN=1 in frames traveling opposite to congestion
DE	Set by edge or network	Marks excess-burst frames as eligible for discard

When a sender receives frames with BECN=1, it should reduce its transmission rate. When a receiver sees FECN=1, it may signal the sender through upper-layer protocols.

Traffic Contract

Parameter	Description
CIR (Committed Information Rate)	Guaranteed throughput the network will deliver (bits/sec)
Bc (Committed Burst Size)	Maximum data allowed during the measurement interval Tc (bits)
Be (Excess Burst Size)	Additional data above Bc that may be transmitted but marked DE (bits)
Tc (Measurement Interval)	Bc / CIR -- the time window for burst measurement

Frames sent within the CIR are delivered normally. Frames in the Be range are marked DE=1 and delivered if capacity allows. Frames exceeding CIR + Be may be dropped immediately.

LMI (Local Management Interface)

LMI provides keepalive and PVC status signaling between a DTE (router) and the Frame Relay switch:

[Diagram: see online version]

LMI Types

LMI Type	Signaling DLCI	Standard
Cisco ("Gang of Four")	1023	Original de facto standard (Cisco, DEC, StrataCom, NorTel)
ANSI T1.617 Annex D	0	ANSI standard
ITU Q.933 Annex A	0	ITU-T standard

All three are functionally similar but use different DLCI values and slightly different message encoding. Both ends must use the same LMI type.

PVC States

State	Meaning
Active	PVC is operational end-to-end
Inactive	PVC exists locally but the remote end is down

Deleted	PVC has been removed from the switch configuration

PVC vs SVC

Feature	PVC (Permanent Virtual Circuit)	SVC (Switched Virtual Circuit)
Setup	Provisioned by operator	Established on demand (Q.933 signaling)
DLCI assignment	Static, configured	Dynamic, assigned during call setup
Availability	Always present	Created and torn down as needed
Use case	Dedicated site-to-site links	On-demand connections (rarely deployed)

PVCs were used almost exclusively in practice. Frame Relay SVCs saw minimal real-world deployment.

Inverse ARP

Inverse ARP (RFC 2390) allows a router to discover the network-layer address (e.g., IP) of the remote device on a DLCI without manual configuration. This is the reverse of regular ARP -- given a DLCI (Layer 2), Inverse ARP discovers the IP address (Layer 3):

Step	Action
1	Router learns DLCI from LMI Status message
2	Router sends Inverse ARP Request on the DLCI
3	Remote router responds with its IP address
4	Local router creates a dynamic DLCI-to-IP mapping

PVC Data Flow

[Diagram: see online version]

DLCI values have **local significance only**. Each switch remaps the DLCI as the frame traverses the network. Router A knows its circuit as DLCI 102, Router B knows the same circuit as DLCI 201, and the switches use different DLCIs internally.

Subinterfaces

Frame Relay routers commonly use subinterfaces to solve split-horizon and broadcast replication issues on multipoint circuits:

Type	Description
Point-to-Point	One subinterface per DLCI -- each acts as a separate link; solves split-horizon for routing protocols

Multipoint	Multiple DLCIs on one subinterface -- behaves like a shared network segment

Frame Relay vs ATM vs MPLS

Feature	Frame Relay	ATM	MPLS
Switching unit	Variable-length frame	Fixed 53-byte cell	Variable-length packet
Max frame/cell	~4096 bytes	48-byte payload	MTU of underlying link
Addressing	DLCI (10-bit typical)	VPI/VCI (8+16 bits)	20-bit label
Congestion signaling	FECN, BECN, DE	CLP, EFCI, ABR rate control	ECN, WRED, QoS via EXP bits
QoS	CIR/Bc/Be traffic contract	Native CBR/VBR/ABR/UBR classes	DiffServ, Traffic Engineering
Error recovery	None (relies on upper layers)	None (relies on upper layers)	None (relies on upper layers)
Heyday	1990s-2000s	Late 1990s	2000s-present

Why Frame Relay Died

Frame Relay was displaced by two technologies: MPLS in the WAN core (providing equivalent virtual circuit service with better traffic engineering and integration with IP routing) and Carrier Ethernet / Metro Ethernet services that offered higher bandwidth at lower cost. Most carriers stopped selling new Frame Relay circuits by the mid-2000s, and the last major Frame Relay networks were decommissioned by the mid-2010s.

Encapsulation

[Diagram: see online version]

Standards

Document	Title
ITU-T Q.922	ISDN Data Link Layer for Frame Mode Bearer Services
ITU-T Q.933	Signaling for Frame Relay Switched Virtual Circuits
ANSI T1.617	Signaling Specification for Frame Relay Bearer Service
FRF.1.2	Frame Relay Forum -- UNI Implementation Agreement
FRF.16	Multilink Frame Relay UNI/NNI
RFC 2390	Inverse ARP
RFC 2427	Multiprotocol Interconnect over Frame Relay

See Also

- HDLC -- Frame Relay's framing is derived from HDLC/Q.922
- ATM -- contemporary cell-based WAN protocol, also displaced by MPLS
- PPP -- alternative point-to-point WAN encapsulation
- Ethernet -- Carrier Ethernet replaced Frame Relay for enterprise WAN
- ARP -- Frame Relay uses Inverse ARP (RFC 2390) for address resolution

HDLC -- High-Level Data Link Control

Standard: ISO 13239 | **Layer:** Data Link (Layer 2) | **Wireshark filter:** `hdlc`

HDLC is a bit-oriented, synchronous data link protocol standardized by ISO. It provides framing, error detection, flow control, and reliable delivery over point-to-point and multipoint links. HDLC is one of the most influential protocols ever designed -- it serves as the foundation for PPP, Frame Relay, LAPB (X.25), LAPD (ISDN D-channel), LAPM (V.42 modems), Cisco HDLC, and IBM's SDLC. Nearly every WAN protocol from the 1980s through the 2000s carried DNA from HDLC's frame structure.

Frame

[Diagram: see online version]

Key Fields

Field	Size	Description
Flag	8 bits	Frame delimiter `0x7E` (01111110) -- marks start and end of frame
Address	8 bits (basic) or multibyte	Identifies station; `0xFF` = all-stations broadcast
Control	8 bits (basic) or 16 bits (extended)	Frame type, sequence numbers, poll/final bit
Information	Variable	Payload data (present in I-frames and some U-frames)
FCS	16 bits (CRC-CCITT) or 32 bits (CRC-32)	Frame Check Sequence for error detection

Bit Stuffing

The flag pattern `01111110` (0x7E) must never appear within the frame body. HDLC achieves this through bit stuffing:

- **Transmitter:** After any five consecutive 1-bits in the data, insert a 0-bit

- **Receiver:** After five consecutive 1-bits, remove the following 0-bit
- **Result:** Six consecutive 1s can only appear as part of a flag or abort (seven 1s = abort)

This is distinct from PPP's byte-stuffing approach over async links, which escapes `0x7E` using `0x7D` followed by the byte XORed with `0x20`.

Control Field

The control byte identifies three frame types based on its first one or two bits:

I-Frame (Information Transfer)

[Diagram: see online version]

Field	Description
Bit 0 = 0	Identifies I-frame
N(S)	Send sequence number (modulo 8 or modulo 128 in extended)
P/F	Poll/Final bit -- command polls for response; response marks final frame
N(R)	Receive sequence number -- acknowledges all frames up to N(R)-1

S-Frame (Supervisory)

[Diagram: see online version]

Type	Code	Name	Description
00	RR	Receive Ready	Acknowledges frames, ready for more
01	REJ	Reject	Go-Back-N -- retransmit from N(R) onward
10	RNR	Receive Not Ready	Acknowledges frames, requests pause (flow control)
11	SREJ	Selective Reject	Retransmit only the single frame N(R)

U-Frame (Unnumbered)

[Diagram: see online version]

The five type bits (split around P/F) encode the command or response:

Type Bits	Name	Direction	Description
00 100	SABM	Command	Set Asynchronous Balanced Mode (8-bit sequence)

11 100	SABME	Command	Set ABM Extended (16-bit control, 128-bit sequence)
00 000	UI	Command	Unnumbered Information (connectionless data)
01 000	DISC	Command	Disconnect -- terminate the link
00 110	UA	Response	Unnumbered Acknowledge (accepts SABM/DISC)
00 001	DM	Response	Disconnected Mode -- link is down
10 001	FRMR	Response	Frame Reject -- protocol error (invalid frame received)

Operating Modes

Mode	Abbreviation	Description
Normal Response Mode	NRM	Secondary station transmits only when polled by primary (legacy, master/slave)
Asynchronous Response Mode	ARM	Secondary can transmit without being polled, but primary manages the link
Asynchronous Balanced Mode	ABM	Both stations are peers with equal control -- **most common mode** (used by PPP, LAPB, LAPD)

Windowing

Mode	Sequence Bits	Window Size	Control Field
Basic (modulo 8)	3-bit N(S), N(R)	Up to 7 outstanding frames	8-bit control
Extended (modulo 128)	7-bit N(S), N(R)	Up to 127 outstanding frames	16-bit control

Extended mode is established via SABME and is used on high-bandwidth or high-latency links where a 7-frame window is insufficient.

ABM Session

[Diagram: see online version]

HDLC Derivatives

Protocol	Based On	Key Differences from HDLC

PPP (RFC 1661)	HDLC framing	Address always 0xFF, Control always 0x03 (UI), adds Protocol field, LCP negotiation, byte stuffing on async links
LAPB (X.25 L2)	HDLC ABM	Strict ABM operation, 8 or 128 sequence, basis of X.25 reliable link layer
LAPD (ISDN Q.921)	HDLC ABM	2-byte address with SAPI + TEI for ISDN D-channel multiplexing
LAPM (V.42)	HDLC ABM	Error-correcting mode for modem connections
Frame Relay (Q.922)	HDLC	Replaces Address/Control with DLCI-based header, no error recovery (relay only)
Cisco HDLC	HDLC	Adds 2-byte proprietary protocol type field after Control (similar to PPP but incompatible)
SDLC (IBM SNA)	Predecessor	IBM's original protocol that HDLC was derived from; NRM only, single-byte address

Cisco HDLC

Cisco's proprietary HDLC variant is the default encapsulation on Cisco serial interfaces. It adds a 2-byte protocol type field after the standard HDLC Control byte:

[Diagram: see online version]

Because the protocol type field is proprietary, Cisco HDLC is incompatible with other vendors' HDLC implementations. Both ends of a serial link must use the same encapsulation -- this is a common misconfiguration when connecting Cisco to non-Cisco equipment (use PPP instead).

Encapsulation

[Diagram: see online version]

Standards

Document	Title
ISO 13239:2002	HDLC -- Consolidated procedures and elements
ISO 3309:1979	HDLC -- Frame structure (original, superseded by ISO 13239)
ITU-T Q.921	LAPD -- ISDN data link layer
ITU-T Q.922	ISDN data link layer for Frame Relay bearer service
RFC 1661	PPP (Point-to-Point Protocol)
RFC 1662	PPP in HDLC-like Framing

See Also

- PPP -- the most widely deployed HDLC derivative
- Frame Relay -- uses HDLC-derived framing with DLCI addressing
- Ethernet -- the LAN counterpart to HDLC's WAN role
- STP -- Layer 2 loop prevention for bridged networks

LACP -- Link Aggregation Control Protocol

Standard: IEEE 802.1AX-2020 | **Layer:** Data Link (Layer 2) | **Wireshark filter:** `lacp`

LACP dynamically negotiates the bundling of multiple physical Ethernet links into a single logical link (Link Aggregation Group -- LAG). This provides increased bandwidth, redundancy, and automatic failover. LACP ensures both ends agree on which links are part of the bundle and detects link failures. It is the standard alternative to static port channeling and is supported by virtually all managed switches, servers, and storage arrays.

LACPDU

[Diagram: see online version]

LACPDUs are 110 bytes, sent to the multicast address `01:80:C2:00:00:02` with EtherType `0x8809` (Slow Protocols).

Key Fields

Field	Size	Description
Subtype	8 bits	0x01 = LACP
Actor System Priority	16 bits	Priority of the local system (lower = higher priority)
Actor System ID	6 bytes	MAC address of the local system
Actor Key	16 bits	Administrative key grouping eligible ports
Actor Port Priority	16 bits	Priority of this port (lower = preferred)
Actor Port Number	16 bits	Port identifier
Actor State	8 bits	State flags (see below)
Partner fields	--	Mirrored information from the remote end

Collector Max Delay	16 bits	Maximum delay before collecting frames (in 10µs units)

State Flags

Bit	Name	Description
0	Activity	1 = Active LACP (initiates), 0 = Passive (responds only)
1	Timeout	1 = Short timeout (3s), 0 = Long timeout (90s)
2	Aggregation	1 = Port can be aggregated
3	Synchronization	1 = Port is in sync with partner
4	Collecting	1 = Port is collecting (receiving) frames
5	Distributing	1 = Port is distributing (sending) frames
6	Defaulted	1 = Using default partner info (no LACPDU received)
7	Expired	1 = Partner info has expired

Negotiation

[Diagram: see online version]

Periodic Exchange

Mode	LACPDU Interval
Short timeout	Every 1 second (detect failure in 3s)
Long timeout	Every 30 seconds (detect failure in 90s)

LAG Formation Rules

For ports to aggregate:

1. Both ends must have matching **Actor Key** values
2. Both ends must have the **Aggregation** flag set
3. At least one end must be in **Active** mode (both can be active)
4. Links must be the same speed and duplex
5. Maximum ports per LAG varies by platform (typically 8-16)

Load Distribution

Traffic is distributed across member links using a hash of frame fields:

Hash Input	Method
Source MAC	L2 hash
Destination MAC	L2 hash
Source + Dest MAC	L2 hash
Source + Dest IP	L3 hash
Source + Dest IP + L4 ports	L3+L4 hash (best distribution)

The hash determines which member link carries each flow -- all packets of the same flow take the same link (preserving order).

LACP Modes

Local Mode	Remote Mode	Result
Active	Active	LAG forms (both initiate)
Active	Passive	LAG forms (active initiates)
Passive	Passive	No LAG (neither initiates)

Encapsulation

[Diagram: see online version]

LACPDUs are slow protocol frames -- they are **not forwarded** by bridges.

Standards

Document	Title
IEEE 802.1AX-2020	Link Aggregation
IEEE 802.3ad	Original link aggregation (merged into 802.1AX)

See Also

- Ethernet -- the links being aggregated
- LLDP -- reports LAG status to neighbors
- STP -- treats a LAG as a single logical link
- 802.1Q -- VLANs trunked over aggregated links

LLDP -- Link Layer Discovery Protocol

Standard: IEEE 802.1AB-2016 | **Layer:** Data Link (Layer 2) | **Wireshark filter:** `lldp`

LLDP is a vendor-neutral, one-way protocol that allows network devices to advertise their identity, capabilities, and neighbors on a local network. Each device periodically sends LLDP frames containing TLV (Type-Length-Value) information to a well-known multicast address. Neighboring devices collect and store this information in an LLDP MIB, enabling network management tools to automatically discover the physical topology. LLDP is essential for VoIP phone provisioning, PoE negotiation, and network documentation.

Frame

[Diagram: see online version]

TLV Format

[Diagram: see online version]

Mandatory TLVs

Type	Name	Description
1	Chassis ID	Device identifier (MAC, hostname, IP, etc.)
2	Port ID	Port identifier (interface name, MAC, etc.)
3	TTL	Time to live in seconds (0 = delete this entry)
0	End of LLDPDU	Marks end of TLV list (type=0, length=0)

Optional TLVs

Type	Name	Description
4	Port Description	Interface description string
5	System Name	Device hostname
6	System Description	Device model, OS, software version
7	System Capabilities	Capabilities bitmap (bridge, router, phone, etc.)
8	Management Address	IP address for device management
127	Organizationally Specific	Vendor extensions (IEEE 802.1, 802.3, LLDP-MED, etc.)

System Capabilities Bitmap

Bit	Capability
0	Other
1	Repeater
2	Bridge
3	WLAN Access Point
4	Router
5	Telephone
6	DOCSIS Cable Device
7	Station Only

Common Organizationally Specific TLVs

IEEE 802.1 (OUI: 00-80-C2)

Subtype	Name	Description
1	Port VLAN ID	Native/untagged VLAN on this port
2	Port and Protocol VLAN ID	Protocol-based VLAN
3	VLAN Name	Name for a VLAN ID
4	Protocol Identity	Protocols supported on this port

IEEE 802.3 (OUI: 00-12-0F)

Subtype	Name	Description
1	MAC/PHY Config/Status	Speed, duplex, auto-negotiation
2	Power via MDI (PoE)	Power class, pair, required power
3	Link Aggregation	LACP status
4	Maximum Frame Size	Maximum supported frame size

LLDP-MED (Media Endpoint Discovery)

LLDP-MED extends LLDP for VoIP devices:

Subtype	Name	Description
1	LLDP-MED Capabilities	Supported MED TLVs
2	Network Policy	VLAN, L2/L3 priority for voice/video/signaling
3	Location Identification	Emergency location (E911)

4	Extended Power-via-MDI	Detailed PoE power info
5	Inventory	Hardware/firmware/software/serial/model

Timing

Parameter	Default
Transmit interval	30 seconds
TTL	120 seconds
Reinit delay	2 seconds
Fast start	1 second (first 4 frames after link up)

Encapsulation

[Diagram: see online version]

LLDP frames are **not forwarded** by bridges -- they are consumed by the directly connected neighbor only.

Standards

Document	Title
IEEE 802.1AB-2016	Station and Media Access Control Connectivity Discovery (LLDP)
ANSI/TIA-1057	LLDP for Media Endpoint Devices (LLDP-MED)

See Also

- Ethernet -- LLDP runs directly on Ethernet
- 802.1Q -- VLAN info advertised via LLDP
- LACP -- aggregation status reported in LLDP
- STP -- another L2 protocol using the same multicast range

PPP -- Point-to-Point Protocol

Standard: RFC 1661 | **Layer:** Data Link (Layer 2) | **Wireshark filter:** ppp

PPP is a data link protocol for establishing a direct connection between two nodes. It provides framing, link negotiation (LCP), authentication (PAP/CHAP/EAP), and network-layer configuration (IPCP for IPv4, IPV6CP for

IPv6). PPP was the standard protocol for dial-up Internet and remains the foundation of PPPoE (broadband DSL/fiber), PPPoA (ATM), L2TP tunnels, and some serial WAN links.

Frame

[Diagram: see online version]

Key Fields

Field	Size	Description
Flag	8 bits	HDLC frame delimiter `0x7E`
Address	8 bits	Always `0xFF` (all-stations broadcast)
Control	8 bits	Always `0x03` (unnumbered information)
Protocol	16 bits	Identifies the encapsulated protocol
Information	Variable	Payload (default max 1500 bytes, negotiable)
FCS	16 or 32 bits	Frame Check Sequence (CRC-16 default, CRC-32 optional)

Address and Control fields can be compressed away via LCP negotiation (ACFC), and the Protocol field can be compressed to 1 byte (PFC).

Protocol Field Values

Value	Protocol
0x0021	IPv4
0x0057	IPv6
0x002B	IPX
0x8021	IPCP (IP Control Protocol)
0x8057	IPV6CP (IPv6 Control Protocol)
0xC021	LCP (Link Control Protocol)
0xC023	PAP (Password Authentication Protocol)
0xC223	CHAP (Challenge Handshake Authentication Protocol)
0xC227	EAP
0x8281	MPLSCP

Link Establishment

[Diagram: see online version]

LCP (Link Control Protocol)

Code	Name	Description
1	Configure-Request	Propose link options
2	Configure-Ack	Accept all options
3	Configure-Nak	Reject option values (suggest alternatives)
4	Configure-Reject	Reject option types entirely
5	Terminate-Request	Close the link
6	Terminate-Ack	Acknowledge closure
9	Echo-Request	Keepalive
10	Echo-Reply	Keepalive response

LCP Options

Type	Name	Description
1	MRU	Maximum Receive Unit (default 1500)
3	Authentication Protocol	PAP (0xC023), CHAP (0xC223), EAP (0xC227)
5	Magic Number	Loop detection
7	PFC	Protocol Field Compression
8	ACFC	Address/Control Field Compression

Authentication

Method	Security	Description
PAP	Weak	Sends password in cleartext
CHAP	Moderate	Challenge-response with MD5 hash
MS-CHAPv2	Moderate	Microsoft variant with mutual authentication
EAP	Strong	Extensible framework (TLS, PEAP, etc.)

PPPoE (PPP over Ethernet)

PPPoE encapsulates PPP in Ethernet frames for broadband access (DSL, fiber):

[Diagram: see online version]

PPPoE Phases

[Diagram: see online version]

PPPoE EtherTypes

EtherType	Phase
0x8863	Discovery (PADI, PADO, PADR, PADS, PADT)
0x8864	Session (PPP data)

MTU Impact

PPPoE adds 8 bytes of overhead, reducing the effective MTU from 1500 to **1492 bytes** -- a common cause of path MTU issues.

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 1661	The Point-to-Point Protocol (PPP)
RFC 1662	PPP in HDLC-like Framing
RFC 1332	PPP IPCP (IP Control Protocol)
RFC 5072	PPP IPV6CP (IPv6 Control Protocol)
RFC 1334	PPP Authentication: PAP and CHAP
RFC 2516	PPP over Ethernet (PPPoE)
RFC 2364	PPP over ATM (PPPoA)

See Also

- Ethernet -- PPPoE carrier
- IPv4 -- configured via IPCP
- RADIUS -- authenticates PPP/PPPoE sessions

STP / RSTP / MSTP -- Spanning Tree Protocols

Standard: IEEE 802.1D (STP/RSTP) / IEEE 802.1Q (MSTP) | **Layer:** Data Link (Layer 2) | **Wireshark filter:** `stp`

STP prevents forwarding loops in networks with redundant Layer 2 paths by selectively blocking ports. Without STP, a single loop causes a broadcast storm that can bring down an entire network in seconds. STP elects a Root Bridge, calculates shortest paths, and blocks redundant links. RSTP (Rapid STP) provides sub-second convergence (vs. 30-50 seconds for classic STP). MSTP maps multiple VLANs to spanning tree instances for efficient use of redundant links.

BPDU (Bridge Protocol Data Unit)

RSTP BPDU

[Diagram: see online version]

Key Fields

Field	Size	Description
Protocol ID	16 bits	Always 0x0000
Version	8 bits	0 = STP, 2 = RSTP, 3 = MSTP
Type	8 bits	0x00 = Config BPDU, 0x02 = RST BPDU, 0x80 = TCN
Flags	8 bits	TC (bit 0), Proposal, Port Role, Learning, Forwarding, Agreement, TCA (bit 7)
Root Bridge ID	8 bytes	Priority (4 bits) + System ID Extension (12 bits) + MAC (6 bytes)
Root Path Cost	4 bytes	Cumulative cost from this bridge to the root
Bridge ID	8 bytes	This bridge's ID (same format as Root Bridge ID)
Port ID	2 bytes	Priority (4 bits) + Port Number (12 bits)
Message Age	2 bytes	Age of this BPDU (in 1/256ths of a second)
Max Age	2 bytes	Maximum BPDU age before discard (default 20s)
Hello Time	2 bytes	Interval between BPDUs (default 2s)
Forward Delay	2 bytes	Time in Listening/Learning states (default 15s)

Port States

Classic STP

State	Learns MACs	Forwards Data	Duration
Disabled	No	No	--
Blocking	No	No	Until topology change
Listening	No	No	Forward Delay (15s)
Learning	Yes	No	Forward Delay (15s)
Forwarding	Yes	Yes	Normal operation

RSTP (simplified)

State	Description
Discarding	Not forwarding (combines Disabled/Blocking/Listening)
Learning	Learning MACs, not yet forwarding
Forwarding	Fully operational

Port Roles

Role	Description
Root Port	Best path to the Root Bridge (one per non-root bridge)
Designated Port	Best path for a segment toward the Root (one per segment)
Alternate Port	Backup path to the Root (RSTP -- blocked, ready for fast failover)
Backup Port	Redundant path on the same segment (rare)
Disabled	Administratively shut down

Root Bridge Election

The bridge with the lowest Bridge ID becomes Root:

```
Bridge ID = Priority (4 bits) + VLAN / Sys-ID-Ext (12 bits) + MAC Address (48 bits)
```

Default priority is 32768. Lower priority wins. If equal, lowest MAC wins.

Example Topology

[Diagram: see online version]

RSTP Convergence

RSTP converges in 1-3 seconds (vs. 30-50 for classic STP) using:

Mechanism	Description
Proposal/Agreement	Rapid handshake to transition to Forwarding
Edge Ports	Ports connected to hosts skip Listening/Learning (like PortFast)
Alternate Ports	Pre-computed backup paths for instant failover

[Diagram: see online version]

MSTP (Multiple Spanning Tree)

MSTP maps VLANs to spanning tree instances, allowing different VLANs to use different paths:

Instance	VLANs	Root Bridge
MSTI 1	10, 20	Switch A
MSTI 2	30, 40	Switch B

This provides load balancing across redundant links.

Path Cost

Link Speed	STP Cost (802.1D-1998)	RSTP Cost (802.1t)
10 Mbps	100	2,000,000
100 Mbps	19	200,000
1 Gbps	4	20,000
10 Gbps	2	2,000
100 Gbps	--	200

Encapsulation

[Diagram: see online version]

STP BPDUs are sent to the well-known multicast address `01:80:C2:00:00:00` with LLC encapsulation.

Standards

Document	Title
IEEE 802.1D-2004	MAC Bridges (includes RSTP)
IEEE 802.1Q-2022	Bridges and Bridged Networks (includes MSTP)
IEEE 802.1w	Rapid Spanning Tree (merged into 802.1D-2004)

See Also

- Ethernet -- the frames STP protects from loops
- 802.1Q -- VLAN tagging (MSTP maps VLANs to instances)
- LLDP -- link-layer discovery
- LACP -- link aggregation (alternative to blocking redundant links)

IEEE 802.1Q -- VLAN Tagging

Standard: IEEE 802.1Q-2022 | **Layer:** Data Link (Layer 2) | **Wireshark filter:** `vlan`

IEEE 802.1Q defines VLAN (Virtual LAN) tagging -- a 4-byte tag inserted into Ethernet frames to identify which virtual network a frame belongs to. VLANs segment a physical network into isolated broadcast domains without requiring separate physical infrastructure. 802.1Q is implemented on virtually every managed switch and is fundamental to enterprise, data center, and service provider network design.

Tagged Frame

[Diagram: see online version]

The 4-byte 802.1Q tag is inserted between the Source MAC and the original EtherType field.

Tag Fields

Field	Size	Description
TPID	16 bits	Tag Protocol Identifier -- `0x8100` for 802.1Q
PCP	3 bits	Priority Code Point -- 802.1p QoS priority (0-7)
DEI	1 bit	Drop Eligible Indicator (formerly CFI)
VID	12 bits	VLAN Identifier (0-4095)

VLAN ID Ranges

VID	Purpose
0	Priority tag only (no VLAN)
1	Default VLAN
2-1001	Normal range

1002-1005	Reserved (Token Ring, FDDI -- Cisco)
1006-4094	Extended range
4095	Reserved

Priority Code Point (PCP)

PCP	Priority	Traffic Type
0	Best Effort	Default
1	Background	Bulk data
2	Excellent Effort	Important business
3	Critical Applications	Signaling
4	Video	Video streaming
5	Voice	VoIP bearer
6	Internetwork Control	Routing protocols
7	Network Control	Spanning tree, LLDP

Port Types

Type	Behavior
Access port	Carries one VLAN; frames are untagged on the wire
Trunk port	Carries multiple VLANs; frames are 802.1Q tagged
Native VLAN	VLAN carried untagged on a trunk (default: VLAN 1)
Voice VLAN	Separate VLAN for VoIP phones (tagged by the phone)

Trunk Operation

[Diagram: see online version]

Q-in-Q (802.1ad -- Double Tagging)

Service providers use a second tag (S-Tag) to tunnel customer VLANs across their network:

[Diagram: see online version]

Tag	TPID	Purpose
S-Tag (outer)	0x88A8	Service provider VLAN
C-Tag (inner)	0x8100	Customer VLAN (preserved)

MTU Impact

The 4-byte tag increases the maximum Ethernet frame from 1518 to **1522 bytes**. Q-in-Q adds another 4 bytes (1526 bytes). Baby giant frame support is needed on all intermediate equipment.

Standards

Document	Title
IEEE 802.1Q-2022	Bridges and Bridged Networks (VLANs, STP, and more)
IEEE 802.1ad	Provider Bridges (Q-in-Q)
IEEE 802.1p	Priority (incorporated into 802.1Q)

See Also

- Ethernet -- base frame that 802.1Q extends
- STP -- prevents loops in VLAN-segmented networks
- LLDP -- discovers VLAN assignments on neighbors
- LACP -- link aggregation across trunk ports

Media & Broadcast

Streaming, broadcast, and media delivery protocols.

8 protocols in this section

AES67 -- High-Performance Audio over IP

Standard: AES67-2018 | **Layer:** Application (Layer 7) | **Wireshark filter:** `rtp`

AES67 is an interoperability standard published by the Audio Engineering Society for high-performance audio-over-IP transport. Rather than inventing a new protocol, AES67 defines a constrained profile of existing standards -- RTP for media transport, IEEE 1588 PTP for synchronization, SDP for session description, and SAP or mDNS for discovery -- creating a common ground where proprietary audio networking ecosystems (Dante, Ravenna, Livewire+, Q-LAN) can exchange audio streams. AES67 delivers uncompressed linear PCM audio with latencies as low as 1 ms over standard Layer 3 IP networks, and serves as the audio transport foundation for SMPTE ST 2110-30 in broadcast.

Architecture

AES67 decomposes audio networking into four functional layers, each built on well-established standards:

[Diagram: see online version]

Protocol Stack

Layer	Protocol	Role
Synchronization	IEEE 1588-2008 (PTPv2)	Sub-microsecond media clock alignment
Media Transport	RTP over UDP	Carries PCM audio samples
Session Description	SDP (RFC 4566)	Describes stream parameters (codec, rate, channels, multicast address)
Discovery	SAP (RFC 2974), mDNS/DNS-SD, NMOS IS-04	Advertise and locate available streams
Network	IPv4 multicast (or unicast)	Standard Layer 3 IP infrastructure

RTP Payload

AES67 uses standard RTP (RFC 3550) with linear PCM payloads. No compression is applied -- samples are packed directly into the RTP payload:

[Diagram: see online version]

Samples are packed in channel-interleaved order: all channels for sample 1, then all channels for sample 2, and so on. For L24 encoding, each sample is a 24-bit signed two's-complement integer in network byte order.

Key Parameters

Parameter	Value

Encoding	L16 (16-bit linear PCM) or L24 (24-bit linear PCM)
Sample rate	48 kHz (mandatory), 44.1 kHz (optional), 96 kHz (optional)
Channels per stream	1 to 64 (typically 8 or 16)
Packet time	1 ms (default, 48 samples at 48 kHz)
Other packet times	125 us (6 samples), 250 us (12 samples), 4 ms (192 samples)
RTP payload type	Dynamic (negotiated via SDP, typically 96-127)
RTP clock rate	Matches sample rate (48000 for 48 kHz)
Transport	UDP unicast or multicast
IP version	IPv4 (mandatory), IPv6 (optional)

Packet Time and Latency

The packet time determines the number of samples per RTP packet and directly impacts latency:

Packet Time	Samples (48 kHz)	Payload (stereo L24)	Total Bandwidth (stereo)	Use Case
125 us	6	36 bytes	~2.9 Mbps	Ultra-low latency
250 us	12	72 bytes	~2.6 Mbps	Low latency
1 ms	48	288 bytes	~2.5 Mbps	Default / recommended
4 ms	192	1152 bytes	~2.4 Mbps	Higher channel count

Bandwidth per Channel

Bit Depth	Sample Rate	Packet Time	Bandwidth per Channel
L24 (24-bit)	48 kHz	1 ms	~1.2 Mbps
L16 (16-bit)	48 kHz	1 ms	~0.8 Mbps
L24 (24-bit)	96 kHz	1 ms	~2.4 Mbps

Bandwidth includes RTP/UDP/IP headers. At 1 ms packet time with L24/48 kHz, a 64-channel stream requires approximately 76 Mbps.

PTP Synchronization (IEEE 1588-2008)

AES67 requires PTPv2 (IEEE 1588-2008) to synchronize media clocks across all devices with sub-microsecond accuracy:

[Diagram: see online version]

AES67 PTP Profile

Parameter	AES67 Default Profile	AES67 Media Profile
PTP version	IEEE 1588-2008 (PTPv2)	IEEE 1588-2008 (PTPv2)
Domain	0	0
Transport	UDP/IPv4 multicast (239.x.x.x)	UDP/IPv4 multicast
Sync interval	1/8 s (125 ms)	1/8 s (125 ms)
Announce interval	1 s	1 s
Delay mechanism	End-to-end (E2E)	End-to-end (E2E)
BMCA	Default (IEEE 1588)	Default (IEEE 1588)
Priority1 / Priority2	128 / 128	128 / 128

Session Description (SDP)

Each AES67 stream is described by an SDP (RFC 4566) document specifying the media format, multicast address, and PTP clock reference:

```
v=0
o=- 1234 5678 IN IP4 192.168.1.10
s=AES67 Mic Array
t=0 0
m=audio 5004 RTP/AVP 98
c=IN IP4 239.69.1.1/32
a=rtpmap:98 L24/48000/8
a=ptime:1
a=mediaclk:direct=0
a=ts-refclk:ptp=IEEE1588-2008:00-1D-C1-FF-FE-00-12-34:0
a=recvonly
```

Key SDP Attributes

Attribute	Description
`rtpmap:98 L24/48000/8`	Payload type 98, L24 encoding, 48 kHz sample rate, 8 channels
`ptime:1`	1 ms packet time
`mediaclk:direct=0`	Media clock offset from RTP timestamp
`ts-refclk:ptp=IEEE1588-2008:...`	PTP clock identity and domain for this stream

Discovery

AES67 does not mandate a single discovery mechanism. Multiple methods are used depending on the ecosystem:

Method	Protocol	Description
SAP	RFC 2974 (UDP 9875, multicast 239.255.255.255)	Session Announcement Protocol -- periodic SDP broadcast

Bonjour/mDNS	RFC 6762/6763 (UDP 5353, multicast 224.0.0.251)	DNS-SD service registration (`_rtsp._tcp` or `_ravenna._tcp`)
NMOS IS-04	AMWA REST API	Networked Media Open Specifications registry and query API
Manual	SDP file import	Direct SDP exchange for static configurations

Stream Setup Flow

[Diagram: see online version]

Interoperability

AES67 serves as a common interoperability layer between proprietary audio networking ecosystems. Each system can generate and receive AES67-compliant streams:

Ecosystem	Vendor	AES67 Interop	Method
Dante	Audinate	Bidirectional	AES67 mode enabled per device
Ravenna	ALC NetworX	Native	Ravenna is AES67-superset
Livewire+	Telos Alliance	Bidirectional	AES67 mode in Axia devices
Q-LAN	QSC	Bidirectional	AES67 compatibility mode
SMPTE ST 2110-30	Broadcast	Native	ST 2110-30 is AES67-compliant
NMOS	AMWA	Discovery	IS-04/IS-05 for AES67 stream management

AES67 vs Dante vs AVB

Feature	AES67	Dante	AVB (IEEE 802.1)
Standard	AES67-2018 (open)	Audinate (proprietary)	IEEE 802.1BA (open)
Layer	Layer 3 (IP/UDP/RTP)	Layer 3 (IP/UDP)	Layer 2 (Ethernet)
Synchronization	PTP (IEEE 1588-2008)	PTP (IEEE 1588)	gPTP (IEEE 802.1AS)
Audio format	L16/L24 PCM, uncompressed	16/24/32-bit PCM	AAF (AVTP Audio Format)
Sample rates	44.1, 48, 96 kHz	44.1, 48, 88.2, 96 kHz	44.1, 48, 96 kHz
Channels per stream	Up to 64	Up to 512 per device	Up to 60 per stream (Class A)
Latency	1-5 ms typical	0.15-5 ms (selectable)	2 ms / 7 hops (Class A)
Infrastructure	Standard IP (Layer 3)	Standard IP (Layer 3)	AVB-capable switches (Layer 2)

Discovery	SAP, mDNS, NMOS	mDNS/DNS-SD, Dante Controller	AVDECC (IEEE 1722.1)
Multicast	Supported (primary)	Unicast and multicast	Multicast (Layer 2)
Bandwidth reservation	None (best effort or QoS)	None (QoS recommended)	SRP (IEEE 802.1Qat) guaranteed
Routing across subnets	Yes (Layer 3)	Yes (Layer 3)	No (Layer 2 only without TSN)
Ecosystem size	Growing (interop standard)	Dominant (100,000+ products)	Niche (automotive, some pro audio)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
AES67-2018	High-performance streaming audio-over-IP interoperability
IEEE 1588-2008	Precision Time Protocol (PTPv2)
RFC 3550	RTP: A Transport Protocol for Real-Time Applications
RFC 3190	RTP Payload Format for 12-bit DAT, 20/24-bit Linear (L24)
RFC 4566	SDP: Session Description Protocol
RFC 2974	SAP: Session Announcement Protocol
RFC 7273	RTP Clock Source Signalling (ts-refclk, mediaclk)
AMWA NMOS IS-04	Discovery & Registration
AMWA NMOS IS-05	Device Connection Management

See Also

- SMPTE ST 2110 -- professional media over IP (ST 2110-30 audio is AES67-compatible)
- RTP -- underlying transport protocol for AES67 audio
- SDP -- session description format for AES67 stream parameters
- mDNS -- multicast DNS used for Bonjour/DNS-SD discovery
- NTP -- time synchronization (PTP is the high-precision variant)
- Dante -- proprietary audio networking with AES67 interoperability
- AVB / TSN -- IEEE Layer 2 audio/video bridging

AVB / TSN -- Audio Video Bridging / Time-Sensitive Networking

Standard: IEEE 802.1BA | **Layer:** Data Link (Layer 2) | **Wireshark filter:** `avtp` or `ieee1722`

AVB (Audio Video Bridging) is a suite of IEEE 802.1 standards that provides guaranteed, low-latency transport of time-sensitive audio and video streams over standard Ethernet at Layer 2, without requiring IP. AVB delivers deterministic timing, bandwidth reservation, and bounded latency -- capabilities absent from standard Ethernet. The IEEE later expanded AVB into TSN (Time-Sensitive Networking), adding scheduled traffic, centralized configuration, and frame replication for industrial, automotive, and aerospace applications. AVB/TSN operates below the IP layer, using a dedicated EtherType (0x22F0) and requiring AVB-capable switches, but in return guarantees worst-case latency of 2 ms over 7 hops for Class A streams.

Standards Suite

AVB (Original Suite)

Standard	Name	Description
IEEE 802.1AS	gPTP	Generalized Precision Time Protocol -- profile of IEEE 1588 for Layer 2
IEEE 802.1Qav	FQTSS	Forwarding and Queuing for Time-Sensitive Streams -- credit-based shaper
IEEE 802.1Qat	SRP	Stream Reservation Protocol -- bandwidth reservation via MMRP/MVRP
IEEE 802.1BA	AVB Systems	Profile combining the above into a complete AVB system

TSN Extensions (Superset of AVB)

Standard	Name	Description
IEEE 802.1Qbv	TAS	Time-Aware Shaper -- scheduled traffic with gate control lists
IEEE 802.1Qcc	SRP Enhancements	Centralized and hybrid stream reservation models
IEEE 802.1CB	FRER	Frame Replication and Elimination for Reliability
IEEE 802.1Qci	PSFP	Per-Stream Filtering and Policing
IEEE 802.1Qcr	ATS	Asynchronous Traffic Shaping
IEEE 802.1AS-2020	gPTP rev	Revised gPTP with multiple domains, hot standby

IEEE 802.1Qch	CQF	Cyclic Queuing and Forwarding

gPTP Synchronization (IEEE 802.1AS)

All AVB/TSN devices synchronize to a common time base using gPTP, a profile of IEEE 1588 optimized for bridged Ethernet networks. gPTP operates at Layer 2 and measures link delay on each hop:

[Diagram: see online version]

gPTP Parameters

Parameter	Value
PTP profile	IEEE 802.1AS (gPTP)
Transport	Layer 2, EtherType 0x88F7
Delay mechanism	Peer-to-peer (Pdelay) only
Sync interval	125 ms (default)
Pdelay interval	1 s (default)
Accuracy	< 1 us across 7 hops (typical)
Domain	Single domain (AVB), multiple domains (802.1AS-2020)
BMCA	gPTP BMCA (similar to IEEE 1588 but simplified)

Stream Reservation Protocol (IEEE 802.1Qat)

Before an AVB talker can transmit, it must reserve bandwidth along the entire path to the listener. SRP uses MMRP (Multiple MAC Registration Protocol) to propagate reservation requests hop by hop:

[Diagram: see online version]

SRP Bandwidth Reservation

Parameter	Class A	Class B
Maximum latency per hop	250 us	2 ms
Maximum latency (7 hops)	2 ms	50 ms
Reserved bandwidth	Up to 75% of link capacity	Up to 75% of link capacity
VLAN Priority (PCP)	3	2
VLAN ID	2 (default)	2 (default)
Observation interval	125 us	250 us

AVTP (IEEE 1722) -- Audio Video Transport Protocol

AVTP is the media transport protocol for AVB, operating at Layer 2 with EtherType 0x22F0. It carries audio, video, clock reference, and control data in a common framing structure:

AVTP Common Header

[Diagram: see online version]

AVTP Key Fields

Field	Size	Description
subtype	7 bits	AVTP data type (AAF=0x02, CVF=0x03, CRF=0x04, etc.)
sv	1 bit	Stream ID valid
version	3 bits	AVTP version (0)
mr	1 bit	Media clock restart
gv	1 bit	Gateway info valid
tv	1 bit	AVTP timestamp valid
sequence_num	8 bits	Per-stream packet sequence counter
tu	1 bit	Timestamp uncertain
stream_id	64 bits	Unique stream identifier (MAC + unique ID)
avtp_timestamp	32 bits	Presentation time (gPTP-derived, nanoseconds)

AVTP Subtypes

Subtype	Value	Name	Description
AAF	0x02	AVTP Audio Format	Uncompressed PCM audio (L16, L24, L32, float)
CVF	0x03	Compressed Video Format	H.264, MJPEG compressed video
CRF	0x04	Clock Reference Format	Clock recovery for non-gPTP media clocks
TSCF	0x05	Time-Synchronous Control Format	Timestamped control messages
NTSCF	0x06	Non-Time-Synchronous Control Format	Non-timestamped control
ACF	--	AVTP Control Format	Vehicle bus tunneling (CAN, LIN, FlexRay)

AAF (AVTP Audio Format) Parameters

Parameter	Value
Bit depths	16, 24, 32-bit integer; 32-bit float
Sample rates	8, 16, 32, 44.1, 48, 88.2, 96, 176.4, 192 kHz
Channels per stream	1 to 60+ (Class A), limited by bandwidth reservation
Frames per packet	Configurable (8 typical at 48 kHz for Class A)
Presentation time	gPTP-derived timestamp in AVTP header

AVDECC (IEEE 1722.1) -- Discovery and Control

AVDECC provides discovery, enumeration, connection management, and control for AVB endpoints:

[Diagram: see online version]

AVDECC Protocols

Protocol	Full Name	Description
ADP	AVDECC Discovery Protocol	Entity advertisement and discovery (multicast)
AECP	AVDECC Enumeration and Control Protocol	Read/write entity descriptors and parameters
ACMP	AVDECC Connection Management Protocol	Establish and tear down stream connections

MAAP (MAC Address Acquisition Protocol)

AVB streams use multicast destination MAC addresses from the reserved range 91:E0:F0:00:00:00 to 91:E0:F0:00:FD:FF. MAAP dynamically allocates these addresses to avoid conflicts:

Parameter	Value
EtherType	0x22F0 (shared with AVTP)
MAC range	91:E0:F0:00:00:00 -- 91:E0:F0:00:FD:FF
Allocation	Dynamic per-stream (MAAP protocol)
Probe count	3 probes before claiming

TSN Traffic Shaping

TSN extends AVB with more sophisticated traffic shaping mechanisms:

Credit-Based Shaper (802.1Qav -- AVB)

The original AVB shaper limits burst size using a credit counter. A stream can only transmit when its credit is non-negative:

Parameter	Description
idleSlope	Rate at which credit accumulates when not transmitting
sendSlope	Rate at which credit depletes when transmitting
maxCredit	Maximum accumulated credit
minCredit	Maximum negative credit (debt from last frame)

Time-Aware Shaper (802.1Qbv -- TSN)

TAS adds scheduled time windows (gates) for different traffic classes, enabling deterministic worst-case latency:

Feature	Description
Gate Control List (GCL)	Cyclic schedule defining which queues can transmit in each time slot
Guard band	Protected interval before gate close to prevent frame truncation
Cycle time	Repetition period of the gate schedule
Time source	gPTP synchronized across all bridges

Layer 2 Operation

AVB/TSN operates entirely at Layer 2 -- no IP stack is required:

Parameter	Value
AVTP EtherType	0x22F0
gPTP EtherType	0x88F7
VLAN	Required (802.1Q tagged, default VLAN ID 2)
Priority (PCP)	3 (Class A), 2 (Class B)
Link speeds	100 Mbps, 1 Gbps, 2.5 Gbps, 5 Gbps, 10 Gbps
Switch requirement	AVB/TSN-capable bridges (hardware support required)
IP encapsulation	Not required, but possible (RTP/IP over TSN)

Use Cases

Domain	Application	Why AVB/TSN
Automotive	In-vehicle audio, camera, ADAS	Deterministic latency, Layer 2 efficiency, weight savings

Pro Audio	Live sound, installed AV (Milan)	Guaranteed bandwidth, interoperability (Milan profile)
Industrial	Motion control, factory automation	Hard real-time, frame replication (FRER)
Aerospace	Avionics data buses	Deterministic, redundant, standard Ethernet
Broadcast	Studio audio/video	Bounded latency, synchronization

Milan

Milan is an AVB profile defined by the Avnu Alliance for pro audio and AV interoperability. It mandates specific AVTP formats, AVDECC behaviors, and SRP parameters to ensure plug-and-play operation across Milan-certified devices.

AVB/TSN vs AES67 vs Dante vs SMPTE ST 2110

Feature	AVB/TSN	AES67	Dante	SMPTE ST 2110
Standard	IEEE 802.1 (open)	AES67-2018 (open)	Audinate (proprietary)	SMPTE (open)
OSI Layer	Layer 2	Layer 3 (IP)	Layer 3 (IP)	Layer 3 (IP)
Synchronization	gPTP (802.1AS)	PTP (IEEE 1588)	PTP (IEEE 1588)	PTP (ST 2059)
Bandwidth guarantee	SRP reservation (guaranteed)	None (QoS/DSCP)	None (QoS/DSCP)	None (QoS/DSCP)
Latency	2 ms / 7 hops (Class A)	1-5 ms (typical)	0.15-5 ms (selectable)	< 1 frame
Routing	Layer 2 only (no IP routing)	Layer 3 routable	Layer 3 routable	Layer 3 routable
Switch requirement	AVB/TSN capable	Standard IP	Standard IP	Managed IP
Media types	Audio + video + control	Audio only	Audio (+ video via Dante AV)	Video + audio + data
Discovery	AVDECC (IEEE 1722.1)	SAP, mDNS, NMOS	mDNS, Dante Controller	NMOS IS-04/IS-05
Video support	CVF (H.264, MJPEG)	No	JPEG 2000 (Dante AV)	Uncompressed / JPEG XS
AES67 interop	Via IP gateway	Native	AES67 mode	ST 2110-30 (AES67)
Redundancy	FRER (802.1CB)	ST 2022-7 (at IP layer)	Dual NIC	ST 2022-7
Ecosystem	Automotive, Milan pro audio	Broadcast interop layer	100,000+ pro audio products	Broadcast facilities

Encapsulation

[Diagram: see online version]

Standards

Document	Title
IEEE 802.1AS	Timing and Synchronization for Time-Sensitive Applications (gPTP)
IEEE 802.1Qav	Forwarding and Queuing for Time-Sensitive Streams (FQTSS)
IEEE 802.1Qat	Stream Reservation Protocol (SRP)
IEEE 802.1BA	Audio Video Bridging Systems
IEEE 1722-2016	Audio Video Transport Protocol (AVTP)
IEEE 1722.1-2013	AVDECC Discovery, Enumeration, Connection management, and Control
IEEE 802.1Qbv	Enhancements for Scheduled Traffic (TAS)
IEEE 802.1Qcc	Stream Reservation Protocol Enhancements and Performance Improvements
IEEE 802.1CB	Frame Replication and Elimination for Reliability (FRER)
IEEE 802.1Qci	Per-Stream Filtering and Policing (PSFP)
Avnu Alliance Milan	Pro AV profile for AVB interoperability

See Also

- AES67 -- Layer 3 audio-over-IP interoperability standard
- Dante -- dominant proprietary audio networking protocol
- SMPTE ST 2110 -- professional media over managed IP networks
- Ethernet -- underlying link layer for AVB/TSN
- VLAN (802.1Q) -- VLAN tagging required by AVB streams
- NTP -- time synchronization (gPTP is the precision Layer 2 variant)
- RTP -- transport protocol used by IP-based alternatives (AES67, ST 2110)

Dante -- Digital Audio Network Through Ethernet

Standard: Audinate Dante | **Layer:** Application (Layer 7) | **Wireshark filter:** `mdns` (discovery) or `rtp` (AES67 mode)

Dante is a proprietary audio (and video) networking technology developed by Audinate that transports uncompressed, multi-channel digital audio over standard Ethernet networks using Layer 3 IP. It is the dominant audio networking protocol in professional audio, integrated into over 100,000 products from more than 500 manufacturers including Shure, Yamaha, Harman, Bose, and Biamp. Dante uses IEEE 1588 PTP for synchronization, mDNS/DNS-SD for zero-configuration device discovery, and a proprietary framing and management protocol for audio routing. Dante supports AES67 bidirectional interoperability and has expanded into video transport with Dante AV.

Architecture

Dante uses a controller-managed architecture where devices discover each other automatically and audio routing is established through Dante Controller software or Dante Domain Manager:

[Diagram: see online version]

Discovery

Dante uses mDNS/DNS-SD (Bonjour) for automatic zero-configuration device discovery on the local subnet. Devices register service instances that describe their capabilities:

Service Types

Service	Protocol	Description
`_netaudio-dbc._udp`	mDNS/DNS-SD	Dante device browsing and control
`_netaudio-arc._udp`	mDNS/DNS-SD	Dante audio routing control
`_netaudio-cmc._udp`	mDNS/DNS-SD	Dante clock management
`_netaudio-dcp._udp`	mDNS/DNS-SD	Dante control protocol

Discovery Methods

Method	Scope	Description
mDNS/DNS-SD	Local subnet	Automatic zero-config discovery (default)
Dante Controller	Single/multi-subnet	Software application for managing routing
Dante Domain Manager	Enterprise	Server-based management, cross-subnet, LDAP auth
Dante Discovery (unicast DNS)	Multi-subnet	DNS-SD via unicast DNS for routed networks

Audio Transport

Dante carries audio as uncompressed PCM samples in a proprietary UDP framing format. Flows are established as unicast (point-to-point) or multicast (one-to-many):

[Diagram: see online version]

Note: The exact Dante framing format is proprietary and not publicly documented. The above represents the general structure based on observed behavior.

Audio Parameters

Parameter	Value
Bit depths	16-bit, 24-bit, 32-bit integer
Sample rates	44.1 kHz, 48 kHz, 88.2 kHz, 96 kHz
Channels per device	Up to 512 transmit and 512 receive (hardware dependent)
Channels per flow	Typically 1-8 (unicast), 1-8 (multicast)
Encoding	Linear PCM, uncompressed
Transport	UDP unicast (default) or UDP multicast
Multicast range	239.255.x.x (auto-assigned)

Latency Presets

Dante offers device-configurable latency presets that determine the receive buffer depth:

Latency Preset	Buffer	Network Hops	Use Case
0.15 ms	150 us	1-2	Single switch, lowest latency
0.25 ms	250 us	1-3	Small networks
0.5 ms	500 us	3-5	Medium networks (default for many devices)
1 ms	1 ms	5-10	Large switched networks
2 ms	2 ms	10+	Very large networks, multiple subnets
5 ms	5 ms	Any	Routed networks, Wi-Fi bridges

Latency is measured from the transmitter DAC to the receiver DAC. All receiving devices on a network should use the same latency setting for synchronous playout.

PTP Synchronization

Dante uses IEEE 1588-2008 (PTPv2) for network-wide clock synchronization. One Dante device is elected as the PTP Grandmaster (clock master) using the Best Master Clock Algorithm (BMCA):

[Diagram: see online version]

Dante PTP Parameters

Parameter	Value
PTP version	IEEE 1588-2008 (PTPv2)
Transport	UDP/IPv4 (ports 319, 320)
Domain	0 (default)
Sync interval	1/8 s (125 ms)

BMCA priority	Automatic or user-configurable (Preferred Master)
Accuracy	Sub-microsecond (< 1 us across managed switches)

Network Requirements

Requirement	Details
Minimum link speed	100 Mbps (limited channels), 1 Gbps recommended
QoS (DSCP)	EF (46) for audio, CS7 (56) for PTP, CS5 (40) for control
IGMP snooping	Required for multicast flows
EEE (Energy Efficient Ethernet)	Must be disabled (802.3az causes timing issues)
Spanning Tree	RSTP recommended, convergence time affects redundancy
Managed switches	Recommended (required for QoS and multicast management)

Ports

Port Range	Protocol	Purpose
319, 320	UDP	PTP (IEEE 1588) synchronization
5353	UDP	mDNS discovery (multicast 224.0.0.251)
8700-8706	TCP/UDP	Dante control and monitoring
14336-14600	UDP	Audio transport (auto-negotiated per flow)
4321, 4440	TCP	Dante Controller communication
38700-38800	UDP	Dante AV video transport

Dante Products and Software

Product	Description
Dante Controller	Free Windows/Mac application for routing and device management
Dante Domain Manager	Enterprise server software: user auth (LDAP), multi-subnet routing, audit logging
Dante Via	Software driver for routing computer audio (any application) as Dante
Dante Virtual Soundcard	ASIO/WDM/Core Audio driver connecting a computer to Dante
Dante AVIO	Hardware adapters (analog, AES3, USB, Bluetooth to Dante)
Dante AV	Video extension using JPEG 2000 compression

Dante AV

Dante AV extends the Dante ecosystem to include synchronized video transport:

Parameter	Value
Video codec	JPEG 2000 (low latency profile)
Video latency	< 1 frame (typical)
Resolutions	Up to 4K UHD (3840x2160)
Synchronization	PTP (shared with audio clock)
Transport	UDP (proprietary framing)
Audio	Standard Dante audio (synchronized with video)

AES67 Interoperability

Dante devices can enable AES67 compatibility mode, allowing bidirectional audio exchange with any AES67-compliant system:

[Diagram: see online version]

AES67 Mode Constraints

Parameter	Dante Native	Dante AES67 Mode
Encoding	16/24/32-bit PCM	L16 or L24 PCM only
Sample rate	44.1-96 kHz	48 kHz (mandatory), 44.1/96 kHz (optional)
Packet time	Device-dependent	1 ms (required for interop)
Transport	Proprietary UDP	Standard RTP/UDP
Discovery	mDNS (Dante)	SAP + mDNS
Channels per flow	Up to 8	Up to 8 per multicast flow

Redundancy

Dante supports network redundancy via dual-NIC (primary and secondary) Ethernet ports on supported devices:

Feature	Description
Mode	Redundant (active backup) or Switched (daisy-chain)
Failover	Automatic, hitless (< 1 ms interruption)
Topology	Two independent network fabrics (primary/secondary)
Requirement	Both NICs must be same speed, separate switches

Dante vs AES67 vs AVB vs MADI

Feature	Dante	AES67	AVB (802.1BA)	MADI (AES10)
Standard	Proprietary (Audinate)	AES67-2018 (open)	IEEE 802.1 (open)	AES10-2008 (open)
Layer	Layer 3 (IP/UDP)	Layer 3 (IP/UDP/RTP)	Layer 2 (Ethernet)	Physical (coax/fiber)
Channels	512 per device	64 per stream	60 per stream (Class A)	64 per link
Sample rates	44.1-96 kHz	44.1, 48, 96 kHz	48, 96 kHz	44.1, 48 kHz
Bit depth	16/24/32-bit	16/24-bit	16/24/32-bit	24-bit
Latency	0.15-5 ms (selectable)	1-5 ms	2 ms / 7 hops (Class A)	Fixed (frame-based)
Distance	Unlimited (IP routed)	Unlimited (IP routed)	Layer 2 domain	2 km (fiber), 100 m (coax)
Sync	PTP (IEEE 1588)	PTP (IEEE 1588)	gPTP (802.1AS)	Word clock / internal
Bandwidth reservation	QoS (DSCP)	QoS (DSCP)	SRP (guaranteed)	Dedicated link
Infrastructure	Standard IP switches	Standard IP switches	AVB-capable switches	Point-to-point
Products	100,000+	Growing (interop)	Limited (pro audio)	Widespread (studios)

Encapsulation

[Diagram: see online version]

In AES67 interop mode:

[Diagram: see online version]

Standards

Document	Title
Audinate Dante	Dante technology overview and specifications
AES67-2018	AES67 interoperability standard
IEEE 1588-2008	Precision Time Protocol (PTPv2)
RFC 6762	Multicast DNS (mDNS)
RFC 6763	DNS-Based Service Discovery (DNS-SD)

See Also

- AES67 -- open interoperability standard for audio over IP
- SMPTE ST 2110 -- professional media over IP (ST 2110-30 audio via AES67)
- AVB / TSN -- IEEE Layer 2 audio/video bridging with guaranteed bandwidth

- RTP -- transport protocol used in Dante AES67 mode
- mDNS -- multicast DNS used for Dante device discovery
- SDP -- session description for AES67 interop streams
- NTP -- time synchronization (PTP is the precision variant used by Dante)

NDI -- Network Device Interface

Standard: NDI SDK | **Layer:** Application (Layer 7) | **Wireshark filter:** `ndi` (limited; proprietary protocol)

NDI is a royalty-free IP video protocol developed by NewTek (now Vizrt) for high-performance, low-latency video transport over standard Ethernet. It enables video sources and receivers to discover each other automatically and exchange video, audio, and metadata frames with minimal configuration. NDI is widely used in live production, streaming, corporate AV, and esports where sub-frame latency video over commodity networks is needed. Unlike SMPTE ST 2110, NDI runs on standard gigabit infrastructure without PTP synchronization or network engineering.

Discovery

NDI sources are discovered automatically using mDNS multicast (UDP port 5353) on the local subnet, or via a centralized NDI Discovery Server for cross-subnet environments:

[Diagram: see online version]

Discovery Methods

Method	Transport	Scope	Port
mDNS (default)	UDP multicast 224.0.0.251	Local subnet	5353
NDI Discovery Server	TCP unicast	Cross-subnet / WAN	5959
NDI 5 Bridge	TCP + reliable UDP	WAN / internet	Configurable

Transport Modes

NDI supports multiple transport variants optimized for different bandwidth and quality tradeoffs:

Variant	Compression	Typical Bandwidth (1080p60)	Latency	Use Case
NDI (full)	SpeedHQ (visually lossless)	~125 Mbps	< 1 frame	LAN production, switching
NDI\	HX	H.264	~12-20 Mbps	2-4 frames
NDI\	HX2	H.264 / H.265	~8-15 Mbps	1-2 frames

NDI\	HX3	H.264 / H.265 / NDI codec	~6-20 Mbps	< 1 frame

Transport Protocols

Mode	Protocol	Description
Reliable TCP (default)	TCP	Single connection, lossless delivery, adaptive bitrate
Reliable UDP (NDI 5+)	UDP with FEC + ARQ	Lower latency, better congestion handling
Multicast UDP	UDP multicast	One-to-many, forward error correction
Unicast UDP	UDP	Single receiver, FEC

Frame Types

NDI carries three frame types over the transport connection:

[Diagram: see online version]

Video Frame

Field	Description
Resolution	Any resolution (common: 720p, 1080i, 1080p, 2160p)
Frame rate	Any rational rate (23.976, 25, 29.97, 30, 50, 59.94, 60, etc.)
Pixel format	UYVY, BGRA, BGRX, NV12, I420, P216, PA16
Alpha channel	Supported via BGRA and PA16 formats
Progressive/interlaced	Both supported; field-based or frame-based
Line stride	Configurable bytes-per-line for memory alignment

Audio Frame

Field	Description
Sample rate	Any rate (common: 44100, 48000, 96000 Hz)
Channels	Any count (mono through immersive audio)
Sample format	32-bit float (interleaved or planar)
Samples per frame	Variable (typically matches video frame duration)

Metadata Frame

Metadata is carried as UTF-8 XML strings, enabling arbitrary structured data exchange between sources and receivers:

```
&lt;ndi_metadata&gt;
  &lt;source name="Camera 1" /&gt;
  &lt;tally on_program="true" on_preview="false" /&gt;
  &lt;custom key="scene"&gt;Interview&lt;/custom&gt;
&lt;/ndi_metadata&gt;
```

Tally

NDI includes bidirectional tally signaling. Receivers send tally status back to sources indicating whether the source is currently on program (live) or preview:

State	Meaning
Program (PGM)	Source is live on air (red tally)
Preview (PVW)	Source is selected for preview (green tally)
Neither	Source is not actively selected

NDI 5 Features

Feature	Description
NDI Bridge	Connects NDI sources across WAN / internet via relay servers
NDI Remote	Allows remote contribution from any location
Reliable UDP	New transport layer with FEC and ARQ for improved performance
NDI KVM	Remote desktop control (keyboard, video, mouse) over NDI
Discovery Server	Centralized cross-subnet source registration

Ports

Port	Protocol	Purpose
5353	UDP	mDNS discovery (multicast 224.0.0.251)
5959	TCP	NDI Discovery Server
5960+	TCP/UDP	Media transport (auto-assigned per source)

NDI vs SDI vs SMPTE ST 2110

Feature	NDI	SDI (3G/12G)	SMPTE ST 2110

Infrastructure	Standard GigE / 10GbE	Dedicated coax (BNC)	Managed 10/25/100GbE
Compression	SpeedHQ (full) / H.264/H.265 (HX)	None	None (ST 2110-20) or JPEG XS (ST 2110-22)
Bandwidth (1080p60)	~125 Mbps (full), ~15 Mbps (HX)	3 Gbps	~3 Gbps (uncompressed)
Latency	< 1 frame (full), 1-4 frames (HX)	Zero (point-to-point)	< 1 frame
Synchronization	NTP / internal	Genlock / black burst	PTP (IEEE 1588)
Discovery	mDNS / Discovery Server	Physical cabling	NMOS IS-04/IS-05
Multicast	Supported	N/A (point-to-point)	Required
Max distance	Network reach (unlimited with Bridge)	~100m (coax), km (fiber)	Network reach
Alpha channel	Yes (BGRA, PA16)	Yes (dual-link, 12-bit)	Yes (ST 2110-20)
Cost	Low (commodity network)	Medium (dedicated infrastructure)	High (managed network + PTP)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
NDI SDK	NDI Software Development Kit and API reference
NDI Advanced SDK	Low-level SDK for hardware integration
NDI Specification	Protocol and ecosystem overview

See Also

- SMPTE ST 2110 -- professional uncompressed media over IP
- SMPTE ST 2022 -- professional video over IP (predecessor to ST 2110)
- RTP -- transport protocol used by ST 2110 and many streaming systems
- RTSP -- streaming control protocol
- mDNS -- multicast DNS used for NDI discovery
- SDP -- session description used by ST 2110

ONVIF -- Open Network Video Interface Forum

Standard: ONVIF Core Specification | **Layer:** Application (Layer 7) | **Wireshark filter:** `xml`

ONVIF is an open industry standard for IP-based security products -- primarily network cameras, video encoders, and NVRs (Network Video Recorders). It defines a common interface for device discovery, media streaming configuration, PTZ (Pan-Tilt-Zoom) control, event handling, analytics, and recording. ONVIF uses SOAP/XML web services for device management and control, while actual video streaming is delegated to RTSP/RTP. Nearly every major IP camera manufacturer supports ONVIF, making it the de facto standard for IP surveillance interoperability.

Profiles

ONVIF organizes functionality into profiles that define conformance requirements:

Profile	Name	Description
Profile S	Streaming	Video/audio streaming, PTZ control, multicast, relay outputs
Profile G	Recording & Storage	Edge storage, recording, search, replay via RTSP
Profile T	Advanced Streaming	H.265, imaging settings, motion alarms, metadata streaming
Profile C	Access Control	Door control, access point management, credential handling
Profile A	Access Control Config	Configuration of access rules, credentials, and schedules
Profile D	Peripheral Devices	Token-based access, peripherals (card readers, REX devices)
Profile M	Metadata & Analytics	Analytics configuration, metadata streaming, scene description

Protocol Stack

[Diagram: see online version]

WS-Discovery (Device Discovery)

ONVIF devices are discovered using WS-Discovery (OASIS standard), which sends SOAP messages over UDP multicast.

Parameter	Value
Multicast Address (IPv4)	239.255.255.250
UDP Port	3702
Message Format	SOAP 1.2 over UDP

Discovery Flow

[Diagram: see online version]

Probe Message (Simplified)

```
&lt;s:Envelope xmlns:s="http://www.w3.org/2003/05/soap-envelope"
             xmlns:a="http://schemas.xmlsoap.org/ws/2004/08/addressing"
             xmlns:d="http://schemas.xmlsoap.org/ws/2005/04/discovery"
             xmlns:tdn="http://www.onvif.org/ver10/network/wsdl"&gt;
  &lt;s:Header&gt;
    &lt;a:Action&gt;http://schemas.xmlsoap.org/ws/2005/04/discovery/Probe&lt;/a:Action&gt;
    &lt;a:MessageID&gt;uuid:msg-1234&lt;/a:MessageID&gt;
    &lt;a:To&gt;urn:schemas-xmlsoap-org:ws:2005:04:discovery&lt;/a:To&gt;
  &lt;/s:Header&gt;
  &lt;s:Body&gt;
    &lt;d:Probe&gt;
      &lt;d:Types&gt;tdn:NetworkVideoTransmitter&lt;/d:Types&gt;
    &lt;/d:Probe&gt;
  &lt;/s:Body&gt;
&lt;/s:Envelope&gt;
```

Authentication

ONVIF uses WS-UsernameToken with password digest for SOAP authentication:

[Diagram: see online version]

The password digest is computed as:

```
PasswordDigest = Base64(SHA-1(Nonce + Created + Password))
```

Field	Description
Username	Plaintext username
PasswordDigest	`Base64(SHA-1(nonce_raw + created_utf8 + password_utf8))`
Nonce	Random value (Base64 encoded), prevents replay
Created	Timestamp (ISO 8601), limits validity window

The WS-UsernameToken header is embedded in the SOAP Security header of every authenticated request. ONVIF also supports HTTP Digest authentication for RTSP streams.

ONVIF Services

Device Management Service

Action	Description

GetDeviceInformation	Manufacturer, model, firmware version, serial number, hardware ID
GetCapabilities	List of supported services and their endpoint URLs
GetServices	Detailed service list with version info (replaces GetCapabilities)
GetNetworkInterfaces	Network interface configuration
SetNetworkInterfaces	Configure IP, DHCP, DNS settings
GetSystemDateAndTime	Camera date/time and NTP settings
SystemReboot	Reboot the device
GetScopes	Discovery scopes (location, hardware, name)
SetScopes	Set discovery scope metadata

Media Service

The Media service manages media profiles -- named configurations that bind video/audio sources, encoders, PTZ, and analytics into a streamable unit.

Action	Description
GetProfiles	List all media profiles
GetProfile	Get a specific profile by token
GetStreamUri	Get the RTSP URI for a profile
GetSnapshotUri	Get the HTTP URI for a JPEG snapshot
GetVideoEncoderConfigurations	List video encoder settings
SetVideoEncoderConfiguration	Set resolution, framerate, bitrate, codec
GetVideoSourceConfigurations	List video source settings
GetAudioEncoderConfigurations	List audio encoder settings

Stream Setup Flow

[Diagram: see online version]

Video Encoder Settings

Parameter	Description
Encoding	H.264, H.265, MPEG-4, JPEG
Resolution	Width x Height (e.g., 1920x1080)
Quality	Encoder quality factor (1-100)
FrameRateLimit	Maximum frame rate
BitrateLimit	Maximum bitrate (kbps)

GovLength	GOP length (keyframe interval)
H264Profile	Baseline, Main, Extended, High
RateControlType	CBR, VBR

PTZ Service

The PTZ service controls pan, tilt, and zoom on motorized cameras and virtual PTZ on fixed cameras.

Action	Description
ContinuousMove	Start continuous movement at given velocity (pan, tilt, zoom)
AbsoluteMove	Move to absolute position (x, y, zoom)
RelativeMove	Move relative to current position
Stop	Stop all PTZ movement
GotoPreset	Move to a saved preset position
SetPreset	Save current position as a preset
RemovePreset	Delete a preset
GetPresets	List all saved presets
GotoHomePosition	Move to home position
GetStatus	Current PTZ position and move status
GetConfigurations	PTZ ranges and limits

PTZ Coordinate System

Axis	Range	Description
Pan	-1.0 to +1.0	Horizontal: -1.0 = far left, +1.0 = far right
Tilt	-1.0 to +1.0	Vertical: -1.0 = down, +1.0 = up
Zoom	0.0 to +1.0	0.0 = wide, 1.0 = maximum telephoto

PTZ Control Flow

[Diagram: see online version]

Event Service

The Event service provides alarm and status notifications using WS-BaseNotification.

Action	Description
CreatePullPointSubscription	Create a subscription and poll for events

PullMessages	Retrieve queued events from the pull point
GetEventProperties	List all supported event topics
Subscribe	Real-time push notification (WS-BaseNotification)
Unsubscribe	Cancel a subscription
Renew	Extend subscription timeout

Common Event Topics

Topic	Description
`tns1:VideoSource/MotionAlarm`	Motion detection triggered
`tns1:RuleEngine/CellMotionDetector/Motion`	Cell-based motion detected
`tns1:RuleEngine/TamperDetector/Tamper`	Camera tamper detected
`tns1:Device/Trigger/DigitalInput`	Digital input (alarm contact) state change
`tns1:Device/HardwareFailure/StorageFailure`	Storage failure (Profile G)
`tns1:VideoAnalytics/...`	Analytics events (line crossing, intrusion)
`tns1:PTZController/PTZPresets/Reached`	PTZ reached a preset position

Pull-Point Event Flow

[Diagram: see online version]

Other Services

Analytics Service

Action	Description
GetSupportedAnalyticsModules	List available analytics (motion, face, LPR, line crossing)
CreateAnalyticsModules	Configure analytics on a profile
GetSupportedRules	List available rule types
CreateRules	Create analytics rules (e.g., line crossing, field intrusion)

Imaging Service

Action	Description
GetImagingSettings	Brightness, contrast, saturation, sharpness, exposure, WDR
SetImagingSettings	Adjust image parameters
GetOptions	Valid ranges for imaging parameters

Move	Focus control (absolute, relative, continuous)

Recording Service (Profile G)

Action	Description
CreateRecording	Create a recording on edge storage
CreateRecordingJob	Start/schedule recording
GetRecordings	List recordings
GetRecordingJobs	List recording jobs and states

Replay Service (Profile G)

Action	Description
GetReplayUri	Get RTSP URI for recorded playback

The replay URI supports RTSP Range headers for seeking through recorded video.

ONVIF SOAP Endpoint Structure

A typical ONVIF device exposes these HTTP endpoints:

Service	Typical Endpoint
Device	`http://<ip>/onvif/device_service`
Media	`http://<ip>/onvif/media_service`
Media2	`http://<ip>/onvif/media2_service`
PTZ	`http://<ip>/onvif/ptz_service`
Events	`http://<ip>/onvif/event_service`
Analytics	`http://<ip>/onvif/analytics_service`
Imaging	`http://<ip>/onvif/imaging_service`
Recording	`http://<ip>/onvif/recording_service`
Replay	`http://<ip>/onvif/replay_service`

All endpoints accept SOAP 1.2 POST requests with `Content-Type: application/soap+xml`.

Encapsulation

[Diagram: see online version]

Standards

Document	Title
ONVIF Core Spec	Core framework, discovery, security
ONVIF Media Service	Media profiles, streaming, encoding
ONVIF PTZ Service	Pan-tilt-zoom control
ONVIF Event Service	Alarms, notifications, pull points
ONVIF Analytics Service	Video analytics configuration
ONVIF Profile S	Streaming conformance
ONVIF Profile G	Recording and storage conformance
ONVIF Profile T	Advanced streaming (H.265, imaging)
WS-Discovery	OASIS Web Services Dynamic Discovery

See Also

- RTSP -- media streaming control (carries ONVIF video)
- RTP -- real-time media transport for video/audio
- UPnP/DLNA -- consumer media sharing with similar discovery patterns
- SIP -- VoIP signaling, sometimes used alongside ONVIF
- HTTP -- transport for SOAP control messages

SMPTE ST 2022 -- Professional Video over IP

Standard: SMPTE ST 2022 | **Layer:** Application (Layer 7) | **Wireshark filter:** `smpte_2022` or `rtp`

SMPTE ST 2022 is a suite of standards for transporting professional video over IP networks, developed as the broadcast industry's first standardized approach to replacing SDI with IP. It covers MPEG transport streams, uncompressed SDI mapping, forward error correction (FEC), and seamless network redundancy. While ST 2110 has largely superseded ST 2022 for new facility designs, ST 2022-7 (seamless protection switching) remains fundamental to ST 2110 deployments, and ST 2022-6 (uncompressed SDI over IP) is still found in many operational broadcast facilities.

Suite of Standards

Standard	Title	Description
ST 2022-1	FEC for MPEG-TS	Forward Error Correction for MPEG transport streams
ST 2022-2	Unidirectional MPEG-TS over IP	Constant bitrate MPEG-TS in UDP/RTP

ST 2022-5	FEC for High Bitrate Media	FEC for uncompressed and high-bitrate streams
ST 2022-6	Uncompressed SDI over IP	Maps entire SDI signal into RTP (HBRMT)
ST 2022-7	Seamless Protection Switching	Dual-path hitless redundancy for any RTP stream

Forward Error Correction (ST 2022-1 / ST 2022-5)

ST 2022-1 defines a 2D FEC scheme based on Pro-MPEG CoP3 that recovers lost packets without retransmission. Media packets are arranged in a matrix, and XOR-based FEC packets are generated for each column and row:

[Diagram: see online version]

FEC Key Fields

Field	Size	Description
SNBase	24 bits	Base sequence number of protected packets
Length Recovery	16 bits	XOR of payload lengths of protected packets
E (Extension)	1 bit	RTP header extension recovery
PT Recovery	7 bits	XOR of payload type fields
Mask	8 bits	Bitmask identifying protected packets
TS Recovery	24 bits	XOR of timestamps of protected packets
Offset	16 bits	Column spacing in the FEC matrix
NA	8 bits	Number of media packets per FEC group
D	1 bit	Direction: 0 = column FEC, 1 = row FEC
Index	16 bits	FEC packet index within the group

FEC Matrix

The 2D FEC matrix arranges media packets in rows and columns. Column FEC protects against burst loss; row FEC protects against random loss:

[Diagram: see online version]

FEC Parameters

Parameter	Symbol	Description
Columns	L	Number of packets per row (1-20)

Rows	D	Number of packets per column (4-20)
Column FEC		Recovers from burst loss up to L consecutive packets
Row FEC		Recovers from random single-packet loss per row
2D Recovery		Can recover multiple losses using both dimensions
Overhead		1/L + 1/D (e.g., L=10, D=10: 20% overhead)

MPEG Transport Stream over IP (ST 2022-2)

ST 2022-2 defines the carriage of MPEG-2 transport streams over IP. Each RTP packet carries 7 TS packets (7 x 188 = 1316 bytes) for a standard MTU:

[Diagram: see online version]

Parameter	Value
RTP payload type	Dynamic (negotiated via SDP)
Clock rate	90 kHz
TS packets per RTP	7 (typical, for 1316-byte payload)
Transport	UDP unicast or multicast
FEC	ST 2022-1 (optional but recommended)

Uncompressed SDI over IP (ST 2022-6)

ST 2022-6 maps an entire SDI signal (including blanking, embedded audio, and ancillary data) directly into RTP using the High Bitrate Media Transport (HBRMT) payload format:

HBRMT Payload Header

[Diagram: see online version]

HBRMT Key Fields

Field	Size	Description
V (Version)	2 bits	HBRMT version (0)
CF (Clock Frequency)	1 bit	0 = not locked, 1 = locked to reference
MAP	4 bits	SDI mapping mode (identifies signal format)
FRCount	8 bits	Frame count (wrapping counter for frame tracking)

R	8 bits	Reference for timing reconstruction

MAP Values (SDI Formats)

MAP	SDI Standard	Data Rate	Description
0	ST 259 (SD-SDI)	270 Mbps	Standard definition
1	ST 292-1 (HD-SDI)	1.485 Gbps	High definition
2	ST 425-1 (3G-SDI) A	2.97 Gbps	3G Level A
3	ST 425-1 (3G-SDI) B-DL	2.97 Gbps	3G Level B (dual-link)
4	ST 2081-10 (6G-SDI)	5.94 Gbps	6G single-link
5	ST 2082-10 (12G-SDI)	11.88 Gbps	12G single-link

Seamless Protection Switching (ST 2022-7)

ST 2022-7 provides hitless redundancy by transmitting identical RTP streams over two physically separate network paths. The receiver reconstructs a single clean output by selecting the first-arriving copy of each packet:

[Diagram: see online version]

ST 2022-7 Architecture

[Diagram: see online version]

ST 2022-7 Parameters

Parameter	Value
Path difference tolerance	Configurable (typically up to 450 ms)
Buffer depth	Must accommodate max path differential
Packet matching	By RTP sequence number
Failover time	Zero (hitless, no frame loss)
Scope	Per-flow (each RTP stream independently protected)
Applicable protocols	Any RTP stream (ST 2022-6, ST 2110, AES67, etc.)

ST 2022 vs ST 2110

Feature	SMPTE ST 2022-6	SMPTE ST 2110
Video mapping	Entire SDI signal (active + blanking)	Active video pixels only
Audio	Embedded in SDI stream	Separate audio flow (ST 2110-30)
Ancillary data	Embedded in SDI stream	Separate data flow (ST 2110-40)

Bandwidth (1080i)	~1.5 Gbps (full SDI rate)	~1.3 Gbps (active video only)
Essence routing	All-or-nothing (single flow)	Independent per-essence routing
Synchronization	PTP (ST 2059)	PTP (ST 2059)
Multicast	Supported	Required
Compression	No (raw SDI mapping)	Optional (ST 2110-22: JPEG XS)
Redundancy	ST 2022-7	ST 2022-7
SDP	One SDP per SDI signal	One SDP per essence flow
Ideal use	SDI migration, simple installations	New IP-native facility builds

Encapsulation

[Diagram: see online version]

Standards

Document	Title
SMPTE ST 2022-1	Forward Error Correction for MPEG-2 TS over IP
SMPTE ST 2022-2	Unidirectional Transport of Constant Bit Rate MPEG-2 TS over IP
SMPTE ST 2022-5	Forward Error Correction for High Bit Rate Media Transport over IP
SMPTE ST 2022-6	High Bit Rate Media Transport over IP (HBRMT)
SMPTE ST 2022-7	Seamless Protection Switching of RTP Datagrams
SMPTE ST 2059-2	SMPTE Profile for Use of IEEE-1588 PTP
Pro-MPEG CoP3	Code of Practice #3: FEC framework (basis for ST 2022-1)

See Also

- SMPTE ST 2110 -- successor suite for professional media over IP
- NDI -- alternative IP video protocol for production
- RTP -- underlying transport protocol for all ST 2022 media
- SDP -- session description for ST 2022 flows
- NTP -- time synchronization (PTP is a related precision protocol)

SMPTE ST 2110 -- Professional Media over Managed IP Networks

Standard: SMPTE ST 2110 | **Layer:** Application (Layer 7) | **Wireshark filter:** `rtp` (with SMPTE dissector plugins)

SMPTE ST 2110 is a suite of standards for transporting professional-grade uncompressed video, audio, and ancillary data over managed IP networks using RTP. It is the broadcast industry's replacement for SDI (Serial Digital Interface), enabling facilities to route any media essence independently over standard IP infrastructure. Unlike SDI's single multiplex, ST 2110 separates video, audio, and data into individual IP flows, each synchronized via PTP (Precision Time Protocol). ST 2110 is used in broadcast studios, OB trucks, and large-scale production facilities worldwide.

Suite of Standards

Standard	Title	Description
ST 2110-10	System Timing and Definitions	PTP-based system clock, epoch, media clock relationships
ST 2110-20	Uncompressed Active Video	Raw raster video in RTP (line-by-line or frame-by-frame)
ST 2110-21	Traffic Shaping and Delivery Timing	Sender pacing models: Narrow, Narrow Linear, Wide
ST 2110-22	Compressed Video	Constant bitrate compressed video (JPEG XS, etc.) in RTP
ST 2110-30	PCM Digital Audio	AES67-compatible uncompressed audio in RTP
ST 2110-31	AES3 Transparent Transport	AES3 audio (with non-PCM data) in RTP
ST 2110-40	Ancillary Data	Captions, timecode, AFD, SCTE-104 in RTP

Architecture

ST 2110 decomposes the traditional SDI multiplex into separate essence flows, each carried in its own RTP multicast stream:

[Diagram: see online version]

PTP Synchronization (ST 2110-10 / ST 2059)

All ST 2110 devices synchronize to a common PTP grandmaster clock. SMPTE ST 2059 defines the PTP profile for professional media:

Parameter	Value
PTP Profile	SMPTE ST 2059-2
Domain	127 (default for broadcast)
Epoch	SMPTE Epoch (1970-01-01 00:00:00 TAI)

Transport	IEEE 1588 over UDP (ports 319, 320) or L2 (EtherType 0x88F7)
Accuracy	Sub-microsecond (< 1 us across the network)
Grandmaster	Dedicated GPS-locked PTP server

Uncompressed Video (ST 2110-20)

RTP Payload Header

Each RTP packet carries a portion of the active video raster. The ST 2110-20 payload header identifies the line and offset:

[Diagram: see online version]

Key Fields

Field	Size	Description
Extended Seq Num	16 bits	High bits of extended RTP sequence number
SRD Length	16 bits	Sample Row Data length in bytes
F (Field ID)	1 bit	0 = field 1 / progressive, 1 = field 2
Line Number	11 bits	Video line number (1-based)
C (Continuation)	1 bit	1 = more SRD headers follow in this packet
SRD Offset	13 bits	Pixel offset from start of line

Common Video Formats and Bandwidth

Format	Resolution	Frame Rate	Bit Depth	Bandwidth
HD 1080i	1920x1080	59.94i	10-bit	~1.5 Gbps
HD 1080p	1920x1080	59.94p	10-bit	~3.0 Gbps
UHD 2160p	3840x2160	59.94p	10-bit	~12.0 Gbps
UHD 2160p	3840x2160	59.94p	12-bit HDR	~14.4 Gbps

Traffic Shaping (ST 2110-21)

ST 2110-21 defines how senders pace packets to prevent network congestion:

Sender Type	Pacing Model	Burst Behavior	Use Case
Narrow (N)	Gapped at line rate	Small, evenly spaced bursts	Hardware senders

Narrow Linear (NL)	Linear spread across frame	Minimal burstiness	Preferred for new designs
Wide (W)	Gapped with larger bursts	Larger bursts allowed	Software senders, gateways

Compressed Video (ST 2110-22)

ST 2110-22 carries constant-bitrate compressed video (typically JPEG XS) in RTP, enabling HD and UHD transport within 10GbE links:

Codec	Typical Ratio	Bandwidth (1080p60)	Latency
JPEG XS (ISO 21122)	5:1 to 10:1	~300-600 Mbps	< 1 ms (encode + decode)
TICO (intoPIX)	4:1	~750 Mbps	< 1 us

Audio (ST 2110-30 / AES67)

ST 2110-30 is fully compatible with AES67 for PCM audio transport:

Parameter	Value
Encoding	Linear PCM (L16, L24)
Sample rate	48 kHz (broadcast standard)
Channels per stream	1 to 64+ (typically 2, 8, or 16)
Packet time	1 ms (48 samples at 48 kHz) or 125 us
RTP payload type	Dynamic (negotiated via SDP)
Compatibility	AES67 interoperable

Ancillary Data (ST 2110-40)

ST 2110-40 carries ancillary data that was previously embedded in SDI VANC/HANC:

Data Type	SMPTE Standard	Description
Closed captions	CEA-708 / CEA-608	Subtitles and captions
Timecode	ST 12-1 (VITC/LTC)	Frame-accurate timecode
AFD	ST 2016-1	Active Format Description (aspect ratio)
SCTE-104	SCTE-104	Ad insertion triggers
Teletext	OP-47	European subtitles

Session Description (SDP)

Each ST 2110 flow is described by an SDP file specifying the connection address, media type, and RTP payload format:

```
v=0
o=- 1234567 1234567 IN IP4 10.0.1.1
s=Camera 1 - Video
t=0 0
m=video 5004 RTP/AVP 96
c=IN IP4 239.1.1.1/32
a=source-filter: incl IN IP4 239.1.1.1 10.0.1.1
a=rtpmap:96 raw/90000
a=fmtp:96 sampling=YCbCr-4:2:2; width=1920; height=1080;
  depth=10; interlace; exactframerate=30000/1001;
  colorimetry=BT709; PM=2110GPM; SSN=ST2110-20:2017;
  TP=2110TPN
a=mediaclk:direct=0
a=ts-refclk:ptp=IEEE1588-2008:AA-BB-CC-FF-FE-DD-EE-FF:127
```

Discovery and Connection Management (NMOS)

NMOS (Networked Media Open Specifications) from AMWA provides discovery and connection management for ST 2110 systems:

Specification	Title	Description
IS-04	Discovery & Registration	Register nodes, devices, senders, receivers via REST API
IS-05	Device Connection Management	Make and break connections between senders and receivers
IS-06	Network Control	Interface to SDN controllers for network resource management
IS-07	Event & Tally	Transport tally, GPIO, and event data
IS-08	Audio Channel Mapping	Map audio channels between senders and receivers

[Diagram: see online version]

Redundancy (ST 2022-7)

ST 2110 systems use SMPTE ST 2022-7 for hitless redundancy. Each flow is transmitted on two physically separate network paths, and receivers reconstruct a seamless output:

[Diagram: see online version]

Both paths carry identical RTP streams with matching sequence numbers. The receiver selects the first-arriving packet and discards duplicates, providing seamless protection against single-path failures.

ST 2110 vs SDI vs NDI

Feature	SMPTE ST 2110	SDI (3G/12G)	NDI
Transport	RTP over managed IP	Dedicated coax/fiber	TCP/UDP over standard IP

Compression	None (20) / JPEG XS (22)	None	SpeedHQ / H.264/H.265
Synchronization	PTP (IEEE 1588)	Genlock / black burst	NTP / internal
Infrastructure	Managed 10/25/100GbE + PTP	BNC coax / fiber	Standard GigE
Discovery	NMOS IS-04/IS-05	Physical cabling	mDNS / Discovery Server
Bandwidth (1080p60)	~3 Gbps (uncompressed)	3 Gbps	~125 Mbps
Redundancy	ST 2022-7 (hitless)	Dual cabling	Application-level
Multicast	Required	N/A	Optional
Essence separation	Yes (independent flows)	No (single multiplex)	No (single stream)
Standards body	SMPTE / AMWA	SMPTE	Vizrt (proprietary)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
SMPTE ST 2110-10	System Timing and Definitions
SMPTE ST 2110-20	Uncompressed Active Video
SMPTE ST 2110-21	Traffic Shaping and Delivery Timing
SMPTE ST 2110-22	Compressed Video
SMPTE ST 2110-30	PCM Digital Audio (AES67 compatible)
SMPTE ST 2110-31	AES3 Transparent Transport
SMPTE ST 2110-40	Ancillary Data
SMPTE ST 2059-2	PTP Profile for Professional Media
SMPTE ST 2022-7	Seamless Protection Switching
AES67	High-Performance Audio over IP
AMWA NMOS IS-04	Discovery & Registration
AMWA NMOS IS-05	Device Connection Management

See Also

- SMPTE ST 2022 -- predecessor video-over-IP suite (FEC, uncompressed SDI mapping)
- NDI -- alternative IP video protocol for production environments
- RTP -- underlying transport for all ST 2110 media flows
- SDP -- session description format used for ST 2110 flow parameters
- NTP -- time synchronization (PTP is a related precision protocol)

UPnP / DLNA -- Universal Plug and Play / Digital Living Network Alliance

Standard: UPnP Device Architecture 2.0 | **Layer:** Application (Layer 7) | **Wireshark filter:** `ssdp`

UPnP is a set of networking protocols that allows devices to discover each other and establish services on a local network without manual configuration. DLNA builds on UPnP AV to define interoperability guidelines for media sharing across consumer electronics -- TVs, speakers, phones, NAS devices, and media players. The protocol stack combines SSDP for discovery, HTTP for description and media transfer, SOAP for action invocation, and GENA for eventing. UPnP/DLNA remains the most widely deployed home media sharing protocol.

Protocol Stack

[Diagram: see online version]

SSDP (Simple Service Discovery Protocol)

SSDP uses HTTP-like messages over UDP multicast to discover and advertise UPnP devices on the network.

Parameter	Value
Multicast Address (IPv4)	239.255.255.250
Multicast Address (IPv6)	ff02::c / ff05::c
UDP Port	1900
Search Target Port	Ephemeral (unicast response)

M-SEARCH (Discovery Request)

```
M-SEARCH * HTTP/1.1
HOST: 239.255.255.250:1900
MAN: "ssdp:discover"
MX: 3
ST: urn:schemas-upnp-org:device:MediaServer:1
```

Field	Description
HOST	Multicast address and port
MAN	Required: `"ssdp:discover"`
MX	Maximum wait time in seconds before responding
ST	Search Target -- device type, service type, or `ssdp:all`

Common Search Targets (ST)

ST Value	Description
`ssdp:all`	All UPnP devices and services
`upnp:rootdevice`	All root devices
`urn:schemas-upnp-org:device:MediaServer:1`	UPnP Media Servers
`urn:schemas-upnp-org:device:MediaRenderer:1`	UPnP Media Renderers
`urn:schemas-upnp-org:service:ContentDirectory:1`	ContentDirectory service
`urn:schemas-upnp-org:service:AVTransport:1`	AVTransport service

NOTIFY (Advertisement)

Devices periodically announce their presence (alive) and departure (byebye):

```
NOTIFY * HTTP/1.1
HOST: 239.255.255.250:1900
CACHE-CONTROL: max-age=1800
LOCATION: http://192.168.1.10:49152/description.xml
NT: urn:schemas-upnp-org:device:MediaServer:1
NTS: ssdp:alive
SERVER: Linux/4.15 UPnP/2.0 MiniDLNA/1.3.0
USN: uuid:4d696e69-444c-164e-9d41-b827eba72d56::urn:schemas-upnp-org:device:MediaServer:1
```

Field	Description
CACHE-CONTROL	How long to cache this advertisement (seconds)
LOCATION	URL of the device description XML
NT	Notification Type -- device or service type
NTS	Notification Sub-type: `ssdp:alive` or `ssdp:byebye`
SERVER	OS, UPnP version, and product name
USN	Unique Service Name -- unique identity of this device/service

Device Discovery Flow

[Diagram: see online version]

UPnP AV Architecture

[Diagram: see online version]

Role	Description

MediaServer	Hosts and serves media content (NAS, PC, camera)
MediaRenderer	Plays/displays media content (TV, speaker, set-top box)
Control Point	Discovers servers and renderers, browses content, controls playback

DLNA Device Classes

Class	Abbreviation	Description
Digital Media Server	DMS	Stores and shares media content
Digital Media Renderer	DMR	Renders (plays) media content pushed to it
Digital Media Player	DMP	Finds and plays content from servers (combined browse + render)
Digital Media Controller	DMC	Finds content on servers and pushes to renderers (3-box model)
Digital Media Printer	DMPr	Prints images from servers

ContentDirectory Service

The ContentDirectory service allows browsing and searching media libraries.

Key Actions

Action	Description
Browse	Navigate the content hierarchy (containers and items)
Search	Query items by metadata (title, artist, class, etc.)
GetSearchCapabilities	List searchable metadata fields
GetSortCapabilities	List sortable metadata fields
GetSystemUpdateID	Check if content has changed since last browse

Browse Action Parameters

Parameter	Description
ObjectID	Container to browse (`0` = root)
BrowseFlag	`BrowseDirectChildren` or `BrowseMetadata`
Filter	Comma-separated list of metadata properties to return (`*` for all)
StartingIndex	Offset for paging (0-based)
RequestedCount	Number of results to return (0 = all)

SortCriteria	Sort order (e.g., `+dc:title`, `-dc:date`)

DIDL-Lite Result Format

Browse and Search return results as DIDL-Lite XML:

```
&lt;DIDL-Lite xmlns="urn:schemas-upnp-org:metadata-1-0/DIDL-Lite/"
            xmlns:dc="http://purl.org/dc/elements/1.1/"
            xmlns:upnp="urn:schemas-upnp-org:metadata-1-0/upnp/"&gt;
  &lt;item id="101" parentID="10" restricted="1"&gt;
    &lt;dc:title&gt;Song Title&lt;/dc:title&gt;
    &lt;upnp:class&gt;object.item.audioItem.musicTrack&lt;/upnp:class&gt;
    &lt;upnp:artist&gt;Artist Name&lt;/upnp:artist&gt;
    &lt;upnp:album&gt;Album Name&lt;/upnp:album&gt;
    &lt;res protocolInfo="http-get:*:audio/mpeg:DLNA.ORG_PN=MP3;DLNA.ORG_OP=01;DLNA.ORG_FLAGS=01700000000000000000
         size="5242880" duration="0:03:45" bitrate="320000"&gt;
      http://192.168.1.10:49152/media/101.mp3
    &lt;/res&gt;
  &lt;/item&gt;
&lt;/DIDL-Lite&gt;
```

UPnP Content Classes

Class	Description
`object.container`	Generic container (folder)
`object.container.album.musicAlbum`	Music album
`object.container.album.photoAlbum`	Photo album
`object.item.audioItem.musicTrack`	Music track
`object.item.videoItem.movie`	Movie
`object.item.imageItem.photo`	Photo

AVTransport Service

The AVTransport service controls media playback on a renderer.

Key Actions

Action	Description
SetAVTransportURI	Load a media URI for playback
Play	Start playback (Speed: "1" for normal)
Pause	Pause playback
Stop	Stop playback
Seek	Seek to a position (REL_TIME, ABS_TIME, TRACK_NR)
Next / Previous	Skip to next/previous track

GetTransportInfo	Get current transport state (PLAYING, PAUSED, STOPPED)
GetPositionInfo	Get current position, duration, and track URI
GetMediaInfo	Get information about current media

Media Playback Flow (3-Box Model)

[Diagram: see online version]

Other UPnP AV Services

ConnectionManager

Action	Description
GetProtocolInfo	List supported media formats (Source and Sink)
PrepareForConnection	Set up a connection for media transfer
ConnectionComplete	Tear down a connection
GetCurrentConnectionIDs	List active connections
GetCurrentConnectionInfo	Details of a specific connection

RenderingControl

Action	Description
GetVolume	Get current volume (0-100)
SetVolume	Set volume level
GetMute	Get mute state
SetMute	Set mute on/off
ListPresets	List available rendering presets
SelectPreset	Activate a preset (e.g., "FactoryDefaults")

DLNA Media Formats and Profile IDs

DLNA mandates baseline format support and uses Profile IDs (DLNA.ORG_PN) to identify specific codec/container combinations:

Profile ID	Format	Description
MP3	audio/mpeg	MPEG-1 Layer 3 audio
LPCM	audio/L16	Linear PCM audio

AAC_ISO_320	audio/mp4	AAC-LC up to 320 kbps
JPEG_SM	image/jpeg	Small JPEG (640x480)
JPEG_LRG	image/jpeg	Large JPEG (up to 4096x4096)
PNG_LRG	image/png	Large PNG
MPEG_PS_NTSC	video/mpeg	MPEG-2 Program Stream (NTSC)
MPEG_TS_HD_NA	video/vnd.dlna.mpeg-tts	MPEG-2 Transport Stream HD
AVC_MP4_MP_SD_AAC	video/mp4	H.264 Main Profile SD + AAC
AVC_MP4_HP_HD_AAC	video/mp4	H.264 High Profile HD + AAC
AVC_TS_MP_HD_AAC	video/vnd.dlna.mpeg-tts	H.264 MP HD in MPEG-TS

DLNA Protocol Info (4th Field)

The `protocolInfo` attribute in DIDL-Lite `<res>` elements follows a 4-field format:

```
&lt;protocol&gt;:&lt;network&gt;:&lt;content-type&gt;:&lt;additional-info&gt;
```

Example:
`http-get:*:video/mp4:DLNA.ORG_PN=AVC_MP4_MP_SD_AAC;DLNA.ORG_OP=01;DLNA.ORG_FLAGS=...`

DLNA.ORG_OP (Operations Parameter)

[Diagram: see online version]

Value	Description
00	No seeking supported
01	Byte-range seek supported
10	Time-based seek supported
11	Both seek modes supported

DLNA.ORG_FLAGS (32 hex characters)

[Diagram: see online version]

Bit	Flag	Description
0 (MSB)	Sender Paced	Server controls transfer rate
1	Limited Operations (time)	Time-based seek supported
2	Limited Operations (byte)	Byte-based seek supported
3	Playcontainer Support	Play container support
4	S0 Increasing	Start of content grows

5	SN Increasing	End of content grows (live)
20	DLNA V1.5 Flag	DLNA 1.5 device
21	HTTP Stalling	Server supports connection stalling
22	Background Transfer	Background transfer mode
23	Interactive Transfer	Interactive transfer mode
24	Streaming Transfer	Streaming transfer mode

DLNA HTTP Headers

Header	Description
`transferMode.dlna.org`	Transfer mode: `Streaming`, `Interactive`, or `Background`
`contentFeatures.dlna.org`	DLNA content features (profile, ops, flags)
`getcontentFeatures.dlna.org`	Request header: `1` to request content features in response
`TimeSeekRange.dlna.org`	Time-based byte-range request/response
`availableSeekRange.dlna.org`	Available seek range for live content
`realTimeInfo.dlna.org`	Real-time info for live streaming

Transfer Modes

Mode	Description
Streaming	Server-paced or client-paced streaming (video/audio playback)
Interactive	Standard HTTP request/response (browsing metadata, images)
Background	Background download (not time-sensitive)

GENA (General Event Notification Architecture)

GENA allows control points to subscribe to state changes from services:

[Diagram: see online version]

Encapsulation

[Diagram: see online version]

Standards

Document	Title

UPnP Device Architecture 2.0	Core UPnP specification (OCF)
UPnP AV Architecture	Audio/Video device and service specifications
UPnP ContentDirectory:4	Content browsing and searching
UPnP AVTransport:3	Media transport control
DLNA Guidelines	Device interoperability and media format guidelines
RFC 6970	UPnP IGD WANIPConnection (NAT traversal)

See Also

- HTTP -- transport for descriptions, control, and media transfer
- RTSP -- alternative streaming control protocol
- mDNS -- complementary local discovery (Bonjour/Avahi)
- ONVIF -- SOAP-based IP camera management with similar discovery

Messaging & Pub/Sub

Message queuing, publish/subscribe, and event-driven protocols.

8 protocols in this section

AMQP -- Advanced Message Queuing Protocol

Standard: AMQP 1.0 (OASIS) | **Layer:** Application (Layer 7) | **Wireshark filter:** `amqp`

AMQP is an open standard for enterprise message-oriented middleware. It provides reliable, brokered message delivery with rich routing semantics -- queues, exchanges, bindings, and acknowledgments. AMQP is used by RabbitMQ, Azure Service Bus, Apache Qpid, and other enterprise messaging systems. AMQP 0-9-1 (RabbitMQ's default) and AMQP 1.0 (OASIS standard) are the two major variants -- they are wire-incompatible despite sharing a name.

AMQP 1.0 Frame

[Diagram: see online version]

Field	Size	Description
Size	4 bytes	Total frame size including header
DOFF	1 byte	Data offset in 4-byte words (min 2 = 8 bytes header)
Type	1 byte	0 = AMQP, 1 = SASL
Channel	2 bytes	Multiplexed session channel
Performative	Variable	AMQP type describing the operation
Payload	Variable	Message data

AMQP 1.0 Performatives

Name	Description
open	Open a connection
begin	Begin a session on a channel
attach	Attach a link (sender or receiver) to a session
flow	Flow control (credit-based)
transfer	Transfer a message
disposition	Acknowledge/settle message delivery
detach	Detach a link
end	End a session
close	Close a connection

Connection Flow

[Diagram: see online version]

AMQP 0-9-1 (RabbitMQ)

The more widely deployed variant in practice (RabbitMQ default):

Routing Model

[Diagram: see online version]

Exchange Types

Type	Routing Logic
direct	Exact routing key match
fanout	Broadcast to all bound queues
topic	Wildcard pattern match (* = one word, # = zero or more)
headers	Match on message header attributes

Key Methods (0-9-1)

Method	Description
connection.open	Open a connection to the broker
channel.open	Open a multiplexed channel
exchange.declare	Create an exchange
queue.declare	Create a queue
queue.bind	Bind a queue to an exchange with a routing key
basic.publish	Publish a message to an exchange
basic.consume	Register a consumer on a queue
basic.deliver	Broker delivers a message to consumer
basic.ack	Consumer acknowledges message
basic.nack	Consumer negatively acknowledges (reject/requeue)

Message Properties

Property	Description
content-type	MIME type of the body
content-encoding	Encoding (e.g., gzip)
delivery-mode	1 = transient, 2 = persistent (survives broker restart)

priority	0-9 priority level
correlation-id	Request-response correlation
reply-to	Queue name for responses
expiration	Message TTL in milliseconds
message-id	Application message identifier
timestamp	Creation timestamp

AMQP vs MQTT vs Kafka

Feature	AMQP	MQTT	Kafka
Model	Queue/Exchange/Binding	Pub/Sub topics	Partitioned log
Broker	Required	Required	Required (cluster)
Routing	Rich (exchanges, bindings)	Topic hierarchy	Partitions + consumer groups
QoS	Ack/Nack/Reject per message	QoS 0/1/2	Offset-based ack
Persistence	Per-message (delivery-mode 2)	Retained messages	Always persisted
Ordering	Per-queue FIFO	Per-topic (QoS dependent)	Per-partition
Use case	Enterprise messaging, workflows	IoT, low-bandwidth	Event streaming, log aggregation

Encapsulation

[Diagram: see online version]

Standards

Document	Title
AMQP 1.0	AMQP Version 1.0 (OASIS Standard)
AMQP 0-9-1	AMQP 0-9-1 Specification (RabbitMQ)

See Also

- MQTT -- lightweight IoT pub-sub
- TCP
- TLS -- AMQPS encryption

IRC -- Internet Relay Chat

Standard: RFC 1459 / RFC 2812 | **Layer:** Application (Layer 7) | **Wireshark filter:** `irc`

IRC is a text-based real-time messaging protocol for group and private communication. Developed in 1988, it remains widely used for open-source project coordination (Libera.Chat, OFTC), community chat, and technical collaboration. IRC uses a simple line-oriented protocol over TCP --- each message is a single line of UTF-8 text terminated by CR-LF, with a maximum length of 512 bytes (including the trailing `\r\n`). Clients connect to servers, which can be linked together to form networks. IRC defined the model of channels, operators, and real-time chat that influenced every messaging platform that followed.

Message Format

Every IRC message follows this structure:

```
[:prefix] command [params] [:trailing]\r\n
```

[Diagram: see online version]

Field	Required	Description
Prefix	No	Source of the message `:nick!user@host` or `:servername` (server-added for relayed messages)
Command	Yes	IRC command (PRIVMSG, JOIN, etc.) or 3-digit numeric reply (001, 353, etc.)
Parameters	Varies	Space-separated parameters (max 15)
Trailing	No	Final parameter prefixed with `:` --- may contain spaces (used for message text)
CR LF	Yes	`\r\n` terminator (messages must not exceed 512 bytes total)

Example Messages

```
:alice!alice@user.example.com PRIVMSG #general :Hello everyone!
PING :server.example.com
:server.example.com 001 alice :Welcome to the IRC Network
JOIN #general
```

Client Registration

When a client connects, it must register with the server:

[Diagram: see online version]

Registration Commands

Command	Parameters	Description
PASS	password	Set connection password (before NICK/USER)
NICK	nickname	Set or change nickname
USER	username mode * :realname	Provide username and real name
CAP	LS / REQ / ACK / END	IRCv3 capability negotiation
QUIT	[:message]	Disconnect from server

Channel Commands

Command	Parameters	Description
JOIN	#channel [key]	Join a channel (creates it if it does not exist)
PART	#channel [:message]	Leave a channel
KICK	#channel nick [:reason]	Remove a user from a channel (requires operator)
INVITE	nick #channel	Invite a user to a channel
TOPIC	#channel [:new topic]	View or set the channel topic
MODE	#channel +/-modes [params]	Set channel or user modes
NAMES	#channel	List users in a channel
LIST	[#channel]	List channels and topics on the server

Messaging Commands

Command	Parameters	Description
PRIVMSG	target :message	Send a message to a user or channel
NOTICE	target :message	Send a notice (must not trigger auto-reply)

Channel Messaging Flow

[Diagram: see online version]

Server Commands

Command	Parameters	Description

PING	:token	Server keepalive request (client must respond with PONG)
PONG	:token	Response to PING
WHO	mask	Query user information
WHOIS	nick	Detailed user information
MOTD	[server]	Request Message of the Day
LUSERS		Server/network statistics
VERSION	[server]	Server version information

Numeric Replies

Code	Name	Description
001	RPL_WELCOME	Welcome message after successful registration
002	RPL_YOURHOST	Server host information
003	RPL_CREATED	Server creation date
004	RPL_MYINFO	Server name, version, available modes
005	RPL_ISUPPORT	Server-supported features (NETWORK, CHANTYPES, PREFIX, etc.)
332	RPL_TOPIC	Channel topic
353	RPL_NAMREPLY	List of nicknames in a channel
366	RPL_ENDOFNAMES	End of NAMES list
372	RPL_MOTD	Message of the Day line
375	RPL_MOTDSTART	Start of MOTD
376	RPL_ENDOFMOTD	End of MOTD
401	ERR_NOSUCHNICK	No such nick/channel
403	ERR_NOSUCHCHANNEL	No such channel
433	ERR_NICKNAMEINUSE	Nickname is already in use
461	ERR_NEEDMOREPARAMS	Not enough parameters
473	ERR_INVITEONLYCHAN	Cannot join invite-only channel
474	ERR_BANNEDFROMCHAN	Cannot join --- banned
475	ERR_BADCHANNELKEY	Cannot join --- wrong key

Channel Modes

Mode	Name	Description

+o nick	Operator	Grants channel operator status (prefix @)
+v nick	Voice	Grants voice in moderated channels (prefix +)
+b mask	Ban	Bans a hostmask from joining or speaking
+i	Invite Only	Only invited users can join
+k key	Key	Requires a password to join
+l count	Limit	Maximum number of users in the channel
+m	Moderated	Only operators and voiced users can speak
+n	No External	Only channel members can send messages to the channel
+s	Secret	Channel hidden from LIST and WHOIS
+t	Topic Lock	Only operators can change the topic

User Modes

Mode	Name	Description
+i	Invisible	User hidden from WHO/NAMES unless in a shared channel
+o	Operator	IRC operator (server admin)
+w	Wallops	Receives WALLOPS (network-wide operator messages)
+r	Registered	Identified with NickServ (varies by implementation)

IRCv3 Extensions

Modern IRC implementations support IRCv3 extensions negotiated via CAP:

Extension	Description
message-tags	Arbitrary key-value metadata on messages
sasl	SASL authentication (PLAIN, EXTERNAL, SCRAM)
server-time	Server-provided timestamp on each message
batch	Group related messages (e.g., history replay)
labeled-response	Correlate commands with their replies
echo-message	Server echoes your messages back (for multi-client sync)
away-notify	Real-time notification when contacts go away/return

account-notify	Notification when users authenticate to services
cap-notify	Dynamic capability add/remove notification
chathistory	Request channel history from the server
multi-prefix	Show all channel prefixes (@+) in NAMES

CTCP (Client-To-Client Protocol)

CTCP messages are embedded within PRIVMSG using `\x01` (SOH) delimiters:

```
PRIVMSG alice :\x01VERSION\x01
PRIVMSG #general :\x01ACTION waves hello\x01
```

Command	Description
ACTION	Emote (/me does something)
VERSION	Request client name/version
PING	Round-trip latency measurement
TIME	Request client's local time
CLIENTINFO	List supported CTCP commands

DCC (Direct Client-to-Client)

DCC establishes direct TCP connections between clients (bypassing the server):

Type	Description
DCC SEND	File transfer (sender opens a listening port)
DCC CHAT	Private chat over a direct TCP connection
DCC RESUME	Resume an interrupted file transfer

DCC is initiated via CTCP: `PRIVMSG bob :\x01DCC SEND filename ip port filesize\x01`

Server Linking

IRC servers can link together to form a network. Each server maintains connections to other servers in a spanning tree topology --- there is exactly one path between any two servers.

Concept	Description
Spanning Tree	Server topology forms a tree (no loops)
Netsplit	A server link breaks, dividing the network; users on each side cannot see users on the other
Netjoin	Link re-established, network merges (burst of JOIN messages)

Services	Pseudoservers providing NickServ (nick registration), ChanServ (channel management), etc.

IRC vs Matrix vs XMPP

Feature	IRC	Matrix	XMPP
Protocol	Line-based text over TCP	JSON over HTTPS	XML streams over TCP
Federation	Server linking (spanning tree)	Homeserver federation (DAG replication)	Server-to-server (S2S)
Persistence	None (ephemeral by default; bouncer or IRCv3 chathistory needed)	Full history (server-stored DAG)	MAM (XEP-0313)
E2EE	None built-in (OTR/OMEMO via clients)	Olm/Megolm	OMEMO (XEP-0384)
File sharing	DCC (direct) or external pastes	Native (mxc:// URIs)	HTTP Upload (XEP-0363)
Identity	Nickname (ephemeral; NickServ for registration)	@user:server (permanent)	user@server (permanent JID)
Message size	512 bytes (IRCv3: 4096+ with tags)	Unlimited (JSON)	Unlimited (XML)
Extensibility	IRCv3 capabilities, CTCP	Event types, MSCs	XEPs (XML extensions)
Clients	HexChat, irssi, WeeChat, Halloy	Element, FluffyChat, Beeper	Conversations, Gajim, Dino
Typical use	Open-source communities, tech chat	Government, enterprise, bridged platforms	Enterprise IM, IoT

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 1459	Internet Relay Chat Protocol (original, 1993)
RFC 2810	IRC: Architecture
RFC 2811	IRC: Channel Management
RFC 2812	IRC: Client Protocol (updated)
RFC 2813	IRC: Server Protocol
IRCv3	IRCv3 Working Group Specifications

See Also

- XMPP --- XML-based federated messaging (more structured, heavier)
- Matrix --- modern federated messaging with history and E2EE
- Telnet --- similar line-based text protocol
- TLS --- encrypts IRC connections on port 6697

Kafka Protocol

Standard: Apache Kafka Protocol Guide | **Layer:** Application (Layer 7) | **Wireshark filter:** `kafka`

The Kafka protocol is a binary, request-response protocol for interacting with Apache Kafka brokers. Kafka is a distributed event streaming platform used for log aggregation, event sourcing, stream processing, and as a durable message bus between microservices. Clients produce records to topics (partitioned, replicated logs) and consumers read them by tracking offsets. The protocol defines how producers, consumers, and admin tools communicate with the broker cluster.

Request/Response Frame

Every Kafka message uses a length-prefixed binary format:

[Diagram: see online version]

Response

[Diagram: see online version]

Key Fields

Field	Size	Description
Size	4 bytes	Length of the remaining message
API Key	16 bits	Identifies the request type
API Version	16 bits	Version of the API being used
Correlation ID	32 bits	Client-assigned ID matching responses to requests
Client ID	Variable	Nullable string identifying the client

API Keys (Request Types)

Key	Name	Description
0	Produce	Write records to a topic partition
1	Fetch	Read records from a topic partition
2	ListOffsets	Get available offsets for a partition
3	Metadata	Get cluster and topic metadata
8	OffsetCommit	Commit consumer offsets
9	OffsetFetch	Fetch committed offsets
10	FindCoordinator	Find the group coordinator broker
11	JoinGroup	Join a consumer group
14	SyncGroup	Synchronize group assignments
18	ApiVersions	Query supported API versions
19	CreateTopics	Create new topics
20	DeleteTopics	Delete topics
22	InitProducerID	Initialize idempotent/transactional producer
32	DescribeConfigs	Query broker/topic configuration
36	SaslAuthenticate	SASL authentication exchange
37	CreatePartitions	Add partitions to topics

Core Concepts

Concept	Description
Topic	Named stream of records (like a database table)
Partition	Ordered, immutable log within a topic
Offset	Sequential position within a partition
Record	Key + value + timestamp + headers
Producer	Writes records to topic partitions
Consumer	Reads records by tracking offsets
Consumer Group	Set of consumers that share partitions (load balancing)
Broker	Kafka server that stores partitions and serves clients
Replication	Each partition has N replicas across brokers
Leader	One replica handles all reads/writes; others are followers

Produce/Consume Flow

[Diagram: see online version]

Consumer Group Rebalancing

[Diagram: see online version]

Record Batch Format (v2)

[Diagram: see online version]

Record Attributes

Bit	Description
0-2	Compression: 0=none, 1=gzip, 2=snappy, 3=lz4, 4=zstd
3	Timestamp type: 0=create, 1=log-append
4	Is transactional
5	Is control batch

Encapsulation

[Diagram: see online version]

Standards

Document	Title
Kafka Protocol Guide	Apache Kafka Protocol Specification
KIP Index	Kafka Improvement Proposals (protocol evolution)

See Also

- AMQP -- enterprise message queuing (different model)
- MQTT -- lightweight IoT messaging
- NATS -- lightweight cloud-native messaging
- TCP

Matrix

Standard: Matrix Specification | **Layer:** Application (Layer 7) | **Wireshark filter:** `http || http2`

Matrix is an open federated protocol for decentralized, real-time communication. It provides interoperable instant messaging, VoIP, and IoT communication with no single point of control. Each user has a homeserver (e.g., matrix.org), and homeservers federate with each other over HTTPS so users on different servers can communicate seamlessly --- like email, but for real-time messaging. Matrix supports end-to-end encryption (Olm/Megolm), bridges to other platforms (Slack, Discord, IRC, WhatsApp, Signal, Telegram), and is used by governments (France, Germany), enterprises, and open-source communities. Major clients include Element, FluffyChat, and Beeper.

Architecture

[Diagram: see online version]

Identifiers

Identifier	Format	Example	Description
User ID	@localpart:domain	@alice:matrix.org	Unique user identifier
Room ID	!opaque_id:domain	!abc123:matrix.org	Unique room identifier
Room Alias	#name:domain	#general:matrix.org	Human-readable room name
Event ID	$opaque_id	$event123	Unique event identifier
Device ID	String	ABCDEF	User's device for E2EE

Room Model

Rooms are the fundamental data structure. Each room is a replicated DAG (Directed Acyclic Graph) of events synchronized across all participating homeservers.

Concept	Description
Room	A shared data structure (chat room, DM, or space) replicated across homeservers
Event	An immutable JSON object in the room DAG (message, state change, membership)
State Event	An event that persists as room configuration (name, topic, power levels, membership)
Timeline Event	An event in the message timeline (messages, reactions, redactions)
DAG	Directed Acyclic Graph of events --- enables eventually consistent replication
State Resolution	Algorithm to deterministically resolve conflicting state across federated servers

Event Types

State Events

Type	Description
m.room.create	Room creation event (first event in any room)
m.room.member	Membership change (join, leave, invite, ban, knock)
m.room.power_levels	Permission levels for actions within the room
m.room.name	Room display name
m.room.topic	Room topic / description
m.room.encryption	Enables E2EE for the room (Megolm settings)
m.room.encrypted	An encrypted event payload
m.room.avatar	Room avatar image
m.room.canonical_alias	Primary room alias
m.room.join_rules	Who can join (public, invite, knock, restricted)
m.room.history_visibility	Who can see history (joined, invited, shared, world_readable)
m.space.child	Links a room as a child of a Space
m.space.parent	Links a room to its parent Space

Message Types (inside m.room.message)

msgtype	Description
m.text	Plain text or formatted (HTML) message
m.image	Image attachment
m.video	Video attachment
m.audio	Audio attachment
m.file	Generic file attachment
m.notice	Bot/automated notification (clients may render differently)
m.emote	Emote (/me action)
m.location	Geographic location

Client-Server API

The Client-Server API is RESTful JSON over HTTPS:

Endpoint	Method	Description
/login	POST	Authenticate and obtain access token
/register	POST	Create a new account

/sync	GET	Long-poll for new events (main sync loop)
/rooms/{roomId}/send/{eventType}/{txnId}	PUT	Send a message event
/rooms/{roomId}/state/{eventType}/{stateKey}	PUT	Set room state
/join/{roomIdOrAlias}	POST	Join a room
/rooms/{roomId}/leave	POST	Leave a room
/rooms/{roomId}/invite	POST	Invite a user
/rooms/{roomId}/messages	GET	Paginate room history
/keys/upload	POST	Upload E2EE device keys
/keys/query	POST	Query other users' device keys
/keys/claim	POST	Claim one-time keys for Olm sessions
/createRoom	POST	Create a new room
/publicRooms	GET	List public rooms on the server

Federation (Server-Server) Flow

When Alice on Server A sends a message to a room with Bob on Server B:

[Diagram: see online version]

Federation uses HTTPS with server-to-server signing keys. Each event is signed by its origin server, and servers validate signatures before accepting events. Server discovery uses `.well-known/matrix/server` or DNS SRV records.

End-to-End Encryption

Matrix uses two cryptographic ratchets for E2EE:

Protocol	Use Case	Based On	Description
Olm	1:1 device sessions	Double Ratchet (Signal)	Establishes encrypted session between two devices
Megolm	Group sessions	Forward ratchet only	Efficient group encryption --- one outbound session per device per room

E2EE Key Exchange

[Diagram: see online version]

Key Verification

Method	Description
SAS (Emoji)	Both devices display 7 emoji --- users compare out of band
QR Code	One device shows QR, other scans --- cross-signing verification
Cross-Signing	Master key signs device keys; verified users auto-trust new devices

Spaces, Threads, and VoIP

Feature	Description
Spaces	Hierarchical grouping of rooms (like Discord servers or Slack workspaces)
Threads	In-room threaded replies (m.thread relation)
VoIP	1:1 and group calls via WebRTC (m.call.invite, m.call.answer, m.call.candidates)
Sliding Sync	MSC3575 --- efficient sync for mobile (request only visible rooms/events)

Bridges

Matrix bridges connect to external platforms, translating messages bidirectionally:

Bridge	Platform	Protocol
mautrix-whatsapp	WhatsApp	WhatsApp Web API
mautrix-signal	Signal	Signal Protocol (libsignal)
mautrix-telegram	Telegram	Telegram Bot/User API
mautrix-slack	Slack	Slack API
mautrix-discord	Discord	Discord API
matrix-appservice-irc	IRC	IRC protocol

Matrix vs XMPP vs Signal

Feature	Matrix	XMPP	Signal
Federation	Yes (any server)	Yes (any server)	No (centralized)
Data model	Room DAG (replicated)	XML streams	Centralized store
E2EE	Olm/Megolm (opt-in per room)	OMEMO (XEP-0384)	Always on (Double Ratchet)
Group encryption	Megolm (efficient)	OMEMO (per-device)	Sender Keys

History	Full history on join (configurable)	MAM (XEP-0313)	No server-side history
Message format	JSON	XML	Protobuf
Bridges	Extensive (Slack, Discord, IRC, WhatsApp)	Limited (Biboumi for IRC)	None
Spaces/hierarchy	Spaces (room grouping)	MUC only	Groups (basic)
VoIP	WebRTC (1:1 and group)	Jingle (XEP-0166)	Built-in calling
Specification	spec.matrix.org	RFCs + XEPs	signal.org/docs

Encapsulation

[Diagram: see online version]

Standards

Document	Title
Matrix Spec	Matrix Specification (Client-Server, Server-Server, Application Service, Identity Service)
MSC Process	Matrix Spec Changes (proposals for new features)
MSC3575	Sliding Sync

See Also

- XMPP --- federated messaging alternative (XML-based)
- HTTP --- underlying transport for Matrix APIs
- WebSocket --- streaming transport for clients

MQTT -- Message Queuing Telemetry Transport

Standard: MQTT v5.0 (OASIS) | **Layer:** Application (Layer 7) | **Wireshark filter:** `mqtt`

MQTT is a lightweight publish-subscribe messaging protocol designed for constrained devices and unreliable networks. A client publishes messages to topics on a broker, and other clients subscribe to those topics to receive messages. MQTT is the dominant IoT messaging protocol -- used by AWS IoT, Azure IoT Hub, Home Assistant, and millions of embedded devices for telemetry, control, and event notification. Its minimal overhead (a 2-byte minimum header) makes it ideal for low-bandwidth, high-latency, or battery-powered environments.

Fixed Header

Every MQTT packet starts with a fixed header:

[Diagram: see online version]

Key Fields

Field	Size	Description
Packet Type	4 bits	Identifies the control packet type
DUP	1 bit	Duplicate delivery (QoS 1/2 retransmission)
QoS	2 bits	Quality of Service level (0, 1, or 2)
Retain	1 bit	Broker stores this as the last known value for the topic
Remaining Length	1-4 bytes	Variable-length encoding of the remaining packet size

Packet Types

Type	Value	Direction	Description
CONNECT	1	Client -> Broker	Initiate a session
CONNACK	2	Broker -> Client	Connection acknowledgment
PUBLISH	3	Either	Publish a message to a topic
PUBACK	4	Either	QoS 1 acknowledgment
PUBREC	5	Either	QoS 2 step 1 -- received
PUBREL	6	Either	QoS 2 step 2 -- release
PUBCOMP	7	Either	QoS 2 step 3 -- complete
SUBSCRIBE	8	Client -> Broker	Subscribe to topics
SUBACK	9	Broker -> Client	Subscription acknowledgment
UNSUBSCRIBE	10	Client -> Broker	Unsubscribe from topics
UNSUBACK	11	Broker -> Client	Unsubscription acknowledgment
PINGREQ	12	Client -> Broker	Keepalive ping
PINGRESP	13	Broker -> Client	Keepalive response
DISCONNECT	14	Either	Graceful disconnect
AUTH	15	Either	Extended authentication (v5.0)

QoS Levels

Level	Name	Guarantee	Flow
0	At most once	Fire and forget -- no acknowledgment	PUBLISH ->
1	At least once	Guaranteed delivery, may duplicate	PUBLISH -> PUBACK
2	Exactly once	Guaranteed single delivery	PUBLISH -> PUBREC -> PUBREL -> PUBCOMP

QoS 2 Flow

[Diagram: see online version]

Topics

Topics are UTF-8 strings with hierarchical levels separated by `/`:

```
home/livingroom/temperature
home/kitchen/humidity
factory/line1/motor3/rpm
```

Wildcards (subscribe only)

Wildcard	Meaning	Example
`+`	Single level	`home/+/temperature` matches `home/kitchen/temperature`
`#`	Multi-level (must be last)	`home/#` matches `home/kitchen/humidity` and all below

Special Topics

Topic	Description
`$SYS/...`	Broker statistics and metadata (convention)

CONNECT Packet

Field	Description
Protocol Name	"MQTT"
Protocol Level	5 (MQTT v5.0) or 4 (v3.1.1)

Connect Flags	Clean Start, Will, Will QoS, Will Retain, Password, Username
Keep Alive	Seconds between keepalive PINGREQs (0 = disabled)
Client ID	Unique client identifier
Will Topic	Topic for Last Will and Testament message
Will Payload	Payload published if client disconnects unexpectedly
Username	Optional authentication username
Password	Optional authentication password/token

Last Will and Testament (LWT)

If a client disconnects abnormally (network failure, no DISCONNECT), the broker publishes the Will message on behalf of the client -- commonly used for "device offline" notifications.

MQTT v5.0 Properties

v5.0 added properties to most packet types:

Property	Description
Message Expiry Interval	TTL for retained/queued messages
Content Type	MIME type of the payload
Response Topic	Topic for request-response patterns
Correlation Data	Request-response correlation
Session Expiry Interval	How long broker keeps session after disconnect
Topic Alias	Short numeric alias for frequently used topics
Receive Maximum	Flow control -- max in-flight QoS 1/2 messages
Shared Subscriptions	Load-balanced delivery across a consumer group

Encapsulation

[Diagram: see online version]

Standards

Document	Title
MQTT v5.0	MQTT Version 5.0 (OASIS Standard)
MQTT v3.1.1	MQTT Version 3.1.1 (widely deployed)
MQTT-SN	MQTT for Sensor Networks (UDP-based variant)

See Also

- TCP
- TLS -- encrypts MQTT connections
- WebSocket -- browser-based MQTT transport
- HTTP -- alternative IoT transport (REST APIs)

NATS

Standard: NATS Protocol (nats.io) | **Layer:** Application (Layer 7) | **Wireshark filter:** `tcp.port == 4222` (no native dissector)

NATS is a lightweight, high-performance messaging system designed for cloud-native, microservice, and IoT architectures. It uses a simple text-based protocol over TCP with publish-subscribe, request-reply, and queue group (load-balanced) messaging patterns. NATS prioritizes simplicity and speed -- the core protocol has only a handful of commands. NATS JetStream adds persistence, at-least-once delivery, and stream processing. NATS is used by Kubernetes (for some distributions), Synadia, and as a service mesh data plane.

Protocol

NATS uses a text-based, line-oriented protocol (CRLF-terminated):

Commands (Client -> Server)

Command	Format	Description
CONNECT	`CONNECT {json}\r\n`	Authentication and options
PUB	`PUB subject [reply] size\r\npayload\r\n`	Publish a message
HPUB	`HPUB subject [reply] hdr_size total_size\r\nheaders\r\npayload\r\n`	Publish with headers
SUB	`SUB subject [queue] sid\r\n`	Subscribe to a subject
UNSUB	`UNSUB sid [max]\r\n`	Unsubscribe
PING	`PING\r\n`	Keepalive
PONG	`PONG\r\n`	Keepalive response

Messages (Server -> Client)

Message	Format	Description
INFO	`INFO {json}\r\n`	Server info (sent on connect)

MSG	`MSG subject sid [reply] size\r\npayload\r\n`	Deliver a message
HMSG	`HMSG subject sid [reply] hdr_size total_size\r\nheaders\r\npayload\r\n`	Deliver with headers
+OK	`+OK\r\n`	Acknowledged (if verbose mode)
-ERR	`-ERR 'message'\r\n`	Error
PING	`PING\r\n`	Server keepalive
PONG	`PONG\r\n`	Client must respond

Session Example

```
S: INFO {"server_id":"abc","version":"2.10.0","max_payload":1048576}\r\n
C: CONNECT {"verbose":false,"name":"myapp"}\r\n
C: SUB weather.&gt; 1\r\n
C: PUB weather.london 11\r\n{"temp":18}\r\n
S: MSG weather.london 1 11\r\n{"temp":18}\r\n
C: PING\r\n
S: PONG\r\n
```

Subject-Based Addressing

Subjects are dot-separated hierarchies with wildcards:

Pattern	Meaning	Example
`foo.bar`	Exact match	Only `foo.bar`
`foo.*`	Single token wildcard	`foo.bar`, `foo.baz` but not `foo.bar.baz`
`foo.>`	Multi-token wildcard (must be last)	`foo.bar`, `foo.bar.baz`, everything under `foo`

Messaging Patterns

Pattern	Description
Pub/Sub	Publisher sends to subject, all subscribers receive
Queue Groups	Subscribers with same queue name load-balance (one receives)
Request/Reply	Publisher sends with reply subject; one subscriber responds

Request/Reply

[Diagram: see online version]

JetStream (Persistence)

JetStream adds persistent streams and consumers on top of core NATS:

Concept	Description
Stream	Persistent, append-only log of messages for a set of subjects
Consumer	A stateful view into a stream (tracks position)
Push Consumer	Server pushes messages to a subscription
Pull Consumer	Client explicitly requests batches
Exactly-once	Deduplication via message ID

Encapsulation

[Diagram: see online version]

Standards

Document	Title
NATS Protocol	NATS Client Protocol specification
NATS JetStream	JetStream persistence documentation

See Also

- MQTT -- IoT-focused alternative
- AMQP -- enterprise messaging with rich routing
- TCP

XMPP -- Extensible Messaging and Presence Protocol

Standard: RFC 6120 | **Layer:** Application (Layer 7) | **Wireshark filter:** `xmpp`

XMPP is an open, decentralized protocol for real-time messaging, presence, and general XML routing. Originally developed as Jabber in 1999, it was standardized by the IETF and is used for instant messaging, multi-user chat, voice/video signaling (Jingle), IoT, and as the foundation of Google Talk (historically), WhatsApp's internal signaling, and ejabberd/Prosody-based deployments. XMPP uses persistent XML streams over TCP and a federated server architecture similar to email -- any XMPP server can communicate with any other.

XML Streams

An XMPP session consists of two XML streams -- one in each direction -- wrapped in `<stream:stream>` root elements:

```
&lt;!-- Client to Server --&gt;
&lt;stream:stream to="example.com" xmlns="jabber:client"
  xmlns:stream="http://etherx.jabber.org/streams" version="1.0"&gt;

&lt;!-- Server to Client --&gt;
&lt;stream:stream from="example.com" id="abc123"
  xmlns="jabber:client" version="1.0"&gt;
```

All stanzas are XML elements within these streams. The streams remain open for the session's lifetime.

Stanza Types

XMPP defines three core stanza types:

Stanza	Purpose	Description
`<message>`	Communication	Deliver a message to a user or room
`<presence>`	Status	Broadcast availability (online, away, DND)
`<iq>`	Info/Query	Request-response operations (get/set/result/error)

Common Attributes

Attribute	Description
from	Sender's JID (set by server)
to	Recipient's JID
id	Stanza identifier (for matching responses)
type	Stanza subtype (see per-stanza tables)
xml:lang	Language of the stanza content

Addressing (JID -- Jabber ID)

```
user@domain/resource
```

Part	Description	Example
user (localpart)	Account name	`alice`
domain	Server hostname	`example.com`
resource	Specific client/session	`phone`, `laptop`
Bare JID	user@domain	`alice@example.com`

Full JID	user@domain/resource	`alice@example.com/phone`

Message Stanza

```
&lt;message from="alice@example.com/phone"
         to="bob@example.com"
         type="chat"&gt;
  &lt;body&gt;Hello Bob!&lt;/body&gt;
&lt;/message&gt;
```

Message Types

Type	Description
chat	One-to-one conversation
groupchat	Multi-user chat (MUC) room message
headline	Alert/notification (no reply expected)
normal	Single message (standalone, like email)
error	Error response

Presence Stanza

```
&lt;presence from="alice@example.com/phone"&gt;
  &lt;show&gt;away&lt;/show&gt;
  &lt;status&gt;In a meeting&lt;/status&gt;
  &lt;priority&gt;5&lt;/priority&gt;
&lt;/presence&gt;
```

Show Values

Value	Meaning
(absent)	Available (online)
away	Away
chat	Actively looking to chat
dnd	Do Not Disturb
xa	Extended Away

Presence Types

Type	Description
(absent)	Available

unavailable	Going offline
subscribe	Request to subscribe to someone's presence
subscribed	Approve a subscription request
unsubscribe	Cancel a subscription
unsubscribed	Deny/revoke a subscription
probe	Server-to-server presence probe
error	Presence error

IQ (Info/Query) Stanza

```
&lt;iq from="alice@example.com/phone"
    to="example.com"
    id="roster1"
    type="get"&gt;
  &lt;query xmlns="jabber:iq:roster"/&gt;
&lt;/iq&gt;
```

IQ Types

Type	Description
get	Request information
set	Provide or update information
result	Successful response
error	Error response

Connection Flow

[Diagram: see online version]

Federation

XMPP servers federate like email -- `alice@server-a.com` can message `bob@server-b.com`:

[Diagram: see online version]

Server-to-server connections use DNS SRV records (`_xmpp-server._tcp.example.com`) for discovery.

Key Extensions (XEPs)

XEP	Name	Description
XEP-0045	Multi-User Chat (MUC)	Group chat rooms

XEP-0060	Publish-Subscribe (PubSub)	Event distribution
XEP-0166	Jingle	Voice/video call signaling (like SIP over XMPP)
XEP-0280	Message Carbons	Sync messages across multiple clients
XEP-0313	Message Archive Management (MAM)	Server-side message history
XEP-0384	OMEMO	End-to-end encryption (Signal Protocol)
XEP-0363	HTTP File Upload	Share files via HTTP URL
XEP-0198	Stream Management	Reliable delivery, session resumption

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 6120	XMPP: Core
RFC 6121	XMPP: Instant Messaging and Presence
RFC 7590	Use of TLS in XMPP
RFC 7395	XMPP Subprotocol for WebSocket
XEP Index	XMPP Extension Protocols (XEPs)

See Also

- TCP
- TLS -- encrypts XMPP connections
- SIP -- alternative signaling for voice/video (Jingle is XMPP's equivalent)
- WebSocket -- browser-based XMPP transport
- DNS -- SRV records for server discovery

ZeroMQ -- ØMQ / ZMTP

Standard: ZMTP 3.1 (rfc.zeromq.org) | **Layer:** Application / Transport (Layer 7) | **Wireshark filter:** `zmtp`

ZeroMQ (also written ØMQ or 0MQ) is a high-performance asynchronous messaging library that provides socket-like abstractions for building distributed systems. Unlike broker-based systems (MQTT, AMQP, NATS), ZeroMQ is **brokerless** -- messages are sent directly between application endpoints with no intermediary. The wire protocol is

ZMTP (ZeroMQ Message Transport Protocol), which defines framing, greeting, handshake, and multiple socket patterns over TCP, IPC, in-process, UDP multicast, and WebSocket.

ZMTP Greeting

When two ZeroMQ sockets connect, they exchange a greeting to negotiate version and security:

[Diagram: see online version]

Field	Size	Description
Signature	1 + 8 + 1 bytes	`0xFF` + 8 padding bytes + `0x7F` (identifies ZMTP)
Version	2 bytes	Major.minor (3.1)
Mechanism	20 bytes	Security mechanism: `NULL`, `PLAIN`, `CURVE`, `GSSAPI`
as-server	1 byte	0 = client role, 1 = server role

Frame Format

ZMTP messages are composed of one or more frames:

Short Frame (0-255 bytes)

[Diagram: see online version]

Long Frame (256+ bytes)

[Diagram: see online version]

Flags

Bit	Name	Description
0	MORE	1 = more frames follow (multi-part message)
1	LONG	1 = size is 8 bytes (uint64); 0 = size is 1 byte
2	COMMAND	1 = this is a command frame (not data)

Socket Patterns

ZeroMQ's core innovation is its socket patterns -- each pattern defines a messaging topology:

REQ-REP (Request-Reply)

[Diagram: see online version]

Strict alternation: send -> recv -> send -> recv. Lock-step request-response.

PUB-SUB (Publish-Subscribe)

[Diagram: see online version]

Publisher sends to all matching subscribers. Subscribers set topic prefix filters. No broker -- publisher fans out directly.

PUSH-PULL (Pipeline)

[Diagram: see online version]

Round-robin distribution to workers. Load-balanced task distribution.

ROUTER-DEALER (Advanced Async)

[Diagram: see online version]

ROUTER tracks sender identity (prepends identity frame). DEALER round-robins. Together they build brokers, proxies, and load balancers.

All Socket Types

Socket	Pattern	Send	Receive	Description
REQ	Request-Reply	Yes (then must recv)	Yes (after send)	Client request
REP	Request-Reply	Yes (after recv)	Yes (then must send)	Server reply
PUB	Pub-Sub	Yes	No	Publish to all subscribers
SUB	Pub-Sub	No	Yes (filtered)	Subscribe to topics
PUSH	Pipeline	Yes	No	Push to a worker
PULL	Pipeline	No	Yes	Pull from a producer
DEALER	Async	Yes (round-robin)	Yes (fair-queue)	Async request/reply
ROUTER	Async	Yes (routed by identity)	Yes (with identity)	Routing/brokering
PAIR	Exclusive	Yes	No	1-to-1 bidirectional (inter-thread)
XPUB	Extended Pub	Yes	Yes (subscriptions)	PUB with subscription visibility
XSUB	Extended Sub	Yes (subscriptions)	Yes	SUB that forwards subscriptions

Multi-Part Messages

ZeroMQ messages can consist of multiple frames (MORE flag):

```
Frame 1: "topic" (MORE=1)
Frame 2: "key:value" (MORE=1)
Frame 3: &lt;binary payload&gt; (MORE=0, final)
```

The receiver gets all frames atomically -- either the entire multi-part message arrives or nothing. This is how PUB-SUB topic filtering works: the first frame is the topic prefix.

Security Mechanisms

Mechanism	Description
NULL	No authentication or encryption
PLAIN	Username + password (plaintext -- use with TLS only)
CURVE	CurveZMQ -- elliptic curve encryption and authentication (NaCl/libsodium)
GSSAPI	Kerberos-based authentication

CurveZMQ

CurveZMQ provides encryption + authentication using Curve25519:

Feature	Description
Key exchange	Curve25519 ECDH
Encryption	XSalsa20 + Poly1305 (NaCl crypto_box)
Authentication	Server has long-term key pair; client has long-term + transient
Perfect forward secrecy	Yes (transient keys per connection)

Transports

Transport	URI	Description
TCP	`tcp://host:port`	Network communication (most common)
IPC	`ipc:///tmp/feed`	Unix domain sockets (same machine, fast)
In-process	`inproc://name`	Inter-thread (zero-copy, fastest)
PGM/EPGM	`pgm://interface;multicast:port`	Reliable multicast (PGM protocol)
UDP	`udp://host:port`	Unreliable datagram (RADIO-DISH pattern)
WebSocket	`ws://host:port`	Browser-compatible (ZMTP over WS)

ZeroMQ vs Other Messaging

Feature	ZeroMQ	NATS	MQTT	AMQP	Kafka
Broker	**None** (peer-to-peer)	Required	Required	Required	Required
Patterns	REQ-REP, PUB-SUB, PUSH-PULL, ROUTER-DEALER	Pub-Sub, Queue, Request-Reply	Pub-Sub	Queue, Exchange, Topic	Partitioned Log
Persistence	None (in-memory only)	JetStream (optional)	Retained messages	Per-message	Always persisted
Language	C (libzmq) + 40+ bindings	Go server + clients	Various	Various	JVM + clients
Latency	Microseconds	Microseconds	Milliseconds	Milliseconds	Milliseconds
Throughput	Millions msg/s	Millions msg/s	Thousands msg/s	Thousands msg/s	Millions msg/s
Use case	Low-latency distributed systems, HPC, trading	Cloud-native microservices	IoT	Enterprise integration	Event streaming

Common Libraries

Language	Library
C	libzmq (reference)
C++	cppzmq
Python	pyzmq
Java	JeroMQ (pure Java) or JZMq (JNI)
Go	pebbe/zmq4, go-zeromq/zmq4
Rust	zmq.rs
Node.js	zeromq.js
C#	NetMQ

Standards

Document	Title
ZMTP 3.1	ZeroMQ Message Transport Protocol
CurveZMQ	CurveZMQ security mechanism
ZeroMQ RFCs	All ZeroMQ specifications
ZeroMQ Guide	Official guide (patterns, examples)

See Also

- MQTT -- broker-based IoT messaging
- AMQP -- broker-based enterprise messaging
- NATS -- lightweight cloud messaging
- Kafka -- distributed event streaming
- TCP -- primary ZeroMQ transport
- DDS / ROS 2 -- another brokerless pub-sub (different domain)

Mobile Sync

Protocols for mobile device synchronization and management.

6 protocols in this section

DeltaSync

Standard: Proprietary (Microsoft, undocumented) | **Layer:** Application (Layer 7) | **Wireshark filter:** `http` (DeltaSync runs over HTTPS)

DeltaSync was Microsoft's proprietary email synchronization protocol for Windows Live Hotmail (later Outlook.com). It provided efficient synchronization of email, contacts, and calendar between the Windows Live Mail desktop client and Microsoft's webmail service. DeltaSync was notable for its compact delta-based synchronization -- only changes were transmitted, reducing bandwidth for mobile and low-speed connections. Microsoft deprecated DeltaSync in 2016, replacing it with Exchange ActiveSync and later with Microsoft Graph/REST APIs.

How DeltaSync Worked

DeltaSync used HTTPS POST requests to Microsoft's sync endpoints:

```
POST https://mail.services.live.com/DeltaSync_v2.0.0/Sync.aspx
Content-Type: application/x-wbxml
```

Like Exchange ActiveSync, DeltaSync used WBXML-encoded XML for request/response bodies, keeping payloads compact for mobile networks.

Endpoints

Endpoint	Purpose
`/DeltaSync_v2.0.0/Sync.aspx`	Email synchronization
`/DeltaSync_v2.0.0/Settings.aspx`	Account settings
`/DeltaSync_v2.0.0/ItemOperations.aspx`	Item operations (fetch, delete)

Core Operations

Operation	Description
Sync	Synchronize email folder contents (delta-based)
GetChanges	Retrieve changes since last sync token
GetItem	Fetch a complete email message
DeleteItem	Delete an email
MoveItem	Move an email between folders
MarkRead	Mark an email as read
SendMessage	Send an email via the service
FolderSync	Synchronize the folder list

Settings	Get/set account settings

Sync Model

DeltaSync used a change-tracking model similar to Exchange ActiveSync:

[Diagram: see online version]

Delta Synchronization

The key design principle was efficiency:

Feature	Description
Sync tokens	Server tracks client's last known state
Header-first	Sync downloads headers first, bodies on demand
Delta only	Only new/changed/deleted items are transmitted
Compressed	WBXML encoding for compact payloads
Batch	Multiple operations in a single HTTP round-trip

Protocol Characteristics

Feature	DeltaSync	EAS	IMAP
Owner	Microsoft (proprietary)	Microsoft (licensed)	IETF (open standard)
Encoding	WBXML over HTTPS	WBXML over HTTPS	Text over TCP/TLS
Push	Polling-based	Direct Push (Ping)	IDLE command
Scope	Email only (later contacts/calendar)	Email, calendar, contacts, tasks	Email only
Clients	Windows Live Mail only	iOS, Android, Windows, third-party	Universal
Server	Hotmail/Outlook.com only	Exchange, third-party	Any IMAP server
Status	**Deprecated** (June 2016)	Active (being replaced by Graph)	Active

Timeline

Year	Event
2005	DeltaSync introduced in Windows Live Mail
2007	Windows Live Mail 2009 uses DeltaSync as primary protocol

2012	Windows Live Mail 2012 adds EAS support alongside DeltaSync
2013	Outlook.com launches (Hotmail rebrand)
2016	Microsoft announces DeltaSync deprecation
June 2016	DeltaSync servers shut down
Post-2016	Windows Live Mail uses IMAP/POP3 as fallback

Why DeltaSync Matters Historically

DeltaSync represented an early attempt at efficient mobile email sync before Exchange ActiveSync was widely licensed. Its delta-based approach -- syncing only changes with compact binary encoding -- influenced the design of later sync protocols. The transition from DeltaSync -> EAS -> Microsoft Graph/REST APIs mirrors the broader industry shift from proprietary binary protocols to standardized HTTP/JSON APIs.

Encapsulation

[Diagram: see online version]

Standards

DeltaSync was never formally standardized or publicly documented:

Resource	Description
Microsoft internal	Protocol specification was not publicly released
MS-ASWBXML	Related WBXML encoding (shared concepts with EAS)

See Also

- Exchange ActiveSync -- Microsoft's replacement sync protocol
- SyncML -- open standard competitor
- WBXML -- encoding format used by DeltaSync
- IMAP -- open email sync protocol (fallback after DeltaSync shutdown)
- HTTP -- transport protocol

Exchange ActiveSync -- EAS

Standard: MS-ASCMD / MS-ASHTTP (Microsoft Open Specifications) | **Layer:** Application (Layer 7) | **Wireshark filter:** `http` (EAS runs over HTTPS with WBXML bodies)

Exchange ActiveSync (EAS) is Microsoft's protocol for synchronizing email, calendar, contacts, tasks, and notes between a mobile device and an Exchange server (or compatible server). It uses HTTP/HTTPS as transport with WBXML-encoded request/response bodies for compact over-the-air transfer. EAS provides push email notification, remote device wipe, policy enforcement, and direct push (long-lived HTTP connections for real-time notifications). It became the dominant mobile sync protocol on iOS, Android, and Windows Phone, and is licensed to many third-party server implementations.

How EAS Works

EAS uses HTTP POST requests to a single endpoint, with the command specified in the query string:

```
POST /Microsoft-Server-ActiveSync?Cmd=Sync&amp;User=alice&amp;DeviceId=ABC123&amp;DeviceType=iPhone
Content-Type: application/vnd.ms-sync.wbxml
```

The request and response bodies are WBXML-encoded XML documents.

Commands

Command	Description
FolderSync	Synchronize the folder hierarchy
FolderCreate	Create a new folder
FolderDelete	Delete a folder
FolderUpdate	Rename or move a folder
Sync	Synchronize items (email, calendar, contacts, tasks, notes)
GetItemEstimate	Get the count of items to sync (plan before downloading)
MoveItems	Move items between folders
Search	Search the mailbox, GAL, or document library
SendMail	Send an email message
SmartReply	Reply to an email (server appends original)
SmartForward	Forward an email (server appends original)
Ping	Register for push notifications (direct push)
ItemOperations	Fetch attachments, empty folder, move items
Provision	Negotiate device security policies
Settings	Get/set OOF, device info, user info
MeetingResponse	Accept, tentatively accept, or decline a meeting
ResolveRecipients	Look up recipients in the GAL
ValidateCert	Validate an S/MIME certificate
Find	Search for items (EAS 16.1+)

Sync Flow

[Diagram: see online version]

SyncKey

The SyncKey is the core state-tracking mechanism:

SyncKey	Meaning
0	Initial sync -- server returns all item IDs
1+	Incremental sync -- only changes since this key

If the server doesn't recognize the SyncKey, it forces a re-sync (reset to 0).

Push Email -- Direct Push (Ping)

EAS provides real-time email notification without polling:

1. Device sends a `Ping` request listing the folders to monitor and a heartbeat interval (up to 3600 seconds)
2. Server holds the HTTP connection open
3. When new mail arrives, the server immediately responds
4. Device issues a `Sync` to fetch the new items
5. Device sends another `Ping` to wait for the next change

This is essentially HTTP long polling, giving push-like behavior without a persistent socket.

Protocol Versions

Version	Exchange	Year	Key Features
2.5	Exchange 2003 SP2	2005	Basic sync, GAL
12.0	Exchange 2007	2006	Document search, HTML email
12.1	Exchange 2007 SP1	2008	Header compression (Base64 binary), bundled sync
14.0	Exchange 2010	2009	SMS sync, conversation view
14.1	Exchange 2010 SP1	2010	S/MIME, IRM, body preferences
16.0	Exchange 2016	2015	Improved Find command
16.1	Exchange Online	2018	Find enhancements, body part preferences

Provisioning (Device Policies)

The Provision command enforces security policies on managed devices:

Policy	Description
DevicePasswordEnabled	Require a device password/PIN
MinDevicePasswordLength	Minimum password length
MaxInactivityTimeDeviceLock	Auto-lock timeout
MaxDevicePasswordFailedAttempts	Wipe after N failed attempts
AllowSimpleDevicePassword	Allow simple PINs (1234, 0000)
DeviceEncryptionEnabled	Require device encryption
AllowCamera	Allow/disallow camera
AllowBrowser	Allow/disallow web browser
RemoteWipe	Remotely erase the device

WBXML Encoding

EAS uses WBXML to compress XML requests/responses for efficient mobile transmission. Each EAS namespace (AirSync, Email, Calendar, Contacts, etc.) has its own WBXML code page with token-to-element mappings.

Code Page	Namespace	Description
0	AirSync	Core sync commands
2	Email	Email-specific elements
4	Calendar	Calendar elements
5	Move	Move operations
7	FolderHierarchy	Folder management
14	Provision	Policy management
15	Search	Search operations
17	AirSyncBase	Common elements (body, attachments)
18	Settings	Device/user settings
20	ItemOperations	Fetch/empty/move operations

Encapsulation

[Diagram: see online version]

EAS always runs over HTTPS. The endpoint path is `/Microsoft-Server-ActiveSync`.

Standards

Document	Title
MS-ASCMD	Exchange ActiveSync: Command Reference Protocol
MS-ASHTTP	Exchange ActiveSync: HTTP Protocol
MS-ASWBXML	Exchange ActiveSync: WBXML Algorithm
MS-ASPROV	Exchange ActiveSync: Provisioning Protocol
MS-ASAIRS	Exchange ActiveSync: AirSyncBase Namespace

See Also

- WBXML -- compact XML encoding used by EAS
- HTTP -- transport protocol
- IMAP -- alternative email access (server-side sync)
- SyncML -- open standard that EAS competed with
- DeltaSync -- Microsoft's earlier Hotmail sync protocol
- TLS -- HTTPS encryption

MM5 -- MMS MMSC-to-HLR Interface

Standard: 3GPP TS 23.140 | **Layer:** Application (Layer 7) | **Wireshark filter:** `gsm_map`

MM5 is the reference interface between the MMS Relay/Server (MMSC) and the Home Location Register (HLR) in the 3GPP Multimedia Messaging Service architecture. The MMSC uses MM5 to query subscriber information needed for message routing and delivery -- specifically, to resolve an MSISDN to a serving network element and to check subscriber service entitlements. MM5 is based on MAP (Mobile Application Part), the same signaling protocol used for voice call routing in GSM/UMTS networks.

MMS Architecture Interfaces

[Diagram: see online version]

MM5 is one of several interfaces defined in the MMS architecture. Each interface serves a different role.

Interface Summary

Interface	Endpoints	Protocol	Purpose
MM1	UE <-> MMSC	HTTP/WSP + WBXML	Message submission and retrieval
MM2	MMSC <-> Message Store	Implementation-specific	Message storage
MM3	MMSC <-> Email	SMTP	Email interworking

MM4	MMSC <-> MMSC	SMTP	Inter-operator MMS relay
MM5	**MMSC <-> HLR**	**MAP (SS7)**	**Subscriber info and routing**
MM6	MMSC <-> User DB	Implementation-specific	User profile management
MM7	MMSC <-> VAS	SOAP/HTTP	Value-added service integration

MM5 Operations

The MMSC uses MM5 to perform MAP queries against the HLR:

MAP Operation	Purpose
SendRoutingInfoForSM	Resolve MSISDN to serving MSC/SGSN address and IMSI
ReportSMDeliveryStatus	Inform HLR of delivery failure (for retry notification)
AlertServiceCentre	HLR notifies MMSC when a subscriber becomes reachable
AnyTimeInterrogation	Query subscriber location and status

Typical MM5 Flow (Message Delivery)

[Diagram: see online version]

Delivery Retry Notification

[Diagram: see online version]

Protocol Stack

MM5 is carried over the SS7 signaling network:

[Diagram: see online version]

In modern deployments, the lower SS7 layers are often replaced by SIGTRAN (SS7 over IP), using M3UA and SCTP.

Standards

Document	Title
3GPP TS 23.140	Multimedia Messaging Service (MMS) -- Functional description; Stage 2
3GPP TS 29.002	Mobile Application Part (MAP) specification
3GPP TS 23.040	Technical realization of SMS (related procedures)

See Also

- SMPP -- alternative interface for SMS submission
- SMTP -- used by MM3 (email) and MM4 (inter-MMSC) interfaces
- WBXML -- encoding used on the MM1 interface
- SS7 -- signaling network that carries MAP

SMPP -- Short Message Peer-to-Peer

Standard: SMPP v3.4 / v5.0 | **Layer:** Application (Layer 7) | **Wireshark filter:** `smpp`

SMPP is the standard protocol for exchanging SMS messages between Short Message Service Centers (SMSCs), External Short Messaging Entities (ESMEs), and routing entities. It is the workhorse of the SMS industry -- used by aggregators, messaging gateways, and applications to submit and receive SMS at high volume. SMPP operates over TCP and uses a binary PDU format with a request/response model. Version 3.4 is the most widely deployed; version 5.0 added features but saw limited adoption.

PDU Header

Every SMPP PDU begins with a 16-byte header:

[Diagram: see online version]

Followed by a variable-length body specific to the command.

Key Fields

Field	Size	Description
Command Length	32 bits	Total PDU length in bytes (header + body)
Command ID	32 bits	Identifies the operation (request or response)
Command Status	32 bits	Error code; 0 = success (only meaningful in responses)
Sequence Number	32 bits	Correlates requests with responses

Field Details

Command IDs

ID	Name	Direction	Description

0x00000001	bind_receiver	ESME -> SMSC	Bind as receiver (MO messages)
0x80000001	bind_receiver_resp	SMSC -> ESME	Response to bind_receiver
0x00000002	bind_transmitter	ESME -> SMSC	Bind as transmitter (MT messages)
0x80000002	bind_transmitter_resp	SMSC -> ESME	Response to bind_transmitter
0x00000004	submit_sm	ESME -> SMSC	Submit a message for delivery
0x80000004	submit_sm_resp	SMSC -> ESME	Response with message ID
0x00000005	deliver_sm	SMSC -> ESME	Deliver a message (MO or DLR)
0x80000005	deliver_sm_resp	ESME -> SMSC	Acknowledge delivery
0x00000006	unbind	Either	Graceful session teardown
0x80000006	unbind_resp	Either	Response to unbind
0x00000009	bind_transceiver	ESME -> SMSC	Bind as both transmitter and receiver
0x80000009	bind_transceiver_resp	SMSC -> ESME	Response to bind_transceiver
0x00000015	enquire_link	Either	Keepalive / connection check
0x80000015	enquire_link_resp	Either	Response to enquire_link
0x00000103	submit_multi	ESME -> SMSC	Submit message to multiple destinations
0x80000103	submit_multi_resp	SMSC -> ESME	Response to submit_multi

Command Status (Error Codes)

Code	Name	Description
0x00000000	ESME_ROK	Success
0x00000001	ESME_RINVMSGLEN	Invalid message length
0x00000002	ESME_RINVCMDLEN	Invalid command length
0x00000003	ESME_RINVCMDID	Invalid command ID
0x00000005	ESME_RALYBND	Already bound
0x0000000D	ESME_RINVSRCADR	Invalid source address
0x0000000E	ESME_RINVDSTADR	Invalid destination address
0x00000045	ESME_RSUBMITFAIL	Submit failed
0x00000058	ESME_RTHROTTLED	Throttled -- too many requests
0x00000088	ESME_RSYSERR	System error

submit_sm Body

The primary PDU for sending messages:

Field	Size	Description
service_type	C-String	Service type (empty = default)
source_addr_ton	1 byte	Source address Type of Number
source_addr_npi	1 byte	Source address Numbering Plan
source_addr	C-String	Sender address/number
dest_addr_ton	1 byte	Destination address Type of Number
dest_addr_npi	1 byte	Destination address Numbering Plan
destination_addr	C-String	Recipient address/number
esm_class	1 byte	Message mode and type
protocol_id	1 byte	Protocol identifier (GSM)
priority_flag	1 byte	Message priority (0-3)
schedule_delivery_time	C-String	Scheduled delivery (empty = immediate)
validity_period	C-String	Message expiry time
registered_delivery	1 byte	Delivery receipt request flags
replace_if_present	1 byte	Replace existing message flag
data_coding	1 byte	Character encoding of the message
sm_default_msg_id	1 byte	Predefined message ID
sm_length	1 byte	Length of short_message
short_message	Variable	Message content (max 254 bytes)
TLVs	Variable	Optional Tag-Length-Value parameters

Data Coding

Value	Encoding	Max Characters (1 SMS)
0x00	GSM 7-bit default	160
0x01	ASCII (IA5)	160
0x03	Latin-1 (ISO-8859-1)	140
0x08	UCS-2 (UTF-16 BE)	70

Type of Number (TON)

Value	Type
0x00	Unknown

0x01	International
0x02	National
0x03	Network Specific
0x05	Alphanumeric
0x06	Abbreviated

Numbering Plan Indicator (NPI)

Value	Plan
0x00	Unknown
0x01	E.164 (ISDN/telephone)
0x03	Data (X.121)
0x06	Land Mobile (E.212)
0x09	Private

Session Flow

[Diagram: see online version]

Optional TLV Parameters

Tag	Name	Description
0x0204	message_payload	Extended message body (>254 bytes)
0x020C	sar_msg_ref_num	Concatenated message reference
0x020E	sar_total_segments	Total segments in concatenated message
0x020F	sar_segment_seqnum	Segment sequence number
0x0427	receipted_message_id	Original message ID (in delivery receipts)
0x001E	message_state	Final message state (in delivery receipts)

Encapsulation

[Diagram: see online version]

Port 2775 is conventional but not standardized; providers often use custom ports.

Standards

Document	Title
SMPP v3.4	Short Message Peer-to-Peer Protocol Specification v3.4
SMPP v5.0	Short Message Peer-to-Peer Protocol Specification v5.0
GSM 03.38	GSM 7-bit default alphabet
GSM 03.40	SMS technical specification

See Also

- TCP
- TLS -- used for encrypted SMPP connections
- SMTP -- email transport (sometimes bridged with SMS)
- MM5 -- MMS inter-MMSC interface

SyncML / OMA DS & DM -- Synchronization Markup Language

Standard: OMA Data Synchronization (DS) v1.2 / OMA Device Management (DM) v1.2 | **Layer:** Application (Layer 7) | **Wireshark filter:** `xml` or `wbxml` (SyncML is XML/WBXML over HTTP)

SyncML is a platform-independent protocol for synchronizing data between a server and client devices. It was developed by the SyncML Initiative (later merged into the Open Mobile Alliance) and covers two use cases: **OMA DS** (Data Synchronization -- contacts, calendar, tasks, notes) and **OMA DM** (Device Management -- firmware updates, configuration, policy). SyncML was the standard sync protocol on feature phones and early smartphones (Nokia, Sony Ericsson, Motorola) before platform-specific cloud sync (iCloud, Google Sync, EAS) took over.

Protocol Overview

SyncML messages are XML documents (or WBXML for compact transmission) exchanged over HTTP, OBEX (Bluetooth/IrDA), or WSP (WAP). Each message contains a header and a body with one or more sync commands.

Message Structure

```
&lt;SyncML&gt;
  &lt;SyncHdr&gt;
    &lt;VerDTD&gt;1.2&lt;/VerDTD&gt;
    &lt;VerProto&gt;SyncML/1.2&lt;/VerProto&gt;
    &lt;SessionID&gt;1&lt;/SessionID&gt;
    &lt;MsgID&gt;1&lt;/MsgID&gt;
    &lt;Target&gt;&lt;LocURI&gt;http://server/syncml&lt;/LocURI&gt;&lt;/Target&gt;
    &lt;Source&gt;&lt;LocURI&gt;IMEI:123456789&lt;/LocURI&gt;&lt;/Source&gt;
    &lt;Cred&gt;...&lt;/Cred&gt;
  &lt;/SyncHdr&gt;
  &lt;SyncBody&gt;
    &lt;Alert&gt;...&lt;/Alert&gt;
    &lt;Sync&gt;...&lt;/Sync&gt;
    &lt;Final/&gt;
  &lt;/SyncBody&gt;
&lt;/SyncML&gt;
```

Key Fields

Element	Description
SyncHdr	Message header (version, session, source, target, credentials)
SessionID	Unique session identifier
MsgID	Sequence number within the session
Target	URI of the recipient
Source	URI of the sender (device IMEI, phone number, etc.)
Cred	Authentication credentials (Basic, MD5 digest, HMAC)
SyncBody	Contains one or more sync commands
Final	Marks the last message in an exchange

Sync Commands (OMA DS)

Command	Description
Alert	Initiate sync, specify sync type
Sync	Container for data operations
Add	Add a new item
Replace	Update an existing item
Delete	Delete an item
Copy	Copy an item
Move	Move an item
Get	Retrieve data
Put	Send device info/capabilities
Map	Map client-side IDs to server-side IDs

Results	Return results of a Get command
Status	Return status code for a command

Sync Types (Alert Codes)

Code	Name	Description
200	Two-way sync	Full bidirectional sync
201	Slow sync	Exchange all items (initial sync or recovery)
202	One-way from client	Client sends changes to server
203	Refresh from client	Client replaces all server data
204	One-way from server	Server sends changes to client
205	Refresh from server	Server replaces all client data
206	Two-way by server	Server-initiated two-way sync
207	One-way from client by server	Server-initiated, client sends
208	Refresh from client by server	Server-initiated, client replaces server
209	One-way from server by server	Server-initiated, server sends
210	Refresh from server by server	Server-initiated, server replaces client

Sync Session Flow

[Diagram: see online version]

Change Detection -- Anchors

SyncML uses **sync anchors** (timestamps or change counters) to track what has been synchronized:

Anchor	Description
Last	Anchor from the previous successful sync
Next	Anchor for this sync session

If the server's Last anchor doesn't match the client's, a slow sync is triggered (exchange all items).

Data Formats

SyncML carries data in various formats depending on the data type:

Data Type	Format	MIME Type
Contacts	vCard 2.1 or 3.0	text/x-vcard or text/vcard

Calendar	vCalendar 1.0 or iCalendar 2.0	text/x-vcalendar or text/calendar
Tasks	vCalendar VTODO	text/x-vcalendar
Notes	Plain text	text/plain
Email	RFC 5322	message/rfc822

Device Management (OMA DM)

OMA DM uses the same SyncML message format but with different commands for managing device configuration:

Command	Description
Get	Read a device management node value
Replace	Set a device management node value
Add	Create a new management node
Delete	Remove a management node
Exec	Execute a command on the device (firmware update, factory reset)
Copy	Copy a management node
Sequence	Execute commands in order
Atomic	Execute commands atomically (all or nothing)

DM Tree

Device configuration is organized as a tree:

```
./DevInfo/DevId       -&gt; IMEI:123456789
./DevInfo/Man         -&gt; Nokia
./DevInfo/Mod         -&gt; N95
./Email/Account1/     -&gt; Email account settings
./WiFi/SSID1/         -&gt; Wi-Fi configuration
./Firmware/Update/    -&gt; Firmware update management
```

Transport Bindings

Transport	Usage
HTTP/HTTPS	Most common (mobile data, Wi-Fi)
OBEX	Bluetooth or IrDA (local sync)
WSP	WAP Session Protocol (legacy mobile)

Status Codes

Code	Meaning
200	OK
201	Item added
212	Authentication accepted
401	Invalid credentials
403	Forbidden
404	Not found
406	Optional feature not supported
407	Credentials required
415	Unsupported media type
500	Server error
508	Refresh required (slow sync needed)

SyncML Ecosystem

Implementation	Description
Nokia PC Suite	Synced contacts/calendar via SyncML
Funambol	Open-source SyncML server
Horde	PHP groupware with SyncML support
ScheduleWorld	Cloud SyncML service (defunct)
Memotoo	Cloud SyncML service
Android (pre-4.0)	Some vendors included SyncML clients

Encapsulation

[Diagram: see online version]

Standards

Document	Title
OMA DS v1.2	OMA Data Synchronization Protocol
OMA DM v1.2	OMA Device Management Protocol
SyncML Representation Protocol	XML/WBXML representation
SyncML HTTP Binding	HTTP transport specification

See Also

- WBXML -- compact encoding used for SyncML on mobile
- HTTP -- primary transport
- Exchange ActiveSync -- Microsoft's competing sync protocol
- IMAP -- email sync alternative

WBXML -- WAP Binary XML

Standard: WAP-192-WBXML | **Layer:** Application (Layer 7) | **Wireshark filter:** `wbxml`

WBXML is a compact binary encoding of XML developed by the WAP Forum (now Open Mobile Alliance) for efficient transmission of XML documents over low-bandwidth mobile networks. Instead of sending verbose text tags, WBXML maps element names, attribute names, and common attribute values to single-byte tokens, dramatically reducing message size. WBXML is used by WAP provisioning, Microsoft Exchange ActiveSync, SyncML, MMS (MM1 interface), and OMA Device Management.

Document Structure

[Diagram: see online version]

All multi-byte integers use a variable-length encoding (high bit = continuation flag, 7 bits of value per byte).

Key Fields

Field	Size	Description
Version	1 byte	WBXML version (e.g., 0x03 = version 1.3)
Public ID	mb_u_int32	Document type identifier (or 0x01 + string table index)
Charset	mb_u_int32	IANA MIBenum character encoding (e.g., 0x6A = UTF-8)
String Table Length	mb_u_int32	Length of the string table in bytes
String Table	Variable	Null-terminated strings referenced by offset
Body	Variable	Stream of tokens representing the XML document

Field Details

Version

Value	WBXML Version
0x00	1.0
0x01	1.1
0x02	1.2
0x03	1.3 (current)

Token Types

The body is a stream of single-byte tokens:

Token	Name	Description
0x00	SWITCH_PAGE	Change the active code page
0x01	END	Close current element
0x02	ENTITY	Character entity (followed by mb_u_int32 code point)
0x03	STR_I	Inline string (followed by null-terminated UTF-8)
0x40	EXT_I_0	Extension token (inline string)
0x43	PI	Processing instruction
0x44	LITERAL	Element tag from string table
0x80	EXT_T_0	Extension token (table reference)
0x83	STR_T	String table reference (followed by offset)
0xC0	EXT_0	Extension token (no data)
0xC3	OPAQUE	Opaque binary data (followed by length + bytes)

Tag Encoding

Element tags are encoded as a single byte. The two high bits have special meaning:

[Diagram: see online version]

Bit	Name	Meaning
7	A (Attributes)	1 = element has attributes
6	C (Content)	1 = element has child content
5-0	Tag	Token value (0x05-0x3F mapped to element names)

Tokens 0x05 through 0x3F represent element names. The mapping from token to element name is defined by the document type's code page (e.g., WML, SI, SL, ActiveSync).

Code Pages

Different document types define different token-to-element mappings:

Public ID	Document Type	Usage
0x02	WML 1.0	WAP Markup Language
0x04	WML 1.1	WAP Markup Language
0x09	WML 1.3	WAP Markup Language
0x05	SI 1.0	Service Indication (push)
0x06	SL 1.0	Service Loading (push)
0x0B	WV CSP 1.1	Wireless Village (IM)
--	ActiveSync	Microsoft Exchange sync (uses literal tags)

String Table

The string table stores commonly repeated strings. References in the token stream point to byte offsets within the table. This avoids repeating the same string multiple times in the document.

Multi-Byte Integer (mb_u_int32)

```
Byte 1: 1xxxxxxx  (continuation, 7 bits of value)
Byte 2: 1xxxxxxx  (continuation, 7 bits of value)
Byte N: 0xxxxxxx  (final byte, 7 bits of value)
```

The high bit of each byte indicates whether more bytes follow. Value is assembled from the 7-bit fields, most significant first.

Example: XML to WBXML

XML:

```
&lt;SI&gt;
  &lt;indication href="http://example.com"&gt;New message&lt;/indication&gt;
&lt;/SI&gt;
```

Becomes approximately 30 bytes of WBXML instead of ~90 bytes of text XML -- a ~70% reduction.

Encapsulation

[Diagram: see online version]

Standards

Document	Title
WAP-192-WBXML	WAP Binary XML Content Format v1.3
W3C NOTE-wbxml	WAP Binary XML Content Format (W3C Note)
MS-ASWBXML	Exchange ActiveSync: WBXML Algorithm
OMA SyncML	SyncML Representation Protocol (uses WBXML)

See Also

- HTTP -- common transport for WBXML content
- SMPP -- alternative messaging protocol (text-based addressing)
- SMTP -- email transport (XML/WBXML used in ActiveSync over HTTP)

Network Management & Monitoring

Network management, monitoring, and observability protocols.

12 protocols in this section

gNMI -- gRPC Network Management Interface

Standard: OpenConfig gNMI Specification | **Layer:** Application (Layer 7) | **Wireshark filter:** `grpc`

gNMI is a gRPC-based protocol for network device configuration and telemetry, developed by the OpenConfig consortium. It provides a modern, high-performance alternative to SNMP and NETCONF for streaming operational data and pushing configuration changes. gNMI uses Protocol Buffers for serialization over HTTP/2 (gRPC), enabling bidirectional streaming, strong typing, and efficient binary encoding. It operates on YANG-modeled data using XPath-style paths and supports streaming telemetry with sub-second granularity -- a fundamental shift from the poll-based approach of SNMP.

gRPC Service Definition

gNMI defines four RPCs in its protobuf service:

```
service gNMI {
  rpc Capabilities(CapabilityRequest) returns (CapabilityResponse);
  rpc Get(GetRequest)                 returns (GetResponse);
  rpc Set(SetRequest)                 returns (SetResponse);
  rpc Subscribe(stream SubscribeRequest) returns (stream SubscribeResponse);
}
```

RPCs

RPC	Type	Description
Capabilities	Unary	Discover supported models, encodings, and gNMI version
Get	Unary	Retrieve snapshots of data from the target
Set	Unary	Modify configuration on the target (transactional)
Subscribe	Bidirectional streaming	Stream telemetry data from the target

Path Encoding

gNMI uses structured paths based on YANG schema nodes:

```
/interfaces/interface[name=eth0]/state/counters/in-octets
```

Encoded as a protobuf `Path` message:

Field	Description
origin	Schema origin (e.g., `openconfig`, `ietf`)

elem[]	Ordered list of path elements
elem.name	Node name in the YANG tree
elem.key	Map of key-value pairs for list entries

Path Examples

Path	Description
`/interfaces/interface[name=eth0]/config/mtu`	MTU config for eth0
`/interfaces/interface[name=eth0]/state/counters`	All counters for eth0
`/network-instances/network-instance[name=default]/protocols`	Routing protocols
`/system/config/hostname`	Device hostname
`/components/component[name=*]/state/temperature`	All component temperatures

Capabilities Exchange

[Diagram: see online version]

CapabilityResponse Fields

Field	Description
supported_models[]	YANG models with name, organization, and version
supported_encodings[]	JSON, JSON_IETF, PROTO, ASCII, BYTES
gnmi_version	Protocol version string (e.g., "0.10.0")

Get Operation

GetRequest Field	Description
path[]	List of paths to retrieve
type	CONFIG, STATE, OPERATIONAL, or ALL
encoding	Desired response encoding

GetResponse Field	Description
notification[]	List of Notification messages with timestamped data

Set Operation

Set is transactional -- all operations in a single SetRequest either succeed or fail atomically:

[Diagram: see online version]

Set Operation Types

Operation	Description
delete	Remove the data at the specified path
replace	Replace the entire subtree at the path (missing leaves reset to defaults)
update	Merge the provided data with existing config (like NETCONF merge)

All operations in a single SetRequest are applied as one transaction. If any operation fails, the entire Set is rolled back.

Subscribe (Streaming Telemetry)

Subscribe is the core differentiator of gNMI -- enabling push-based, real-time telemetry:

Subscription Modes

Mode	Description
STREAM	Long-lived subscription; target continuously sends updates
ONCE	One-shot retrieval; target sends current values then closes
POLL	Client-driven; client sends Poll messages to trigger updates on demand

Stream Sub-Modes

Sub-Mode	Description
TARGET_DEFINED	Target chooses the best mode per leaf (vendor-optimized)
ON_CHANGE	Send update only when the value changes
SAMPLE	Send current value at a fixed interval (sample_interval)

Streaming Telemetry Flow

[Diagram: see online version]

Notification Message

Field	Description

timestamp	Nanoseconds since Unix epoch
prefix	Common path prefix for all updates in this notification
update[]	List of path-value pairs (updated/current data)
delete[]	List of paths removed from the data tree
atomic	If true, all updates must be treated as a single snapshot

Encoding Formats

Encoding	Description
JSON	Standard JSON (deprecated in favor of JSON_IETF)
JSON_IETF	JSON per RFC 7951 (YANG-to-JSON mapping rules)
PROTO	Binary protobuf encoding of typed values
ASCII	CLI-style text (vendor-specific)
BYTES	Raw bytes (opaque to gNMI)

gNOI (gRPC Network Operations Interface)

gNOI extends the gRPC management model with operational RPCs beyond config/telemetry:

gNOI Service	Description
System	Reboot, SetPackage, SwitchControlProcessor, Ping, Traceroute, Time
OS	Install, Activate, Verify device OS images
Cert	Rotate, Install, GetCertificates -- certificate management
File	Get, Put, Stat, Remove -- file operations on device
BGP	ClearBGPNeighbor -- clear BGP sessions
Interface	ClearInterfaceCounters
Layer2	ClearLLDPInterface, ClearSpanningTree
Healthz	Get, Check, Acknowledge -- component health
Factory Reset	Start -- return device to factory defaults

gNMI vs NETCONF vs SNMP

Feature	gNMI	NETCONF	SNMP
Transport	gRPC/HTTP2 (TLS)	SSH (port 830)	UDP (port 161/162)
Encoding	Protobuf, JSON_IETF	XML	ASN.1 BER

Data model	YANG (OpenConfig)	YANG	MIB (SMI)
Streaming telemetry	Native (Subscribe)	Limited (RFC 5277)	No (poll only)
Config transactions	Yes (Set)	Yes (lock/commit)	No
Sub-second telemetry	Yes (SAMPLE mode)	No	No
On-change updates	Yes (ON_CHANGE mode)	Notification streams	Traps only
Bidirectional streaming	Yes (gRPC)	No	No
Binary efficiency	High (protobuf)	Low (XML)	Medium (BER)
Human-readable	JSON_IETF option	Somewhat (XML)	No
Origin	OpenConfig (2015)	IETF (2006/2011)	IETF (1988/2002)
Operations (gNOI)	Reboot, OS install, cert, file	Via vendor RPCs	Very limited

Encapsulation

[Diagram: see online version]

Default port varies by implementation (commonly 9339, 6030, or 57400). No IANA-assigned port.

Standards

Document	Title
gNMI Specification	gRPC Network Management Interface specification
gNMI Proto	gNMI Protocol Buffers definition
gNOI Repository	gRPC Network Operations Interface definitions
OpenConfig Models	OpenConfig YANG models
RFC 7951	JSON Encoding of Data Modeled with YANG
RFC 7950	The YANG 1.1 Data Modeling Language

See Also

- NETCONF -- XML/SSH-based YANG datastore management
- SNMP -- legacy polling-based network monitoring
- RESTCONF -- HTTP-based YANG datastore access
- OTLP -- application-layer observability telemetry

NETCONF -- Network Configuration Protocol

Standard: RFC 6241 | **Layer:** Application (Layer 7) | **Wireshark filter:** `netconf`

NETCONF is an IETF protocol for installing, manipulating, and deleting the configuration of network devices. It uses an RPC-based communication model where a client (manager) sends XML-encoded requests to a server (agent) over a secure transport, primarily SSH. Configuration data is modeled using YANG (RFC 7950), a rich data modeling language that defines the structure, constraints, and semantics of device configuration and state data. NETCONF supports multiple configuration datastores (running, candidate, startup) and provides transactional operations with locking, validation, and commit/rollback.

NETCONF Message Framing

NETCONF 1.0 (End-of-Message)

Messages are delimited by the character sequence `]]>]]>`:

```
&lt;?xml version="1.0" encoding="UTF-8"?&gt;
&lt;rpc message-id="1" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0"&gt;
  &lt;get-config&gt;&lt;source&gt;&lt;running/&gt;&lt;/source&gt;&lt;/get-config&gt;
&lt;/rpc&gt;
]]&gt;]]&gt;
```

NETCONF 1.1 (Chunked Framing)

Messages use chunked framing (RFC 6242) with explicit byte counts:

[Diagram: see online version]

Element	Description
`\n#<size>\n`	Chunk header with byte count in decimal
chunk-data	XML message bytes (up to chunk-size)
`\n##\n`	End-of-message marker

NETCONF Layers

[Diagram: see online version]

RPC Message Structure

All NETCONF messages use a simple RPC envelope:

Message	Root Element	Description
Request	`<rpc>`	Client-to-server operation request
Reply	`<rpc-reply>`	Server-to-client response

Notification	`<notification>`	Server-to-client async event

Key Attributes

Attribute	Description
message-id	Matches requests to replies (required on `<rpc>`)
xmlns	Namespace -- `urn:ietf:params:xml:ns:netconf:base:1.0`

Operations

Operation	Description
`<get>`	Retrieve running config and device state data
`<get-config>`	Retrieve all or part of a configuration datastore
`<edit-config>`	Modify a configuration datastore
`<copy-config>`	Copy entire configuration between datastores or to/from URL
`<delete-config>`	Delete a configuration datastore (not running)
`<lock>`	Lock a datastore for exclusive access
`<unlock>`	Release a datastore lock
`<close-session>`	Gracefully close the NETCONF session
`<kill-session>`	Force-close another NETCONF session
`<commit>`	Commit candidate configuration to running (`:candidate` capability)
`<discard-changes>`	Revert candidate datastore to current running config
`<validate>`	Validate a datastore or inline config (`:validate` capability)

edit-config Operations

The `operation` attribute on elements within `<edit-config>` controls merge behavior:

Operation	Description
merge	Merge with existing config (default)
replace	Replace the target element entirely
create	Create new element; error if it already exists
delete	Delete the element; error if it does not exist
remove	Delete the element if it exists; no error if absent

Datastores

[Diagram: see online version]

Datastore	Description
running	Currently active configuration on the device
candidate	Working copy for staged changes before commit
startup	Configuration loaded at boot (if separate from running)

Capability Exchange

When a NETCONF session starts, both sides exchange `<hello>` messages advertising their capabilities:

[Diagram: see online version]

Common Capabilities

Capability URI (suffix)	Description
`:base:1.0`	NETCONF base protocol (RFC 4741)
`:base:1.1`	NETCONF base with chunked framing (RFC 6241)
`:candidate`	Candidate configuration datastore
`:confirmed-commit`	Commit with automatic rollback timer
`:rollback-on-error`	Roll back entire edit-config on any error
`:validate`	Configuration validation support
`:startup`	Separate startup datastore
`:writable-running`	Direct edit of running datastore
`:xpath`	XPath filter support
`:notification`	Event notification support (RFC 5277)
`:with-defaults`	Control retrieval of default values

Edit-Config with Candidate Commit

[Diagram: see online version]

Notification Subscription

[Diagram: see online version]

YANG Data Modeling Language

YANG (RFC 7950) defines the schema for NETCONF data:

YANG Node Types

Node Type	Description
module	Top-level unit of YANG definition; defines namespace
container	Grouping node with no value itself (like a directory)
list	Sequence of entries identified by key leaf(s)
leaf	Single scalar value (string, integer, boolean, etc.)
leaf-list	Sequence of scalar values (like a leaf array)
choice / case	Mutually exclusive branches
rpc	Defines an operation with input/output
notification	Defines an async event structure
augment	Extends another module's data tree
deviation	Documents how an implementation differs from the model
grouping / uses	Reusable node templates
typedef	Named type definition with constraints

YANG Built-in Types

Type	Description
string	Text with optional length/pattern constraints
int8, int16, int32, int64	Signed integers
uint8, uint16, uint32, uint64	Unsigned integers
boolean	true or false
enumeration	One of a fixed set of named values
bits	Set of named bit flags
binary	Base64-encoded binary data
leafref	Reference to another leaf in the data tree
identityref	Reference to a YANG identity
union	One of several types
empty	Presence/absence indicator (no value)
inet:ipv4-address	IPv4 address (from ietf-inet-types)
inet:ipv6-address	IPv6 address (from ietf-inet-types)
yang:date-and-time	Timestamp (from ietf-yang-types)

Example YANG Module

```
module example-interfaces {
  namespace "urn:example:interfaces";
  prefix if;

  container interfaces {
    list interface {
      key "name";
      leaf name        { type string; }
      leaf enabled     { type boolean; default true; }
      leaf mtu         { type uint16 { range "68..9000"; } }
      leaf ip-address  { type inet:ipv4-address; }
    }
  }
}
```

Maps to NETCONF XML:

```
&lt;interfaces&gt;
  &lt;interface&gt;
    &lt;name&gt;eth0&lt;/name&gt;
    &lt;enabled&gt;true&lt;/enabled&gt;
    &lt;mtu&gt;1500&lt;/mtu&gt;
    &lt;ip-address&gt;10.0.0.1&lt;/ip-address&gt;
  &lt;/interface&gt;
&lt;/interfaces&gt;
```

NETCONF Error Handling

Error Tag	Description
in-use	Resource is locked by another session
invalid-value	Value does not meet YANG constraints
data-missing	Required data is absent
data-exists	Element already exists (create operation)
operation-not-supported	Operation not supported by server
lock-denied	Lock held by another session
access-denied	Insufficient authorization
rollback-failed	Rollback-on-error failed

NETCONF vs SNMP vs RESTCONF

Feature	NETCONF	SNMP	RESTCONF
Transport	SSH (port 830), TLS	UDP (port 161/162)	HTTPS (port 443)
Encoding	XML	ASN.1 BER	JSON or XML
Data model	YANG	MIB (SMI)	YANG

Config operations	Full CRUD + commit	SetRequest only	HTTP CRUD
Transactions	Yes (lock, validate, commit)	No	Partial (YANG Patch)
Candidate config	Yes	No	No (typically)
Rollback	Yes (confirmed-commit)	No	No
Streaming events	Yes (RFC 5277)	Traps/Informs	SSE
Human-readable	Somewhat (XML)	No (BER)	Yes (JSON)
Tooling	ncclient, YANG Explorer	Net-SNMP, snmpwalk	curl, Postman
Adoption	All major vendors	Universal	Growing

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 6241	Network Configuration Protocol (NETCONF)
RFC 6242	Using the NETCONF Protocol over SSH
RFC 7950	The YANG 1.1 Data Modeling Language
RFC 5277	NETCONF Event Notifications
RFC 6470	NETCONF Base Notifications
RFC 7895	YANG Module Library
RFC 8040	RESTCONF Protocol (HTTP-based alternative)
RFC 8341	Network Configuration Access Control Model (NACM)
RFC 8525	YANG Library
RFC 4741	NETCONF (original, obsoleted by 6241)

See Also

- SNMP -- polling/trap-based management predecessor
- RESTCONF -- HTTP-based YANG datastore access
- gNMI -- gRPC-based streaming telemetry and config
- SSH -- primary transport for NETCONF

NetFlow / IPFIX -- IP Flow Information Export

Standard: RFC 7011 (IPFIX) / Cisco NetFlow v5/v9 | **Layer:** Application (Layer 7) | **Wireshark filter:** `netflow` or `cflow`

NetFlow and IPFIX are protocols for exporting IP traffic flow records from routers, switches, and firewalls to a collector for traffic analysis, capacity planning, billing, and security monitoring. A "flow" is a set of packets sharing common attributes (src/dst IP, ports, protocol) observed during a time window. NetFlow v5 is the original Cisco format; NetFlow v9 introduced templates; IPFIX is the IETF standardization of NetFlow v9 with extensions.

NetFlow v5 Record

Each flow record is a fixed 48-byte structure:

[Diagram: see online version]

NetFlow v5 Header

[Diagram: see online version]

Key Fields

Field	Size	Description
Version	16 bits	Protocol version (5, 9, or 10 for IPFIX)
Flow Count	16 bits	Number of flow records in this packet
System Uptime	32 bits	Milliseconds since device boot
Unix Seconds/Nanoseconds	64 bits	Current time
Flow Sequence	32 bits	Sequence counter of total flows exported
Source/Dest IP	32 bits	Flow endpoints
Source/Dest Port	16 bits	Transport ports
Protocol	8 bits	IP protocol number (6=TCP, 17=UDP, etc.)
Packet Count	32 bits	Packets in this flow
Byte Count	32 bits	Total bytes in this flow
TCP Flags	8 bits	Cumulative OR of all TCP flags seen
ToS	8 bits	IP Type of Service / DSCP
Source/Dest AS	16 bits	BGP autonomous system numbers
Input/Output Interface	16 bits	SNMP ifIndex of ingress/egress interface

IPFIX (NetFlow v10) Message

IPFIX uses a template-based format -- the exporter first sends template definitions, then data records referencing those templates:

IPFIX Header

[Diagram: see online version]

Set Types

Set ID	Type	Description
2	Template Set	Defines the format of data records
3	Options Template	Defines format for scope/option records
256+	Data Set	Flow records using a template

Template Record

[Diagram: see online version]

Common Information Elements (IEs)

IE ID	Name	Size	Description
1	octetDeltaCount	8	Bytes in the flow
2	packetDeltaCount	8	Packets in the flow
4	protocolIdentifier	1	IP protocol number
5	ipClassOfService	1	ToS / DSCP
6	tcpControlBits	2	TCP flags
7	sourceTransportPort	2	Source port
8	sourceIPv4Address	4	Source IPv4
10	ingressInterface	4	SNMP ifIndex (ingress)
11	destinationTransportPort	2	Dest port
12	destinationIPv4Address	4	Dest IPv4
14	egressInterface	4	SNMP ifIndex (egress)
15	ipNextHopIPv4Address	4	Next-hop IP
16	bgpSourceAsNumber	4	Source AS
17	bgpDestinationAsNumber	4	Dest AS
21	flowEndSysUpTime	4	Flow end timestamp
22	flowStartSysUpTime	4	Flow start timestamp

27	sourceIPv6Address	16	Source IPv6
28	destinationIPv6Address	16	Dest IPv6
61	flowDirection	1	0=ingress, 1=egress
136	flowEndReason	1	Why the flow ended
148	flowId	8	Unique flow identifier

Export Flow

[Diagram: see online version]

Flow Cache and Timeouts

Timeout	Default	Description
Active timeout	30 minutes	Export long-lived flows periodically
Inactive timeout	15 seconds	Export flows with no new packets
TCP FIN/RST	Immediate	Export completed TCP sessions
Cache full	Immediate	Export oldest flows when cache is exhausted

Sampling

To reduce CPU load on high-speed routers, NetFlow supports sampling:

Method	Description
Deterministic (1-in-N)	Sample every Nth packet
Random (1-in-N)	Randomly sample with probability 1/N
sFlow	Interface-counter + packet-sample combined protocol

NetFlow v5 vs v9 vs IPFIX

Feature	v5	v9	IPFIX (v10)
Format	Fixed 48-byte records	Template-based	Template-based
IPv6	No	Yes	Yes
MPLS	No	Yes	Yes
Variable fields	No	Yes (templates)	Yes (templates)
Enterprise fields	No	No	Yes (PEN)
Standard	Cisco proprietary	Cisco proprietary	IETF RFC 7011

Transport	UDP only	UDP	UDP, TCP, SCTP

Common Collectors

Software	Type	Description
ntopng	Open-source	Web-based flow analysis
nfdump/nfsen	Open-source	CLI flow tools + web frontend
Elasticsearch + Logstash	Open-source	ELK stack with NetFlow input
SolarWinds NTA	Commercial	Network Traffic Analyzer
Kentik	Commercial/SaaS	Cloud-based network analytics
PRTG	Commercial	Network monitoring with flow support

Encapsulation

[Diagram: see online version]

Ports are configurable; 2055 and 9995-9996 are common for NetFlow, 4739 is IANA-assigned for IPFIX.

Standards

Document	Title
RFC 7011	IPFIX Protocol Specification
RFC 7012	IPFIX Information Model
RFC 7015	IPFIX Flow Aggregation
RFC 5101	IPFIX (original, superseded by 7011)
Cisco NetFlow v5	NetFlow v5 format documentation
Cisco NetFlow v9	NetFlow v9 specification

See Also

- SNMP -- complementary monitoring (device status vs traffic flows)
- Syslog -- complementary logging
- UDP -- primary transport
- BGP -- AS path information included in flow records
- OTLP -- modern observability telemetry

OpenFlow

Standard: ONF OpenFlow Switch Specification 1.3.5 | **Layer:** Application (Layer 7) | **Wireshark filter:** `openflow` or `openflow_v4`

OpenFlow is the foundational protocol of Software-Defined Networking (SDN), enabling an external controller to program the forwarding behavior of network switches. It separates the control plane (routing decisions) from the data plane (packet forwarding), allowing centralized, programmable network management. The controller communicates with switches over a secure TCP connection, installing flow rules that define how packets are matched and what actions to apply. OpenFlow transforms switches from autonomous devices with embedded control logic into simple forwarding elements driven by a logically centralized controller.

OpenFlow Header

Every OpenFlow message begins with an 8-byte header:

[Diagram: see online version]

Key Fields

Field	Size	Description
Version	8 bits	Protocol version (0x01=1.0, 0x02=1.1, 0x03=1.2, 0x04=1.3, 0x06=1.5)
Type	8 bits	Message type (see Message Types table)
Length	16 bits	Total message length including header (in bytes)
xid	32 bits	Transaction ID -- matches requests to replies

Message Types (OpenFlow 1.3)

Controller-to-Switch

Type	Value	Description
HELLO	0	Version negotiation on connection startup
ERROR	1	Error notification
ECHO_REQUEST	2	Keepalive / latency measurement
ECHO_REPLY	3	Response to Echo Request
FEATURES_REQUEST	5	Request switch capabilities
FEATURES_REPLY	6	Switch capabilities (datapath ID, buffers, tables)

GET_CONFIG_REQUEST	7	Request switch configuration
GET_CONFIG_REPLY	8	Switch configuration response
SET_CONFIG	9	Set switch configuration
PACKET_OUT	13	Send a packet out of a switch port
FLOW_MOD	14	Add, modify, or delete flow table entries
GROUP_MOD	15	Add, modify, or delete group table entries
PORT_MOD	16	Modify port configuration
TABLE_MOD	17	Modify table configuration
STATS_REQUEST	18	Request statistics
STATS_REPLY	19	Statistics response
BARRIER_REQUEST	20	Ensure all prior messages are processed
BARRIER_REPLY	21	Confirmation that barrier is complete
ROLE_REQUEST	24	Set controller role (master/slave/equal)
ROLE_REPLY	25	Controller role response
METER_MOD	29	Add, modify, or delete meter entries

Switch-to-Controller (Async)

Type	Value	Description
PACKET_IN	10	Forward a packet to the controller
FLOW_REMOVED	11	Notify controller that a flow was removed
PORT_STATUS	12	Notify controller of port state change

Switch Connection Setup

[Diagram: see online version]

Flow Table

Each switch contains one or more flow tables. Packets are matched against entries in priority order:

Flow Entry Structure

[Diagram: see online version]

Component	Description
Match Fields	Packet header fields to match against

Priority	Higher value = higher priority (0-65535)
Counters	Packet count, byte count, duration
Instructions	Actions to apply to matching packets
Idle Timeout	Remove flow after N seconds of inactivity (0 = never)
Hard Timeout	Remove flow after N seconds regardless (0 = never)
Cookie	Opaque value set by the controller for identification

Match Fields (OXM)

Field	OXM Type	Description
IN_PORT	0	Ingress switch port
ETH_DST	3	Destination MAC address
ETH_SRC	4	Source MAC address
ETH_TYPE	5	EtherType (0x0800=IPv4, 0x0806=ARP, 0x86DD=IPv6)
VLAN_VID	6	VLAN ID
VLAN_PCP	7	VLAN priority
IP_DSCP	8	IP DSCP (differentiated services)
IP_PROTO	10	IP protocol (6=TCP, 17=UDP, 1=ICMP)
IPV4_SRC	11	Source IPv4 address (with mask)
IPV4_DST	12	Destination IPv4 address (with mask)
TCP_SRC	13	TCP source port
TCP_DST	14	TCP destination port
UDP_SRC	15	UDP source port
UDP_DST	16	UDP destination port
ICMPV4_TYPE	19	ICMPv4 type
ARP_OP	21	ARP opcode
IPV6_SRC	26	Source IPv6 address
IPV6_DST	27	Destination IPv6 address
TUNNEL_ID	38	Logical tunnel metadata

Instructions

Instruction	Description
Apply-Actions	Execute actions immediately on the packet
Write-Actions	Add actions to the packet's action set (executed at end of pipeline)

Clear-Actions	Clear the action set
Goto-Table	Continue processing at a specified table
Write-Metadata	Write metadata for use by subsequent tables
Meter	Direct packet to a meter for rate limiting

Actions

Action	Description
OUTPUT	Forward packet to a port (or CONTROLLER, FLOOD, ALL, IN_PORT)
SET_FIELD	Modify a packet header field (rewrite MAC, IP, port, etc.)
PUSH_VLAN	Push a new VLAN tag onto the packet
POP_VLAN	Remove the outermost VLAN tag
PUSH_MPLS	Push an MPLS label
POP_MPLS	Pop an MPLS label
GROUP	Forward to a group table entry
SET_QUEUE	Set the output queue for QoS
DROP	(implicit) No output action = drop the packet

Pipeline Processing

Packets traverse multiple flow tables in sequence:

[Diagram: see online version]

Each table processes the packet independently. A table-miss entry (priority=0, match-all) defines what happens when no flow matches -- typically send to controller via PACKET_IN.

Reactive Forwarding (Packet-In / Flow-Mod / Packet-Out)

[Diagram: see online version]

FLOW_MOD Commands

Command	Value	Description
ADD	0	Add a new flow entry
MODIFY	1	Modify matching flow entries
MODIFY_STRICT	2	Modify entry with exact match + priority
DELETE	3	Delete matching flow entries

DELETE_STRICT	4	Delete entry with exact match + priority

Group Table

Groups enable complex forwarding like multicast, load balancing, and fast failover:

Group Type	Description
ALL	Execute all action buckets (multicast/broadcast)
SELECT	Execute one bucket (load balancing -- round-robin or weighted)
INDIRECT	Execute a single defined bucket (next-hop indirection)
FAST_FAILOVER	Execute first live bucket (link failover)

Meter Table

Meters provide per-flow rate limiting:

Meter Band Type	Description
DROP	Drop packets exceeding the rate
DSCP_REMARK	Lower DSCP value for exceeding packets

Controller Roles (Multi-Controller)

Role	Description
EQUAL	Full access (default); all controllers are equal
MASTER	Full read/write; only one master at a time
SLAVE	Read-only; receives async messages based on config

OpenFlow vs Traditional Networking

Aspect	OpenFlow / SDN	Traditional Networking
Control plane	Centralized (external controller)	Distributed (per-device)
Forwarding decisions	Controller programs flow tables	Each device runs routing protocols
Configuration	Programmatic (controller API)	CLI / SNMP per device
Flexibility	Arbitrary match/action rules	Fixed protocol behavior
Vendor lock-in	Reduced (standard southbound API)	High (proprietary CLI/features)
Convergence	Controller-driven (can be instant)	Protocol-dependent (seconds to minutes)

Scalability	Controller must handle all switches	Each device operates independently
Failure mode	Controller failure = no new flows	Devices continue autonomously
Network visibility	Global view at controller	Per-device local view
Use cases	Data centers, WANs, research, campus	Enterprise, ISP, general purpose

Version History

Version	Year	Key Additions
1.0	2009	Single table, basic match/action
1.1	2011	Multiple tables, groups, MPLS
1.2	2011	IPv6 matching, controller roles
1.3	2012	Meters, per-flow rate limiting, table-miss entry, OXM match
1.4	2013	Optical ports, bundles, synchronized tables
1.5	2014	Egress tables, copy-field action, scheduled bundles

Encapsulation

[Diagram: see online version]

Port 6653 is the IANA-assigned port (since OpenFlow 1.3.3). Port 6633 was used by earlier implementations and remains common. TLS is recommended for production deployments.

Standards

Document	Title
OpenFlow 1.3.5	OpenFlow Switch Specification 1.3.5 (most widely deployed)
OpenFlow 1.5.1	OpenFlow Switch Specification 1.5.1 (latest)
ONF TR-523	OpenFlow Table Type Patterns
ONF	Open Networking Foundation (standards body)

See Also

- SNMP -- device monitoring (complementary to SDN control)
- NETCONF -- configuration management for network devices
- LLDP -- topology discovery used alongside SDN
- BGP -- traditional distributed routing protocol

OTLP -- OpenTelemetry Protocol

Standard: OpenTelemetry Specification (opentelemetry.io) | **Layer:** Application (Layer 7) | **Wireshark filter:** `http2` or `grpc` (OTLP/gRPC) or `http` (OTLP/HTTP)

OTLP is the native protocol for OpenTelemetry -- the industry-standard observability framework for collecting traces, metrics, and logs from applications and infrastructure. OTLP defines how telemetry data is serialized (Protocol Buffers) and transported (gRPC or HTTP) from instrumented applications to observability backends (Jaeger, Prometheus, Grafana, Datadog, Honeycomb, etc.). It is the wire protocol behind the OpenTelemetry Collector and SDKs in 11+ languages.

Signal Types

OTLP carries three telemetry signals:

Signal	Description	Example
Traces	Distributed request traces (spans)	HTTP request -> DB query -> cache lookup
Metrics	Measurements over time	CPU usage, request count, latency histogram
Logs	Structured log records	Error message with trace context

Transport Variants

Transport	Endpoint	Content-Type	Description
OTLP/gRPC	`host:4317`	application/grpc	Full gRPC with streaming, preferred
OTLP/HTTP (protobuf)	`host:4318/v1/{signal}`	application/x-protobuf	HTTP POST with protobuf body
OTLP/HTTP (JSON)	`host:4318/v1/{signal}`	application/json	HTTP POST with JSON body (human-readable)

HTTP Endpoints

Signal	Path
Traces	`/v1/traces`
Metrics	`/v1/metrics`
Logs	`/v1/logs`

Data Model

Traces

[Diagram: see online version]

Key Trace Fields

Field	Type	Description
trace_id	16 bytes	Globally unique trace identifier
span_id	8 bytes	Unique span identifier within a trace
parent_span_id	8 bytes	Parent span (empty for root span)
name	string	Operation name
kind	enum	CLIENT, SERVER, PRODUCER, CONSUMER, INTERNAL
start_time_unix_nano	uint64	Start timestamp (nanoseconds since epoch)
end_time_unix_nano	uint64	End timestamp
status	Status	OK, ERROR, UNSET
attributes	KeyValue[]	Span metadata (HTTP method, DB statement, etc.)
events	Event[]	Timestamped events within the span
links	Link[]	Causal references to other spans/traces

Metrics

Metric Type	Description	Example
Gauge	Current value	CPU usage, temperature, queue depth
Sum (Counter)	Monotonically increasing total	Request count, bytes sent
Histogram	Distribution of values	Request latency distribution
ExponentialHistogram	Compact histogram with exponential buckets	High-cardinality latency data
Summary	Pre-computed quantiles (legacy)	P50, P90, P99 latency

Logs

Field	Description
time_unix_nano	Log timestamp

severity_number	1-24 (maps to TRACE, DEBUG, INFO, WARN, ERROR, FATAL)
severity_text	Human-readable severity ("ERROR")
body	Log message (string or structured)
attributes	Structured metadata
trace_id	Correlation with distributed trace
span_id	Correlation with specific span

Architecture

[Diagram: see online version]

Collector Pipeline

Component	Description
Receivers	Accept telemetry (OTLP, Prometheus, Jaeger, Zipkin)
Processors	Transform, filter, batch, sample
Exporters	Send to backends (OTLP, Prometheus, Jaeger, vendor-specific)

Protocol Flow

OTLP/gRPC

[Diagram: see online version]

OTLP/HTTP

```
POST /v1/traces HTTP/1.1
Host: collector:4318
Content-Type: application/x-protobuf

[protobuf-encoded ExportTraceServiceRequest]
```

Response: HTTP 200 with `ExportTraceServiceResponse`.

Partial failures return HTTP 200 with `partial_success` indicating how many items were rejected.

Semantic Conventions

OpenTelemetry defines standard attribute names:

Attribute	Description
`service.name`	Logical service name
`service.version`	Service version
`http.request.method`	HTTP method (GET, POST, etc.)
`http.response.status_code`	HTTP status code
`url.full`	Full request URL
`db.system`	Database type (postgresql, redis, etc.)
`db.statement`	Database query
`rpc.system`	RPC system (grpc, aws-api)
`messaging.system`	Messaging system (kafka, rabbitmq)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
OTLP Specification	OpenTelemetry Protocol Specification
OTel Data Model	OpenTelemetry Specification (data model)
Semantic Conventions	Standard attribute names
Proto definitions	Protobuf schema for OTLP

See Also

- gRPC -- primary transport for OTLP
- HTTP -- alternative transport (OTLP/HTTP)
- TCP

PTP -- Precision Time Protocol

Standard: IEEE 1588-2019 | **Layer:** Application (Layer 7) | **Wireshark filter:** `ptp`

PTP synchronizes clocks across networked devices with sub-microsecond accuracy, far exceeding what NTP can achieve. It is the timing backbone for telecom networks, financial trading systems, power grids, broadcast media, and industrial automation. PTP uses a master-slave hierarchy where a Grandmaster clock distributes time to all other

clocks in a PTP domain. Hardware timestamping at the physical layer is key to achieving nanosecond-level precision. The protocol was originally defined as IEEE 1588-2002 (PTPv1) and significantly revised in IEEE 1588-2008 (PTPv2); the current edition is IEEE 1588-2019.

Header

[Diagram: see online version]

The PTP common header is 34 bytes (272 bits). All PTP messages begin with this header, followed by message-type-specific fields and optional TLVs.

Key Fields

Field	Size	Description
transportSpecific	4 bits	Transport-specific flags (e.g., 0x1 for 802.1AS)
messageType	4 bits	Type of PTP message
versionPTP	4 bits	PTP version number (2 for PTPv2)
messageLength	16 bits	Total message length in bytes
domainNumber	8 bits	PTP domain (0-127), allowing multiple independent PTP instances
flags	16 bits	Bitfield for two-step, unicast, leap second indicators, etc.
correctionField	64 bits	Nanoseconds correction (fixed-point, 48.16 format) accumulated by transparent clocks
sourcePortIdentity	80 bits	Unique identity: clockIdentity (64 bits) + portNumber (16 bits)
sequenceId	16 bits	Sequence number for matching requests to responses
controlField	8 bits	Legacy field for backward compatibility with PTPv1
logMessageInterval	8 bits	Message transmission rate as log2 seconds (e.g., -3 = 8 per second)

Message Types

Event Messages (Timestamped)

Value	Name	Description
0x0	Sync	Master-to-slave time synchronization

0x1	Delay_Req	Slave-to-master delay measurement request (E2E)
0x2	Pdelay_Req	Peer delay measurement request (P2P)
0x3	Pdelay_Resp	Peer delay measurement response (P2P)

General Messages (Not Timestamped at Hardware Level)

Value	Name	Description
0x8	Follow_Up	Carries precise Sync departure timestamp (two-step clocks)
0x9	Delay_Resp	Master's response to Delay_Req with receive timestamp
0xA	Pdelay_Resp_Follow_Up	Carries precise Pdelay_Resp departure timestamp
0xB	Announce	Clock quality advertisement for BMCA
0xC	Signaling	Negotiation messages (unicast, intervals)
0xD	Management	Configuration and monitoring of PTP nodes

Flags Field

Bit	Name	Description
0	alternateMasterFlag	Sender is an alternate master
1	twoStepFlag	Follow_Up message will follow (two-step clock)
2	unicastFlag	Message is unicast (not multicast)
5	PTP profile Specific 1	Defined by PTP profile
6	PTP profile Specific 2	Defined by PTP profile
8	leap61	Last minute of current UTC day has 61 seconds
9	leap59	Last minute of current UTC day has 59 seconds
10	currentUtcOffsetValid	The UTC offset field is valid
11	ptpTimescale	Using PTP (TAI) timescale
12	timeTraceable	Timescale is traceable to a primary reference
13	frequencyTraceable	Frequency is traceable to a primary reference

Clock Synchronization

Two-Step Clock (Most Common)

The master sends a Sync message and records the exact departure timestamp afterward, then sends it in a Follow_Up:

[Diagram: see online version]

One-Step Clock

The master embeds the precise departure timestamp directly in the Sync message as it leaves the hardware, eliminating the need for a Follow_Up. This requires hardware capable of modifying the packet on the fly.

Delay Measurement

End-to-End (E2E) Delay Measurement

Used with Ordinary Clocks and Boundary Clocks. The slave measures delay to the master:

[Diagram: see online version]

Peer-to-Peer (P2P) Delay Measurement

Used with Transparent Clocks. Each link measures its own delay independently:

[Diagram: see online version]

P2P delay measurement works per-link, so each network hop knows its own delay. Transparent clocks update the correctionField as Sync messages pass through, accumulating total path delay.

Best Master Clock Algorithm (BMCA)

BMCA selects the Grandmaster by comparing Announce messages from all clocks. Clocks are compared using these fields, in priority order:

Priority	Field	Description
1	priority1	User-configurable (0-255, lower wins)
2	clockClass	Clock quality class (e.g., 6 = primary reference)
3	clockAccuracy	Expected accuracy (e.g., 0x21 = within 100 ns)
4	offsetScaledLogVariance	Clock stability metric
5	priority2	Tiebreaker (0-255, lower wins)

6	clockIdentity	Unique ID (lowest wins as final tiebreaker)

Clock Types

Type	Description
Ordinary Clock (OC)	Single-port clock; acts as master or slave
Boundary Clock (BC)	Multi-port clock; slave on one port, master on others; re-generates Sync messages
Transparent Clock (TC)	Forwards PTP messages and updates correctionField with residence time; does not participate in BMCA
Grandmaster (GM)	The root time source for the entire PTP domain; selected by BMCA

PTP Profiles

Profile	Standard	Domain	Key Characteristics
Default	IEEE 1588	General	Multicast, E2E or P2P, domain 0
Telecom (full)	ITU-T G.8275.1	Telecom	Layer 2 multicast, BC at every hop, sub-us accuracy
Telecom (partial)	ITU-T G.8275.2	Telecom	Unicast, IP transport, BCs not at every hop
Power	IEEE C37.238	Power grid	Layer 2, 1-second Sync rate, VLAN tagged
AES67 / SMPTE ST 2059	AES67	Broadcast media	Domain 0, PTP over IP multicast, sub-us for audio/video sync
Automotive	IEEE 802.1AS (gPTP)	In-vehicle	P2P only, Layer 2, simplified BMCA

PTP vs NTP

Feature	PTP (IEEE 1588)	NTP (RFC 5905)
Typical accuracy	Sub-microsecond	1-50 milliseconds
Best-case accuracy	< 1 ns (with hardware)	~100 us (LAN, SW timestamps)
Hardware timestamping	Yes (essential for best accuracy)	Optional (rarely used)
Network support needed	Boundary / Transparent clocks help	None (end-to-end)
Transport	UDP 319/320 or Layer 2 (0x88F7)	UDP port 123
Message rate	Configurable (1-128 per second)	Typically every 64-1024 seconds

Hierarchy	Grandmaster selected by BMCA	Stratum levels (manual config)
Use cases	Telecom, finance, broadcast, power	General-purpose server/desktop sync
Complexity	High (profiles, clock types, HW support)	Low (software-only)

Encapsulation

PTP can run over UDP/IPv4, UDP/IPv6, or directly over Ethernet:

[Diagram: see online version]

Multicast Addresses

Scope	IPv4	IPv6	Ethernet
Default domain (all)	224.0.1.129	ff0x::181	01:1B:19:00:00:00
Peer delay (P2P)	224.0.0.107	ff02::6B	01:80:C2:00:00:0E

Standards

Document	Title
IEEE 1588-2019	Precision Clock Synchronization Protocol (current edition)
IEEE 1588-2008	PTPv2 (widely deployed edition)
ITU-T G.8275.1	PTP Telecom Profile (full timing support)
ITU-T G.8275.2	PTP Telecom Profile (partial timing support)
IEEE C37.238	PTP Power Profile
IEEE 802.1AS	Generalized PTP (gPTP) for automotive/TSN

See Also

- NTP -- software-based time synchronization
- Ethernet -- PTP Layer 2 transport (EtherType 0x88F7)

RESTCONF

Standard: RFC 8040 | **Layer:** Application (Layer 7) | **Wireshark filter:** `http` or `http2`

RESTCONF is an HTTP-based protocol for accessing YANG-modeled data on network devices, providing a RESTful alternative to NETCONF. It maps YANG data nodes to HTTP resources identified by URIs, using standard HTTP

methods (GET, POST, PUT, PATCH, DELETE) for CRUD operations. Data is encoded as JSON (application/yang-data+json) or XML (application/yang-data+xml). RESTCONF runs over HTTPS and is designed for automation tools and web developers who are more familiar with REST APIs than NETCONF's XML-RPC model. It supports event notifications via Server-Sent Events (SSE).

Resource URI Structure

RESTCONF resources are accessed via a well-known root path:

```
https://{host}/.well-known/host-meta
  -&gt; returns the RESTCONF API root (typically /restconf)
```

[Diagram: see online version]

URI Patterns

URI Pattern	Description
`{+restconf}/data`	Root of the unified datastore
`{+restconf}/data/{path}`	Specific YANG data node
`{+restconf}/data/{module}:{container}`	Top-level container from a module
`{+restconf}/data/{module}:{list}={key}`	Specific list entry by key
`{+restconf}/data/{module}:{list}={key1},{key2}`	List entry with composite key
`{+restconf}/operations/{module}:{rpc}`	Invoke a YANG RPC or action
`{+restconf}/data/ietf-restconf-monitoring:restconf-state`	Server capabilities

Example URIs

URI	Description
`/restconf/data/ietf-interfaces:interfaces`	All interfaces
`/restconf/data/ietf-interfaces:interfaces/interface=eth0`	Interface eth0
`/restconf/data/ietf-interfaces:interfaces/interface=eth0/mtu`	MTU of eth0
`/restconf/operations/ietf-system:restart`	Restart the device

HTTP Method Mapping

HTTP Method	RESTCONF Operation	NETCONF Equivalent	Description
GET	Read	`<get>`, `<get-config>`	Retrieve data resource
POST	Create	`<edit-config>` (create)	Create a new data resource

PUT	Create or Replace	`<edit-config>` (replace)	Create or replace entire resource
PATCH	Merge	`<edit-config>` (merge)	Merge data into existing resource
DELETE	Delete	`<edit-config>` (delete)	Remove a data resource
POST (on operations)	RPC	`<rpc>`	Invoke a YANG-defined operation

Media Types

Media Type	Description
`application/yang-data+json`	JSON encoding per RFC 7951
`application/yang-data+xml`	XML encoding per RFC 7950
`application/yang-patch+json`	YANG Patch in JSON (RFC 8072)
`application/yang-patch+xml`	YANG Patch in XML (RFC 8072)

CRUD Operations

[Diagram: see online version]

Query Parameters

Parameter	Description	Example
`depth`	Limit subtree depth (1 = no children, unbounded = all)	`?depth=2`
`fields`	Select specific fields to return	`?fields=name;mtu`
`filter`	XPath filter expression (if supported)	`?filter=/if:interfaces/if:interface[name='eth0']`
`content`	config, nonconfig, or all	`?content=config`
`with-defaults`	report-all, trim, explicit, report-all-tagged	`?with-defaults=report-all`
`insert`	Where to insert in ordered list: first, last, before, after	`?insert=first`
`point`	Reference entry for before/after insert	`?point=/list=key3`

YANG Patch (RFC 8072)

YANG Patch enables ordered, batch edit operations in a single HTTP PATCH request:

[Diagram: see online version]

YANG Patch Edit Operations

Operation	Description
create	Create new resource; fail if it exists
delete	Remove resource; fail if it does not exist
insert	Insert into ordered list (with point/where)
merge	Merge with existing data (like PATCH)
move	Reorder entry in an ordered list
replace	Replace existing resource; fail if it does not exist
remove	Remove resource if it exists; no error if absent

Event Notifications (SSE)

RESTCONF delivers YANG notifications via Server-Sent Events (SSE) over HTTP:

[Diagram: see online version]

SSE Message Format

```
event: notification
data: {
data:   "ietf-restconf:notification": {
data:     "eventTime": "2025-03-21T10:15:30Z",
data:     "ietf-interfaces:link-down": {
data:       "if-name": "eth1"
data:     }
data:   }
data: }
```

HTTP Status Codes

Status Code	RESTCONF Meaning
200 OK	Successful GET, PATCH, or POST (RPC)
201 Created	Resource successfully created (POST)
204 No Content	Successful PUT, PATCH, or DELETE (no body)
400 Bad Request	Malformed request or invalid data
401 Unauthorized	Authentication required
403 Forbidden	Insufficient authorization (NACM)
404 Not Found	Resource does not exist
405 Method Not Allowed	HTTP method not supported on this resource
409 Conflict	Resource already exists (POST create) or data conflict

412 Precondition Failed	ETag/If-Match condition not met
415 Unsupported Media Type	Content-Type not supported

RESTCONF vs NETCONF vs gNMI

Feature	RESTCONF	NETCONF	gNMI
Transport	HTTPS (port 443)	SSH (port 830)	gRPC/HTTP2 (TLS)
Encoding	JSON or XML	XML	Protobuf, JSON_IETF
Data model	YANG	YANG	YANG (OpenConfig)
API style	RESTful (resource-oriented)	RPC (operation-oriented)	RPC (protobuf service)
Candidate datastore	No (typically)	Yes	No
Transactions	Per-request (YANG Patch for batch)	Full lock/commit	Per-Set atomic
Streaming telemetry	SSE (limited)	Notifications (RFC 5277)	Native bidirectional (Subscribe)
Tooling	curl, Postman, any HTTP client	ncclient, specialized tools	gNMIc, gRPC clients
Learning curve	Low (familiar REST/JSON)	Medium (XML/SSH)	Medium (gRPC/protobuf)
Bulk operations	YANG Patch	Native	Set with multiple operations
Adoption	Growing (REST-friendly)	Widespread	Growing (telemetry focus)

Encapsulation

[Diagram: see online version]

RESTCONF MUST use HTTPS. The API root is typically `/restconf` but is discovered via `/.well-known/host-meta`.

Standards

Document	Title
RFC 8040	RESTCONF Protocol
RFC 8072	YANG Patch Media Type
RFC 7951	JSON Encoding of Data Modeled with YANG
RFC 7950	The YANG 1.1 Data Modeling Language
RFC 8341	Network Configuration Access Control Model (NACM)
RFC 8340	YANG Tree Diagrams
RFC 8525	YANG Library

RFC 8040 Errata	RESTCONF Errata

See Also

- NETCONF -- SSH/XML-based YANG datastore management
- gNMI -- gRPC-based streaming telemetry and config
- SNMP -- legacy polling-based network monitoring

sFlow

Standard: RFC 3176 / sFlow v5 Specification | **Layer:** Application (Layer 7) | **Wireshark filter:** `sflow`

sFlow is a sampling-based network monitoring technology for high-speed switched and routed networks. It combines random packet sampling (1-in-N) with periodic interface counter polling to provide real-time, network-wide traffic visibility with minimal impact on device performance. sFlow agents embedded in switches and routers export sampled data as UDP datagrams to a central collector for analysis. Unlike flow-cache-based approaches (NetFlow/IPFIX), sFlow is stateless on the agent -- each sampled packet is immediately exported, enabling linear scalability even at 100 Gbps+ link speeds.

sFlow v5 Datagram

[Diagram: see online version]

Key Fields

Field	Size	Description
Version	32 bits	sFlow datagram version (5)
Agent Address Type	32 bits	1 = IPv4, 2 = IPv6
Agent Address	32 or 128 bits	IP address of the sFlow agent
Sub-Agent ID	32 bits	Identifies the sub-agent (e.g., line card) within the device
Sequence Number	32 bits	Per-agent datagram sequence number (detects lost datagrams)
System Uptime	32 bits	Milliseconds since agent last rebooted
Number of Samples	32 bits	Count of sample records in this datagram
Sample Records	Variable	Flow Samples and/or Counter Samples

Sample Types

Each sFlow datagram contains one or more sample records. The two primary types are:

Enterprise	Format	Sample Type	Description
0	1	Flow Sample	Sampled packet header and metadata
0	2	Counter Sample	Interface and system counters
0	3	Flow Sample (expanded)	Extended format with 32-bit source ID
0	4	Counter Sample (expanded)	Extended format with 32-bit source ID

Flow Sample

A Flow Sample is generated when a packet is randomly selected for sampling:

[Diagram: see online version]

Flow Sample Fields

Field	Size	Description
Sequence Number	32 bits	Per-source sample sequence (detects drops)
Source ID	32 bits	High 8 bits: type (0=ifIndex, 1=sMonitor, 2=VLAN); low 24 bits: index
Sampling Rate	32 bits	Current sampling rate (e.g., 1-in-4096)
Sample Pool	32 bits	Total packets observed since agent start
Drops	32 bits	Packets dropped due to resource limits
Input Interface	32 bits	SNMP ifIndex of ingress interface (0x3FFFFFFF = unknown)
Output Interface	32 bits	SNMP ifIndex of egress interface (0x3FFFFFFF = unknown, 0x80000000+ = multiple/broadcast)
Number of Flow Records	32 bits	Count of flow records following

Flow Record Types

Enterprise	Format	Type	Description
0	1	Raw Packet Header	Sampled packet header bytes (up to 128 bytes typically)

0	2	Ethernet Frame Data	Source/dest MAC, EtherType, frame length
0	3	IPv4 Data	Extracted IP header fields
0	4	IPv6 Data	Extracted IPv6 header fields
0	1001	Extended Switch	Source/dest VLAN and priority
0	1002	Extended Router	Next-hop, source/dest mask lengths
0	1003	Extended Gateway	BGP AS path, communities, local pref
0	1004	Extended User	Source/dest user ID (authentication)
0	1005	Extended URL	URL direction and content

Raw Packet Header Record

[Diagram: see online version]

Field	Size	Description
Header Protocol	32 bits	Protocol of the sampled header (1 = Ethernet)
Frame Length	32 bits	Original length of the sampled frame
Stripped Bytes	32 bits	Bytes stripped before sampling (e.g., preamble, FCS)
Header Length	32 bits	Number of header bytes included
Header Bytes	Variable	First N bytes of the packet (typically 128)

Counter Sample

Counter Samples are polled periodically (default every 20 seconds) and report cumulative interface statistics:

[Diagram: see online version]

Counter Record Types

Enterprise	Format	Type	Description
0	1	Generic Interface	Standard interface counters (ifInOctets, ifOutOctets, etc.)
0	2	Ethernet Interface	Ethernet-specific counters (collisions, CRC errors)
0	3	Token Ring	Token Ring counters
0	4	100 BaseVG	VG counters

0	5	VLAN	Per-VLAN counters
0	1001	Processor	CPU utilization, memory usage, total/free memory

Generic Interface Counters

Field	Size	Description
ifIndex	32 bits	SNMP interface index
ifType	32 bits	Interface type (6 = Ethernet, 53 = PPP, etc.)
ifSpeed	64 bits	Interface speed in bps
ifDirection	32 bits	0 = unknown, 1 = full-duplex, 2 = half-duplex
ifStatus	32 bits	Bit 0: ifAdminStatus, Bit 1: ifOperStatus
ifInOctets	64 bits	Input byte counter
ifInUcastPkts	32 bits	Input unicast packets
ifInMulticastPkts	32 bits	Input multicast packets
ifInBroadcastPkts	32 bits	Input broadcast packets
ifInDiscards	32 bits	Input discarded packets
ifInErrors	32 bits	Input error packets
ifInUnknownProtos	32 bits	Input unknown protocol packets
ifOutOctets	64 bits	Output byte counter
ifOutUcastPkts	32 bits	Output unicast packets
ifOutMulticastPkts	32 bits	Output multicast packets
ifOutBroadcastPkts	32 bits	Output broadcast packets
ifOutDiscards	32 bits	Output discarded packets
ifOutErrors	32 bits	Output error packets
ifPromiscuousMode	32 bits	Promiscuous mode (0 = false, 1 = true)

Sampling and Collection Flow

[Diagram: see online version]

sFlow vs NetFlow/IPFIX

Feature	sFlow	NetFlow v5/v9	IPFIX
Approach	Packet sampling + counters	Flow cache export	Flow cache export

Agent state	Stateless (no flow cache)	Stateful (maintains flow cache)	Stateful (maintains flow cache)
Sampling	Random 1-in-N packet	Deterministic or sampled	Deterministic or sampled
Data exported	Raw packet headers	Aggregated flow records	Aggregated flow records
Counter export	Built-in periodic polling	Separate (SNMP)	Separate (SNMP)
Layer 2 visibility	Yes (MAC, VLAN, etc.)	Limited (v9 can include)	Yes (templates)
Transport	UDP (port 6343)	UDP (port 2055 typical)	UDP/TCP/SCTP (port 4739)
Scalability	Linear (stateless)	CPU-bound (flow cache)	CPU-bound (flow cache)
Non-IP traffic	Yes (any EtherType)	No (IP only)	Mostly IP
Standard	RFC 3176 + sflow.org v5	Cisco proprietary	IETF RFC 7011
Latency	Near real-time	Depends on timeouts	Depends on timeouts

Configuration Example

Typical sFlow agent configuration on a switch:

Parameter	Typical Value	Description
Collector IP	10.0.0.100	sFlow collector address
Collector Port	6343	UDP destination port
Sampling Rate	1-in-4096	Packet sampling rate (adjust for link speed)
Polling Interval	20 seconds	Counter polling frequency
Header Size	128 bytes	Bytes of packet header to capture
Agent Address	Switch management IP	Source address in sFlow datagrams

Recommended Sampling Rates

Link Speed	Sampling Rate
100 Mbps	1-in-256
1 Gbps	1-in-1024 to 1-in-4096
10 Gbps	1-in-4096 to 1-in-8192
40 Gbps	1-in-8192 to 1-in-16384
100 Gbps	1-in-16384 to 1-in-65536

Encapsulation

[Diagram: see online version]

sFlow datagrams are sent over UDP. Port 6343 is IANA-assigned. The collector address and port are configurable on the agent.

Standards

Document	Title
RFC 3176	InMon Corporation's sFlow: A Method for Monitoring Traffic in Switched and Routed Networks (Informational)
sFlow v5	sFlow Version 5 specification (sflow.org)
sFlow.org	sFlow standards body and resources
IANA port 6343	sFlow UDP port assignment

See Also

- NetFlow/IPFIX -- flow-cache-based traffic monitoring
- SNMP -- complementary device and interface monitoring
- OTLP -- modern observability telemetry protocol

SMPTE Timecode -- SMPTE ST 12M / LTC / VITC

Standard: SMPTE ST 12-1:2014 | **Layer:** Physical / Application | **Wireshark filter:** N/A (analog/embedded signal; appears in ST 2110-40 ancillary data as `smpte_2110`)

SMPTE timecode is a standardized labeling system for uniquely identifying every frame of video or film with a timestamp in hours:minutes:seconds:frames format. It is used throughout broadcast television, film production, music recording, and live events to synchronize multiple pieces of equipment -- cameras, audio recorders, video editors, lighting consoles, and playout servers. The two primary forms are LTC (Linear Timecode), an audio-frequency signal carried on a dedicated track or cable, and VITC (Vertical Interval Timecode), embedded in the vertical blanking interval of analog video signals. Modern IP-based workflows carry timecode as ancillary data within SMPTE ST 2110-40 streams.

LTC Frame (80 bits)

LTC encodes one timecode value per video frame as an 80-bit word using Manchester bi-phase encoding on an audio-frequency carrier.

[Diagram: see online version]

The 16-bit sync word `0x3FFD` at the end of every frame allows receivers to identify frame boundaries and determine playback direction (the sync word reads differently when tape runs in reverse).

Key Fields

Field	Size	Description
Frame Units	4 bits	Ones digit of frame number (0-9)
Frame Tens	2 bits	Tens digit of frame number (0-2)
Seconds Units	4 bits	Ones digit of seconds (0-9)
Seconds Tens	3 bits	Tens digit of seconds (0-5)
Minutes Units	4 bits	Ones digit of minutes (0-9)
Minutes Tens	3 bits	Tens digit of minutes (0-5)
Hours Units	4 bits	Ones digit of hours (0-9)
Hours Tens	2 bits	Tens digit of hours (0-2)
Drop Frame Flag	1 bit	1 = drop-frame timecode (29.97 fps)
Color Frame Flag	1 bit	1 = locked to color framing sequence
Clock Flag	1 bit	1 = timecode is locked to real-time clock
Binary Group Flags	2 bits	Indicate how user bits should be interpreted
Bi-phase Correction	1 bit	Ensures each frame has even number of transitions
User Bits	32 bits (8x4)	32 bits of user-defined data (8 groups of 4 bits)
Sync Word	16 bits	Fixed pattern `0x3FFD` for frame synchronization

Frame Rates

Frame Rate	Standard	Region	Notes
24 fps	Film	Worldwide	Cinema standard
25 fps	PAL/SECAM	Europe, Asia, Africa	Broadcast standard for 50 Hz regions
29.97 fps	NTSC	Americas, Japan	Actual rate for NTSC color television
30 fps	NTSC (B&W)	Americas	Original black-and-white rate, used in audio/music

Drop-Frame Timecode

NTSC color video runs at 29.97 fps, not exactly 30 fps. Over time, non-drop timecode drifts from real wall-clock time (approximately 3.6 seconds per hour). Drop-frame timecode compensates by skipping frame numbers 00 and 01 at the start of each minute, except every 10th minute.

Minute	Frames Skipped	Example
:01 through :09	Skip frames :00 and :01	01:02:00;02 follows 01:01:59;29
:00, :10, :20, :30, :40, :50	No skip (count normally)	01:10:00;00 follows 01:09:59;29

Drop-frame timecode is denoted with semicolons (01:00:00;00) while non-drop uses colons (01:00:00:00).

VITC (Vertical Interval Timecode)

VITC is embedded in the vertical blanking interval (VBI) of analog video signals, typically on lines 14-16 and 16-18 of the video field.

Feature	LTC	VITC
Carrier	Audio signal on track/cable	Video signal in VBI lines
Encoding	Manchester bi-phase modulation	NRZ (Non-Return-to-Zero)
Bits per frame	80	90 (80 data + CRC + sync)
Readable at high speed	Yes (shuttle, fast-forward)	No (lost at high speeds)
Readable at pause/slow	No (signal too low frequency)	Yes (embedded in each frame)
Redundancy	One copy per frame	Two identical lines per field
Connectable via	Audio cable (XLR, BNC)	Part of video signal
Standards	SMPTE ST 12-1	SMPTE ST 12-2

In practice, professional video decks generate both LTC and VITC simultaneously to ensure timecode is readable at all transport speeds.

User Bits

Each LTC/VITC frame carries 32 bits (8 nibbles) of user-defined data interleaved with the timecode digits. Common uses:

Use	Description
Date code	Store recording date (day/month/year) in BCD
Reel number	Identify tape reel or media clip
Scene/take	Encode scene and take numbers for film
Character set	ASCII characters (requires multiple frames)
Custom flags	Production-specific metadata

The Binary Group Flags in the LTC frame indicate how user bits should be interpreted (unspecified, BCD date, page/line character set, etc.).

MIDI Timecode (MTC)

MTC carries SMPTE timecode over MIDI connections using quarter-frame messages, allowing music sequencers and DAWs to synchronize with video equipment.

MTC Message	MIDI Status	Description
Quarter Frame	0xF1	Sends one nibble of timecode per message (8 messages = 2 frames)
Full Frame	0xF0 (SysEx)	Sends complete timecode value in one message (used for locate)

Quarter-frame messages transmit two per frame, taking 2 full frames (8 messages) to send a complete timecode value. This creates a 2-frame offset that receivers must compensate for.

Timecode over IP

SMPTE ST 2110-40

In modern IP-based broadcast facilities, timecode is carried as ancillary data within SMPTE ST 2110-40 RTP streams. The timecode appears as a SMPTE ST 12-1 ancillary data packet embedded in the RTP payload alongside other metadata (closed captions, AFD, etc.).

[Diagram: see online version]

Timecode Display Format

```
HH:MM:SS:FF     (non-drop frame)
HH:MM:SS;FF     (drop-frame, semicolons)
```

Examples:

- `01:23:45:12` -- 1 hour, 23 minutes, 45 seconds, frame 12 (non-drop)
- `00:59:59;29` -- last frame before the 1-hour mark (29.97 drop-frame)
- `01:00:00;02` -- first displayable frame after the hour (frames 00 and 01 skipped at :00 because it is not a 10th minute)

Standards

Document	Title
SMPTE ST 12-1:2014	Time and Control Code (timecode definition)
SMPTE ST 12-2:2014	Time and Control Code -- Vertical Interval (VITC)
SMPTE ST 12-3:2016	Time and Control Code -- Transmission by Data Packets
SMPTE ST 2110-40	Ancillary Data over IP (carries timecode in IP workflows)
IEC 60461	Time and control code for video/audio (international equivalent)
MIDI 1.0 Specification	MIDI Timecode (MTC) quarter-frame messages

See Also

- PTP (IEEE 1588) -- precision clock synchronization used alongside timecode in IP facilities
- MIDI -- carries MIDI Timecode (MTC) for music/video sync
- DMX512 -- lighting protocol often synchronized via timecode

SNMP -- Simple Network Management Protocol

Standard: RFC 3416 | **Layer:** Application (Layer 7) | **Wireshark filter:** `snmp`

SNMP is the standard protocol for monitoring and managing network devices -- routers, switches, servers, printers, UPS units, and virtually any IP-connected equipment. It uses a manager-agent model where a management station polls agents running on devices for status information, and agents can send unsolicited traps/notifications when events occur. Data is organized in a hierarchical Management Information Base (MIB). SNMP is implemented in nearly every piece of network equipment ever manufactured.

Message (SNMPv2c)

SNMP messages are encoded in ASN.1 BER:

[Diagram: see online version]

Key Fields

Field	Encoding	Description
Version	INTEGER	SNMP version (0 = v1, 1 = v2c, 3 = v3)
Community	OCTET STRING	Community string (v1/v2c -- acts as password)
PDU	SEQUENCE	Protocol Data Unit -- the operation and data

PDU Structure

[Diagram: see online version]

Field	Description
Request ID	Matches requests to responses
Error Status	0 = noError, or an error code
Error Index	Points to the variable binding that caused the error

Variable Bindings	List of OID + value pairs

Operations (PDU Types)

Type	Tag	Name	Description
0	0xA0	GetRequest	Read one or more OID values
1	0xA1	GetNextRequest	Read the next OID in the tree (for walking)
2	0xA2	GetResponse	Agent's response to a request
3	0xA3	SetRequest	Write a value to an OID
4	0xA4	Trap (v1)	Unsolicited event notification
5	0xA5	GetBulkRequest (v2c+)	Efficient retrieval of large tables
6	0xA6	InformRequest (v2c+)	Acknowledged notification (manager-to-manager)
7	0xA7	SNMPv2-Trap (v2c+)	Unsolicited event notification (v2c format)

Request/Response Flow

[Diagram: see online version]

OID (Object Identifier)

OIDs form a global tree. Every manageable object has a unique dotted-number address:

```
iso.org.dod.internet.mgmt.mib-2.system.sysDescr.0
 1 . 3 . 6 . 1  . 2  . 1   . 1    . 1       .0
```

Common OIDs (MIB-2)

OID	Name	Description
1.3.6.1.2.1.1.1.0	sysDescr	Device description string
1.3.6.1.2.1.1.3.0	sysUpTime	Time since last reboot (hundredths of seconds)
1.3.6.1.2.1.1.5.0	sysName	Device hostname
1.3.6.1.2.1.2.2.1.2	ifDescr	Interface descriptions (table)
1.3.6.1.2.1.2.2.1.8	ifOperStatus	Interface operational status (1=up, 2=down)

1.3.6.1.2.1.2.2.1.10	ifInOctets	Inbound byte counter
1.3.6.1.2.1.2.2.1.16	ifOutOctets	Outbound byte counter
1.3.6.1.2.1.4.20.1.1	ipAdEntAddr	IP addresses on the device
1.3.6.1.2.1.25.1.1.0	hrSystemUptime	Host uptime

OID Tree Structure

[Diagram: see online version]

Error Codes

Code	Name	Description
0	noError	Success
1	tooBig	Response too large for transport
2	noSuchName	OID not found (v1)
3	badValue	Invalid value in SetRequest
5	genErr	General error
6	noAccess	OID exists but not writable
11	resourceUnavailable	Cannot allocate resource

SNMPv3 Security

SNMPv3 adds proper authentication and encryption (v1/v2c use plaintext community strings):

Security Level	Auth	Privacy	Description
noAuthNoPriv	No	No	Username only (like v2c)
authNoPriv	Yes	No	HMAC-MD5 or HMAC-SHA authentication
authPriv	Yes	Yes	Authentication + AES/DES encryption

SNMPv3 Message

[Diagram: see online version]

Version Comparison

Feature	v1	v2c	v3
Community strings	Yes	Yes	No (users)

GetBulk	No	Yes	Yes
Inform	No	Yes	Yes
64-bit counters	No	Yes	Yes
Authentication	Community (plaintext)	Community (plaintext)	HMAC-SHA/MD5
Encryption	None	None	AES/DES

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 3411	SNMPv3 Architecture
RFC 3412	SNMPv3 Message Processing
RFC 3414	SNMPv3 User-based Security Model (USM)
RFC 3416	SNMPv2 Protocol Operations
RFC 3417	SNMPv2 Transport Mappings
RFC 3418	MIB for SNMP (MIB-2 base)
RFC 1157	SNMPv1 (original)

See Also

- UDP
- RADIUS -- another protocol on every network device

Syslog

Standard: RFC 5424 | **Layer:** Application (Layer 7) | **Wireshark filter:** `syslog`

Syslog is the standard protocol for sending log messages from devices, servers, and applications to a centralized log collector. Virtually every network device, Unix/Linux server, firewall, and application can emit syslog messages. The protocol is intentionally simple -- a text message with a priority value, sent over UDP (or TCP/TLS for reliable delivery). Syslog is the foundation of centralized logging, SIEM systems, and operational monitoring.

Message Format (RFC 5424)

```
&lt;priority&gt;version timestamp hostname app-name procid msgid structured-data msg
```

Example:

```
&lt;165&gt;1 2026-03-21T14:30:00.123Z router1.example.com snmpd 2741 - - SNMP trap: linkDown on interface eth0
```

Header Fields

Field	Description	Example
Priority	`<facility x 8 + severity>` in angle brackets	`<165>` = local4.notice
Version	Syslog protocol version (1 for RFC 5424)	`1`
Timestamp	ISO 8601 with optional fractional seconds	`2026-03-21T14:30:00.123Z`
Hostname	Originating host (FQDN, IP, or hostname)	`router1.example.com`
App-Name	Application or process name	`snmpd`
ProcID	Process ID or instance identifier	`2741`
MsgID	Message type identifier	`-` (nil)
Structured-Data	Key-value metadata in brackets	`[exampleSDID@32473 iut="3"]`
Msg	Free-form log message text	`SNMP trap: linkDown...`

Priority (PRI)

The priority value encodes both facility and severity:

```
PRI = facility x 8 + severity
```

Facility Values

Code	Facility	Description
0	kern	Kernel messages
1	user	User-level messages
2	mail	Mail system
3	daemon	System daemons
4	auth	Security/authorization
5	syslog	Syslog daemon itself
6	lpr	Printer subsystem

7	news	Network news
8	uucp	UUCP subsystem
9	cron	Clock/cron daemon
10	authpriv	Security/authorization (private)
11	ftp	FTP daemon
16-23	local0-local7	Local use (configurable)

Severity Levels

Code	Severity	Keyword	Description
0	Emergency	emerg	System is unusable
1	Alert	alert	Immediate action required
2	Critical	crit	Critical conditions
3	Error	err	Error conditions
4	Warning	warning	Warning conditions
5	Notice	notice	Normal but significant
6	Informational	info	Informational messages
7	Debug	debug	Debug-level messages

Priority Examples

PRI	Facility	Severity	Meaning
`<0>`	kern (0)	emerg (0)	Kernel emergency
`<34>`	auth (4)	crit (2)	Critical auth failure
`<165>`	local4 (20)	notice (5)	Local4 notice
`<191>`	local7 (23)	debug (7)	Local7 debug

BSD Format (RFC 3164 -- Legacy)

The older, still widely used format:

```
&lt;PRI&gt;Mmm dd hh:mm:ss hostname tag: message
```

Example:

```
&lt;134&gt;Mar 21 14:30:00 webserver01 nginx: 192.168.1.100 - GET /api/health 200
```

Field	Description

PRI	Same as RFC 5424
Timestamp	`Mmm dd hh:mm:ss` (no year, no timezone!)
Hostname	Originating host
Tag	Application name + optional PID in brackets
Message	Free-form text

Structured Data (RFC 5424)

Structured data provides machine-parseable key-value pairs:

```
[timeQuality tzKnown="1" isSynced="1" syncAccuracy="60000"]
[origin ip="192.168.1.1" software="rsyslogd" swVersion="8.2312.0"]
```

Format: `[SD-ID param="value" param="value"]`

IANA-registered SD-IDs:

SD-ID	Description
timeQuality	Timestamp quality (synced, accuracy)
origin	Message origin (IP, software, version)
meta	Metadata (sequenceId, sysUpTime)

Transport

Transport	Port	Reliability	Standard
UDP	514	Unreliable (fire-and-forget)	RFC 5426
TCP	514 or 601	Reliable (connection-oriented)	RFC 6587
TLS over TCP	6514	Reliable + encrypted	RFC 5425
DTLS over UDP	6514	Encrypted datagram	RFC 6012

UDP vs TCP

Feature	UDP (traditional)	TCP/TLS (modern)
Delivery guarantee	None	Yes
Message framing	One message per datagram	Octet counting or newline delimited
Encryption	None	TLS (RFC 5425)
Backpressure	None (messages dropped)	TCP flow control
When to use	LAN, non-critical	WAN, compliance, security events

Common Implementations

Software	Role	Notes
rsyslog	Server/client	Most common on Linux; supports TCP, TLS, filtering, templates
syslog-ng	Server/client	Advanced parsing, routing, and transformation
systemd-journald	Client	Linux journal; can forward to syslog
Splunk	Collector/SIEM	Enterprise log analysis and search
Graylog	Collector	Open-source log management (GELF + syslog)
Fluentd/Fluent Bit	Collector	Cloud-native log forwarding
Logstash	Collector	Part of Elastic Stack (ELK)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 5424	The Syslog Protocol
RFC 3164	The BSD Syslog Protocol (legacy, still widely used)
RFC 5425	TLS Transport Mapping for Syslog
RFC 5426	Transmission of Syslog Messages over UDP
RFC 6587	Transmission of Syslog Messages over TCP

See Also

- UDP -- traditional syslog transport
- TCP -- reliable syslog transport
- TLS -- encrypted syslog (RFC 5425)
- SNMP -- complementary monitoring protocol (traps vs logs)
- OTLP -- modern observability telemetry (traces + metrics + logs)

TR-069 (CWMP) / TR-369 (USP)

Standard: Broadband Forum TR-069 | **Layer:** Application (Layer 7) | **Wireshark filter:** `http` (TR-069 is SOAP/HTTP)

TR-069 (CPE WAN Management Protocol / CWMP) is how ISPs remotely manage customer premises equipment -- routers, ONTs, set-top boxes, and gateways. It uses SOAP over HTTP/HTTPS between the CPE and an ACS (Auto Configuration Server), enabling remote configuration, firmware updates, diagnostics, and monitoring without truck rolls. TR-369 (User Services Platform / USP) is its modern replacement, using Protocol Buffers over MQTT, WebSocket, or other transports. Both use the same TR-181 data model, a hierarchical parameter tree describing every configurable aspect of the device.

TR-069 (CWMP)

Session Architecture

[Diagram: see online version]

Component	Role
CPE	Customer Premises Equipment -- runs a TR-069 agent
ACS	Auto Configuration Server -- ISP's management platform
Connection Request	ACS-initiated HTTP GET to CPE (port 7547) triggers CPE to connect back

Session Flow

[Diagram: see online version]

Inform Message

The Inform is the cornerstone of TR-069 -- every session begins with one. The CPE sends an Inform to the ACS containing:

Inform Field	Description
DeviceId	Manufacturer, OUI, ProductClass, SerialNumber
Event	List of event codes that triggered the session
MaxEnvelopes	Max SOAP envelopes per HTTP message
CurrentTime	CPE's current time
RetryCount	Number of retries for this session attempt
ParameterList	Key parameter values (forced + notification-triggered)

Event Codes

Code	Name	Trigger

0 BOOTSTRAP	First boot	CPE contacts ACS for the first time (factory reset or new device)
1 BOOT	Reboot	CPE has rebooted
2 PERIODIC	Timer	Periodic inform interval expired
4 VALUE CHANGE	Parameter changed	A parameter with active notification changed value
6 CONNECTION REQUEST	ACS-initiated	ACS sent a Connection Request to port 7547
7 TRANSFER COMPLETE	Download/upload done	File transfer completed
8 DIAGNOSTICS COMPLETE	Diag done	A diagnostic test (ping, traceroute, speed test) finished
M Download	Manufacturer	Download requested by ACS
M Reboot	Manufacturer	Reboot requested by ACS

RPC Methods

Method	Direction	Description
Inform	CPE -> ACS	Session initiation with device identity and events
GetParameterValues	ACS -> CPE	Read one or more parameter values
SetParameterValues	ACS -> CPE	Write one or more parameter values
GetParameterNames	ACS -> CPE	Discover available parameters (walk the tree)
AddObject	ACS -> CPE	Create a new instance (e.g., new port forwarding rule)
DeleteObject	ACS -> CPE	Delete an instance
Download	ACS -> CPE	Instruct CPE to download a file (firmware, config)
Upload	ACS -> CPE	Instruct CPE to upload a file (config backup, logs)
Reboot	ACS -> CPE	Reboot the CPE
FactoryReset	ACS -> CPE	Restore factory defaults
GetRPCMethods	Both	Discover supported RPC methods
ScheduleInform	ACS -> CPE	Schedule a future Inform
SetVouchers	ACS -> CPE	Install vouchers (rare)

Data Model (TR-181)

TR-069 uses a hierarchical dot-separated parameter tree. TR-181 (Device:2) is the current standard data model; TR-098 (InternetGatewayDevice:1) is the legacy model.

Parameter Path	Type	Description
Device.DeviceInfo.Manufacturer	string	Device manufacturer name
Device.DeviceInfo.SoftwareVersion	string	Current firmware version
Device.WiFi.Radio.{i}.Channel	uint	WiFi channel number
Device.WiFi.Radio.{i}.Enable	bool	WiFi radio enabled
Device.WiFi.SSID.{i}.SSID	string	WiFi network name
Device.IP.Interface.{i}.IPv4Address.{i}.IP Address	string	IPv4 address on an interface
Device.NAT.PortMapping.{i}.ExternalPort	uint	Port forwarding external port
Device.DNS.Client.Server.{i}.DNSServer	string	DNS server address
Device.ManagementServer.URL	string	ACS URL
Device.ManagementServer.PeriodicInformInterval	uint	Inform interval in seconds

`{i}` indicates a multi-instance object (table row). AddObject creates new instances; DeleteObject removes them.

Connection Request

When the ACS needs to reach the CPE outside of a scheduled session:

1. ACS sends **HTTP GET** to the CPE's Connection Request URL (typically `http://<CPE-IP>:7547/`)
2. CPE authenticates the request (HTTP Digest)
3. CPE initiates a new CWMP session to the ACS (Inform with event 6 CONNECTION REQUEST)

Parameter	Description
Port	7547 (default CPE listening port)
Authentication	HTTP Digest (username/password)
NAT traversal	Problematic -- CPE behind NAT may not be reachable; STUN or periodic Inform used as workarounds

TR-069 Security

Mechanism	Description
TLS/HTTPS	CPE connects to ACS over HTTPS
HTTP Digest	Authentication for both ACS URL access and Connection Requests
Client certificates	Optional mutual TLS authentication
ACS URL	Typically provisioned during manufacturing or via DHCP option 43

TR-369 (USP -- User Services Platform)

TR-369 is the modern replacement for TR-069, designed for IoT-scale management with better transport flexibility and security:

Architecture

[Diagram: see online version]

TR-069 Term	TR-369 Term
CPE	Agent
ACS	Controller
SOAP/XML	Protocol Buffers (protobuf)
HTTP/HTTPS	MQTT, WebSocket, STOMP, CoAP
Session-based	Message-based (no session)

USP Record (Protobuf)

Every USP exchange is wrapped in a USP Record:

[Diagram: see online version]

Field	Description
version	USP protocol version
to_id	Destination endpoint ID (e.g., `os::012345-ABCdef`)
from_id	Source endpoint ID
payload_security	0 = PLAINTEXT, 1 = TLS12
mac_signature	HMAC or digital signature over the record
sender_cert	X.509 certificate of the sender
payload	Serialized USP Message (protobuf)

USP Message Flow

[Diagram: see online version]

USP Message Types

Message	Direction	Description
Get	Controller -> Agent	Read parameter values
GetResp	Agent -> Controller	Parameter values response
Set	Controller -> Agent	Write parameter values

SetResp	Agent -> Controller	Write confirmation
Add	Controller -> Agent	Create object instance
Delete	Controller -> Agent	Delete object instance
Operate	Controller -> Agent	Execute a command (ping, traceroute, firmware update)
Notify	Agent -> Controller	Event notification (value change, boot, operation complete)
GetSupportedDM	Controller -> Agent	Discover supported data model
GetInstances	Controller -> Agent	List object instances
GetSupportedProtocol	Both	Protocol version negotiation

USP Transports

Transport	Protocol	Port	Use Case
MQTT	MQTT 3.1.1 / 5.0	8883 (TLS)	Primary -- scalable pub/sub for large deployments
WebSocket	RFC 6455	443	NAT-friendly, browser-compatible
STOMP	STOMP 1.2	61613	Enterprise messaging integration
CoAP	RFC 7252	5684 (DTLS)	Constrained IoT devices

USP Advantages over CWMP

Feature	TR-069 (CWMP)	TR-369 (USP)
Encoding	SOAP/XML (verbose)	Protocol Buffers (compact)
Transport	HTTP/HTTPS only	MQTT, WebSocket, STOMP, CoAP
Model	Session-based (CPE connects to ACS)	Message-based (bidirectional, event-driven)
NAT traversal	Problematic (Connection Request)	Solved (MQTT broker, WebSocket)
Scalability	Moderate (HTTP sessions)	High (MQTT pub/sub)
Security	TLS + HTTP Digest	TLS + protobuf signatures + E2E encryption
Data model	TR-181 (Device:2)	TR-181 (same -- full compatibility)
IoT support	Limited	Designed for IoT (lightweight, async)
Multi-controller	No (single ACS)	Yes (multiple controllers with permissions)
Commands	RPC methods	Operate (extensible command framework)

TR-069 vs TR-369 Comparison

Aspect	TR-069 (CWMP)	TR-369 (USP)
Year	2004	2018
Encoding	SOAP 1.1 / XML	Protocol Buffers
Transport	HTTP/HTTPS	MQTT, WebSocket, STOMP, CoAP
Session model	Session (Inform starts, empty POST ends)	Stateless messages
Data model	TR-181 Device:2 / TR-098 IGD:1	TR-181 Device:2
CPE port	7547 (Connection Request)	N/A (broker-mediated)
Authentication	HTTP Digest, TLS certs	TLS, X.509, protobuf signatures
Firmware mgmt	Download RPC	Operate command
Bulk data	TR-157 Bulk Data Collection	Native Notify + Get
Deployment	Ubiquitous (billions of devices)	Growing (new deployments)

Standards

Document	Title
TR-069	CPE WAN Management Protocol (CWMP)
TR-369	User Services Platform (USP)
TR-181	Device:2 Data Model
TR-098	InternetGatewayDevice:1 Data Model (legacy)
TR-104	VoIP data model for TR-069
TR-135	STB data model for TR-069
TR-157	Bulk Data Collection for TR-069
TR-106	Data Model Template (CWMP/USP)

See Also

- GPON -- ONTs managed via TR-069/OMCI
- EPON -- ONUs managed via TR-069/OAM
- SNMP -- alternative network management protocol
- MQTT -- primary transport for USP
- HTTP -- transport for CWMP (SOAP over HTTP)

Naming, Addressing & Discovery

Name resolution, addressing, and service discovery protocols.

12 protocols in this section

BOOTP / PXE -- Bootstrap Protocol / Preboot Execution Environment

Standard: RFC 951 | **Layer:** Application (Layer 7) | **Wireshark filter:** `bootp` or `dhcp`

BOOTP is the original protocol for automatically assigning IP addresses and boot information to diskless workstations. It was designed to replace RARP with a richer bootstrap mechanism, allowing a client to discover its IP address, a TFTP server address, and a boot filename in a single exchange. DHCP (RFC 2131) evolved from BOOTP, extending it with address leasing, additional options, and dynamic allocation while maintaining backward-compatible message format. PXE (Preboot Execution Environment) builds on DHCP/BOOTP to provide a standardized method for booting computers over the network, downloading a Network Bootstrap Program (NBP) via TFTP or HTTP.

BOOTP / DHCP Message

The BOOTP and DHCP message formats are identical in their fixed fields. DHCP extends BOOTP through the options field.

[Diagram: see online version]

The fixed header is 236 bytes. In BOOTP, the final field is a 64-byte vendor-specific area. In DHCP, the options field begins with magic cookie `0x63825363` and is variable-length.

Key Fields

Field	Size	Description
Op	8 bits	Message type: 1 = BOOTREQUEST (client), 2 = BOOTREPLY (server)
HType	8 bits	Hardware address type (1 = Ethernet)
HLen	8 bits	Hardware address length (6 for Ethernet MAC)
Hops	8 bits	Relay agent hop count, incremented by each relay
XID	32 bits	Transaction ID chosen by client to match requests and replies
Secs	16 bits	Seconds elapsed since client began boot process
Flags	16 bits	Bit 0 = Broadcast flag; bits 1-15 reserved
ciaddr	32 bits	Client's current IP address (set if client is bound)
yiaddr	32 bits	"Your" IP address assigned by the server

siaddr	32 bits	Next server IP address (TFTP server for boot file)
giaddr	32 bits	Relay agent IP address
chaddr	16 bytes	Client hardware address (MAC), zero-padded
sname	64 bytes	Server hostname (null-terminated string, optional)
file	128 bytes	Boot filename (null-terminated string, optional)
vend/options	64+ bytes	Vendor-specific area (BOOTP) or DHCP options

BOOTP vs DHCP

Feature	BOOTP	DHCP
Address allocation	Static (admin-configured table)	Dynamic, automatic, or manual
Lease concept	No (permanent assignment)	Yes (time-limited leases)
Options field	64 bytes vendor-specific	Variable-length, extensible
Message types	Request and Reply only	Discover, Offer, Request, Ack, Nak, Release, Inform, Decline
Client state machine	Simple one-exchange	Multi-step DORA process
RFC	RFC 951	RFC 2131
Backward compatible	--	Yes, uses same message format

PXE Boot Process

Classic PXE Boot (DHCP + TFTP)

[Diagram: see online version]

ProxyDHCP Flow (Separate Boot Server)

When the DHCP server cannot be modified to include PXE options, a ProxyDHCP server provides boot information on a separate port.

[Diagram: see online version]

PXE DHCP Options

Option	Name	Description

60	Vendor Class Identifier	Client sends `PXEClient:Arch:xxxxx:UNDI:yyyzzz`
66	TFTP Server Name	Hostname or IP of the TFTP boot server
67	Bootfile Name	Path to the Network Bootstrap Program (e.g., `pxelinux.0`)
97	Client Machine Identifier (UUID)	16-byte GUID/UUID of the client system
43	Vendor-Specific Information	PXE-specific sub-options (boot menu, discovery control)

Client System Architecture (Option 60 / RFC 4578)

The architecture type in the vendor class identifier tells the server what kind of bootloader to serve.

Type	Architecture
0	Intel x86 BIOS
6	EFI IA32
7	EFI x86-64 (UEFI)
9	EFI x86-64 (UEFI, alternate)
10	EFI ARM 32-bit
11	EFI ARM 64-bit (AARCH64)

UEFI HTTP Boot

Modern UEFI firmware supports HTTP Boot as an alternative to TFTP, offering significant performance improvements for large boot images.

Feature	TFTP (Classic PXE)	HTTP Boot (UEFI)
Transport	UDP, stop-and-wait	TCP, streaming
Speed	Slow (512-byte blocks default)	Fast (full TCP throughput)
Encryption	None	HTTPS supported
Firmware support	BIOS and UEFI	UEFI only
DHCP option	Option 67 (filename)	Option 59 (Boot File URL)
Example path	`pxelinux.0`	`http://boot.example.com/boot.efi`

iPXE Extensions

iPXE is an open-source PXE implementation that extends the standard PXE environment with additional capabilities. It can be chain-loaded from a standard PXE ROM.

Feature	Standard PXE	iPXE
Download protocols	TFTP only	TFTP, HTTP, HTTPS, iSCSI, FCoE, AoE
Scripting	None	Built-in script engine
Boot from SAN	No	iSCSI, FCoE, AoE
DNS support	No	Yes
Wireless boot	No	Wi-Fi supported
VLAN tagging	No	802.1Q support
Embedding	ROM only	ROM, USB, ISO, chain-load

iPXE Chain-Loading Flow

[Diagram: see online version]

Encapsulation

[Diagram: see online version]

BOOTP and PXE use the same ports as DHCP: client sends from UDP port 68 to server port 67. ProxyDHCP uses port 4011 for unicast boot service queries.

Standards

Document	Title
RFC 951	Bootstrap Protocol (BOOTP)
RFC 1497	BOOTP Vendor Information Extensions
RFC 1542	Clarifications and Extensions for BOOTP
RFC 2131	Dynamic Host Configuration Protocol (DHCP)
RFC 4578	DHCP Options for PXE (Client Architecture)
RFC 5970	DHCPv6 Options for Network Boot
Intel PXE Specification v2.1	Preboot Execution Environment Specification
RFC 7440	TFTP Windowsize Option (improves PXE transfer speed)

See Also

- DHCP -- BOOTP's successor, uses the same message format
- TFTP -- default file transfer protocol for PXE boot
- HTTP -- used by UEFI HTTP Boot and iPXE
- Ethernet -- underlying link layer for PXE

DHCP -- Dynamic Host Configuration Protocol

Standard: RFC 2131 | **Layer:** Application (Layer 7) | **Wireshark filter:** `dhcp`

DHCP automatically assigns IP addresses and network configuration (subnet mask, default gateway, DNS servers) to devices joining a network. It follows a client-server model where the client broadcasts a discovery request and the server responds with an address lease. DHCP replaced the older BOOTP protocol and is backward-compatible with it. DHCPv6 (RFC 8415) serves a similar role for IPv6 networks.

Message

[Diagram: see online version]

The full message is 236 bytes of fixed fields followed by a variable-length Options field (starting with the magic cookie `0x63825363`).

Key Fields

Field	Size	Description
Op	8 bits	Message type: 1 = BOOTREQUEST (client), 2 = BOOTREPLY (server)
HType	8 bits	Hardware address type (1 = Ethernet)
HLen	8 bits	Hardware address length (6 for Ethernet MAC)
Hops	8 bits	Relay agent hop count
XID	32 bits	Transaction ID chosen by client, used to match replies
Secs	16 bits	Seconds elapsed since client began address acquisition
Flags	16 bits	Bit 0 = Broadcast flag; bits 1-15 reserved
ciaddr	32 bits	Client's current IP (if bound/renewing)
yiaddr	32 bits	"Your" IP -- address offered/assigned to client
siaddr	32 bits	Next server IP (for TFTP boot)
giaddr	32 bits	Relay agent IP address
chaddr	16 bytes	Client hardware address (MAC), zero-padded
sname	64 bytes	Server hostname (optional)

file	128 bytes	Boot filename (optional)
Options	Variable	DHCP options, starting with magic cookie `0x63825363`

Field Details

DORA Process (Discover, Offer, Request, Acknowledge)

[Diagram: see online version]

Message Types (Option 53)

Value	Type	Description
1	DHCPDISCOVER	Client looking for servers
2	DHCPOFFER	Server offers an address
3	DHCPREQUEST	Client requests the offered address
4	DHCPDECLINE	Client rejects the address (conflict detected)
5	DHCPACK	Server confirms the lease
6	DHCPNAK	Server rejects the request
7	DHCPRELEASE	Client releases the address
8	DHCPINFORM	Client requests config without address assignment

Common Options

Option	Name	Description
1	Subnet Mask	Client's subnet mask
3	Router	Default gateway(s)
6	DNS Servers	DNS server address(es)
12	Hostname	Client's hostname
15	Domain Name	Client's domain suffix
28	Broadcast Address	Subnet broadcast address
42	NTP Servers	Network time server(s)
51	Lease Time	Address lease duration in seconds
53	Message Type	DHCP message type (see table above)
54	Server Identifier	IP of the DHCP server
58	Renewal (T1) Time	Seconds until client should renew

59	Rebinding (T2) Time	Seconds until client should rebind
61	Client Identifier	Unique client ID (often MAC)
255	End	Marks end of options

Encapsulation

[Diagram: see online version]

Client sends from port 68 to server port 67. Server replies from port 67 to client port 68.

Standards

Document	Title
RFC 2131	Dynamic Host Configuration Protocol
RFC 2132	DHCP Options and BOOTP Vendor Extensions
RFC 4361	Node-specific Client Identifiers for DHCP
RFC 8415	DHCPv6 (Dynamic Host Configuration Protocol for IPv6)
RFC 951	BOOTP -- predecessor protocol

See Also

- UDP
- IPv4
- DNS -- DHCP typically assigns DNS server addresses
- ARP -- used after DHCP to verify address availability

DHCP Options Reference

Standard: RFC 2132 | **Layer:** Application (Layer 7) | **Wireshark filter:** `dhcp.option`

DHCP options carry configuration parameters and control information between DHCP clients and servers. Options are appended to the fixed-field DHCP message (after the magic cookie `0x63825363`) using a Type-Length-Value (TLV) encoding. The DHCP options mechanism was inherited from BOOTP vendor extensions and greatly expanded. This document is a reference for the most commonly encountered DHCP options in production networks, including those critical for network booting, relay agents, and vendor-specific provisioning.

Option TLV Format

Each option is encoded as a single-byte code, a single-byte length, and a variable-length value:

[Diagram: see online version]

Field	Size	Description
Code	1 byte	Option type identifier (0-255)
Length	1 byte	Length of the value field in bytes (not present for options 0 and 255)
Value	Variable	Option data, format depends on the option code

Options 0 (Pad) and 255 (End) are single-byte options with no length or value fields.

Options by Function: Network Configuration

These options configure the client's IP stack:

Option	Name	Len	Format	Description
1	Subnet Mask	4	IP address	Client's subnet mask (e.g., `255.255.255.0`)
3	Router	4n	IP list	Default gateway(s), in preference order
6	Domain Name Server	4n	IP list	DNS server(s), in preference order
15	Domain Name	Variable	String	Client's DNS domain suffix (e.g., `example.com`)
28	Broadcast Address	4	IP address	Subnet broadcast address
33	Static Route	8n	IP pairs	Destination/router pairs (obsoleted by Option 121)
42	NTP Servers	4n	IP list	Network Time Protocol server(s)
119	Domain Search List	Variable	DNS encoded	DNS search list (compressed domain names per RFC 1035)
121	Classless Static Routes	Variable	CIDR + IP	Classless routes: subnet/mask-length + next-hop (RFC 3442)

Options by Function: Lease and Timing

These options control address lease duration and renewal timing:

Option	Name	Len	Format	Description
51	IP Address Lease Time	4	uint32	Lease duration in seconds
58	Renewal (T1) Time	4	uint32	Seconds until client enters RENEWING state (default: 50% of lease)
59	Rebinding (T2) Time	4	uint32	Seconds until client enters REBINDING state (default: 87.5% of lease)

Options by Function: DHCP Protocol

These options are used by the DHCP protocol itself:

Option	Name	Len	Format	Description
53	DHCP Message Type	1	uint8	1=Discover, 2=Offer, 3=Request, 4=Decline, 5=ACK, 6=NAK, 7=Release, 8=Inform
54	Server Identifier	4	IP address	IP address of the DHCP server
55	Parameter Request List	Variable	uint8 list	List of option codes the client is requesting
57	Maximum DHCP Message Size	2	uint16	Max DHCP message size the client will accept
61	Client Identifier	Variable	type + ID	Unique client ID: first byte is hardware type, rest is identifier (often MAC)

Options by Function: Client Identification

These options identify the client to the server:

Option	Name	Len	Format	Description
12	Hostname	Variable	String	Client's hostname (sent by client)
60	Vendor Class Identifier	Variable	String	Identifies client type/vendor (e.g., `MSFT 5.0`, `PXEClient:Arch:00000`)
61	Client Identifier	Variable	type + ID	Unique client identifier (see above)

77	User Class	Variable	String	User-defined class (for policy assignment)
97	Client Machine Identifier (UUID/GUID)	17	type + UUID	Client UUID (type byte 0 + 16-byte UUID; used in PXE environments)

Options by Function: Boot and Provisioning

These options support network booting (PXE/BOOTP) and device provisioning:

Option	Name	Len	Format	Description
66	TFTP Server Name	Variable	String	TFTP server hostname or IP for boot file download
67	Bootfile Name	Variable	String	Boot filename on the TFTP server
150	TFTP Server Address	4n	IP list	TFTP server IP(s) -- Cisco extension for IP phones
252	WPAD (Web Proxy Auto-Discovery)	Variable	String	URL to proxy auto-config (PAC) file (e.g., `http://wpad.example.com/wpad.dat`)

Options by Function: Vendor-Specific

Option	Name	Len	Format	Description
43	Vendor-Specific Information	Variable	TLV-encoded	Vendor-defined sub-options; format depends on Option 60 value

Option 43 carries vendor-specific sub-options encoded as nested TLV (code-length-value) within the value field. The interpretation depends on the Vendor Class (Option 60). Common uses include Wi-Fi controller discovery, IP phone provisioning, and thin-client configuration.

Option 82: Relay Agent Information

Option 82 (RFC 3046) is inserted by DHCP relay agents to identify the network location of the client. It contains sub-options:

[Diagram: see online version]

Option 82 Sub-options

Sub-option	Name	Description
1	Circuit ID	Identifies the physical port/VLAN where the client is connected (e.g., switch port, DSLAM slot)
2	Remote ID	Identifies the relay agent itself (e.g., switch MAC address, hostname)
5	Link Selection	Subnet the relay agent wants the server to allocate from
6	Subscriber ID	ISP subscriber identifier
9	Vendor-Specific Information	Vendor sub-options within Option 82
11	Server Identifier Override	Override server ID for relay scenarios
151	Virtual Subnet Selection	Select address pool by virtual subnet

Option 82 is widely used by ISPs, managed switches, and wireless controllers to assign IP addresses based on physical location rather than just client identity.

Option 121: Classless Static Routes

Option 121 (RFC 3442) provides classless static routes, replacing the obsolete Option 33. Each route is encoded as:

Field	Size	Description
Subnet mask width	1 byte	Number of significant bits (0-32)
Subnet	0-4 bytes	Significant octets of the destination subnet (only enough bytes to cover mask width)
Router	4 bytes	Next-hop IP address

A mask width of 0 indicates a default route. When Option 121 is present, the client must ignore Option 3 (Router) per the RFC.

Special Options

Option	Name	Description
0	Pad	Single byte (0x00), no length/value -- used for alignment
255	End	Single byte (0xFF), no length/value -- marks end of options
52	Option Overload	Indicates that sname/file fields carry additional options (1=file, 2=sname, 3=both)

Complete Quick Reference

Code	Name	RFC
1	Subnet Mask	2132
3	Router	2132
6	Domain Name Server	2132
12	Hostname	2132
15	Domain Name	2132
28	Broadcast Address	2132
42	NTP Servers	2132
43	Vendor-Specific Information	2132
51	IP Address Lease Time	2132
53	DHCP Message Type	2132
54	Server Identifier	2132
55	Parameter Request List	2132
57	Maximum DHCP Message Size	2132
58	Renewal (T1) Time	2132
59	Rebinding (T2) Time	2132
60	Vendor Class Identifier	2132
61	Client Identifier	2132
66	TFTP Server Name	2132
67	Bootfile Name	2132
77	User Class	3004
82	Relay Agent Information	3046
97	Client Machine Identifier (UUID)	4578
119	Domain Search List	3397
121	Classless Static Routes	3442
150	TFTP Server Address (Cisco)	--
252	WPAD	--
255	End	2132

Standards

Document	Title
RFC 2132	DHCP Options and BOOTP Vendor Extensions
RFC 3046	DHCP Relay Agent Information Option (Option 82)
RFC 3442	Classless Static Route Option (Option 121)

RFC 3004	User Class Option (Option 77)
RFC 3397	Domain Search Option (Option 119)
RFC 4578	DHCP PXE Options (Options 93, 94, 97)
RFC 2131	Dynamic Host Configuration Protocol

See Also

- DHCP -- the protocol that carries these options
- DHCPv6 -- IPv6 DHCP uses a different option format (16-bit code + 16-bit length)
- DNS -- Options 6, 15, and 119 configure DNS behavior

DHCPv6 -- Dynamic Host Configuration Protocol for IPv6

Standard: RFC 8415 | **Layer:** Application (Layer 7) | **Wireshark filter:** `dhcpv6`

DHCPv6 assigns IPv6 addresses and network configuration to hosts, similar to DHCPv4 but redesigned for IPv6. It operates alongside SLAAC (Stateless Address Autoconfiguration) -- Router Advertisements indicate whether to use DHCPv6 via the M (Managed) and O (Other Config) flags. DHCPv6 uses UDP on ports 546 (client) and 547 (server), with multicast for server discovery. It provides both stateful address assignment (IA_NA, IA_PD) and stateless configuration (DNS servers, NTP, domain search list).

Message Format

[Diagram: see online version]

Message Types

Type	Name	Direction	Description
1	SOLICIT	Client -> Server	Discover DHCPv6 servers
2	ADVERTISE	Server -> Client	Server offers configuration
3	REQUEST	Client -> Server	Client requests offered config
4	CONFIRM	Client -> Server	Verify address is still valid (after link change)
5	RENEW	Client -> Server	Extend lease (to the same server)
6	REBIND	Client -> Server	Extend lease (to any server)
7	REPLY	Server -> Client	Server response to any client message

8	RELEASE	Client -> Server	Release assigned address
9	DECLINE	Client -> Server	Address conflict detected
10	RECONFIGURE	Server -> Client	Server triggers client reconfiguration
11	INFORMATION-REQUEST	Client -> Server	Request config without address (stateless)
12	RELAY-FORW	Relay -> Server	Relay-forwarded message
13	RELAY-REPL	Server -> Relay	Server reply to relay

SARR Exchange (Solicit, Advertise, Request, Reply)

[Diagram: see online version]

For stateless DHCPv6 (O flag = 1, M flag = 0):

[Diagram: see online version]

Option Format

[Diagram: see online version]

Common Options

Code	Name	Description
1	Client Identifier (DUID)	Unique client identity
2	Server Identifier (DUID)	Unique server identity
3	IA_NA (Non-Temporary Address)	Request/assign a permanent IPv6 address
5	IA Address	An IPv6 address within an IA_NA
6	Option Request Option (ORO)	List of requested option codes
7	Preference	Server preference (0-255, higher wins)
8	Elapsed Time	Milliseconds since client began exchange
13	Status Code	Success/failure status
23	DNS Recursive Name Server	DNS server addresses
24	Domain Search List	DNS search suffixes
25	IA_PD (Prefix Delegation)	Request a delegated prefix
26	IA Prefix	A delegated prefix within IA_PD
31	SNTP Servers	SNTP/NTP server addresses
56	NTP Server	NTP server (RFC 5908)

82	SOL_MAX_RT	Maximum Solicit timeout

Status Codes (Option 13)

Code	Name	Description
0	Success	Operation successful
1	UnspecFail	Failure, no specific reason
2	NoAddrsAvail	No addresses available
3	NoBinding	Binding not found (stale request)
4	NotOnLink	Prefix not appropriate for the link
5	UseMulticast	Client must use multicast
6	NoPrefixAvail	No prefixes available for delegation

DUID (DHCP Unique Identifier)

Each client and server has a DUID for identification:

Type	Name	Content
1	DUID-LLT	Link-layer address + time
2	DUID-EN	Enterprise number + identifier
3	DUID-LL	Link-layer address only
4	DUID-UUID	UUID (RFC 6355)

Prefix Delegation (IA_PD)

DHCPv6-PD allows a router (e.g., home gateway) to request an entire prefix from the ISP:

[Diagram: see online version]

DHCPv6 vs DHCPv4

Feature	DHCPv4	DHCPv6
Discovery	Broadcast (255.255.255.255)	Multicast (ff02::1:2)
Ports	Client 68, Server 67	Client 546, Server 547
Client ID	MAC or Client-ID option	DUID (mandatory)
Address assignment	Yes (only method)	Optional (SLAAC can also assign)
Prefix delegation	No	Yes (IA_PD)

Relay	GIADDR field in header	Dedicated RELAY-FORW/RELAY-REPL messages
Message format	Fixed header + options	Minimal header + all options

Interaction with SLAAC

RA Flags	Addressing	Other Config	Method
M=0, O=0	SLAAC	SLAAC (RDNSS)	Fully stateless
M=0, O=1	SLAAC	DHCPv6 (stateless)	Common: SLAAC for address, DHCPv6 for DNS
M=1, O=1	DHCPv6 (stateful)	DHCPv6	Full DHCPv6 management

Multicast Addresses

Address	Scope	Purpose
ff02::1:2	Link-local	All DHCPv6 relay agents and servers
ff05::1:3	Site-local	All DHCPv6 servers

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 8415	DHCPv6 (unified spec, replaces RFC 3315/3633/3736)
RFC 3315	DHCPv6 (original, superseded by 8415)
RFC 3633	DHCPv6 Prefix Delegation (merged into 8415)
RFC 5908	NTP Server Option for DHCPv6

See Also

- DHCP -- IPv4 equivalent
- IPv6 -- the network layer DHCPv6 configures
- ICMPv6 -- Router Advertisements control DHCPv6 usage (M/O flags)
- UDP

DNS -- Domain Name System

Standard: RFC 1035 | **Layer:** Application (Layer 7) | **Wireshark filter:** `dns`

DNS is the Internet's naming system, translating human-readable domain names (e.g., `example.com`) into IP addresses and other resource records. It uses a hierarchical, distributed database served by a global network of name servers. DNS primarily uses UDP port 53 for queries, falling back to TCP for large responses or zone transfers.

Message Header

[Diagram: see online version]

The header is followed by four variable-length sections: Question, Answer, Authority, and Additional.

Key Fields

Field	Size	Description
Transaction ID	16 bits	Matches replies to requests
QR	1 bit	0 = query, 1 = response
Opcode	4 bits	Type of query
AA	1 bit	Authoritative Answer -- server is authoritative for the domain
TC	1 bit	Truncated -- response was truncated (retry with TCP)
RD	1 bit	Recursion Desired -- client wants recursive resolution
RA	1 bit	Recursion Available -- server supports recursion
Z	3 bits	Reserved (must be zero)
RCODE	4 bits	Response code
QDCOUNT	16 bits	Number of questions
ANCOUNT	16 bits	Number of answer resource records
NSCOUNT	16 bits	Number of authority resource records
ARCOUNT	16 bits	Number of additional resource records

Field Details

Opcode

Value	Meaning
0	Standard query (QUERY)
1	Inverse query (IQUERY, obsolete)
2	Server status request (STATUS)
4	Notify (RFC 1996)
5	Update (RFC 2136, dynamic DNS)

Response Code (RCODE)

Value	Name	Meaning
0	NOERROR	No error
1	FORMERR	Format error in query
2	SERVFAIL	Server failure
3	NXDOMAIN	Name does not exist
4	NOTIMP	Not implemented
5	REFUSED	Query refused by policy

Question Section

Each question entry contains:

Field	Description
QNAME	Domain name, encoded as a sequence of labels
QTYPE	Record type being queried (A, AAAA, MX, etc.)
QCLASS	Class (almost always IN = Internet)

Common Record Types

Type	Value	Description
A	1	IPv4 address
NS	2	Authoritative name server
CNAME	5	Canonical name (alias)
SOA	6	Start of Authority
PTR	12	Pointer (reverse DNS)
MX	15	Mail exchange
TXT	16	Text record (SPF, DKIM, verification)
AAAA	28	IPv6 address

SRV	33	Service locator
OPT	41	EDNS pseudo-record
DS	43	Delegation Signer (DNSSEC)
RRSIG	46	DNSSEC signature
DNSKEY	48	DNSSEC public key
HTTPS	65	HTTPS service binding

Resource Record Format

[Diagram: see online version]

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 1034	Domain Names -- Concepts and Facilities
RFC 1035	Domain Names -- Implementation and Specification
RFC 6891	Extension Mechanisms for DNS (EDNS0)
RFC 4033	DNS Security Introduction and Requirements (DNSSEC)
RFC 7858	DNS over TLS (DoT)
RFC 8484	DNS Queries over HTTPS (DoH)
RFC 2136	Dynamic Updates in DNS

See Also

- UDP
- TCP
- DHCP -- often assigns DNS server addresses
- TLS -- used by DNS over TLS (DoT)

DNSSEC -- DNS Security Extensions

Standard: RFC 4033 / RFC 4034 / RFC 4035 | **Layer:** Application (Layer 7) | **Wireshark filter:** `dns.flags.ad` (Authenticated Data) or `dns.type == 46` (RRSIG)

DNSSEC adds cryptographic authentication to DNS, allowing resolvers to verify that DNS responses have not been tampered with and genuinely originate from the authoritative zone. It does **not** encrypt DNS traffic -- it provides **data origin authentication** and **integrity protection** via digital signatures. DNSSEC introduces new record types (RRSIG, DNSKEY, DS, NSEC/NSEC3) and builds a chain of trust from the signed DNS root zone down through TLDs to individual domains. A validating resolver checks signatures at each level to confirm authenticity.

New Record Types

Record Type	Code	Purpose
RRSIG	46	Signature over an RRset (one per record type per zone)
DNSKEY	48	Public key for the zone (used to verify RRSIGs)
DS	43	Delegation Signer -- hash of a child zone's KSK (links parent to child)
NSEC	47	Authenticated denial of existence (proves a name/type does not exist)
NSEC3	50	Hashed denial of existence (prevents zone walking)
NSEC3PARAM	51	Parameters for NSEC3 hash computation

RRSIG Record

An RRSIG record contains a digital signature covering an entire RRset (all records of one type at one name).

[Diagram: see online version]

Field	Size	Description
Type Covered	16 bits	DNS record type this RRSIG covers (e.g., 1=A, 28=AAAA, 15=MX)
Algorithm	8 bits	Cryptographic algorithm used (see algorithm table)
Labels	8 bits	Number of labels in the owner name (used for wildcard detection)
Original TTL	32 bits	TTL of the covered RRset (used in signature computation)
Signature Expiration	32 bits	Unix timestamp when signature expires
Signature Inception	32 bits	Unix timestamp when signature becomes valid

Key Tag	16 bits	Short identifier to match this RRSIG to the correct DNSKEY
Signer's Name	Variable	Domain name of the zone that generated the signature
Signature	Variable	The cryptographic signature bytes

DNSKEY Record

A DNSKEY record publishes the zone's public key. There are two key roles:

[Diagram: see online version]

Field	Size	Description
Flags	16 bits	Bit 7 = Zone Key (must be 1), Bit 15 = SEP (Secure Entry Point, indicates KSK)
Protocol	8 bits	Always 3 (DNSSEC)
Algorithm	8 bits	Cryptographic algorithm
Public Key	Variable	Public key material (algorithm-specific encoding)

ZSK vs KSK

Key Type	Flags Value	Description
ZSK (Zone Signing Key)	256 (bit 7 set)	Signs all RRsets in the zone. Shorter key, rotated frequently.
KSK (Key Signing Key)	257 (bits 7+15 set)	Signs only the DNSKEY RRset. Longer key, rotated infrequently. Its hash is published as a DS record in the parent zone.

Chain of Trust

DNSSEC validation follows a hierarchical chain from the root zone down:

[Diagram: see online version]

Chain of Trust Linkage

[Diagram: see online version]

DS Record

The DS (Delegation Signer) record is published in the **parent** zone and contains a hash of the child zone's KSK, linking the two zones:

Field	Size	Description
Key Tag	16 bits	Tag of the child's DNSKEY being referenced
Algorithm	8 bits	Algorithm of the child's DNSKEY
Digest Type	8 bits	Hash algorithm: 1=SHA-1, 2=SHA-256, 4=SHA-384
Digest	Variable	Hash of the child's DNSKEY record (owner name + RDATA)

Cryptographic Algorithms

Number	Algorithm	Status
5	RSA/SHA-1	Deprecated
7	RSASHA1-NSEC3-SHA1	Deprecated
8	RSA/SHA-256	Widely deployed
10	RSA/SHA-512	Supported
13	ECDSA P-256/SHA-256	Recommended (smaller keys/signatures)
14	ECDSA P-384/SHA-384	Supported
15	Ed25519	Recommended (smallest, fastest)
16	Ed448	Supported

Authenticated Denial of Existence

DNSSEC must prove that a queried name or record type does **not** exist, without allowing an attacker to forge "no such name" responses.

NSEC (Next Secure)

NSEC records form a sorted chain of all names in the zone. Each NSEC record points to the next name and lists the record types that exist at the current name. A gap in the chain proves a name does not exist.

Drawback	Description
Zone walking	An attacker can enumerate all names in a zone by following the NSEC chain

NSEC3 (RFC 5155)

NSEC3 replaces plain names with hashed names, preventing zone enumeration:

Field	Description
Hash Algorithm	1 = SHA-1
Flags	Bit 0 = Opt-Out (skip unsigned delegations)
Iterations	Number of additional hash iterations
Salt	Random salt prepended before hashing
Next Hashed Owner	Hash of the next existing owner name
Type Bit Maps	Record types at this hashed owner name

Feature	NSEC	NSEC3
Name representation	Plaintext domain names	SHA-1 hashes of domain names
Zone walking	Trivial	Computationally expensive (requires brute-forcing hashes)
Zone file size	Moderate	Larger (hashed records)
Response size	Moderate	Larger
CPU cost	Lower	Higher (hashing)

Key Rollover

Keys must be periodically rotated for security. Two main strategies:

ZSK Rollover (Pre-Publish)

Step	Action
1	Publish new ZSK in DNSKEY RRset (not yet signing)
2	Wait for old DNSKEY TTL to expire (propagation)
3	Sign all RRsets with new ZSK, remove old signatures
4	Wait for old RRSIG TTL to expire
5	Remove old ZSK from DNSKEY RRset

KSK Rollover (Double-DS)

Step	Action
1	Generate new KSK, add to DNSKEY RRset, sign DNSKEY with both KSKs
2	Add DS record for new KSK in parent zone (keep old DS too)
3	Wait for parent DS TTL to expire
4	Remove old KSK from DNSKEY RRset
5	Remove old DS from parent zone

DNSSEC-Related DNS Flags

Flag	Bit	Description
DO (DNSSEC OK)	EDNS(0)	Client requests DNSSEC records in response
AD (Authenticated Data)	Header	Resolver asserts all data in the answer is authenticated
CD (Checking Disabled)	Header	Client tells resolver to skip DNSSEC validation

Encapsulation

[Diagram: see online version]

DNSSEC records are carried as standard DNS records in DNS responses. The DO bit in the EDNS(0) OPT record signals that the client supports DNSSEC.

Standards

Document	Title
RFC 4033	DNS Security Introduction and Requirements
RFC 4034	Resource Records for the DNS Security Extensions
RFC 4035	Protocol Modifications for the DNS Security Extensions
RFC 5155	DNS Security (DNSSEC) Hashed Authenticated Denial of Existence (NSEC3)
RFC 6781	DNSSEC Operational Practices, Version 2
RFC 8624	Algorithm Implementation Requirements and Usage Guidance for DNSSEC

See Also

- DNS -- the base DNS protocol that DNSSEC extends
- DoH -- DNS over HTTPS (provides confidentiality; DNSSEC provides authenticity)
- DoT -- DNS over TLS (provides confidentiality; DNSSEC provides authenticity)
- DANE -- DNS-Based Authentication of Named Entities (relies on DNSSEC)
- X.509 -- certificate-based authentication (alternative trust model)

DoH -- DNS over HTTPS

Standard: RFC 8484 | **Layer:** Application (Layer 7) | **Wireshark filter:** `dns` (only with TLS key log; otherwise `tls`)

DNS over HTTPS encapsulates standard DNS queries and responses inside HTTPS, providing confidentiality, integrity, and authentication for DNS resolution. Because DoH uses the same port as regular HTTPS (443/tcp), DNS traffic blends with normal web traffic and cannot be easily blocked or distinguished by network middleboxes. DoH uses standard DNS wire format (RFC 1035) as the payload, transported via HTTP/2 or HTTP/3. Clients can send queries using either HTTP GET (base64url-encoded) or POST (binary body).

Query Methods

DoH supports two HTTP methods for sending DNS queries:

GET Method

The DNS query is base64url-encoded (without padding) and sent as the `dns` query parameter:

```
GET /dns-query?dns=AAABAAABAAAAAAAAA3d3dwdleGFtcGxlA2NvbQAAAQAB HTTP/2
Accept: application/dns-message
```

POST Method

The DNS query is sent as binary in the HTTP body:

```
POST /dns-query HTTP/2
Content-Type: application/dns-message
Accept: application/dns-message
Content-Length: 33

&lt;binary DNS wire format query&gt;
```

Method	Content-Type	DNS Encoding	Cacheability
GET	N/A (query in URL)	base64url in `?dns=` parameter	Cacheable by HTTP caches
POST	`application/dns-message`	Binary wire format in body	Not cached by intermediaries

GET Query Flow

[Diagram: see online version]

POST Query Flow

[Diagram: see online version]

Key Fields

Field	Value	Description
URI Template	`https://{host}/dns-query{?dns}`	Standard DoH endpoint path
Content-Type	`application/dns-message`	MIME type for DNS wire format
Accept	`application/dns-message`	Client indicates expected response format
DNS Payload	RFC 1035 wire format	Standard DNS message (same bytes as UDP DNS)
HTTP Version	HTTP/2 or HTTP/3	Multiplexed streams for parallel queries
Port	443/tcp	Standard HTTPS port

HTTP/2 Benefits

Feature	Benefit for DNS
Multiplexing	Multiple DNS queries in parallel over one TLS connection
Header compression (HPACK)	Reduces overhead for repeated headers
Server push	Server could proactively push related records (not widely used)
Connection reuse	Amortize TLS handshake cost across many queries

Padding (RFC 8467)

DNS messages have variable length that can reveal the queried domain. RFC 8467 recommends padding DoH queries and responses to fixed block sizes using EDNS(0) Padding (Option Code 12):

Recommendation	Query Padding	Response Padding
RFC 8467	Pad to 128-byte blocks	Pad to 468-byte blocks

Major Providers

Provider	DoH Endpoint	Notes
Cloudflare	`https://cloudflare-dns.com/dns-query`	Also `https://1.1.1.1/dns-query`
Google	`https://dns.google/dns-query`	Also supports JSON API
Quad9	`https://dns.quad9.net/dns-query`	Malware blocking
NextDNS	`https://dns.nextdns.io/{config_id}`	Customizable filtering

AdGuard	`https://dns.adguard-dns.com/dns-query`	Ad/tracker blocking

DoH vs DoT vs DNS vs DNSCrypt

Feature	DNS (Plain)	DoT	DoH	DNSCrypt
Encryption	None	TLS	TLS (via HTTPS)	X25519-XSalsa20Poly1305
Port	53 (UDP/TCP)	853 (TCP)	443 (TCP)	443 or 5443 (UDP/TCP)
Visibility	Fully visible	Dedicated port (detectable)	Blends with HTTPS	Dedicated port
Blockable	Easily	By blocking port 853	Hard (same as blocking HTTPS)	By blocking port
Standardization	RFC 1035	RFC 7858	RFC 8484	Community spec
HTTP features	No	No	Yes (caching, multiplexing)	No
Wire format	DNS	DNS over TLS	DNS over HTTP over TLS	Encrypted DNS

Privacy and Censorship Considerations

Consideration	Description
Query privacy	Encrypted queries prevent ISP/network snooping
Centralization risk	DoH often shifts trust from ISP to a few large providers
Enterprise visibility	DoH bypasses corporate DNS monitoring and content filters
Censorship resistance	Cannot block DoH without blocking all HTTPS to the provider
Server authentication	TLS certificates authenticate the DoH server
SNI leakage	TLS SNI may reveal the DoH server (mitigated by ECH)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 8484	DNS Queries over HTTPS (DoH)

RFC 8467	Padding Policies for Extension Mechanisms for DNS (EDNS(0))
RFC 1035	Domain Names -- Implementation and Specification
RFC 7540	HTTP/2

See Also

- DNS -- the underlying DNS wire format and semantics
- DoT -- DNS over TLS on dedicated port 853
- DNSSEC -- cryptographic authentication of DNS data (complementary to DoH)
- TLS -- transport encryption layer
- HTTP -- application protocol used by DoH

DoT -- DNS over TLS

Standard: RFC 7858 | **Layer:** Application (Layer 7) | **Wireshark filter:** `dns` (with TLS key log) or `tls && tcp.port == 853`

DNS over TLS encrypts standard DNS queries by wrapping them in a TLS connection on a dedicated port (853/tcp). Unlike plain DNS, which sends queries and responses in cleartext over UDP/TCP port 53, DoT prevents eavesdropping and tampering by network intermediaries. The DNS wire format (RFC 1035) is preserved unchanged -- each DNS message is length-prefixed and sent over the TLS stream, exactly as DNS-over-TCP works but inside a TLS tunnel. DoT was standardized before DoH and uses a dedicated port, making it straightforward to deploy but also straightforward to block.

Message Framing

DoT uses the same TCP framing as DNS-over-TCP: a 2-byte length prefix followed by the DNS message.

[Diagram: see online version]

Field	Size	Description
Length	2 bytes	Length of the following DNS message in bytes (network byte order)
DNS Message	Variable	Standard DNS wire format message (RFC 1035)

Multiple DNS messages can be sent sequentially over the same TLS connection, each with its own length prefix.

Connection Flow

[Diagram: see online version]

Key Parameters

Parameter	Value	Description
Port	853/tcp	Dedicated well-known port for DoT
TLS Version	TLS 1.2 or 1.3	TLS 1.3 preferred (1-RTT handshake)
DNS Format	RFC 1035 wire format	Identical to DNS-over-TCP payload
Framing	2-byte length prefix	Same as DNS-over-TCP (RFC 1035 Section 4.2.2)
Connection reuse	Yes	Persistent TLS session for multiple queries
ALPN	Not required	No ALPN token defined for DoT (unlike DoH)

TLS Requirements

Requirement	Description
Server Certificate	X.509 certificate for the DoT server (validates server identity)
Certificate Pinning	Optional SPKI (Subject Public Key Info) pinning for known resolvers
Session Resumption	TLS session tickets reduce reconnection overhead
SNI	Server Name Indication reveals the DoT server hostname
Minimum Version	TLS 1.2 required; TLS 1.3 recommended

Usage Profiles (RFC 8310)

RFC 8310 defines two usage profiles for DoT:

Profile	Name	Description
Strict	Strict Privacy	Client MUST authenticate the server (certificate validation, optional SPKI pin). Connection fails if authentication fails. No fallback to plain DNS.
Opportunistic	Opportunistic Privacy	Client attempts DoT but falls back to plain DNS if port 853 is blocked or TLS fails. May skip server authentication.

Strict Mode

[Diagram: see online version]

Opportunistic Mode

[Diagram: see online version]

Padding (EDNS(0) Padding)

DNS message lengths can reveal queried domain names even through TLS. The EDNS(0) Padding option (RFC 7830) adds padding to DNS messages:

[Diagram: see online version]

Recommendation	Query Padding	Response Padding
RFC 8467	Pad to 128-byte blocks	Pad to 468-byte blocks

DoT vs DoH

Feature	DoT	DoH
Port	853 (dedicated)	443 (shared with HTTPS)
Detectability	Easy (distinct port)	Hard (blends with web traffic)
Blockability	Easy (block port 853)	Hard (requires blocking HTTPS)
Protocol overhead	Lower (just TLS)	Higher (TLS + HTTP/2 framing)
HTTP features	None	Caching, multiplexing, server push
Enterprise monitoring	Visible as DNS traffic	Hidden in HTTPS traffic
NAT/firewall traversal	May be blocked	Usually passes (port 443 open)
Standardization	RFC 7858 (2016)	RFC 8484 (2018)

Major Providers

Provider	DoT Server	IP Address
Cloudflare	`cloudflare-dns.com`	1.1.1.1, 1.0.0.1
Google	`dns.google`	8.8.8.8, 8.8.4.4
Quad9	`dns.quad9.net`	9.9.9.9
AdGuard	`dns.adguard-dns.com`	94.140.14.14

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 7858	DNS over Transport Layer Security (DNS over TLS)
RFC 8310	Usage Profiles for DNS over TLS and DNS over DTLS
RFC 7830	The EDNS(0) Padding Option
RFC 8467	Padding Policies for Extension Mechanisms for DNS
RFC 1035	Domain Names -- Implementation and Specification

See Also

- DNS -- the underlying DNS protocol and wire format
- DoH -- DNS over HTTPS (alternative encrypted DNS transport)
- DNSSEC -- cryptographic authentication of DNS data (complementary to DoT)
- TLS -- transport encryption layer

LDAP -- Lightweight Directory Access Protocol

Standard: RFC 4511 | **Layer:** Application (Layer 7) | **Wireshark filter:** `ldap`

LDAP is a protocol for accessing and managing distributed directory services over IP. A directory is a specialized database optimized for reads, lookups, and hierarchical data -- think phone books, organizational structures, and user accounts. LDAP is the standard way applications authenticate users and look up identity information. It is the protocol behind Microsoft Active Directory, OpenLDAP, and most enterprise identity systems. LDAP uses a tree-structured namespace (the Directory Information Tree) where each entry is identified by a Distinguished Name (DN).

Message Envelope

Every LDAP message is encoded using ASN.1 BER (Basic Encoding Rules) and wrapped in an LDAPMessage envelope:

[Diagram: see online version]

Field	Encoding	Description
Message ID	INTEGER	Matches requests to responses (client-assigned)
Protocol Op	CHOICE	The LDAP operation (Bind, Search, Modify, etc.)
Controls	SEQUENCE (optional)	Extended operation controls

LDAP uses ASN.1 BER encoding throughout, not fixed binary offsets -- the diagram above is conceptual. Each field is a TLV (Tag-Length-Value) triplet.

Operations

Op	Tag	Name	Description
0	0x60	BindRequest	Authenticate to the directory
1	0x61	BindResponse	Authentication result
2	0x42	UnbindRequest	Close the connection
3	0x63	SearchRequest	Search for entries
4	0x64	SearchResultEntry	One matching entry
5	0x65	SearchResultDone	Search complete
6	0x66	ModifyRequest	Modify an entry's attributes
7	0x67	ModifyResponse	Modify result
8	0x68	AddRequest	Add a new entry
9	0x69	AddResponse	Add result
10	0x4A	DelRequest	Delete an entry
11	0x6B	DelResponse	Delete result
12	0x6C	ModDNRequest	Rename/move an entry
13	0x6D	ModDNResponse	Rename result
14	0x6E	CompareRequest	Compare an attribute value
15	0x6F	CompareResponse	Compare result
16	0x50	AbandonRequest	Cancel an outstanding operation
23	0x77	ExtendedRequest	Extended operation (StartTLS, password modify, etc.)
24	0x78	ExtendedResponse	Extended operation result
25	0x79	IntermediateResponse	Multi-step operation progress

Result Codes

Code	Name	Meaning
0	success	Operation completed successfully
1	operationsError	Server-side error
2	protocolError	Malformed request
3	timeLimitExceeded	Search time limit reached

4	sizeLimitExceeded	Search size limit reached
7	authMethodNotSupported	Requested auth not supported
8	strongerAuthRequired	Stronger authentication needed
16	noSuchAttribute	Attribute does not exist
32	noSuchObject	Entry does not exist (DN not found)
34	invalidDNSyntax	Malformed DN
48	inappropriateAuthentication	Wrong authentication type
49	invalidCredentials	Wrong username or password
50	insufficientAccessRights	Permission denied
53	unwillingToPerform	Server refuses the operation
65	objectClassViolation	Entry violates schema
68	entryAlreadyExists	DN already in use

Key Concepts

Distinguished Name (DN)

Every entry has a unique DN that describes its position in the tree:

```
cn=John Smith,ou=People,dc=example,dc=com
```

Component	Meaning
dc	Domain Component
ou	Organizational Unit
cn	Common Name
uid	User ID
o	Organization
c	Country

Directory Information Tree (DIT)

[Diagram: see online version]

Search Request

A search is the most common LDAP operation:

Parameter	Description

Base DN	Where in the tree to start searching
Scope	baseObject (just this entry), singleLevel (children), wholeSubtree (all descendants)
Deref Aliases	How to handle aliases (never, search, find, always)
Size Limit	Maximum entries to return (0 = no limit)
Time Limit	Maximum seconds for the search (0 = no limit)
Types Only	Return attribute names only (no values)
Filter	Boolean expression selecting entries
Attributes	Which attributes to return

Search Filters

Filters use a prefix notation enclosed in parentheses:

Filter	Meaning
`(cn=John Smith)`	Equality -- cn equals "John Smith"
`(cn=John*)`	Substring -- cn starts with "John"
`(uidNumber>=1000)`	Greater or equal
`(objectClass=*)`	Presence -- attribute exists
`(&(objectClass=person)(uid=jsmith))`	AND -- both conditions
`(\	(dept=Sales)(dept=Marketing))`
`(!(accountDisabled=TRUE))`	NOT -- negation

Authentication (Bind)

Method	Description
Anonymous	No credentials (Bind with empty DN and password)
Simple	DN + plaintext password (requires TLS)
SASL	Framework for pluggable auth mechanisms

Common SASL mechanisms:

Mechanism	Description
PLAIN	Username + password (requires TLS)
EXTERNAL	Use TLS client certificate
GSSAPI	Kerberos (Active Directory default)
DIGEST-MD5	Challenge-response (deprecated)
CRAM-MD5	Challenge-response (deprecated)

Common Object Classes

Class	Description	Typical Attributes
inetOrgPerson	Person entry	uid, cn, sn, mail, userPassword
organizationalUnit	Container/folder	ou, description
groupOfNames	Static group	cn, member (list of DNs)
posixAccount	UNIX account	uid, uidNumber, gidNumber, homeDirectory
dcObject	Domain component	dc

Session Flow

[Diagram: see online version]

Ports

Port	Service	Encryption
389	LDAP	None or STARTTLS
636	LDAPS	Implicit TLS from connection start
3268	Global Catalog	Active Directory cross-domain search
3269	Global Catalog (SSL)	AD Global Catalog over TLS

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 4510	LDAP Technical Specification Road Map
RFC 4511	LDAP: The Protocol
RFC 4512	LDAP: Directory Information Models
RFC 4513	LDAP: Authentication Methods and Security Mechanisms
RFC 4514	LDAP: String Representation of Distinguished Names
RFC 4515	LDAP: String Representation of Search Filters
RFC 4516	LDAP: Uniform Resource Locator
RFC 4519	LDAP: Schema for User Applications

RFC 4522	LDAP: Binary Encoding Option

See Also

- TCP
- TLS -- encrypts LDAP connections (LDAPS / StartTLS)
- DNS -- SRV records for LDAP server discovery (`_ldap._tcp.example.com`)
- HTTP -- REST/SCIM increasingly used alongside LDAP for identity

mDNS / DNS-SD -- Multicast DNS / DNS Service Discovery

Standard: RFC 6762 (mDNS) / RFC 6763 (DNS-SD) | **Layer:** Application (Layer 7) | **Wireshark filter:** `mdns`

mDNS provides DNS-like name resolution on a local network without a DNS server. Instead of querying a server, mDNS queries are sent via multicast -- all devices on the link receive the query and the owner of the name responds. DNS-SD builds on mDNS to discover available services (printers, file shares, web servers, AirPlay, Chromecast) on the local network. Together they power Apple's Bonjour, Linux's Avahi, and Google's Cast discovery. The `.local` top-level domain is reserved for mDNS.

How mDNS Works

[Diagram: see online version]

All queries and responses use the same DNS message format (RFC 1035) but are sent to/from:

Parameter	Value
IPv4 Multicast Address	224.0.0.251
IPv6 Multicast Address	ff02::fb
UDP Port	5353
Domain	`.local`

mDNS vs Unicast DNS

Feature	Unicast DNS	mDNS
Server	Required (recursive resolver)	None (peer-to-peer)
Transport	Unicast UDP/TCP port 53	Multicast UDP port 5353
Domain	Any	`.local` only
Scope	Global Internet	Link-local only

Record registration	Zone file / dynamic DNS	Self-registration (announce)
Conflict resolution	Authority of the zone	Probing + announcement
TTL	Typically long (hours-days)	Short (75 min default for host records)

Name Registration

When a device joins the network, it probes to check if its desired name is already taken, then announces it:

[Diagram: see online version]

If another device responds during probing, the new device must pick a different name (e.g., `mydevice-2.local`).

DNS-SD (Service Discovery)

DNS-SD uses standard DNS record types (SRV, TXT, PTR) to advertise and discover services:

Service Browsing

[Diagram: see online version]

DNS-SD Record Types

Record	Name Format	Purpose
PTR	`_service._proto.local`	Service type browsing (enumerate instances)
SRV	`Instance._service._proto.local`	Hostname and port of a service instance
TXT	`Instance._service._proto.local`	Key-value metadata about the service
A/AAAA	`hostname.local`	IP address of the host

Service Name Format

```
Instance Name._service._protocol.local
```

Component	Description	Example
Instance Name	Human-readable service name	"Office Printer"
_service	Service type (IANA-registered)	`_ipp`
_protocol	Transport protocol	`_tcp` or `_udp`
.local	mDNS domain	`.local`

Common Service Types

Service Type	Description
`_http._tcp`	Web server
`_https._tcp`	Secure web server
`_ipp._tcp`	Internet Printing Protocol (AirPrint)
`_printer._tcp`	LPR/LPD printer
`_smb._tcp`	Windows file sharing (SMB)
`_afpovertcp._tcp`	Apple Filing Protocol
`_nfs._tcp`	NFS file sharing
`_ssh._tcp`	SSH remote access
`_sftp-ssh._tcp`	SFTP
`_ftp._tcp`	FTP
`_airplay._tcp`	Apple AirPlay
`_raop._tcp`	AirPlay audio (Remote Audio Output)
`_googlecast._tcp`	Google Chromecast
`_spotify-connect._tcp`	Spotify Connect
`_homekit._tcp`	Apple HomeKit
`_hap._tcp`	HomeKit Accessory Protocol
`_mqtt._tcp`	MQTT broker
`_coap._udp`	CoAP server

TXT Record Format

Key-value pairs, each prefixed with a length byte:

```
pdl=application/pdf
Color=T
Duplex=T
URF=W8,SRGB24,CP1,RS300
```

Boolean values: key present = true, key absent = false, `key=` with empty value is valid.

Implementations

Software	Platform	Description
Bonjour	macOS, iOS, Windows	Apple's mDNS/DNS-SD implementation
Avahi	Linux	Open-source mDNS/DNS-SD daemon
Windows mDNS	Windows 10+	Built-in mDNS responder
nss-mdns	Linux	NSS plugin for `.local` resolution

dns-sd (tool)	macOS	Command-line service discovery
avahi-browse	Linux	Command-line service browser

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 6762	Multicast DNS
RFC 6763	DNS-Based Service Discovery
RFC 2782	SRV Records (used by DNS-SD)
IANA Service Types	Registered service type names

See Also

- DNS -- unicast DNS that mDNS mirrors
- DHCP -- often used alongside mDNS for network configuration
- UDP -- mDNS transport
- LLDP -- alternative link-layer discovery

NTP -- Network Time Protocol

Standard: RFC 5905 | **Layer:** Application (Layer 7) | **Wireshark filter:** `ntp`

NTP synchronizes clocks between computer systems over a network. It uses a hierarchical system of time sources organized by stratum level, where stratum 0 is a reference clock (GPS, atomic clock), stratum 1 is directly connected to a reference, and so on. NTP can achieve sub-millisecond accuracy on a LAN and typically within tens of milliseconds over the Internet. SNTP (Simple NTP) is a simplified subset for less demanding use cases.

Packet

[Diagram: see online version]

The NTP packet is 48 bytes (384 bits) without extension fields or authentication. NTP timestamps are 64 bits: 32 bits for seconds since 1 January 1900 and 32 bits for the fractional second.

Key Fields

Field	Size	Description
LI	2 bits	Leap Indicator -- upcoming leap second warning
VN	3 bits	Version Number (currently 4)
Mode	3 bits	Operating mode
Stratum	8 bits	Stratum level of the server
Poll	8 bits	Maximum polling interval as log2 seconds
Precision	8 bits	Clock precision as log2 seconds
Root Delay	32 bits	Round-trip delay to the reference source (fixed-point)
Root Dispersion	32 bits	Maximum error relative to the reference (fixed-point)
Reference ID	32 bits	Identifier of the reference source
Reference Timestamp	64 bits	Time the system clock was last set
Origin Timestamp	64 bits	Time at the client when the request departed (T1)
Receive Timestamp	64 bits	Time at the server when the request arrived (T2)
Transmit Timestamp	64 bits	Time at the server when the response departed (T3)

Field Details

Leap Indicator (LI)

Value	Meaning
0	No warning
1	Last minute of the day has 61 seconds
2	Last minute of the day has 59 seconds
3	Clock not synchronized (alarm)

Mode

Value	Meaning
1	Symmetric active
2	Symmetric passive

3	Client
4	Server
5	Broadcast
6	NTP control message
7	Private use

Stratum

Value	Meaning
0	Unspecified or invalid
1	Primary server (directly attached to reference clock)
2-15	Secondary servers (synchronized via NTP)
16	Unsynchronized

Reference ID

For stratum 1, this is a 4-character ASCII string identifying the reference source:

ID	Source
GPS	Global Positioning System
PPS	Pulse Per Second
ATOM	Atomic clock
CDMA	CDMA reference
DCF	DCF77 longwave (Germany)
LOCL	Local (uncalibrated) clock

For stratum 2+, this is the IPv4 address of the reference server (or first 4 bytes of the MD5 hash of the IPv6 address).

Clock Offset Calculation

The client records the time the reply arrived (T4) and computes:

- **Offset** = ((T2 - T1) + (T3 - T4)) / 2
- **Round-trip delay** = (T4 - T1) - (T3 - T2)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 5905	Network Time Protocol Version 4: Protocol and Algorithms
RFC 5906	NTPv4 Autokey Public-Key Authentication
RFC 8915	Network Time Security for NTP
RFC 4330	Simple Network Time Protocol (SNTP) Version 4
RFC 7822	NTP Extension Fields

See Also

- UDP
- DHCP -- can distribute NTP server addresses (option 42)

Wake-on-LAN -- WOL

Standard: AMD Magic Packet Technology (White Paper, 1995) | **Layer:** Data Link (Layer 2) / Application | **Wireshark filter:** `wol`

Wake-on-LAN (WOL) is a mechanism for remotely powering on a computer over the network by sending a specially crafted "magic packet." The target machine's network interface card (NIC) remains minimally powered in sleep, hibernate, or shutdown states, listening for its own MAC address pattern. When the NIC detects a valid magic packet, it signals the motherboard to initiate a full power-on sequence. WOL is widely used in enterprise environments for after-hours patching, remote administration, and energy-saving power management, as well as by home users for remote access to sleeping machines.

Magic Packet

[Diagram: see online version]

The magic packet is exactly 102 bytes: 6 bytes of `0xFF` followed by the target machine's 48-bit MAC address repeated 16 times. An optional 4-byte or 6-byte password may be appended for SecureON.

Key Fields

Field	Size	Description
Synchronization stream	6 bytes	Six bytes of `0xFF` (all ones), marks start of magic packet
Target MAC address	96 bytes	Target machine's 6-byte MAC address repeated 16 times

SecureON password	0, 4, or 6 bytes	Optional password for Secure Wake-on-LAN

Wake-on-LAN Flow

Local Subnet Wake

[Diagram: see online version]

Wake-on-WAN (Across Subnets)

[Diagram: see online version]

Transport Methods

The magic packet can be delivered via several transport mechanisms. It only needs to arrive as a data payload -- the NIC scans every incoming frame for the pattern regardless of protocol.

Method	Description
UDP Broadcast (port 7 or 9)	Most common: sent to `255.255.255.255` or subnet broadcast on port 7 (Echo) or 9 (Discard)
Raw Ethernet frame	EtherType `0x0842`, no IP/UDP headers required, NIC matches at Layer 2
UDP Directed Broadcast	Sent to a specific subnet's broadcast address (e.g., `10.0.1.255`) for cross-subnet wake
Subnet unicast	Sent to the target's last-known IP with a static ARP entry to prevent ARP failure

SecureON (Secure Wake-on-LAN)

Some NICs support a password that must be appended to the magic packet. The NIC stores the password in non-volatile memory and only triggers wake if the password matches.

Feature	Standard WOL	SecureON
Authentication	None (any sender with MAC knowledge)	4-byte or 6-byte password required
Password storage	N/A	Stored in NIC EEPROM/firmware
Security level	Low (MAC addresses are easily discovered)	Marginal improvement (password sent in cleartext)
Configuration	BIOS/UEFI + OS driver	NIC firmware utility

Requirements

For Wake-on-LAN to function, several conditions must be met:

Requirement	Description
NIC support	Network adapter must support WOL (nearly all modern NICs do)
BIOS/UEFI setting	WOL or "Wake on PCI/PCIe" must be enabled in firmware settings
OS driver configuration	NIC driver must enable WOL before shutdown (e.g., `ethtool -s eth0 wol g` on Linux)
Standby power	NIC must receive standby power (5V SB from ATX PSU) during sleep/shutdown
Link state	NIC must maintain Ethernet link while system is sleeping (link LED stays on)
Switch port	Switch port must remain active (not disabled by spanning tree or power saving)
Broadcast routing	For WAN wake: routers must forward directed broadcasts (`ip directed-broadcast` on Cisco)

WOL vs IPMI / BMC

For enterprise servers, Intelligent Platform Management Interface (IPMI) provides a more robust out-of-band management alternative.

Feature	Wake-on-LAN	IPMI / BMC
Power on	Yes	Yes
Power off	No	Yes (hard or graceful)
Power cycle	No	Yes
Console access	No	Yes (SOL, KVM)
Health monitoring	No	Yes (fans, temps, voltages)
BIOS/UEFI access	No	Yes (remote KVM)
Dedicated NIC	No (shares main NIC)	Yes (dedicated management port)
Authentication	None or weak password	Strong (user/pass, RAKP)
Cost	Free (built into NIC)	Requires BMC hardware
Protocol	Magic packet (Layer 2)	IPMI over LAN (UDP 623)

Common WOL Tools

Tool	Platform	Description
`etherwake`	Linux	Send magic packets from command line
`wakeonlan`	Linux/macOS	Perl-based WOL utility

`wol`	Linux	Simple command-line WOL sender
`wolcmd`	Windows	Command-line magic packet sender
PowerShell	Windows	`Send-WolPacket` or custom script using `[Net.Sockets.UdpClient]`

Encapsulation

[Diagram: see online version]

The magic packet can ride directly in an Ethernet frame (EtherType `0x0842`) or inside a UDP datagram on port 7 or 9. The NIC pattern-matches the payload regardless of encapsulation.

Standards

Document	Title
AMD Magic Packet Technology	Original AMD white paper defining the magic packet format
IEEE 802.3	Ethernet standard, defines PME (Power Management Event) signaling
ACPI Specification	Advanced Configuration and Power Interface, defines system sleep states (S1-S5)

See Also

- Ethernet -- transport layer for magic packets
- ARP -- static ARP entries needed for unicast WOL across subnets
- DHCP -- machine acquires IP after waking up
- UDP -- common transport for magic packets

Network Layer

Network layer (Layer 3) protocols for packet routing and delivery.

11 protocols in this section

GRE -- Generic Routing Encapsulation

Standard: RFC 2784 | **Layer:** Network (Layer 3) | **Wireshark filter:** `gre`

GRE is a tunneling protocol that encapsulates a wide variety of network-layer protocols inside point-to-point virtual links. It wraps an inner packet (the payload) with a GRE header and an outer IP header, creating a tunnel between two endpoints. GRE is simple and stateless -- no encryption, no authentication, no reliability by default. It is widely used for site-to-site tunnels, carrier networks (MPLS over GRE), connecting non-contiguous subnets, and as the transport for PPTP VPN. GRE is often paired with IPsec for encryption.

Header

[Diagram: see online version]

The minimum GRE header is 4 bytes. Optional fields add up to 12 more bytes depending on the flags.

Key Fields

Field	Size	Description
C (Checksum)	1 bit	1 = Checksum and Reserved1 fields present
R (Routing)	1 bit	Deprecated (RFC 2784); must be 0
K (Key)	1 bit	1 = Key field present
S (Sequence)	1 bit	1 = Sequence Number field present
s (Strict Source)	1 bit	Deprecated; must be 0
Recur	3 bits	Deprecated; must be 0
A (Ack)	1 bit	Enhanced GRE (RFC 2637, PPTP)
Flags	4 bits	Reserved; must be 0
Version	3 bits	0 = standard GRE, 1 = Enhanced GRE (PPTP)
Protocol Type	16 bits	EtherType of the encapsulated payload
Checksum	16 bits	Optional; over GRE header + payload
Key	32 bits	Optional; tunnel identifier (demultiplexing)
Sequence Number	32 bits	Optional; for ordering

Protocol Type Values

Value	Encapsulated Protocol

0x0800	IPv4
0x86DD	IPv6
0x6558	Transparent Ethernet Bridging (GRETAP)
0x880B	PPP (used by PPTP)
0x8847	MPLS Unicast

Encapsulation

A GRE tunnel wraps the inner packet with a GRE header and new outer IP header:

[Diagram: see online version]

Full Stack Example

[Diagram: see online version]

Common Uses

Use Case	Description
Site-to-site tunnel	Connect remote networks over the Internet
GRE over IPsec	GRE provides multicast/multiprotocol; IPsec provides encryption
PPTP	GRE carries PPP for VPN (Enhanced GRE, version 1)
GRETAP	Tunnel Ethernet frames (bridge remote L2 segments)
MPLS over GRE	Carry MPLS labels across non-MPLS networks
ERSPAN	Cisco remote SPAN -- mirror traffic via GRE

GRE over IPsec

[Diagram: see online version]

GRE alone has no security. Wrapping GRE in IPsec adds encryption and authentication while preserving GRE's ability to carry multicast and non-IP protocols.

MTU Considerations

GRE adds overhead that reduces the effective MTU:

Component	Size
Outer IP header	20 bytes
GRE header (minimum)	4 bytes

GRE header (with key + seq)	12 bytes
Total overhead	**24-32 bytes**

With a 1500-byte Ethernet MTU, the inner packet MTU is typically **1476 bytes** (or less with IPsec).

Standards

Document	Title
RFC 2784	Generic Routing Encapsulation (GRE)
RFC 2890	Key and Sequence Number Extensions to GRE
RFC 1701	GRE (original, partially superseded)
RFC 2637	PPTP (uses Enhanced GRE)
RFC 7676	IPv6 over GRE

See Also

- IPv4 -- outer and inner layer; GRE is IP protocol 47
- IPsec -- commonly paired with GRE for encrypted tunnels
- MPLS -- can be carried over GRE
- PPP -- carried by GRE in PPTP

ICMP -- Internet Control Message Protocol

Standard: RFC 792 | **Layer:** Network (Layer 3) | **Wireshark filter:** `icmp`

ICMP is a supporting protocol for IPv4 that carries diagnostic and error messages. It is used by network tools like `ping` and `traceroute` and by routers to report delivery problems back to the source. ICMP messages are encapsulated inside IP packets but are considered part of the network layer, not the transport layer. IPv6 has its own version: ICMPv6 (RFC 4443).

Header

[Diagram: see online version]

The first 4 bytes are common to all ICMP messages. The remaining content depends on the Type and Code.

Key Fields

Field	Size	Description
Type	8 bits	ICMP message type
Code	8 bits	Subtype providing additional context
Checksum	16 bits	Error check over the entire ICMP message
Rest of Header	32 bits	Content varies by message type

Field Details

Common Types

Type	Name	Description
0	Echo Reply	Response to ping
3	Destination Unreachable	Packet could not be delivered
4	Source Quench	Congestion control (deprecated)
5	Redirect	Router advises a better route
8	Echo Request	Ping request
9	Router Advertisement	Router announces its presence
10	Router Solicitation	Host asks for router info
11	Time Exceeded	TTL expired (used by traceroute)
12	Parameter Problem	Bad header field
13	Timestamp Request	Request round-trip time measurement
14	Timestamp Reply	Reply to timestamp request

Echo Request / Reply (Type 8 / 0)

Used by `ping`. The identifier and sequence number allow matching replies to requests.

[Diagram: see online version]

Destination Unreachable (Type 3)

[Diagram: see online version]

Code	Meaning
0	Network Unreachable
1	Host Unreachable
2	Protocol Unreachable
3	Port Unreachable

4	Fragmentation Needed (DF set)
5	Source Route Failed
6	Destination Network Unknown
7	Destination Host Unknown
13	Communication Administratively Prohibited

Time Exceeded (Type 11)

Sent when TTL reaches zero (code 0) or fragment reassembly times out (code 1). This is how `traceroute` works -- it sends packets with incrementally increasing TTL values.

Code	Meaning
0	TTL expired in transit
1	Fragment reassembly time exceeded

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 792	Internet Control Message Protocol
RFC 1122	Requirements for Internet Hosts -- ICMP requirements
RFC 4443	ICMPv6 for IPv6
RFC 1191	Path MTU Discovery

See Also

- IPv4
- IPv6 -- ICMPv6 replaces ICMP and ARP for IPv6

ICMPv6 -- Internet Control Message Protocol for IPv6

Standard: RFC 4443 | **Layer:** Network (Layer 3) | **Wireshark filter:** `icmpv6`

ICMPv6 is the IPv6 equivalent of ICMP, but with a significantly expanded role. Beyond basic diagnostics (ping, traceroute) and error reporting, ICMPv6 absorbs functions that were separate protocols in IPv4 -- most notably ARP (replaced by Neighbor Discovery) and IGMP (replaced by Multicast Listener Discovery). ICMPv6 is mandatory for IPv6 operation; you cannot firewall it away like you sometimes can with ICMPv4.

Header

[Diagram: see online version]

The checksum covers a pseudo-header (source/dest IPv6 addresses, payload length, next header = 58), the ICMPv6 header, and the message body.

Key Fields

Field	Size	Description
Type	8 bits	Message type (0-127 = error, 128-255 = informational)
Code	8 bits	Subtype providing context
Checksum	16 bits	Over pseudo-header + ICMPv6 message
Message Body	Variable	Type-specific content

Message Types

Error Messages (Type 0-127)

Type	Name	Description
1	Destination Unreachable	Packet could not be delivered
2	Packet Too Big	MTU exceeded (path MTU discovery)
3	Time Exceeded	Hop limit reached (traceroute)
4	Parameter Problem	Malformed header field

Informational Messages (Type 128-255)

Type	Name	Description
128	Echo Request	Ping request
129	Echo Reply	Ping reply
130	Multicast Listener Query	MLD -- router asks who listens (replaces IGMP Query)
131	Multicast Listener Report	MLD -- host reports group membership

132	Multicast Listener Done	MLD -- host leaves multicast group
133	Router Solicitation (RS)	NDP -- host asks for router info
134	Router Advertisement (RA)	NDP -- router announces itself and prefixes
135	Neighbor Solicitation (NS)	NDP -- resolve IPv6 -> MAC (replaces ARP request)
136	Neighbor Advertisement (NA)	NDP -- respond with MAC (replaces ARP reply)
137	Redirect	NDP -- router redirects to a better next-hop
143	MLDv2 Report	Multicast Listener Discovery v2

Neighbor Discovery Protocol (NDP)

NDP is the most critical function of ICMPv6 -- it replaces ARP, ICMP Router Discovery, and ICMP Redirect from IPv4.

Neighbor Solicitation (Type 135) -- "ARP Request"

[Diagram: see online version]

Sent to the solicited-node multicast address `ff02::1:ffXX:XXXX` (last 24 bits of the target IP).

Neighbor Advertisement (Type 136) -- "ARP Reply"

[Diagram: see online version]

Flag	Name	Description
R	Router	1 = sender is a router
S	Solicited	1 = response to a Neighbor Solicitation
O	Override	1 = override existing neighbor cache entry

Address Resolution Flow (Replaces ARP)

[Diagram: see online version]

Router Solicitation (Type 133)

[Diagram: see online version]

Sent by a host at boot to `ff02::2` (all-routers multicast) to request immediate Router Advertisements rather than waiting for the next periodic RA.

Router Advertisement (Type 134)

[Diagram: see online version]

Flag	Name	Description
M	Managed	1 = use DHCPv6 for address configuration
O	Other Config	1 = use DHCPv6 for other info (DNS, NTP)

RA Options

Type	Name	Description
1	Source Link-Layer Address	Router's MAC address
3	Prefix Information	Network prefix for SLAAC (address autoconfiguration)
5	MTU	Link MTU
25	RDNSS	Recursive DNS Server addresses (RFC 8106)
31	DNSSL	DNS Search List (RFC 8106)

Prefix Information Option (Type 3)

[Diagram: see online version]

Flag	Name	Description
L	On-Link	1 = prefix is on-link (hosts can communicate directly)
A	Autonomous	1 = hosts can use this prefix for SLAAC

SLAAC (Stateless Address Autoconfiguration)

[Diagram: see online version]

Duplicate Address Detection (DAD)

Before using a new IPv6 address, the host sends a Neighbor Solicitation for its own address with source `::` (unspecified). If no Neighbor Advertisement comes back, the address is unique.

Destination Unreachable (Type 1)

Code	Meaning
0	No route to destination

1	Communication administratively prohibited
2	Beyond scope of source address
3	Address unreachable
4	Port unreachable
5	Source address failed ingress/egress policy
6	Reject route to destination

Packet Too Big (Type 2)

[Diagram: see online version]

IPv6 does **not** allow routers to fragment packets. If a packet exceeds the link MTU, the router drops it and sends a Packet Too Big message with the MTU value. The source then adjusts its packet size. This is Path MTU Discovery -- it's mandatory in IPv6.

Multicast Listener Discovery (MLD)

MLD replaces IGMP for IPv6 multicast management:

ICMPv6 Type	MLD Equivalent	IGMP Equivalent
130	Multicast Listener Query	IGMP Membership Query
131	Multicast Listener Report (v1)	IGMP Membership Report
132	Multicast Listener Done	IGMP Leave Group
143	MLDv2 Report	IGMPv3 Report

ICMPv6 vs ICMPv4

Function	IPv4	IPv6
Ping	ICMP Echo (type 8/0)	ICMPv6 Echo (type 128/129)
Traceroute	ICMP Time Exceeded (type 11)	ICMPv6 Time Exceeded (type 3)
Path MTU	ICMP Fragmentation Needed (type 3/4)	ICMPv6 Packet Too Big (type 2)
Address resolution	ARP (separate protocol)	NDP Neighbor Solicitation/Advertisement (ICMPv6 135/136)
Router discovery	ICMP Router Solicitation/Advertisement	NDP RS/RA (ICMPv6 133/134)
Redirect	ICMP Redirect (type 5)	ICMPv6 Redirect (type 137)
Multicast membership	IGMP (separate protocol)	MLD (ICMPv6 130-132, 143)
Duplicate address detection	Gratuitous ARP	NDP DAD (ICMPv6 135 with :: source)

Key Multicast Addresses

Address	Scope	Purpose
`ff02::1`	Link-local	All nodes
`ff02::2`	Link-local	All routers
`ff02::1:ffXX:XXXX`	Link-local	Solicited-node (NDP target lookup)
`ff02::16`	Link-local	MLDv2 reports
`ff02::1:2`	Link-local	DHCPv6 servers and relay agents

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 4443	ICMPv6
RFC 4861	Neighbor Discovery for IPv6 (NDP)
RFC 4862	IPv6 Stateless Address Autoconfiguration (SLAAC)
RFC 3810	Multicast Listener Discovery Version 2 (MLDv2)
RFC 8106	IPv6 RA Options for DNS (RDNSS, DNSSL)
RFC 7559	Packet-Too-Big Message for IPv6 (PMTUD)
RFC 3971	SEcure Neighbor Discovery (SEND)

See Also

- IPv6 -- ICMPv6 is mandatory for IPv6 operation
- ICMP -- IPv4 equivalent (much simpler scope)
- ARP -- IPv4 address resolution (replaced by NDP in IPv6)
- IGMP -- IPv4 multicast management (replaced by MLD in IPv6)
- Ethernet -- link-layer addresses resolved by NDP

IGMP -- Internet Group Management Protocol

Standard: RFC 3376 (IGMPv3) | **Layer:** Network (Layer 3) | **Wireshark filter:** `igmp`

IGMP manages multicast group membership between hosts and their local router. When a host wants to receive multicast traffic (e.g., IPTV, live streaming, stock feeds), it sends an IGMP report to join the multicast group. The local router uses this to decide which multicast streams to forward to the local subnet. IGMP operates between hosts and their first-hop router only -- multicast routing between routers uses PIM.

IGMPv3 Message

[Diagram: see online version]

Message Types

Type	Name	Direction	Description
0x11	Membership Query	Router -> Hosts	Ask which groups hosts are subscribed to
0x16	Membership Report (v2)	Host -> Router	Report group membership
0x22	Membership Report (v3)	Host -> Router	Report with source filtering
0x17	Leave Group (v2)	Host -> Router	Leave a multicast group

IGMPv3 Report

[Diagram: see online version]

Group Record

Field	Size	Description
Record Type	8 bits	MODE_IS_INCLUDE, MODE_IS_EXCLUDE, etc.
Aux Data Len	8 bits	Length of auxiliary data (usually 0)
Number of Sources	16 bits	Source addresses for source-specific multicast
Multicast Address	32 bits	Group being joined/left
Source Addresses	32 bits each	Sources to include or exclude

Join/Leave Flow

[Diagram: see online version]

IGMP Snooping

Layer 2 switches examine IGMP messages to learn which ports need multicast traffic, preventing unnecessary flooding to all ports:

[Diagram: see online version]

Multicast Address Ranges

Range	Scope	Description
224.0.0.0/24	Link-local	IGMP, OSPF, VRRP (not forwarded by routers)
224.0.0.1	Link-local	All hosts
224.0.0.2	Link-local	All routers
224.0.0.22	Link-local	IGMPv3 reports
224.0.1.0/24	Internetwork	NTP, SLP, etc.
239.0.0.0/8	Administratively scoped	Private multicast (like RFC 1918 for unicast)

Encapsulation

[Diagram: see online version]

IGMP is carried directly in IP packets with protocol number 2 and TTL=1.

Standards

Document	Title
RFC 3376	IGMPv3
RFC 2236	IGMPv2
RFC 4541	IGMP and MLD Snooping Considerations

See Also

- IPv4 -- IGMP is an IPv4 protocol
- OSPF -- uses multicast 224.0.0.5/6
- VRRP -- uses multicast 224.0.0.18

IPv4 -- Internet Protocol version 4

Standard: RFC 791 | **Layer:** Network (Layer 3) | **Wireshark filter:** `ip`

IP is the principal communications protocol for relaying datagrams across network boundaries. It provides logical addressing and routing, allowing packets to traverse heterogeneous networks to reach a destination identified by a unique IP address. IPv4 uses 32-bit addresses and is the foundation of the modern Internet.

Header

[Diagram: see online version]

Key Fields

Field	Size	Description
Version	4 bits	IP version; always 4 for IPv4
IHL	4 bits	Internet Header Length in 32-bit words (min 5 = 20 bytes)
DSCP	6 bits	Differentiated Services Code Point (QoS)
ECN	2 bits	Explicit Congestion Notification
Total Length	16 bits	Entire packet size in bytes (header + payload), max 65535
Identification	16 bits	Unique ID for reassembling fragmented datagrams
Flags	3 bits	Fragmentation control flags
Fragment Offset	13 bits	Position of fragment in original datagram (in 8-byte units)
TTL	8 bits	Hop limit; decremented by each router, packet discarded at 0
Protocol	8 bits	Upper-layer protocol identifier
Header Checksum	16 bits	Error check of the header only (recomputed at each hop)
Source Address	32 bits	Sender's IPv4 address
Destination Address	32 bits	Recipient's IPv4 address
Options	Variable	Optional; rarely used in practice

Field Details

DSCP / ECN (formerly Type of Service)

The original ToS byte was redefined by RFC 2474 (DSCP) and RFC 3168 (ECN):

[Diagram: see online version]

Legacy ToS Precedence values (bits 0-2 of original ToS):

Value	Meaning
111	Network Control
110	Internetwork Control
101	CRITIC/ECP
100	Flash Override
011	Flash
010	Immediate
001	Priority
000	Routine

Flags

[Diagram: see online version]

Bit	Name	Description
0	Reserved	Must be zero
1	DF	Don't Fragment -- 0 = may fragment, 1 = don't fragment
2	MF	More Fragments -- 0 = last fragment, 1 = more fragments follow

Protocol

Identifies the upper-layer protocol encapsulated in the payload. Assigned by IANA.

Value	Protocol
1	ICMP
2	IGMP
6	TCP
17	UDP
41	IPv6 encapsulation
47	GRE
50	ESP (IPsec)
51	AH (IPsec)
89	OSPF
132	SCTP

IP Addressing

IPv4 uses 32-bit addresses written in dotted-decimal notation (e.g., `192.168.1.1`).

Range	Purpose
10.0.0.0/8	Private (RFC 1918)
172.16.0.0/12	Private (RFC 1918)
192.168.0.0/16	Private (RFC 1918)
127.0.0.0/8	Loopback
169.254.0.0/16	Link-local
224.0.0.0/4	Multicast
255.255.255.255	Broadcast

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 791	Internet Protocol -- primary specification
RFC 1918	Address Allocation for Private Internets
RFC 2474	Definition of the Differentiated Services Field (DSCP)
RFC 3168	The Addition of Explicit Congestion Notification (ECN) to IP
RFC 8200	Internet Protocol, Version 6 (IPv6) Specification

See Also

- IPv6
- ICMP
- TCP
- UDP
- Ethernet

IPsec -- Internet Protocol Security

Standard: RFC 4301 | **Layer:** Network (Layer 3) | **Wireshark filter:** `esp` or `ah` or `isakmp`

IPsec is a suite of protocols that provides encryption, authentication, and integrity for IP packets. It operates at the network layer, transparently protecting all traffic between two endpoints without changes to applications. IPsec is the foundation of site-to-site VPNs, remote access VPNs, and host-to-host encryption. It consists of two transport protocols (ESP and AH), a key exchange protocol (IKE), and a Security Association database.

Components

[Diagram: see online version]

ESP Header (Encapsulating Security Payload)

ESP is the primary IPsec protocol -- it provides both encryption and authentication:

[Diagram: see online version]

Field	Size	Description
SPI	32 bits	Identifies the Security Association (SA)
Sequence Number	32 bits	Anti-replay protection (monotonically increasing)
Payload Data	Variable	Encrypted original packet (or transport header + data)
Padding	0-255 bytes	Aligns to block cipher boundary
Pad Length	8 bits	Number of padding bytes
Next Header	8 bits	Protocol of the encrypted payload (TCP=6, UDP=17, etc.)
ICV	Variable	Integrity check (HMAC or AEAD tag, typically 12-16 bytes)

The SPI + Sequence Number (8 bytes) are not encrypted. Everything from Payload Data through Next Header is encrypted. The ICV covers everything from SPI through Next Header.

AH Header (Authentication Header)

AH provides authentication and integrity but **no encryption** (rarely used today):

[Diagram: see online version]

Modes

Transport Mode

Encrypts only the payload -- the original IP header is preserved:

[Diagram: see online version]

Used for: host-to-host encryption, L2TP/IPsec VPN.

Tunnel Mode

Encrypts the entire original packet and wraps it with a new IP header:

[Diagram: see online version]

Used for: site-to-site VPN, remote access VPN.

IKEv2 (Internet Key Exchange)

IKE negotiates Security Associations and manages keys. IKEv2 uses UDP port 500 (or 4500 for NAT traversal):

IKEv2 Header

[Diagram: see online version]

IKEv2 Exchanges

[Diagram: see online version]

Exchange Types

Type	Name	Description
34	IKE_SA_INIT	Negotiate IKE SA parameters and DH exchange
35	IKE_AUTH	Authenticate and create first Child SA
36	CREATE_CHILD_SA	Create additional SAs or rekey
37	INFORMATIONAL	Delete SAs, notify errors, keepalive

Security Association (SA)

An SA is a unidirectional agreement defining:

Parameter	Description
SPI	Unique identifier for this SA
Protocol	ESP or AH
Encryption algorithm	AES-CBC, AES-GCM, ChaCha20-Poly1305, etc.
Integrity algorithm	HMAC-SHA256, HMAC-SHA384, or AEAD implicit
Key material	Encryption and authentication keys
Lifetime	Time or byte limit before rekeying

Mode	Transport or Tunnel
Traffic selectors	Which traffic this SA protects (IP ranges, ports)

Common Algorithms

Category	Algorithms
Encryption	AES-128-GCM (preferred), AES-256-GCM, AES-CBC, ChaCha20-Poly1305
Integrity	SHA-256, SHA-384, SHA-512 (or implicit with AEAD)
DH Groups	Group 14 (2048-bit MODP), Group 19 (256-bit ECP), Group 20 (384-bit ECP)
Authentication	RSA signatures, ECDSA, PSK (pre-shared key), EAP

NAT Traversal (NAT-T)

When NAT is detected (via NAT_DETECTION_SOURCE_IP/DESTINATION_IP payloads), IKE switches from UDP 500 to **UDP 4500**, and ESP packets are encapsulated in UDP:

[Diagram: see online version]

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 4301	Security Architecture for IP
RFC 4303	IP Encapsulating Security Payload (ESP)
RFC 4302	IP Authentication Header (AH)
RFC 7296	Internet Key Exchange Protocol Version 2 (IKEv2)
RFC 3948	UDP Encapsulation of IPsec ESP Packets (NAT-T)
RFC 7383	IKEv2 Message Fragmentation

See Also

- IPv4 -- IPsec protects IP traffic (protocol 50/51)
- GRE -- commonly paired with IPsec for multiprotocol VPN
- TLS -- application-layer alternative to IPsec
- UDP -- IKE and NAT-T use UDP

IPv6 -- Internet Protocol version 6

Standard: RFC 8200 | **Layer:** Network (Layer 3) | **Wireshark filter:** `ipv6`

IPv6 is the successor to IPv4, designed to address the exhaustion of the 32-bit IPv4 address space. It uses 128-bit addresses, providing approximately 3.4 x 10^38 unique addresses. IPv6 simplifies the header format compared to IPv4 (removing the checksum, options, and fragmentation fields from the base header), supports mandatory IPsec, and introduces extension headers for optional features.

Header

[Diagram: see online version]

The fixed header is always 40 bytes (320 bits), with no variable-length options -- a deliberate simplification over IPv4.

Key Fields

Field	Size	Description
Version	4 bits	IP version; always 6
Traffic Class	6 bits	Differentiated Services (DSCP) for QoS
ECN	2 bits	Explicit Congestion Notification
Flow Label	20 bits	Identifies a flow for special handling by routers
Payload Length	16 bits	Size of the payload in bytes (excludes the 40-byte header)
Next Header	8 bits	Identifies the type of the next header (extension or upper-layer)
Hop Limit	8 bits	Equivalent to IPv4 TTL; decremented at each hop
Source Address	128 bits	Sender's IPv6 address
Destination Address	128 bits	Recipient's IPv6 address

Field Details

Next Header

Uses the same values as IPv4's Protocol field (IANA registry), plus extension header types:

Value	Header Type
0	Hop-by-Hop Options
6	TCP
17	UDP
43	Routing
44	Fragment
50	ESP (IPsec)
51	AH (IPsec)
58	ICMPv6
59	No Next Header
60	Destination Options
135	Mobility

Extension Headers

IPv6 replaces IPv4's variable-length options with a chain of extension headers. Each header contains a Next Header field pointing to the next one:

[Diagram: see online version]

Recommended processing order per RFC 8200:

1. Hop-by-Hop Options (examined by every node)
2. Destination Options (for first destination)
3. Routing
4. Fragment
5. Authentication (AH)
6. Encapsulating Security Payload (ESP)
7. Destination Options (for final destination)
8. Upper-layer header (TCP, UDP, ICMPv6, etc.)

IPv6 Addressing

128-bit addresses written in colon-separated hexadecimal (e.g., `2001:0db8:85a3::8a2e:0370:7334`).

Prefix	Purpose
`::1/128`	Loopback
`::/0`	Default route
`fe80::/10`	Link-local
`fc00::/7`	Unique local (ULA)
`2000::/3`	Global unicast
`ff00::/8`	Multicast

Key Differences from IPv4

Feature	IPv4	IPv6
Address size	32 bits	128 bits
Header size	20-60 bytes (variable)	40 bytes (fixed)
Header checksum	Yes	No (removed, relies on link-layer and transport)
Fragmentation	Routers and sender	Sender only (via extension header)
ARP	Yes	No (replaced by NDP/ICMPv6)
Broadcast	Yes	No (replaced by multicast)
IPsec	Optional	Mandatory support

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 8200	Internet Protocol, Version 6 (IPv6) Specification
RFC 4291	IP Version 6 Addressing Architecture
RFC 4443	ICMPv6 for IPv6
RFC 4861	Neighbor Discovery for IPv6 (NDP)
RFC 4862	IPv6 Stateless Address Autoconfiguration (SLAAC)
RFC 3587	IPv6 Global Unicast Address Format

See Also

- IPv4
- TCP
- UDP
- Ethernet

IS-IS -- Intermediate System to Intermediate System

Standard: ISO 10589 / RFC 5308 | **Layer:** Network (Layer 3) | **Wireshark filter:** `isis`

IS-IS is a link-state interior gateway routing protocol that runs directly over the data link layer (not over IP). Originally designed for OSI networking (CLNP), it was extended to support IPv4 and IPv6 and is now one of the two dominant IGPs alongside OSPF. IS-IS is the preferred IGP for large service provider and data center networks due to its protocol-agnostic design, fast convergence, and efficient flooding. It is the routing protocol of choice for MPLS and Segment Routing deployments.

PDU Types

Type	Name	Description
15	L1 LAN Hello	Level 1 hello on broadcast networks
16	L2 LAN Hello	Level 2 hello on broadcast networks
17	P2P Hello	Hello on point-to-point links
18	L1 LSP	Level 1 Link State PDU
20	L2 LSP	Level 2 Link State PDU
24	L1 CSNP	Level 1 Complete Sequence Numbers PDU
25	L2 CSNP	Level 2 Complete Sequence Numbers PDU
26	L1 PSNP	Level 1 Partial Sequence Numbers PDU
27	L2 PSNP	Level 2 Partial Sequence Numbers PDU

Common Header

[Diagram: see online version]

Key Concepts

Hierarchy -- Levels

Level	Scope	Equivalent
Level 1 (L1)	Intra-area routing	OSPF intra-area
Level 2 (L2)	Inter-area (backbone) routing	OSPF backbone (area 0)
L1/L2	Routes between levels (like OSPF ABR)	Area Border Router

[Diagram: see online version]

NET (Network Entity Title)

IS-IS uses a CLNP-style address called a NET:

```
49.0001.1921.6800.1001.00
```

Component	Value	Description
AFI	49	Private addressing (most common)
Area ID	0001	IS-IS area identifier
System ID	1921.6800.1001	Unique per router (often derived from loopback IP)
SEL	00	NSAP selector (always 00 for IS-IS)

IS-IS vs OSPF

Feature	IS-IS	OSPF
Runs on	Directly over L2 (no IP needed)	Over IP (protocol 89)
Addressing	NET / NSAP	IP-based (router ID)
Levels/Areas	L1 (area) / L2 (backbone)	Area 0 backbone + numbered areas
Multi-protocol	Native (IPv4, IPv6, MPLS via TLVs)	Separate: OSPFv2 (IPv4), OSPFv3 (IPv6)
LSA/LSP size	Up to 1492 bytes	Up to ~65535 bytes
Extension mechanism	TLVs (easy to extend)	LSA types (more rigid)
Provider preference	Dominant in SP networks	Common in enterprise
Segment Routing	Native TLV support	Supported but IS-IS more common

TLV (Type-Length-Value) Extensions

IS-IS is extended via TLVs -- adding new capabilities without changing the core protocol:

TLV	Name	Description
2	IS Neighbors	Adjacencies to other IS-IS routers
128	IP Internal Reachability	IPv4 prefixes (old format)
130	IP External Reachability	External IPv4 prefixes
135	Extended IP Reachability	IPv4 prefixes with TE metrics (wide metrics)
137	Dynamic Hostname	Hostname mapping for router
236	IPv6 Reachability	IPv6 prefixes
22	Extended IS Reachability	Adjacencies with TE metrics

Encapsulation

[Diagram: see online version]

IS-IS does **not** run over IP. It runs directly over the data link layer using:

- EtherType `0xFE` (ISO/LLC/SNAP encapsulation on Ethernet)
- Multicast MACs: `0180.C200.0014` (L1) and `0180.C200.0015` (L2)

Standards

Document	Title
ISO 10589	IS-IS for intra-domain routing (original)
RFC 1195	Use of IS-IS for routing in TCP/IP and dual environments
RFC 5305	IS-IS Extensions for Traffic Engineering
RFC 5308	Routing IPv6 with IS-IS
RFC 8667	IS-IS Extensions for Segment Routing

See Also

- OSPF -- alternative link-state IGP
- BGP -- used alongside IS-IS for inter-AS routing
- MPLS -- IS-IS commonly drives MPLS label distribution
- Ethernet -- IS-IS runs directly on Ethernet

MPLS -- Multiprotocol Label Switching

Standard: RFC 3031 | **Layer:** Between Layer 2 and 3 ("Layer 2.5") | **Wireshark filter:** `mpls`

MPLS is a high-performance packet forwarding mechanism used in carrier and enterprise backbone networks. Instead of performing a full IP routing lookup at each hop, MPLS routers (Label Switch Routers) forward packets based on short fixed-length labels. This enables traffic engineering, fast reroute, VPNs (L3VPN, L2VPN/VPLS), and QoS at scale. MPLS sits between the data link and network layers -- often called "Layer 2.5."

Label Stack Entry

[Diagram: see online version]

Each label stack entry is exactly 4 bytes (32 bits). Labels can be stacked -- multiple entries form a label stack, processed last-in-first-out.

Key Fields

Field	Size	Description
Label	20 bits	Forwarding identifier (0-1048575)
TC (Traffic Class)	3 bits	QoS / DiffServ (formerly EXP bits)
S (Bottom of Stack)	1 bit	1 = this is the last (bottom) label in the stack
TTL	8 bits	Time to Live -- decremented at each LSR

Reserved Label Values

Label	Name	Description
0	IPv4 Explicit Null	Pop label, use IPv4 DSCP for QoS
1	Router Alert	Deliver to local control plane
2	IPv6 Explicit Null	Pop label, use IPv6 TC for QoS
3	Implicit Null	Penultimate Hop Popping (PHP) -- pop before final hop
4-6	Reserved	
7	Entropy Label Indicator	Signals next label is entropy (RFC 6790)
13	GAL (Generic Associated Channel)	OAM channel (RFC 5586)
14	OAM Alert	MPLS OAM
15	Extension	Extended label space

Label Operations

Operation	Description
Push	Add a label to the top of the stack (ingress)
Swap	Replace the top label with a new one (transit)
Pop	Remove the top label (egress or PHP)

Forwarding Example

[Diagram: see online version]

Label Distribution

Labels are distributed between LSRs using signaling protocols:

Protocol	RFC	Description
LDP	RFC 5036	Label Distribution Protocol -- automatic label assignment
RSVP-TE	RFC 3209	Resource Reservation Protocol with Traffic Engineering
MP-BGP	RFC 4364	Labels distributed via BGP for L3VPN
Segment Routing	RFC 8402	Labels encoded in the packet by the ingress (no signaling)

LDP Session

[Diagram: see online version]

Label Stack (Multiple Labels)

MPLS supports stacked labels for nested services:

[Diagram: see online version]

Common two-label stacks:

Outer Label	Inner Label	Use Case
Transport (LDP/RSVP)	VPN (BGP)	L3VPN (RFC 4364)
Transport	PW (pseudowire)	L2VPN / VPLS
Transport	Entropy	Load balancing

MPLS Applications

L3VPN (RFC 4364)

[Diagram: see online version]

- PE routers maintain per-customer VRFs (Virtual Routing and Forwarding tables)
- MP-BGP distributes VPN labels between PE routers
- Customer routes are isolated from each other and from the core

Traffic Engineering (RSVP-TE)

- Explicit paths through the network (not shortest-path)
- Bandwidth reservation
- Fast Reroute (FRR) -- sub-50ms failover

Segment Routing

Modern alternative to LDP/RSVP-TE:

- No label distribution protocol needed
- Source routing -- ingress encodes the full path as a label stack
- Works with MPLS data plane (SR-MPLS) or IPv6 (SRv6)

Encapsulation

[Diagram: see online version]

EtherType	Description
0x8847	MPLS unicast
0x8848	MPLS multicast

Standards

Document	Title
RFC 3031	Multiprotocol Label Switching Architecture
RFC 3032	MPLS Label Stack Encoding
RFC 5036	LDP Specification
RFC 3209	RSVP-TE: Extensions for LSP Tunnels
RFC 4364	BGP/MPLS IP Virtual Private Networks (L3VPN)
RFC 8402	Segment Routing Architecture
RFC 5921	MPLS Transport Profile (MPLS-TP) Framework

See Also

- Ethernet -- carries MPLS frames
- IPv4 -- the network layer MPLS often encapsulates
- BGP -- distributes VPN labels in L3VPN
- OSPF -- IGP that drives LDP label assignment

OSPF -- Open Shortest Path First

Standard: RFC 2328 | **Layer:** Network (Layer 3) | **Wireshark filter:** `ospf`

OSPF is a link-state interior gateway protocol (IGP) used for routing within a single autonomous system. Each router maintains a complete topology map of the network (the link-state database) and uses Dijkstra's SPF algorithm to compute shortest paths. OSPF supports hierarchical routing through areas, VLSM/CIDR, equal-cost multi-path

routing, and fast convergence. It runs directly over IP (protocol number 89), not over TCP or UDP. OSPFv3 (RFC 5340) extends OSPF for IPv6.

Header

All OSPF packets share a common 24-byte header:

[Diagram: see online version]

Key Fields

Field	Size	Description
Version	8 bits	OSPF version (2 for OSPFv2, 3 for OSPFv3)
Type	8 bits	Packet type
Packet Length	16 bits	Total packet length including header
Router ID	32 bits	Unique identifier of the originating router
Area ID	32 bits	OSPF area this packet belongs to (0.0.0.0 = backbone)
Checksum	16 bits	Standard IP checksum over the packet (excluding auth)
Auth Type	16 bits	Authentication type
Authentication	64 bits	Authentication data

Field Details

Packet Types

Type	Name	Description
1	Hello	Neighbor discovery and keepalive
2	Database Description (DBD)	Summary of link-state database
3	Link State Request (LSR)	Request specific LSAs
4	Link State Update (LSU)	Carry one or more LSAs
5	Link State Acknowledgment (LSAck)	Acknowledge received LSAs

Hello Packet

Sent periodically (default every 10 seconds on broadcast networks) to discover and maintain neighbor relationships:

Field	Description

Network Mask	Subnet mask of the interface
Hello Interval	Seconds between Hello packets
Options	Optional capabilities (E, MC, N/P, DC, EA)
Router Priority	Priority for DR/BDR election (0 = ineligible)
Dead Interval	Seconds before declaring a neighbor dead (default 40)
Designated Router	IP of the current DR
Backup Designated Router	IP of the current BDR
Neighbor List	Router IDs of known neighbors

Authentication Types

Value	Type
0	Null (no authentication)
1	Simple password (plaintext, 8 bytes)
2	Cryptographic (MD5 or SHA, per RFC 5709)

LSA Types

Type	Name	Description
1	Router LSA	Links of a router within an area
2	Network LSA	Routers attached to a multi-access network (from DR)
3	Summary LSA (Network)	Inter-area routes (from ABR)
4	Summary LSA (ASBR)	Route to an ASBR (from ABR)
5	AS External LSA	Routes external to the AS (from ASBR)
7	NSSA External LSA	External routes in Not-So-Stubby Areas

Neighbor State Machine

[Diagram: see online version]

Encapsulation

[Diagram: see online version]

OSPF uses multicast addresses `224.0.0.5` (AllSPFRouters) and `224.0.0.6` (AllDRouters).

Standards

Document	Title
RFC 2328	OSPF Version 2
RFC 5340	OSPF for IPv6 (OSPFv3)
RFC 3101	OSPF Not-So-Stubby Areas (NSSA)
RFC 5709	OSPFv2 HMAC-SHA Cryptographic Authentication
RFC 2370	OSPF Opaque LSAs

See Also

- IPv4
- BGP -- exterior gateway protocol (between autonomous systems)

VRRP -- Virtual Router Redundancy Protocol

Standard: RFC 5798 | **Layer:** Network (Layer 3) | **Wireshark filter:** `vrrp`

VRRP provides automatic default gateway failover for hosts on a LAN. Two or more routers share a virtual IP address and a virtual MAC address. One router is elected Master and handles all traffic to the virtual IP; the others are Backups that take over within seconds if the Master fails. VRRP is essential for high-availability network designs -- virtually every enterprise and data center network uses it (or Cisco's proprietary HSRP) for gateway redundancy.

Packet

[Diagram: see online version]

Key Fields

Field	Size	Description
Version	4 bits	VRRP version (3 for VRRPv3)
Type	4 bits	1 = Advertisement (only type defined)
VRID	8 bits	Virtual Router ID (1-255) -- identifies the VRRP group
Priority	8 bits	1-254 (higher = more preferred); 255 = IP address owner
Count IP Addrs	8 bits	Number of virtual IP addresses
Max Advert Interval	12 bits	Centiseconds between advertisements (default 100 = 1 second)

Checksum	16 bits	Checksum over the VRRP packet
IP Addresses	32 bits each	The virtual IP address(es) this VRID protects

How VRRP Works

[Diagram: see online version]

Election and Failover

[Diagram: see online version]

Priority

Priority	Meaning
255	Router owns the virtual IP (IP address is configured on this router's interface)
101-254	High priority (manually configured)
100	Default priority
1-99	Low priority
0	Master is shutting down (triggers immediate failover)

Virtual MAC Address

VRRP uses a well-known MAC address based on the VRID:

IP Version	Virtual MAC Format
IPv4	`00-00-5E-00-01-{VRID}`
IPv6	`00-00-5E-00-02-{VRID}`

This ensures that the MAC doesn't change during failover -- hosts don't need to update their ARP caches.

Preemption

When a higher-priority router recovers, it can preempt the current Master:

Setting	Behavior
Preempt (default)	Higher-priority router takes over immediately
No-preempt	Current Master keeps the role until it fails

VRRP vs HSRP

Feature	VRRP	HSRP (Cisco)
Standard	IETF (RFC 5798)	Cisco proprietary
Default priority	100	100
Timers	Advertise: 1s, holddown: ~3s	Hello: 3s, holddown: 10s
Multicast	224.0.0.18 (IPv4), ff02::12 (IPv6)	224.0.0.2 (v1), 224.0.0.102 (v2)
Virtual MAC	00-00-5E-00-01-xx	00-00-0C-07-AC-xx
IP owner	Supported (priority 255)	Not supported
IPv6	Native (VRRPv3)	HSRPv2

Encapsulation

[Diagram: see online version]

VRRP is carried directly in IP packets with protocol number 112, TTL=255, sent to multicast address `224.0.0.18`.

Standards

Document	Title
RFC 5798	Virtual Router Redundancy Protocol (VRRP) Version 3
RFC 3768	VRRP Version 2 (IPv4 only)

See Also

- IPv4
- ARP -- gratuitous ARP used during failover
- OSPF -- routing protocol often used alongside VRRP
- IGMP -- VRRP uses multicast 224.0.0.18

Remote Access

Protocols for remote login, desktop access, and terminal services.

2 protocols in this section

SSH -- Secure Shell

Standard: RFC 4253 | **Layer:** Application (Layer 7) | **Wireshark filter:** `ssh`

SSH provides encrypted remote login, command execution, and tunneling over an insecure network. It replaced older unencrypted protocols like Telnet and rlogin. SSH is built from three sub-protocols layered on top of each other: the Transport Layer Protocol (key exchange and encryption), the User Authentication Protocol, and the Connection Protocol (multiplexed channels for sessions, port forwarding, etc.).

Binary Packet

Every SSH message after the version exchange is wrapped in a binary packet:

[Diagram: see online version]

Key Fields

Field	Size	Description
Packet Length	32 bits	Length of packet excluding itself and MAC
Padding Length	8 bits	Length of random padding
Payload	Variable	The SSH message data
Random Padding	Variable	Padding to align to cipher block size (min 4 bytes)
MAC	Variable	Message Authentication Code (if negotiated)

After encryption is negotiated, the entire packet (except MAC) is encrypted and the MAC covers the unencrypted data plus a sequence number.

Protocol Layers

[Diagram: see online version]

Field Details

Version Exchange

Before binary packets begin, both sides exchange a plaintext version string:

```
SSH-2.0-OpenSSH_9.6\r\n
```

Format: `SSH-protoversion-softwareversion [comments]\r\n`

Transport Layer Message Types

Value	Name	Description
1	SSH_MSG_DISCONNECT	Connection terminated
2	SSH_MSG_IGNORE	Ignored data (anti-traffic analysis)
20	SSH_MSG_KEXINIT	Key exchange begins
21	SSH_MSG_NEWKEYS	Key exchange complete, switch to new keys
30-49		Key exchange method-specific messages

Key Exchange (KEXINIT)

The KEXINIT message contains ordered lists of supported algorithms:

Field	Description
cookie	16 random bytes
kex_algorithms	Key exchange methods (e.g., curve25519-sha256)
server_host_key_algorithms	Host key types (e.g., ssh-ed25519, rsa-sha2-512)
encryption_algorithms_c2s	Ciphers client-to-server (e.g., aes256-gcm)
encryption_algorithms_s2c	Ciphers server-to-client
mac_algorithms_c2s	MAC algorithms client-to-server
mac_algorithms_s2c	MAC algorithms server-to-client
compression_algorithms_c2s	Compression client-to-server
compression_algorithms_s2c	Compression server-to-client

User Authentication Message Types

Value	Name	Description
50	SSH_MSG_USERAUTH_REQUEST	Client authentication attempt
51	SSH_MSG_USERAUTH_FAILURE	Authentication failed (lists remaining methods)
52	SSH_MSG_USERAUTH_SUCCESS	Authentication succeeded
60	SSH_MSG_USERAUTH_PK_OK	Public key is acceptable

Authentication methods: `publickey`, `password`, `keyboard-interactive`, `hostbased`

Connection Protocol Message Types

Value	Name	Description
90	SSH_MSG_CHANNEL_OPEN	Open a new channel
91	SSH_MSG_CHANNEL_OPEN_CONFIRMATION	Channel opened
93	SSH_MSG_CHANNEL_WINDOW_ADJUST	Flow control window update
94	SSH_MSG_CHANNEL_DATA	Data on a channel
96	SSH_MSG_CHANNEL_EOF	No more data
97	SSH_MSG_CHANNEL_CLOSE	Close the channel
98	SSH_MSG_CHANNEL_REQUEST	Channel-specific request (shell, exec, subsystem)

Common Algorithms (OpenSSH defaults)

Category	Algorithms
Key Exchange	curve25519-sha256, diffie-hellman-group16-sha512
Host Key	ssh-ed25519, rsa-sha2-512, rsa-sha2-256
Cipher	chacha20-poly1305, aes256-gcm, aes128-gcm
MAC	(implicit with AEAD ciphers), hmac-sha2-256-etm

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 4251	SSH Protocol Architecture
RFC 4252	SSH Authentication Protocol
RFC 4253	SSH Transport Layer Protocol
RFC 4254	SSH Connection Protocol
RFC 4256	Generic Message Exchange Authentication (keyboard-interactive)
RFC 8709	Ed25519 and Ed448 Public Key Algorithms for SSH
RFC 8332	Use of RSA Keys with SHA-256 and SHA-512
RFC 8731	Curve25519 and Curve448 Key Exchange for SSH

See Also

- TCP
- TLS -- alternative encryption layer for other protocols

Telnet

Standard: RFC 854 | **Layer:** Application (Layer 7) | **Wireshark filter:** `telnet`

Telnet provides bidirectional, interactive text-oriented communication over TCP. It was the original remote login protocol of the Internet, allowing a user to operate a command-line interface on a remote machine as if sitting at the console. Telnet transmits everything -- including passwords -- in plaintext, making it fundamentally insecure. It has been almost entirely replaced by SSH for remote access, but remains in use for network device management (router/switch consoles), legacy mainframe access (TN3270/TN5250), and as the underlying protocol for MUDs and BBS systems. Many application protocols (SMTP, HTTP, POP3) use Telnet-like command syntax.

Protocol Overview

Telnet has no fixed binary header. It uses a simple in-band command mechanism where the byte `0xFF` (IAC -- Interpret As Command) signals a command sequence within the data stream:

Command Sequence

[Diagram: see online version]

Key Fields

Field	Size	Description
IAC	8 bits	0xFF -- Interpret As Command (escape byte)
Command	8 bits	Telnet command code
Option	8 bits	Option code (for WILL/WONT/DO/DONT)

All other bytes in the stream are user data (terminal I/O). A literal `0xFF` in user data is escaped as `0xFF 0xFF`.

Commands

Code	Name	Description
240 (0xF0)	SE	Sub-negotiation End

241 (0xF1)	NOP	No Operation
242 (0xF2)	DM	Data Mark (synch signal)
243 (0xF3)	BRK	Break
244 (0xF4)	IP	Interrupt Process
245 (0xF5)	AO	Abort Output
246 (0xF6)	AYT	Are You There?
247 (0xF7)	EC	Erase Character
248 (0xF8)	EL	Erase Line
249 (0xF9)	GA	Go Ahead (half-duplex signal)
250 (0xFA)	SB	Sub-negotiation Begin
251 (0xFB)	WILL	Sender will perform option
252 (0xFC)	WONT	Sender will not perform option
253 (0xFD)	DO	Sender requests receiver to perform option
254 (0xFE)	DONT	Sender requests receiver to stop option
255 (0xFF)	IAC	Interpret As Command (or data escape when doubled)

Option Negotiation

Options are negotiated using a four-way model:

[Diagram: see online version]

Common Options

Code	Name	RFC	Description
0	Binary	RFC 856	8-bit clean transmission
1	Echo	RFC 857	Remote echo of typed characters
3	Suppress Go Ahead	RFC 858	Full-duplex operation
5	Status	RFC 859	Report option status
24	Terminal Type	RFC 1091	Negotiate terminal emulation type
25	End of Record	RFC 885	Mark record boundaries
31	Window Size (NAWS)	RFC 1073	Transmit terminal dimensions
32	Terminal Speed	RFC 1079	Report connection speed
34	Linemode	RFC 1184	Line-at-a-time input processing

36	Environment	RFC 1408	Pass environment variables
39	New Environment	RFC 1572	Extended environment variable exchange

Sub-negotiation Example (Terminal Type)

```
IAC SB TERMINAL-TYPE SEND IAC SE          (server asks)
IAC SB TERMINAL-TYPE IS "xterm-256color" IAC SE   (client responds)
```

Session Flow

[Diagram: see online version]

NVT (Network Virtual Terminal)

Telnet defines a Network Virtual Terminal as a common reference point:

Feature	NVT Default
Character set	US-ASCII (7-bit)
End of line	CR LF (0x0D 0x0A)
Echo	Local (until negotiated otherwise)
Mode	Half-duplex with Go Ahead (until suppressed)

Telnet Variants

Variant	Description
TN3270	IBM 3270 terminal emulation (mainframe access)
TN5250	IBM 5250 terminal emulation (AS/400 access)
Telnet over TLS	Encrypted Telnet (port 992, rarely used -- SSH preferred)

Security

Telnet has **no encryption whatsoever**. All data -- including usernames, passwords, and session content -- is transmitted in plaintext and trivially captured by anyone on the network path. This is why SSH replaced Telnet for virtually all remote administration.

Threat	Impact
Credential theft	Passwords visible in packet captures
Session hijacking	Attacker can inject commands mid-session

Eavesdropping	All commands and output readable
MITM	No server authentication

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 854	Telnet Protocol Specification
RFC 855	Telnet Option Specifications
RFC 856	Telnet Binary Transmission
RFC 857	Telnet Echo Option
RFC 858	Telnet Suppress Go Ahead
RFC 1091	Telnet Terminal-Type Option
RFC 1073	Telnet Window Size Option (NAWS)
RFC 2217	Telnet Com Port Control (serial port redirection)

See Also

- SSH -- encrypted replacement for Telnet
- TCP

Robotics

Communication protocols used in robotics and autonomous systems.

5 protocols in this section

CANopen

Standard: CiA 301 (CAN in Automation) | **Layer:** Application (Layer 7, over CAN) | **Wireshark filter:** canopen

CANopen is a higher-layer protocol running on CAN bus that provides standardized device profiles, network management, and data exchange for embedded systems. It defines how to configure devices (Object Dictionary via SDO), exchange real-time process data (PDO), handle errors (EMCY), and synchronize operations (SYNC/TIME). CANopen is widely used in robotics (joint controllers, grippers), medical devices, maritime, and industrial automation.

Communication Model

[Diagram: see online version]

CAN-ID Allocation (Predefined Connection Set)

CANopen uses the 11-bit CAN identifier to encode function code + node ID:

[Diagram: see online version]

CAN-ID Range	Function Code	Object	Direction
0x000	0x0	NMT	Master -> all
0x080	0x1	SYNC	Master -> all
0x080 + NodeID	0x1	EMCY	Node -> all
0x180 + NodeID	0x3	TPDO1	Node -> bus
0x200 + NodeID	0x4	RPDO1	Bus -> node
0x280 + NodeID	0x5	TPDO2	Node -> bus
0x300 + NodeID	0x6	RPDO2	Bus -> node
0x380 + NodeID	0x7	TPDO3	Node -> bus
0x400 + NodeID	0x8	RPDO3	Bus -> node
0x480 + NodeID	0x9	TPDO4	Node -> bus
0x500 + NodeID	0xA	RPDO4	Bus -> node
0x580 + NodeID	0xB	SDO response	Node -> master
0x600 + NodeID	0xC	SDO request	Master -> node
0x700 + NodeID	0xE	Heartbeat/NMT Error Control	Node -> all

Object Dictionary

Every CANopen device has an Object Dictionary (OD) -- a structured database of all configuration parameters and process data, indexed by 16-bit index + 8-bit sub-index:

Index Range	Category	Description
0x0000	--	Not used
0x1000-0x1FFF	Communication	Device type, error register, heartbeat, SDO/PDO config
0x2000-0x5FFF	Manufacturer-specific	Vendor-defined parameters
0x6000-0x9FFF	Standardized device profile	Profile-specific (e.g., CiA 402 motion control)

Key Objects

Index	Name	Description
0x1000	Device Type	Device type and profile number
0x1001	Error Register	Current error status
0x1008	Manufacturer Device Name	Human-readable name
0x1017	Producer Heartbeat Time	Heartbeat interval (ms)
0x1400	RPDO1 Communication Parameter	CAN-ID, transmission type
0x1600	RPDO1 Mapping Parameter	Which OD entries map to PDO
0x1800	TPDO1 Communication Parameter	CAN-ID, transmission type
0x1A00	TPDO1 Mapping Parameter	Which OD entries map to PDO

SDO (Service Data Object)

SDO provides confirmed read/write access to any Object Dictionary entry -- used for configuration:

SDO Protocol

[Diagram: see online version]

Command	CS byte	Description
Download initiate (write)	0x22/0x23	Client writes to server
Download response	0x60	Server confirms write
Upload initiate (read)	0x40	Client requests read
Upload response	0x42/0x43	Server returns data
Abort	0x80	Error (abort code in data)

SDO Example (Read Device Type)

[Diagram: see online version]

PDO (Process Data Object)

PDOs carry real-time process data without protocol overhead -- pure CAN data frames mapped to Object Dictionary entries:

Feature	SDO	PDO
Purpose	Configuration	Real-time data
Overhead	4 bytes protocol + data	Data only (0-8 bytes)
Confirmation	Yes (request/response)	No (broadcast)
Trigger	On request	Event, timer, SYNC, or remote request
Typical data	Parameters, firmware	Position, velocity, I/O states

PDO Transmission Types

Type	Trigger	Description
0	Acyclic on SYNC	Transmitted on next SYNC after data change
1-240	Cyclic on Nth SYNC	Transmitted every N SYNC messages
254	Event-driven (manufacturer)	Vendor-defined trigger
255	Event-driven (profile)	Profile-defined trigger

NMT (Network Management)

NMT controls the state machine of all nodes:

[Diagram: see online version]

NMT Commands (CAN-ID 0x000, 2 bytes)

CS	Command	Description
0x01	Start Remote Node	Enter Operational
0x02	Stop Remote Node	Enter Stopped
0x80	Enter Pre-Operational	Leave Operational
0x81	Reset Node	Full reset
0x82	Reset Communication	Reset communication layer

Node ID 0 = all nodes (broadcast).

Heartbeat

Each node periodically sends a heartbeat (CAN-ID = 0x700 + NodeID):

[Diagram: see online version]

State	Value	Description
Boot-up	0x00	Node just started
Stopped	0x04	Node is stopped
Operational	0x05	Node is operational
Pre-Operational	0x7F	Node is pre-operational

Device Profiles

CiA Profile	Application	Description
CiA 401	Generic I/O	Digital/analog I/O modules
CiA 402	Motion Control	Servo drives, stepper motors (most common in robotics)
CiA 404	Measuring Devices	Encoders, sensors
CiA 406	Encoders	Rotary and linear encoders
CiA 410	Inclinometers	Tilt sensors

CiA 402 -- Motion Control (Robotics)

Object	Index	Description
Controlword	0x6040	State machine control (enable, start, halt)
Statusword	0x6041	State machine status
Target Position	0x607A	Target position (profile position mode)
Target Velocity	0x60FF	Target velocity (profile velocity mode)
Target Torque	0x6071	Target torque (torque mode)
Actual Position	0x6064	Current position (encoder feedback)
Actual Velocity	0x606C	Current velocity

Standards

Document	Title
CiA 301	CANopen Application Layer and Communication Profile
CiA 402	Device profile for drives and motion control
CiA 306	Electronic Data Sheet (EDS) specification

See Also

- CAN -- physical/data link layer CANopen runs on
- EtherCAT -- CoE maps CANopen over industrial Ethernet
- DDS / ROS 2 -- higher-level robotics middleware
- MAVLink -- drone command protocol (some use CAN transport)
- PROFIBUS -- alternative fieldbus

DDS / ROS 2 -- Data Distribution Service

Standard: OMG DDS v1.4 / RTPS v2.3 | **Layer:** Application (Layer 7) | **Wireshark filter:** `rtps`

DDS (Data Distribution Service) is a middleware protocol for real-time, publish-subscribe data distribution. It is the default communication layer for **ROS 2** (Robot Operating System 2), the dominant framework for robotics software. DDS provides automatic peer discovery, strongly-typed topics, configurable Quality of Service (QoS), and decentralized architecture -- no broker required. The wire protocol is RTPS (Real-Time Publish-Subscribe), which runs over UDP multicast for discovery and UDP unicast for data.

Architecture

[Diagram: see online version]

DDS is **brokerless** -- publishers and subscribers discover each other directly using multicast, then communicate peer-to-peer.

RTPS Wire Protocol

RTPS Message

[Diagram: see online version]

Key Fields

Field	Size	Description
Protocol	4 bytes	Magic bytes `RTPS` (0x52545053)
Version	2 bytes	Protocol version (major.minor)
Vendor ID	2 bytes	DDS implementation identifier
GUID Prefix	12 bytes	Globally unique identifier for this participant

Submessages	Variable	One or more submessages (data, heartbeat, ack, etc.)

Submessage Header

[Diagram: see online version]

Submessage Types

ID	Name	Description
0x01	PAD	Padding
0x06	ACKNACK	Acknowledge/negative-acknowledge data
0x07	HEARTBEAT	Inform readers of available data sequence numbers
0x09	GAP	Indicate irrelevant sequence numbers
0x12	INFO_TS	Timestamp for subsequent submessages
0x14	INFO_SRC	Source participant info
0x15	DATA	User data payload
0x16	DATA_FRAG	Fragmented data

Discovery

DDS uses a two-phase discovery protocol:

Simple Participant Discovery Protocol (SPDP)

Participants announce themselves via UDP multicast:

[Diagram: see online version]

Parameter	Default Value
Discovery multicast group	239.255.0.1
Discovery port	7400 + (250 x domain_id) + participant_id offsets
Announcement period	30 seconds (default)

Simple Endpoint Discovery Protocol (SEDP)

After participants discover each other, they exchange their endpoints (DataWriters/DataReaders) via reliable unicast:

[Diagram: see online version]

ROS 2 Integration

ROS 2 maps its concepts onto DDS:

ROS 2 Concept	DDS Concept
Node	Domain Participant
Topic	Topic (with type)
Publisher	DataWriter
Subscriber	DataReader
Service	Request/Reply Topics (two topics)
Action	Multiple Topics (goal, result, feedback, status)
QoS Profile	DDS QoS policies
Domain ID	DDS Domain ID (network isolation)

ROS 2 Topic Wire Format

ROS 2 messages (e.g., `geometry_msgs/Twist`) are serialized using **CDR** (Common Data Representation) encoding and carried as RTPS DATA submessage payloads.

Common ROS 2 DDS Implementations

Implementation	Vendor	ROS 2 RMW
Fast DDS	eProsima	rmw_fastrtps (default)
Cyclone DDS	Eclipse	rmw_cyclonedds
Connext DDS	RTI	rmw_connextdds
GurumDDS	Gurum Networks	rmw_gurumdds

Quality of Service (QoS)

DDS provides rich QoS policies -- critical for robotics where some data needs reliability and some needs low latency:

QoS Policy	Options	Description
Reliability	BEST_EFFORT, RELIABLE	Guaranteed delivery vs. lowest latency
Durability	VOLATILE, TRANSIENT_LOCAL, TRANSIENT, PERSISTENT	Whether late joiners receive historical data
History	KEEP_LAST(n), KEEP_ALL	How many samples to retain
Deadline	Duration	Maximum expected time between samples
Liveliness	AUTOMATIC, MANUAL	How to detect dead writers

Lifespan	Duration	How long data is valid
Ownership	SHARED, EXCLUSIVE	Multiple or single writer per topic instance

ROS 2 QoS Profiles

Profile	Reliability	Durability	History	Use Case
Sensor data	BEST_EFFORT	VOLATILE	KEEP_LAST(5)	Camera, LiDAR, IMU
Parameters	RELIABLE	TRANSIENT_LOCAL	KEEP_LAST(1)	Config parameters
Services	RELIABLE	VOLATILE	KEEP_ALL	Request-response
Default	RELIABLE	VOLATILE	KEEP_LAST(10)	General purpose

Encapsulation

[Diagram: see online version]

Port	Usage
7400 + offsets	Discovery (multicast and unicast, per domain/participant)
7401 + offsets	User data (unicast)

DDS implementations also support TCP and shared memory transports for scenarios where UDP multicast is unavailable.

Standards

Document	Title
OMG DDS v1.4	Data Distribution Service for Real-Time Systems
OMG DDSI-RTPS v2.5	DDS Interoperability Wire Protocol (RTPS)
OMG DDS-XTypes v1.3	Extensible and Dynamic Topic Types
OMG DDS Security v1.1	DDS Security Specification
ROS 2 Design	ROS 2 middleware abstraction design

See Also

- UDP -- primary transport for RTPS
- MQTT -- alternative IoT pub-sub (broker-based, simpler)
- RTP -- real-time media transport (different domain)

EtherCAT -- Ethernet for Control Automation Technology

Standard: IEC 61158 / ETG Specification | **Layer:** Data Link (Layer 2) | **Wireshark filter:** `ecat` or `ecatmb`

EtherCAT is a real-time industrial Ethernet protocol designed by Beckhoff Automation for high-performance motion control, I/O, and sensor applications. Its key innovation is "processing on the fly" -- each slave device reads/writes its data from/to the Ethernet frame as it passes through, without buffering the entire frame. This achieves cycle times as low as 12.5 µs for 100 digital I/O or 100 µs for 100 servo drives. EtherCAT is widely used in CNC machines, robotics, semiconductor manufacturing, and packaging equipment.

How EtherCAT Works

Unlike standard Ethernet where each device receives, processes, and re-sends frames, EtherCAT frames pass **through** each slave in sequence. Each slave reads its input data and inserts its output data on the fly:

[Diagram: see online version]

The frame travels through all slaves in a ring (or open line with internal return) and comes back to the master in a single Ethernet frame cycle.

Frame Structure

EtherCAT frames are standard Ethernet frames with EtherType `0x88A4`:

[Diagram: see online version]

EtherCAT Header

[Diagram: see online version]

Datagram

[Diagram: see online version]

Field	Size	Description
Command	8 bits	Read/write type and addressing mode
Index	8 bits	Identifies the datagram (for matching)
Address	32 bits	Slave address (position, configured, or logical)
Length	11 bits	Data field length
C (Circulating)	1 bit	Frame has circulated back to master
M (More)	1 bit	More datagrams follow
IRQ	16 bits	Interrupt request from slaves

Data	Variable	Process data (read by/written by slaves on the fly)
WKC	16 bits	Working Counter -- incremented by each slave that successfully processes

Commands (Addressing Modes)

Cmd	Name	Addressing	Description
1	APRD	Auto-increment position	Read from slave at physical position
2	APWR	Auto-increment position	Write to slave at physical position
4	FPRD	Configured station address	Read from slave by configured address
5	FPWR	Configured station address	Write to slave by configured address
7	BRD	Broadcast	Read from all slaves
8	BWR	Broadcast	Write to all slaves
10	LRD	Logical address	Read from logical memory-mapped address
11	LWR	Logical address	Write to logical memory-mapped address
12	LRW	Logical address	Simultaneous read/write (most common for process data)

Logical Addressing (LRW)

The most efficient mode -- the master defines a logical address map. Each slave is configured to read/write specific byte ranges. One LRW datagram can exchange process data with all slaves simultaneously:

```
Logical Address Map:
Offset 0-3:   Slave 1 outputs (master writes, slave reads)
Offset 4-7:   Slave 2 outputs
Offset 8-11:  Slave 1 inputs (slave writes, master reads)
Offset 12-15: Slave 2 inputs
```

Slave States

[Diagram: see online version]

State	Description
INIT	Slave detected, mailbox not active

PRE-OP	Mailbox active (configuration via CoE/SoE/FoE)
SAFE-OP	Process data input active, outputs safe (zero)
OP	Full operation -- inputs and outputs active

Protocols over EtherCAT (Mailbox)

EtherCAT's mailbox carries higher-layer protocols for configuration:

Protocol	Name	Description
CoE	CANopen over EtherCAT	Object dictionary, SDO/PDO mapping (most common)
SoE	Servo Profile over EtherCAT	SERCOS drive parameters
FoE	File Access over EtherCAT	Firmware upload/download
EoE	Ethernet over EtherCAT	Standard Ethernet tunneling through EtherCAT
AoE	ADS over EtherCAT	Beckhoff TwinCAT access

Performance

Metric	Typical Value
Cycle time	12.5 µs (100 digital I/O), 100 µs (100 servo axes)
Jitter	< 1 µs
Max slaves	> 65,000 (theoretical), hundreds (practical)
Max frame size	1518 bytes (standard Ethernet)
Bandwidth	100 Mbps (Fast Ethernet), 1 Gbps (EtherCAT G)
Topology	Line, star, tree (with junction slaves)
Max cable length	100 m per segment (standard Ethernet copper)

Distributed Clocks (DC)

EtherCAT provides hardware-based clock synchronization across all slaves with < 1 µs accuracy:

Feature	Description
Reference clock	First slave with DC capability
Propagation delay	Measured and compensated automatically
Sync accuracy	< 100 ns typical
Sync modes	Free-run, SM-synchronous, DC-synchronous

EtherCAT vs Other Industrial Ethernet

Feature	EtherCAT	PROFINET IRT	EtherNet/IP
Cycle time	12.5 µs	250 µs	1 ms
Jitter	< 1 µs	< 1 µs	~1 ms
Topology	Line/star/tree	Star (switched)	Star (switched)
Processing	On the fly	Switched	Switched
Standard Ethernet	EtherType 0x88A4	Modified Ethernet	Standard TCP/UDP/IP
Application layer	CoE (CANopen)	PROFIdrive	CIP

Standards

Document	Title
IEC 61158	Industrial communication networks -- Fieldbus
ETG Specification	EtherCAT Technology Group specifications
IEC 61800-7	Adjustable speed electrical power drive systems (servo profiles)

See Also

- Ethernet -- EtherCAT uses standard Ethernet frames
- CAN -- CoE maps CANopen over EtherCAT
- CANopen -- application layer often used with EtherCAT
- PROFIBUS -- legacy fieldbus EtherCAT replaces

MAVLink -- Micro Air Vehicle Link

Standard: MAVLink Protocol (mavlink.io) | **Layer:** Application (Layer 7) | **Wireshark filter:** `mavlink`

MAVLink is a lightweight binary messaging protocol for communicating with drones, unmanned vehicles, and robotic systems. It is the standard protocol for ArduPilot, PX4, QGroundControl, and most open-source autopilot systems. MAVLink defines ~300 message types covering telemetry (GPS, attitude, battery), commands (takeoff, land, waypoint), parameters, and file transfer. It is designed for unreliable links -- messages are self-contained, with CRC integrity checking and optional signing.

Packet (MAVLink v2)

[Diagram: see online version]

Key Fields

Field	Size	Description
STX	8 bits	Start byte: `0xFD` (v2) or `0xFE` (v1)
Payload Length	8 bits	Length of payload in bytes
Incompat Flags	8 bits	Flags that must be understood (bit 0 = signed)
Compat Flags	8 bits	Flags that can be ignored if not understood
Sequence	8 bits	Packet sequence number (0-255, wrapping)
System ID	8 bits	Sending system (vehicle=1, GCS=255)
Component ID	8 bits	Sending component (autopilot=1, camera=100, gimbal=154)
Message ID	24 bits	Identifies the message type
Payload	0-255 bytes	Message-specific data (little-endian)
CRC	16 bits	CRC-16/MCRF4XX over header + payload + CRC_EXTRA
Signature	13 bytes	Optional: link ID + timestamp + SHA-256 hash (first 6 bytes)

Common Messages

Telemetry

MSG ID	Name	Key Fields	Description
0	HEARTBEAT	type, autopilot, base_mode, system_status	System alive indicator (1 Hz)
1	SYS_STATUS	battery_voltage, battery_remaining, errors_count	System health
24	GPS_RAW_INT	lat, lon, alt, fix_type, satellites_visible	Raw GPS data
30	ATTITUDE	roll, pitch, yaw, rollspeed, pitchspeed, yawspeed	Vehicle orientation (Euler angles)
31	ATTITUDE_QUATERNION	q1, q2, q3, q4	Vehicle orientation (quaternion)
32	LOCAL_POSITION_NED	x, y, z, vx, vy, vz	Local position (North-East-Down)
33	GLOBAL_POSITION_INT	lat, lon, alt, relative_alt, vx, vy, vz	Global position (WGS84)

74	VFR_HUD	airspeed, groundspeed, heading, throttle, alt, climb	Heads-up display data
147	BATTERY_STATUS	voltages, current, remaining, temperature	Detailed battery info

Commands

MSG ID	Name	Description
76	COMMAND_LONG	Send a command (MAV_CMD) with up to 7 parameters
77	COMMAND_ACK	Acknowledge a command (result code)
11	SET_MODE	Set flight mode (AUTO, GUIDED, LAND, RTL, etc.)

Common MAV_CMDs (in COMMAND_LONG)

CMD	Name	Description
22	NAV_TAKEOFF	Take off to specified altitude
21	NAV_LAND	Land at current or specified location
20	NAV_RETURN_TO_LAUNCH	Return to home position
16	NAV_WAYPOINT	Navigate to a waypoint
400	COMPONENT_ARM_DISARM	Arm or disarm motors
176	DO_SET_MODE	Change flight mode
183	DO_SET_SERVO	Control a servo output
2500	DO_CONTROL_VIDEO	Start/stop video capture

Mission / Waypoint

MSG ID	Name	Description
44	MISSION_COUNT	Total number of mission items
39	MISSION_ITEM_INT	Individual waypoint (lat/lon as int32)
40	MISSION_REQUEST	Request a specific mission item
47	MISSION_ACK	Mission transfer complete

Parameters

MSG ID	Name	Description
21	PARAM_REQUEST_LIST	Request all parameters

22	PARAM_VALUE	Parameter name + value
23	PARAM_SET	Set a parameter value

Typical GCS-Vehicle Flow

[Diagram: see online version]

Transport

MAVLink is transport-agnostic -- it runs over:

Transport	Typical Use
Serial (UART)	Autopilot <-> companion computer, telemetry radio
UDP	GCS <-> vehicle (SITL simulation, Wi-Fi)
TCP	GCS <-> vehicle (reliable link)
USB	Direct connection for configuration
Radio (SiK, RFD900)	Long-range telemetry (serial over radio)

Default UDP ports: 14550 (GCS), 14540 (onboard API).

Message Signing (v2)

MAVLink v2 supports optional message signing to prevent spoofing:

Field	Size	Description
Link ID	1 byte	Identifies the communication channel
Timestamp	6 bytes	10µs resolution timestamp
Signature	6 bytes	First 6 bytes of SHA-256(secret_key + header + payload + CRC + link_id + timestamp)

MAVLink v1 vs v2

Feature	v1	v2
STX	0xFE	0xFD
Message ID	8 bits (256 messages)	24 bits (16M messages)
Signing	No	Optional
Incompat/Compat flags	No	Yes
Payload truncation	No	Yes (trailing zeros removed)

Standards

Document	Title
MAVLink Protocol	MAVLink Developer Guide
MAVLink Messages	Common message definitions
ArduPilot MAVLink	ArduPilot MAVLink reference

See Also

- DDS / ROS 2 -- alternative robotics middleware (larger systems)
- CAN -- MAVLink can run over CAN (DroneCAN/UAVCAN)
- UART -- common MAVLink transport
- UDP -- GCS communication transport

ROS 1 -- Robot Operating System -- TCPROS/UDPROS

Standard: ROS Wiki (wiki.ros.org) | **Layer:** Application (Layer 7) | **Wireshark filter:** `tcp` (TCPROS is plain TCP with a custom header)

ROS 1 is the original Robot Operating System middleware that dominated robotics from 2007-2020 and remains widely deployed. Unlike ROS 2 (which uses DDS), ROS 1 uses a centralized architecture with a **ROS Master** (name server) for node discovery and custom transport protocols -- **TCPROS** (reliable, default) and **UDPROS** (unreliable, for large data like point clouds). Nodes communicate via topics (pub/sub), services (request/response), and parameters.

Architecture

[Diagram: see online version]

Key difference from ROS 2: the ROS Master is a **single point of failure** -- if it dies, no new connections can be established (existing ones continue).

Communication Patterns

Pattern	Mechanism	Description
Publish/Subscribe	Topics	Many-to-many, asynchronous (TCPROS/UDPROS)
Request/Response	Services	One-to-one, synchronous (TCPROS)

Shared State	Parameter Server	Key-value store on the ROS Master (XMLRPC)
Persistent Stream	Actions	Long-running tasks with feedback (built on topics)

TCPROS Protocol

TCPROS is the default transport for topic data and service calls over TCP:

Connection Handshake

[Diagram: see online version]

Connection Header

A series of length-prefixed key=value fields:

```
4 bytes: total header length
4 bytes: field length
N bytes: "callerid=/camera_driver"
4 bytes: field length
N bytes: "topic=/image_raw"
4 bytes: field length
N bytes: "type=sensor_msgs/Image"
4 bytes: field length
N bytes: "md5sum=060021388200f6f0f447d0fcd9c64743"
```

Header Fields

Field	Description
callerid	Node name of the sender
topic	Topic name
type	Message type (package/MessageType)
md5sum	MD5 hash of the message definition (compatibility check)
message_definition	Full message definition text (optional)
tcp_nodelay	"1" to set TCP_NODELAY (disable Nagle's algorithm)
latching	"1" if publisher sends last message to new subscribers
service	Service name (for service connections)
persistent	"1" for persistent service connections

Data Messages

After the header exchange, each message is sent as:

[Diagram: see online version]

Messages are serialized using ROS's custom serialization format -- fields are packed sequentially in definition order, with 4-byte length prefixes for variable-length types (strings, arrays).

UDPROS Protocol

UDPROS is used for large, lossy-tolerant data (point clouds, images in degraded networks):

UDPROS Header

[Diagram: see online version]

Opcode	Name	Description
0	DATA0	First (or only) fragment
1	DATAN	Subsequent fragment
2	PING	Keepalive
3	ERR	Error

UDPROS fragments messages larger than the UDP MTU and does **not** retransmit lost fragments -- the application sees either a complete message or nothing.

ROS Message Serialization

ROS messages are defined in `.msg` files:

```
# sensor_msgs/Image.msg
Header header
uint32 height
uint32 width
string encoding
uint8 is_bigendian
uint32 step
uint8[] data
```

Serialization Rules

Type	Serialization
bool, uint8, int8	1 byte
uint16, int16	2 bytes, little-endian
uint32, int32, float32	4 bytes, little-endian
uint64, int64, float64	8 bytes, little-endian
string	4 bytes length + UTF-8 bytes
array[]	4 bytes length + elements

fixed array[N]	N elements (no length prefix)
Header	uint32 seq + time stamp + string frame_id
time, duration	uint32 secs + uint32 nsecs

ROS Master (XMLRPC API)

The ROS Master provides name registration and lookup via XMLRPC over HTTP:

Method	Description
registerPublisher	Announce a publisher for a topic
registerSubscriber	Subscribe to a topic
unregisterPublisher	Remove a publisher
unregisterSubscriber	Remove a subscriber
registerService	Register a service
lookupService	Find a service provider
getTopicTypes	List all topics and their types
getSystemState	Full list of publishers, subscribers, services

ROS 1 vs ROS 2

Feature	ROS 1	ROS 2
Middleware	Custom (TCPROS/UDPROS)	DDS (standard)
Discovery	Centralized (ROS Master)	Decentralized (DDS multicast)
Single point of failure	Yes (Master)	No
QoS	None (best-effort or reliable TCP)	DDS QoS (reliability, durability, deadline, etc.)
Real-time	Not designed for RT	Supports RT via DDS + RTOS
Security	None (anyone can connect)	DDS Security (SROS2)
Language support	C++, Python (custom clients)	C, C++, Python (via rcl abstraction)
Serialization	Custom	CDR (DDS standard)
Multi-robot	Difficult	Namespace/domain isolation

Encapsulation

[Diagram: see online version]

Standards

ROS 1 is open source with community-maintained specifications:

Resource	Description
TCPROS Spec	TCPROS transport protocol
UDPROS Spec	UDPROS transport protocol
Connection Header	Header field definitions
Master API	XMLRPC API specification
Serialization	Message serialization format

See Also

- DDS / ROS 2 -- modern successor using DDS
- MAVLink -- drone protocol (often bridged with ROS)
- TCP -- TCPROS transport
- UDP -- UDPROS transport

Routing

Routing protocols for path computation and network convergence.

8 protocols in this section

BGP -- Border Gateway Protocol

Standard: RFC 4271 | **Layer:** Application (Layer 7) | **Wireshark filter:** bgp

BGP is the routing protocol that holds the Internet together. As an exterior gateway protocol (EGP), it exchanges routing information between autonomous systems (ASes), enabling networks operated by different organizations to reach each other. BGP is a path-vector protocol -- each route carries the full AS path it has traversed, enabling loop detection and policy-based routing decisions. BGP uses TCP port 179 for reliable delivery.

Message Header

All BGP messages share a common 19-byte header:

[Diagram: see online version]

Key Fields

Field	Size	Description
Marker	128 bits	All 1s (for compatibility); used for synchronization
Length	16 bits	Total message length including header (19-4096 bytes)
Type	8 bits	Message type

Field Details

Message Types

Type	Name	Description
1	OPEN	Establish a BGP session
2	UPDATE	Advertise or withdraw routes
3	NOTIFICATION	Report an error (terminates the session)
4	KEEPALIVE	Maintain the session (sent every 60s by default)
5	ROUTE-REFRESH	Request re-advertisement of routes (RFC 2918)

OPEN Message

Sent after TCP connection is established to negotiate session parameters:

[Diagram: see online version]

Field	Description
Version	BGP version (always 4)
My AS Number	Sender's 16-bit AS number (use capability for 32-bit)
Hold Time	Max seconds between KEEPALIVE/UPDATE (0 = no keepalives)
BGP Identifier	Router ID (typically an IPv4 address)
Optional Parameters	Capabilities (4-byte ASN, multiprotocol, route refresh, etc.)

UPDATE Message

Carries route advertisements and withdrawals:

Field	Description
Withdrawn Routes Length	Length of withdrawn routes field
Withdrawn Routes	Prefixes being withdrawn (variable)
Total Path Attr Length	Length of path attributes field
Path Attributes	Attributes describing the route (variable)
NLRI	Network Layer Reachability Information -- prefixes being advertised

Path Attributes

Attribute	Type	Description
ORIGIN	Well-known mandatory	How the route was learned (IGP, EGP, incomplete)
AS_PATH	Well-known mandatory	Sequence of AS numbers the route has traversed
NEXT_HOP	Well-known mandatory	IP address of the next-hop router
MULTI_EXIT_DISC (MED)	Optional non-transitive	Metric for choosing between multiple entry points
LOCAL_PREF	Well-known discretionary	Preference within the local AS (higher = preferred)
ATOMIC_AGGREGATE	Well-known discretionary	Route has been aggregated
AGGREGATOR	Optional transitive	AS and router ID that performed aggregation
COMMUNITY	Optional transitive	Tags for routing policy (RFC 1997)
LARGE_COMMUNITY	Optional transitive	Extended community tags (RFC 8092)
MP_REACH_NLRI	Optional non-transitive	Multiprotocol reachability (IPv6, VPN, etc.)

MP_UNREACH_NLRI	Optional non-transitive	Multiprotocol withdrawal

NOTIFICATION Error Codes

Code	Name
1	Message Header Error
2	OPEN Message Error
3	UPDATE Message Error
4	Hold Timer Expired
5	Finite State Machine Error
6	Cease

BGP Session Establishment

[Diagram: see online version]

eBGP vs iBGP

Feature	eBGP	iBGP
Peers	Different ASes	Same AS
TTL	1 (default)	255
AS_PATH	Prepends local AS	Does not modify
Next-hop	Changes to self	Preserves original
Full mesh	Not required	Required (or use route reflectors)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 4271	A Border Gateway Protocol 4 (BGP-4)
RFC 6793	BGP Support for Four-Octet Autonomous System Numbers
RFC 4760	Multiprotocol Extensions for BGP-4
RFC 4456	BGP Route Reflection
RFC 1997	BGP Communities Attribute

RFC 8092	BGP Large Communities Attribute
RFC 7606	Revised Error Handling for BGP UPDATE Messages
RFC 8203	BGP Administrative Shutdown Communication

See Also

- TCP
- OSPF -- interior gateway protocol (within an AS)
- IPv4

EIGRP -- Enhanced Interior Gateway Routing Protocol

Standard: RFC 7868 (Informational) | **Layer:** Network (Layer 3) | **Wireshark filter:** `eigrp`

EIGRP is an advanced distance-vector (hybrid) interior gateway routing protocol originally developed by Cisco. It uses the Diffusing Update Algorithm (DUAL) for rapid convergence and loop-free routing. EIGRP maintains a topology table with feasible successors, enabling instant failover without route recomputation. Unlike pure distance-vector protocols, EIGRP sends partial updates only when routes change, conserving bandwidth. Cisco published EIGRP as an informational RFC in 2016, though it remains most widely deployed on Cisco platforms.

Packet Header

All EIGRP packets share a common 20-byte header:

[Diagram: see online version]

Key Fields

Field	Size	Description
Version	1 byte	EIGRP version (currently 2)
Opcode	1 byte	Packet type (Update, Query, Reply, Hello, etc.)
Checksum	2 bytes	Checksum over the entire EIGRP packet
Flags	4 bytes	Init (0x01), Conditional Receive (0x02), Restart (0x04), End-of-Table (0x08)
Sequence Number	4 bytes	Sequence number for reliable delivery
Acknowledgment	4 bytes	Sequence number being acknowledged (0 = none)

Virtual Router ID	2 bytes	Virtual router identifier (0x0001 = unicast AF, 0x0002 = multicast AF)
AS Number	2 bytes	Autonomous System number for this EIGRP domain

Opcodes

Opcode	Name	Description
1	Update	Advertise route information to neighbors
3	Query	Ask neighbors for a route (DUAL active state)
4	Reply	Response to a Query with route information
5	Hello	Neighbor discovery and keepalive
10	SIA-Query	Stuck-In-Active query (prevents SIA timeout)
11	SIA-Reply	Response to SIA-Query

TLV Types

EIGRP uses TLVs within packets to carry route and parameter data:

Type	Name	Description
0x0001	EIGRP Parameters	K-values and holdtime
0x0002	Authentication	MD5 or SHA-256 authentication data
0x0003	Sequence	List of addresses for reliable multicast
0x0004	Software Version	IOS/EIGRP software version
0x0005	Next Multicast Sequence	Next multicast sequence number
0x0102	Internal Route (IPv4)	Classic internal route entry
0x0103	External Route (IPv4)	Redistributed route entry
0x0402	Internal Route (IPv6)	IPv6 internal route entry
0x0403	External Route (IPv6)	IPv6 external route entry
0x0602	Wide Internal Route	64-bit wide metric internal route
0x0603	Wide External Route	64-bit wide metric external route

DUAL Algorithm

The Diffusing Update Algorithm (DUAL) guarantees loop-free routing and fast convergence:

Key Concepts

Term	Definition
Successor	The best next-hop router for a destination (lowest feasible distance)
Feasible Distance (FD)	The best metric (composite cost) to a destination from this router
Reported Distance (RD)	The metric a neighbor reports for reaching a destination
Feasible Successor (FS)	A backup next-hop whose RD is less than the current FD
Feasibility Condition	RD of neighbor < FD of current successor (guarantees loop-free path)
Active state	Route is being recomputed via Query/Reply diffusion
Passive state	Route is stable and in use

Route Selection and Failover

[Diagram: see online version]

Active Route Recovery (No Feasible Successor)

[Diagram: see online version]

Metric Calculation

Classic Metric (32-bit)

EIGRP composite metric uses K-values to weight components:

```
Metric = 256 * ( K1*BW + (K2*BW)/(256-Load) + K3*Delay ) * ( K5/(K4+Reliability) )
```

K-Value	Default	Component
K1	1	Bandwidth
K2	0	Load
K3	1	Delay
K4	0	Reliability
K5	0	Reliability (if K5=0, the reliability term is not used)

With defaults (K1=1, K2=0, K3=1, K4=0, K5=0):

```
Metric = 256 * (BW + Delay)
Where:
  BW = 10^7 / minimum bandwidth in kbps along the path
  Delay = sum of delays in tens of microseconds along the path
```

Wide Metrics (64-bit)

Modern EIGRP (Named Mode) uses 64-bit wide metrics to handle high-speed interfaces:

Component	Classic	Wide Metric
Throughput	10^7 / BW (kbps)	10^7 * 65536 / BW (kbps)
Latency	Delay / 10 (usec)	Delay * 65536 / 10 (usec)
Metric range	32-bit	64-bit (top 32 bits used in RIB)
Scalability	Cannot differentiate > 1 Gbps	Differentiates up to 100+ Gbps

Neighbor Discovery and Adjacency

[Diagram: see online version]

Hello and Hold Timers

Interface Type	Hello Interval	Hold Time
LAN, point-to-point (high bandwidth)	5 seconds	15 seconds
Multipoint (T1 and below)	60 seconds	180 seconds

Named EIGRP Mode

Named EIGRP (introduced in IOS 15.0(1)M) provides a hierarchical configuration structure:

Component	Purpose
Address-Family	IPv4 or IPv6 configuration under a single process
af-interface	Per-interface settings (hello, holdtime, authentication, etc.)
topology	Route filtering, redistribution, variance, offset-lists

Benefits over classic mode: unified configuration for IPv4/IPv6, wide metrics, remote neighbor support, and per-AF/per-interface fine-tuning.

EIGRP vs OSPF

Feature	EIGRP	OSPF
Algorithm	DUAL (distance-vector/hybrid)	Dijkstra SPF (link-state)

Protocol	IP protocol 88	IP protocol 89
Update type	Partial, bounded updates	LSA flooding within areas
Convergence	Instant (feasible successor) or Query/Reply	SPF recalculation
Metric	Composite (bandwidth, delay, etc.)	Cost (based on bandwidth)
Equal-cost load balancing	Yes (up to 32 paths)	Yes (up to 32 paths)
Unequal-cost load balancing	Yes (variance command)	No
Hierarchy	Flat (stub routing available)	Areas (backbone + non-backbone)
Multi-vendor	Limited (Cisco-centric)	Broad multi-vendor support
Standards	RFC 7868 (Informational)	RFC 2328 (Standards Track)
IPv6 support	EIGRP for IPv6 / Named Mode	OSPFv3

Encapsulation

[Diagram: see online version]

EIGRP runs directly over IP with protocol number 88. Hello, Update, Query, and Reply packets are sent to the multicast address `224.0.0.10` (IPv4) or `FF02::A` (IPv6). ACKs are sent unicast.

Standards

Document	Title
RFC 7868	Cisco's Enhanced Interior Gateway Routing Protocol (EIGRP)

See Also

- OSPF -- link-state IGP alternative
- BGP -- exterior gateway protocol
- RIP -- simpler distance-vector protocol
- IS-IS -- another link-state IGP
- IPv4 -- EIGRP runs over IP protocol 88

HSRP -- Hot Standby Router Protocol

Standard: RFC 2281 (Informational, HSRPv1) | **Layer:** Network (Layer 3) | **Wireshark filter:** `hsrp`

HSRP is a Cisco proprietary first-hop redundancy protocol (FHRP) that provides automatic default gateway failover for hosts on a LAN. Two or more routers form an HSRP group sharing a virtual IP address. One router is elected

Active and handles all traffic to the virtual IP; the other serves as Standby and takes over if the Active router fails. HSRP is widely deployed on Cisco networks for gateway high availability -- it predates the IETF-standard VRRP and remains common in Cisco-only environments.

HSRPv1 Packet

[Diagram: see online version]

HSRPv2 Packet

[Diagram: see online version]

Key Fields

Field	Size	Description
Version	1 byte	0 = HSRPv1, 2 = HSRPv2
Opcode	1 byte	Message type (Hello, Coup, Resign)
State	1 byte	Current HSRP state of the sender
Hellotime	1 byte	Seconds between Hello messages (default 3)
Holdtime	1 byte	Seconds before declaring Active router down (default 10)
Priority	1 byte	Router priority (0-255, default 100, higher = preferred)
Group	1 byte (v1) / 2 bytes (v2)	HSRP group number
Authentication	8 bytes (v1)	Plain-text authentication string (default: "cisco")
Virtual IP	4 bytes	The shared virtual gateway IP address

Opcodes

Opcode	Name	Description
0	Hello	Periodic keepalive; announces state, priority, timers
1	Coup	Sent by a router claiming to be the new Active (preemption)
2	Resign	Sent by the Active router when it is giving up the role

States

HSRP routers transition through the following states:

State	Value	Description
Initial	0	HSRP starting; no configuration or interface not up
Learn	1	Waiting to hear from the Active router to learn the virtual IP
Listen	2	Knows the virtual IP; not Active or Standby; monitoring Hellos
Speak	4	Sending Hellos, participating in Active/Standby election
Standby	8	Candidate to become Active; monitors the Active router
Active	16	Currently forwarding traffic for the virtual IP address

State Machine

[Diagram: see online version]

Failover Sequence

[Diagram: see online version]

Virtual MAC Address

Version	Virtual MAC Format	Range
HSRPv1	`0000.0C07.ACxx`	xx = group number (0x00-0xFF, groups 0-255)
HSRPv2 (IPv4)	`0000.0C9F.F000`-`0000.0C9F.FFFF`	12-bit group in last 3 nibbles (groups 0-4095)
HSRPv2 (IPv6)	`0005.73A0.0000`-`0005.73A0.0FFF`	12-bit group (groups 0-4095)

HSRPv1 vs HSRPv2

Feature	HSRPv1	HSRPv2
Group range	0-255	0-4095
Multicast address	224.0.0.2	224.0.0.102
Virtual MAC	0000.0C07.ACxx	0000.0C9F.Fxxx
Authentication	Plain-text (8 bytes)	MD5 authentication
IPv6 support	No	Yes

Millisecond timers	No	Yes
Source address	Interface IP	Virtual IP
Transport	UDP port 1985	UDP port 1985

Priority and Preemption

Priority	Meaning
255	Maximum (not reserved like VRRP)
101-254	High priority (manually configured)
100	Default
1-99	Low priority
0	Triggers immediate resignation from Active role

Interface Tracking

HSRP can adjust priority based on tracked interface or object status:

Tracking Type	Behavior
Interface tracking	Decrement priority when a tracked interface goes down
Object tracking	Decrement priority based on IP SLA, route existence, etc.
Decrement value	Configurable (default 10); if priority drops below Standby, failover occurs

HSRP vs VRRP vs GLBP

Feature	HSRP	VRRP	GLBP
Standard	Cisco proprietary	IETF RFC 5798	Cisco proprietary
Active/Master	One Active router	One Master router	One AVG + multiple AVFs
Load balancing	No (single Active)	No (single Master)	Yes (per-host or round-robin)
Default priority	100	100	100
Hello timer	3 seconds	1 second	3 seconds
Hold timer	10 seconds	~3 seconds	10 seconds
Multicast (default)	224.0.0.2 (v1)	224.0.0.18	224.0.0.102
Virtual MAC	0000.0C07.ACxx	0000.5E00.01xx	0007.B400.xxyy
IP owner concept	No	Yes (priority 255)	No
Preemption default	Disabled	Enabled	Disabled
Groups per interface	Multiple	Multiple	Multiple

IPv6	HSRPv2	VRRPv3	Yes

Encapsulation

[Diagram: see online version]

HSRP packets are sent via UDP port 1985 to multicast addresses:

- HSRPv1: `224.0.0.2` (all routers), TTL=1
- HSRPv2: `224.0.0.102`, TTL=1

Standards

Document	Title
RFC 2281	Cisco Hot Standby Router Protocol (HSRP)

See Also

- VRRP -- IETF-standard alternative to HSRP
- EIGRP -- Cisco routing protocol often deployed alongside HSRP
- OSPF -- IGP commonly used with HSRP
- ARP -- gratuitous ARP used during HSRP failover

LDP -- Label Distribution Protocol

Standard: RFC 5036 | **Layer:** Network (Layer 3) | **Wireshark filter:** `ldp`

LDP is a protocol for distributing MPLS labels between Label Switching Routers (LSRs). Each router uses LDP to announce the labels it has assigned to Forwarding Equivalence Classes (FECs), typically IP prefixes from its routing table. This creates Label Switched Paths (LSPs) that follow the shortest IGP path. LDP uses a two-phase approach: UDP (port 646) for neighbor discovery via Hello messages, then TCP (port 646) for reliable session establishment and label exchange.

LDP PDU Header

Every LDP message is carried inside an LDP PDU with a 10-byte header:

[Diagram: see online version]

Key Fields

Field	Size	Description
Version	16 bits	LDP version; always 1
PDU Length	16 bits	Total length of the PDU excluding the version and length fields
LDP Identifier	32 bits	Router ID of the sending LSR (typically a loopback address)
Label Space ID	16 bits	Identifies the label space (0 for per-platform label space)

LDP Message Header

Each message within a PDU has its own header:

[Diagram: see online version]

Field	Size	Description
U (Unknown)	1 bit	Action when message type is unknown: 0 = send Notification, 1 = ignore
Message Type	15 bits	Type of LDP message
Message Length	16 bits	Length of the message body (excluding type and length)
Message ID	32 bits	Unique identifier for matching requests to responses

Field Details

Message Types

Type	Name	Description
0x0100	Notification	Signal errors or advisory information
0x0200	Hello	Neighbor discovery (sent via UDP)
0x0300	Initialization	Negotiate session parameters after TCP connection
0x0201	KeepAlive	Maintain the TCP session (default every 60s)
0x0300	Address	Announce interface addresses to the peer
0x0301	Address Withdraw	Withdraw previously announced addresses
0x0400	Label Mapping	Bind a label to a FEC
0x0401	Label Request	Request a label for a FEC

0x0402	Label Withdraw	Withdraw a previously advertised label binding
0x0403	Label Release	Release a label received from a peer
0x0404	Label Abort Request	Abort an outstanding label request

Hello Message

Sent as UDP multicast (224.0.0.2, port 646) or targeted UDP unicast:

Parameter	Description
Hold Time	How long to maintain adjacency without a Hello (default 15s for link, 45s for targeted)
T bit	0 = link Hello (multicast), 1 = targeted Hello (unicast)
R bit	Request targeted Hello response

Label Mapping TLVs

TLV	Description
FEC TLV	Forwarding Equivalence Class (prefix, host address, wildcard, or CR-LSP)
Label TLV	Generic label (20-bit MPLS label value)
Hop Count TLV	Loop detection for non-TTL-based networks
Path Vector TLV	List of LSR IDs for additional loop detection

FEC Element Types

Type	Description
Wildcard	All FECs (used in Label Withdraw for mass withdrawal)
Prefix	IP prefix (IPv4 or IPv6) with prefix length
Host Address	Specific host address binding

Label Distribution Modes

Mode	Description
Downstream Unsolicited (DU)	LSR advertises labels to peers without being asked (most common)
Downstream on Demand (DoD)	LSR only sends labels when requested
Independent	LSR assigns labels independently of its peers
Ordered	LSR only assigns labels after receiving a label from its downstream peer

Liberal retention	Keep labels from all peers (faster convergence, more memory)
Conservative retention	Keep labels only from the next-hop peer (less memory)

LDP Session Establishment

[Diagram: see online version]

Session Maintenance

LDP sessions are maintained through periodic KeepAlive messages. If no LDP message is received within the negotiated KeepAlive timeout (default 180 seconds), the session is torn down and all associated label bindings are removed.

Graceful Restart

LDP supports graceful restart (RFC 3478), allowing an LSR to preserve its MPLS forwarding state across a control-plane restart. The restarting router signals its intent via the FT Session TLV in the Initialization message, and its peers maintain their label bindings for a configurable recovery period.

Encapsulation

[Diagram: see online version]

Discovery uses UDP multicast (or unicast for targeted sessions). All other messages use a TCP session on port 646 for reliable, ordered delivery.

Standards

Document	Title
RFC 5036	LDP Specification
RFC 5283	LDP Extension for Inter-Area LSPs
RFC 3478	Graceful Restart Mechanism for LDP
RFC 5561	LDP Capabilities
RFC 7552	Updates to LDP for IPv6

See Also

- MPLS -- label switching data plane that LDP populates
- RSVP -- constraint-based LSP signaling (traffic engineering)
- BGP -- distributes labels for L3VPN and segment routing

PIM -- Protocol Independent Multicast

Standard: RFC 7761 (PIM-SM) | **Layer:** Network (Layer 3) | **Wireshark filter:** `pim`

PIM is the dominant multicast routing protocol used to build distribution trees that deliver multicast traffic from sources to receivers across an IP network. It is called "protocol independent" because it does not include its own topology discovery mechanism -- instead, it relies on whatever unicast routing protocol (OSPF, BGP, IS-IS, etc.) has populated the routing table. PIM runs directly over IP (protocol number 103) and comes in two main flavors: Sparse Mode (PIM-SM), which uses explicit joins toward a Rendezvous Point, and Dense Mode (PIM-DM), which floods traffic and prunes branches with no receivers.

PIM Header

All PIM messages share a common 4-byte header:

[Diagram: see online version]

Key Fields

Field	Size	Description
Version	4 bits	PIM version; always 2
Type	4 bits	Message type (Hello, Join/Prune, Register, etc.)
Reserved	8 bits	Set to zero on transmission
Checksum	16 bits	Checksum over the entire PIM message

Field Details

Message Types

Type	Name	Description
0	Hello	Neighbor discovery and option negotiation
1	Register	Encapsulate multicast data from source DR to RP
2	Register-Stop	RP tells source DR to stop sending Register messages
3	Join/Prune	Join or leave a multicast tree branch
4	Bootstrap	Distribute RP-set information across the domain

5	Assert	Elect a single forwarder when multiple routers share a LAN
8	Candidate-RP-Advertisement	Announce willingness to serve as RP

Hello Message

Sent periodically (default 30 seconds) to discover PIM neighbors and negotiate capabilities:

Option	Description
Hold Time	How long to consider the neighbor alive (default 105s)
DR Priority	Priority for Designated Router election (higher wins)
Generation ID	Random value to detect neighbor restarts
LAN Prune Delay	Parameters for prune override on multi-access networks

Register Message

[Diagram: see online version]

Flag	Description
B (Border)	Set when the Register is from a domain border
N (Null-Register)	Probe to check if Register-Stop is still active

Join/Prune Message

Field	Description
Upstream Neighbor	Address of the upstream router being joined/pruned
Hold Time	How long the join/prune state is valid
Number of Groups	Count of multicast groups in this message
Joined Sources	Sources to join (list of (S,G) or (*,G) entries)
Pruned Sources	Sources to prune

PIM-SM Operation

Rendezvous Point (RP)

The RP is a central meeting point in PIM Sparse Mode. Sources register with the RP, and receivers join toward the RP. Once traffic is flowing, the last-hop router can switch to a shortest-path tree (SPT) directly from the source for optimal delivery.

RP Selection Methods:

Method	Description
Static RP	Manually configured on all routers
Bootstrap Router (BSR)	BSR collects Candidate-RP advertisements and distributes the RP-set
Auto-RP	Cisco proprietary; uses 224.0.1.39 and 224.0.1.40
Anycast RP	Multiple RPs share the same IP, synchronized via MSDP

Multicast Tree Building (PIM-SM)

[Diagram: see online version]

PIM-SM vs PIM-DM

Feature	PIM-SM (Sparse Mode)	PIM-DM (Dense Mode)
Model	Explicit join (pull)	Flood and prune (push)
RP required	Yes	No
Default tree	Shared tree via RP	Source tree (shortest path)
Scalability	Good for sparse receivers	Poor at scale; floods everywhere
Use case	Enterprise and Internet multicast	Small/dense networks (largely deprecated)

Assert Mechanism

When two PIM routers on the same LAN both forward the same multicast stream, an Assert message is exchanged. The router with the best route metric to the source (or RP) wins and continues forwarding; the loser stops.

Assert field	Purpose
RPT bit	Whether the Assert is for a shared or source tree
Metric Preference	Preference of the unicast routing protocol
Route Metric	Metric to the source or RP

Encapsulation

[Diagram: see online version]

PIM is carried directly in IP packets with protocol number 103. Hello messages are sent to the ALL-PIM-ROUTERS multicast group (224.0.0.13 for IPv4, ff02::d for IPv6) with TTL/Hop Limit of 1.

Standards

Document	Title

RFC 7761	Protocol Independent Multicast - Sparse Mode (PIM-SM)
RFC 3973	Protocol Independent Multicast - Dense Mode (PIM-DM)
RFC 5059	Bootstrap Router (BSR) Mechanism for PIM
RFC 4610	Anycast-RP Using Protocol Independent Multicast (PIM)
RFC 3446	Anycast Rendezvous Point (RP) Mechanism using PIM and MSDP

See Also

- IGMP -- host-to-router multicast group membership
- OSPF -- unicast routing that PIM relies on
- BGP -- carries multiprotocol multicast routes (MBGP)

RIP -- Routing Information Protocol

Standard: RFC 2453 (RIPv2) / RFC 2080 (RIPng) | **Layer:** Application (Layer 7) | **Wireshark filter:** `rip`

RIP is one of the oldest distance-vector routing protocols, using hop count as its sole metric to determine the best path to a destination. Each router periodically broadcasts (RIPv1) or multicasts (RIPv2) its entire routing table to neighbors every 30 seconds. RIP is simple to configure and understand, making it suitable for small networks, but its 15-hop limit, slow convergence, and lack of scalability have led to its replacement by OSPF and EIGRP in most modern networks. RIPv2 added classless routing (VLSM support), multicast updates, and authentication. RIPng extends RIP for IPv6.

RIPv2 Packet

[Diagram: see online version]

Key Fields

Packet Header

Field	Size	Description
Command	1 byte	1 = Request, 2 = Response
Version	1 byte	1 = RIPv1, 2 = RIPv2
Zero	2 bytes	Must be zero (unused)

Route Entry (20 bytes each, max 25 per packet)

Field	Size	Description
Address Family	2 bytes	0x0002 for IP (0xFFFF for authentication entry)
Route Tag	2 bytes	Tag for external route identification (RIPv2 only)
IP Address	4 bytes	Destination network address
Subnet Mask	4 bytes	Subnet mask (RIPv2 only; zero in RIPv1)
Next Hop	4 bytes	Next-hop IP (0.0.0.0 = use sender; RIPv2 only)
Metric	4 bytes	Hop count to destination (1-15 valid, 16 = unreachable)

Command Types

Command	Name	Description
1	Request	Ask for all or specific routes from a neighbor
2	Response	Send routing table entries (periodic or triggered)

RIPv1 vs RIPv2 vs RIPng

Feature	RIPv1	RIPv2	RIPng
RFC	RFC 1058	RFC 2453	RFC 2080
Addressing	Classful (no subnet mask)	Classless (VLSM/CIDR)	IPv6 prefixes
Updates	Broadcast (255.255.255.255)	Multicast (224.0.0.9)	Multicast (FF02::9)
Transport	UDP port 520	UDP port 520	UDP port 521
Authentication	None	Plain-text or MD5	IPsec (external)
Route tags	No	Yes	Yes
Next-hop field	No (implicit)	Yes	Yes (link-local)
Max metric	15 (16 = infinity)	15 (16 = infinity)	15 (16 = infinity)
Subnet mask in update	No	Yes	Prefix length

RIPv2 Authentication Entry

When authentication is enabled, the first route entry slot is used for authentication:

[Diagram: see online version]

Auth Type	Value	Description
Plain-text	2	16-byte cleartext password
MD5	3	MD5 keyed digest (RFC 2082)

Distance-Vector Algorithm

RIP uses the Bellman-Ford algorithm. Each router:

1. Receives routing tables from directly connected neighbors
2. Adds its own hop cost (1) to each received metric
3. Selects the route with the lowest metric for each destination
4. Advertises its routing table to all neighbors every 30 seconds

Loop Prevention Mechanisms

Mechanism	Description
Split horizon	Do not advertise a route back out the interface it was learned on
Poison reverse	Advertise learned routes back with metric 16 (unreachable)
Triggered updates	Send immediate update when a route changes (do not wait for timer)
Hold-down timer	After a route is marked unreachable, ignore new routes to it for a period
Maximum hop count	16 = infinity; prevents count-to-infinity (limits network diameter to 15 hops)

Timers

Timer	Default	Description
Update	30 seconds	Interval between periodic full routing table broadcasts
Invalid	180 seconds	Time without an update before a route is marked unreachable
Hold-down	180 seconds	After invalidation, period to reject inferior routes
Flush	240 seconds	Time after which an invalid route is removed from the table

Timer Interaction

[Diagram: see online version]

Convergence Example

[Diagram: see online version]

RIP vs OSPF vs EIGRP

Feature	RIP	OSPF	EIGRP
Algorithm	Bellman-Ford (distance-vector)	Dijkstra SPF (link-state)	DUAL (hybrid)
Metric	Hop count (max 15)	Cost (bandwidth-based)	Composite (BW, delay, etc.)
Convergence	Slow (minutes)	Fast (seconds)	Very fast (sub-second with FS)
Scalability	Small networks only	Large networks (areas)	Large networks (stub/summary)
Update method	Full table every 30s	LSA flooding on change	Partial updates on change
Transport	UDP port 520	IP protocol 89	IP protocol 88
VLSM/CIDR	RIPv2 only	Yes	Yes
Authentication	RIPv2: MD5/plain	MD5, SHA	MD5, SHA
Equal-cost paths	No (typically)	Yes	Yes
Unequal-cost paths	No	No	Yes (variance)
Administrative distance	120	110	90 (internal), 170 (external)
Standards	RFC 2453	RFC 2328	RFC 7868 (Informational)

Encapsulation

[Diagram: see online version]

- **RIPv1:** UDP port 520, broadcast to `255.255.255.255`
- **RIPv2:** UDP port 520, multicast to `224.0.0.9`
- **RIPng:** UDP port 521, multicast to `FF02::9`

Standards

Document	Title
RFC 1058	Routing Information Protocol (RIPv1)
RFC 2453	RIP Version 2
RFC 2080	RIPng for IPv6
RFC 2082	RIP-2 MD5 Authentication
RFC 1723	RIP Version 2 (obsoleted by RFC 2453)

See Also

- OSPF -- link-state IGP (preferred for larger networks)
- EIGRP -- advanced distance-vector/hybrid protocol
- BGP -- exterior gateway protocol
- IS-IS -- another link-state IGP
- IPv4 -- RIP routes IPv4 networks
- IPv6 -- RIPng routes IPv6 networks

RSVP -- Resource Reservation Protocol

Standard: RFC 2205 | **Layer:** Network (Layer 3) | **Wireshark filter:** `rsvp`

RSVP is a signaling protocol that reserves resources (bandwidth, buffer space) along a network path to provide Quality of Service (QoS) guarantees for data flows. While the original RSVP was designed for IntServ-style reservations, its most important modern use is RSVP-TE (Traffic Engineering), which signals MPLS Label Switched Paths (LSPs) with explicit route control and fast reroute capabilities. RSVP runs directly over IP (protocol number 46) and uses a soft-state model -- reservations must be periodically refreshed or they time out.

Common Header

[Diagram: see online version]

Key Fields

Field	Size	Description
Version	4 bits	RSVP version; always 1
Flags	4 bits	0x01 = Refresh-reduction capable
Msg Type	8 bits	Message type (Path, Resv, Error, Teardown)
RSVP Checksum	16 bits	Checksum over the full RSVP message
Send TTL	8 bits	TTL value when the message was sent (detects non-RSVP hops)
RSVP Length	16 bits	Total length of the RSVP message in bytes

Field Details

Message Types

Type	Name	Direction	Description
1	Path	Sender -> Receiver	Establish path state at each hop; carry sender info
2	Resv	Receiver -> Sender	Request resource reservation along the path
3	PathErr	Hop -> Sender	Report an error in Path processing
4	ResvErr	Hop -> Receiver	Report an error in Resv processing
5	PathTear	Sender -> Receiver	Remove path state and associated reservations
6	ResvTear	Receiver -> Sender	Remove reservation state
7	ResvConf	Sender -> Receiver	Confirm a reservation was installed

RSVP Objects

RSVP messages carry a sequence of typed objects. Each object has a class and a C-Type:

Object	Description
SESSION	Identifies the data flow (destination IP, protocol, port/LSP ID)
RSVP_HOP	Previous/next RSVP hop address and logical interface
TIME_VALUES	Refresh period for soft-state maintenance
SENDER_TEMPLATE	Source IP and port identifying the sender
SENDER_TSPEC	Traffic specification (token bucket: rate, burst, peak rate)
FLOWSPEC	Requested QoS (guaranteed or controlled-load service)
FILTER_SPEC	Identifies which senders the reservation applies to
STYLE	Reservation style (Fixed-Filter, Shared-Explicit, Wildcard-Filter)

RSVP-TE Extensions (RFC 3209)

RSVP-TE adds objects for MPLS traffic engineering:

Object	Description
LABEL_REQUEST	Request a label assignment from downstream
LABEL	Assigned MPLS label (carried upstream in Resv)
EXPLICIT_ROUTE (ERO)	Ordered list of hops the LSP must traverse
RECORD_ROUTE (RRO)	Records the actual path taken (for loop detection)
SESSION_ATTRIBUTE	LSP name, setup/hold priority, flags (local protection, etc.)

FAST_REROUTE	Request facility or one-to-one backup protection
DETOUR	Identify backup path for one-to-one fast reroute

ERO (Explicit Route Object)

The ERO encodes a sequence of subobjects specifying the desired LSP path:

Subobject type	Description
IPv4 prefix (strict)	Must traverse this exact next hop
IPv4 prefix (loose)	Must reach this node, path between is flexible
AS number	Traverse a specific autonomous system
Unnumbered interface	Identified by router ID and interface index

Reservation Styles

Style	Abbreviation	Description
Fixed-Filter	FF	Distinct reservation per sender
Shared-Explicit	SE	Shared reservation among listed senders
Wildcard-Filter	WF	Shared reservation for all senders

MPLS LSP Signaling (RSVP-TE)

[Diagram: see online version]

Fast Reroute (FRR)

RSVP-TE supports sub-50ms protection switching using pre-computed backup paths:

FRR Type	Description
Facility Backup	Pre-established bypass tunnel protects many LSPs (NNHOP or NHOP)
One-to-One Backup	Dedicated detour path for each protected LSP

The Point of Local Repair (PLR) detects the failure and immediately reroutes traffic onto the backup path, then signals the head-end to recompute the primary path.

Encapsulation

[Diagram: see online version]

RSVP is carried directly in IP packets with protocol number 46. Unlike BGP or LDP, RSVP does not use TCP or UDP -- it is its own IP protocol with its own reliability mechanisms (refresh and summary refresh via RFC 2961).

Standards

Document	Title
RFC 2205	Resource ReSerVation Protocol (RSVP)
RFC 3209	RSVP-TE: Extensions to RSVP for LSP Tunnels
RFC 4090	Fast Reroute Extensions to RSVP-TE for LSP Tunnels
RFC 2961	RSVP Refresh Overhead Reduction Extensions
RFC 3473	Generalized MPLS Signaling -- RSVP-TE Extensions (GMPLS)
RFC 5440	Path Computation Element (PCE) Communication Protocol

See Also

- MPLS -- label switching data plane that RSVP-TE signals
- LDP -- alternative MPLS label distribution protocol

SRv6 -- Segment Routing over IPv6

Standard: RFC 8986 | **Layer:** Network (Layer 3) | **Wireshark filter:** `ipv6.routing.type == 4`

Segment Routing over IPv6 (SRv6) encodes forwarding instructions as an ordered list of IPv6 addresses (segments) in the Segment Routing Header (SRH), an IPv6 routing extension header. Each segment identifier (SID) is a 128-bit IPv6 address that represents an instruction -- forward to a node, follow a specific link, or apply a network function. SRv6 combines the source-routing paradigm of segment routing with the native IPv6 data plane, eliminating the need for MPLS labels while enabling sophisticated network programming.

Segment Routing Header (SRH)

The SRH (Routing Type 4) is inserted as an IPv6 extension header:

[Diagram: see online version]

Key Fields

Field	Size	Description
Next Header	8 bits	Type of the header following the SRH

Hdr Ext Len	8 bits	Length of the SRH in 8-octet units (not counting first 8 octets)
Routing Type	8 bits	Always 4 for Segment Routing
Segments Left	8 bits	Index (0-based) into the segment list pointing to the active segment
Last Entry	8 bits	Index of the last element in the segment list
Flags	8 bits	Currently unused; must be zero
Tag	16 bits	Packet tagging for policy-based routing
Segment List	128 bits each	Ordered list of SIDs (IPv6 addresses), in reverse order

Field Details

SID Format

An SRv6 SID is a 128-bit IPv6 address composed of three parts:

[Diagram: see online version]

Part	Description
Locator	Routable IPv6 prefix that identifies the node owning the SID
Function	Identifies the local behavior (instruction) to execute
Arguments	Optional parameters for the function (flow ID, VPN context, etc.)

SRv6 Endpoint Behaviors (RFC 8986)

Behavior	Name	Description
End	Endpoint	Decrement Segments Left; update IPv6 DA with next segment
End.X	Endpoint with Layer-3 cross-connect	Forward to a specific next hop via a specific interface
End.T	Endpoint with table lookup	Look up the inner destination in a specific routing table
End.DX2	Endpoint with L2 cross-connect	Decapsulate and forward the L2 frame to an interface
End.DX4	Endpoint with IPv4 cross-connect	Decapsulate and forward to an IPv4 next hop
End.DX6	Endpoint with IPv6 cross-connect	Decapsulate and forward to an IPv6 next hop

End.DT4	Endpoint with IPv4 table lookup	Decapsulate and look up in an IPv4 routing table (L3VPN)
End.DT6	Endpoint with IPv6 table lookup	Decapsulate and look up in an IPv6 routing table (L3VPN)
End.DT46	Endpoint with dual-stack table lookup	Decapsulate and look up in an IP routing table (dual-stack L3VPN)
End.B6.Encaps	Endpoint with SRH encapsulation	Insert a new IPv6 header with SRH
End.BM	Endpoint with SR-MPLS encapsulation	Encapsulate with MPLS labels
H.Encaps	Headend with SRH encapsulation	Encapsulate original packet with outer IPv6 + SRH
H.Encaps.Red	Headend with reduced SRH	Omit the last SID from the SRH (it is the DA)

Network Programming Model

SRv6 treats the network as a programmable infrastructure. Each node advertises its SIDs (via IGP extensions or BGP), and the head-end composes a segment list that chains together network instructions:

[Diagram: see online version]

SRH Optional TLVs

TLV Type	Name	Description
1	Padding	Align the SRH to a boundary
5	HMAC	Keyed hash for SRH integrity verification
6	NSH Carrier	Carries Network Service Header metadata

SRv6 vs SR-MPLS

Feature	SRv6	SR-MPLS
SID size	128 bits (IPv6 address)	20 bits (MPLS label)
Data plane	Native IPv6	MPLS
MTU overhead	Higher (each SID = 16 bytes)	Lower (each label = 4 bytes)
Network programming	Rich function set per SID	Limited to forwarding actions
Midpoint state	Stateless (instructions in packet header)	Stateless (labels in stack)
Hardware support	Requires IPv6 extension header support	Mature MPLS silicon
Deployment	Greenfield / IPv6-native networks	Brownfield / existing MPLS networks
VPN support	End.DT4, End.DT6, End.DX*	VPN labels
Encapsulation	IPv6 + SRH	MPLS label stack

IGP extensions	IS-IS SRv6, OSPFv3 SRv6	IS-IS SR, OSPF SR

SRv6 Encapsulation Modes

Mode	Description
H.Encaps	New outer IPv6 header + SRH encapsulates the original packet
H.Encaps.Red	Reduced encapsulation: last SID placed in DA, not in the SRH
H.Insert	SRH inserted into the existing IPv6 header (no outer encapsulation)
H.Insert.Red	Reduced SRH inserted into the existing IPv6 header

Encapsulation

[Diagram: see online version]

SRv6 uses IPv6 Routing Extension Header (Next Header = 43) with Routing Type 4. The inner payload can be IPv4, IPv6, or Layer 2 frames depending on the endpoint behavior. No MPLS headers are required.

Standards

Document	Title
RFC 8986	Segment Routing over IPv6 (SRv6) Network Programming
RFC 8754	IPv6 Segment Routing Header (SRH)
RFC 9252	BGP Overlay Services Based on Segment Routing over IPv6
RFC 9256	Segment Routing Policy Architecture
RFC 9352	IS-IS Extensions to Support SRv6
RFC 8402	Segment Routing Architecture

See Also

- IPv6 -- underlying data plane for SRv6
- MPLS -- alternative label switching data plane (SR-MPLS)
- BGP -- distributes SRv6 SIDs for overlay services

Security & Authentication

Security, authentication, and cryptographic protocols.

15 protocols in this section

802.1X / EAP -- Port-Based Network Access Control

Standard: IEEE 802.1X-2020 / RFC 3748 (EAP) | **Layer:** Data Link (Layer 2) | **Wireshark filter:** `eapol` or `eap`

802.1X is a port-based access control framework that authenticates devices before granting them network access. It's the standard for WPA/WPA2/WPA3 Enterprise Wi-Fi and wired NAC (Network Access Control). The framework has three roles: the supplicant (client), the authenticator (switch/AP), and the authentication server (RADIUS). EAP (Extensible Authentication Protocol) is the pluggable mechanism that carries the actual authentication exchange -- various EAP methods provide different credential types.

Architecture

[Diagram: see online version]

Component	Role	Examples
Supplicant	Client seeking access	Laptop, phone, IoT device
Authenticator	Enforces access control	Wi-Fi AP, Ethernet switch
Authentication Server	Validates credentials	FreeRADIUS, Microsoft NPS, Cisco ISE

The authenticator is a pass-through -- it relays EAP between the supplicant (over L2) and the RADIUS server (over IP).

EAPOL Frame (EAP over LAN)

[Diagram: see online version]

EAPOL Packet Types

Type	Name	Description
0	EAP-Packet	Carries an EAP frame
1	EAPOL-Start	Supplicant initiates authentication
2	EAPOL-Logoff	Supplicant logs off
3	EAPOL-Key	WPA/WPA2 4-way handshake key frames
5	EAPOL-Encapsulated-ASF-Alert	Alert messages

EAPOL frames use Ethernet type `0x888E` and are sent to multicast `01:80:C2:00:00:03`.

EAP Frame

[Diagram: see online version]

EAP Codes

Code	Name	Description
1	Request	Server -> client (challenge)
2	Response	Client -> server (answer)
3	Success	Authentication succeeded
4	Failure	Authentication failed

EAP Types

Type	Name	Description
1	Identity	Request/provide username
2	Notification	Display message to user
3	Nak	Client rejects proposed EAP method
4	MD5-Challenge	MD5 challenge-response (like CHAP)
6	GTC	Generic Token Card (one-time password)
13	EAP-TLS	Mutual TLS with client certificate
21	EAP-TTLS	TLS tunnel + inner auth (PAP, MSCHAPv2, etc.)
25	EAP-PEAP	TLS tunnel + inner EAP (MSCHAPv2, GTC)
43	EAP-FAST	Cisco's tunneled method (PAC-based)
18	EAP-SIM	GSM SIM card authentication
23	EAP-AKA	UMTS/LTE SIM authentication
50	EAP-AKA'	Enhanced AKA for 5G/LTE

Authentication Flow (EAP-PEAP)

[Diagram: see online version]

Common EAP Methods

Method	Client Cert	Server Cert	Inner Auth	Security	Deployment
EAP-TLS	Required	Required	N/A (mutual TLS)	Highest	Enterprises with PKI
EAP-PEAP	No	Required	MSCHAPv2 or GTC	High	Most common enterprise Wi-Fi

EAP-TTLS	No	Required	PAP, CHAP, MSCHAPv2	High	Popular in education (eduroam)
EAP-FAST	No	Optional	MSCHAPv2, GTC	High	Cisco environments
EAP-SIM	SIM card	No	GSM auth vectors	High	Carrier Wi-Fi offload
EAP-AKA	USIM card	No	UMTS/LTE auth	High	Carrier Wi-Fi offload
EAP-MD5	No	No	MD5 challenge	Low	Testing only (no key derivation)

Port States

The authenticator's port has two virtual states:

State	Traffic Allowed	When
Unauthorized	Only EAPOL frames	Before successful authentication
Authorized	All traffic	After EAP-Success + key exchange

Wired 802.1X

On a wired switch, the physical port is blocked until authentication:

[Diagram: see online version]

The switch can assign the authenticated device to a specific VLAN, apply an ACL, or quarantine it based on RADIUS attributes.

RADIUS Attributes for 802.1X

Attribute	Description
Tunnel-Type (64)	VLAN (value 13)
Tunnel-Medium-Type (65)	802 (value 6)
Tunnel-Private-Group-ID (81)	VLAN number or name
Filter-Id (11)	ACL name to apply
Session-Timeout (27)	Re-authentication interval
EAP-Message (79)	Encapsulated EAP data
Message-Authenticator (80)	HMAC-MD5 integrity (mandatory for EAP)
MS-MPPE-Recv-Key	PMK for WPA key derivation
MS-MPPE-Send-Key	PMK for WPA key derivation

MAB (MAC Authentication Bypass)

For devices that don't support 802.1X (printers, cameras, IoT):

[Diagram: see online version]

Encapsulation

[Diagram: see online version]

Standards

Document	Title
IEEE 802.1X-2020	Port-Based Network Access Control
RFC 3748	Extensible Authentication Protocol (EAP)
RFC 5216	EAP-TLS Authentication Protocol
RFC 7170	Tunnel EAP (TEAP)
RFC 2716	PPP EAP-TLS

See Also

- WPA2 / WPA3 -- Wi-Fi security built on 802.1X
- RADIUS -- authentication server
- Kerberos -- credential source for EAP-TLS/PEAP
- 802.11 (Wi-Fi) -- wireless LAN
- Ethernet -- wired 802.1X
- TLS -- used inside EAP-TLS/PEAP/TTLS

ACME -- Automatic Certificate Management Environment

Standard: RFC 8555 | **Layer:** Application (Layer 7) | **Wireshark filter:** `http2 or tls`

ACME is a protocol for automating the issuance, renewal, and revocation of X.509 TLS certificates. Developed by the Internet Security Research Group (ISRG) for Let's Encrypt, ACME eliminates the manual, error-prone process of obtaining certificates. The protocol runs over HTTPS, using JSON request/response bodies protected by JWS (JSON Web Signature). An ACME client proves control of a domain through challenge-response validation, then submits a Certificate Signing Request (CSR) and receives a signed certificate -- all without human intervention. Let's Encrypt is the most widely used ACME CA, but the protocol is supported by many certificate authorities.

Protocol Overview

ACME communication is JSON over HTTPS. All POST requests from the client are JWS-signed (using the `Flattened JSON Serialization` with a protected header, payload, and signature). The server uses standard HTTPS responses with JSON bodies.

JWS-Protected Request

[Diagram: see online version]

Field	Description
protected	Base64url-encoded JSON header: `alg`, `nonce`, `url`, `kid` (or `jwk` for new accounts)
payload	Base64url-encoded JSON body (empty string for POST-as-GET)
signature	Base64url-encoded signature over `protected.payload`

Directory Resource

Every ACME server exposes a directory at a well-known URL (e.g., `/directory`) listing available endpoints:

Field	Description
`newNonce`	URL to request a fresh anti-replay nonce
`newAccount`	URL to create or look up an account
`newOrder`	URL to submit a new certificate order
`revokeCert`	URL to revoke a certificate
`keyChange`	URL to change the account key
`meta`	Optional metadata: terms of service, website, CA identifiers

Order Lifecycle

An order tracks the certificate issuance process through these states:

State	Description
`pending`	Order created; authorizations need to be completed
`ready`	All authorizations satisfied; client may finalize (submit CSR)
`processing`	CA is issuing the certificate
`valid`	Certificate has been issued and is available for download
`invalid`	An authorization failed or the order expired

[Diagram: see online version]

Challenge Types

The client proves domain control through one of these challenge types:

Challenge	Type String	Method	Description
HTTP-01	`http-01`	HTTP	Place a token file at `http://<domain>/.well-known/acme-challenge/<token>`
DNS-01	`dns-01`	DNS	Create a TXT record at `_acme-challenge.<domain>` with a digest of the key authorization
TLS-ALPN-01	`tls-alpn-01`	TLS	Respond on port 443 with a self-signed certificate containing the validation token (ALPN protocol `acme-tls/1`)

Key Authorization

For all challenges, the key authorization string is: `<token>.<account-key-thumbprint>`

For `dns-01`, the TXT record value is the base64url-encoded SHA-256 digest of the key authorization.

Full Certificate Issuance Flow

[Diagram: see online version]

DNS-01 Challenge Flow

[Diagram: see online version]

Certificate Renewal

ACME certificates (especially Let's Encrypt) have short lifetimes (typically 90 days). Renewal follows the same order/authorize/finalize flow. Clients like Certbot and acme.sh automate this via cron jobs or systemd timers, typically renewing 30 days before expiration.

Certificate Revocation

A client can revoke a certificate by POSTing to the `revokeCert` endpoint:

Field	Description

`certificate`	Base64url-encoded DER of the certificate to revoke
`reason`	Optional revocation reason code (RFC 5280: 0=unspecified, 1=keyCompromise, etc.)

The request can be signed by the account key or the certificate's private key.

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 8555	Automatic Certificate Management Environment (ACME)
RFC 8737	ACME TLS-ALPN-01 Challenge Extension
RFC 8738	ACME IP Identifier Validation Extension
RFC 7515	JSON Web Signature (JWS) -- used for request signing
RFC 8823	Extensions to ACME for Subdomains

See Also

- TLS -- the certificates ACME issues are used to secure TLS connections
- HTTP -- ACME runs over HTTPS
- DNS -- dns-01 challenge validates via DNS TXT records
- WireGuard -- modern VPN that also uses X.509-adjacent key management

FIDO2 / WebAuthn -- Fast Identity Online 2 / Web Authentication

Standard: W3C WebAuthn Level 3 / FIDO Alliance CTAP 2.2 | **Layer:** Application (Layer 7) | **Wireshark filter:** N/A (browser API + local authenticator)

FIDO2 is the passwordless authentication standard that replaces passwords with public-key cryptography. It consists of two complementary specifications: **WebAuthn** (a W3C browser API for relying parties to create and verify credentials) and **CTAP** (the FIDO Alliance protocol for communicating with hardware authenticators like security keys). Together they enable phishing-resistant authentication -- the private key never leaves the authenticator, credentials are origin-bound, and there is nothing to phish, replay, or stuff. FIDO2 is supported by all major browsers and platforms, and powers passkeys (synced multi-device FIDO credentials).

Architecture

[Diagram: see online version]

Authenticator Types

Type	Description	Examples
Platform (Internal)	Built into the device; uses biometrics or device PIN	Touch ID, Face ID, Windows Hello, Android biometrics
Roaming (Cross-Platform)	External hardware token; portable across devices	YubiKey, Google Titan, Feitian, SoloKeys
Synced (Passkey)	Platform credential synced across devices via cloud	iCloud Keychain, Google Password Manager, 1Password

Registration Ceremony (Create Credential)

[Diagram: see online version]

Authentication Ceremony (Get Assertion)

[Diagram: see online version]

PublicKeyCredentialCreationOptions

Field	Type	Description
challenge	BufferSource	Random bytes from server (minimum 16 bytes) -- prevents replay
rp	Object	Relying party: `{ id: "example.com", name: "Example Corp" }`
user	Object	User: `{ id: <buffer>, name: "alice@example.com", displayName: "Alice" }`
pubKeyCredParams	Array	Allowed algorithms: `[{ type: "public-key", alg: -7 }]` (-7 = ES256, -257 = RS256)
timeout	Number	Ceremony timeout in milliseconds
attestation	String	`"none"` (default), `"indirect"`, or `"direct"`
authenticatorSelection	Object	Platform vs cross-platform, resident key requirement, user verification
excludeCredentials	Array	Credential IDs to prevent re-registration

AuthenticatorAttestationResponse

Field	Description
clientDataJSON	JSON: `{ type: "webauthn.create", challenge: "...", origin: "https://example.com" }`
attestationObject	CBOR-encoded: `{ fmt, attStmt, authData }`

authData (within attestationObject)

[Diagram: see online version]

Flags Byte

Bit	Name	Description
0	UP	User Present -- user interacted with authenticator
2	UV	User Verified -- biometric or PIN verified
6	AT	Attested Credential Data present
7	ED	Extension Data present

AuthenticatorAssertionResponse

Field	Description
clientDataJSON	JSON: `{ type: "webauthn.get", challenge: "...", origin: "https://example.com" }`
authenticatorData	rpIdHash + flags + signCount (no credential data)
signature	ECDSA/RSA signature over `authData \
userHandle	User ID (from registration), enables username-less login

CTAP2 (Client to Authenticator Protocol)

CTAP2 defines the wire protocol between the platform (browser/OS) and the authenticator hardware:

Transports

Transport	Description	Framing
USB HID	USB Human Interface Device	64-byte HID reports
NFC	Near Field Communication	ISO 7816-4 APDUs (extended length)
BLE	Bluetooth Low Energy	FIDO BLE service (GATT)

Hybrid	QR code + BLE + cloud assist	Cross-device (phone as authenticator)

CTAP2 Commands

Command	Byte	Description
authenticatorMakeCredential	0x01	Create a new credential (registration)
authenticatorGetAssertion	0x02	Sign a challenge (authentication)
authenticatorGetInfo	0x04	Query authenticator capabilities and supported features
authenticatorClientPIN	0x06	Set/change/get PIN token, get retries
authenticatorReset	0x07	Factory reset (remove all credentials)
authenticatorGetNextAssertion	0x08	Get next assertion when multiple credentials match
authenticatorCredentialManagement	0x0A	Enumerate and delete resident credentials
authenticatorSelection	0x0B	Prompt user to select this authenticator (touch)
authenticatorLargeBlobs	0x0C	Store/retrieve large blobs associated with credentials
authenticatorConfig	0x0D	Configure authenticator settings

CTAP2 Message Format

[Diagram: see online version]

Response:

[Diagram: see online version]

Passkeys (Multi-Device FIDO Credentials)

Passkeys extend FIDO2 by allowing credential private keys to sync across a user's devices through platform cloud services:

Feature	Traditional FIDO2 Key	Passkey
Storage	Single hardware device	Synced via cloud (iCloud, Google, etc.)
Recovery	Backup key required	Restored from cloud backup
Cross-device	Each device needs own credential	Available on all synced devices
Phishing-resistant	Yes	Yes (origin-bound)
Discoverable	Optional	Always (resident key)

Algorithms

COSE Algorithm ID	Name	Description
-7	ES256	ECDSA with P-256 and SHA-256 (most common)
-8	EdDSA	Edwards-curve Digital Signature Algorithm (Ed25519)
-257	RS256	RSASSA-PKCS1-v1_5 with SHA-256
-37	PS256	RSASSA-PSS with SHA-256

FIDO2 vs Password vs TOTP

Feature	FIDO2 / Passkeys	Password	TOTP (Google Authenticator)
Phishing-resistant	Yes (origin-bound)	No	No (code can be phished in real-time)
Replay-resistant	Yes (challenge-response)	No	Partial (time-limited)
Server breach risk	Public key only (useless to attacker)	Hashed password (crackable)	Shared secret (reusable)
User experience	Biometric touch / tap	Type password	Open app, type 6 digits
Account recovery	Platform sync / backup key	Email reset	Backup codes
Credential stuffing	Impossible	Common attack	N/A (second factor)
Man-in-the-middle	Prevented (TLS channel binding)	Possible	Possible
Zero-knowledge	Yes (private key never transmitted)	No (password sent to server)	No (both sides know secret)

Attestation Formats

Format	Description
packed	Standard FIDO2 attestation (self-attestation or CA-signed)
tpm	Trusted Platform Module attestation
android-key	Android hardware keystore attestation
android-safetynet	Android SafetyNet / Play Integrity
apple	Apple Anonymous Attestation
fido-u2f	Backward-compatible U2F attestation
none	No attestation (privacy-preserving default)

Standards

Document	Title
W3C WebAuthn Level 3	Web Authentication: An API for accessing Public Key Credentials
FIDO CTAP 2.2	Client to Authenticator Protocol
FIDO Metadata Service	Authenticator metadata for trust decisions
RFC 8809	FIDO Alliance attestation statement format identifiers
COSE (RFC 9053)	CBOR Object Signing and Encryption (algorithm identifiers)

See Also

- WireGuard -- modern cryptographic protocol (different domain, similar philosophy of simplicity)
- Bluetooth LE -- CTAP2 transport for roaming authenticators
- NFC -- CTAP2 transport for tap-to-authenticate
- USB -- CTAP2 transport (USB HID)

Kerberos

Standard: RFC 4120 | **Layer:** Application (Layer 7) | **Wireshark filter:** `kerberos`

Kerberos is a network authentication protocol that uses symmetric-key cryptography and a trusted third party (the Key Distribution Center) to authenticate users and services without sending passwords over the network. It is the default authentication protocol in Microsoft Active Directory and is used by every Windows domain login, file share access, and service authentication in enterprise networks. Kerberos provides mutual authentication, single sign-on (SSO), and delegated authentication via ticket-based credentials.

Core Concepts

Component	Description
KDC (Key Distribution Center)	Trusted server that issues tickets (runs AS + TGS)
AS (Authentication Service)	Authenticates users and issues TGTs
TGS (Ticket-Granting Service)	Issues service tickets in exchange for TGTs
TGT (Ticket-Granting Ticket)	Proves identity to the TGS (like a passport)
Service Ticket (ST)	Proves identity to a specific service (like a boarding pass)
Principal	Identity (user or service): `user@REALM` or `service/host@REALM`
Realm	Administrative domain (typically uppercase domain name: `EXAMPLE.COM`)

Keytab	File containing a principal's long-term key (service accounts)

Authentication Flow

[Diagram: see online version]

Message Types

Type	Name	Direction	Description
10	AS-REQ	Client -> KDC	Request a TGT (initial authentication)
11	AS-REP	KDC -> Client	TGT + session key
12	TGS-REQ	Client -> KDC	Request a service ticket (present TGT)
13	TGS-REP	KDC -> Client	Service ticket + service session key
14	AP-REQ	Client -> Service	Present service ticket to access a service
15	AP-REP	Service -> Client	Mutual authentication confirmation
30	KRB-ERROR	Any -> Any	Error response

Ticket Structure

[Diagram: see online version]

The encrypted part (readable only by the target service or KDC) contains:

Field	Description
Flags	Ticket flags (forwardable, renewable, etc.)
Session Key	Shared key for client-service communication
Client Realm	Client's realm
Client Name	Client's principal name
Auth Time	When the client authenticated
Start Time	When the ticket becomes valid
End Time	When the ticket expires
Renew Till	Maximum renewal time
Client Addresses	Optional IP restriction

Ticket Flags

Flag	Description
FORWARDABLE	TGT can be forwarded to another host
FORWARDED	This is a forwarded ticket
PROXIABLE	Ticket can be used to obtain proxy tickets
RENEWABLE	Ticket can be renewed past its expiration
PRE-AUTHENT	Client was pre-authenticated
OK-AS-DELEGATE	Service is trusted for delegation

Pre-Authentication

Modern Kerberos requires pre-authentication (PA-DATA) in the AS-REQ to prevent offline password attacks:

PA-Type	Name	Description
2	PA-ENC-TIMESTAMP	Timestamp encrypted with user's key (proves knowledge of password)
11	PA-ETYPE-INFO2	Supported encryption types and salt
16	PA-PK-AS-REQ	PKINIT -- certificate-based authentication
128	PA-PAC-REQUEST	Request a Privilege Attribute Certificate (Windows)
136	PA-FX-FAST	Flexible Authentication Secure Tunneling (armor)

Encryption Types

etype	Name	Status
17	aes128-cts-hmac-sha1-96	Current standard
18	aes256-cts-hmac-sha1-96	Current standard (default in modern AD)
23	rc4-hmac (NTLM hash)	Deprecated (still common in legacy)
3	des-cbc-md5	Obsolete (disabled by default)

Error Codes

Code	Name	Common Cause
6	KDC_ERR_C_PRINCIPAL_UNKNOWN	User not found in KDC database
14	KDC_ERR_ETYPE_NOSUPP	No common encryption type

18	KDC_ERR_PREAUTH_FAILED	Wrong password
23	KDC_ERR_KEY_EXPIRED	Password expired
24	KDC_ERR_PREAUTH_REQUIRED	Pre-authentication needed (normal first step)
25	KDC_ERR_SERVER_NOMATCH	Service principal not found
34	KDC_ERR_BADOPTION	Invalid ticket flag request
37	KDC_ERR_S_PRINCIPAL_UNKNOWN	Service not found in KDC database
68	KDC_ERR_WRONG_REALM	Cross-realm referral

Active Directory Integration

In Windows AD, Kerberos is tightly integrated:

AD Concept	Kerberos Mapping
Domain	Realm
Domain Controller	KDC (AS + TGS)
User login	AS-REQ/AS-REP
Accessing \\\\server\\share	TGS-REQ for `cifs/server@REALM`
Group membership	PAC (Privilege Attribute Certificate) in ticket
Trust relationship	Cross-realm TGT exchange

Service Principal Names (SPNs)

SPN Format	Example	Service
`HTTP/server`	`HTTP/web01.example.com`	Web server
`CIFS/server`	`CIFS/fs01.example.com`	File share (SMB)
`MSSQLSvc/server:port`	`MSSQLSvc/db01:1433`	SQL Server
`LDAP/server`	`LDAP/dc01.example.com`	LDAP (Domain Controller)
`krbtgt/REALM`	`krbtgt/EXAMPLE.COM`	TGS itself (TGT target)

Kerberos Delegation

Type	Description	Use Case
Unconstrained	Service can impersonate the client to any other service	Legacy (security risk)
Constrained	Service can impersonate only to specified services	Web app -> SQL server

Resource-Based Constrained	Target service specifies who can delegate to it	Modern AD recommended
S4U2Self / S4U2Proxy	Protocol transition extensions	Service accounts, web SSO

Encapsulation

[Diagram: see online version]

Kerberos uses UDP 88 by default, falling back to TCP 88 for messages exceeding the UDP datagram size (common with PAC-heavy tickets in AD).

Standards

Document	Title
RFC 4120	The Kerberos Network Authentication Service (V5)
RFC 4121	The Kerberos V5 GSSAPI Mechanism
RFC 6113	Kerberos Pre-Authentication Framework (FAST)
RFC 4556	PKINIT (Public Key Cryptography for Initial Auth)
MS-KILE	Kerberos Protocol Extensions (Microsoft AD)
MS-PAC	Privilege Attribute Certificate (AD group membership)

See Also

- LDAP -- directory service Kerberos often authenticates against
- RADIUS -- alternative AAA (often wraps Kerberos via EAP)
- SMB -- Windows file sharing authenticated via Kerberos
- DNS -- SRV records for KDC discovery (`_kerberos._tcp.example.com`)
- UDP -- primary transport
- TCP -- fallback for large messages

NTLM -- NT LAN Manager

Standard: MS-NLMP (Microsoft Open Specifications) | **Layer:** Application / Authentication | **Wireshark filter:** `ntlmssp`

NTLM is a challenge-response authentication protocol developed by Microsoft. It authenticates clients without transmitting passwords over the network -- instead, the server sends a random challenge and the client proves knowledge of the password by computing a cryptographic response. NTLM operates as a Security Support Provider (SSP) via the NTLMSSP interface and is embedded in many protocols including HTTP, SMB, LDAP, SMTP, and

DCE/RPC. While still widely deployed in Windows environments, NTLM is considered legacy and is being deprecated in favor of Kerberos due to fundamental security weaknesses including vulnerability to pass-the-hash, relay, and brute-force attacks.

Three-Message Challenge-Response

NTLM authentication uses exactly three messages:

[Diagram: see online version]

Type 1 -- NEGOTIATE_MESSAGE

Sent by the client to initiate authentication and advertise supported features:

[Diagram: see online version]

Field	Size	Description
Signature	8 bytes	Always `NTLMSSP\0` (null-terminated)
Message Type	4 bytes	Always 0x00000001
Negotiate Flags	4 bytes	Bitfield of requested capabilities
Domain Name	Variable	Client's domain (optional)
Workstation Name	Variable	Client's workstation name (optional)
Version	8 bytes	OS version info (present if NEGOTIATE_VERSION flag set)

Type 2 -- CHALLENGE_MESSAGE

Sent by the server with a random challenge for the client to sign:

[Diagram: see online version]

Field	Size	Description
Signature	8 bytes	Always `NTLMSSP\0`
Message Type	4 bytes	Always 0x00000002
Target Name	Variable	Server's domain or machine name
Negotiate Flags	4 bytes	Flags reflecting negotiated capabilities
Server Challenge	8 bytes	Random nonce used to compute the response
Target Info	Variable	AV_PAIR structures (domain, server name, DNS, timestamp)

Target Info AV_PAIRs

AV ID	Name	Description
0x0001	MsvAvNbComputerName	Server NetBIOS name
0x0002	MsvAvNbDomainName	Domain NetBIOS name
0x0003	MsvAvDnsComputerName	Server DNS hostname
0x0004	MsvAvDnsDomainName	Domain DNS name
0x0007	MsvAvTimestamp	FILETIME timestamp (used in NTLMv2)
0x0009	MsvAvTargetName	SPN of the target server
0x000A	MsvAvChannelBindings	MD5 hash of TLS channel binding

Type 3 -- AUTHENTICATE_MESSAGE

Sent by the client with the computed response proving password knowledge:

[Diagram: see online version]

Field	Size	Description
LM Response	24 bytes (v1) or variable	LM compatibility response
NTLM Response	24 bytes (v1) or variable	NT response (NTLMv1 or NTLMv2)
Domain Name	Variable	User's domain (Unicode)
User Name	Variable	Account name (Unicode)
Workstation Name	Variable	Client machine name (Unicode)
Encrypted Random Session Key	16 bytes	Session key encrypted with response key (for signing/sealing)
MIC	16 bytes	Message Integrity Code over all three messages (NTLMv2)

NTLMv1 vs NTLMv2

Property	NTLMv1	NTLMv2
Hash algorithm	DES-based (MD4 of password -> DES key)	HMAC-MD5
Challenge	8-byte server challenge only	Server challenge + client challenge + timestamp + target info
Response size	24 bytes (LM + NTLM)	Variable (NTProofStr + client blob)
Replay protection	None	Timestamp in blob prevents replay
Security	Weak -- vulnerable to rainbow tables, DES brute-force	Stronger -- but still hash-based (no PFS)
LM hash	Optional (can be disabled)	Not used (sent as zeros or dummy)
Default since	Windows NT 4.0	Windows Vista / Server 2008

Recommendation	Disable immediately	Use only when Kerberos is unavailable

NTLMv2 Response Computation

Step	Operation
1	NT Hash = MD4(Unicode(password))
2	NTLMv2 Hash = HMAC-MD5(NT Hash, Unicode(uppercase(username) + domain))
3	Build client blob (timestamp, client challenge, target info)
4	NTProofStr = HMAC-MD5(NTLMv2 Hash, server challenge + client blob)
5	NTLMv2 Response = NTProofStr + client blob

Negotiate Flags

Key flags negotiated between client and server:

Flag	Bit	Description
NEGOTIATE_UNICODE	0	Unicode strings
NEGOTIATE_OEM	1	OEM character set
REQUEST_TARGET	2	Server should return target name
NEGOTIATE_SIGN	4	Message signing requested
NEGOTIATE_SEAL	5	Message encryption (sealing) requested
NEGOTIATE_LM_KEY	7	LAN Manager session key
NEGOTIATE_NTLM	9	NTLM authentication (always set)
NEGOTIATE_ANONYMOUS	11	Anonymous connection
NEGOTIATE_EXTENDED_SESSIONS ECURITY	19	Extended session security (NTLMv2 session security)
NEGOTIATE_TARGET_INFO	23	Target info block present
NEGOTIATE_VERSION	25	Version field present
NEGOTIATE_128	29	128-bit session key
NEGOTIATE_KEY_EXCH	30	Key exchange for session key
NEGOTIATE_56	31	56-bit encryption

Protocols That Carry NTLM

NTLM is embedded in numerous protocols as an authentication sub-protocol:

Protocol	Mechanism
HTTP	`Authorization: NTLM <base64>` header (via WWW-Authenticate negotiation)
SMB	NTLMSSP in SMB Session Setup
LDAP	SASL NTLM bind
SMTP / IMAP	AUTH NTLM command
DCE/RPC	NTLMSSP authentication context
MSSQL	NTLM via TDS login packets
WinRM / PowerShell Remoting	NTLM over HTTP (WSMAN)

Security Weaknesses

Attack	Description
Pass-the-Hash (PtH)	The NT hash alone is sufficient to authenticate -- no password needed
NTLM Relay	Attacker relays Type 1/2/3 messages between victim and target server
Brute-force / Rainbow tables	NTLMv1 hashes are DES-based and fast to crack
Credential forwarding	No mutual authentication -- server is not verified
No Perfect Forward Secrecy	Compromised hash exposes all past sessions
Reflection attacks	Attacker reflects challenge back to the client (mitigated in NTLMv2)

Why NTLM Is Being Deprecated

Microsoft has announced plans to deprecate NTLM in Windows. Key reasons include the lack of mutual authentication, the pass-the-hash vulnerability (attackers with hash access can authenticate without the password), and the absence of modern protections like channel binding by default. Kerberos provides mutual authentication, delegation, and is based on tickets rather than password hashes.

NTLM vs Kerberos

Feature	NTLM	Kerberos
Authentication model	Challenge-response (password hash)	Ticket-based (KDC issues tickets)
Mutual authentication	No (server not verified)	Yes (both sides verified)
Third-party trust	No (direct client-server)	Yes (KDC / Domain Controller)
Delegation	No	Yes (constrained and unconstrained)
Pass-the-hash vulnerable	Yes	No (uses session keys)
Requires DC connectivity	Only for domain validation	Yes (for ticket granting)

Protocol overhead	3 messages	6+ messages (including TGT + service ticket)
Clock sensitivity	None	Requires synchronized clocks (5-minute window)
Offline authentication	Yes (cached credentials)	Limited
Modern recommendation	Deprecated / fallback only	Preferred for all Windows authentication

Standards

Document	Title
MS-NLMP	NT LAN Manager (NTLM) Authentication Protocol
MS-NTHT	NTLM Over HTTP Protocol
RFC 4178	SPNEGO (used to negotiate NTLM vs Kerberos)

See Also

- Kerberos -- preferred replacement for NTLM in Active Directory environments
- SMB -- primary protocol that uses NTLM authentication
- LDAP -- directory protocol that supports NTLM bind
- TLS -- transport-layer security (NTLM can be carried over TLS)
- OAuth 2.0 -- modern token-based authorization framework

OAuth 2.0 / OpenID Connect

Standard: RFC 6749 | **Layer:** Application (Layer 7) | **Wireshark filter:** `http contains "Bearer" or http.request.uri contains "oauth" or http.request.uri contains "openid"`

OAuth 2.0 is an authorization framework that enables third-party applications to obtain limited access to an HTTP service on behalf of a resource owner. It replaces the pattern of sharing passwords with delegated, scoped, revocable access tokens. OpenID Connect (OIDC) is an identity layer built on top of OAuth 2.0 that adds authentication, providing a standardized way to verify a user's identity and obtain basic profile information via ID Tokens (JWTs).

Core Concepts

Component	Description
Resource Owner	The user who owns the data and grants access
Client	The application requesting access (web app, mobile app, CLI)

Authorization Server (AS)	Issues tokens after authenticating the resource owner (e.g., Okta, Auth0, Keycloak)
Resource Server (RS)	The API that accepts access tokens (e.g., `api.example.com`)
Authorization Code	Short-lived code exchanged for tokens (never exposed to the browser)
Access Token	Credential used to call protected APIs (typically a JWT or opaque string)
Refresh Token	Long-lived credential used to obtain new access tokens without re-authentication
ID Token	JWT containing user identity claims (OIDC only)
Scope	Permission boundary (e.g., `openid profile email read:documents`)

Grant Types

Grant Type	Use Case	RFC
Authorization Code	Server-side web apps, SPAs (with PKCE), mobile apps	RFC 6749 Section 4.1
Authorization Code + PKCE	Public clients (SPAs, native apps) -- now recommended for all clients	RFC 7636
Client Credentials	Machine-to-machine (no user involved)	RFC 6749 Section 4.4
Device Authorization	Input-constrained devices (smart TVs, CLI tools)	RFC 8628
Refresh Token	Obtain new access token without user interaction	RFC 6749 Section 6
~~Implicit~~	~~SPAs (deprecated -- use Authorization Code + PKCE instead)~~	~~RFC 6749 Section 4.2~~
~~Resource Owner Password~~	~~Direct username/password (deprecated)~~	~~RFC 6749 Section 4.3~~

Authorization Code Flow

The most common and secure flow for interactive users:

[Diagram: see online version]

Authorization Request Parameters

Parameter	Required	Description
response_type	Yes	`code` for authorization code flow
client_id	Yes	Client identifier issued during registration

redirect_uri	Yes	Where to send the user after authorization
scope	Recommended	Space-separated list of requested permissions
state	Recommended	Opaque value for CSRF protection (client verifies on callback)
code_challenge	PKCE	Base64url-encoded SHA-256 hash of the code verifier
code_challenge_method	PKCE	`S256` (recommended) or `plain`

Token Request Parameters

Parameter	Required	Description
grant_type	Yes	`authorization_code`
code	Yes	The authorization code received from the AS
redirect_uri	Yes	Must match the original request
client_id	Yes	Client identifier
client_secret	Confidential clients	Client secret (not sent by public clients)
code_verifier	PKCE	Original random string used to generate code_challenge

Authorization Code Flow with PKCE

PKCE (Proof Key for Code Exchange, pronounced "pixy") prevents authorization code interception attacks. It is now recommended for all OAuth clients, not just public ones.

[Diagram: see online version]

Client Credentials Flow

For server-to-server communication where no user is involved:

[Diagram: see online version]

Token Types

Access Token

Used to authorize API requests. Typically short-lived (minutes to hours).

Format	Description

JWT (Self-Contained)	Signed JSON token -- resource server validates locally by checking signature
Opaque (Reference)	Random string -- resource server must call AS introspection endpoint to validate

Refresh Token

Used to obtain new access tokens. Longer-lived (hours to days). Must be stored securely and is only sent to the token endpoint, never to resource servers.

ID Token (OIDC)

A JWT that contains authenticated user identity claims. Only present when the `openid` scope is requested.

JWT Structure

JSON Web Tokens (RFC 7519) consist of three Base64url-encoded parts separated by dots:

[Diagram: see online version]

Format: `xxxxx.yyyyy.zzzzz`

JWT Header

Field	Description
alg	Signing algorithm (`RS256`, `ES256`, `EdDSA`, `PS256`)
typ	Token type (`JWT`)
kid	Key ID -- identifies which key from the JWKS was used to sign

JWT Payload (Claims)

Registered Claims (RFC 7519)

Claim	Name	Description
iss	Issuer	Who issued the token (`https://auth.example.com`)
sub	Subject	Unique identifier for the user
aud	Audience	Intended recipient(s) of the token (client_id or API identifier)
exp	Expiration	Token expiry (Unix timestamp)
nbf	Not Before	Token not valid before this time
iat	Issued At	When the token was issued

jti	JWT ID	Unique token identifier (for revocation / replay prevention)

Common OIDC Claims

Claim	Scope Required	Description
name	profile	Full name
given_name	profile	First name
family_name	profile	Last name
email	email	Email address
email_verified	email	Whether email has been verified
picture	profile	Profile picture URL
locale	profile	Locale (e.g., `en-US`)
updated_at	profile	Last profile update time

JWT Signature

The signature is computed over the encoded header and payload:

```
RSASSA-PKCS1-v1_5-SIGN(
  SHA-256(BASE64URL(header) + "." + BASE64URL(payload)),
  private_key
)
```

The resource server verifies the signature using the public key from the AS's JWKS endpoint.

OpenID Connect

OIDC adds identity and authentication to OAuth 2.0:

OIDC Feature	Description
ID Token	JWT proving user authentication (contains `sub`, `iss`, `aud`, `exp`, `iat`, `nonce`)
UserInfo Endpoint	`GET /userinfo` with access token to retrieve additional claims
Discovery	`GET /.well-known/openid-configuration` returns all endpoint URLs and capabilities
JWKS Endpoint	`GET /jwks` provides public keys for token signature verification
Dynamic Registration	Clients can register programmatically (RFC 7591)
Session Management	Front-channel and back-channel logout

OIDC Discovery Document

Fetched from `https://{issuer}/.well-known/openid-configuration`:

Field	Description
issuer	Authorization server identifier
authorization_endpoint	URL for authorization requests
token_endpoint	URL for token exchange
userinfo_endpoint	URL for user profile claims
jwks_uri	URL for JSON Web Key Set (public signing keys)
scopes_supported	Available scopes (e.g., `openid`, `profile`, `email`)
response_types_supported	Supported response types (`code`, `id_token`, `token`)
grant_types_supported	Supported grant types
id_token_signing_alg_values_supported	Signing algorithms for ID tokens
token_endpoint_auth_methods_supported	Client auth methods (`client_secret_basic`, `client_secret_post`, `private_key_jwt`)

OIDC Standard Scopes

Scope	Claims Returned
openid	`sub` (required scope for OIDC)
profile	`name`, `family_name`, `given_name`, `middle_name`, `nickname`, `picture`, `updated_at`, etc.
email	`email`, `email_verified`
address	`address` (structured JSON)
phone	`phone_number`, `phone_number_verified`
offline_access	Requests a refresh token

Token Validation

Access Token Validation at Resource Server

Step	Description
1. Extract	Parse `Authorization: Bearer <token>` header
2. Decode	Base64url-decode the three JWT parts
3. Verify Signature	Fetch JWKS from `jwks_uri`, find key by `kid`, verify signature
4. Check `exp`	Reject if token is expired
5. Check `iss`	Verify issuer matches expected authorization server

6. Check aud	Verify audience includes this resource server's identifier
7. Check Scopes	Verify token has required scope(s) for the requested operation

Token Introspection (RFC 7662)

For opaque tokens, the resource server calls the AS:

```
POST /introspect
token=&lt;opaque_access_token&gt;

Response:
{
  "active": true,
  "scope": "read write",
  "client_id": "app123",
  "sub": "user456",
  "exp": 1700000000
}
```

Token Revocation (RFC 7009)

```
POST /revoke
token=&lt;refresh_token&gt;&amp;token_type_hint=refresh_token
```

Security Considerations

Threat	Mitigation
Authorization code interception	PKCE (code_challenge / code_verifier)
CSRF attacks	`state` parameter validated on callback
Token leakage in browser history	Use `response_mode=fragment` or POST; avoid tokens in URLs
Token theft	Short-lived access tokens, token binding, sender-constrained tokens (DPoP)
Open redirect	Strict `redirect_uri` validation (exact match)
Refresh token theft	Rotation (issue new refresh token on each use, invalidate old one)
JWT algorithm confusion	Validate `alg` header against expected algorithms; never accept `none`
JWKS spoofing	Pin `jwks_uri` to trusted issuer; validate TLS certificate

Encapsulation

[Diagram: see online version]

OAuth 2.0 and OIDC are carried entirely over HTTPS. Authorization requests use HTTP redirects (302). Token requests use HTTP POST. API calls carry the access token in the `Authorization: Bearer` header.

Standards

Document	Title
RFC 6749	The OAuth 2.0 Authorization Framework
RFC 6750	The OAuth 2.0 Authorization Framework: Bearer Token Usage
RFC 7636	Proof Key for Code Exchange by OAuth Public Clients (PKCE)
RFC 7519	JSON Web Token (JWT)
RFC 7517	JSON Web Key (JWK)
RFC 7662	OAuth 2.0 Token Introspection
RFC 7009	OAuth 2.0 Token Revocation
RFC 8628	OAuth 2.0 Device Authorization Grant
RFC 9449	OAuth 2.0 Demonstrating Proof of Possession (DPoP)
OpenID Connect Core 1.0	OpenID Connect Core specification
OpenID Connect Discovery 1.0	OpenID Connect Discovery specification
RFC 7591	OAuth 2.0 Dynamic Client Registration

See Also

- TLS -- transport security (OAuth 2.0 requires HTTPS)
- HTTP -- underlying protocol for all OAuth/OIDC exchanges
- SAML -- alternative federated identity protocol (XML-based)
- Kerberos -- ticket-based authentication for enterprise networks
- RADIUS -- AAA protocol often used alongside OAuth in network access

RADIUS -- Remote Authentication Dial-In User Service

Standard: RFC 2865 | **Layer:** Application (Layer 7) | **Wireshark filter:** `radius`

RADIUS is the dominant AAA (Authentication, Authorization, and Accounting) protocol for network access control. It authenticates users connecting via Wi-Fi (WPA Enterprise), VPN, dial-up, wired 802.1X, and ISP broadband, and tracks their usage for billing and auditing. RADIUS uses a client-server model where the Network Access Server (NAS) -- a switch, AP, or VPN concentrator -- is the RADIUS client, and a central RADIUS server validates credentials. It is implemented in virtually every enterprise network.

Packet

[Diagram: see online version]

Key Fields

Field	Size	Description
Code	8 bits	Packet type
Identifier	8 bits	Matches requests to responses (0-255)
Length	16 bits	Total packet length (20-4096 bytes)
Authenticator	128 bits	Request: random nonce; Response: MD5 hash for integrity
Attributes	Variable	TLV-encoded attribute-value pairs

Packet Types (Code)

Code	Name	Direction	Description
1	Access-Request	NAS -> Server	Authentication request with credentials
2	Access-Accept	Server -> NAS	Authentication successful
3	Access-Reject	Server -> NAS	Authentication failed
4	Accounting-Request	NAS -> Server	Usage data (start, stop, interim)
5	Accounting-Response	Server -> NAS	Acknowledge accounting data
11	Access-Challenge	Server -> NAS	Additional information needed (e.g., OTP, EAP)
12	Status-Server	NAS -> Server	Keepalive / server status check
13	Status-Client	Server -> NAS	Response to status check

Attribute Format

[Diagram: see online version]

Common Attributes

Type	Name	Description
1	User-Name	Username being authenticated

2	User-Password	Encrypted password (MD5 with shared secret)
4	NAS-IP-Address	IP of the NAS sending the request
5	NAS-Port	Physical or virtual port on the NAS
6	Service-Type	Requested service (Login, Framed, etc.)
7	Framed-Protocol	Protocol for framed access (PPP, SLIP)
8	Framed-IP-Address	IP to assign to the user
12	Framed-MTU	Maximum transmission unit
18	Reply-Message	Text message to display to user
24	State	Session state for multi-round exchanges
25	Class	Opaque value passed to accounting
26	Vendor-Specific	Vendor-defined attributes (encapsulated)
27	Session-Timeout	Maximum session time in seconds
29	Termination-Action	Action on session end (Default, RADIUS-Request)
30	Called-Station-Id	Phone number or SSID called
31	Calling-Station-Id	Phone number or MAC of caller
32	NAS-Identifier	String identifying the NAS
40	Acct-Status-Type	Start (1), Stop (2), Interim-Update (3)
44	Acct-Session-Id	Unique session identifier
46	Acct-Session-Time	Session duration in seconds
47	Acct-Input-Octets	Bytes received from user
48	Acct-Output-Octets	Bytes sent to user
61	NAS-Port-Type	Physical port type (Ethernet, Wireless, Virtual)
79	EAP-Message	Encapsulated EAP data
80	Message-Authenticator	HMAC-MD5 integrity check (required for EAP)

Authentication Flow

Simple (PAP)

[Diagram: see online version]

EAP (802.1X / WPA Enterprise)

[Diagram: see online version]

Accounting

[Diagram: see online version]

EAP Methods

Method	Description
EAP-TLS	Mutual TLS with client certificates (most secure)
EAP-PEAP	TLS tunnel + inner MSCHAPv2 (most common in enterprise Wi-Fi)
EAP-TTLS	TLS tunnel + inner PAP/CHAP/MSCHAPv2
EAP-FAST	Cisco's tunnel method with PAC provisioning
EAP-SIM/AKA	SIM card authentication (mobile carrier Wi-Fi offload)

Vendor-Specific Attributes (VSA)

Attribute type 26 encapsulates vendor-defined attributes:

[Diagram: see online version]

Common vendors: Cisco (9), Microsoft (311), Juniper (2636).

Encapsulation

[Diagram: see online version]

Legacy ports 1645 (auth) and 1646 (accounting) are still seen on older equipment.

Standards

Document	Title
RFC 2865	Remote Authentication Dial In User Service (RADIUS)
RFC 2866	RADIUS Accounting
RFC 3579	RADIUS Support for EAP
RFC 5176	Dynamic Authorization Extensions (CoA, Disconnect)
RFC 6614	TLS Encryption for RADIUS (RadSec)
RFC 3748	Extensible Authentication Protocol (EAP)

See Also

- UDP
- TLS -- used by RadSec and EAP-TLS
- LDAP -- RADIUS servers often authenticate against LDAP/AD
- SNMP -- another ubiquitous network management protocol

SAML 2.0 -- Security Assertion Markup Language

Standard: OASIS SAML 2.0 Core | **Layer:** Application (Layer 7) | **Wireshark filter:** `http contains "SAMLRequest" or http contains "SAMLResponse" or http contains "saml"`

SAML 2.0 is an XML-based open standard for exchanging authentication and authorization data between an Identity Provider (IdP) and a Service Provider (SP). It enables web-based single sign-on (SSO), allowing users to authenticate once with their organization's IdP and access multiple SPs without re-entering credentials. SAML is the dominant SSO protocol in enterprise environments and is widely supported by identity providers such as Active Directory Federation Services (AD FS), Okta, Ping Identity, and Shibboleth.

Core Concepts

Component	Description
Identity Provider (IdP)	Authenticates users and issues SAML assertions (e.g., AD FS, Okta, Azure AD)
Service Provider (SP)	Application that relies on the IdP for authentication (e.g., Salesforce, AWS Console)
Assertion	XML document containing authentication, attribute, and authorization statements
Binding	Method of transporting SAML messages (HTTP-Redirect, HTTP-POST, SOAP, Artifact)
Profile	Combination of assertions, protocols, and bindings for a use case (e.g., Web Browser SSO)
Metadata	XML document describing an IdP or SP's endpoints, certificates, and capabilities
Name ID	Unique identifier for the subject (user) -- email, persistent ID, or transient ID
Relay State	Opaque value that preserves the user's original destination URL across the SSO flow

SP-Initiated SSO Flow

The most common flow. The user starts at the SP and is redirected to the IdP:

[Diagram: see online version]

AuthnRequest (SP to IdP)

The SP generates and sends this to request authentication:

Field	Description
ID	Unique request identifier (for correlation with response)
Version	`2.0`
IssueInstant	Timestamp of request creation (UTC)
Destination	IdP's SSO endpoint URL
AssertionConsumerServiceURL	SP's ACS endpoint where the response should be POSTed
Issuer	SP's entity ID (unique identifier)
NameIDPolicy	Requested format for the user identifier (e.g., email, persistent, transient)
RequestedAuthnContext	Requested authentication strength (e.g., password, MFA)
ForceAuthn	If `true`, IdP must re-authenticate even if session exists
IsPassive	If `true`, IdP must not interact with user (silent check)

IdP-Initiated SSO Flow

The user starts at the IdP portal and selects a service:

[Diagram: see online version]

Note: IdP-initiated SSO is more susceptible to replay attacks because there is no corresponding AuthnRequest with an ID to correlate against.

SAML Assertion Structure

A SAML Response contains one or more Assertions. Each Assertion is a signed XML document:

[Diagram: see online version]

Key Assertion Elements

Element	Description
Issuer	Entity ID of the IdP that created the assertion
Signature	XML Digital Signature (RSA-SHA256 or similar) over the assertion or response
Subject	Identifies the authenticated user
Subject / NameID	User identifier (email, persistent ID, transient ID, or Windows domain name)

Subject / SubjectConfirmation	How the subject is confirmed (`urn:oasis:names:tc:SAML:2.0:cm:bearer`)
Subject / SubjectConfirmationData	Constraints: `NotOnOrAfter`, `Recipient` (ACS URL), `InResponseTo` (request ID)
Conditions	Validity constraints for the assertion
Conditions / AudienceRestriction	Entity ID(s) of the intended SP(s)
Conditions / NotBefore	Assertion not valid before this time
Conditions / NotOnOrAfter	Assertion not valid after this time
AuthnStatement	Describes the authentication event
AuthnStatement / AuthnInstant	When the user authenticated at the IdP
AuthnStatement / SessionIndex	Session identifier for single logout correlation
AuthnStatement / AuthnContext	How the user authenticated (password, MFA, certificate, Kerberos)
AttributeStatement	User attributes (name, email, roles, groups)

NameID Formats

Format URI (shortened)	Description
`...:nameid-format:emailAddress`	Email address (e.g., `user@example.com`)
`...:nameid-format:persistent`	Opaque persistent identifier (privacy-preserving)
`...:nameid-format:transient`	Temporary identifier (changes per session)
`...:nameid-format:unspecified`	IdP decides the format
`...:nameid-format:WindowsDomainQualifiedName`	`DOMAIN\username` (AD environments)
`...:nameid-format:kerberos`	`user@REALM` (Kerberos principal)

Authentication Context Classes

Context Class (shortened)	Meaning
`...:ac:classes:Password`	Password authentication
`...:ac:classes:PasswordProtectedTransport`	Password over TLS
`...:ac:classes:X509`	X.509 certificate
`...:ac:classes:Kerberos`	Kerberos ticket
`...:ac:classes:TLSClient`	TLS client certificate
`...:ac:classes:unspecified`	Unspecified method

Bindings

SAML bindings define how messages are transported between IdP and SP:

Binding	Transport	Message Size	Use Case
HTTP-Redirect	GET query parameter (deflate + base64)	Small only (URL length limit)	AuthnRequest from SP to IdP
HTTP-POST	Hidden HTML form field (base64)	Large messages supported	SAMLResponse from IdP to SP
HTTP-Artifact	Small reference token, then SOAP back-channel resolve	Any size (resolved server-side)	When assertions must not pass through the browser
SOAP	Direct HTTPS call between servers	Any size	Artifact resolution, Attribute Query, single logout

HTTP-Redirect Binding

The SAMLRequest is deflated, base64-encoded, and URL-encoded in the query string:

```
GET /sso?SAMLRequest=&lt;deflate+base64+urlencode&gt;&amp;RelayState=&lt;target_url&gt;&amp;SigAlg=...&amp;Signa
```

Signatures are applied to the query string itself (not inside the XML).

HTTP-POST Binding

The SAMLResponse is base64-encoded and delivered via a self-submitting HTML form:

```
&lt;form method="POST" action="https://sp.example.com/acs"&gt;
  &lt;input type="hidden" name="SAMLResponse" value="&lt;base64-encoded XML&gt;" /&gt;
  &lt;input type="hidden" name="RelayState" value="&lt;target_url&gt;" /&gt;
&lt;/form&gt;
&lt;script&gt;document.forms[0].submit();&lt;/script&gt;
```

Signatures are embedded inside the XML (`<ds:Signature>`).

Artifact Binding Flow

[Diagram: see online version]

Metadata

SAML metadata is an XML document that describes an entity's SSO configuration. It enables automated trust setup between IdPs and SPs.

IdP Metadata Key Elements

Element	Description
EntityDescriptor / entityID	Unique identifier for the IdP
IDPSSODescriptor	Contains IdP-specific SSO configuration

SingleSignOnService	SSO endpoint URL and binding type
SingleLogoutService	SLO endpoint URL and binding type
KeyDescriptor (signing)	X.509 certificate used to sign assertions
KeyDescriptor (encryption)	X.509 certificate used to encrypt assertions
NameIDFormat	Supported NameID formats

SP Metadata Key Elements

Element	Description
EntityDescriptor / entityID	Unique identifier for the SP
SPSSODescriptor	Contains SP-specific SSO configuration
AssertionConsumerService	ACS endpoint URL(s), binding, and index
SingleLogoutService	SLO endpoint URL and binding type
KeyDescriptor (signing)	X.509 certificate for signed AuthnRequests
KeyDescriptor (encryption)	X.509 certificate for encrypted assertions
NameIDFormat	Requested NameID format(s)
AuthnRequestsSigned	Whether the SP signs AuthnRequests (`true`/`false`)
WantAssertionsSigned	Whether the SP requires signed assertions (`true`/`false`)

Single Logout (SLO)

[Diagram: see online version]

SLO is notoriously fragile in practice because any SP that is unreachable or returns an error can break the logout chain.

Assertion Validation Checklist

Step	Validation
1	Verify XML digital signature against IdP's public certificate (from metadata)
2	Check `Issuer` matches the expected IdP entity ID
3	Check `Destination` matches this SP's ACS URL
4	Check `AudienceRestriction` includes this SP's entity ID
5	Check `NotBefore` and `NotOnOrAfter` -- assertion must be within validity window
6	Check `SubjectConfirmationData / NotOnOrAfter` is not expired

7	Check `SubjectConfirmationData / Recipient` matches this SP's ACS URL
8	Check `InResponseTo` matches the original AuthnRequest ID (SP-initiated only)
9	Verify the assertion has not been previously consumed (replay prevention)
10	If assertion is encrypted, decrypt using SP's private key before validation

Security Considerations

Threat	Mitigation
Assertion forgery	Require XML digital signatures; validate against IdP certificate from trusted metadata
XML Signature wrapping	Use strict schema validation; verify signed element is the one being processed
Replay attacks	Enforce `NotOnOrAfter`; track consumed assertion IDs; validate `InResponseTo`
Man-in-the-middle	All endpoints must use HTTPS; sign AuthnRequests
Assertion eavesdropping	Encrypt assertions using SP's public key (`EncryptedAssertion`)
IdP-initiated SSO risks	No `InResponseTo` to validate -- higher replay risk; prefer SP-initiated flow
Open redirect via RelayState	Validate RelayState URLs against an allowlist
XXE / XML injection	Disable external entity resolution and DTD processing in XML parser

SAML vs OAuth 2.0 / OIDC

Feature	SAML 2.0	OAuth 2.0 / OIDC
Primary purpose	Authentication (SSO)	Authorization (delegated access) + Authentication (OIDC)
Data format	XML	JSON (JWT)
Token type	SAML Assertion (XML)	Access Token + ID Token (JWT)
Transport	HTTP Redirect, HTTP POST, SOAP	HTTPS (REST APIs, redirects)
Mobile / SPA support	Poor (XML parsing, large payloads)	Native (compact JSON, PKCE)
Enterprise adoption	Dominant in legacy enterprise SSO	Growing rapidly, especially for APIs and modern apps
API authorization	Not designed for API access	Core use case
Token size	Large (XML, often 5-20 KB)	Small (JWT, typically 1-2 KB)

Complexity	High (XML signatures, canonicalization, multiple bindings)	Lower (JSON, standard HTTPS flows)
Specification	OASIS (multiple documents)	IETF RFCs + OpenID Foundation
Logout	Single Logout (fragile in practice)	Back-channel logout, front-channel logout

Encapsulation

[Diagram: see online version]

SAML messages are carried entirely over HTTPS. AuthnRequests typically use HTTP-Redirect (GET with query parameters). SAMLResponses typically use HTTP-POST (hidden form fields). Artifact resolution and SLO back-channel use SOAP over HTTPS.

Standards

Document	Title
SAML 2.0 Core	Assertions and Protocols
SAML 2.0 Bindings	Protocol Bindings (HTTP-Redirect, HTTP-POST, SOAP, Artifact)
SAML 2.0 Profiles	Profiles including Web Browser SSO and Single Logout
SAML 2.0 Metadata	Metadata schema for IdP and SP configuration exchange
SAML 2.0 Conformance	Conformance Requirements and Test Procedures
XML Signature	W3C XML Digital Signatures (used by SAML for assertion signing)
XML Encryption	W3C XML Encryption (used for encrypted assertions)

See Also

- OAuth 2.0 / OIDC -- modern alternative for SSO and API authorization
- TLS -- transport security (SAML requires HTTPS)
- HTTP -- underlying transport protocol
- Kerberos -- often used as IdP authentication method behind SAML
- RADIUS -- AAA protocol used in network access alongside SAML federation
- 802.1X -- network access control that can integrate with SAML-based identity systems

SCIM -- System for Cross-domain Identity Management

Standard: RFC 7643 / RFC 7644 | **Layer:** Application (Layer 7) | **Wireshark filter:** `http` or `http2`

SCIM is a RESTful protocol for automating the provisioning and deprovisioning of user identities across systems. When an employee joins, changes roles, or leaves an organization, SCIM allows identity providers (Okta, Azure AD, OneLogin) to automatically create, update, or disable accounts in downstream service providers (Slack, GitHub, AWS, Salesforce). SCIM standardizes the schema for user and group resources, the HTTP API for CRUD operations, and the filtering/pagination syntax, replacing proprietary provisioning integrations with a single open standard.

Request Structure

SCIM uses standard HTTP methods on RESTful endpoints:

[Diagram: see online version]

Endpoints

Endpoint	Methods	Description
/Users	GET, POST	List/search users, create a user
/Users/{id}	GET, PUT, PATCH, DELETE	Read, replace, modify, or delete a specific user
/Groups	GET, POST	List/search groups, create a group
/Groups/{id}	GET, PUT, PATCH, DELETE	Read, replace, modify, or delete a specific group
/Bulk	POST	Execute multiple operations in a single request
/ServiceProviderConfig	GET	Discover server capabilities (supported features)
/ResourceTypes	GET	Discover available resource types and their schemas
/Schemas	GET	Retrieve the full schema definitions
/Me	GET, PATCH	Current authenticated user (optional)

HTTP Operations

Method	Operation	Success Code	Description
GET	Read	200	Retrieve a resource or list of resources
POST	Create	201	Create a new resource
PUT	Replace	200	Fully replace a resource
PATCH	Modify	200	Partially update specific attributes

DELETE	Remove	204	Delete (or deactivate) a resource

User Resource

Core attributes for a SCIM User:

Attribute	Type	Required	Description
schemas	String[]	Yes	Schema URIs (e.g., "urn:ietf:params:scim:schemas:core:2.0:User")
id	String	Server-set	Unique server-assigned identifier
externalId	String	No	Identifier from the provisioning client
userName	String	Yes	Unique login identifier (often email)
name.givenName	String	No	First name
name.familyName	String	No	Last name
name.formatted	String	No	Full formatted name
emails[]	Complex	No	Email addresses (value, type, primary)
phoneNumbers[]	Complex	No	Phone numbers (value, type)
active	Boolean	No	Account enabled/disabled status
groups[]	Complex	Read-only	Groups the user belongs to
roles[]	Complex	No	Roles assigned to the user
title	String	No	Job title
department	String	No	Department name (Enterprise extension)
manager	Complex	No	Manager reference (Enterprise extension)

Group Resource

Attribute	Type	Required	Description
schemas	String[]	Yes	Schema URIs
id	String	Server-set	Unique identifier
displayName	String	Yes	Human-readable group name
members[]	Complex	No	Array of {value (user id), display, type}

User Provisioning Flow

[Diagram: see online version]

Group Membership Sync

[Diagram: see online version]

Filtering

SCIM supports a rich filter syntax for querying resources:

Operator	Example	Description
eq	`userName eq "alice"`	Equals
ne	`active ne true`	Not equals
co	`name.familyName co "Smith"`	Contains
sw	`userName sw "a"`	Starts with
ew	`emails.value ew "@corp.com"`	Ends with
gt, ge, lt, le	`meta.lastModified gt "2025-01-01"`	Comparison
and, or, not	`active eq true and department eq "Eng"`	Logical combinators

Example: `GET /scim/v2/Users?filter=userName eq "alice@corp.com" and active eq true`

Pagination

Parameter	Description
startIndex	1-based index of the first result to return
count	Maximum number of results per page
totalResults	Total number of matching resources (in response)
itemsPerPage	Actual number of results returned (in response)

Bulk Operations

```
POST /scim/v2/Bulk
Content-Type: application/scim+json
```

Field	Description
schemas	["urn:ietf:params:scim:api:messages:2.0:BulkRequest"]
Operations[]	Array of {method, path, bulkId, data}
failOnErrors	Number of errors before the server stops processing

Each operation specifies an HTTP method, path, and optional data. The `bulkId` allows cross-referencing newly created resources within the same batch.

Discovery

Endpoint	Returns
/ServiceProviderConfig	Supported features: patch, bulk, filter, changePassword, sort, etag, authSchemes
/ResourceTypes	Available types (User, Group) with schema URIs and endpoints
/Schemas	Full attribute definitions including mutability, type, and constraints

SCIM vs LDAP

Feature	SCIM	LDAP
Protocol	REST over HTTPS	Binary over TCP (port 389/636)
Data format	JSON	ASN.1 / BER encoding
Authentication	OAuth 2.0 Bearer tokens	SASL, Simple Bind
Direction	Push (IdP provisions to SP)	Pull (client queries directory)
Schema	Predefined User/Group + extensions	Flexible, schema-on-write
Filtering	URL query parameters	LDAP filter syntax (RFC 4515)
Use case	Cloud provisioning across domains	On-premise directory access
Firewall	HTTPS (port 443) friendly	Requires port 389/636 open
Bulk operations	Built-in /Bulk endpoint	LDIF import (offline)
Real-time sync	Webhook or polling	Persistent search / syncrepl
Adoption	SaaS (Okta, Azure AD, Google)	Enterprise (Active Directory, OpenLDAP)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 7643	SCIM: Core Schema
RFC 7644	SCIM: Protocol
RFC 7642	SCIM: Definitions, Overview, Concepts, and Requirements

See Also

- HTTP -- SCIM transport
- OAuth 2.0 -- typical authentication for SCIM endpoints
- TLS -- encrypts SCIM traffic

Signal Protocol -- Double Ratchet

Standard: Signal Protocol Specifications | **Layer:** Application (Layer 7) | **Wireshark filter:** N/A (end-to-end encrypted; only ciphertext visible on the wire)

The Signal Protocol is an end-to-end encryption protocol providing forward secrecy and post-compromise security for instant messaging. It combines the X3DH (Extended Triple Diffie-Hellman) key agreement protocol with the Double Ratchet algorithm to establish and maintain encrypted sessions. Originally developed for Signal (formerly TextSecure), it is now used by WhatsApp (2+ billion users), Google Messages (RCS), Facebook Messenger, and Skype. The protocol ensures that even if long-term keys are compromised, past messages remain secret (forward secrecy) and future messages become secure again after a single round trip (post-compromise security / self-healing).

Components

The Signal Protocol consists of two main components:

Component	Purpose	Description
X3DH	Initial key agreement	Establishes a shared secret between two parties, even if the recipient is offline
Double Ratchet	Ongoing encryption	Derives unique keys for every message using a ratcheting mechanism

X3DH Key Types

Each user publishes a set of keys to the server:

Key	Type	Lifetime	Description
Identity Key (IK)	Curve25519	Long-term	Permanent identity key pair; represents the user
Signed Pre-Key (SPK)	Curve25519	Medium-term	Signed by IK; rotated periodically (e.g., weekly)
One-Time Pre-Keys (OPK)	Curve25519	Single use	Pool of keys; each consumed by one session setup
Ephemeral Key (EK)	Curve25519	Single use	Generated fresh by the initiator for each new session

X3DH Key Agreement

When Alice wants to start a session with Bob (who may be offline), she fetches Bob's pre-key bundle from the server and computes four DH exchanges:

[Diagram: see online version]

Exchange	Initiator Key	Responder Key	Purpose
DH1	IK_A (identity)	SPK_B (signed pre-key)	Authenticates Alice to Bob
DH2	EK_A (ephemeral)	IK_B (identity)	Authenticates Bob to Alice
DH3	EK_A (ephemeral)	SPK_B (signed pre-key)	Forward secrecy (ephemeral component)
DH4	EK_A (ephemeral)	OPK_B (one-time pre-key)	Additional forward secrecy (optional, if OPK available)

The shared secret is: `SK = KDF(DH1 || DH2 || DH3 || DH4)`

This SK becomes the initial root key for the Double Ratchet.

X3DH Flow

[Diagram: see online version]

Double Ratchet Algorithm

After X3DH establishes the initial shared secret, the Double Ratchet takes over. It combines two ratcheting mechanisms:

Ratchet	Type	Advances When	Provides
DH Ratchet	Asymmetric	Each message exchange (new DH key pair)	Post-compromise security
Symmetric Ratchet	KDF Chain	Every message sent/received	Per-message keys

Three KDF Chains

The Double Ratchet maintains three KDF (Key Derivation Function) chains:

Chain	Purpose	Advances
Root Chain	Derives new chain keys when DH ratchet advances	On each DH ratchet step
Sending Chain	Derives message keys for outgoing messages	On each sent message
Receiving Chain	Derives message keys for incoming messages	On each received message

Message Format

[Diagram: see online version]

Field	Description
DH Ratchet Public Key	Sender's current ratchet public key (Curve25519, 32 bytes)
Previous Chain Length	Number of messages in the previous sending chain
Message Number	Index in the current sending chain
Ciphertext	AES-256-CBC + HMAC-SHA-256 encrypted message

Double Ratchet Message Exchange

[Diagram: see online version]

Security Properties

Property	Description
Forward Secrecy	Compromised long-term keys cannot decrypt past messages --- ephemeral keys are deleted
Post-Compromise Security	After a key compromise, the ratchet "heals" on the next DH exchange --- future messages are secure
Per-Message Keys	Every message uses a unique encryption key derived from the KDF chain
Out-of-Order Delivery	Skipped message keys are cached temporarily to handle messages arriving out of order
Deniability	No cryptographic proof that a specific user sent a message (both parties can forge MACs)
Asynchronous Setup	X3DH allows session establishment even when the recipient is offline

Sealed Sender (Signal-specific)

Signal extends the protocol with sealed sender --- the server does not learn who sent a message:

Component	Description
Sender Certificate	Short-lived certificate signed by Signal, includes sender identity and device
Encrypted Envelope	Sender encrypts the sender certificate along with the message using the recipient's identity key
Server Routing	Server routes based on recipient only; sender identity is inside the encrypted payload

Group Messaging (Sender Keys)

For group chats, Signal uses Sender Keys for efficiency:

Concept	Description
Sender Key	Each group member generates a signing key pair and a symmetric chain key
Distribution	Sender Keys are distributed to all group members via pairwise Signal Protocol sessions
Encryption	Sender encrypts once with their chain key; all members can decrypt
Ratchet	Chain key ratchets forward with each message (forward secrecy within the chain)
Limitation	No post-compromise security until sender key is rotated (on membership change)

Signal Protocol vs PGP/GPG vs Matrix Olm

Feature	Signal Protocol	PGP/GPG	Matrix Olm/Megolm
Forward secrecy	Yes (DH ratchet)	No (static RSA/ECC keys)	Partial (Megolm: forward only)
Post-compromise security	Yes (heals on DH exchange)	No	No (Megolm sessions are one-directional)
Key management	Automatic (transparent to user)	Manual (key exchange, web of trust)	Automatic (server-assisted)
Asynchronous setup	Yes (X3DH pre-keys)	Yes (public key on keyserver)	Yes (one-time keys on homeserver)
Group encryption	Sender Keys	Encrypt to each recipient	Megolm (single ratchet per room)
Deniability	Yes (no binding signatures)	No (signatures are non-repudiable)	Partial
Key rotation	Every message exchange (DH ratchet)	Manual	Per-session (Megolm)

Adoption	Signal, WhatsApp, Google Messages, FB Messenger	Email encryption, file signing	Matrix/Element ecosystem
Primitives	Curve25519, AES-256, HMAC-SHA-256	RSA/ECC, AES, SHA-2	Curve25519, AES-256, HMAC-SHA-256
Specification	Open (signal.org/docs)	RFC 4880	Open (spec.matrix.org)

Cryptographic Primitives

Function	Algorithm	Purpose
Key Agreement	Curve25519 (X25519)	DH key exchange (X3DH and ratchet)
Signing	Ed25519	Identity key signatures, pre-key signatures
Symmetric Encryption	AES-256-CBC	Message encryption
MAC	HMAC-SHA-256	Message authentication
KDF	HKDF-SHA-256	Derive chain keys and message keys

Standards

Document	Title
X3DH Specification	Extended Triple Diffie-Hellman Key Agreement Protocol
Double Ratchet Specification	The Double Ratchet Algorithm
Sesame Specification	Session Management (multi-device)
Sender Key Specification	Group Messaging with Sender Keys

See Also

- TLS --- transport-layer encryption (different layer; Signal is application-layer E2EE)
- Matrix --- federated messaging using Olm/Megolm (derived from Signal Protocol)
- XMPP --- messaging protocol with OMEMO extension (uses Signal Protocol)
- WireGuard --- VPN encryption (network-layer, different use case)

TACACS+ -- Terminal Access Controller Access-Control System Plus

Standard: RFC 8907 | **Layer:** Application (Layer 7) | **Wireshark filter:** `tacplus`

TACACS+ is an AAA (Authentication, Authorization, and Accounting) protocol used primarily for device administration -- controlling who can log into network equipment (routers, switches, firewalls) and what commands they can execute. Unlike RADIUS, which combines authentication and authorization in a single exchange, TACACS+ separates all three AAA functions into independent operations. TACACS+ encrypts the entire packet body (not just the password), runs over TCP port 49 for reliable delivery, and provides per-command authorization -- making it the standard for managing Cisco and multi-vendor network infrastructure.

Packet Header

[Diagram: see online version]

Key Fields

Field	Size	Description
Major Version	8 bits	Protocol major version (0xC0 = TACACS+)
Minor Version	4 bits	Minor version (0x00 = default, 0x01 = minor v1)
Type	4 bits	Packet type (Authentication, Authorization, Accounting)
Seq No	8 bits	Sequence number (starts at 1, increments per packet in session)
Flags	8 bits	Control flags (encryption, single-connect)
Session ID	32 bits	Random value identifying the session
Length	32 bits	Length of the body (after this header)

Field Details

Type

Value	Name	Description
0x01	TAC_PLUS_AUTHEN	Authentication
0x02	TAC_PLUS_AUTHOR	Authorization
0x03	TAC_PLUS_ACCT	Accounting

Flags

Bit	Name	Description
0	TAC_PLUS_UNENCRYPTED_FLAG	Body is unencrypted (debugging only)

2	TAC_PLUS_SINGLE_CONNECT_FLAG	Multiplex sessions on a single TCP connection

Packet Types

Authentication

Authentication uses three packet sub-types exchanged in a multi-step conversation:

Packet	Direction	Description
START	Client -> Server	Initiates authentication (username, authen_type, action)
REPLY	Server -> Client	Server response (PASS, FAIL, GETDATA, GETUSER, GETPASS, ERROR, RESTART)
CONTINUE	Client -> Server	Client provides requested data (password, OTP, etc.)

Authentication START Body

[Diagram: see online version]

Authentication REPLY Body

[Diagram: see online version]

Status	Value	Description
TAC_PLUS_AUTHEN_STATUS_PASS	0x01	Authentication successful
TAC_PLUS_AUTHEN_STATUS_FAIL	0x02	Authentication failed
TAC_PLUS_AUTHEN_STATUS_GETDATA	0x03	Server requests additional data
TAC_PLUS_AUTHEN_STATUS_GETUSER	0x04	Server requests username
TAC_PLUS_AUTHEN_STATUS_GETPASS	0x05	Server requests password
TAC_PLUS_AUTHEN_STATUS_ERROR	0x07	Server error
TAC_PLUS_AUTHEN_STATUS_RESTART	0x06	Restart authentication with START

Authorization

Authorization uses two packet sub-types:

Packet	Direction	Description

REQUEST	Client -> Server	Request permission for an action (with attribute-value pairs)
RESPONSE	Server -> Client	PASS_ADD, PASS_REPL, FAIL, ERROR (with attribute-value pairs)

Authorization REQUEST Body

[Diagram: see online version]

Accounting

Accounting uses two packet sub-types:

Packet	Direction	Description
REQUEST	Client -> Server	Report event (START, STOP, WATCHDOG with attribute-value pairs)
REPLY	Server -> Client	SUCCESS, ERROR (acknowledgement)

Accounting Flags

Flag	Value	Description
TAC_PLUS_ACCT_FLAG_START	0x02	Session began
TAC_PLUS_ACCT_FLAG_STOP	0x04	Session ended
TAC_PLUS_ACCT_FLAG_WATCHDOG	0x08	Interim update (session still active)

Encryption

TACACS+ encrypts the entire packet body using a pseudo-random pad derived from the shared secret, session ID, version, and sequence number:

```
pad = MD5(session_id + key + version + seq_no + prev_pad)
encrypted_body = body XOR pad
```

Each packet in a session uses a different pad (sequence number increments), providing per-packet encryption. The header is always sent in the clear to allow demultiplexing.

Authentication Flow

Login (ASCII)

[Diagram: see online version]

Command Authorization

[Diagram: see online version]

Accounting

[Diagram: see online version]

Common Attribute-Value Pairs

Attribute	Example	Description
service	shell	Type of service requested
cmd	show	Command being authorized
cmd-arg	running-config	Command argument
priv-lvl	15	Privilege level (0-15, Cisco)
protocol	ip	Protocol context
acl	10	Access list to apply
addr	10.0.0.0/8	Network address
timeout	60	Session timeout in seconds
task_id	1	Unique task identifier for accounting

TACACS+ vs RADIUS

Feature	TACACS+	RADIUS
Transport	TCP (port 49)	UDP (ports 1812/1813)
Encryption	Entire body encrypted	Only User-Password encrypted
AAA separation	Separate authen/author/acct	Auth+authz combined in one exchange
Command authorization	Per-command (granular)	Not supported
Protocol control	Multilink, multi-protocol	Limited (attribute-based)
Vendor support	Primarily Cisco, growing multi-vendor	Universal (every NAS)
Primary use	Device administration (CLI access)	Network access (Wi-Fi, VPN, 802.1X)
Accounting	START/STOP/WATCHDOG	Start/Stop/Interim-Update
Standards	RFC 8907 (2020)	RFC 2865 (2000)
Packet size	No practical limit (TCP)	Max 4096 bytes (UDP)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 8907	The TACACS+ Protocol
RFC 9476	TACACS+ TLS 1.3 (obfuscation removal)

See Also

- RADIUS -- the other AAA protocol (focused on network access)
- 802.1X -- port-based network access control (uses RADIUS or TACACS+)
- Kerberos -- authentication protocol used in Active Directory
- SSH -- commonly authorized via TACACS+

TLS -- Transport Layer Security

Standard: RFC 8446 | **Layer:** Application / Presentation (Layer 6-7) | **Wireshark filter:** `tls`

TLS provides encryption, authentication, and integrity for communication over a network. It sits between the transport layer (TCP) and the application layer, creating a secure channel for protocols like HTTPS, SMTPS, IMAPS, and others. TLS 1.3 is the current version, significantly simplifying the handshake and removing insecure legacy features from TLS 1.2.

Record Protocol

Every TLS message is wrapped in a record:

[Diagram: see online version]

Key Fields

Field	Size	Description
Content Type	8 bits	Type of the enclosed message
Protocol Version	16 bits	TLS version (see below)
Length	16 bits	Payload length in bytes (max 16384 + 2048)
Fragment	Variable	The message payload (encrypted after handshake)

Field Details

Content Type

Value	Type
20	ChangeCipherSpec (legacy, not used in TLS 1.3)
21	Alert
22	Handshake
23	Application Data
24	Heartbeat (RFC 6520)

Protocol Version

Value	Version
0x0301	TLS 1.0 (deprecated)
0x0302	TLS 1.1 (deprecated)
0x0303	TLS 1.2
0x0303	TLS 1.3 (uses 0x0303 in record, negotiated via extensions)

Note: TLS 1.3 reuses the 0x0303 version in the record header for backward compatibility. The actual version is negotiated in the `supported_versions` extension.

TLS 1.3 Handshake

[Diagram: see online version]

TLS 1.3 completes in a single round trip (1-RTT), or zero round trips (0-RTT) for resumed connections.

TLS 1.2 Handshake (for comparison)

[Diagram: see online version]

TLS 1.2 requires two round trips (2-RTT).

Handshake Message Types

Type	Name	Description
1	ClientHello	Client initiates handshake
2	ServerHello	Server responds with chosen parameters
4	NewSessionTicket	Server provides resumption ticket

8	EncryptedExtensions	Additional server extensions (TLS 1.3)
11	Certificate	Server (or client) certificate chain
13	CertificateRequest	Server requests client certificate
15	CertificateVerify	Signature proving certificate ownership
20	Finished	Handshake integrity verification

Alert Levels and Descriptions

Level	Meaning
1	Warning
2	Fatal (connection terminated)

Common alert descriptions:

Value	Description
0	close_notify (graceful shutdown)
10	unexpected_message
20	bad_record_mac
40	handshake_failure
42	bad_certificate
43	unsupported_certificate
44	certificate_revoked
45	certificate_expired
48	unknown_ca
70	protocol_version
112	unrecognized_name (SNI mismatch)

TLS 1.3 Cipher Suites

Suite	Description
TLS_AES_128_GCM_SHA256	AES-128 in GCM mode
TLS_AES_256_GCM_SHA384	AES-256 in GCM mode
TLS_CHACHA20_POLY1305_SHA256	ChaCha20 stream cipher

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 8446	The Transport Layer Security (TLS) Protocol Version 1.3
RFC 5246	TLS Protocol Version 1.2
RFC 6066	TLS Extensions (including SNI)
RFC 8449	Record Size Limit Extension
RFC 9001	Using TLS to Secure QUIC
RFC 8996	Deprecating TLS 1.0, TLS 1.1, and DTLS 1.0

See Also

- TCP
- HTTP -- primary user of TLS (HTTPS)
- DNS -- DNS over TLS (DoT)
- SSH -- alternative encryption for remote access

WireGuard

Standard: WireGuard Whitepaper | **Layer:** Network / VPN (Layer 3) | **Wireshark filter:** wg

WireGuard is a modern VPN protocol designed for simplicity, speed, and strong cryptography. Its entire codebase is roughly 4,000 lines (vs. hundreds of thousands for OpenVPN or IPsec), making it auditable and fast. WireGuard uses a fixed set of modern cryptographic primitives -- no cipher negotiation, no legacy algorithm support. It is built into the Linux kernel (since 5.6), and available on all major platforms. WireGuard operates at Layer 3, tunneling IP packets inside encrypted UDP datagrams.

Message Types

WireGuard uses only four message types:

Type	Value	Name	Description
1	0x01	Handshake Initiation	Begin key exchange (initiator -> responder)
2	0x02	Handshake Response	Complete key exchange (responder -> initiator)
3	0x03	Cookie Reply	DoS mitigation response
4	0x04	Transport Data	Encrypted tunnel data

Handshake Initiation (Type 1)

[Diagram: see online version]

Handshake Response (Type 2)

[Diagram: see online version]

Transport Data (Type 4)

[Diagram: see online version]

Field	Size	Description
Type	1 byte	Always 4 for transport data
Receiver Index	4 bytes	Identifies the session at the receiver
Counter	8 bytes	Nonce / anti-replay counter (monotonically increasing)
Encrypted Packet	Variable	ChaCha20-Poly1305 AEAD of the inner IP packet

Cryptographic Primitives

WireGuard uses exactly these -- no negotiation, no options:

Function	Algorithm	Purpose
Key Exchange	Curve25519 (ECDH)	Derive shared secrets
Symmetric Encryption	ChaCha20-Poly1305	Encrypt tunnel traffic (AEAD)
Hashing	BLAKE2s	Key derivation, MAC
Key Derivation	HKDF	Derive session keys from DH results
MAC	Keyed BLAKE2s	MAC1 and MAC2 in handshake

Handshake (Noise IK Pattern)

WireGuard uses the **Noise Protocol Framework** (Noise_IKpsk2):

[Diagram: see online version]

The handshake completes in **1 round trip** (1-RTT). Session keys are rotated every 2 minutes or 2^64 messages.

Key Concepts

Concept	Description
Interface	A virtual network interface (e.g., `wg0`) with a private key
Peer	A remote endpoint identified by its public key
Allowed IPs	IP ranges this peer is authorized to send/receive (acts as routing + ACL)
Endpoint	The peer's public IP:port (can change -- roaming supported)
Persistent Keepalive	Optional periodic keepalive to maintain NAT mappings

Configuration Example

```
[Interface]
PrivateKey = yAnz5TF+lXXJte14tji3zlMNq+hd2rYUIgJBgB3fBmk=
Address = 10.0.0.1/24
ListenPort = 51820

[Peer]
PublicKey = xTIBA5rboUvnH4htodjb6e697QjLERt1NAB4mZqp8Dg=
AllowedIPs = 10.0.0.2/32, 192.168.1.0/24
Endpoint = 203.0.113.50:51820
PersistentKeepalive = 25
```

WireGuard vs IPsec vs OpenVPN

Feature	WireGuard	IPsec	OpenVPN
Codebase	~4,000 lines	400,000+ lines	100,000+ lines
Handshake	1-RTT	2-RTT (IKEv2)	2+ RTT (TLS)
Cipher negotiation	None (fixed)	Extensive	Extensive
State	Stateless (silent until data)	IKE SA state machine	TLS session state
Roaming	Built-in (endpoint updates on valid packet)	Requires MOBIKE	Reconnect
Kernel integration	Native Linux (since 5.6)	Native (xfrm)	Userspace (tun)
Performance	Fastest	Fast	Slowest
Protocol	UDP only	ESP (IP protocol 50) or UDP	TCP or UDP

Encapsulation

[Diagram: see online version]

Standards

Document	Title

WireGuard Whitepaper	WireGuard: Next Generation Kernel Network Tunnel
Noise Protocol Framework	Noise_IKpsk2 pattern used by WireGuard

See Also

- IPsec -- traditional VPN (more complex, more features)
- UDP -- WireGuard transport
- GRE -- unencrypted tunneling alternative

WPA2 / WPA3 -- Wi-Fi Protected Access

Standard: IEEE 802.11i (WPA2) / Wi-Fi Alliance WPA3 | **Layer:** Data Link (Layer 2) | **Wireshark filter:** `eapol`

WPA2 and WPA3 are the security protocols that protect Wi-Fi networks from eavesdropping and unauthorized access. WPA2 (2004) uses AES-CCMP encryption with a 4-way handshake to establish per-session keys. WPA3 (2018) replaces the Pre-Shared Key exchange with SAE (Simultaneous Authentication of Equals), providing forward secrecy and resistance to offline dictionary attacks. Both support Personal (password) and Enterprise (802.1X/RADIUS) modes.

Security Evolution

Standard	Year	Encryption	Key Exchange	Status
WEP	1997	RC4 (broken)	Static key	**Broken -- never use**
WPA	2003	TKIP (RC4, improved)	4-way handshake (PSK)	**Deprecated**
WPA2	2004	AES-128-CCMP	4-way handshake (PSK)	Current
WPA3	2018	AES-128-CCMP or AES-256-GCMP	SAE (Dragonfly)	Current (recommended)

Modes

Mode	Authentication	Key Source	Use Case
Personal (PSK/SAE)	Shared password	Derived from passphrase + SSID	Home, small office
Enterprise (802.1X)	Per-user credentials via RADIUS	Per-session from EAP exchange	Corporate, campus

WPA2 -- 4-Way Handshake

The 4-way handshake derives per-session encryption keys from the PMK (Pairwise Master Key) without transmitting the PMK itself:

Key Hierarchy

[Diagram: see online version]

PSK Derivation

```
PMK = PBKDF2(SHA1, passphrase, SSID, 4096 iterations, 256 bits)
```

4-Way Handshake

[Diagram: see online version]

PTK Derivation

```
PTK = PRF-384(PMK, "Pairwise key expansion",
              min(AA,SA) || max(AA,SA) || min(ANonce,SNonce) || max(ANonce,SNonce))
```

Where AA = Authenticator Address (AP MAC), SA = Supplicant Address (client MAC).

PTK Component	Size	Purpose
KCK	128 bits	MIC calculation for EAPOL frames
KEK	128 bits	Encrypt GTK in message 3
TK	128 bits	Encrypt unicast data (AES-CCMP)

EAPOL-Key Frame

[Diagram: see online version]

WPA3 -- SAE (Simultaneous Authentication of Equals)

SAE replaces the PSK 4-way handshake with a zero-knowledge proof based on the Dragonfly key exchange:

SAE Handshake

[Diagram: see online version]

WPA3 Improvements

Feature	WPA2	WPA3
Key exchange	PSK (PBKDF2 -> 4-way handshake)	SAE (Dragonfly -> 4-way handshake)
Offline attack	Vulnerable (captured handshake can be brute-forced)	Resistant (zero-knowledge proof)
Forward secrecy	No (same PMK from same password)	Yes (fresh PMK per session)
Open networks	Unencrypted	OWE (Opportunistic Wireless Encryption)
Enterprise (192-bit)	Optional	Mandatory 192-bit mode (CNSA suite)
PMF (Protected Management Frames)	Optional	Mandatory

WPA3 OWE (Opportunistic Wireless Encryption)

OWE encrypts open networks (coffee shops, airports) without a password using a Diffie-Hellman exchange:

[Diagram: see online version]

Encryption Algorithms

Protocol	Cipher	Key Size	Integrity
WPA (TKIP)	RC4	128 bits	Michael MIC (64 bits)
WPA2 (CCMP)	AES-CTR	128 bits	AES-CBC-MAC (64 bits)
WPA3 (CCMP-256)	AES-CTR	256 bits	AES-CBC-MAC (128 bits)
WPA3 (GCMP-256)	AES-GCM	256 bits	GHASH (128 bits)

CCMP Encryption

[Diagram: see online version]

The Packet Number (PN) is a 48-bit counter -- it must never repeat for the same key, providing replay protection.

Protected Management Frames (PMF -- 802.11w)

PMF prevents deauthentication attacks by encrypting/signing management frames:

Frame	Without PMF	With PMF
Deauthentication	Spoofable (trivial DoS)	Authenticated (MIC-protected)
Disassociation	Spoofable	Authenticated
Action frames	Unprotected	Encrypted

PMF is optional in WPA2 and **mandatory in WPA3**.

Key Caching and Roaming

Mechanism	Standard	Description
PMK Caching (PMKSA)	802.11i	Cache PMK for faster reconnection
Pre-authentication	802.11i	Authenticate with next AP before roaming
802.11r (FT)	802.11r	Fast BSS Transition -- roam in <50ms
OKC	Vendor	Opportunistic Key Caching across APs

802.11r Fast Roaming

[Diagram: see online version]

Encapsulation

[Diagram: see online version]

Standards

Document	Title
IEEE 802.11i-2004	MAC Security Enhancements (WPA2 basis)
IEEE 802.11-2020	Consolidated WLAN standard (includes 802.11i, 802.11w, SAE)
IEEE 802.11w-2009	Protected Management Frames
IEEE 802.11r-2008	Fast BSS Transition
RFC 7664	Dragonfly Key Exchange (SAE basis)
Wi-Fi Alliance WPA3	WPA3 certification program

See Also

- 802.11 (Wi-Fi) -- base Wi-Fi framing and PHY
- 802.1X / EAP -- enterprise authentication framework
- RADIUS -- authentication server for WPA Enterprise
- Kerberos -- sometimes used via EAP-FAST/EAP-TLS

X.509 Certificates

Standard: RFC 5280 / ITU-T X.509 | **Layer:** Application (Layer 7) | **Wireshark filter:** `x509ce` or `x509af` or `tls.handshake.certificate`

X.509 is the standard format for public key certificates used across TLS/HTTPS, code signing, email (S/MIME), VPNs, and virtually every system that relies on PKI (Public Key Infrastructure). An X.509 v3 certificate binds a public key to an identity (domain name, organization, email) and is signed by a Certificate Authority (CA). The certificate chain model -- end entity, intermediate CA, root CA -- is the foundation of trust on the internet. X.509 certificates are ASN.1 structures encoded in DER (binary) or PEM (base64 with headers).

Certificate Structure (X.509 v3)

[Diagram: see online version]

The outer certificate is a SEQUENCE of three fields: the TBSCertificate (the data that is signed), the signature algorithm identifier, and the signature itself.

TBSCertificate

[Diagram: see online version]

Key Fields

Field	Description
Version	Certificate format version: v1 (0), v2 (1), v3 (2). Virtually all modern certs are v3
Serial Number	Unique integer assigned by the issuing CA (up to 20 octets). Must be unique per CA
Signature Algorithm	Algorithm the CA used to sign this certificate (e.g., sha256WithRSAEncryption, ecdsa-with-SHA256)
Issuer	Distinguished Name of the CA that signed this certificate
Validity	notBefore and notAfter timestamps (UTCTime or GeneralizedTime) defining the certificate's valid period
Subject	Distinguished Name of the entity this certificate identifies
Subject Public Key Info	The public key algorithm and the public key itself (RSA modulus+exponent, EC point, etc.)
Extensions	v3 extensions that add constraints, key usage, alternative names, revocation info, etc.
Signature Value	The CA's digital signature over the DER-encoded TBSCertificate

Distinguished Name (DN) Components

Attribute	OID	Example	Description

CN	2.5.4.3	www.example.com	Common Name (historically the domain; now SAN is authoritative)
O	2.5.4.10	Example Inc.	Organization
OU	2.5.4.11	Engineering	Organizational Unit (deprecated for TLS certs)
C	2.5.4.6	US	Country (2-letter ISO 3166)
ST	2.5.4.8	California	State or Province
L	2.5.4.7	San Francisco	Locality (city)

Example DN: `C=US, ST=California, L=San Francisco, O=Example Inc., CN=www.example.com`

Key Extensions (v3)

Extension	OID	Critical	Description
Subject Alternative Name (SAN)	2.5.29.17	Yes	The authoritative list of identities: DNS names, IPs, emails, URIs
Key Usage	2.5.29.15	Yes	Permitted operations: digitalSignature, keyEncipherment, keyCertSign, etc.
Extended Key Usage (EKU)	2.5.29.37	Varies	Purpose: serverAuth (TLS server), clientAuth, codeSigning, emailProtection
Basic Constraints	2.5.29.19	Yes	Whether this is a CA cert and the maximum chain depth (pathLenConstraint)
Authority Key Identifier (AKI)	2.5.29.35	No	Identifies the signing CA's key (for chain building)
Subject Key Identifier (SKI)	2.5.29.14	No	Hash of this certificate's public key (referenced by child certs' AKI)
CRL Distribution Points	2.5.29.31	No	URLs where the CA publishes Certificate Revocation Lists
Authority Information Access (AIA)	1.3.6.1.5.5.7.1.1	No	OCSP responder URL and CA Issuers URL (for chain discovery)
Certificate Policies	2.5.29.32	Varies	OIDs indicating validation level (DV, OV, EV)
Name Constraints	2.5.29.30	Yes	Restricts which names a sub-CA can issue for (permitted/excluded subtrees)

SCT List (CT)	1.3.6.1.4.1.11129.2.4.2	No	Signed Certificate Timestamps from Certificate Transparency logs

Key Usage Bits

Bit	Usage	Typical Certificate
0	digitalSignature	TLS server/client, code signing
1	nonRepudiation (contentCommitment)	Document signing
2	keyEncipherment	RSA TLS server (key transport)
3	dataEncipherment	Rare
4	keyAgreement	ECDH TLS server
5	keyCertSign	CA certificates only
6	cRLSign	CA certificates only

Extended Key Usage

OID	Name	Description
1.3.6.1.5.5.7.3.1	serverAuth	TLS/HTTPS server certificate
1.3.6.1.5.5.7.3.2	clientAuth	TLS client certificate
1.3.6.1.5.5.7.3.3	codeSigning	Code and software signing
1.3.6.1.5.5.7.3.4	emailProtection	S/MIME email signing/encryption
1.3.6.1.5.5.7.3.8	timeStamping	Trusted timestamp signing
1.3.6.1.5.5.7.3.9	OCSPSigning	OCSP response signing

Certificate Chain Validation

[Diagram: see online version]

Chain Structure

[Diagram: see online version]

Certificate Lifecycle

CSR (Certificate Signing Request -- PKCS#10)

[Diagram: see online version]

ACME Protocol (Let's Encrypt)

[Diagram: see online version]

Revocation

CRL (Certificate Revocation List)

Field	Description
Issuer	The CA that issued the CRL
This Update	When this CRL was published
Next Update	When the next CRL will be published
Revoked Certificates	List of serial numbers, revocation dates, and reason codes
Signature	CA's signature over the CRL

Reason codes: unspecified (0), keyCompromise (1), cACompromise (2), affiliationChanged (3), superseded (4), cessationOfOperation (5), certificateHold (6).

OCSP (Online Certificate Status Protocol)

[Diagram: see online version]

OCSP Stapling

Approach	Description
Standard OCSP	Client contacts OCSP responder directly (privacy concern, latency)
OCSP Stapling (TLS extension)	Server fetches OCSP response periodically and includes it in the TLS handshake
OCSP Must-Staple (extension)	Certificate includes the must-staple extension -- client rejects if no stapled response

OCSP stapling is preferred: it avoids the client revealing which sites it visits to the OCSP responder, and eliminates the extra round trip.

Encoding Formats

Format	Extension	Encoding	Contains	Description
DER	.der, .cer	Binary (ASN.1 DER)	Single certificate	Raw binary encoding; used by Java, Windows

PEM	.pem, .crt	Base64 + headers	One or more certs/keys	`-----BEGIN CERTIFICATE-----` ... `-----END CERTIFICATE-----`
PKCS#7	.p7b, .p7c	DER or PEM	Certificate chain (no private key)	Used for chain distribution
PKCS#12 / PFX	.p12, .pfx	Binary (encrypted)	Cert + private key + chain	Password-protected bundle; used for import/export
PKCS#8	.key	DER or PEM	Private key only	`-----BEGIN PRIVATE KEY-----` (or ENCRYPTED PRIVATE KEY)

PEM Example

```
-----BEGIN CERTIFICATE-----
MIIFazCCA1OgAwIBAgIRAIIQz7DSQONZRGPgu2OCiwAwDQYJKoZIhvcNAQEL
BQAwTzELMAkGA1UEBhMCVVMxKTAnBgNVBAoTIEludGVybmV0IFNlY3VyaXR5
... (base64 encoded DER) ...
-----END CERTIFICATE-----
```

Key Types and Algorithms

Algorithm	Key Size	Signature Size	Performance	Adoption
RSA	2048 bits	256 bytes	Slow sign, fast verify	Universal (legacy + modern)
RSA	4096 bits	512 bytes	Very slow sign	High-security CAs
ECDSA P-256	256 bits	~72 bytes	Fast	Widely supported
ECDSA P-384	384 bits	~104 bytes	Fast	High-security applications
Ed25519	256 bits	64 bytes	Fastest	Growing (not yet in all TLS stacks)

Signature Algorithms

OID	Name	Description
1.2.840.113549.1.1.11	sha256WithRSAEncryption	RSA signature with SHA-256 (most common)
1.2.840.113549.1.1.12	sha384WithRSAEncryption	RSA signature with SHA-384
1.2.840.113549.1.1.13	sha512WithRSAEncryption	RSA signature with SHA-512
1.2.840.10045.4.3.2	ecdsa-with-SHA256	ECDSA signature with SHA-256
1.2.840.10045.4.3.3	ecdsa-with-SHA384	ECDSA signature with SHA-384

1.3.101.112	Ed25519	EdDSA signature (no separate hash)

Certificate Transparency (CT)

CT requires CAs to submit all issued certificates to public, append-only logs. Browsers (Chrome, Safari, Apple platforms) require valid SCTs (Signed Certificate Timestamps) for trust.

Component	Role
CT Log	Append-only Merkle tree of all issued certificates (operated by Google, Cloudflare, etc.)
SCT	Signed Certificate Timestamp -- proof the cert was submitted to a log
Monitor	Watches logs for unauthorized certificates for a domain
Auditor	Verifies log integrity (inclusion and consistency proofs)

SCT Delivery Methods

Method	Description
Embedded in certificate	CA requests SCTs before issuance, embeds them in the SCT List extension
TLS extension	Server sends SCTs in the TLS handshake (signed_certificate_timestamp extension)
OCSP Stapling	SCTs included in the stapled OCSP response

Certificate Validation Levels

Level	Abbreviation	Validation	Visual Indicator
Domain Validation	DV	CA verifies domain control only (HTTP/DNS challenge)	Padlock only
Organization Validation	OV	CA verifies organization identity (business records)	Padlock + org name in cert
Extended Validation	EV	Strict identity verification (legal entity, physical address)	Padlock (formerly green bar, deprecated in browsers)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 5280	X.509 v3 Certificate and CRL Profile (PKIX)
RFC 6960	OCSP (Online Certificate Status Protocol)
RFC 8555	ACME (Automatic Certificate Management Environment)
RFC 6962	Certificate Transparency
RFC 2986	PKCS#10 -- Certificate Signing Request (CSR)
RFC 7468	PEM Encoding for PKI structures
RFC 7633	TLS Features Extension (OCSP Must-Staple)
ITU-T X.509	Original X.509 specification (ITU-T)
ITU-T X.690	ASN.1 DER/BER encoding rules

See Also

- WireGuard -- VPN using X25519 keys (no X.509 PKI)
- DANE -- DNS-based certificate pinning (alternative to CA trust)
- SMTP -- STARTTLS uses X.509 certificates
- MQTT -- IoT protocol supporting mutual TLS with X.509

Serial Protocols

Serial communication protocols for point-to-point and multi-drop links.

6 protocols in this section

RS-232 -- EIA/TIA-232

Standard: EIA/TIA-232-F | **Layer:** Physical (Layer 1) | **Wireshark filter:** N/A (sub-packet-capture)

RS-232 is the classic serial port standard, defining the electrical signaling, voltage levels, pin assignments, and mechanical connectors for point-to-point serial communication. It was originally designed in 1960 for connecting terminals to modems and became the universal PC serial port (COM port). RS-232 uses single-ended signaling with voltage swings of ±3V to ±15V. While largely replaced by USB for PC peripherals, RS-232 remains widespread in industrial equipment, networking gear (console ports), scientific instruments, and embedded systems.

Electrical Characteristics

Parameter	Specification
Signal type	Single-ended (unbalanced)
Logic 1 (Mark)	-3V to -15V
Logic 0 (Space)	+3V to +15V
Invalid region	-3V to +3V
Max cable length	~15 m (50 ft) at standard rates
Max data rate	20 kbps (original spec); practical up to 115.2 kbps
Topology	Point-to-point (DTE to DCE)

Note: The logic sense is inverted from TTL -- a logic 1 is a negative voltage.

Connector Pinout

DE-9 (commonly called DB-9)

The 9-pin DE-9 connector is the most common RS-232 connector (replaced the original 25-pin DB-25):

Pin	Name	Direction (DTE)	Description
1	DCD	Input	Data Carrier Detect
2	RxD	Input	Receive Data
3	TxD	Output	Transmit Data
4	DTR	Output	Data Terminal Ready
5	GND	--	Signal Ground
6	DSR	Input	Data Set Ready
7	RTS	Output	Request to Send
8	CTS	Input	Clear to Send

9	RI	Input	Ring Indicator

DB-25

Pin	Name	Direction (DTE)	Description
1	Shield	--	Protective ground
2	TxD	Output	Transmit Data
3	RxD	Input	Receive Data
4	RTS	Output	Request to Send
5	CTS	Input	Clear to Send
6	DSR	Input	Data Set Ready
7	GND	--	Signal Ground
8	DCD	Input	Data Carrier Detect
20	DTR	Output	Data Terminal Ready
22	RI	Input	Ring Indicator

Signal Lines

Data Lines

Signal	Description
TxD (Transmit Data)	Data from DTE to DCE
RxD (Receive Data)	Data from DCE to DTE

Handshake Lines

Signal	Description
RTS (Request to Send)	DTE is ready to send data
CTS (Clear to Send)	DCE is ready to receive data
DTR (Data Terminal Ready)	DTE is powered on and ready
DSR (Data Set Ready)	DCE is powered on and ready
DCD (Data Carrier Detect)	DCE has established a connection (carrier detected)
RI (Ring Indicator)	DCE signals incoming call (modem)

Hardware Flow Control (RTS/CTS)

[Diagram: see online version]

DTE vs DCE

Term	Full Name	Examples	TxD Pin
DTE	Data Terminal Equipment	Computer, terminal, router	Output
DCE	Data Communications Equipment	Modem, CSU/DSU	Input

Null Modem (DTE-to-DTE)

When connecting two DTEs directly (e.g., two computers), a null modem cable crosses the data and handshake lines:

DTE 1		DTE 2
TxD (3)	->	RxD (2)
RxD (2)	<-	TxD (3)
RTS (7)	->	CTS (8)
CTS (8)	<-	RTS (7)
DTR (4)	->	DSR (6) + DCD (1)
DSR (6) + DCD (1)	<-	DTR (4)
GND (5)	--	GND (5)

Data Framing

RS-232 uses UART framing:

[Diagram: see online version]

Common configuration: 9600 baud, 8N1 (8 data bits, no parity, 1 stop bit).

Standards

Document	Title
EIA/TIA-232-F	Interface Between DTE and DCE Employing Serial Binary Data Interchange
ITU-T V.24	List of definitions for interchange circuits (signal definitions)
ITU-T V.28	Electrical characteristics for unbalanced interchange circuits
ISO 2110	25-pin connector assignment

See Also

- UART -- the framing protocol RS-232 carries

- RS-485 -- differential multi-drop alternative
- RS-422 -- differential point-to-point alternative

RS-422 -- EIA/TIA-422

Standard: EIA/TIA-422-B | **Layer:** Physical (Layer 1) | **Wireshark filter:** N/A (sub-packet-capture)

RS-422 is a differential serial standard for point-to-point (or point-to-multipoint) communication. It uses balanced signaling like RS-485 for noise immunity and long-distance capability, but is designed around a single driver with up to 10 receivers -- making it inherently full-duplex with separate transmit and receive pairs. RS-422 is commonly found in industrial control, Apple Macintosh serial ports (pre-USB), and as the physical layer for protocols requiring full-duplex differential signaling.

Electrical Characteristics

Parameter	Specification
Signal type	Differential (balanced)
Signal lines	4 (TxD+/TxD–, RxD+/RxD–) + GND
Logic 1 (Mark)	A < B, differential > +200 mV
Logic 0 (Space)	A > B, differential < -200 mV
Driver output voltage	±2V to ±5V differential
Receiver sensitivity	±200 mV minimum
Max cable length	1200 m (4000 ft)
Max data rate	10 Mbps (at short distances)
Max receivers	10 per driver (1 driver only)
Topology	Point-to-point or point-to-multipoint
Duplex	Full (separate TX and RX pairs)

Wiring

Point-to-Point (most common)

[Diagram: see online version]

Wire	Name	Description
TxD+ (B)	Transmit Data +	Non-inverting transmit
TxD– (A)	Transmit Data –	Inverting transmit

RxD+ (B)	Receive Data +	Non-inverting receive
RxD– (A)	Receive Data –	Inverting receive
GND	Signal Ground	Common reference

Point-to-Multipoint

One driver can feed up to 10 receivers on a shared receive bus. Only one device transmits on each pair:

```
[Driver] ---+--- [Receiver 1]
            +--- [Receiver 2]
            +--- [Receiver N (max 10)]
```

Termination

A 100-120Ω termination resistor at the far end of each differential pair:

Pair	Termination
TxD+/TxD– (at receiver end)	100-120Ω
RxD+/RxD– (at receiver end)	100-120Ω

Data Framing

RS-422 defines only the electrical layer. Data framing is typically UART:

[Diagram: see online version]

RS-422 vs RS-485

Feature	RS-422	RS-485
Drivers per bus	1	32 (or 256 high-Z)
Receivers per bus	10	32 (or 256 high-Z)
Duplex	Full (4-wire native)	Half (2-wire) or Full (4-wire)
Direction control	Not needed (dedicated TX/RX)	DE pin required in half-duplex
Multi-master	No	Yes
Electrical compatibility	RS-485 receivers can receive RS-422	RS-422 receivers may not tolerate RS-485 bus contention

RS-485 is a superset in many practical ways -- an RS-485 transceiver can communicate with RS-422 devices, but not always the reverse.

Standards

Document	Title
EIA/TIA-422-B	Electrical Characteristics of Balanced Voltage Digital Interface Circuits
ITU-T V.11	Electrical Characteristics for Balanced Double-Current Interchange Circuits
ISO 8482	Twisted pair multipoint interconnections

See Also

- UART -- framing protocol carried over RS-422
- RS-232 -- single-ended point-to-point alternative
- RS-485 -- multi-drop extension of RS-422

RS-485 -- EIA/TIA-485

Standard: EIA/TIA-485-A | **Layer:** Physical (Layer 1) | **Wireshark filter:** N/A (sub-packet-capture)

RS-485 is a differential serial communication standard designed for multi-drop networks over long distances in noisy environments. It uses a balanced (differential) signal pair, providing far greater noise immunity and cable lengths than RS-232. RS-485 supports up to 32 unit loads on a single bus (256 with high-impedance receivers), distances up to 1200 m, and data rates up to 10 Mbps. It is the dominant physical layer in industrial protocols like Modbus RTU, PROFIBUS, and BACnet MS/TP.

Electrical Characteristics

Parameter	Specification
Signal type	Differential (balanced)
Signal lines	2 (A/B or D+/D–) + GND recommended
Logic 1 (Mark)	V(A) < V(B), differential > +200 mV
Logic 0 (Space)	V(A) > V(B), differential < -200 mV
Driver output voltage	±1.5V to ±5V differential
Receiver sensitivity	±200 mV minimum
Max cable length	1200 m (4000 ft) at lower baud rates
Max data rate	10 Mbps (at short distances)
Max devices	32 unit loads (256 with 1/8 unit load receivers)
Topology	Multi-drop bus

Distance vs Data Rate

Data Rate	Approximate Max Distance
100 kbps	1200 m
500 kbps	300 m
1 Mbps	120 m
10 Mbps	12 m

Bus Topology

Half-Duplex (2-wire)

The most common RS-485 configuration -- all devices share a single differential pair and take turns transmitting:

[Diagram: see online version]

Wire	Name	Description
A (D–)	Non-inverting / TxD– / RxD–	Negative when transmitting logic 0
B (D+)	Inverting / TxD+ / RxD+	Positive when transmitting logic 0
GND	Signal Ground	Recommended to prevent common-mode issues

Note: The A/B naming convention is inconsistent across manufacturers. Some label A as non-inverting (TIA standard), others as inverting. Always verify with the datasheet.

Full-Duplex (4-wire)

Uses two differential pairs -- one for each direction. Electrically identical to RS-422 in a point-to-point configuration, but can also support multi-drop with a single master:

Pair	Direction
TxD+/TxD–	Master transmit / slave receive
RxD+/RxD–	Slave transmit / master receive

Bus Configuration

Termination

A 120Ω termination resistor is placed at each end of the bus to match the characteristic impedance of twisted pair cable and prevent signal reflections:

```
[Master]---/\/\/\---[Device 1]---[Device 2]---/\/\/\---[Device N]
         120Ω                                          120Ω
```

Termination is critical at higher baud rates and longer cable runs. It is typically not needed at low speeds (<9600 baud) over short distances.

Biasing

When no device is transmitting, the bus is in an indeterminate state. Bias resistors pull the bus to a known idle (mark) state:

Resistor	Connection	Typical Value
Pull-up	B (D+) to Vcc	390Ω - 750Ω
Pull-down	A (D–) to GND	390Ω - 750Ω

Direction Control

In half-duplex mode, the transceiver's driver must be enabled only when transmitting and disabled at all other times. This is typically controlled by a Driver Enable (DE) pin:

[Diagram: see online version]

Timing the DE signal is critical -- asserting too late clips the start bit; deasserting too early clips the stop bit.

Data Framing

RS-485 defines only the electrical layer. The data framing is typically UART:

[Diagram: see online version]

The higher-level protocol (Modbus, PROFIBUS, DMX512, etc.) defines message structure, addressing, and error checking on top of this.

RS-485 vs RS-232 vs RS-422

Feature	RS-232	RS-422	RS-485
Signal type	Single-ended	Differential	Differential
Topology	Point-to-point	Point-to-point (multi-drop receive)	Multi-drop
Max devices	1 driver, 1 receiver	1 driver, 10 receivers	32 drivers, 32 receivers
Max distance	15 m	1200 m	1200 m
Max data rate	20 kbps	10 Mbps	10 Mbps
Duplex	Full	Full	Half (2-wire) or Full (4-wire)

Common Protocols Using RS-485

Protocol	Application	Typical Rate
Modbus RTU	Industrial automation	9600 - 115200 bps
PROFIBUS DP	Factory automation	9.6 kbps - 12 Mbps
BACnet MS/TP	Building automation	9600 - 115200 bps
DMX512	Stage lighting	250 kbps
DALI (via bus)	Lighting control	1200 bps
M-Bus (via adapter)	Metering	300 - 9600 bps

Standards

Document	Title
EIA/TIA-485-A	Electrical Characteristics of Generators and Receivers for Use in Balanced Digital Multipoint Systems
ISO 8482	Twisted pair multipoint interconnections

See Also

- UART -- framing protocol carried over RS-485
- RS-232 -- single-ended point-to-point alternative
- RS-422 -- differential point-to-point alternative
- CAN -- another differential bus used in automotive/industrial

UART -- Universal Asynchronous Receiver/Transmitter

Standard: No single standard (framing convention) | **Layer:** Data Link / Physical | **Wireshark filter:** N/A (sub-packet-capture)

UART is the most common asynchronous serial communication scheme, used for decades in everything from PC serial ports to embedded systems, GPS modules, and debug consoles. It defines a framing protocol -- how bits are packaged into characters -- rather than electrical signaling (which is defined by standards like RS-232 or RS-485). UART requires no clock line; both sides agree on a baud rate in advance, and the start bit provides synchronization.

Frame

[Diagram: see online version]

The frame above shows the minimum case. A typical configuration is 8N1: 8 data bits, no parity, 1 stop bit (10 bits total per character).

Key Fields

Field	Size	Description
Start Bit	1 bit	Always logic 0; signals the beginning of a frame
Data Bits	5-9 bits	The payload, transmitted LSB first
Parity Bit	0-1 bit	Optional error detection
Stop Bit(s)	1-2 bits	Always logic 1; marks the end of the frame

Field Details

Idle State

The line idles at logic high (mark). A transition to low (space) indicates the start bit.

```
      ___     ___________     ___
Line: |   |___|           |___|   |___
      Idle Start D0 D1 ... Stop Idle Start ...
```

Data Bits

Most modern usage is 8 data bits, but 5 (Baudot), 7 (ASCII), and 9 (address-mark multi-drop) are also used. Data is transmitted **LSB first**.

Parity

Setting	Description
None (N)	No parity bit -- most common
Even (E)	Parity bit set so total 1s (data + parity) is even
Odd (O)	Parity bit set so total 1s is odd
Mark	Parity bit always 1
Space	Parity bit always 0

Common Configurations

Notation: data bits / parity / stop bits

Config	Total Bits	Usage

8N1	10	Most common default
8E1	11	Modbus RTU, industrial
8O1	11	Some legacy systems
8N2	11	Some slower receivers
7E1	10	Legacy ASCII terminals
9N1	11	Multi-drop addressing (RS-485)

Common Baud Rates

Baud Rate	Bit Time	Common Use
300	3.33 ms	Legacy modems
9600	104 µs	Default for many devices, GPS NMEA
19200	52 µs	Industrial equipment
38400	26 µs	Faster serial devices
57600	17.4 µs	Bluetooth SPP default
115200	8.7 µs	Most common high-speed default
230400	4.3 µs	Fast embedded debug
460800	2.2 µs	High-speed embedded
921600	1.1 µs	Maximum for many USB-serial adapters
1000000+	<1 µs	Custom high-speed applications

Error Detection

Error	Cause	Detection
Framing error	Stop bit not detected at expected time	Hardware check
Parity error	Data corrupted in transit	Parity bit mismatch
Overrun error	New byte arrived before previous was read	Hardware FIFO overflow
Break condition	Line held low for longer than one frame	Intentional signal

Flow Control

UART itself has no flow control. It is typically added at a higher layer:

Method	Description
None	No flow control -- risk of overrun at high speeds
Hardware (RTS/CTS)	Dedicated signal lines; receiver deasserts CTS when buffer full
Software (XON/XOFF)	In-band control characters (0x11 = XON, 0x13 = XOFF)

Physical Layer Implementations

UART defines framing only. The electrical interface depends on the standard:

[Diagram: see online version]

Standards

UART framing is not defined by a single standard but is described in the context of various specifications:

Document	Title
EIA/TIA-232	Interface between DTE and DCE (includes UART framing)
EIA/TIA-485	Electrical characteristics of balanced digital multipoint
16550 UART datasheet	De facto UART hardware reference (TI/National Semiconductor)

See Also

- RS-232 -- point-to-point electrical standard using UART framing
- RS-485 -- multi-drop differential electrical standard
- RS-422 -- point-to-point differential electrical standard

XMODEM / YMODEM

Standard: Ward Christensen's XMODEM Protocol, Chuck Forsberg's YMODEM Spec | **Layer:** Application (serial file transfer) | **Wireshark filter:** N/A (serial protocol, not IP)

XMODEM is the first widely used file transfer protocol for microcomputers, created by Ward Christensen in 1977 for CP/M systems. It enabled ordinary users to reliably transfer files over noisy telephone lines using modems and terminal programs. YMODEM (Chuck Forsberg, 1985) extended XMODEM with batch file transfers, filenames, file sizes, and larger blocks. Together they defined the era of BBS file transfers and remain supported by virtually every terminal emulator.

XMODEM Block (Checksum)

[Diagram: see online version]

XMODEM-CRC Block

[Diagram: see online version]

XMODEM-1K Block

[Diagram: see online version]

YMODEM Block 0 (File Header)

[Diagram: see online version]

Block 0 carries metadata: the filename (null-terminated), followed by the file size in ASCII decimal, modification date as an octal Unix timestamp, optional file mode, and optional serial number, all space-separated within the 128-byte data field.

Key Fields

Field	Size	Description
SOH	1 byte	Start of Header (0x01) -- indicates 128-byte data block
STX	1 byte	Start of Text (0x02) -- indicates 1024-byte data block (XMODEM-1K / YMODEM)
Block Number	1 byte	Sequence number, starts at 1 (or 0 for YMODEM header), wraps at 255
Block Complement	1 byte	One's complement of block number (255 - block#) for integrity
Data	128 or 1024 bytes	File data, padded with SUB (0x1A) if final block is short
Checksum	1 byte	Sum of all data bytes mod 256 (original XMODEM only)
CRC-16	2 bytes	CRC-CCITT (polynomial 0x1021), big-endian (XMODEM-CRC and later)

Control Characters

Character	Hex	Name	Purpose
SOH	0x01	Start of Header	Begins 128-byte data block
STX	0x02	Start of Text	Begins 1024-byte data block
EOT	0x04	End of Transmission	Sender signals file complete
ACK	0x06	Acknowledge	Receiver confirms block received OK

NAK	0x15	Negative Acknowledge	Receiver requests retransmission (or initiates checksum mode)
CAN	0x18	Cancel	Abort transfer -- send two or more to confirm
'C'	0x43	CRC request	Receiver sends to request CRC mode instead of checksum
SUB	0x1A	Substitute / Ctrl-Z	Padding byte for short final blocks

XMODEM File Transfer

[Diagram: see online version]

The receiver is the driver: it initiates the transfer by sending NAK (for checksum mode) or 'C' (for CRC mode). The sender does not begin until it receives this initiation character. If a block is corrupted, the receiver NAKs and the sender retransmits. After 10 consecutive retries on the same block, the transfer is aborted.

YMODEM Batch Transfer

[Diagram: see online version]

YMODEM wraps each file with a Block 0 header containing metadata. After the last file, an empty Block 0 (null filename) signals the end of the batch. The receiver confirms each EOT with a NAK/EOT/ACK handshake to guard against line noise.

YMODEM-G (Streaming Variant)

YMODEM-G is a streaming variant designed for error-free links (hardware flow control or error-correcting modems). The receiver sends 'G' instead of 'C' to initiate. The sender then streams blocks without waiting for per-block ACKs. Any error causes the transfer to abort entirely -- there is no per-block retransmission.

Protocol Comparison

Feature	XMODEM	XMODEM-CRC	XMODEM-1K	YMODEM	YMODEM-G
Year	1977	~1982	~1983	1985	1985
Block size	128	128	1024	1024 (default)	1024
Error check	Checksum (8-bit)	CRC-CCITT (16-bit)	CRC-CCITT (16-bit)	CRC-CCITT (16-bit)	CRC-CCITT (16-bit)
Filename sent	No	No	No	Yes (Block 0)	Yes (Block 0)
File size sent	No	No	No	Yes	Yes
Batch transfer	No	No	No	Yes	Yes

Flow	ACK per block	ACK per block	ACK per block	ACK per block	Streaming (no per-block ACK)
Crash recovery	No	No	No	No	No
Initiation	Receiver sends NAK	Receiver sends 'C'	Receiver sends 'C'	Receiver sends 'C'	Receiver sends 'G'

Error Handling

Condition	Behavior
Bad checksum/CRC	Receiver sends NAK, sender retransmits block
Bad block number	Receiver sends NAK
Duplicate block (previous block#)	Receiver sends ACK (already received) and discards
10 consecutive NAKs on same block	Transfer aborted
Timeout (no response)	Retry, then abort after 10 attempts
CAN received (2+ times)	Transfer cancelled by remote end

Historical Significance

XMODEM was the first practical file transfer protocol for personal computers. Ward Christensen wrote it in 8080 assembly for CP/M and placed it in the public domain, allowing it to spread across every BBS and terminal program of the 1980s. Its simplicity -- the entire protocol can be implemented in a few hundred lines of code -- made it the foundation for the YMODEM and ZMODEM protocols that followed. The receiver-driven design was well-suited to the half-duplex modems of the era, and the SUB padding convention (inherited from CP/M's lack of exact file sizes) persisted for years.

Standards

Document	Title
XMODEM Protocol Reference	Ward Christensen's original 1977 specification
XMODEM/CRC Overview	CRC extension documented in Forsberg's YMODEM spec
YMODEM Batch File Transfer Protocol	Chuck Forsberg's YMODEM specification (1985)

See Also

- ZMODEM -- advanced streaming protocol with crash recovery
- UART -- underlying asynchronous serial framing
- RS-232 -- electrical standard for serial communication
- FTP -- TCP/IP-based file transfer protocol
- SFTP / SCP -- encrypted file transfer over SSH

ZMODEM

Standard: Chuck Forsberg's ZMODEM Protocol | **Layer:** Application (serial file transfer) | **Wireshark filter:** N/A (serial protocol, not IP)

ZMODEM is an advanced serial file transfer protocol designed by Chuck Forsberg in 1986. It was the protocol of choice on BBSes and Unix systems throughout the late 1980s and 1990s, largely replacing XMODEM and YMODEM. ZMODEM's key innovations are streaming data transfer (no per-block ACK), crash recovery (resume interrupted transfers at the exact byte offset), auto-start (the `rz`/`sz` commands can trigger automatically), variable-length data subpackets, and 32-bit CRC. These features made it dramatically faster than its predecessors, especially on high-latency links.

Hex Header

[Diagram: see online version]

Hex headers are human-readable (all printable characters) and used for initial negotiation frames. The two ZPAD characters (0x2A, asterisk) allow the receiver to detect the start of a ZMODEM session. ZDLE (0x18) followed by 'B' marks this as a hex-encoded header.

Binary Header (16-bit CRC)

[Diagram: see online version]

Binary Header (32-bit CRC)

[Diagram: see online version]

Binary headers are more compact and used during data transfer. The header type character distinguishes encoding: 'A' = binary with 16-bit CRC, 'B' = hex with 16-bit CRC, 'C' = binary with 32-bit CRC.

Key Fields

Field	Size	Description
ZPAD	1 byte	Padding character (0x2A, asterisk) -- allows receiver to detect ZMODEM
ZDLE	1 byte	Escape character (0x18) -- marks header start and escapes special bytes in data
Header type	1 byte	'A' (binary/CRC16), 'B' (hex/CRC16), or 'C' (binary/CRC32)
Frame type	1 byte	Identifies the frame function (ZRQINIT, ZFILE, ZDATA, etc.)

Flags / Position	4 bytes	Frame-type-dependent: file position offset, capability flags, or parameters
CRC	2 or 4 bytes	CRC-16 (CRC-CCITT) or CRC-32, covering type + flags

ZDLE Encoding

ZDLE (0x18) is the escape character for binary data. When a byte that must be escaped appears in the data stream, it is replaced by ZDLE followed by the byte XORed with 0x40. Bytes that must be escaped:

Byte	Hex	Reason
ZDLE	0x18	Escape character itself
XON	0x11	Software flow control
XOFF	0x13	Software flow control
XON \	0x80	0x91
XOFF \	0x80	0x93
0x10	0x10	DLE -- interferes with some modems
CR following @	0x0D	Telenet escape sequence avoidance

Frame Type Codes

Code	Name	Direction	Description
0	ZRQINIT	Sender	Request receiver initialization
1	ZRINIT	Receiver	Receiver capabilities and initialization
2	ZSINIT	Sender	Define attention string, set parameters
3	ZACK	Both	Acknowledge, position report
4	ZFILE	Sender	File name, attributes, and metadata
5	ZSKIP	Receiver	Skip this file
6	ZNAK	Receiver	Last header was garbled -- retransmit
7	ZABORT	Receiver	Abort batch transfer
8	ZFIN	Sender	Finish session
9	ZRPOS	Receiver	Resume data from this position (crash recovery)
10	ZDATA	Sender	Data follows at given position
11	ZEOF	Sender	End of file at given position

12	ZFERR	Receiver	Fatal file I/O error
13	ZCRC	Receiver	Request file CRC (for comparison)
14	ZCHALLENGE	Receiver	Security challenge
15	ZCOMPL	Sender	Request is complete
16	ZCAN	Both	Cancel -- 5 CAN chars (0x18)
17	ZFREECNT	Sender	Request free disk space
18	ZCOMMAND	Sender	Execute command on remote system

Data Subpackets

After a ZDATA header, the sender transmits variable-length data subpackets. Each subpacket ends with a ZDLE-encoded terminator byte followed by CRC:

Terminator	Name	Meaning
ZCRCE	CRC next, end	End of data, no response expected
ZCRCG	CRC next, go	More data follows, no response expected
ZCRCQ	CRC next, query	More data follows, ACK requested (window sync)
ZCRCW	CRC next, wait	End of data, ACK required before continuing

The sender can stream many ZCRCG subpackets without pausing. ZCRCQ is inserted periodically to implement sliding window flow control. ZCRCW forces a full stop-and-wait synchronization.

ZMODEM File Transfer

[Diagram: see online version]

The sender streams data subpackets continuously, only pausing at ZCRCQ/ZCRCW boundaries for receiver acknowledgment. This eliminates the round-trip delay that slowed XMODEM. The receiver interrupts with ZRPOS only when it detects an error.

Crash Recovery (Resume)

[Diagram: see online version]

When a transfer is interrupted and restarted, the receiver checks what it already has on disk. On receiving ZFILE, if a partial file exists with a matching name and size, the receiver responds with ZRPOS at the last known good offset instead of position 0. The sender seeks to that offset and resumes transmission.

Window Management

ZMODEM supports a sliding window for flow control without stopping the data stream. The sender tracks how many unacknowledged bytes are outstanding. When the window fills, it inserts a ZCRCQ subpacket and waits for ZACK before continuing. The receiver's ZRINIT flags advertise its buffer size.

Window Strategy	Description
Full streaming	No window limit, sender blasts at full speed (reliable links)
Windowed	Sender limits unacknowledged bytes, uses ZCRCQ for sync
Segmented	ZCRCW at each segment boundary (unreliable links)

Protocol Comparison

Feature	XMODEM	YMODEM	ZMODEM	Kermit
Year	1977	1985	1986	1981
Block size	128 fixed	1024 fixed	Variable (up to ~8K)	Variable
Error check	Checksum / CRC-16	CRC-16	CRC-16 or CRC-32	1-3 byte check
Streaming	No (stop-and-wait)	No (stop-and-wait)	Yes	Optional (sliding windows)
Crash recovery	No	No	Yes (ZRPOS)	Yes (in extended Kermit)
Batch transfer	No	Yes	Yes	Yes
Auto-start	No	No	Yes (rz/sz)	No
Filename transfer	No	Yes	Yes	Yes
File size known	No	Yes	Yes	Optional
7-bit safe	No	No	Configurable (ZDLE encoding)	Yes (designed for it)

Auto-Start

ZMODEM's auto-start feature detects the beginning of a transfer without user intervention. When the sender starts `sz`, it emits a ZRQINIT frame preceded by `rz\r` as a text string. Terminal emulators that recognize this string automatically launch the receiver (`rz`). This made ZMODEM significantly easier to use than XMODEM/YMODEM, which required the user to manually invoke the receiver.

Standards

Document	Title
ZMODEM Protocol	Chuck Forsberg's ZMODEM specification (1986)
ZMODEM Technical Reference	Detailed technical description (Stratus)

ZMODEM Application Note	Forsberg's YMODEM/ZMODEM combined document

See Also

- XMODEM / YMODEM -- predecessor protocols
- UART -- underlying asynchronous serial framing
- RS-232 -- electrical standard for serial communication
- FTP -- TCP/IP-based file transfer protocol
- SFTP / SCP -- encrypted file transfer over SSH

Space & Satellite

Protocols designed for space communication and satellite networks.

4 protocols in this section

CCSDS -- Consultative Committee for Space Data Systems

Standard: CCSDS 133.0-B (Space Packet Protocol) | **Layer:** Full stack (Physical through Application) | **Wireshark filter:** `ccsds`

CCSDS defines the international standard protocols for spacecraft communication, used by virtually every space agency: NASA, ESA, JAXA, ROSCOSMOS, ISRO, and CNSA. The protocol stack covers space packet encapsulation, telemetry (TM) and telecommand (TC) data link framing, file delivery (CFDP), and delay-tolerant networking (DTN). CCSDS protocols carry data between spacecraft (or instruments) and ground stations, then onward to mission control. Missions from the ISS and Mars rovers to the James Webb Space Telescope and Voyager probes rely on CCSDS framing.

Protocol Stack

[Diagram: see online version]

Space Packet Primary Header (6 bytes)

[Diagram: see online version]

The Space Packet is the fundamental data unit. The primary header is always 6 bytes (48 bits), followed by an optional secondary header and user data field.

Key Fields (Space Packet)

Field	Size	Description
Version	3 bits	Packet version number (always 000 for current CCSDS)
Type	1 bit	0 = telemetry (TM), 1 = telecommand (TC)
Secondary Header Flag	1 bit	1 = secondary header present
APID	11 bits	Application Process Identifier (0-2047) -- routes packet to instrument/subsystem
Sequence Flags	2 bits	00 = continuation, 01 = first segment, 10 = last segment, 11 = unsegmented
Sequence Count	14 bits	Packet sequence counter (0-16383), wraps around
Data Length	16 bits	Number of bytes in data field minus 1 (0 = 1 byte, 65535 = 65536 bytes)

APID Allocation

APID Range	Usage
0	Idle packet
1-63	Typically reserved or system-level
64-2031	Mission-assigned (instruments, subsystems)
2032-2046	Reserved
2047	Idle packet (all 1s fill)

Sequence Flags

Value	Meaning
00	Continuation segment of a multi-packet group
01	First segment of a multi-packet group
10	Last segment of a multi-packet group
11	Unsegmented (complete packet, most common)

TM (Telemetry) Transfer Frame

The TM frame carries space packets from spacecraft to ground. Maximum frame length is 2048 bytes.

[Diagram: see online version]

TM Transfer Frame Fields

Field	Size	Description
ASM	32 bits	Attached Sync Marker (0x1ACFFC1D) -- frame synchronization
Version	2 bits	Transfer frame version (00 for TM)
Spacecraft ID (SCID)	10 bits	Identifies the spacecraft (0-1023)
Virtual Channel ID (VCID)	3 bits	Multiplexes up to 8 virtual channels per spacecraft
Master Channel Frame Counter	8 bits	Counts all frames from spacecraft
Virtual Channel Frame Counter	8 bits	Counts frames per virtual channel
TF Secondary Header Flag	1 bit	1 = secondary header present
Synchronization Flag	1 bit	0 = octet-synchronized packets, 1 = other
Packet Order Flag	1 bit	0 = forward order

Segment Length ID	2 bits	Segment header size (11 = no segmentation)
First Header Pointer	11 bits	Offset to first packet header in data field
Data Field	Variable	Contains space packets (or idle fill)
CLCW	32 bits	Command Link Control Word -- operational control for TC acknowledgment
FECF	16 bits	Frame Error Control Field (CRC-16-CCITT, optional)

TC (Telecommand) Transfer Frame

Shorter frames for commands sent from ground to spacecraft. Uses COP-1 (Command Operation Procedure) for reliable delivery with sequence control and retransmission.

Field	Size	Description
Version	2 bits	Transfer frame version (00 for TC)
Bypass Flag	1 bit	0 = sequence controlled (Type-A), 1 = expedited (Type-B)
Control Command Flag	1 bit	0 = data, 1 = control command
SCID	10 bits	Spacecraft ID
VCID	6 bits	Virtual channel (up to 64 channels)
Frame Length	10 bits	Frame length minus 1
Frame Sequence Number	8 bits	For COP-1 sequence control
Data Field	Variable	Contains space packets or directives
FECF	16 bits	CRC-16 (mandatory for TC)

AOS (Advanced Orbiting Systems) Transfer Frame

For high-data-rate missions (Earth observation, ISS). Supports larger frames, insert zones for time-critical data, and Multiplexing Protocol Data Units (MPDUs).

Feature	TM Frame	AOS Frame
Max frame size	2048 bytes	Configurable (up to 65535)
Virtual channels	8 (3 bits)	64 (6 bits)
Insert zone	No	Yes (fixed-length, for real-time data)
Bitstream service	No	Yes
Frame counter	8 bits	24 bits

Telemetry Flow

[Diagram: see online version]

CCSDS File Delivery Protocol (CFDP)

CFDP provides reliable file transfer in space environments with long delays and intermittent links.

Feature	Description
Modes	Unreliable (Class 1) and Reliable (Class 2, with NAK retransmission)
Segmentation	Files split into PDUs with sequence numbers
Store-and-forward	Intermediate nodes can store and retransmit
Checksum	32-bit modular checksum for file integrity
Metadata	File name, size, and optional user messages
Use cases	Mars rover data relay, ISS experiment files

Delay-Tolerant Networking (DTN)

The Bundle Protocol (BP) provides interplanetary networking:

Feature	Description
Bundle	Self-contained message with source/destination, expiration, priority
Store-and-forward	Nodes hold bundles until a link becomes available
Custody transfer	Receiving node takes responsibility for delivery
Convergence layers	Runs over TCP, LTP (Licklint Transmission Protocol), or CCSDS links
Demonstrated	ISS (operational since 2018), Mars relay (experimental)

Link Characteristics

Mission Type	Band	Downlink Rate	Round-Trip Light Time
LEO (ISS, Earth Obs)	S/X	150-800 Mbps	~5 ms
Lunar (Artemis)	S/Ka	20-100 Mbps	2.5 sec
Mars (Perseverance)	X/UHF relay	0.5-2 Mbps (relay)	4-24 min
Deep space (Voyager)	S/X	160 bps	20+ hours
L2 (JWST)	Ka	28 Mbps	10 sec

Notable Missions Using CCSDS

Mission	Agency	CCSDS Protocols
ISS	NASA/ESA/JAXA/ROSCOSMOS	TM/TC, AOS, CFDP, DTN
Mars Curiosity/Perseverance	NASA	Space Packet, Proximity-1 relay, CFDP
James Webb Space Telescope	NASA/ESA/CSA	TM/TC, Space Packet (Ka-band)
Voyager 1 & 2	NASA	TM (legacy, pre-standard)
Artemis / Gateway	NASA	TM/TC, DTN, AOS
Rosetta / Philae	ESA	TM/TC, Proximity-1
Chandrayaan / Mangalyaan	ISRO	TM/TC, Space Packet

Standards

Document	Title
CCSDS 133.0-B-2	Space Packet Protocol
CCSDS 132.0-B-3	TM Space Data Link Protocol
CCSDS 232.0-B-4	TC Space Data Link Protocol
CCSDS 732.0-B-4	CCSDS File Delivery Protocol (CFDP)
CCSDS 131.0-B-4	TM Synchronization and Channel Coding
CCSDS 734.2-B-1	CCSDS Bundle Protocol Specification
CCSDS 211.0-B-2	Proximity-1 Space Link Protocol
CCSDS 735.1-B-1	AOS Space Data Link Protocol

See Also

- GPS -- satellite navigation using broadcast messages (not CCSDS-framed)
- ADS-B -- aircraft position broadcast
- LRIT/HRIT -- weather satellite broadcast built on CCSDS framing
- DVB-S2 -- digital video broadcast via satellite

GPS Navigation Message -- L1 C/A

Standard: IS-GPS-200 (GPS Interface Control Document) | **Layer:** Physical / Data Link (radio broadcast) | **Wireshark filter:** N/A (radio signal, decoded by GPS receivers)

GPS (Global Positioning System) is a satellite-based radionavigation system operated by the U.S. Space Force. The L1 C/A (Coarse/Acquisition) signal at 1575.42 MHz is the primary civilian signal, broadcasting a 50 bps navigation message modulated onto a 1.023 Mcps Gold code using BPSK. A receiver decodes messages from 4+ satellites to compute position via pseudorange trilateration. The navigation message provides satellite clock corrections, orbital ephemeris, ionospheric models, and almanac data for the entire constellation.

Frame Structure

The navigation message is organized into a 25-frame **superframe** (12.5 minutes total). Each frame contains 5 **subframes** of 300 bits each (6 seconds per subframe, 30 seconds per frame). Each subframe contains 10 **words** of 30 bits (24 data bits + 6 parity bits).

[Diagram: see online version]

Each subframe = 10 words x 30 bits = 300 bits. At 50 bps, one subframe takes 6 seconds.

Word Structure (30 bits)

[Diagram: see online version]

All words use Hamming-based parity with bits 25-30 providing error detection. The last two parity bits of the previous word feed into the encoding of the next word.

TLM Word (Word 1 of Every Subframe)

[Diagram: see online version]

HOW Word (Word 2 of Every Subframe)

[Diagram: see online version]

Key Fields

Field	Size	Description
Preamble	8 bits	Fixed pattern 10001011 -- marks start of each subframe
TLM Message	14 bits	Telemetry data (reserved for authorized users)
Integrity Status	1 bit	Signal integrity flag
TOW Count	17 bits	Time of Week in 6-second epochs (0-100799), truncated count at start of next subframe
Alert Flag	1 bit	1 = satellite may be unhealthy, use at own risk

Anti-Spoof Flag	1 bit	1 = anti-spoofing mode active (P(Y) code encrypted)
Subframe ID	3 bits	Identifies subframe (1-5)

Subframe Contents

Subframe 1 -- Clock and Health

Transmitted every 30 seconds (same data for each frame).

Field	Bits	Description
GPS Week Number	10	Week count since Jan 6, 1980 (mod 1024, rollover every ~19.7 years)
SV Accuracy (URA)	4	User Range Accuracy index
SV Health	6	Satellite health summary
IODC	10	Issue of Data, Clock -- change indicates new clock parameters
T_GD	8	Group delay differential (L1-L2 correction), seconds x 2^-31
t_oc	16	Clock data reference time
af2	8	Clock drift rate (sec/sec^2)
af1	16	Clock drift (sec/sec)
af0	22	Clock bias (seconds)

Subframes 2 and 3 -- Ephemeris

Keplerian orbital elements describing the satellite's precise orbit. Updated every 2 hours.

Parameter	Subframe	Description
IODE	2, 3	Issue of Data, Ephemeris (8 bits, must match IODC lower 8 bits)
C_rs	2	Sine correction to orbital radius (meters)
delta_n	2	Mean motion difference (rad/sec)
M_0	2	Mean anomaly at reference time (radians)
C_uc	2	Cosine correction to argument of latitude (radians)
e	2	Eccentricity (dimensionless)
C_us	2	Sine correction to argument of latitude (radians)

sqrt_A	2	Square root of semi-major axis (meters^1/2)
t_oe	2	Ephemeris reference time (seconds)
C_ic	3	Cosine correction to inclination (radians)
OMEGA_0	3	Longitude of ascending node at weekly epoch (radians)
C_is	3	Sine correction to inclination (radians)
i_0	3	Inclination angle at reference time (radians)
C_rc	3	Cosine correction to orbital radius (meters)
omega	3	Argument of perigee (radians)
OMEGA_dot	3	Rate of right ascension (rad/sec)
i_dot	3	Rate of inclination (rad/sec)

Subframe 4 -- Almanac, Ionosphere, UTC (25 pages)

Each of the 25 frames carries a different page:

Pages	Content
1, 6, 11, 16, 21	Reserved
2, 3, 4, 5, 7, 8, 9, 10	Almanac for SVs 25-32 and other data
13	NMCT (Navigation Message Correction Table)
14, 15	Reserved for system use
17	Special messages (ASCII text)
18	Ionospheric correction (Klobuchar alpha_0-3, beta_0-3) and UTC parameters
25	SV health for SVs 1-24 and almanac reference time

Subframe 5 -- Almanac for SVs 1-24 (25 pages)

Pages	Content
1-24	Almanac data for SVs 1-24 (one SV per page)
25	SV health for SVs 1-24, almanac reference time and week

Superframe Structure

[Diagram: see online version]

Position Fix

[Diagram: see online version]

Minimum 4 satellites required: 3 for spatial position (X, Y, Z) plus 1 to solve for receiver clock bias. More satellites improve accuracy through overdetermined solution.

Signal Characteristics

Parameter	L1 C/A
Carrier frequency	1575.42 MHz (154 x 10.23 MHz)
Code type	Gold code (C/A)
Code rate	1.023 Mcps (Megachips/sec)
Code length	1023 chips (1 ms period)
Data rate	50 bps
Modulation	BPSK
Signal power at surface	-158.5 dBW (minimum)
Bandwidth	~2.046 MHz

GNSS Comparison

System	Country	Civilian Signal	Frequency	Access	Satellites
GPS	USA	L1 C/A	1575.42 MHz	CDMA	31 (nominal 24)
GLONASS	Russia	L1OF	1598.0625-1605.375 MHz	FDMA + CDMA	24
Galileo	EU	E1 OS	1575.42 MHz	CDMA	30 (nominal 24)
BeiDou	China	B1C	1575.42 MHz	CDMA	35+
QZSS	Japan	L1 C/A	1575.42 MHz	CDMA	4 (regional)
NavIC	India	L5	1176.45 MHz	CDMA	7 (regional)

GPS Modernization

Signal	Frequency	Status	Key Improvement
L1 C/A	1575.42 MHz	Operational (legacy)	Original civilian signal
L2C	1227.60 MHz	Operational	Second civilian frequency, ionospheric correction
L5	1176.45 MHz	Operational	Safety-of-life, higher power, longer code

L1C	1575.42 MHz	Deploying (GPS III+)	CNAV-2 message, interoperable with Galileo/BeiDou

L1C uses the CNAV (Civil Navigation) message format: variable-length messages, forward error correction (LDPC), and faster data delivery than the legacy 25-page superframe structure.

Standards

Document	Title
IS-GPS-200	GPS L1 C/A Interface Control Document
IS-GPS-705	GPS L5 Interface Control Document
IS-GPS-800	GPS L1C Interface Control Document

See Also

- NMEA -- standard format for GPS receiver output sentences
- RTCM -- differential GNSS corrections for centimeter-level accuracy
- ADS-B -- aircraft surveillance using GPS-derived position
- NTP -- network time synchronization (GPS is a stratum 0 source)

LRIT / HRIT -- Weather Satellite Downlink

Standard: CGMS LRIT/HRIT Global Specification | **Layer:** Application / Data Link (radio broadcast) | **Wireshark filter:** N/A (radio signal, decoded by SatDump / goestools)

LRIT (Low Rate Information Transmission) and HRIT (High Rate Information Transmission) are the standard protocols for broadcasting weather satellite imagery and data products to ground receivers. Defined by the Coordination Group for Meteorological Satellites (CGMS), they are used by GOES (USA), Meteosat (Europe), Himawari (Japan), Fengyun (China), and other geostationary weather satellites. The protocols are built on the CCSDS space packet and TM transfer frame standards, encapsulating image files, text bulletins, and meteorological data within Virtual Channel Data Units (VCDUs) broadcast continuously on L-band frequencies.

Protocol Layers

[Diagram: see online version]

VCDU (Virtual Channel Data Unit)

The VCDU is based on the CCSDS TM Transfer Frame. Each VCDU is 892 bytes for GOES HRIT.

[Diagram: see online version]

VCDU Fields

Field	Size	Description
Version	2 bits	Frame version (01 for AOS/VCDU)
Spacecraft ID	8 bits	Identifies satellite (e.g., GOES-16 = 0xC0)
Virtual Channel ID	6 bits	Multiplexes data streams (images, admin, DCS, fill)
VCDU Counter	24 bits	Sequential frame counter (0-16777215)
Signaling Field	8 bits	Replay flag, VC frame count cycle
M_PDU Header	16 bits	First Header Pointer -- offset to first CP_PDU header in data zone
M_PDU Data Zone	Variable	Contains CP_PDUs (CCSDS space packets carrying LRIT/HRIT data)
Reed-Solomon Check	128 bytes	RS(255,223) parity for error correction

GOES HRIT Virtual Channels

VCID	Content
0	Full disk images (Band 2, visible)
1	Full disk images (Band 7-16, IR)
2	EMWIN (Emergency Managers Weather Information Network)
3	Full disk images (additional bands)
4	GOES-R relay data
5	DCS (Data Collection System)
63	Fill frames (idle)

LRIT/HRIT File Structure

Each file transmitted via LRIT/HRIT consists of a primary header followed by optional secondary headers and the data payload.

Primary Header (16 bytes)

[Diagram: see online version]

Primary Header Fields

Field	Size	Description
Header Type	8 bits	Always 0x00 for primary header
Header Record Length	16 bits	Length of this header record (16 bytes)
File Type Code	8 bits	Type of file being transmitted
Total Header Length	32 bits	Combined length of all headers (bytes)
Data Field Length	64 bits	Length of the data field (bits)

File Type Codes

Code	Description
0	Image data (satellite imagery)
1	GTS messages (WMO Global Telecommunication System)
2	Alphanumeric text
3	Encryption key message
128	EMWIN data (GOES)
129+	Mission-specific / user-defined

Secondary Headers

Header Type	Description
1	Image structure (columns, lines, bits per pixel)
2	Image navigation (projection parameters, sub-satellite point)
3	Image data function (calibration lookup table)
4	Annotation text (human-readable metadata)
5	Timestamp (days since epoch, milliseconds of day)
6	Ancillary text (key=value metadata)
7	Key header (encryption parameters)
128	Segment identification (for multi-segment images)
129	NOAA-specific header
130	Header structure record
131	Rice compression parameters

Image Structure Header

Field	Size	Description
Header Type	8 bits	0x01

Header Record Length	16 bits	Length of this record
Bits per Pixel	8 bits	Typically 8, 10, or 16
Number of Columns	16 bits	Image width in pixels
Number of Lines	16 bits	Image height in pixels
Compression Flag	8 bits	0 = none, 1 = lossless (Rice), 2 = lossy (JPEG)

Image Reception Flow

[Diagram: see online version]

Signal Parameters

Parameter	GOES LRIT	GOES HRIT	GOES GRB
Frequency	1691.0 MHz	1694.1 MHz	1686.6 MHz
Data rate	128 kbps	400 kbps (HRIT/EMWIN)	31 Mbps
Modulation	BPSK	BPSK	DVB-S2 (QPSK/8PSK)
FEC	Viterbi (r=1/2) + RS(255,223)	Viterbi (r=1/2) + RS(255,223)	LDPC + BCH
Polarization	Linear	Linear	Dual circular
Dish size (typical)	0.6-1.0 m	1.0-1.8 m	3.8 m
Full disk interval	3 hours	10-15 minutes	10-15 min (all bands)

Comparison of Weather Satellite Downlink Formats

Format	Data Rate	Coverage	Reception Difficulty	Satellite Examples
APT	2.4 kbps	2 channels, low-res	Easy (V-dipole, RTL-SDR)	NOAA 15/18/19 (LEO, analog)
LRPT	72 kbps	3 channels, medium-res	Moderate (V-dipole, SDR)	Meteor-M2 (LEO, digital)
LRIT	128 kbps	Select bands, reduced-res	Moderate (1m dish, LNA)	GOES (GEO)
HRIT	400 kbps - 3.5 Mbps	Most bands, full-res	Moderate-hard (1m+ dish)	GOES, Himawari, Meteosat
GRB	31 Mbps	All 16 bands, full-res	Hard (3.8m dish, DVB-S2)	GOES-R series
HRD	18.7 Mbps	All channels	Hard (dedicated ground station)	Meteosat Third Generation

Rice Compression (CCSDS 121.0)

LRIT/HRIT uses CCSDS 121.0 Rice compression for lossless image data reduction:

Parameter	Description
Algorithm	Extended Rice (adaptive entropy coding)
Block size	Typically 16 pixels per block
Bits per pixel	8-16 (preserves full dynamic range)
Compression ratio	~1.5:1 to 2.5:1 (image dependent)
Standard	CCSDS 121.0-B-3

Preprocessing splits pixel values into a predictable component (coded with Golomb-Rice) and a remainder, achieving efficient lossless compression well-suited to smooth satellite imagery.

Hobbyist Reception

Receiving GOES HRIT is achievable with consumer-grade equipment:

Component	Example	Purpose
Dish	1.0-1.2 m offset parabolic	Signal collection
Feed	Patch antenna or helix (1694 MHz)	L-band feed at focus
LNA	NooElec SAWbird+ GOES	Low-noise amplifier + filter
SDR	RTL-SDR v3 or Airspy Mini	IQ sampling
Software	SatDump, goestools	Demod, decode, image output
Pointing	Fixed mount aimed at GOES satellite	Geostationary = no tracking needed

Standards

Document	Title
CGMS LRIT/HRIT Global Spec	CGMS LRIT/HRIT Global Specification
CCSDS 132.0-B-3	TM Space Data Link Protocol (VCDU framing)
CCSDS 133.0-B-2	Space Packet Protocol (CP_PDU encapsulation)
CCSDS 121.0-B-3	Lossless Data Compression (Rice)
CCSDS 131.0-B-4	TM Synchronization and Channel Coding
NOAA GOES-R PUG	GOES-R Product User Guide (HRIT/EMWIN)

See Also

- CCSDS -- the space data system standards that LRIT/HRIT is built upon

- DVB-S2 -- used by GRB (GOES Rebroadcast) for full-rate downlink
- GPS -- satellite-based navigation signal (also L-band broadcast)
- ADS-B -- aircraft broadcast protocol (similar hobbyist SDR reception)

RTCM 3.x -- Differential GNSS Corrections

Standard: RTCM Standard 10403.3 | **Layer:** Application (over serial, HTTP, or radio) | **Wireshark filter:** `rtcm` (limited support)

RTCM (Radio Technical Commission for Maritime Services) SC-104 defines the standard format for transmitting differential GNSS corrections from a base station (known position) to a rover receiver. RTCM 3.x is the current version, enabling RTK (Real-Time Kinematic) positioning with centimeter-level accuracy. Corrections include pseudorange and carrier-phase observations, base station coordinates, and satellite ephemerides. RTCM data is commonly transported over serial links (UART), radio modems (UHF/VHF), or the Internet via NTRIP.

RTCM 3.x Frame

[Diagram: see online version]

Field	Size	Description
Preamble	8 bits	Fixed 0xD3 -- synchronization byte
Reserved	6 bits	Set to 0
Message Length	10 bits	Length of data message in bytes (0-1023)
Data Message	0-1023 bytes	RTCM message content (starts with 12-bit message type ID)
CRC-24Q	24 bits	Qualcomm CRC-24 for error detection

Total frame overhead: 6 bytes (3 header + 3 CRC). Maximum frame size: 1029 bytes.

Data Message Header

Every data message begins with a 12-bit message type number:

[Diagram: see online version]

The exact field layout varies per message type, but the first 12 bits always identify the message.

Key Message Types

Legacy Observation Messages

Message	Description
1001	GPS L1 observations (compact)
1002	GPS L1 observations (extended -- includes pseudorange, carrier phase, lock time)
1003	GPS L1/L2 observations (compact)
1004	GPS L1/L2 observations (extended -- full dual-frequency)

Station and Antenna

Message	Description
1005	Station ARP (Antenna Reference Point) in ECEF XYZ, no antenna height
1006	Station ARP in ECEF XYZ with antenna height
1007	Antenna descriptor (name string)
1008	Antenna descriptor with serial number
1033	Receiver and antenna descriptor strings

Ephemeris

Message	Description
1019	GPS satellite ephemeris
1020	GLONASS satellite ephemeris
1042	BeiDou satellite ephemeris
1045	Galileo F/NAV ephemeris
1046	Galileo I/NAV ephemeris

GLONASS Observations

Message	Description
1009	GLONASS L1 observations (compact)
1010	GLONASS L1 observations (extended)
1011	GLONASS L1/L2 observations (compact)
1012	GLONASS L1/L2 observations (extended)
1230	GLONASS code-phase biases

MSM (Multiple Signal Messages) -- Modern Format

MSM messages are the preferred modern format supporting all GNSS constellations uniformly. MSM types indicate data richness:

MSM Type	Content	Typical Use
MSM1	Pseudorange only (compact)	Low bandwidth
MSM2	Pseudorange (extended)	--
MSM3	Pseudorange + carrier phase (no Doppler/SNR)	--
MSM4	Pseudorange + carrier phase + Doppler + SNR (compact)	Common for RTK
MSM5	Full pseudorange + carrier phase + Doppler + SNR (extended)	Common for RTK
MSM6	High-resolution pseudorange + carrier phase	--
MSM7	Full high-resolution observations	Highest precision

MSM by Constellation

Constellation	MSM1	MSM4	MSM5	MSM7
GPS	1071	1074	1075	1077
GLONASS	1081	1084	1085	1087
Galileo	1091	1094	1095	1097
SBAS	1101	1104	1105	1107
BeiDou	1111	1114	1115	1117
QZSS	1121	1124	1125	1127
NavIC/IRNSS	1131	1134	1135	1137

Proprietary Messages

Message	Source	Description
4072	u-blox	Reference station metadata (PVT, moving base)
4094	Various	Proprietary extensions

RTK Correction Flow

[Diagram: see online version]

NTRIP (Networked Transport of RTCM via Internet Protocol)

NTRIP allows streaming RTCM corrections over the Internet using HTTP/1.1. It consists of three components:

[Diagram: see online version]

NTRIP Architecture

Component	Role
NTRIP Server	Connects to caster, sends RTCM data from a base station
NTRIP Caster	Internet relay, maintains source table, authenticates clients
NTRIP Client	Connects to caster, receives RTCM corrections for rover
Source Table	Lists available mountpoints with position, format, carrier info

NTRIP v2.0 supports HTTP/1.1 chunked transfer, RTSP-based streaming, and UDP transport.

Positioning Methods

Method	Accuracy	Corrections	Convergence
Standalone GNSS	2-5 m	None	Instant
DGNSS (Differential)	0.5-1 m	RTCM pseudorange corrections	Instant
RTK (Real-Time Kinematic)	1-2 cm	RTCM carrier-phase observations	5-30 sec (fix)
Network RTK (VRS/MAC)	1-2 cm	NTRIP from CORS network	5-30 sec
PPP (Precise Point Positioning)	2-5 cm	Precise orbits/clocks (SSR)	15-30 min
PPP-RTK	1-3 cm	SSR + atmospheric corrections	1-5 min

Transport Options

Transport	Medium	Range	Typical Use
Serial (UART)	Cable	Direct connection	Base to rover (survey)
UHF/VHF radio modem	400-470 MHz	5-20 km	Agriculture, survey
NTRIP (HTTP)	Internet	Global	CORS networks, cellular rovers
LoRa	868/915 MHz	2-15 km	Low-power, long-range
Cellular (4G/5G)	Mobile network	Network coverage	Machine control, autonomous vehicles
Wi-Fi	2.4/5 GHz	~50 m	Local reference station

Standards

Document	Title
RTCM 10403.3	RTCM Standard for Differential GNSS Services, Version 3.3
RTCM 10410.1	NTRIP Version 2.0
BKG NTRIP	NTRIP Caster reference implementation (BKG)

See Also

- GPS -- the navigation message that RTCM corrections enhance
- NMEA -- receiver output format, also used for GGA in NTRIP
- LoRa -- low-power radio transport for RTCM
- HTTP -- transport protocol used by NTRIP

Storage Networking

Storage area networking and block-level storage protocols.

3 protocols in this section

Fibre Channel -- FC

Standard: INCITS FC-FS-5 | **Layer:** Full stack (FC-0 through FC-4) | **Wireshark filter:** `fc`

Fibre Channel is a high-speed, lossless network technology designed primarily for storage area networks (SANs). It provides in-order, guaranteed delivery of raw block data between servers and storage arrays at speeds from 1 Gbps to 256 Gbps per link. FC defines its own five-layer protocol stack (FC-0 through FC-4), with the upper layer (FC-4) mapping protocols like SCSI (FCP) and mainframe channels (FICON) onto the fabric. Fibre Channel over Ethernet (FCoE) encapsulates FC frames in Ethernet for convergence with data center networks.

FC Frame Structure

[Diagram: see online version]

Key Fields

Field	Size	Description
SOF	4 bytes	Start of Frame delimiter (defines frame class)
R_CTL	8 bits	Routing Control -- frame category and type
D_ID	24 bits	Destination port address (Domain, Area, Port)
S_ID	24 bits	Source port address
Type	8 bits	Upper-layer protocol (0x08 = FCP/SCSI, 0x20 = FC-CT)
F_CTL	24 bits	Frame Control -- sequence context, exchange role, end flags
SEQ_ID	8 bits	Identifies frames within a sequence
SEQ_CNT	16 bits	Frame order within a sequence
OX_ID	16 bits	Originator Exchange ID -- identifies the I/O operation
RX_ID	16 bits	Responder Exchange ID -- assigned by target
Parameter	32 bits	Relative offset for data frames
CRC	32 bits	CRC-32 over header and payload
EOF	4 bytes	End of Frame delimiter

Field Details

R_CTL (Routing Control)

R_CTL	Category	Meaning
0x00-0x0F	Device Data	Data frames (FCP_DATA)
0x02	Device Data	Unsolicited data
0x04	Device Data	Solicited data
0x06	Device Data	Unsolicited command
0x20-0x2F	Extended Link Services	ELS frames (FLOGI, PLOGI)
0x80-0x8F	FC-4 Link Data	FCP_CMND, FCP_XFER_RDY, FCP_RSP

F_CTL (Frame Control) Key Bits

Bit	Name	Description
23	Exchange Originator	1 = sent by originator of the exchange
22	Sequence Recipient	1 = sent by sequence recipient
21	First Sequence	1 = first sequence of the exchange
20	Last Sequence	1 = last sequence of the exchange
19	End Sequence	1 = last frame of the sequence
16	Transfer Sequence Initiative	1 = pass sequence initiative

Type Field

Type	Protocol	Description
0x00	BLS	Basic Link Services (ABTS, BA_ACC)
0x01	ELS	Extended Link Services (FLOGI, PLOGI, LOGO)
0x08	FCP	SCSI over Fibre Channel
0x12	FICON	IBM mainframe channel
0x20	FC-CT	Common Transport (name server queries)

FC Address Format

A 24-bit FC address is structured as Domain (8 bits) + Area (8 bits) + Port (8 bits):

Address	Meaning
0xFFFFFF	Broadcast
0xFFFFFE	Fabric F_Port (fabric login)

0xFFFFFC	Directory Server (Name Server)
0xFFFFFB	Fabric Controller
0x000000	Unknown (not yet assigned)

FC Protocol Stack

[Diagram: see online version]

Fabric Login (FLOGI / PLOGI)

[Diagram: see online version]

FCP (SCSI over Fibre Channel) Exchange

[Diagram: see online version]

Classes of Service

Class	Delivery	Flow Control	Description
Class 2	Connectionless	Credit-based	Acknowledged, with possible out-of-order; rare
Class 3	Connectionless	Credit-based	Unacknowledged datagram -- dominant in SANs
Class F	Connectionless	Credit-based	Switch-to-switch inter-switch link frames

Class 3 is overwhelmingly used in modern FC SANs; the fabric provides lossless, in-order delivery.

Flow Control

Level	Mechanism	Description
Buffer-to-Buffer (BB)	BB_Credit	Controls frames between adjacent ports (link level)
End-to-End (EE)	EE_Credit	Controls frames between N_Ports (rarely used with Class 3)

BB_Credit is exchanged during FLOGI/PLOGI. A sender can transmit BB_Credit frames before waiting for R_RDY primitives.

Zoning and LUN Masking

Mechanism	Layer	Description
Zoning	Fabric (switch)	Restricts which ports can communicate -- like VLANs
Hard Zoning	Port-based	Enforced by switch hardware (D_ID filtering)
Soft Zoning	WWN-based	Enforced by name server (can be bypassed)
LUN Masking	Target/array	Controls which initiators can see which LUNs

FC Speeds

Generation	Line Rate	Throughput (per lane)	Encoding
1GFC	1.0625 Gbaud	100 MB/s	8b/10b
2GFC	2.125 Gbaud	200 MB/s	8b/10b
4GFC	4.25 Gbaud	400 MB/s	8b/10b
8GFC	8.5 Gbaud	800 MB/s	8b/10b
16GFC	14.025 Gbaud	1600 MB/s	64b/66b
32GFC	28.05 Gbaud	3200 MB/s	64b/66b
64GFC	57.8 Gbaud	6400 MB/s	256b/257b

Fibre Channel over Ethernet (FCoE)

FCoE encapsulates FC frames directly in Ethernet, allowing FC traffic to share a converged Ethernet fabric:

[Diagram: see online version]

Parameter	Value
EtherType	0x8906
Requires	DCB (Data Center Bridging) -- PFC, ETS, DCBX
MTU	2500+ bytes (baby jumbo for 2112-byte FC payload)
FIP	FCoE Initialization Protocol (discovery, FLOGI over Ethernet)

Standards

Document	Title
INCITS FC-FS-5	Fibre Channel Framing and Signaling
INCITS FC-PI-7	Fibre Channel Physical Interface

INCITS FC-LS-4	Fibre Channel Link Services
INCITS FC-GS-8	Fibre Channel Generic Services (Name Server)
INCITS FCP-5	Fibre Channel Protocol for SCSI
INCITS FC-BB-6	Fibre Channel Backbone (includes FCoE)

See Also

- iSCSI -- SCSI over TCP/IP (lower cost alternative)
- NVMe-oF -- NVMe over Fabrics (including FC-NVMe)
- Ethernet -- FCoE converges FC onto Ethernet
- RDMA -- competing high-performance fabric technology

iSCSI -- Internet Small Computer Systems Interface

Standard: RFC 7143 | **Layer:** Application (SCSI over TCP/IP) | **Wireshark filter:** `iscsi`

iSCSI maps the SCSI command set onto TCP/IP, allowing block-level storage access over standard Ethernet networks. An initiator (client) sends SCSI commands encapsulated in iSCSI PDUs to a target (storage device or array) over one or more TCP connections on port 3260. iSCSI brought SAN-class storage to commodity networks, eliminating the need for dedicated Fibre Channel fabrics. It remains widely deployed in enterprise environments alongside newer protocols like NVMe-oF.

Basic Header Segment (BHS) -- 48 bytes

Every iSCSI PDU begins with a 48-byte Basic Header Segment:

[Diagram: see online version]

Key Fields

Field	Size	Description
Opcode	6 bits	PDU type (command, response, data, login, etc.)
I (Immediate)	1 bit	Immediate delivery -- not queued in command window
F (Final)	1 bit	Final PDU in a sequence
TotalAHSLength	8 bits	Length of Additional Header Segments in 4-byte words
DataSegmentLength	24 bits	Length of data segment in bytes

LUN	64 bits	Logical Unit Number -- identifies storage volume
ITT (Initiator Task Tag)	32 bits	Unique tag matching commands to responses
CmdSN	32 bits	Command Sequence Number -- ordered command delivery
ExpStatSN	32 bits	Expected Status Sequence Number -- flow control

PDU Types (Opcodes)

Initiator Opcodes

Opcode	Name	Description
0x00	NOP-Out	Keepalive / ping from initiator
0x01	SCSI Command	Carries a SCSI CDB to the target
0x02	Task Management Request	Abort task, LUN reset, target reset
0x03	Login Request	Session establishment and negotiation
0x04	Text Request	Parameter negotiation after login
0x05	Data-Out	Write data from initiator to target
0x06	Logout Request	Session teardown
0x10	SNACK	Request retransmission of a PDU

Target Opcodes

Opcode	Name	Description
0x20	NOP-In	Keepalive / ping response from target
0x21	SCSI Response	Status and sense data for a SCSI command
0x22	Task Management Response	Result of a task management request
0x23	Login Response	Login phase reply with negotiation parameters
0x24	Text Response	Parameter negotiation reply
0x25	Data-In	Read data from target to initiator
0x26	Logout Response	Logout acknowledgment
0x31	Ready to Transfer (R2T)	Target solicits write data from initiator
0x32	Async Message	Asynchronous event notification
0x3F	Reject	Target rejects a PDU

Login Phase

The login phase establishes a session and negotiates parameters before any SCSI commands can be exchanged:

[Diagram: see online version]

Login Phases

Phase	CSG Value	Description
Security Negotiation	0	Authentication (CHAP, SRP, Kerberos)
Operational Negotiation	1	Negotiate parameters (burst lengths, queuing)
Full Feature Phase	3	SCSI command processing

SCSI Read Flow

[Diagram: see online version]

SCSI Write Flow

[Diagram: see online version]

Key Negotiation Parameters

Parameter	Default	Description
MaxRecvDataSegmentLength	8192	Maximum data segment size per PDU
MaxBurstLength	262144	Maximum unsolicited + solicited data per command
FirstBurstLength	65536	Maximum unsolicited data with immediate write
MaxConnections	1	Number of TCP connections per session
MaxOutstandingR2T	1	Outstanding R2T PDUs before more data
InitialR2T	Yes	Require R2T before sending write data
ImmediateData	Yes	Allow data with the command PDU
ErrorRecoveryLevel	0	0=session, 1=digest, 2=connection recovery
HeaderDigest	None	CRC32C integrity check on headers
DataDigest	None	CRC32C integrity check on data

Naming and Discovery

iSCSI Qualified Name (IQN)

Format: `iqn.YYYY-MM.reversed.domain:identifier`

Example: `iqn.2024-01.com.example:storage.lun0`

Discovery Methods

Method	Description
Static	Manually configured target addresses
SendTargets	Query a known target portal for available targets
iSNS	Internet Storage Name Service -- centralized discovery and management
SLP	Service Location Protocol (less common)

SendTargets Discovery

[Diagram: see online version]

Session Architecture

Concept	Description
Session	Initiator-to-target relationship (one or more connections)
Connection	Single TCP connection within a session
ISID	Initiator Session ID -- identifies the session
TSIH	Target Session Identifying Handle -- assigned by target
CID	Connection ID within a session
CmdSN Window	Flow control -- target advertises ExpCmdSN to MaxCmdSN

Protocol Comparison

Feature	iSCSI	Fibre Channel	NVMe-oF (TCP)
Transport	TCP/IP (Ethernet)	Dedicated FC fabric	TCP/IP (Ethernet)
Default Port	3260	N/A (lossless fabric)	4420
Latency	100-500 us	10-50 us	50-200 us
Protocol Overhead	SCSI + iSCSI + TCP	SCSI + FCP	NVMe (native)
Queue Depth	Negotiated (128-256 typical)	Per-exchange	64K queues x 64K depth

Network	Standard Ethernet	FC switches (dedicated)	Standard Ethernet
Cost	Low (commodity HW)	High (dedicated fabric)	Low-Medium

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 7143	Internet Small Computer System Interface (iSCSI) Protocol (consolidated)
RFC 7144	iSCSI SCSI Features Update
RFC 4171	iSNS -- Internet Storage Name Service
RFC 3723	Securing Block Storage Protocols over IP (IPsec for iSCSI)
RFC 3720	iSCSI (original specification, obsoleted by RFC 7143)

See Also

- Fibre Channel -- dedicated storage fabric protocol
- NVMe-oF -- NVMe over Fabrics (lower latency alternative)
- RDMA -- iSER (iSCSI Extensions for RDMA) runs iSCSI over RDMA

NVMe-oF -- NVMe over Fabrics

Standard: NVMe over Fabrics 1.1 (nvmexpress.org) | **Layer:** Application / Transport | **Wireshark filter:** `nvme`

NVMe over Fabrics extends the NVMe protocol beyond a single machine's PCIe bus, allowing hosts to access remote NVMe storage over a network fabric. NVMe-oF preserves the high-parallelism, low-latency architecture of local NVMe -- with up to 65,535 I/O queues each supporting 65,536 outstanding commands -- while transporting commands and data over RDMA (RoCE v2, InfiniBand), TCP, or Fibre Channel. The result is significantly lower latency and higher IOPS than iSCSI for disaggregated storage.

NVMe Architecture (Local vs Fabric)

[Diagram: see online version]

Capsule Structure

NVMe-oF uses capsules to carry NVMe commands, responses, and data over a fabric:

Command Capsule

[Diagram: see online version]

Response Capsule (Completion Queue Entry)

[Diagram: see online version]

Key Fields

Field	Size	Description
Opcode	8 bits	NVMe command opcode (Read=0x02, Write=0x01, Identify=0x06)
Flags	8 bits	Fused operation, PRP/SGL type
Command ID	16 bits	Unique tag for matching completions to commands
NSID	32 bits	Namespace ID -- identifies the storage volume
SGL	128 bits	Scatter-Gather List -- describes data buffer locations
SQ Head Pointer	16 bits	Submission Queue head -- flow control
Status	15 bits	Completion status (success, error codes)
P (Phase)	1 bit	Phase tag -- toggled to indicate new completion

NVMe-oF Transports

Transport Binding Summary

Transport	Port	Encapsulation	Latency	Requires
RDMA (RoCE v2)	4420	NVMe capsule in RDMA Send/Write	10-30 us	Lossless Ethernet (DCB) or RoCE resilience
RDMA (InfiniBand)	4420	NVMe capsule in IB verbs	5-15 us	InfiniBand fabric
TCP	4420	NVMe/TCP PDU over TCP	50-200 us	Standard Ethernet
FC (FC-NVMe)	N/A	NVMe capsule in FC frames	20-50 us	FC fabric (32G/64G)

NVMe/TCP PDU Header

[Diagram: see online version]

PDU Type	Value	Description
ICReq	0x00	Initialize Connection Request
ICResp	0x01	Initialize Connection Response
H2CData	0x02	Host-to-Controller Data
C2HData	0x03	Controller-to-Host Data
CapsuleCmd	0x04	Command Capsule
CapsuleResp	0x05	Response Capsule
H2CTermReq	0x06	Host-to-Controller Terminate Request
C2HTermReq	0x07	Controller-to-Host Terminate Request
R2T	0x09	Ready to Transfer

Submission Queue / Completion Queue Model

[Diagram: see online version]

Queue Parameters

Parameter	Value	Description
Admin Queue	1 pair	Queue pair 0 -- for management commands
I/O Queues	Up to 65,535	One per CPU core is typical
Queue Depth	Up to 65,536	Outstanding commands per queue
Total Commands	~4 billion	65,535 queues x 65,536 depth

Discovery Service

NVMe-oF provides a centralized Discovery Controller for locating storage targets:

[Diagram: see online version]

NVMe Qualified Name (NQN)

Format: `nqn.YYYY-MM.reversed.domain:identifier`

Well-Known NQN	Purpose
`nqn.2014-08.org.nvmexpress.discovery`	Discovery Controller

`nqn.2014-08.org.nvmexpress:uuid:<UUID>`	UUID-based naming

NVMe-oF over RDMA Flow

[Diagram: see online version]

Performance Comparison

Metric	iSCSI	NVMe/TCP	NVMe/RDMA	NVMe/FC
Latency	100-500 us	50-200 us	10-30 us	20-50 us
IOPS (per target)	~200K	~500K	~1M+	~800K
CPU Overhead	High (TCP + iSCSI + SCSI)	Medium (TCP + NVMe)	Very low (kernel bypass)	Low
Queue Depth	~256 typical	65K queues x 65K	65K queues x 65K	65K queues x 65K
Multi-queue	Limited	Yes (per-core)	Yes (per-core)	Yes (per-core)

Encapsulation (TCP Transport)

[Diagram: see online version]

Encapsulation (RDMA Transport)

[Diagram: see online version]

Standards

Document	Title
NVMe Base Spec 2.1	NVM Express Base Specification
NVMe-oF 1.1	NVMe over Fabrics Specification
NVMe/TCP Transport Binding	TCP Transport Binding Specification
RFC 7143	iSCSI (for comparison)
INCITS FC-NVMe-2	FC-NVMe Fibre Channel Transport

See Also

- iSCSI -- SCSI over TCP/IP (predecessor technology)
- RDMA -- transport substrate for NVMe/RDMA (RoCE, InfiniBand)
- Fibre Channel -- FC-NVMe transport
- PCIe -- local NVMe transport (NVMe was designed for PCIe)

Telecommunications

Telecommunications signaling, switching, and transport protocols.

20 protocols in this section

5G NR -- New Radio

Standard: 3GPP TS 38 series | **Layer:** Full stack | **Wireshark filter:** `nr-rrc` or `ngap` or `nas-5gs` or `pfcp`

5G NR is the fifth-generation mobile radio access technology, designed for three use cases: enhanced Mobile Broadband (eMBB -- high speed), Ultra-Reliable Low-Latency Communication (URLLC -- <1ms latency for industrial/automotive), and massive Machine-Type Communication (mMTC -- millions of IoT devices). 5G NR introduces a flexible numerology (variable subcarrier spacing), massive MIMO (up to 256 antennas), mmWave support (24-100 GHz), and a service-based core architecture (5GC).

Architecture (5GC -- 5G Core)

[Diagram: see online version]

5GC Network Functions (NFs)

NF	Full Name	Replaces (LTE)	Role
AMF	Access and Mobility Management	MME (mobility part)	Registration, mobility, security
SMF	Session Management Function	MME (session part) + P-GW-C	PDU session, QoS, IP allocation
UPF	User Plane Function	S-GW-U + P-GW-U	Packet forwarding, QoS enforcement
UDM	Unified Data Management	HSS (data)	Subscription management
AUSF	Authentication Server Function	HSS (auth)	5G-AKA authentication
PCF	Policy Control Function	PCRF	Policy and charging rules
NSSF	Network Slice Selection Function	-- (new)	Selects network slice
NEF	Network Exposure Function	-- (new)	API exposure to applications
NRF	NF Repository Function	-- (new)	Service discovery for NFs
UDR	Unified Data Repository	-- (new)	Central data storage

Key Interfaces

Interface	Between	Protocol	Purpose
N1	UE <-> AMF	NAS-5GS	Registration, auth, session management
N2	gNB <-> AMF	NGAP (SCTP)	Control plane signaling
N3	gNB <-> UPF	GTP-U (UDP)	User data tunneling

N4	SMF <-> UPF	PFCP (UDP)	User plane rules and reporting
N6	UPF <-> DN	IP	External data network
Xn	gNB <-> gNB	XnAP (SCTP)	Handover, dual connectivity

Radio Interface

Parameter	Sub-6 GHz	mmWave
Frequency	410 MHz - 7.125 GHz (FR1)	24.25 - 52.6 GHz (FR2)
Bandwidth	Up to 100 MHz	Up to 400 MHz
Subcarrier spacing	15, 30, 60 kHz	60, 120 kHz
Modulation	QPSK, 16/64/256-QAM	QPSK, 16/64/256-QAM
MIMO	Up to 256 antenna elements (Massive MIMO)	Up to 1024 elements (beamforming)
Peak DL	~4.7 Gbps (100 MHz, 256-QAM, 4x4)	~20 Gbps (theoretical)
Peak UL	~2.5 Gbps	~10 Gbps (theoretical)
Latency	~4 ms (eMBB), <1 ms (URLLC)	~1 ms

Flexible Numerology

Numerology (μ)	Subcarrier Spacing	Slot Duration	Slots/Subframe	Use Case
0	15 kHz	1 ms	1	Sub-6 GHz, compatible with LTE
1	30 kHz	0.5 ms	2	Sub-6 GHz, typical
2	60 kHz	0.25 ms	4	Sub-6 GHz or mmWave
3	120 kHz	0.125 ms	8	mmWave
4	240 kHz	0.0625 ms	16	mmWave (SS/PBCH blocks)

Shorter slots = lower latency. The network can mix numerologies for different use cases.

Deployment Modes

Mode	Description
NSA (Non-Standalone)	5G NR radio + LTE core (EPC). Uses LTE for control plane.
SA (Standalone)	5G NR radio + 5G core (5GC). Full 5G capabilities.
EN-DC	E-UTRA NR Dual Connectivity (LTE master + NR secondary)

NR-DC	NR Dual Connectivity (NR master + NR secondary)

NSA vs SA

[Diagram: see online version]

Network Slicing

5G allows the network to be divided into virtual slices, each tailored for a specific use case:

Slice Type (SST)	Name	Use Case
1	eMBB	Enhanced Mobile Broadband (video, web)
2	URLLC	Ultra-Reliable Low Latency (industry 4.0, autonomous driving)
3	mMTC	Massive Machine Type Communication (IoT sensors)
4	V2X	Vehicle-to-Everything

Each slice is identified by an S-NSSAI (Single Network Slice Selection Assistance Information) = SST + optional SD (Slice Differentiator).

Security (5G-AKA)

5G improves on LTE security:

Feature	LTE	5G
Authentication	EPS-AKA	5G-AKA or EAP-AKA'
SUPI privacy	IMSI sent in cleartext	SUCI (encrypted SUPI) -- prevents IMSI catchers
Integrity (user plane)	Optional (rarely used)	Mandatory support
Key hierarchy	KASME -> KeNB	KAUSF -> KSEAF -> KAMF -> KgNB

SUCI (Subscription Concealed Identifier)

[Diagram: see online version]

The IMSI is never sent in cleartext over the air -- a major improvement over GSM/LTE.

5G vs LTE vs GSM

Feature	GSM (2G)	LTE (4G)	5G NR

Year	1991	2009	2019
Access	TDMA	OFDMA	OFDMA (flexible)
Peak DL	14 kbps	300 Mbps - 3 Gbps	20 Gbps (theoretical)
Latency	~300 ms	~10-20 ms	~1-4 ms
Core	Circuit + Packet (GPRS)	All-IP (EPC)	Service-Based (5GC)
Voice	Circuit-switched	VoLTE (SIP/IMS)	VoNR (SIP/IMS)
Slicing	No	Limited	Native
mmWave	No	No	Yes (FR2)

Standards

Document	Title
3GPP TS 38.300	NR overall description
3GPP TS 38.331	NR RRC Protocol
3GPP TS 24.501	NAS Protocol (5GMM + 5GSM)
3GPP TS 38.211	NR physical channels and modulation
3GPP TS 23.501	5G System architecture
3GPP TS 33.501	5G security architecture

See Also

- LTE -- 4G predecessor
- GSM -- 2G predecessor
- GTP -- user plane tunneling (GTP-U on N3)
- Diameter -- replaced by HTTP/2-based SBI in 5GC
- SIP -- VoNR voice calls via IMS

Diameter

Standard: RFC 6733 | **Layer:** Application (Layer 7) | **Wireshark filter:** `diameter`

Diameter is the AAA (Authentication, Authorization, and Accounting) protocol for modern mobile networks. It replaced RADIUS in 3GPP networks starting with LTE, providing reliable transport (TCP/SCTP), larger message sizes, and better extensibility. Diameter is used between the MME and HSS for subscriber authentication (S6a), between the P-GW and PCRF for policy/charging (Gx), and in many other 3GPP interfaces. The name is a play on words -- Diameter is "twice RADIUS."

Message Format

[Diagram: see online version]

Key Fields

Field	Size	Description
Version	8 bits	Always 1
Message Length	24 bits	Total message length including header
R flag	1 bit	1 = Request, 0 = Answer
P flag	1 bit	Proxiable (can be forwarded by agents)
E flag	1 bit	Error (indicates an error answer)
T flag	1 bit	Retransmitted message
Command Code	24 bits	Identifies the operation
Application-ID	32 bits	Identifies the Diameter application
Hop-by-Hop ID	32 bits	Matches requests/answers on a single hop
End-to-End ID	32 bits	Matches requests/answers end-to-end

AVP (Attribute-Value Pair)

[Diagram: see online version]

Field	Description
AVP Code	Identifies the attribute (IANA-registered or vendor-specific)
V flag	1 = Vendor-ID field present
M flag	1 = Mandatory (receiver must understand)
P flag	1 = End-to-end encryption needed
AVP Length	Length of entire AVP including header
Vendor-ID	IANA enterprise number (if V=1)
Data	Attribute value (typed: OctetString, Unsigned32, UTF8String, Grouped, etc.)

Base Protocol Commands

Code	Name	Description
257	CER / CEA	Capabilities Exchange (session setup)

258	RAR / RAA	Re-Auth (server requests re-authentication)
271	ACR / ACA	Accounting
274	ASR / ASA	Abort-Session (server terminates session)
280	DWR / DWA	Device-Watchdog (keepalive)
282	DPR / DPA	Disconnect-Peer (graceful shutdown)

3GPP Diameter Applications

Interface	Application-ID	Command	Between	Purpose
S6a	16777251	AIR/AIA	MME <-> HSS	Authentication Info (auth vectors)
S6a	16777251	ULR/ULA	MME <-> HSS	Update Location (subscription data)
Gx	16777238	CCR/CCA	P-GW <-> PCRF	Credit-Control (QoS policy, charging)
Gy	4	CCR/CCA	P-GW <-> OCS	Online Charging (real-time prepaid)
Rx	16777236	AAR/AAA	P-CSCF <-> PCRF	QoS for IMS/VoLTE sessions
Sh	16777217	UDR/UDA	AS <-> HSS	User Data (IMS application server)
Cx	16777216	MAR/MAA	I-CSCF/S-CSCF <-> HSS	IMS registration
SWx	16777265	MAR/MAA	ePDG <-> HSS	WiFi calling authentication
S13	16777252	ECR/ECA	MME <-> EIR	Equipment Identity Check (IMEI)

Session Flow (S6a -- LTE Attach)

[Diagram: see online version]

Common AVPs

AVP Code	Name	Description
1	User-Name	Username (often IMSI for 3GPP)
25	Class	Opaque data passed between sessions
263	Session-Id	Unique session identifier

264	Origin-Host	Sending node's FQDN
268	Result-Code	Success/failure (2001=success)
277	Auth-Session-State	Stateful or stateless
283	Destination-Realm	Target realm for routing
293	Destination-Host	Target host (if known)
416	CC-Request-Type	Initial, Update, Termination (for Gx/Gy)
444	Subscription-Id	Subscriber identity (IMSI, MSISDN)

Result Codes

Code	Name	Description
2001	DIAMETER_SUCCESS	Request completed successfully
3001	DIAMETER_COMMAND_UNSUPPORTED	Command not supported
3002	DIAMETER_UNABLE_TO_DELIVER	Cannot route to destination
4001	DIAMETER_AUTHENTICATION_REJECTED	Auth failed
5001	DIAMETER_AVP_UNSUPPORTED	Required AVP not understood
5004	DIAMETER_MISSING_AVP	Required AVP missing
5012	DIAMETER_UNABLE_TO_COMPLY	General failure

Diameter vs RADIUS

Feature	RADIUS	Diameter
Transport	UDP (unreliable)	TCP or SCTP (reliable)
Max message size	~4096 bytes	~16 MB
Security	Shared secret (per-packet)	IPsec or TLS (per-connection)
Agent types	Proxy only	Proxy, Relay, Redirect, Translation
Error handling	Silent discard	Explicit error answers
Failover	Application-dependent	Built-in (TCP/SCTP transport)
Server-initiated	Limited (CoA, DM)	Native (RAR, ASR)
Capability negotiation	None	CER/CEA at connection setup
Use case	Wi-Fi/VPN/ISP	3GPP mobile core (LTE/5G)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 6733	Diameter Base Protocol
RFC 4006	Diameter Credit-Control Application
3GPP TS 29.272	S6a/S6d (MME/SGSN <-> HSS)
3GPP TS 29.212	Gx (Policy and Charging Control)
3GPP TS 29.214	Rx (IMS QoS)
3GPP TS 29.229	Cx/Dx (IMS registration)

See Also

- RADIUS -- predecessor AAA protocol
- LTE -- the network Diameter was built for
- 5G NR -- 5GC replaces Diameter with HTTP/2-based SBI
- GTP -- user/control plane tunneling alongside Diameter
- TCP -- primary transport
- SCTP -- alternative transport

DOCSIS -- Data Over Cable Service Interface Specification

Standard: CableLabs DOCSIS 3.1 / 4.0 | **Layer:** Physical / Data Link (Layers 1-2) | **Wireshark filter:** `docsis`

DOCSIS is the standard for delivering broadband Internet over hybrid fiber-coaxial (HFC) cable television networks. It allows cable operators to provide high-speed data alongside television services on the same coaxial infrastructure. DOCSIS defines downstream (headend to subscriber) and upstream (subscriber to headend) communication using frequency-division multiplexing on the cable plant. DOCSIS 3.1 supports multi-gigabit speeds, and DOCSIS 4.0 enables symmetric 10 Gbps.

DOCSIS Versions

Version	Year	Downstream	Upstream	Key Features
1.0	1997	38 Mbps	9 Mbps	Basic data over cable
1.1	2001	38 Mbps	9 Mbps	QoS, fragmentation

2.0	2002	38 Mbps	27 Mbps	Improved upstream (A-TDMA, S-CDMA)
3.0	2006	1 Gbps+	100 Mbps+	Channel bonding (up to 32 DS / 8 US)
3.1	2013	10 Gbps	1-2 Gbps	OFDM, 4096-QAM, wider channels
4.0	2017	10 Gbps	6 Gbps	Full Duplex DOCSIS (FDX), Extended Spectrum

Network Architecture

[Diagram: see online version]

Component	Description
CMTS	Cable Modem Termination System -- headend equipment (operator side)
HFC	Hybrid Fiber-Coaxial -- fiber to the neighborhood, coax to the home
Fiber Node	Converts between fiber optics and coaxial RF
CM	Cable Modem -- customer premises equipment
CPE	Customer devices behind the cable modem

Frequency Plan

[Diagram: see online version]

Direction	DOCSIS 3.0	DOCSIS 3.1
Downstream	6/8 MHz SC-QAM channels (up to 256-QAM)	Up to 192 MHz OFDM channels (up to 4096-QAM)
Upstream	6.4 MHz channels (up to 64-QAM)	Up to 96 MHz OFDMA channels (up to 4096-QAM)

MAC Frame

Downstream

[Diagram: see online version]

Field	Size	Description
FC	8 bits	Frame control -- type, encryption, extended header flag

MAC Parm	8 bits	Depends on frame type (EHDR length or MAC type)
Length/SID	16 bits	PDU length (downstream) or Service ID (upstream)
EHDR	Variable	Extended header (BPI+, fragmentation, etc.)
PDU	Variable	Payload -- typically an Ethernet frame
CRC	32 bits	CRC over the entire MAC frame

Frame Control Types

FC Type	Name	Description
00	Packet PDU	Data frame (Ethernet payload)
01	ATM PDU	ATM cell (legacy, rare)
10	Reserved	--
11	MAC Management	CMTS-CM signaling messages

Registration and Ranging

Cable Modem Initialization

[Diagram: see online version]

Key MAC Management Messages

Type	Name	Description
1	SYNC	Downstream timing synchronization
2	UCD	Upstream Channel Descriptor (modulation, frequency)
3	MAP	Upstream bandwidth allocation map
4	RNG-REQ	Ranging Request
5	RNG-RSP	Ranging Response (timing/power corrections)
6	REG-REQ	Registration Request
7	REG-RSP	Registration Response
11	UCC-REQ	Upstream Channel Change Request
18	DBC-REQ	Dynamic Bonding Change Request
29	CM-STATUS	Cable Modem Status Report
33	DCD	Downstream Channel Descriptor

35	MDD	MAC Domain Descriptor

Upstream Access -- TDMA/OFDMA

The upstream is a shared medium. The CMTS allocates timeslots to cable modems using MAP messages:

Mechanism	DOCSIS Version	Description
TDMA	1.x-3.0	Time Division Multiple Access -- modems transmit in assigned time slots
S-CDMA	2.0-3.0	Synchronous Code Division -- multiple modems share same timeslot with codes
OFDMA	3.1+	Orthogonal Frequency Division Multiple Access -- modems assigned subcarrier groups

Contention and Grants

[Diagram: see online version]

Channel Bonding (DOCSIS 3.0)

Multiple channels are bonded together for higher throughput:

Direction	DOCSIS 3.0	DOCSIS 3.1
Downstream	Up to 32 x 6 MHz channels = ~1.2 Gbps	2 x 192 MHz OFDM = ~10 Gbps
Upstream	Up to 8 x 6.4 MHz channels = ~120 Mbps	2 x 96 MHz OFDMA = ~1.5 Gbps

Security (BPI+ -- Baseline Privacy Interface Plus)

Feature	Description
Authentication	X.509 certificates (CM has manufacturer certificate)
Key exchange	Authenticated key agreement (CMTS -> CM)
Encryption	AES-128-CBC (traffic encryption keys per SAID)
Key refresh	TEKs rotated periodically

DOCSIS 4.0

Feature	Description

FDX (Full Duplex)	Same frequencies used for both upstream and downstream simultaneously
Extended Spectrum	Upstream expanded up to 684 MHz
Target speeds	10 Gbps downstream, 6 Gbps upstream
Backward compatible	Coexists with DOCSIS 3.1 on the same plant

Encapsulation

[Diagram: see online version]

Unlike DSL (which typically uses PPPoE), DOCSIS presents a standard Ethernet interface to the customer. Most cable ISPs use IPoE (IP over Ethernet) with DHCP for addressing.

Standards

Document	Title
DOCSIS 3.1 PHY	DOCSIS 3.1 Physical Layer Specification
DOCSIS 3.1 MAC	DOCSIS 3.1 MAC and Upper Layer Protocols
DOCSIS 4.0	DOCSIS 4.0 Specification
DOCSIS 3.0	DOCSIS 3.0 Specifications
BPI+	Baseline Privacy Plus Interface Specification

See Also

- xDSL -- telephone-line alternative to cable broadband
- Ethernet -- interface presented to customer
- DHCP -- IP address assignment for cable modems
- IPv4

E1

Standard: ITU-T G.703 / G.704 | **Layer:** Physical (Layer 1) | **Wireshark filter:** N/A (sub-packet-capture)

E1 is the digital transmission standard used outside North America and Japan, carrying 2.048 Mbps over two twisted pairs or coaxial cable. It multiplexes 32 timeslots of 64 kbps each using TDM. Timeslot 0 is reserved for framing/synchronization and timeslot 16 is typically used for signaling, leaving 30 channels for voice or data. E1 is the fundamental building block of the European/international digital hierarchy (PDH) and is used for voice trunks, ISDN PRI, SS7 signaling, and data circuits.

Frame

An E1 frame is 256 bits (32 bytes), transmitted 8,000 times per second:

[Diagram: see online version]

Field	Size	Description
Timeslot 0	8 bits	Frame alignment and service bits
Timeslots 1-15	8 bits each	Bearer channels (voice/data)
Timeslot 16	8 bits	Signaling (CAS) or bearer (CCS)
Timeslots 17-31	8 bits each	Bearer channels (voice/data)

Total: 32 x 8 = 256 bits x 8,000 frames/sec = **2,048,000 bps**

Key Parameters

Parameter	Value
Line rate	2.048 Mbps
Total timeslots	32 x DS0 (64 kbps)
Bearer channels	30 (TS1-15, TS17-31)
Frame size	256 bits (32 bytes)
Frame rate	8,000 frames/sec
Encoding	HDB3 (High Density Bipolar 3)
Cable	2 twisted pairs (120Ω) or coax (75Ω)
Connector	RJ-48C (twisted pair) or BNC (coax)

Frame Structure

Timeslot 0 (Framing)

Alternating frames carry Frame Alignment Signal (FAS) and Not-FAS patterns:

Even frames (FAS):

[Diagram: see online version]

Odd frames (Not-FAS):

[Diagram: see online version]

Field	Description
Si	International bit (spare, set to 1)

0011011	Frame Alignment Signal -- fixed pattern for frame sync
A	Remote Alarm Indication (RAI); 1 = alarm
Sa4-Sa8	Spare bits (used for CRC-4 in extended framing)

CRC-4 Multiframe

When CRC-4 is enabled, 16 frames form a multiframe. The Sa bits in timeslot 0 carry a CRC-4 over the preceding sub-multiframe, providing error detection.

Timeslot 16 (Signaling)

CAS (Channel Associated Signaling)

16 frames form a signaling multiframe. Each frame carries signaling for two channels (A/B/C/D bits):

[Diagram: see online version]

Frame 0 of the multiframe contains the Multiframe Alignment Signal: `0000 1101`

Bits	Typical Meaning
A=1, B=1	Off-hook (active)
A=0, B=0	On-hook (idle)
A=1, B=0	Seizure / wink

CCS (Common Channel Signaling)

When timeslot 16 carries signaling using HDLC (LAPD for ISDN PRI or SS7), all 64 kbps are available for the D-channel, and all 30 bearer channels get full 64 kbps.

Configurations

Configuration	TS0	TS16	Bearer	Total Bearer
CAS (30B+D)	Framing	CAS signaling	TS1-15, TS17-31	30 x 64 kbps
PRI ISDN (30B+D)	Framing	D-channel (Q.931)	TS1-15, TS17-31	30 x 64 kbps
Unframed	--	--	All 32 timeslots	2.048 Mbps clear channel
Fractional E1	Framing	--	Selected timeslots	N x 64 kbps

Line Coding

Encoding	Description
HDB3 (High Density Bipolar 3)	Bipolar AMI with substitution of 4 consecutive zeros to maintain clock sync. Always used on E1.

HDB3 replaces strings of 4 zeros with a pattern containing a bipolar violation, ensuring sufficient signal transitions for clock recovery.

PDH Hierarchy (International)

Level	Rate	Channels	Composition
E0	64 kbps	1	Single channel
E1	2.048 Mbps	32 (30 bearer)	32 x E0
E2	8.448 Mbps	128	4 x E1
E3	34.368 Mbps	512	4 x E2 (16 x E1)
E4	139.264 Mbps	2048	4 x E3 (64 x E1)

E1 vs T1

Parameter	E1	T1
Line rate	2.048 Mbps	1.544 Mbps
Total timeslots	32	24 + 1 framing bit
Bearer channels	30	23 (PRI) or 24 (CAS)
Signaling channel	TS16 (dedicated)	TS24 or robbed-bit
Frame size	256 bits	193 bits
Line coding	HDB3	AMI or B8ZS
Framing overhead	TS0 (in-slot)	1 bit per frame (out-of-slot)
Geography	Europe, Asia, Africa, South America	North America, Japan

Standards

Document	Title
ITU-T G.703	Physical/electrical characteristics of hierarchical digital interfaces
ITU-T G.704	Synchronous frame structures at 1544, 6312, 2048, 8488, and 44736 kbit/s
ITU-T G.706	Frame alignment and CRC procedures
ITU-T G.732	Characteristics of primary PCM multiplex equipment at 2048 kbit/s

See Also

- T1 -- North American equivalent (1.544 Mbps, 24 channels)
- ISDN -- PRI signaling over E1
- SS7 -- signaling carried in E1 timeslots

EPON / 10G-EPON -- Ethernet Passive Optical Network

Standard: IEEE 802.3ah | **Layer:** Physical + Data Link (Layers 1-2) | **Wireshark filter:** epon

EPON is the IEEE-standardized approach to passive optical networking, using native Ethernet framing rather than a custom encapsulation like GPON's GEM. It shares the same physical topology -- OLT to passive splitter to ONUs -- but manages upstream access through MPCP (Multi-Point Control Protocol), an extension to Ethernet. EPON is dominant in Asia (especially Japan and China) while GPON dominates in Europe and the Americas. 10G-EPON (IEEE 802.3av) extends the architecture to 10 Gbps, with asymmetric (10G/1G) and symmetric (10G/10G) variants.

Architecture

[Diagram: see online version]

Parameter	EPON (802.3ah)	10G-EPON (802.3av)
Downstream rate	1.25 Gbps	10.3125 Gbps
Upstream rate	1.25 Gbps	1.25 Gbps (asym) or 10.3125 Gbps (sym)
Downstream wavelength	1490 nm	1577 nm
Upstream wavelength	1310 nm	1270 nm
Split ratio	1:16 / 1:32	1:16 / 1:32
Max reach	20 km	20 km
Line coding	8b/10b	64b/66b (10G)
FEC	RS(255,239) optional	RS(255,223) mandatory

Key Difference from GPON

EPON carries **standard Ethernet frames** directly over the PON -- no GEM encapsulation, no GTC framing. The Ethernet frame itself is the transport unit. This simplicity is EPON's primary advantage:

Aspect	EPON	GPON
Transport unit	Ethernet frame	GEM frame (5-byte header)
Fragmentation	No (full Ethernet frames)	Yes (GEM fragments large frames)
Overhead	Lower (native Ethernet)	Higher (GEM + GTC headers)

Efficiency	~95% at 1G	~93% at 2.5G
Upstream control	MPCP (GATE/REPORT)	BWmap in GTC header
Management	IEEE 802.3ah OAM	OMCI (ITU-T G.988)
TDM support	Not native (use CES)	Native GEM TDM mapping

MPCP (Multi-Point Control Protocol)

MPCP manages the point-to-multipoint Ethernet link by controlling upstream TDMA access. It uses Ethernet frames with EtherType **0x8808** (MAC Control):

GATE Message (OLT to ONU)

The OLT sends GATE messages to grant upstream transmission windows to each ONU:

[Diagram: see online version]

Field	Size	Description
Opcode	2 bytes	0x0002 = GATE
Timestamp	4 bytes	OLT's local time when frame was sent
Number of grants	4 bits	Number of grant windows (0-4)
Grant Start Time	4 bytes	When the ONU may begin transmitting (32-bit time)
Grant Length	2 bytes	Duration of the grant (in time quanta = 16 ns)
Sync Time	4 bytes	Used for clock synchronization

A single GATE message can carry up to 4 grant windows, allowing burst scheduling for different priority queues.

REPORT Message (ONU to OLT)

ONUs send REPORT messages to inform the OLT of their queue depths for DBA:

[Diagram: see online version]

Field	Size	Description
Opcode	2 bytes	0x0003 = REPORT
Timestamp	4 bytes	ONU's local time (used for RTT calculation)
Number of queue sets	1 byte	Number of queue set reports included
Report bitmap	2 bytes	Bitmask of which queue lengths are reported
Queue length	2 bytes each	Pending data per queue (in time quanta)

The OLT uses REPORT data to run its DBA algorithm and determine grant sizes for the next cycle.

LLID (Logical Link ID)

Each ONU is assigned a unique LLID during registration, embedded in the Ethernet preamble:

[Diagram: see online version]

LLID Value	Meaning
0x7FFF	Broadcast (all ONUs)
0x0001 - 0x7FFE	Unicast (specific ONU)
Mode bit = 0	Single ONU (unicast LLID)
Mode bit = 1	OLT emulated (broadcast/multicast)

Downstream: The OLT broadcasts all frames; each ONU checks the LLID in the preamble and only accepts frames matching its LLID (or broadcast).

Upstream: Each ONU stamps its LLID in the preamble so the OLT knows which ONU sent each frame.

ONU Discovery and Registration

[Diagram: see online version]

Discovery Process Details

1. **Discovery GATE:** OLT periodically opens a discovery window (broadcast GATE with a grant for unregistered ONUs)
2. **Register Request:** New ONUs transmit their MAC address with a random delay to reduce collisions
3. **Register:** OLT assigns a unique LLID and provides timing correction (RTT compensation)
4. **Register Ack:** ONU confirms the LLID assignment
5. **Normal operation:** OLT begins sending unicast GATE messages; ONU sends REPORT messages

OAM (IEEE 802.3ah)

EPON uses IEEE 802.3ah OAM for link management, carried on EtherType **0x8809**:

Function	Description
Discovery	OAM capability negotiation between OLT and ONU
Link monitoring	Error counters, symbol errors, frame errors
Remote fault indication	ONU signals link problems to OLT
Remote loopback	OLT places ONU in loopback mode for testing
Variable retrieval	Read ONU MIB variables
Organization-specific	Vendor extensions (DPoE, CTC, etc.)

OAM PDU Types

Code	Type	Description
0x00	Information	Periodic keepalive + capability exchange
0x01	Event Notification	Threshold-crossing alarm
0x02	Variable Request	Read MIB variables
0x03	Variable Response	MIB variable values
0x04	Loopback Control	Enable/disable remote loopback
0xFE	Organization Specific	Vendor/operator extensions

DPoE (DOCSIS Provisioning of EPON)

CableLabs defined DPoE to allow cable operators to deploy EPON using their existing DOCSIS provisioning infrastructure:

Component	Role
DPoE OLT	Looks like a CMTS to the provisioning system
DPoE ONU	Looks like a cable modem to the provisioning system
DPoE System	Maps DOCSIS provisioning messages to OAM

DPoE enables TFTP-based config file provisioning, BPI+ security, and DOCSIS QoS classifiers over EPON -- letting cable operators use their existing OSS/BSS.

10G-EPON and Beyond

10G-EPON (IEEE 802.3av)

Mode	Downstream	Upstream	Wavelengths
10G/1G (asymmetric)	10.3125 Gbps	1.25 Gbps	1577nm down, 1310nm up
10G/10G (symmetric)	10.3125 Gbps	10.3125 Gbps	1577nm down, 1270nm up

Can coexist with 1G-EPON on the same fiber using WDM.

25G/50G-EPON (IEEE 802.3ca)

Mode	Downstream	Upstream
25G	25.78125 Gbps	10 or 25 Gbps
50G	2 x 25G (channel bonding)	2 x 25G

Uses NRZ modulation at 25G per wavelength, with optional channel bonding for 50G.

EPON vs GPON Comparison

Feature	EPON	GPON
Standards body	IEEE	ITU-T
Transport	Native Ethernet	GEM encapsulation
Upstream control	MPCP (GATE/REPORT)	BWmap in GTC
Management	802.3ah OAM	OMCI (G.988)
DS / US rate	1G / 1G	2.5G / 1.2G
Split ratio	1:32 typical	1:64 typical
TDM support	Circuit Emulation Service	Native GEM mapping
Fragmentation	None	GEM fragments large frames
Market	Japan, China, cable (DPoE)	Europe, Americas, global
10G upgrade	10G-EPON (802.3av)	XGS-PON (G.9807)
Next-gen	25G/50G-EPON (802.3ca)	50G-PON (G.9804)

Standards

Document	Title
IEEE 802.3ah-2004	Ethernet in the First Mile (EPON)
IEEE 802.3av-2009	10Gb/s Ethernet Passive Optical Network (10G-EPON)
IEEE 802.3ca-2020	25Gb/s and 50Gb/s EPON
CableLabs DPoE	DOCSIS Provisioning of EPON
IEEE 802.3-2022 Clause 64	MPCP specification
IEEE 802.3-2022 Clause 57	802.3ah OAM specification

See Also

- GPON -- ITU-T counterpart using GEM encapsulation
- 50G-PON / NG-PON2 -- next-generation PON technologies
- Ethernet -- the frame format EPON carries natively
- LACP -- link aggregation (related 802.3 control protocol)
- LLDP -- discovery protocol often run over PON
- TR-069 / TR-369 -- remote management of ONUs

GPON / XGS-PON -- Gigabit Passive Optical Network

Standard: ITU-T G.984 | **Layer:** Physical + Data Link (Layers 1-2) | **Wireshark filter:** gpon

GPON is the dominant fiber-to-the-premises (FTTP) technology, delivering gigabit broadband over a point-to-multipoint passive optical network. A single fiber from an OLT (Optical Line Terminal) at the central office reaches up to 64 subscribers via passive splitters, with each subscriber connected through an ONT/ONU (Optical Network Terminal/Unit). GPON uses wavelength-division multiplexing to separate downstream (1490nm) and upstream (1310nm) traffic on the same fiber. XGS-PON (G.9807) is the 10G symmetric successor, coexisting on the same infrastructure using different wavelengths.

Architecture

[Diagram: see online version]

Parameter	GPON (G.984)	XGS-PON (G.9807)
Downstream rate	2.488 Gbps	9.953 Gbps
Upstream rate	1.244 Gbps	9.953 Gbps
Downstream wavelength	1490 nm	1577 nm
Upstream wavelength	1310 nm	1270 nm
Split ratio	1:32 / 1:64	1:32 / 1:64
Max reach	20 km (up to 60 km with extenders)	20 km
Encryption	AES-128 (downstream)	AES-256
Frame period	125 us	125 us

GTC Downstream Frame

The GTC (GPON Transmission Convergence) layer organizes data into 125 us frames. Downstream frames are broadcast to all ONTs; each ONT filters by its Alloc-ID and GEM Port-ID.

[Diagram: see online version]

PCBd (Physical Control Block downstream) Fields

Field	Size	Description
Psync	4 bytes	Fixed pattern `0xB6AB31E0` for frame synchronization
Ident	4 bytes	Superframe counter (30 bits) + FEC indicator (1 bit) + reserved

PLOAMd	13 bytes	Downstream PLOAM (Physical Layer OAM) message
BIP	1 byte	Bit-interleaved parity over previous frame (error monitoring)
Plend	4 bytes x 2	Payload length and BWmap length (transmitted twice for reliability)
BWmap	8 bytes x N	Bandwidth map: upstream timeslot allocations for each ONU

BWmap Entry

Each BWmap entry grants an ONU a specific upstream transmission window:

Field	Size	Description
Alloc-ID	12 bits	T-CONT allocation identifier
Flags	12 bits	PLOAMu flag, FEC, send DBRu
StartTime	16 bits	Start of upstream burst (in bytes, from frame start)
StopTime	16 bits	End of upstream burst
CRC	8 bits	CRC-8 over the entry

GTC Upstream Frame

Upstream uses TDMA: each ONU transmits in its assigned timeslot per the BWmap. Bursts from different ONUs are interleaved in the 125 us frame.

[Diagram: see online version]

GEM Header (GPON Encapsulation Method)

GEM encapsulates Ethernet frames, TDM traffic, and OMCI messages into GEM frames for transport over the PON:

[Diagram: see online version]

GEM Key Fields

Field	Size	Description
PLI	12 bits	Payload Length Indicator (0-4095 bytes)
Port-ID	12 bits	GEM port identifier -- maps to a service flow (up to 4096 ports)
PTI	3 bits	Payload Type Indicator
HEC	13 bits	Header Error Control (BCH code + parity -- corrects 2-bit errors)

PTI Values

PTI	Meaning
000	User data, not end of fragment
001	User data, end of fragment
010	OAM (OMCI)
100	GEM idle frame

GEM can fragment large Ethernet frames across multiple GEM frames, and multiple small frames can share a single GEM payload through multiplexing.

PLOAM Messages (Physical Layer OAM)

PLOAM messages carry control-plane signaling between OLT and ONUs. Each message is 13 bytes:

[Diagram: see online version]

Key PLOAM Messages

Message	Direction	Description
Upstream_Overhead	OLT -> ONU	Configures preamble, delimiter, burst profile
Assign_ONU-ID	OLT -> ONU	Assigns ONU-ID after serial number match
Ranging_Time	OLT -> ONU	Sets equalization delay for ONU time alignment
Serial_Number_Request	OLT -> all	Requests serial numbers during discovery
Serial_Number_ONU	ONU -> OLT	ONU responds with its serial number
Registration	OLT -> ONU	Requests password for authentication
Key_Update	ONU -> OLT	Sends new AES encryption key
Deactivate_ONU-ID	OLT -> ONU	Deactivates an ONU
Encrypted_Port-ID	OLT -> ONU	Enables/disables encryption on a GEM port
Physical_Equipment_Error	ONU -> OLT	Reports hardware failure

ONU Activation (Discovery and Ranging)

[Diagram: see online version]

DBA (Dynamic Bandwidth Allocation)

The OLT dynamically allocates upstream bandwidth to ONUs based on traffic demand. Each ONU has one or more T-CONTs (Transmission Containers) representing traffic queues:

T-CONT Types

Type	Priority	Behavior	Use Case
T-CONT 1	Highest	Fixed bandwidth -- guaranteed, always allocated	Voice (TDM), real-time
T-CONT 2	High	Assured bandwidth -- guaranteed rate, may burst	Video, premium data
T-CONT 3	Medium	Assured + non-assured -- guaranteed minimum, best-effort surplus	Business data
T-CONT 4	Low	Best-effort only -- no guarantee	Residential internet
T-CONT 5	Mixed	Combination of all types in one container	Mixed services

DBA Methods

Method	Mechanism	Latency
Status Reporting (SR-DBA)	ONU sends DBRu (buffer occupancy report) to OLT	Higher (report + grant cycle)
Traffic Monitoring (TM-DBA)	OLT infers demand from idle GEM frames	Lower (no report needed)
Hybrid	Combines SR-DBA and TM-DBA	Balanced

OMCI (ONT Management and Control Interface)

OMCI runs over a dedicated GEM port (typically Port-ID 65535) and manages ONT configuration:

Managed Entity	Purpose
GEM Port Network CTP	GEM port configuration
MAC Bridge Port Config	Bridge port mapping
VLAN Tagging Filter	VLAN filtering rules
T-CONT	Upstream bandwidth container
Traffic Descriptor	QoS profile (CIR/PIR)
IP Host Config	ONT management IP
PPTP Ethernet UNI	Physical Ethernet port

Extended VLAN Tagging	Complex VLAN operations (translate, add, remove)

OMCI uses a request/response model: the OLT sends Get/Set/Create/Delete operations on Managed Entities, and the ONT responds with results.

XGS-PON Differences

Feature	GPON (G.984)	XGS-PON (G.9807)
Downstream rate	2.488 Gbps	9.953 Gbps
Upstream rate	1.244 Gbps	9.953 Gbps
Downstream wavelength	1490 nm	1577 nm
Upstream wavelength	1310 nm	1270 nm
Encapsulation	GEM (5-byte header)	XGEM (variable, 8-byte header)
Framing	GTC	XGTC (larger BWmap, flexible burst profiles)
Encryption	AES-128	AES-256
FEC	RS(255,239) optional	RS(248,216) mandatory downstream
PLOAM	13 bytes	48 bytes (larger, more message types)
ONU activation	Serial number + password	+ mutual authentication (OMCI-based)

GPON Security

Mechanism	Direction	Description
AES-128 encryption (GPON)	Downstream	ONU generates key, sends to OLT via PLOAM
AES-256 encryption (XGS-PON)	Downstream	Enhanced encryption with key exchange
Password authentication	Upstream	ONU sends password during registration
Serial number verification	Both	Each ONU has unique vendor-assigned serial
Port-ID filtering	Downstream	ONUs only process GEM frames matching their ports

Note: GPON upstream traffic is inherently isolated by TDMA (each ONU can only transmit in its allocated window), but upstream is **not encrypted** by default. A compromised ONU could potentially sniff downstream broadcast traffic for other ONUs if encryption is disabled.

Wavelength Coexistence

GPON and XGS-PON can coexist on the same fiber using a Coexistence Element (CEx) or WDM filter:

Technology	Downstream	Upstream
GPON	1490 nm	1310 nm
XGS-PON	1577 nm	1270 nm
RF Video overlay	1550 nm	N/A

This allows operators to incrementally upgrade subscribers from GPON to XGS-PON without replacing the fiber or splitters.

PON Technology Comparison

Feature	GPON	XGS-PON	EPON (1G)	10G-EPON
Standard	ITU-T G.984	ITU-T G.9807	IEEE 802.3ah	IEEE 802.3av
DS / US rate	2.5G / 1.2G	10G / 10G	1G / 1G	10G / 10G
Encapsulation	GEM	XGEM	Native Ethernet	Native Ethernet
Max split	1:64	1:64	1:32	1:32
Frame period	125 us	125 us	Variable	Variable
DBA mechanism	BWmap + T-CONT	BWmap + T-CONT	MPCP GATE/REPORT	MPCP
Management	OMCI	OMCI	OAM (802.3ah)	OAM
Primary market	Worldwide	Worldwide	Asia (Japan, China)	Asia, cable

Standards

Document	Title
ITU-T G.984.1	GPON -- General characteristics
ITU-T G.984.2	GPON -- Physical media dependent layer
ITU-T G.984.3	GPON -- Transmission convergence layer (GTC)
ITU-T G.984.4	GPON -- ONT management and control interface (OMCI)
ITU-T G.9807.1	XGS-PON -- 10-Gigabit-capable symmetric PON
ITU-T G.988	ONU management and control interface (OMCI) specification
ITU-T G.987	XG-PON (10G asymmetric -- predecessor to XGS-PON)

See Also

- EPON -- IEEE-based passive optical network (Ethernet-native)
- 50G-PON / NG-PON2 -- next-generation PON technologies
- Ethernet -- carried as payload inside GEM frames
- PPP -- PPPoE often used over GPON for subscriber access

- TR-069 / TR-369 -- remote management of ONTs
- VLAN (802.1Q) -- VLAN tagging configured via OMCI

GSM -- Global System for Mobile Communications

Standard: 3GPP TS 05 series (GSM) | **Layer:** Full stack (Physical through Application) | **Wireshark filter:** `gsm_a` or `gsm_map` or `lapdm`

GSM is the second-generation (2G) digital cellular standard that replaced analog mobile networks in the 1990s and became the most widely deployed mobile technology in history (billions of subscribers). It defines the radio interface (TDMA-based), core network architecture (MSC, HLR, VLR), signaling (MAP over SS7), and security (A5 encryption, SIM-based authentication). While voice is being sunset, GSM's architecture and concepts (SIM cards, IMSI, roaming) carry through to LTE and 5G.

Architecture

[Diagram: see online version]

Network Elements

Element	Full Name	Role
MS	Mobile Station	Handset + SIM
BTS	Base Transceiver Station	Radio tower
BSC	Base Station Controller	Manages multiple BTS, handover
MSC	Mobile Switching Center	Call routing, auth, handover
HLR	Home Location Register	Subscriber database (master)
VLR	Visitor Location Register	Temporary subscriber data (current MSC)
AuC	Authentication Center	Generates auth triplets
EIR	Equipment Identity Register	IMEI blacklist
SMSC	Short Message Service Center	SMS store-and-forward
SGSN	Serving GPRS Support Node	Packet data (2.5G GPRS)
GGSN	Gateway GPRS Support Node	Internet gateway (GPRS)

Radio Interface (Um)

Parameter	Value

Frequency bands	850, 900, 1800, 1900 MHz
Access method	TDMA (Time Division Multiple Access)
Timeslots per carrier	8
Channel bandwidth	200 kHz
Modulation	GMSK (Gaussian Minimum Shift Keying)
Frame duration	4.615 ms
Data rate (voice)	13 kbps (full rate), 5.6 kbps (half rate)
Data rate (GPRS)	Up to 171.2 kbps (8 timeslots, CS-4)
Data rate (EDGE)	Up to 473.6 kbps (8 timeslots, MCS-9)

TDMA Frame

[Diagram: see online version]

Each timeslot = 577 µs. 8 timeslots = 1 TDMA frame (4.615 ms). Each timeslot carries one normal burst of 148 bits.

Channel Types

Channel	Direction	Purpose
TCH	Both	Traffic Channel (voice/data)
BCCH	Downlink	Broadcast Control (cell info, frequency list)
CCCH (PCH, AGCH, RACH)	Both	Common Control (paging, access grant, random access)
SDCCH	Both	Standalone Dedicated Control (SMS, location update)
SACCH	Both	Slow Associated Control (measurements, timing advance)
FACCH	Both	Fast Associated Control (handover commands, replaces TCH)

Identifiers

Identifier	Full Name	Description
IMSI	International Mobile Subscriber Identity	Unique subscriber ID on SIM (MCC+MNC+MSIN)
TMSI	Temporary Mobile Subscriber Identity	Temporary ID assigned by VLR (privacy)
IMEI	International Mobile Equipment Identity	Unique handset ID (15 digits)
MSISDN	Mobile Subscriber ISDN Number	Phone number (E.164)

LAI	Location Area Identity	MCC+MNC+LAC (tracks subscriber location)
Cell ID	Cell Identifier	Individual cell within a location area

IMSI Structure

[Diagram: see online version]

Field	Description	Example
MCC	Mobile Country Code	310 (USA), 234 (UK)
MNC	Mobile Network Code	410 (AT&T), 15 (Vodafone)
MSIN	Mobile Subscriber Identification Number	Unique within the operator

Authentication

GSM uses challenge-response authentication based on a shared secret (Ki) stored on the SIM and in the AuC:

[Diagram: see online version]

Security Algorithms

Algorithm	Purpose	Notes
A3	Authentication (SRES generation)	On SIM card
A5/1	Encryption (strong)	Used in most networks
A5/2	Encryption (weak, export version)	**Broken -- deprecated**
A5/3 (KASUMI)	Encryption (3G-strength for 2G)	Upgrade path
A8	Key generation (Kc from Ki+RAND)	On SIM card

Call Setup

[Diagram: see online version]

GSM Evolution

Generation	Technology	Data Rate	Standard
2G GSM	Circuit-switched voice + SMS	9.6 kbps CSD	3GPP Release 97/98
2.5G GPRS	Packet-switched data overlay	171 kbps	3GPP Release 97
2.75G EDGE	Enhanced GPRS (8PSK modulation)	474 kbps	3GPP Release 4

3G UMTS	W-CDMA, new radio interface	384 kbps - 2 Mbps	3GPP Release 99
3.5G HSPA	High-Speed Packet Access	14.4 Mbps DL	3GPP Release 5/6
4G LTE	OFDMA, all-IP	150 Mbps+	3GPP Release 8

Standards

Document	Title
3GPP TS 05.01	Physical layer on the radio path
3GPP TS 04.08	Mobile radio interface layer 3 (call control, mobility)
3GPP TS 03.20	Security-related network functions
3GPP TS 09.02	MAP specification (SS7 signaling)

See Also

- LTE -- 4G successor
- 5G NR -- 5G successor
- SS7 -- signaling network GSM uses
- ISDN -- MSC uses ISUP for PSTN interworking
- T1 / E1 -- backhaul between BTS and BSC

ISDN -- Integrated Services Digital Network

Standard: ITU-T I-series / Q.920-Q.931 | **Layer:** Physical / Data Link / Network (Layers 1-3) | **Wireshark filter:** `isdn` or `q931`

ISDN is a set of standards for digital transmission of voice, data, and signaling over the traditional telephone network. It replaced analog signaling with digital end-to-end connectivity, offering faster call setup, caller ID, and multiple simultaneous channels on a single line. ISDN defines two service levels: Basic Rate Interface (BRI) for homes/small offices and Primary Rate Interface (PRI) for enterprises/carriers. While largely superseded by VoIP, ISDN PRI remains widely deployed in enterprise PBX systems and as a carrier interconnect.

Interface Types

Interface	Channels	Total Bandwidth	Usage
BRI (2B+D)	2 x 64 kbps B + 1 x 16 kbps D	144 kbps (192 kbps gross)	Residential, small office
PRI (T1)	23 x 64 kbps B + 1 x 64 kbps D	1.544 Mbps	Enterprise (North America/Japan)

PRI (E1)	30 x 64 kbps B + 1 x 64 kbps D	2.048 Mbps	Enterprise (international)

Channel Types

Channel	Rate	Purpose
B (Bearer)	64 kbps	Voice or data payload
D (Delta)	16 kbps (BRI) / 64 kbps (PRI)	Signaling (Q.931) and packet data
H0	384 kbps	6 x B channels bonded
H11	1536 kbps	24 x B channels (full T1)
H12	1920 kbps	30 x B channels (full E1)

Protocol Stack

[Diagram: see online version]

LAPD Frame (Q.921 -- Layer 2)

[Diagram: see online version]

Field	Size	Description
Flag	8 bits	HDLC frame delimiter `0x7E`
SAPI	6 bits	Service Access Point Identifier
C/R	1 bit	Command/Response
TEI	7 bits	Terminal Endpoint Identifier (0-63 non-auto, 64-126 auto, 127 broadcast)
Control	8-16 bits	Frame type (I, S, or U frame)
Information	Variable	Q.931 message payload
FCS	16 bits	Frame Check Sequence (CRC-16)

SAPI Values

SAPI	Purpose
0	Q.931 call control signaling
16	X.25 packet data
63	LAPD management

Q.931 Message (Layer 3)

[Diagram: see online version]

Field	Size	Description
Protocol Discriminator	8 bits	Always 0x08 for Q.931
Call Reference Length	8 bits	Length of call reference value (1-2 bytes)
Call Reference Value	8-16 bits	Identifies the call this message relates to
Message Type	8 bits	The Q.931 message type
Information Elements	Variable	Parameters for the message

Q.931 Message Types

Type	Name	Direction	Description
0x05	SETUP	Either	Initiate a call
0x07	CONNECT	Called -> Calling	Call answered
0x0F	CONNECT ACK	Calling -> Called	Acknowledge answer
0x02	CALL PROCEEDING	Network -> User	Call is being routed
0x01	ALERTING	Called -> Calling	Called party is ringing
0x45	DISCONNECT	Either	Initiate call clearing
0x4D	RELEASE	Either	Release the call reference
0x5A	RELEASE COMPLETE	Either	Call reference released
0x03	PROGRESS	Network -> User	In-band information available
0x20	STATUS ENQUIRY	Either	Request status
0x7D	STATUS	Either	Report current call state

Call Setup Flow

[Diagram: see online version]

Key Information Elements

IE	Name	Description
0x04	Bearer Capability	Type of service (speech, data, video)
0x14	Date/Time	Current date and time
0x18	Channel Identification	B-channel to use
0x1C	Facility	Supplementary service invocation
0x1E	Progress Indicator	In-band information status
0x28	Display	Text to display (caller name, etc.)

0x6C	Calling Party Number	Caller ID number and type
0x70	Called Party Number	Destination number
0x74	Redirecting Number	Original called number (on forwarding)

ISDN Reference Points

Point	Between	Description
U	NT1 <-> Network	Two-wire subscriber line (North America)
S/T	TE <-> NT	Four-wire bus (up to 8 terminals on BRI)
R	TE2 <-> TA	Non-ISDN device to terminal adapter

[Diagram: see online version]

Standards

Document	Title
ITU-T Q.921	LAPD -- ISDN Data Link Layer
ITU-T Q.931	ISDN Call Control (Layer 3)
ITU-T I.430	BRI Layer 1 specification
ITU-T I.431	PRI Layer 1 specification
ITU-T Q.932	Generic procedures for supplementary services

See Also

- SS7 -- carrier signaling network (ISUP handles ISDN calls between switches)
- T1 -- physical carrier for PRI in North America
- E1 -- physical carrier for PRI internationally
- SIP -- VoIP successor to ISDN signaling

LTE -- Long Term Evolution -- 4G

Standard: 3GPP TS 36 series | **Layer:** Full stack | **Wireshark filter:** `lte_rrc` or `nas-eps` or `s1ap` or `diameter`

LTE is the fourth-generation (4G) mobile broadband standard, designed as an all-IP network with no circuit-switched voice (VoLTE uses SIP/IMS for voice over the data path). It uses OFDMA for the downlink and SC-FDMA for the uplink, achieving peak rates of 300 Mbps DL / 75 Mbps UL (LTE) and 3 Gbps DL (LTE-Advanced Pro). LTE's flat

architecture eliminates legacy BSC/RNC controllers, connecting the eNodeB directly to the core network (EPC).

Architecture (EPC -- Evolved Packet Core)

[Diagram: see online version]

Core Network Elements

Element	Full Name	Role
eNodeB	Evolved NodeB	Base station (radio + some control)
MME	Mobility Management Entity	Signaling: attach, handover, paging, security
S-GW	Serving Gateway	User plane anchor for handover between eNodeBs
P-GW	PDN Gateway	Internet gateway, IP address allocation, policy
HSS	Home Subscriber Server	Subscriber database (replaces HLR)
PCRF	Policy and Charging Rules Function	QoS policies, charging rules

Interfaces

Interface	Between	Protocol	Purpose
Uu	UE <-> eNodeB	LTE air interface	Radio access
S1-MME	eNodeB <-> MME	S1AP (SCTP)	Control plane signaling
S1-U	eNodeB <-> S-GW	GTP-U (UDP)	User data tunneling
S5/S8	S-GW <-> P-GW	GTP (C+U)	Inter-gateway tunneling
S6a	MME <-> HSS	Diameter	Authentication, subscription data
S11	MME <-> S-GW	GTPv2-C	Session management
X2	eNodeB <-> eNodeB	X2AP (SCTP)	Handover, load balancing
Gx	P-GW <-> PCRF	Diameter	Policy and charging control
SGi	P-GW <-> Internet	IP	External data network

Radio Interface

Parameter	Value
Downlink access	OFDMA
Uplink access	SC-FDMA
Frequency bands	700 MHz - 3.5 GHz (many bands)

Channel bandwidth	1.4, 3, 5, 10, 15, 20 MHz
Subcarrier spacing	15 kHz
Modulation	QPSK, 16-QAM, 64-QAM (256-QAM in Cat 11+)
MIMO	2x2, 4x4 (up to 8x8 in LTE-A)
Peak DL rate	300 Mbps (Cat 5), 3 Gbps (Cat 20, LTE-A Pro)
Peak UL rate	75 Mbps (Cat 5), 1.5 Gbps (Cat 20)
Latency	~10-20 ms (user plane)
Frame duration	10 ms (20 slots of 0.5 ms)

Resource Grid

[Diagram: see online version]

Each subframe = 2 slots = 1 ms. One radio frame = 10 subframes = 10 ms.

A **Resource Block** = 12 subcarriers x 1 slot (0.5 ms) = 84 resource elements (normal CP).

Protocol Stack

[Diagram: see online version]

Layer Functions

Layer	Function
NAS (Non-Access Stratum)	UE <-> MME: attach, authentication, bearer setup, tracking area update
RRC (Radio Resource Control)	UE <-> eNodeB: connection setup, measurement config, handover, RRC states
PDCP	Header compression (ROHC), ciphering, integrity (control plane)
RLC	Segmentation/reassembly, ARQ (AM mode), reordering
MAC	Scheduling (uplink grants, downlink assignments), HARQ, multiplexing
PHY	Modulation, coding, MIMO, OFDMA/SC-FDMA

Attach Procedure

[Diagram: see online version]

Security

Algorithm	Purpose	Standard
AES (EEA2)	User/control plane encryption	Default
SNOW 3G (EEA1)	User/control plane encryption	Alternative
ZUC (EEA3)	User/control plane encryption	Chinese standard
AES-CMAC (EIA2)	Integrity protection	Default
KASME	Root key derived from AKA	Per-session

Key Hierarchy

[Diagram: see online version]

LTE Categories

Category	Max DL	Max UL	MIMO	Year
Cat 1	10 Mbps	5 Mbps	1x1	2009
Cat 4	150 Mbps	50 Mbps	2x2	2009
Cat 6	300 Mbps	50 Mbps	2x2 CA	2013
Cat 9	450 Mbps	50 Mbps	3CC CA	2014
Cat 12	600 Mbps	100 Mbps	4x4 MIMO	2015
Cat 16	1 Gbps	150 Mbps	4x4, 3CC	2016
Cat 20	2 Gbps	316 Mbps	8x8, 5CC	2017
Cat-M1	1 Mbps	1 Mbps	1x1	IoT (eMTC)
Cat-NB1	0.026 Mbps	0.066 Mbps	1x1	IoT (NB-IoT)

Standards

Document	Title
3GPP TS 36.300	E-UTRA / E-UTRAN overall description
3GPP TS 36.331	RRC Protocol
3GPP TS 24.301	NAS Protocol (EMM + ESM)
3GPP TS 36.211	Physical channels and modulation
3GPP TS 23.401	EPS architecture

See Also

- GSM -- 2G predecessor
- 5G NR -- 5G successor

- GTP -- user plane tunneling
- Diameter -- authentication and policy signaling
- SIP -- VoLTE voice calls
- SS7 -- legacy signaling (interworking via SGs)

5G NAS -- Non-Access Stratum

Standard: 3GPP TS 24.501 | **Layer:** Application (NAS signaling) | **Wireshark filter:** `nas-5gs`

5G NAS is the signaling protocol between the UE and the AMF (Access and Mobility Management Function) for mobility management and session management in 5G networks. It operates on the N1 interface -- a logical connection where the gNB acts as a transparent relay, encapsulating NAS messages inside NGAP on the N2 interface. 5G NAS consists of two sub-protocols: 5GMM (5G Mobility Management) for registration, authentication, and security, and 5GSM (5G Session Management) for PDU session lifecycle. It replaces EMM/ESM from 4G LTE.

Protocol Stack

[Diagram: see online version]

The gNB does not interpret NAS messages -- it encapsulates them as opaque NAS PDU IEs within NGAP InitialUEMessage, DownlinkNASTransport, and UplinkNASTransport.

NAS Message Header (Plain)

[Diagram: see online version]

Key Fields

Field	Size	Description
Extended Protocol Discriminator	8 bits	0x7E = 5GMM, 0x2E = 5GSM
Security Header Type	4 bits	0 = plain, 1 = integrity protected, 2 = integrity + ciphered, 3 = integrity + new security context, 4 = integrity + ciphered + new security context
Message Type	8 bits	Identifies the specific NAS message

Extended Protocol Discriminator

Value	Protocol
0x7E	5GS Mobility Management (5GMM)

0x2E	5GS Session Management (5GSM)

Security Protected NAS Message

After security is established, NAS messages are integrity-protected and optionally ciphered:

[Diagram: see online version]

Field	Size	Description
MAC	32 bits	Integrity check value computed over Sequence Number + NAS message
Sequence Number	8 bits	NAS COUNT (mod 256) for replay protection
Plain NAS Message	Variable	The actual NAS message (encrypted with NEA if ciphered)

Security Algorithms

Algorithm Type	ID 0	ID 1	ID 2	ID 3
NIA (integrity)	Null	128-NIA1 (SNOW)	128-NIA2 (AES)	128-NIA3 (ZUC)
NEA (ciphering)	Null (NEA0)	128-NEA1 (SNOW)	128-NEA2 (AES)	128-NEA3 (ZUC)

NAS vs AS (Access Stratum)

Aspect	NAS (Non-Access Stratum)	AS (Access Stratum)
Endpoints	UE <-> AMF	UE <-> gNB
Protocol	5G NAS (TS 24.501)	NR RRC (TS 38.331)
Scope	Mobility management, session management	Radio resource control, measurements
Security	NAS encryption + integrity (NEA/NIA)	PDCP encryption + integrity (AS keys)
Transparency	gNB cannot read NAS	gNB terminates AS

5GMM Messages (Mobility Management)

Registration

Message Type	Value	Direction	Purpose
Registration Request	0x41	UE -> AMF	Request registration (initial, mobility, periodic, emergency)

Registration Accept	0x42	AMF -> UE	Registration successful (5G-GUTI, allowed NSSAI, TAI list)
Registration Complete	0x43	UE -> AMF	Acknowledge registration accept
Registration Reject	0x44	AMF -> UE	Registration denied (with 5GMM cause)

Authentication

Message Type	Value	Direction	Purpose
Authentication Request	0x56	AMF -> UE	Send RAND + AUTN (5G-AKA) or EAP message
Authentication Response	0x57	UE -> AMF	Send RES* (5G-AKA) or EAP response
Authentication Failure	0x59	UE -> AMF	Authentication failed (sync failure, MAC failure)
Authentication Result	0x5A	AMF -> UE	EAP success/failure
Authentication Reject	0x58	AMF -> UE	Reject (invalid credentials)

Security

Message Type	Value	Direction	Purpose
Security Mode Command	0x5D	AMF -> UE	Activate NAS security (selected algorithms, key set)
Security Mode Complete	0x5E	UE -> AMF	Confirm security activation (carries encrypted Registration data)
Security Mode Reject	0x5F	UE -> AMF	UE cannot comply with security request

Other 5GMM

Message Type	Value	Direction	Purpose
Deregistration Request	0x45	Either	Detach from network
Deregistration Accept	0x46	Either	Confirm detach
Service Request	0x4C	UE -> AMF	Request service (idle to connected, data, signaling)
Service Accept	0x4E	AMF -> UE	Service granted
Service Reject	0x4D	AMF -> UE	Service denied (with cause)
Configuration Update Command	0x54	AMF -> UE	Update GUTI, TAI list, NSSAI, LADN

Configuration Update Complete	0x55	UE -> AMF	Acknowledge configuration update
Identity Request	0x5B	AMF -> UE	Request SUCI, SUPI, 5G-GUTI, IMEI
Identity Response	0x5C	UE -> AMF	Provide requested identity

5GSM Messages (Session Management)

Message Type	Value	Direction	Purpose
PDU Session Establishment Request	0xC1	UE -> AMF -> SMF	Request new PDU session (PDU type, SSC mode)
PDU Session Establishment Accept	0xC2	SMF -> AMF -> UE	Session created (QoS rules, IP address, S-NSSAI)
PDU Session Establishment Reject	0xC3	SMF -> AMF -> UE	Session denied (with 5GSM cause)
PDU Session Modification Request	0xC5	Either	Modify existing session (QoS, rules)
PDU Session Modification Complete	0xCA	UE -> SMF	Acknowledge modification
PDU Session Modification Reject	0xC7	Either	Modification denied
PDU Session Release Request	0xC9	UE -> SMF	Request session teardown
PDU Session Release Command	0xCB	SMF -> UE	Network-initiated release
PDU Session Release Complete	0xCC	UE -> SMF	Confirm release
PDU Session Authentication Command	0xC5	SMF -> UE	Secondary authentication for DN
PDU Session Authentication Complete	0xC6	UE -> SMF	Auth response for DN

Registration Types

Type	Value	Description
Initial Registration	1	First registration or after deregistration
Mobility Registration Update	2	Tracking Area Update equivalent (UE moved to new TA)
Periodic Registration Update	3	Periodic keepalive (like periodic TAU in LTE)
Emergency Registration	4	Emergency services without full credentials

5G Identifiers

5G-GUTI (Globally Unique Temporary Identifier)

[Diagram: see online version]

Field	Size	Description
MCC + MNC	5-6 digits	PLMN identity (country + operator)
AMF Region ID	8 bits	Identifies a region of AMFs
AMF Set ID	10 bits	Identifies a set of AMFs within the region
AMF Pointer	6 bits	Identifies a specific AMF within the set
5G-TMSI	32 bits	Temporary subscriber identity assigned by AMF

The combination of AMF Region ID + AMF Set ID + AMF Pointer forms the GUAMI (Globally Unique AMF Identifier).

Authentication Methods

Method	Description
5G-AKA	Native 5G authentication (RAND, AUTN, RES*) using USIM
EAP-AKA'	Extensible Authentication Protocol variant (also USIM-based)
EAP-TLS	Certificate-based authentication (IoT, private networks)

Initial Registration Flow

[Diagram: see online version]

PDU Session Establishment Flow

[Diagram: see online version]

5G NAS vs 4G NAS Comparison

Feature	5G NAS (5GMM/5GSM)	4G NAS (EMM/ESM)
Standard	3GPP TS 24.501	3GPP TS 24.301
Core network endpoint	AMF	MME
Protocol discriminator	0x7E (5GMM), 0x2E (5GSM)	0x07 (EMM), 0x02 (ESM)
Registration	Registration Request/Accept	Attach Request/Accept

Mobility update	Mobility Registration Update	Tracking Area Update
Session	PDU Session Establishment	PDN Connectivity + Activate Default Bearer
Session types	IPv4, IPv6, IPv4v6, Ethernet, Unstructured	IPv4, IPv6, IPv4v6
Identity	SUCI (concealed SUPI), 5G-GUTI	IMSI (plain), GUTI
Privacy	SUPI concealment (ECIES encryption)	IMSI sent in clear (initial attach)
Authentication	5G-AKA, EAP-AKA', EAP-TLS	EPS-AKA
Security algorithms	NEA0-3, NIA0-3	EEA0-3, EIA0-3
Network slicing	S-NSSAI in registration	Not supported
QoS model	Flow-based (QoS rules, QFI)	Bearer-based (EPS bearer, QCI)

Standards

Document	Title
3GPP TS 24.501	5GMM and 5GSM NAS specification
3GPP TS 33.501	5G security architecture and procedures
3GPP TS 24.301	4G NAS (EMM/ESM) -- predecessor
3GPP TS 23.003	Numbering, addressing, identification (SUPI, GUTI)
3GPP TS 23.501	5G system architecture

See Also

- NGAP -- N2 protocol that carries NAS messages between gNB and AMF
- PFCP -- N4 user plane control (SMF programs UPF after PDU session setup)
- GTP -- user plane tunneling for PDU sessions
- 5G NR -- radio interface (Access Stratum)
- LTE -- 4G predecessor (uses EMM/ESM NAS)
- 5G SBI -- service-based interfaces between core NFs (AMF, AUSF, UDM, SMF)

NGAP -- Next Generation Application Protocol

Standard: 3GPP TS 38.413 | **Layer:** Application (signaling) | **Wireshark filter:** ngap

NGAP is the control plane protocol for the N2 interface between the gNB (5G base station) and the AMF (Access and Mobility Management Function) in the 5G Core. It replaces S1AP from 4G LTE and carries UE-associated and non-UE-associated signaling, including NAS message transport, UE context management, PDU session resource

management, handover, and paging. NGAP runs over SCTP (multi-homed, multi-stream) and uses ASN.1 with Packed Encoding Rules (PER) for message encoding.

5G RAN Architecture (N2 Interface)

[Diagram: see online version]

NGAP PDU Structure

All NGAP messages are encoded as a single NGAP-PDU, which is one of three choices:

[Diagram: see online version]

PDU Types

PDU Type	Description
InitiatingMessage	Starts a procedure (request, indication, or command)
SuccessfulOutcome	Positive response to an InitiatingMessage
UnsuccessfulOutcome	Negative response (with cause) to an InitiatingMessage

Each message contains a Procedure Code identifying the operation, a Criticality flag (reject, ignore, notify), and a sequence of Information Elements (IEs) encoded in ASN.1 PER.

Key Fields

Field	Size	Description
Procedure Code	8 bits	Identifies the NGAP procedure (e.g., 21 = NGSetup, 14 = InitialContextSetup)
Criticality	2 bits	reject (0), ignore (1), notify (2) -- how receiver handles unknown IEs
AMF-UE-NGAP-ID	40 bits	AMF-assigned UE identifier for this NGAP association
RAN-UE-NGAP-ID	32 bits	gNB-assigned UE identifier for this NGAP association

UE Context Identification

The pair (AMF-UE-NGAP-ID, RAN-UE-NGAP-ID) uniquely identifies a UE context on the N2 interface. The RAN side assigns the RAN-UE-NGAP-ID in the first uplink message (e.g., InitialUEMessage), and the AMF assigns the AMF-UE-NGAP-ID in its first response (e.g., InitialContextSetupRequest).

NGAP Procedures

Interface Management (Non-UE-Associated)

Procedure	Direction	Purpose
NG Setup	gNB -> AMF	Establish N2 association (exchange capabilities, PLMN, TAIs)
RAN Configuration Update	gNB -> AMF	gNB reports changed configuration (cells, TAIs)
AMF Configuration Update	AMF -> gNB	AMF reports changed GUAMI, PLMN list
NG Reset	Either	Reset all or selected UE contexts on N2
Error Indication	Either	Report protocol errors
Overload Start/Stop	AMF -> gNB	AMF signals load control
AMF Status Indication	AMF -> gNB	AMF availability per GUAMI

UE Context Management

Procedure	Direction	Purpose
Initial UE Message	gNB -> AMF	First NAS message from UE (carries NAS PDU + establishment cause)
Downlink NAS Transport	AMF -> gNB	Deliver NAS message to UE
Uplink NAS Transport	gNB -> AMF	Forward NAS message from UE
Initial Context Setup	AMF -> gNB	Establish UE context (security, PDU sessions, UE capabilities)
UE Context Release (Command/Complete)	AMF -> gNB	Tear down UE context (idle, detach, failure)
UE Context Modification	AMF -> gNB	Update existing UE context

PDU Session Resource Management

Procedure	Direction	Purpose
PDU Session Resource Setup	AMF -> gNB	Establish PDU session resources (QoS flows, GTP-U tunnels)
PDU Session Resource Modify	AMF -> gNB	Modify existing PDU session QoS/tunnel parameters
PDU Session Resource Release	AMF -> gNB	Release PDU session resources
PDU Session Resource Notify	gNB -> AMF	gNB notifies of PDU session status changes

Mobility (Handover)

Procedure	Direction	Purpose

Handover Required	gNB -> AMF	Source gNB requests handover
Handover Command	AMF -> gNB	AMF instructs source gNB with target cell info
Handover Request	AMF -> gNB	AMF requests target gNB to prepare resources
Handover Request Acknowledge	gNB -> AMF	Target gNB confirms resources prepared
Handover Notify	gNB -> AMF	Target gNB confirms UE arrival
Handover Cancel	gNB -> AMF	Source gNB cancels in-progress handover
Path Switch Request	gNB -> AMF	Xn handover -- new gNB requests path update
Paging	AMF -> gNB	Page idle UE in a Tracking Area

Initial Registration Flow (UE Attach)

[Diagram: see online version]

N2 Handover (Inter-gNB via AMF)

[Diagram: see online version]

NGAP vs S1AP Comparison

Feature	NGAP (5G)	S1AP (4G LTE)
Standard	3GPP TS 38.413	3GPP TS 36.413
Interface	N2 (gNB - AMF)	S1-MME (eNodeB - MME)
Transport	SCTP	SCTP
Encoding	ASN.1 PER	ASN.1 PER
UE ID (network side)	AMF-UE-NGAP-ID (40 bits)	MME-UE-S1AP-ID (32 bits)
UE ID (RAN side)	RAN-UE-NGAP-ID (32 bits)	eNB-UE-S1AP-ID (24 bits)
Session concept	PDU Session (multiple QoS flows)	EPS Bearer (one QoS per bearer)
QoS model	Flow-based (QFI per flow)	Bearer-based (QCI per bearer)
NAS transport	Carries 5G NAS (5GMM/5GSM)	Carries EPS NAS (EMM/ESM)
Network slicing	Supported (S-NSSAI in messages)	Not supported
Handover	N2 handover + Xn handover	S1 handover + X2 handover
AMF selection	Based on GUAMI, S-NSSAI	Based on GUMMEI

Standards

Document	Title
3GPP TS 38.413	NGAP specification
3GPP TS 38.401	NG-RAN architecture
3GPP TS 38.300	NR and NG-RAN overall description
3GPP TS 23.501	5G system architecture
3GPP TS 36.413	S1AP (4G predecessor)

See Also

- NAS 5G -- 5G Non-Access Stratum (carried inside NGAP)
- PFCP -- Packet Forwarding Control Protocol (N4 user plane control)
- GTP -- GPRS Tunneling Protocol (user plane tunnels on N3/N9)
- SCTP -- transport for NGAP
- 5G NR -- 5G New Radio (air interface)
- LTE -- 4G predecessor (uses S1AP)
- Diameter -- AAA protocol (used alongside NGAP in 4G, replaced by SBI in 5G)

PFCP -- Packet Forwarding Control Protocol

Standard: 3GPP TS 29.244 | **Layer:** Application (user plane control) | **Wireshark filter:** `pfcp`

PFCP is the control protocol for the N4 interface between the SMF (Session Management Function) and UPF (User Plane Function) in the 5G Core. It separates user plane forwarding from control plane logic, allowing the SMF to install packet detection, forwarding, QoS, and usage reporting rules on the UPF. PFCP also applies to 4G EPC with CUPS (Control and User Plane Separation), operating between SGW-C/SGW-U and PGW-C/PGW-U. It runs over UDP port 8805.

5G User Plane Architecture

[Diagram: see online version]

PFCP Header

[Diagram: see online version]

Key Fields

Field	Size	Description
Version	3 bits	PFCP version (currently 1)
MP	1 bit	Message Priority present (for node-related messages)
S	1 bit	SEID flag -- 1 = SEID field present (session messages)
Message Type	8 bits	Identifies the PFCP message
Message Length	16 bits	Length of payload after this field (excludes first 4 bytes)
SEID	64 bits	Session Endpoint Identifier -- identifies a PFCP session
Sequence Number	24 bits	Matches requests to responses

SEID

The SEID (Session Endpoint Identifier) uniquely identifies a PFCP session at each endpoint (SMF and UPF). Each side assigns its own SEID during Session Establishment, similar to how GTP uses TEIDs. Node-level messages (heartbeat, association) do not carry a SEID (S=0).

Message Types

Node-Level Messages (S=0, no SEID)

Type	Name	Direction	Purpose
1	Heartbeat Request	Either	Path keepalive
2	Heartbeat Response	Either	Keepalive reply (includes Recovery Timestamp)
5	Association Setup Request	Either	Establish PFCP association (capabilities, features)
6	Association Setup Response	Either	Confirm association
7	Association Update Request	Either	Update association parameters
8	Association Update Response	Either	Confirm update
9	Association Release Request	Either	Tear down association
10	Association Release Response	Either	Confirm release
12	Node Report Request	UPF -> SMF	UPF reports node-level events (user plane path failure)
13	Node Report Response	SMF -> UPF	Acknowledge node report

Session-Level Messages (S=1, with SEID)

Type	Name	Direction	Purpose
50	Session Establishment Request	SMF -> UPF	Create a new PFCP session (install PDRs, FARs, QERs, URRs)
51	Session Establishment Response	UPF -> SMF	Confirm with F-SEID, created resources
52	Session Modification Request	SMF -> UPF	Update rules in an existing session
53	Session Modification Response	UPF -> SMF	Confirm modification
54	Session Deletion Request	SMF -> UPF	Delete a PFCP session (remove all rules)
55	Session Deletion Response	UPF -> SMF	Confirm deletion with final usage reports
56	Session Report Request	UPF -> SMF	UPF reports events (usage, DL data notification)
57	Session Report Response	SMF -> UPF	Acknowledge report

PFCP Rules (Information Elements)

The core of PFCP is the rule model. The SMF installs rules on the UPF that define how to detect, forward, rate-limit, and measure user plane packets:

[Diagram: see online version]

PDR (Packet Detection Rule)

IE	Description
PDR ID	Unique identifier for this rule
Precedence	Priority (lower = higher priority, evaluated first)
PDI (Packet Detection Information)	Match criteria for incoming packets
-- Source Interface	Core, Access, CP-function, or SGi-LAN
-- UE IP Address	Subscriber IP to match
-- GTP-U TEID	Tunnel endpoint to match (for N3/N9 encapsulated traffic)
-- SDF Filter	5-tuple filter (src/dst IP, ports, protocol)
-- QFI	QoS Flow Identifier to match
FAR ID	Associated forwarding action
QER ID(s)	Associated QoS enforcement rules
URR ID(s)	Associated usage reporting rules

FAR (Forwarding Action Rule)

IE	Description
FAR ID	Unique identifier
Apply Action	Forward, Drop, Buffer, Duplicate, Notify CP
Forwarding Parameters	Destination interface, outer header creation
-- Destination Interface	Access (toward gNB), Core (toward DN), CP-function
-- Outer Header Creation	GTP-U encapsulation (TEID + peer IP for tunneling)
-- Redirect Information	HTTP redirect or forwarding to another destination

QER (QoS Enforcement Rule)

IE	Description
QER ID	Unique identifier
Gate Status	UL gate (open/closed), DL gate (open/closed)
MBR (Maximum Bit Rate)	UL MBR, DL MBR
GBR (Guaranteed Bit Rate)	UL GBR, DL GBR
QFI	QoS Flow Identifier to mark on forwarded packets

URR (Usage Reporting Rule)

IE	Description
URR ID	Unique identifier
Measurement Method	Volume, Duration, Event
Reporting Triggers	Periodic, Volume threshold, Time threshold, Start/Stop of traffic
Measurement Period	Timer for periodic reporting
Volume Threshold	Byte count trigger

PDU Session Establishment (PFCP Session Setup)

[Diagram: see online version]

Usage Reporting

[Diagram: see online version]

PFCP vs GTP-C Comparison

Feature	PFCP (5G / 4G CUPS)	GTP-C v2 (4G LTE)
Standard	3GPP TS 29.244	3GPP TS 29.274
Interface	N4 (SMF - UPF)	S11 (MME - SGW), S5 (SGW - PGW)
Transport	UDP 8805	UDP 2123
Session ID	SEID (64 bits)	TEID (32 bits)
Architecture	Separated control/user plane	Combined control/user in SGW, PGW
Rule model	PDR/FAR/QER/URR/BAR	Bearer-based QoS (EPS bearer)
QoS granularity	Per-flow (QFI)	Per-bearer (QCI)
Usage reporting	Built-in (URR)	Separate Gy/Gz Diameter
Scope	User plane rule programming	Session/bearer management
Encoding	Grouped TLV IEs	Grouped TLV IEs

Standards

Document	Title
3GPP TS 29.244	PFCP specification (N4 interface)
3GPP TS 23.501	5G system architecture
3GPP TS 23.502	5G procedures
3GPP TS 29.281	GTP-U (user plane tunneling controlled by PFCP)
3GPP TS 23.214	CUPS architecture (4G control/user plane separation)

See Also

- GTP -- user plane tunneling (N3/N9) controlled by PFCP
- NGAP -- N2 signaling (carries PDU session setup to gNB)
- NAS 5G -- UE-AMF signaling that triggers PFCP sessions
- 5G SBI -- service-based interfaces between core NFs
- Diameter -- 4G AAA/charging (partially replaced by PFCP usage reporting)
- SCTP -- transport for NGAP (PFCP itself uses UDP)

50G-PON / NG-PON2 -- Next-Generation PON

Standard: ITU-T G.9804 | **Layer:** Physical + Data Link (Layers 1-2) | **Wireshark filter:** N/A (emerging technology)

50G-PON and NG-PON2 represent the next generation of passive optical networking beyond XGS-PON. 50G-PON (ITU-T G.9804) targets 50 Gbps downstream / 25 Gbps upstream on a single wavelength, using advanced modulation (NRZ or PAM4) and coexisting with existing GPON and XGS-PON deployments on the same fiber. NG-PON2 (ITU-T G.989) takes a different approach, combining multiple 10G wavelength pairs (TWDM) for 40-80 Gbps aggregate capacity with wavelength-tunable ONUs. While NG-PON2 saw limited deployment due to tunable optic costs, 50G-PON has emerged as the industry's chosen path forward.

50G-PON (ITU-T G.9804)

Overview

50G-PON is designed as a smooth evolutionary step from XGS-PON, reusing the same ODN (Optical Distribution Network) infrastructure -- fiber, splitters, and cabinets. It achieves higher speeds through faster optics and optional advanced modulation, while maintaining the same fundamental PON architecture.

[Diagram: see online version]

Key Parameters

Parameter	50G-PON (G.9804)
Downstream rate	49.7664 Gbps
Upstream rate	24.8832 Gbps (asym) or 49.7664 Gbps (sym)
Downstream wavelength	1340-1344 nm
Upstream wavelength	1290-1310 nm
Modulation (downstream)	NRZ at 50G or PAM4 at 25 Gbaud
Modulation (upstream)	NRZ at 25G (asym) or NRZ/PAM4 at 50G (sym)
Split ratio	1:32 / 1:64 (same as GPON/XGS-PON)
Max reach	20 km (40 km with reach extenders)
Encryption	AES-256
FEC	LDPC (Low-Density Parity-Check)
Line coding	64b/66b
Frame period	125 us

Modulation Options

Approach	Symbol Rate	Modulation	Advantage
NRZ at 50G	~50 Gbaud	NRZ (2-level)	Simple, proven, but requires 50G optics
PAM4 at 25G	~25 Gbaud	PAM4 (4-level)	Uses 25G optics (cheaper), but lower noise margin

The industry consensus is moving toward **NRZ at ~25 Gbaud** for the 24.8832 Gbps upstream and advanced FEC to close the link budget. Downstream uses either approach depending on vendor implementation.

Framing (Enhanced XGTC)

50G-PON extends the XGTC framing from XGS-PON:

Feature	XGS-PON (G.9807)	50G-PON (G.9804)
Frame period	125 us	125 us
Encapsulation	XGEM	Enhanced XGEM
XGEM header	8 bytes	8 bytes (extended fields)
PLOAM	48 bytes	48 bytes (enhanced)
BWmap	Per Alloc-ID	Per Alloc-ID (larger burst sizes)
FEC	RS(248,216)	LDPC
Encryption	AES-256	AES-256

AMCC (Auxiliary Management and Control Channel)

50G-PON introduces AMCC for out-of-band ONU management during activation, before the main data channel is established:

Feature	Description
Purpose	ONU discovery, wavelength assignment, initial provisioning
Rate	Low-rate (e.g., 155 Mbps) management channel
Wavelength	Separate from data wavelengths
Usage	Used during ONU activation before high-speed link is up
Shared with	NG-PON2 also uses AMCC

Coexistence with Legacy PON

50G-PON is designed to coexist with GPON and XGS-PON on the same fiber using WDM. A Coexistence Element (CEx) at the OLT side multiplexes/demultiplexes the wavelengths:

[Diagram: see online version]

NG-PON2 (ITU-T G.989)

Overview

NG-PON2 uses Time and Wavelength Division Multiplexed PON (TWDM-PON) -- multiple wavelength pairs, each carrying 10G, for aggregate capacity of 40-80 Gbps. ONUs have **tunable optics** that can lock to any of the available wavelengths, allowing dynamic wavelength assignment.

[Diagram: see online version]

Key Parameters

Parameter	NG-PON2 (G.989)
Architecture	TWDM-PON (Time and Wavelength Division Multiplexing)
Wavelength pairs	4 (baseline) to 8 (extended)
Rate per wavelength	10 Gbps downstream / 10 Gbps upstream
Aggregate downstream	40 Gbps (4 x 10G) to 80 Gbps (8 x 10G)
Aggregate upstream	40 Gbps (4 x 10G) to 80 Gbps (8 x 10G)
Downstream band	1596-1603 nm (4 wavelengths in 100 GHz grid)
Upstream band	1524-1544 nm (4 wavelengths in 100 GHz grid)
Split ratio	1:64
Max reach	40 km
ONU optics	Tunable laser + tunable filter
Encryption	AES-256

TWDM Wavelength Plan

Wavelength Pair	Downstream (nm)	Upstream (nm)
Channel 1	1596.34	1524.50
Channel 2	1597.19	1528.77
Channel 3	1598.04	1533.07
Channel 4	1598.89	1537.40

Each channel operates independently as a 10G-PON, sharing the same ODN. ONUs tune to one wavelength pair at a time.

PtP WDM Overlay

NG-PON2 also supports a Point-to-Point WDM overlay for dedicated wavelength services:

Feature	TWDM	PtP WDM
Topology	Point-to-multipoint	Point-to-point (dedicated wavelength)
Bandwidth	Shared 10G per wavelength	Dedicated wavelength per ONU
Use case	Residential, business	Mobile fronthaul, enterprise leased line
Wavelength band	C-band (as above)	Additional wavelengths in expanded spectrum

ONU Wavelength Assignment

[Diagram: see online version]

NG-PON2 Deployment Challenges

Challenge	Impact
Tunable optics cost	Significantly more expensive than fixed-wavelength ONUs
Complexity	Wavelength management, tuning time, inventory
Market timing	Arrived when XGS-PON was "good enough" for most deployments
50G-PON competition	Single-wavelength 50G is simpler and cheaper
Limited vendors	Fewer ONU/OLT vendors compared to GPON/XGS-PON

Wavelength Plan (Full Coexistence)

All ITU-T PON generations can coexist on the same fiber:

Technology	Downstream (nm)	Upstream (nm)	Notes
GPON	1480-1500	1290-1330	Widest deployment
XGS-PON	1575-1580	1260-1280	10G symmetric
50G-PON	1340-1344	1290-1310	Next generation
NG-PON2 TWDM	1596-1603	1524-1544	Multi-wavelength
RF Video	1550-1560	N/A	Analog/QAM video overlay

This careful wavelength allocation means an operator can deploy all generations simultaneously on the same fiber and splitter infrastructure.

Technology Comparison

Feature	GPON	XGS-PON	50G-PON	NG-PON2
Standard	G.984	G.9807	G.9804	G.989
DS rate	2.5 Gbps	10 Gbps	50 Gbps	40-80 Gbps (aggregate)
US rate	1.2 Gbps	10 Gbps	25-50 Gbps	40-80 Gbps (aggregate)
Wavelengths	1 pair	1 pair	1 pair	4-8 pairs
ONU optics	Fixed	Fixed	Fixed	Tunable
Modulation	NRZ	NRZ	NRZ or PAM4	NRZ
FEC	RS optional	RS mandatory	LDPC	RS mandatory

Encryption	AES-128	AES-256	AES-256	AES-256
Split ratio	1:64	1:64	1:64	1:64
Max reach	20 km	20 km	20 km	40 km
Deployment status	Mature	Active deployment	Standards complete, early trials	Limited deployment
ONU cost	Low	Low-moderate	Moderate	High (tunable)

IEEE Alternatives

For completeness, the IEEE track pursues similar goals:

Standard	Rate	Status
IEEE 802.3ca (25G-EPON)	25G DS / 10G or 25G US	Published 2020
IEEE 802.3ca (50G-EPON)	50G (2 x 25G bonded)	Published 2020
IEEE 802.3cs	400G+ EPON (study)	Under study

Standards

Document	Title
ITU-T G.9804.1	50G-PON -- General requirements
ITU-T G.9804.2	50G-PON -- Physical media dependent layer
ITU-T G.9804.3	50G-PON -- Transmission convergence layer
ITU-T G.989.1	NG-PON2 -- General requirements
ITU-T G.989.2	NG-PON2 -- Physical media dependent layer
ITU-T G.989.3	NG-PON2 -- Transmission convergence layer
IEEE 802.3ca-2020	25G/50G EPON (IEEE alternative)

See Also

- GPON -- current-generation ITU-T PON (2.5G/1.2G)
- EPON -- IEEE-based PON using native Ethernet
- Ethernet -- payload carried over PON
- TR-069 / TR-369 -- CPE management for PON ONTs

5G SBI -- Service-Based Interface

Standard: 3GPP TS 29.500 | **Layer:** Application (HTTP/2 + JSON) | **Wireshark filter:** `http2`

5G SBI is the service-based architecture (SBA) framework for communication between Network Functions (NFs) in the 5G Core. Unlike 4G's point-to-point interfaces (S6a, Gx, S11, each with its own protocol), 5G NFs expose RESTful APIs over HTTP/2 with JSON payloads. Any NF can discover and consume services from any other NF through the NRF (NF Repository Function). This architecture enables cloud-native deployment, independent NF scaling, and service mesh patterns.

5G Service-Based Architecture

[Diagram: see online version]

Note: The UPF uses PFCP on N4, not SBI -- it is a user plane element, not part of the service-based architecture.

Network Functions and Services

NF	Service Name	Key Operations
AMF	Namf_Communication	N1N2MessageTransfer, UEContextTransfer, registration notification
AMF	Namf_EventExposure	Subscribe/notify for UE reachability, location, connectivity
SMF	Nsmf_PDUSession	CreateSMContext, UpdateSMContext, ReleaseSMContext
SMF	Nsmf_EventExposure	Subscribe/notify for PDU session events
NRF	Nnrf_NFManagement	NFRegister, NFUpdate, NFDeregister, NFStatusSubscribe
NRF	Nnrf_NFDiscovery	NFDiscover (query NF instances by type, services, S-NSSAI)
AUSF	Nausf_UEAuthentication	Authenticate, deregister notification
UDM	Nudm_UECM	Registration, deregistration of AMF for subscriber
UDM	Nudm_SDM	Subscription data retrieval, subscribe to data change
UDM	Nudm_UEAuthentication	Get authentication data (vectors)
PCF	Npcf_AMPolicy	Create/update access and mobility policy
PCF	Npcf_SMPolicyControl	Create/update/delete session policy (QoS, charging rules)
NSSF	Nnssf_NSSelection	Slice selection (get allowed/configured NSSAI, target AMF set)
NEF	Nnef_EventExposure	Expose network events to external AF (applications)

API Structure

All SBI APIs follow a common URL pattern:

```
{apiRoot}/{apiName}/{apiVersion}/{resource}
```

Component	Example	Description
apiRoot	`https://amf.5gc.mnc001.mcc001.3gppnetwork.org`	NF base URI (FQDN or IP)
apiName	`namf-comm`	Service name (lowercase, hyphenated)
apiVersion	`v1`	API version
resource	`/ue-contexts/{ueContextId}/n1-n2-messages`	Resource path

Full example:
`https://amf.5gc.example.com/namf-comm/v1/ue-contexts/{ueContextId}/n1-n2-messages`

HTTP Methods

HTTP Method	SBI Usage
POST	Create resource, trigger action, custom operations
GET	Retrieve resource, query/discover
PUT	Full replacement of resource
PATCH	Partial update (JSON Merge Patch or JSON Patch)
DELETE	Remove resource

Service Operation Patterns

Pattern	Mechanism	Example
Request-Response	HTTP request + response	POST Nsmf_PDUSession_CreateSMContext
Subscribe-Notify	POST subscription, then callback POST on events	POST Nnrf_NFManagement_NFStatusSubscribe

For subscribe-notify, the consumer provides a callback URI in the subscription request. The producer POSTs notifications to that URI when events occur.

NF Discovery and Registration

[Diagram: see online version]

NF Profile (Registration Data)

Field	Description
nfInstanceId	UUID of the NF instance
nfType	AMF, SMF, UDM, AUSF, NRF, PCF, NSSF, NEF, etc.
nfStatus	REGISTERED, SUSPENDED, UNDISCOVERABLE
fqdn	Fully Qualified Domain Name
ipv4Addresses / ipv6Addresses	IP endpoints
nfServices	List of services (name, versions, scheme, URI)
sNssais	Supported network slices (SST + SD)
plmnList	Supported PLMNs (MCC + MNC)
capacity	Relative capacity (for load balancing)
priority	Selection priority
load	Current load (0-100)
nfServicePersistence	Whether NF supports service persistence

Authorization (OAuth 2.0)

NF-to-NF communication is authorized using OAuth 2.0 with the NRF acting as the authorization server:

[Diagram: see online version]

The access token is a signed JWT containing the consumer NF instance ID, producer NF type, allowed services (scope), and expiration.

SCP (Service Communication Proxy)

The SCP provides indirect communication between NFs, acting as a service mesh layer:

[Diagram: see online version]

Communication Model	Description
Direct (Model A)	Consumer discovers producer via NRF, calls directly
Indirect without delegated discovery (Model C)	Consumer discovers via NRF, routes through SCP
Indirect with delegated discovery (Model D)	Consumer sends to SCP, SCP discovers and routes

The SCP handles load balancing, topology hiding, monitoring, and message forwarding. In Models C/D, the SCP adds `3gpp-Sbi-Target-apiRoot` headers to route requests.

PDU Session Establishment via SBI

[Diagram: see online version]

SBI vs 4G Point-to-Point Interfaces

Aspect	5G SBI	4G EPC Interfaces
Architecture	Service-based (any NF to any NF)	Point-to-point (fixed interface pairs)
Transport	HTTP/2 over TCP/TLS	Diameter (SCTP/TCP), GTPv2-C (UDP)
Encoding	JSON (text-based)	Diameter AVPs (binary), GTPv2-C IEs (binary)
Discovery	Dynamic via NRF	Static configuration or DNS
Authorization	OAuth 2.0 (JWT tokens)	Diameter CER/CEA peer exchange
API style	RESTful (resources, HTTP methods)	Command-response (request/answer codes)
Versioning	URI-based (/v1, /v2)	Diameter Application-ID
Extensibility	New NF services, OpenAPI specs	New Diameter AVPs, command codes
4G equivalent interfaces	--	S6a (MME-HSS), Gx (PGW-PCRF), S11 (MME-SGW)
Deployment	Cloud-native, containerized, scalable	Appliance-based, monolithic

4G-to-5G Interface Mapping

4G Interface	Protocol	5G Equivalent	5G Protocol
S6a (MME - HSS)	Diameter	AMF - UDM/AUSF	SBI (Nudm, Nausf)
S11 (MME - SGW)	GTPv2-C	AMF - SMF	SBI (Nsmf)
Gx (PGW - PCRF)	Diameter	SMF - PCF	SBI (Npcf)
S5/S8 control (SGW - PGW)	GTPv2-C	SMF - UPF	PFCP (N4, not SBI)
Rx (AF - PCRF)	Diameter	AF - NEF/PCF	SBI (Nnef)

Standards

Document	Title
3GPP TS 29.500	5G SBI framework and technical realization
3GPP TS 29.501	SBI principles and guidelines
3GPP TS 29.510	NRF services (discovery, management)
3GPP TS 29.502	SMF services (Nsmf)
3GPP TS 29.518	AMF services (Namf)
3GPP TS 29.509	AUSF services (Nausf)

3GPP TS 29.503	UDM services (Nudm)
3GPP TS 29.507	PCF services (Npcf)
3GPP TS 23.501	5G system architecture

See Also

- NGAP -- N2 RAN signaling (gNB to AMF, not SBI)
- PFCP -- N4 user plane control (SMF to UPF, not SBI)
- NAS 5G -- UE-AMF signaling carried via SBI between AMF and SMF
- GTP -- user plane tunneling (N3/N9)
- Diameter -- 4G AAA/policy protocol replaced by SBI
- LTE -- 4G architecture with point-to-point interfaces

SMS / MMS -- Short Message Service / Multimedia Messaging Service

Standard: 3GPP TS 23.040 (SMS) / 3GPP TS 23.140 (MMS) | **Layer:** Application (over SS7 MAP / IP) | **Wireshark filter:** `gsm_sms` or `gsm_sms_ud` or `mmse`

SMS is a store-and-forward messaging service that allows sending text messages of up to 160 characters (GSM 7-bit encoding) between mobile devices. Originally defined as part of GSM, SMS routes through a Short Message Service Center (SMSC) and uses the SS7 MAP signaling network for delivery. MMS extends messaging to support images, audio, video, and rich text by leveraging WAP and HTTP for content delivery through a Multimedia Messaging Service Center (MMSC). Together, SMS and MMS form the foundation of mobile messaging, with billions of messages delivered daily worldwide.

SMS Architecture

[Diagram: see online version]

Network Elements

Element	Role
MS	Mobile Station (handset)
MSC	Mobile Switching Center -- routes SMS to/from SMSC
SMSC	Short Message Service Center -- store-and-forward hub
HLR	Home Location Register -- provides routing info for delivery

SMS TP-PDU Types

PDU Type	Direction	Description
SMS-SUBMIT	MS -> SMSC	Mobile-originated message
SMS-SUBMIT-REPORT	SMSC -> MS	Acknowledgment of SMS-SUBMIT
SMS-DELIVER	SMSC -> MS	Mobile-terminated message
SMS-DELIVER-REPORT	MS -> SMSC	Acknowledgment of SMS-DELIVER
SMS-STATUS-REPORT	SMSC -> MS	Delivery status of a previously sent message
SMS-COMMAND	MS -> SMSC	Command to SMSC regarding a previous message

SMS-SUBMIT PDU

[Diagram: see online version]

Key Fields

Field	Size	Description
SCA	Variable	Service Centre Address (SMSC number)
PDU-Type	1 byte	Message type and flag bits
MR	1 byte	Message Reference -- sequence number set by MS
DA / OA	Variable	Destination Address (SUBMIT) or Originating Address (DELIVER)
PID	1 byte	Protocol Identifier (0x00 = default SMS)
DCS	1 byte	Data Coding Scheme (encoding and message class)
VP	0, 1, or 7 bytes	Validity Period (duration SMSC retains message)
UDL	1 byte	User Data Length (characters or octets)
UD	Variable	User Data (message content, optionally with UDH)

PDU-Type Bits (SMS-SUBMIT)

[Diagram: see online version]

Bit(s)	Field	Description

0-1	TP-MTI	Message Type Indicator (01 = SMS-SUBMIT)
2	TP-RD	Reject Duplicates (1 = reject duplicate MR)
3-4	TP-VPF	Validity Period Format (00=none, 01=enhanced, 10=relative, 11=absolute)
5	TP-SRR	Status Report Request (1 = request delivery report)
6	TP-UDHI	User Data Header Indicator (1 = UDH present in UD)
7	TP-RP	Reply Path (1 = reply path set)

Data Coding Scheme

DCS Value	Encoding	Max Chars per SMS
0x00	GSM 7-bit default alphabet	160
0x04	8-bit binary data	140
0x08	UCS-2 (UTF-16 BE)	70

GSM 7-bit Packing

160 characters x 7 bits = 1,120 bits = 140 bytes. Characters are packed right-to-left, filling 7 bits per character into 8-bit octets.

Concatenated SMS (Long Messages)

Messages exceeding the single-SMS character limit are split across multiple SMS segments using a User Data Header (UDH).

Concatenation UDH

[Diagram: see online version]

Field	Size	Description
UDHL	1 byte	User Data Header Length (0x05 for 8-bit ref, 0x06 for 16-bit ref)
IEI	1 byte	Information Element Identifier (0x00 = concatenation, 8-bit ref)
IEDL	1 byte	Information Element Data Length (0x03 for 8-bit ref)
Reference	1 byte	Concatenation reference number (same across all parts)

Total Parts	1 byte	Total number of segments
Part Number	1 byte	Sequence number of this segment (1-based)

With the UDH overhead, each segment carries 153 characters (GSM 7-bit) or 67 characters (UCS-2) instead of the full single-SMS capacity.

SMS-SUBMIT Flow

[Diagram: see online version]

SMS Transport Variants

Transport	Network	Protocol Stack
SMS over MAP/SS7	2G/3G (GSM, UMTS)	RP -> MAP -> TCAP -> SCCP -> MTP
SMS over SGs	4G (LTE)	NAS -> SGs -> MAP -> TCAP -> SCCP
SMS over IMS	VoLTE, 5G	SIP MESSAGE -> IP-SM-GW -> SMSC
SMS over NAS	5G NR	NAS -> AMF -> SMSF -> SMSC

MMS (Multimedia Messaging Service)

MMS extends SMS to support multimedia content (images, audio, video, contacts, calendar entries) through a WAP/HTTP-based architecture centered on the Multimedia Messaging Service Center (MMSC).

MMS Architecture

[Diagram: see online version]

MMS Transaction Flow

[Diagram: see online version]

MMS PDU Types

PDU Type	Direction	Description
M-Send.req	Phone -> MMSC	Submit multimedia message
M-Send.conf	MMSC -> Phone	Submission acknowledgment
M-Notification.ind	MMSC -> Phone	Notification of waiting message (via WAP Push)

M-Retrieve.conf	MMSC -> Phone	Message content delivery (HTTP response)
M-Acknowledge.ind	Phone -> MMSC	Retrieval acknowledgment
M-Delivery.ind	MMSC -> Phone	Delivery report
M-Read-Rec.ind	Phone -> MMSC	Read receipt
M-Forward.req	Phone -> MMSC	Forward a stored message

MMS Headers

Header	Description
X-Mms-Message-Type	PDU type identifier (M-Send.req, M-Notification.ind, etc.)
X-Mms-Transaction-ID	Correlates requests with responses
X-Mms-MMS-Version	MMS version (e.g., 1.0, 1.2, 1.3)
From	Sender address (phone number or email)
To	Recipient address(es)
Subject	Message subject line
Content-Type	Top-level content type (multipart/mixed or multipart/related)
X-Mms-Message-Class	personal, advertisement, informational, auto
X-Mms-Expiry	Message expiry time on MMSC
X-Mms-Message-Size	Size of the multimedia content in bytes
X-Mms-Content-Location	URL to retrieve the message (in M-Notification.ind)

MMS Content Structure

MMS messages are encoded as multipart MIME bodies, typically using SMIL (Synchronized Multimedia Integration Language) for layout:

Part	Content-Type	Purpose
SMIL	application/smil	Presentation layout (slide timing, positioning)
Text	text/plain	Text portion of each slide
Image	image/jpeg, image/gif, image/png	Picture attachments
Audio	audio/amr, audio/mpeg	Audio clips
Video	video/3gpp, video/mp4	Video clips
vCard	text/x-vcard	Contact cards
vCalendar	text/x-vcalendar	Calendar entries

MMS PDUs are binary-encoded using a WBXML-like format (WSP content type and header encoding) for compact transmission.

SMS vs MMS vs RCS

Feature	SMS	MMS	RCS (Rich Communication Services)
Content	Text only	Multimedia (images, video, audio)	Multimedia, interactive cards, actions
Max Size	160 chars (7-bit) / 70 chars (UCS-2)	~300 KB - 1 MB (carrier dependent)	Up to 100 MB
Encoding	GSM 7-bit, UCS-2, 8-bit	MIME multipart (WBXML headers)	HTTP/JSON
Transport	SS7 MAP / NAS	WAP Push + HTTP	IMS / HTTP (MSRP, HTTP)
Store-and-Forward	Yes (SMSC)	Yes (MMSC)	Yes (RCS hub)
Delivery Reports	Optional (Status Report)	Optional (M-Delivery.ind)	Built-in (read receipts, typing)
Group Messaging	Separate SMS to each recipient	Group MMS (single MMS to group)	Native group chat
Encryption	None (air interface only)	None (air interface only)	Optional (E2E in some implementations)
Fallback	N/A	Falls back to SMS	Falls back to SMS/MMS
Era	1992+	2002+	2016+

Standards

Document	Title
3GPP TS 23.040	Technical realization of the Short Message Service (SMS)
3GPP TS 23.038	Alphabets and language-specific information (GSM 7-bit)
3GPP TS 24.011	SMS point-to-point (relay layer)
3GPP TS 29.002	MAP specification (SMS routing over SS7)
3GPP TS 23.140	MMS -- Functional description; Stage 2
OMA MMS	OMA MMS Architecture and Protocols
3GPP TS 23.040 Annex E	Concatenated SMS (UDH specification)

See Also

- WAP -- MMS notification delivered via WAP Push
- SMPP -- application-level SMS submission protocol
- GSM -- mobile network architecture for SMS delivery
- USSD -- session-based alternative to SMS

- WBXML -- binary encoding used in MMS PDUs

SS7 -- Signaling System No. 7

Standard: ITU-T Q.700 series | **Layer:** Full stack (Layers 1-7) | **Wireshark filter:** `mtp3` or `isup` or `sccp` or `tcap`

SS7 is the signaling protocol suite used by telephone networks worldwide to set up and tear down calls, route SMS, manage number portability, provide roaming, and support intelligent network services. It operates on a dedicated signaling network separate from the voice/data bearer channels. SS7 was developed in the 1970s-80s and remains the backbone of PSTN and mobile signaling, though modern networks increasingly carry SS7 over IP using SIGTRAN.

Protocol Stack

[Diagram: see online version]

MTP (Message Transfer Part)

MTP Level 1 (Physical)

The physical signaling link -- typically a 64 kbps timeslot within a T1 or E1 frame, or an IP connection (SIGTRAN).

MTP Level 2 (Data Link)

Provides reliable delivery of signaling messages over a single link using HDLC-like framing:

[Diagram: see online version]

Field	Size	Description
Flag	8 bits	HDLC flag `0x7E` -- frame delimiter
BSN	7 bits	Backward Sequence Number (acknowledgment)
BIB	1 bit	Backward Indicator Bit (retransmission control)
FSN	7 bits	Forward Sequence Number
FIB	1 bit	Forward Indicator Bit
LI	6 bits	Length Indicator (length of SIF + SIO)
SIO	8 bits	Service Information Octet (service indicator + sub-service)
SIF	Variable	Signaling Information Field (payload)

CRC-16	16 bits	Error check

MTP Level 3 (Network)

Routes messages between signaling points. Key concepts:

Term	Description
Originating Point Code (OPC)	Address of the sending node
Destination Point Code (DPC)	Address of the receiving node
Service Indicator (SI)	Identifies the user part (ISUP, SCCP, etc.)
Signaling Link Selection (SLS)	Load balancing across link sets

Service Indicator Values

SI	User Part
0	Signaling Network Management
1	Signaling Network Testing
3	SCCP
4	TUP (Telephone User Part)
5	ISUP
6	DUP (Data User Part)

Point Codes

Variant	Format	Size
ITU	14 bits (3-8-3)	Zone-Area-SP
ANSI	24 bits (8-8-8)	Network-Cluster-Member

ISUP (ISDN User Part)

ISUP handles call setup and teardown. Key messages:

Abbreviation	Message	Direction	Description
IAM	Initial Address Message	Forward	Initiates a call (contains called number)
ACM	Address Complete	Backward	Called party is being alerted
ANM	Answer	Backward	Called party has answered
REL	Release	Either	Request to release the circuit

RLC	Release Complete	Either	Circuit released
CPG	Call Progress	Backward	In-band information available
CON	Connect	Backward	Alternative to ANM

Basic Call Flow

[Diagram: see online version]

SCCP (Signaling Connection Control Part)

Provides global title routing and connection-oriented/connectionless transport above MTP3. Used by TCAP, MAP, and INAP.

Service	Description
Global Title Translation	Routes messages using E.164 numbers instead of point codes
Connection-oriented	Virtual circuits for long transactions
Connectionless	Datagram service for short queries

TCAP (Transaction Capabilities Application Part)

Provides a transaction framework for remote operations. Used by MAP, INAP, and CAP.

Component	Description
Invoke	Request a remote operation
Return Result	Successful response
Return Error	Error response
Reject	Protocol error

MAP (Mobile Application Part)

Used in mobile networks for:

Operation	Purpose
UpdateLocation	Register subscriber at new MSC/VLR
SendRoutingInfo	Resolve MSISDN for call routing
SendRoutingInfoForSM	Resolve MSISDN for SMS delivery
InsertSubscriberData	Push subscriber profile from HLR to VLR
AuthenticationInfo	Retrieve authentication vectors

SIGTRAN (SS7 over IP)

Modern SS7 transport replaces MTP1/MTP2 with IP:

[Diagram: see online version]

Protocol	RFC	Replaces
M2UA	RFC 3331	MTP2
M3UA	RFC 4666	MTP3
SUA	RFC 3868	SCCP
SCTP	RFC 9260	MTP2 reliable delivery

Standards

Document	Title
ITU-T Q.700	Introduction to SS7
ITU-T Q.703	MTP Level 2 -- Signaling Link
ITU-T Q.704	MTP Level 3 -- Signaling Network Functions
ITU-T Q.711-Q.716	SCCP
ITU-T Q.761-Q.764	ISUP
ITU-T Q.771-Q.775	TCAP
3GPP TS 29.002	MAP specification
RFC 4666	M3UA -- MTP3 User Adaptation Layer

See Also

- ISDN -- uses SS7 (ISUP) for call signaling
- T1 -- physical bearer for SS7 links in North America
- E1 -- physical bearer for SS7 links internationally
- MM5 -- MMS interface using MAP over SS7
- SMPP -- SMS protocol that interfaces with SS7/MAP

T1 -- DS1

Standard: ANSI T1.403 / ITU-T G.703 | **Layer:** Physical (Layer 1) | **Wireshark filter:** N/A (sub-packet-capture)

T1 is a digital transmission standard carrying 1.544 Mbps over two twisted pairs (one for each direction). It multiplexes 24 channels of 64 kbps (DS0) using time-division multiplexing (TDM). T1 is the fundamental digital carrier in North America and Japan, used for voice trunks, PRI ISDN, point-to-point data, and SS7 signaling links. The signal format is DS1; "T1" strictly refers to the physical line, though the terms are used interchangeably.

Frame

A T1 frame is 193 bits, transmitted 8,000 times per second:

[Diagram: see online version]

Field	Size	Description
F (Framing bit)	1 bit	Framing, synchronization, and signaling
Channel 1-24	8 bits each	24 timeslots, each carrying one DS0 (64 kbps)

Total: 1 + (24 x 8) = 193 bits x 8,000 frames/sec = **1,544,000 bps**

Key Parameters

Parameter	Value
Line rate	1.544 Mbps
Channels	24 x DS0 (64 kbps)
Frame size	193 bits
Frame rate	8,000 frames/sec
Encoding	AMI or B8ZS
Cable	2 twisted pairs (4 wires), 100Ω
Max distance	~1.8 km (6,000 ft) without repeaters
Connector	RJ-48C (8-pin)

Framing Formats

D4 (Superframe -- SF)

Groups 12 consecutive frames into a superframe. The framing bit follows the pattern `100011011100` for alignment:

[Diagram: see online version]

- **Framing bits (odd frames):** Terminal framing `1 0 1 0 1 0` -- frame sync
- **Framing bits (even frames):** Signaling framing `0 0 1 1 1 0` -- multiframe sync
- **Robbed-bit signaling:** In frames 6 and 12, the LSB of each channel is "robbed" for A/B signaling bits

ESF (Extended Superframe)

Groups 24 frames into an extended superframe. The framing bit is divided into three functions:

Bits (frame positions)	Purpose
4, 8, 12, 16, 20, 24	FDL -- Facilities Data Link (4 kbps data channel)
2, 6, 10, 14, 18, 22	CRC-6 -- Error checking over the superframe
1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23	Frame alignment: `001011`

- **Robbed-bit signaling:** In frames 6, 12, 18, 24 -- provides A/B/C/D signaling bits
- **CRC-6:** Detects transmission errors without taking the circuit down
- **FDL:** 4 kbps out-of-band data channel for performance monitoring and alarms

Line Coding

Encoding	Description
AMI (Alternate Mark Inversion)	1s alternate between +V and -V; 0s are no pulse. Must enforce ones-density rules.
B8ZS (Bipolar with 8-Zero Substitution)	Replaces 8 consecutive zeros with a violation pattern; maintains clock sync. Standard for ESF.

Signaling

Channel Associated Signaling (CAS / Robbed-Bit)

In voice applications, the LSB of each channel is periodically "robbed" to carry A/B (SF) or A/B/C/D (ESF) signaling bits:

Bits	State (typical E&M)
A=0, B=0	On-hook (idle)
A=1, B=1	Off-hook (active)
A=1, B=0	Wink (acknowledgment)

This reduces each voice channel from 64 kbps to 56 kbps effective payload.

Common Channel Signaling (CCS)

Channel 24 (or another designated channel) carries ISDN PRI signaling (Q.931) or SS7 instead of voice. The remaining 23 channels carry bearer traffic.

Configuration	Signaling	Bearer Channels
CAS (robbed-bit)	In-band per channel	24 x 56 kbps

PRI (ISDN)	Channel 24 (D)	23 x 64 kbps B + 1 x 64 kbps D
SS7	Dedicated timeslot(s)	Remaining channels

DS Hierarchy (North America)

Level	Rate	Channels	Composition
DS0	64 kbps	1	Single channel
DS1 (T1)	1.544 Mbps	24	24 x DS0
DS2 (T2)	6.312 Mbps	96	4 x DS1
DS3 (T3)	44.736 Mbps	672	28 x DS1
DS4 (T4)	274.176 Mbps	4032	6 x DS3

Standards

Document	Title
ANSI T1.403	DS1 Electrical Interface
ITU-T G.703	Physical/electrical characteristics of hierarchical digital interfaces
ITU-T G.704	Synchronous frame structures
ANSI T1.107	DS1/DS3 Digital Hierarchy Formats

See Also

- E1 -- international equivalent (2.048 Mbps, 32 channels)
- ISDN -- PRI signaling over T1
- SS7 -- signaling carried in T1 timeslots

USSD -- Unstructured Supplementary Service Data

Standard: 3GPP TS 22.090 / 3GPP TS 23.090 / 3GPP TS 24.090 | **Layer:** Application (over MAP/SS7 or Diameter/SIP) | **Wireshark filter:** `gsm_map` (USSD is carried within MAP operations)

USSD is a real-time, session-based GSM signaling protocol that allows mobile phones to communicate with network applications through short text commands. Unlike SMS, which is store-and-forward, USSD maintains a live session between the handset and the network -- responses arrive in real time and the session stays open for multi-step interactions. USSD is widely used for balance inquiries, prepaid top-up, mobile money services (M-Pesa), operator self-service menus, call forwarding setup, and SIM Toolkit applications. It operates over the SS7 MAP signaling

network (2G/3G) and has been adapted for 4G/5G via Diameter and SIP.

USSD String Format

USSD commands follow a structured dialing string pattern:

```
*service_code#
*service_code*param1*param2#
*service_code*param1*param2*param3#
**service_code*param#
```

Common USSD Codes

Code	Purpose	Example
`*#06#`	Display IMEI	Shows handset identity
`*100#`	Balance inquiry	Prepaid balance (carrier-specific)
`*#21#`	Query call forwarding	Check if call forwarding is active
`**21*number#`	Set call forwarding	Forward calls to another number
`##21#`	Cancel call forwarding	Disable call forwarding
`*131*amount*number#`	Airtime transfer	Send balance to another subscriber
`*144#`	Mobile money menu	M-Pesa / mobile banking (carrier-specific)

Architecture

2G/3G (MAP/SS7)

[Diagram: see online version]

4G/5G (Diameter/SIP)

[Diagram: see online version]

Network Elements

Element	Role
MS / UE	Mobile device initiating or responding to USSD
MSC	Mobile Switching Center -- relays USSD via MAP
HLR	Home Location Register -- routes USSD or handles directly
USSD Gateway	Terminates USSD sessions, connects to application servers
Application Server	Business logic (mobile money, balance check, menus)

MAP Operations

USSD is carried within SS7 MAP (Mobile Application Part) using three operations:

MAP Operation	Direction	Description
ProcessUnstructuredSS-Request	MS -> Network	Mobile-initiated USSD request
UnstructuredSS-Request	Network -> MS	Network-initiated prompt (expects response)
UnstructuredSS-Notify	Network -> MS	Network-initiated one-way notification (no response expected)

MAP USSD Parameters

Parameter	Description
USSD-DataCodingScheme	Encoding of the USSD string (typically GSM 7-bit)
USSD-String	The USSD message content (up to 182 characters)
MSISDN	Subscriber phone number
AlertingPattern	Optional alerting indication

Data Coding Scheme

DCS	Encoding	Max Characters
0x0F	GSM 7-bit default alphabet	182
0x48	UCS-2 (UTF-16)	91

The GSM 7-bit default alphabet allows 182 characters per USSD message (compared to 160 for SMS), because USSD carries 160 bytes of user data and 7-bit packing yields 182 characters from 160 bytes.

Session Model

USSD sessions are stateful and real-time. The network maintains session context between exchanges, allowing multi-step menu navigation:

Property	USSD	SMS
Delivery model	Real-time, session-based	Store-and-forward
Session state	Maintained across exchanges	Stateless
Timeout	30-180 seconds (network-defined)	Up to 72 hours (SMSC VP)
Billing	Per-session (often free or low-cost)	Per-message
Max length	182 chars (GSM 7-bit)	160 chars (GSM 7-bit)

Direction	Bidirectional within session	Unidirectional per PDU

USSD Session Flow

Basic Balance Inquiry

[Diagram: see online version]

Multi-Step Menu Navigation (Mobile Money)

[Diagram: see online version]

USSD vs SMS vs STK

Feature	USSD	SMS	STK (SIM Toolkit)
Session	Real-time, interactive	Store-and-forward, stateless	On-device menus + SMS/USSD
Latency	Sub-second response	Seconds to minutes	Instant (local), varies (network)
Storage	No message storage	Stored on SMSC and handset	Menus stored on SIM
Max Length	182 chars (GSM 7-bit)	160 chars (GSM 7-bit)	Varies by SIM capacity
Cost	Often free / low-cost	Per-message charge	Free (SIM-based)
Works Offline	No (requires active radio)	Store-and-forward handles offline	Menus available; actions need network
Data Input	Free-form text reply	Full SMS compose	Predefined menus, limited input
Encryption	SS7 signaling (not encrypted E2E)	SS7 signaling (not encrypted E2E)	SIM-level security (OTA keys)
Use Cases	Balance, mobile money, menus	Messaging, notifications, 2FA	Operator menus, payments

USSD over 4G/5G

In LTE and 5G networks, traditional SS7-based USSD is adapted:

Network	USSD Transport	Protocol
2G GSM	MAP over SS7 (TCAP/SCCP/MTP)	3GPP TS 24.090
3G UMTS	MAP over SS7 (same as 2G)	3GPP TS 24.090
4G LTE	SGs interface to MSC (fallback) or IMS-based	3GPP TS 29.338 (Diameter)

5G NR	IMS-based or NEF API exposure	3GPP TS 29.338 / TS 29.522

In 4G/5G, USSD requests may be tunneled through NAS signaling to an IMS USSD gateway, or routed via the SGs interface to a legacy MSC for MAP-based handling.

Security Considerations

Concern	Description
No end-to-end encryption	USSD traverses SS7 signaling in cleartext
SS7 vulnerabilities	SS7 network access enables interception and session hijacking
PIN over USSD	Mobile money PINs sent in cleartext USSD strings
Session hijacking	Attacker with SS7 access can inject UnstructuredSS-Request
SIM swapping	Social engineering to obtain victim SIM and access USSD services

Standards

Document	Title
3GPP TS 22.090	Unstructured Supplementary Service Data (USSD) -- Stage 1 (service description)
3GPP TS 23.090	Unstructured Supplementary Service Data (USSD) -- Stage 2 (functional description)
3GPP TS 24.090	Unstructured Supplementary Service Data (USSD) -- Stage 3 (protocol specification)
3GPP TS 29.002	MAP specification (carries USSD operations)
3GPP TS 23.038	Alphabets and language-specific information (GSM 7-bit DCS)
3GPP TS 29.338	Diameter-based USSD signaling for LTE

See Also

- SMS -- store-and-forward messaging (complementary to USSD)
- GSM -- mobile network architecture
- WAP -- WAP stack can use USSD as a bearer via WDP
- SMPP -- application-level protocol for SMS/USSD gateway integration

WAP -- Wireless Application Protocol

Standard: OMA WAP Specifications | **Layer:** Full stack (Transport through Application) | **Wireshark filter:** `wsp` or `wtp` or `wtls` or `wap`

WAP is a suite of protocols developed by the WAP Forum (later absorbed into the Open Mobile Alliance) to bring Internet-like services to mobile phones over low-bandwidth, high-latency 2G networks. WAP defined its own protocol stack optimized for constrained devices: WML instead of HTML, WBXML binary encoding, a compressed HTTP-like session layer (WSP), a lightweight transaction layer (WTP), optional security (WTLS), and bearer adaptation (WDP). WAP 1.x required a WAP Gateway to translate between the WAP stack and HTTP/TCP/IP. WAP 2.0 converged toward standard Internet protocols (XHTML-MP, TCP/IP, TLS), and WAP was eventually superseded by direct mobile web browsing.

WAP Protocol Stack

[Diagram: see online version]

Stack Layers

Layer	Role	Internet Equivalent
WAE	Content and scripting (WML, WMLScript, WBXML)	HTML, JavaScript
WSP	Session management, HTTP-like methods, push	HTTP
WTP	Transaction reliability (3 classes)	Subset of TCP semantics
WTLS	Encryption, authentication, integrity	TLS / SSL
WDP	Datagram adaptation over diverse bearers	UDP

WAP Gateway Model

WAP 1.x required a gateway to bridge between the WAP protocol stack on the phone and the Internet:

[Diagram: see online version]

WAP Ports

Port	Service
9200	WSP connectionless (WDP)
9201	WSP connection-oriented (WTP)
9202	WSP + WTLS connectionless
9203	WSP + WTLS connection-oriented

WSP (Wireless Session Protocol)

WSP provides HTTP-like functionality with binary-encoded headers, session resume, and push capability.

WSP PDU

[Diagram: see online version]

WSP PDU Types

Value	PDU Type	Description
0x01	Connect	Session establishment request
0x02	ConnectReply	Session establishment response
0x03	Redirect	Redirect client to another proxy
0x04	Reply	Response to a method invocation
0x05	Disconnect	Session termination
0x06	Push	Unsolicited content from server
0x07	ConfirmedPush	Push requiring acknowledgment
0x08	Suspend	Suspend session (session resume)
0x09	Resume	Resume a suspended session
0x40	GET	Retrieve a resource
0x60	POST	Submit data to a resource

WSP Method IDs

ID	Method
0x40	GET
0x41	OPTIONS
0x42	HEAD
0x43	DELETE
0x44	TRACE
0x60	POST
0x61	PUT

WSP Content Type Codes (Selected)

Code	Content Type
0x04	text/plain

0x08	text/vnd.wap.wml
0x12	text/vnd.wap.wmlscript
0x14	application/vnd.wap.wbxml
0x29	application/vnd.wap.wmlc
0x2A	application/vnd.wap.wmlscriptc
0x30	application/vnd.wap.multipart.mixed

WTP (Wireless Transaction Protocol)

WTP provides transaction-level reliability without the overhead of a full TCP connection. It defines three transaction classes.

Transaction Classes

Class	Name	Reliability	Description
0	Unreliable Push	None	One-way message, no acknowledgment
1	Reliable Push	Ack	One-way message with acknowledgment
2	Reliable Invoke	Invoke + Result + Ack	Request/response with acknowledgment

WTP PDU

[Diagram: see online version]

WTP PDU Types

Value	PDU Type	Description
0x01	Invoke	Start a transaction (client request)
0x02	Result	Transaction result (server response)
0x03	Ack	Acknowledgment
0x04	Abort	Transaction abort
0x05	Segmented Invoke	Segmented large invoke PDU
0x06	Segmented Result	Segmented large result PDU
0x07	Negative Ack	Selective reject for missing segments

WTP Field Details

Field	Bits	Description
CON	1	Continuation flag (more PDUs follow)
GTR	1	Group trailer (last in a segmented group)
TT/R	1	Transaction trailer / Retransmission flag
PDU Type	4	Identifies the PDU type (Invoke, Result, Ack, Abort)
TID	16	Transaction ID (correlates request with response)

WTP Class 2 Transaction

[Diagram: see online version]

WTLS (Wireless Transport Layer Security)

WTLS provides TLS-like security optimized for wireless networks with smaller certificates, shorter handshakes, and reduced computational overhead.

WTLS Record Types

Type	Name	Description
1	ChangeCipherSpec	Switch to negotiated cipher
2	Alert	Warning or fatal error
3	Handshake	Key exchange and authentication
4	Application Data	Encrypted application payload

Authentication Modes

Mode	Client Auth	Server Auth	Description
Anonymous	No	No	No certificates; vulnerable to MITM
Server	No	Yes	Server certificate only (most common)
Mutual	Yes	Yes	Both client and server certificates

Certificate Types

Type ID	Format	Description
1	WTLS	Compact WAP-specific certificate format

2	X.509	Standard Internet PKI certificate
3	X9.68	URL reference to a certificate

WTLS Cipher Suites (Selected)

Suite	Key Exchange	Cipher	Hash
1	NULL	NULL	SHA-1
2	ECDH_anon	RC5_CBC_56	SHA-1
3	ECDH_anon	RC5_CBC_56	MD5
4	ECDH_ECDSA	RC5_CBC_56	SHA-1
6	RSA_anon	RC5_CBC_56	SHA-1
7	RSA	RC5_CBC_56	SHA-1
9	RSA	DES_CBC_56	SHA-1
10	RSA	3DES_CBC_EDE	SHA-1

WDP (Wireless Datagram Protocol)

WDP adapts the upper WAP layers to diverse wireless bearers. When the bearer is IP-based, WDP maps directly to UDP. For non-IP bearers, WDP provides bearer-specific adaptation.

Bearer Adaptation

Bearer	Transport	WDP Mapping
GSM CSD	Circuit-switched data	IP over PPP/CSD, WDP = UDP
GSM SMS	Store-and-forward	WDP payload in SMS user data
GSM USSD	Session-based signaling	WDP payload in USSD string
GPRS	Packet-switched data	IP/UDP directly
CDMA	IS-95/IS-2000	UDP/IP or SMS bearer
UDP/IP	Any IP network	WDP = UDP (direct mapping)

WAE (Wireless Application Environment)

WML Content Types

Content	Description
WML	Wireless Markup Language -- card/deck metaphor for mobile pages
WMLScript	Lightweight scripting language (compiled to bytecode)

WBXML	Binary encoding of WML for compact transmission
WTA	Wireless Telephony Application -- access to phone features

WML Push

WAP Push allows servers to send content to mobile devices without a prior request:

[Diagram: see online version]

Push Components

Component	Protocol	Description
Push Initiator (PI)	PAP (Push Access Protocol)	Application sending the push content
Push Proxy Gateway (PPG)	OTA (Over The Air)	Translates PAP to bearer-specific push delivery
WAP Client	WSP Push / SMS	Receives and renders push content

Push Content Types

Type	Description
SI (Service Indication)	Notification with URL -- user chooses to load
SL (Service Loading)	Automatically loads the URL
CO (Cache Operation)	Invalidates cached content on the device

WAP 1.x vs WAP 2.0 vs Mobile HTTP

Feature	WAP 1.x	WAP 2.0	Modern Mobile Web
Markup	WML (card/deck)	XHTML-MP	HTML5
Encoding	WBXML binary	Text XML/XHTML	Text (gzip compressed)
Session Layer	WSP	HTTP (TCP)	HTTP/2, HTTP/3
Transaction Layer	WTP	TCP	TCP, QUIC
Security	WTLS	TLS	TLS 1.2/1.3
Transport	WDP (bearer-adapted)	TCP/IP	TCP/IP, UDP/IP
Gateway Required	Yes (WAP Gateway)	Optional (WAP Proxy)	No
Push	WAP Push (PAP/OTA)	WAP Push or OMA Push	Web Push (RFC 8030)
Scripting	WMLScript	ECMAScript Mobile Profile	JavaScript (full)
Era	1998-2004	2002-2008	2007-present

Standards

Document	Title
WAP-230-WSP	Wireless Session Protocol Specification
WAP-224-WTP	Wireless Transaction Protocol Specification
WAP-261-WTLS	Wireless Transport Layer Security Specification
WAP-259-WDP	Wireless Datagram Protocol Specification
WAP-191-WML	Wireless Markup Language Specification
WAP-192-WBXML	WAP Binary XML Content Format
WAP-235-PushOTA	Push Over-The-Air Protocol
WAP-247-PAP	Push Access Protocol

See Also

- HTTP -- WSP is modeled after HTTP
- TLS -- WTLS is derived from TLS
- WBXML -- binary encoding used by WAP
- SMS -- bearer for WAP push and WDP
- USSD -- alternative bearer for WDP
- GSM -- mobile network WAP operates over

xDSL -- Digital Subscriber Line

Standard: ITU-T G.992 (ADSL) / G.993 (VDSL) | **Layer:** Physical / Data Link (Layers 1-2) | **Wireshark filter:** N/A (sub-packet-capture; ATM/PPPoE visible above)

xDSL is a family of technologies that deliver high-speed data over existing copper telephone lines. "x" is a placeholder for the specific variant -- ADSL, VDSL, SDSL, etc. DSL uses frequencies above the voice band (0-4 kHz) to transmit data, allowing simultaneous voice and data on the same line. A splitter or filter separates voice and data at each end. DSL is the most widely deployed broadband technology globally, connecting hundreds of millions of subscribers.

DSL Variants

Variant	Standard	Downstream	Upstream	Max Distance	Description
ADSL	G.992.1	8 Mbps	1 Mbps	5.5 km	Asymmetric -- first widely deployed DSL

ADSL2	G.992.3	12 Mbps	1.3 Mbps	5.5 km	Improved modulation, power management
ADSL2+	G.992.5	24 Mbps	1.4 Mbps	5.5 km	Doubled downstream bandwidth (2.2 MHz)
SDSL	--	2.3 Mbps	2.3 Mbps	3 km	Symmetric -- business use
VDSL	G.993.1	52 Mbps	16 Mbps	1.5 km	Very high bitrate, shorter range
VDSL2	G.993.2	100 Mbps	100 Mbps	300 m (at max speed)	Profiles up to 30 MHz bandwidth
G.fast	G.9700/G.9701	1 Gbps	1 Gbps	250 m	Up to 212 MHz bandwidth
G.mgfast	G.9710/G.9711	10 Gbps	10 Gbps	100 m	Up to 848 MHz, emerging

Frequency Plan

ADSL2+

[Diagram: see online version]

VDSL2

[Diagram: see online version]

VDSL2 uses multiple upstream (US) and downstream (DS) bands, interleaved to reduce crosstalk. The exact band plan varies by profile and region (Annex A/B).

Modulation

Technique	Used By	Description
DMT (Discrete Multi-Tone)	ADSL, VDSL	Divides bandwidth into 256-4096 subcarriers (tones), each carrying QAM-modulated data
QAM (per tone)	All xDSL	2-15 bits per tone depending on SNR
Vectoring	VDSL2, G.fast	Cancels crosstalk between lines in the same bundle using DSP

DMT (Discrete Multi-Tone)

Each 4.3125 kHz subcarrier (tone) is independently loaded with as many bits as the SNR allows:

SNR per Tone	Bits per Tone	Modulation
~7 dB	1	BPSK
~13 dB	2	QPSK
~19 dB	4	16-QAM
~25 dB	6	64-QAM
~37 dB	10	1024-QAM
~49 dB	14	16384-QAM
~55 dB	15	32768-QAM (VDSL2/G.fast)

Tones in frequency ranges with high noise or attenuation carry fewer bits or are disabled entirely. This adaptive loading is what makes DSL distance-sensitive -- lines close to the DSLAM have better SNR and higher speeds.

Protocol Stack

[Diagram: see online version]

ATM vs PTM

Transport	Used By	Description
ATM (Asynchronous Transfer Mode)	ADSL, ADSL2+	Fixed 53-byte cells; high overhead
PTM (Packet Transfer Mode)	VDSL2, G.fast	Ethernet-like framing; 64/65-byte encapsulation; lower overhead

Most ADSL deployments use ATM with AAL5 encapsulation. VDSL2 and newer use PTM for better efficiency.

Network Architecture

[Diagram: see online version]

Component	Description
CPE	Customer Premises Equipment (DSL modem/router)
Splitter/Filter	Separates voice (POTS) from data (DSL) frequencies
DSLAM	DSL Access Multiplexer -- terminates DSL lines, aggregates to backhaul
BNG/BRAS	Broadband Network Gateway -- PPPoE termination, authentication, IP assignment

FTTx + DSL (Fiber to the Cabinet)

Modern deployments use fiber to a street cabinet with a DSLAM, then short copper runs to premises. Shorter copper = higher DSL speeds:

[Diagram: see online version]

Initialization

DSL line training (sync) takes 10-60 seconds:

Phase	Description
Handshake	CPE and DSLAM identify capabilities (G.994.1 / G.hs)
Channel Discovery	Measure line characteristics, identify usable tones
Training	Fine-tune equalizer, echo cancellation
Channel Analysis	Determine bits per tone (bit loading)
Exchange	Exchange operating parameters
Showtime	Data transfer begins

Key Performance Factors

Factor	Effect
Line length	Longer = more attenuation = lower speed
Wire gauge	Thicker (24 AWG) better than thin (26 AWG)
Crosstalk	Adjacent DSL lines in the same bundle interfere
Bridge taps	Unterminated wire stubs cause reflections
Noise	AM radio, impulse noise, REIN degrade performance
Temperature	Hot weather increases attenuation

Standards

Document	Title
ITU-T G.992.1	ADSL
ITU-T G.992.3	ADSL2
ITU-T G.992.5	ADSL2+
ITU-T G.993.2	VDSL2
ITU-T G.9701	G.fast
ITU-T G.994.1	Handshake (G.hs)
ITU-T G.997.1	Physical layer management for xDSL

See Also

- PPP / PPPoE -- data link protocol used over most DSL connections
- Ethernet -- delivered over VDSL2 PTM
- DOCSIS -- cable modem alternative to DSL
- RADIUS -- authenticates PPPoE sessions
- T1 / E1 -- legacy digital access (DSL replaced leased lines for many uses)

Transport Layer

Transport layer (Layer 4) protocols for reliable and unreliable delivery.

5 protocols in this section

DCCP -- Datagram Congestion Control Protocol

Standard: RFC 4340 | **Layer:** Transport (Layer 4) | **Wireshark filter:** `dccp`

DCCP is a transport-layer protocol that provides unreliable, congestion-controlled datagram delivery. It fills the gap between TCP (reliable, congestion-controlled) and UDP (unreliable, no congestion control) by adding negotiable congestion control to datagrams without the overhead of reliability. DCCP is designed for applications where timely delivery matters more than completeness -- streaming media, online gaming, VoIP, and real-time telemetry. It uses IP protocol number 33 and supports feature negotiation, allowing endpoints to agree on congestion control mechanisms.

Generic Header

All DCCP packets share a generic header. The X bit determines whether the sequence number is 24 or 48 bits:

[Diagram: see online version]

When X=1 (extended sequence numbers), the sequence number field expands to 48 bits:

[Diagram: see online version]

Key Fields

Field	Size	Description
Source Port	16 bits	Sender's port number
Destination Port	16 bits	Receiver's port number
Data Offset	8 bits	Offset from start of header to payload (in 32-bit words)
CCVal	4 bits	Used by the congestion control mechanism (CCID-specific)
CsCov (Checksum Coverage)	4 bits	Portion of packet covered by checksum (0 = entire packet)
Checksum	16 bits	Internet checksum (covers header + data per CsCov)
Res	3 bits	Reserved, must be zero
Type	4 bits	Packet type (0-9)
X (Extended)	1 bit	0 = 24-bit sequence numbers, 1 = 48-bit sequence numbers
Sequence Number	24 or 48 bits	Packet sequence number

Packet Types

Type	Value	Name	Description
0	0x0	DCCP-Request	Client initiates connection (like TCP SYN)
1	0x1	DCCP-Response	Server accepts connection (like TCP SYN-ACK)
2	0x2	DCCP-Data	Application data
3	0x3	DCCP-Ack	Acknowledgment only (no data)
4	0x4	DCCP-DataAck	Data plus acknowledgment (piggybacked)
5	0x5	DCCP-CloseReq	Server requests connection close
6	0x6	DCCP-Close	Close the connection
7	0x7	DCCP-Reset	Abort the connection (with reason code)
8	0x8	DCCP-Sync	Re-synchronize sequence numbers after loss
9	0x9	DCCP-SyncAck	Acknowledge a Sync

Connection Lifecycle

DCCP uses a three-way handshake for connection setup, similar to TCP:

[Diagram: see online version]

Server-Initiated Close

The server can request the client to initiate the close:

[Diagram: see online version]

Congestion Control IDs (CCIDs)

DCCP's key innovation is pluggable congestion control, negotiated via feature negotiation during connection setup:

CCID	Name	RFC	Behavior
2	TCP-like Congestion Control	RFC 4341	AIMD (additive increase, multiplicative decrease) -- mimics TCP Reno/NewReno
3	TCP-Friendly Rate Control (TFRC)	RFC 4342	Equation-based -- smoother rate, avoids halving; good for streaming

CCID 2 vs CCID 3

Property	CCID 2 (TCP-like)	CCID 3 (TFRC)
Rate response to loss	Halves sending rate (like TCP)	Smoothly adjusts via TCP throughput equation
Burstiness	Bursty (like TCP)	Smooth sending rate
Best for	Bulk data transfer, applications tolerant of rate variation	Streaming media, VoIP, applications needing stable rate
Ack mechanism	Ack Vectors (selective ack)	Loss event rates
Fairness	TCP-fair	TCP-friendly (does not starve TCP)

Feature Negotiation

DCCP uses options in DCCP-Request and DCCP-Response to negotiate features:

Feature	Description
CCID	Congestion Control ID for each half-connection
Allow Short Seqno	Permit 24-bit sequence numbers (X=0)
Sequence Window	Size of valid sequence number window
ECN Incapable	Whether endpoint cannot use ECN
Ack Ratio	How many data packets per Ack (CCID 2)
Send Ack Vector	Whether Ack Vectors are sent (CCID 2)

Each half-connection can use a different CCID -- for example, CCID 2 in one direction and CCID 3 in the other.

Service Codes

DCCP connections are identified by a 32-bit Service Code in the DCCP-Request, allowing a server to demultiplex to the correct application (similar to TCP's role of port numbers, but application-level):

Example	Service Code	Description
SIP (audio)	1717858423	VoIP signaling
RTP (audio)	Application-defined	Media stream

Reset Codes

DCCP-Reset packets include a reason code:

Code	Name	Description
0	Unspecified	No reason given
1	Closed	Normal close

2	Aborted	Connection aborted
3	No Connection	No matching connection
4	Packet Error	Invalid packet
5	Option Error	Invalid option
6	Mandatory Error	Mandatory option not understood
7	Connection Refused	Service Code not available
8	Bad Service Code	Service Code does not match
9	Too Busy	Server overloaded
10	Bad Init Cookie	Init Cookie validation failed
11	Aggression Penalty	Rate exceeded what congestion control allows

DCCP vs TCP vs UDP vs SCTP

Feature	DCCP	TCP	UDP	SCTP
IP Protocol	33	6	17	132
Reliable delivery	No	Yes	No	Yes (per-stream)
Ordered delivery	No	Yes	No	Optional per-stream
Congestion control	Yes (pluggable)	Yes (built-in)	No	Yes
Connection-oriented	Yes	Yes	No	Yes
Multi-homing	No	No	No	Yes
Multi-streaming	No	No	No	Yes
Message boundaries	Yes	No (byte stream)	Yes	Yes
Handshake	3-way	3-way	None	4-way
Use case	Streaming, VoIP, gaming	Web, file transfer	DNS, NTP, gaming	Telephony signaling

Use Cases

Application	Why DCCP
Streaming media	Timely delivery matters more than completeness; smooth rate (CCID 3)
Online gaming	Low-latency datagrams; stale data is worthless
VoIP	Congestion control without reliability overhead
Telemetry	Congestion-safe sensor data without TCP's retransmission delay

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 4340	Datagram Congestion Control Protocol (DCCP)
RFC 4341	Profile for DCCP Congestion Control ID 2: TCP-like
RFC 4342	Profile for DCCP Congestion Control ID 3: TFRC
RFC 5595	The DCCP Service Codes
RFC 5596	DCCP Simultaneous-Open
RFC 6773	DCCP-UDP: A Datagram Congestion Control Protocol UDP Encapsulation for NAT Traversal

See Also

- TCP -- reliable, congestion-controlled byte stream
- UDP -- unreliable datagrams without congestion control
- SCTP -- reliable, multi-stream transport
- QUIC -- modern encrypted transport over UDP
- RTP -- often carried over DCCP for congestion-controlled media

QUIC

Standard: RFC 9000 | **Layer:** Transport (Layer 4) | **Wireshark filter:** `quic`

QUIC is a modern transport protocol built on UDP that provides reliable, multiplexed, encrypted connections with reduced latency. Originally developed by Google and standardized by the IETF, QUIC integrates TLS 1.3 encryption into the transport layer, eliminating the separate TLS handshake. It supports multiple independent streams within a single connection (no head-of-line blocking), connection migration across network changes, and 0-RTT connection resumption. QUIC is the transport for HTTP/3 and is rapidly replacing TCP+TLS for web traffic.

Packet (Short Header -- most data packets)

[Diagram: see online version]

Packet (Long Header -- handshake packets)

[Diagram: see online version]

Key Fields

Field	Size	Description
Header Form	1 bit	0 = Short header, 1 = Long header
Fixed Bit	1 bit	Always 1 (for demuxing from other protocols)
Type (long)	2 bits	Initial, 0-RTT, Handshake, or Retry
Spin Bit (short)	1 bit	Latency measurement signal
Key Phase (short)	1 bit	Indicates which key generation is in use
Version	32 bits	QUIC version (1 = RFC 9000)
Dest Connection ID	0-20 bytes	Identifies the connection at the receiver
Source Connection ID	0-20 bytes	Identifies the connection at the sender (long header only)
Packet Number	1-4 bytes	Per-packet-number-space sequence number
Payload	Variable	Encrypted frames

Long Header Types

Type	Value	Description
Initial	0x00	First handshake packet (carries CRYPTO frames with TLS ClientHello)
0-RTT	0x01	Early data sent before handshake completes
Handshake	0x02	Continues the TLS handshake
Retry	0x03	Server requests address validation

Frame Types

QUIC payloads contain one or more frames:

Type	Name	Description
0x00	PADDING	No-op padding
0x01	PING	Keepalive / elicit ACK
0x02-0x03	ACK	Acknowledge received packets
0x04	RESET_STREAM	Abruptly terminate a stream
0x05	STOP_SENDING	Request peer stop sending on a stream

0x06	CRYPTO	TLS handshake data (replaces TLS record layer)
0x07	NEW_TOKEN	Token for future 0-RTT connection
0x08-0x0F	STREAM	Application data on a stream
0x10	MAX_DATA	Connection-level flow control
0x11	MAX_STREAM_DATA	Stream-level flow control
0x12-0x13	MAX_STREAMS	Limit on number of streams
0x18	NEW_CONNECTION_ID	Provide alternative connection IDs
0x19	RETIRE_CONNECTION_ID	Retire a connection ID
0x1A	PATH_CHALLENGE	Path validation
0x1B	PATH_RESPONSE	Response to path validation
0x1C-0x1D	CONNECTION_CLOSE	Close the connection (with error)
0x1E	HANDSHAKE_DONE	Server confirms handshake complete

Connection Establishment

1-RTT Handshake

[Diagram: see online version]

0-RTT Resumption

[Diagram: see online version]

0-RTT data is vulnerable to replay attacks -- servers must handle this.

Streams

QUIC multiplexes multiple independent streams within a single connection:

Stream ID Bits	Type
Least significant 2 bits = 00	Client-initiated, bidirectional
Least significant 2 bits = 01	Server-initiated, bidirectional
Least significant 2 bits = 10	Client-initiated, unidirectional
Least significant 2 bits = 11	Server-initiated, unidirectional

Key advantage: **no head-of-line blocking**. A lost packet on stream 1 doesn't stall streams 2 and 3 (unlike TCP where all streams share one byte sequence).

Connection Migration

QUIC connections are identified by Connection IDs, not IP:port tuples. When a device changes networks (Wi-Fi -> cellular), it can:

1. Send a PATH_CHALLENGE on the new path
2. Receive PATH_RESPONSE validating the path
3. Continue the connection seamlessly -- no new handshake needed

QUIC vs TCP+TLS

Feature	TCP + TLS 1.3	QUIC
Handshake	2 RTT (TCP + TLS) or 1 RTT (TFO + TLS)	1 RTT (0-RTT for resumption)
Encryption	Optional (STARTTLS)	Mandatory (built-in TLS 1.3)
Multiplexing	Application-level only	Native streams (no HOL blocking)
Connection migration	Not supported	Supported via Connection IDs
Loss recovery	Per-connection	Per-stream
Header encryption	No	Partial (packet number encrypted)
Middlebox ossification	High (TCP options often stripped)	Resistant (encrypted, UDP-based)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 9000	QUIC: A UDP-Based Multiplexed and Secure Transport
RFC 9001	Using TLS to Secure QUIC
RFC 9002	QUIC Loss Detection and Congestion Control
RFC 9114	HTTP/3
RFC 9221	QUIC Datagrams (unreliable)
RFC 9250	DNS over Dedicated QUIC Connections

See Also

- UDP -- QUIC runs over UDP
- TCP -- the transport protocol QUIC aims to replace
- TLS -- QUIC integrates TLS 1.3
- HTTP -- HTTP/3 runs over QUIC

SCTP -- Stream Control Transmission Protocol

Standard: RFC 9260 | **Layer:** Transport (Layer 4) | **Wireshark filter:** `sctp`

SCTP is a reliable, message-oriented transport protocol that combines features of TCP and UDP. It provides reliable delivery like TCP but preserves message boundaries (not a byte stream), supports multiple independent streams within a single association (no head-of-line blocking), and offers multi-homing (multiple IP addresses per endpoint for failover). SCTP is used by SIGTRAN (SS7 over IP), Diameter (LTE AAA), WebRTC data channels (inside DTLS), and some HPC applications.

Packet

[Diagram: see online version]

An SCTP packet has a 12-byte common header followed by one or more chunks.

Key Fields

Field	Size	Description
Source Port	16 bits	Sender's port
Destination Port	16 bits	Receiver's port
Verification Tag	32 bits	Association identifier (anti-spoofing)
Checksum	32 bits	CRC-32c over the entire packet
Chunks	Variable	One or more typed chunks

Chunk Format

[Diagram: see online version]

Chunk Types

Type	Name	Description
0	DATA	User data
1	INIT	Initiate an association
2	INIT ACK	Acknowledge initiation (with cookie)
3	SACK	Selective Acknowledgment
4	HEARTBEAT	Path keepalive

5	HEARTBEAT ACK	Heartbeat response
6	ABORT	Abort the association
7	SHUTDOWN	Graceful close (no more data from sender)
8	SHUTDOWN ACK	Acknowledge shutdown
9	ERROR	Report error conditions
10	COOKIE ECHO	Send cookie for association validation
11	COOKIE ACK	Acknowledge cookie
14	SHUTDOWN COMPLETE	Close complete
0xC0	FORWARD TSN	Skip undeliverable chunks (RFC 3758)

Association Setup (4-Way Handshake)

SCTP uses a 4-way handshake with a cookie to prevent SYN flood attacks:

[Diagram: see online version]

No state is allocated at the responder until the COOKIE ECHO is validated -- DoS resistant by design.

Key Features

Multi-Streaming

Multiple independent streams within one association -- loss on stream 1 doesn't block stream 2:

Feature	TCP	SCTP
Streams	1 byte stream	N independent message streams
HOL blocking	Yes (one lost segment blocks all)	No (only affected stream stalls)
Message boundaries	No (byte stream)	Yes (message-oriented)

Multi-Homing

Each endpoint can have multiple IP addresses. If one path fails, traffic fails over:

[Diagram: see online version]

HEARTBEAT chunks monitor reachability of each path. On failure, traffic switches to an alternate path.

Selective Acknowledgment (SACK)

[Diagram: see online version]

SCTP vs TCP vs UDP

Feature	TCP	UDP	SCTP
Connection	Stream	Connectionless	Association
Reliability	Full	None	Full or partial (PR-SCTP)
Message boundaries	No	Yes	Yes
Multi-streaming	No	N/A	Yes
Multi-homing	No	No	Yes
Head-of-line blocking	Yes	No	No (per stream)
Setup	3-way handshake	None	4-way handshake (cookie)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 9260	Stream Control Transmission Protocol (current)
RFC 3758	Partial Reliability Extension (PR-SCTP)
RFC 5061	Dynamic Address Reconfiguration

See Also

- TCP -- reliable byte-stream transport
- UDP -- unreliable datagram transport
- SS7 -- SIGTRAN carries SS7 over SCTP
- WebRTC -- data channels use SCTP over DTLS

TCP -- Transmission Control Protocol

Standard: RFC 9293 | **Layer:** Transport (Layer 4) | **Wireshark filter:** `tcp`

TCP provides reliable, ordered, connection-oriented byte-stream delivery over IP. It handles segmentation, flow control (sliding window), congestion control, and retransmission of lost data. TCP is the transport behind HTTP, SSH, SMTP, FTP, and most connection-oriented Internet applications.

Header

[Diagram: see online version]

The minimum header size is 20 bytes (Data Offset = 5). Options extend it up to 60 bytes.

Key Fields

Field	Size	Description
Source Port	16 bits	Sender's port number
Destination Port	16 bits	Receiver's port number
Sequence Number	32 bits	Byte offset of the first data byte in this segment
Acknowledgment Number	32 bits	Next sequence number the sender expects to receive
Data Offset	4 bits	Header length in 32-bit words (min 5, max 15)
Reserved	3 bits	Reserved for future use; must be zero
Flags	9 bits	Control flags
Window Size	16 bits	Receiver's available buffer space in bytes
Checksum	16 bits	Error check over header, data, and pseudo-header
Urgent Pointer	16 bits	Offset from sequence number to last urgent data byte
Options	Variable	Optional fields (MSS, window scale, timestamps, SACK, etc.)

Field Details

Flags

[Diagram: see online version]

Flag	Name	Description
NS	ECN-Nonce	ECN nonce concealment protection
CWR	Congestion Window Reduced	Sender reduced its congestion window
ECE	ECN-Echo	ECN capability or congestion experienced
URG	Urgent	Urgent pointer field is valid
ACK	Acknowledgment	Acknowledgment number field is valid

PSH	Push	Receiver should deliver data to application immediately
RST	Reset	Abort the connection
SYN	Synchronize	Initiate a connection (synchronize sequence numbers)
FIN	Finish	Sender has finished sending data

Three-Way Handshake

[Diagram: see online version]

Connection Teardown

[Diagram: see online version]

Common Options

Kind	Length	Name	Description
0	1	End of Options	Marks end of options list
1	1	No-Operation	Padding for alignment
2	4	MSS	Maximum Segment Size (typically 1460 for Ethernet)
3	3	Window Scale	Multiplier for Window Size field (up to 2^14)
4	2	SACK Permitted	Selective Acknowledgment is supported
5	Variable	SACK	Selective Acknowledgment data blocks
8	10	Timestamps	Round-trip time measurement and PAWS

Well-Known Ports

Port	Service
20, 21	FTP
22	SSH
23	Telnet
25	SMTP
53	DNS
80	HTTP
110	POP3

143	IMAP
443	HTTPS (TLS)
993	IMAPS
3389	RDP

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 9293	Transmission Control Protocol (TCP) -- current spec (replaces RFC 793)
RFC 793	Transmission Control Protocol -- original specification
RFC 7323	TCP Extensions for High Performance (window scaling, timestamps)
RFC 2018	TCP Selective Acknowledgment Options (SACK)
RFC 5681	TCP Congestion Control
RFC 6298	Computing TCP's Retransmission Timer
RFC 3168	Explicit Congestion Notification (ECN)

See Also

- UDP
- IPv4
- IPv6
- TLS
- HTTP

UDP -- User Datagram Protocol

Standard: RFC 768 | **Layer:** Transport (Layer 4) | **Wireshark filter:** udp

UDP provides a minimal, connectionless transport service. It adds port-based multiplexing and an optional checksum to IP, but offers no reliability, ordering, or flow control. This simplicity makes UDP ideal for latency-sensitive applications (VoIP, gaming, video streaming), service discovery (DNS, DHCP, mDNS), and as the foundation for protocols that implement their own reliability (QUIC, DTLS).

Header

[Diagram: see online version]

The UDP header is exactly 8 bytes -- one of the simplest headers in the protocol stack.

Key Fields

Field	Size	Description
Source Port	16 bits	Sender's port (optional; 0 if not used)
Destination Port	16 bits	Receiver's port
Length	16 bits	Total datagram size in bytes (header + payload), minimum 8
Checksum	16 bits	Error check over header, data, and pseudo-header (optional in IPv4, mandatory in IPv6)

Field Details

Checksum

The checksum is computed over a pseudo-header (source IP, destination IP, protocol, UDP length), the UDP header, and the payload. In IPv4, a checksum value of 0 means no checksum was computed. In IPv6, the checksum is mandatory per RFC 8200.

Well-Known Ports

Port	Service
53	DNS
67, 68	DHCP
69	TFTP
123	NTP
161, 162	SNMP
443	QUIC / HTTP/3
500	IKE (IPsec)
514	Syslog
1900	SSDP (UPnP)
4500	IPsec NAT Traversal
5353	mDNS

5060	SIP

UDP vs TCP

Feature	UDP	TCP
Connection	Connectionless	Connection-oriented
Reliability	None	Retransmission, ordering
Flow Control	None	Sliding window
Header Size	8 bytes	20-60 bytes
Latency	Lower	Higher (handshake + ACKs)
Broadcast/Multicast	Supported	Not supported

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 768	User Datagram Protocol
RFC 8085	UDP Usage Guidelines
RFC 8200	IPv6 -- mandates UDP checksum

See Also

- TCP
- DNS
- DHCP
- NTP
- IPv4

Tunneling & VPN

Tunneling and VPN protocols for encapsulation and secure transit.

8 protocols in this section

EVPN -- Ethernet VPN

Standard: RFC 7432 | **Layer:** Control Plane (BGP address family) | **Wireshark filter:** `bgp.evpn`

EVPN is a BGP-based control plane for Layer 2 and Layer 3 VPN services over an IP/MPLS or IP/VXLAN fabric. It replaces traditional flood-and-learn with a control-plane-driven MAC (and IP) advertisement model, eliminating most BUM (Broadcast, Unknown unicast, Multicast) flooding. EVPN is the standard overlay control plane for modern data center fabrics (VXLAN/EVPN), service provider Ethernet VPN services, and multi-tenant networks. It supports all-active multihoming, MAC mobility, ARP/ND suppression, and integrated routing and bridging (IRB).

BGP EVPN Address Family

EVPN is carried as a BGP address family using Multiprotocol BGP (MP-BGP):

Parameter	Value
AFI	25 (L2VPN)
SAFI	70 (EVPN)
TCP Port	179 (standard BGP)
Encapsulation	VXLAN (RFC 8365), MPLS, or NVGRE

EVPN NLRI (Route Types)

All EVPN routes are carried in BGP UPDATE messages as MP_REACH_NLRI or MP_UNREACH_NLRI attributes. Each route type serves a distinct function:

Route Type	Name	Purpose
1	Ethernet Auto-Discovery (A-D)	Multihoming -- fast convergence, aliasing, split horizon
2	MAC/IP Advertisement	Advertise MAC and optional IP bindings
3	Inclusive Multicast Ethernet Tag	Discover PEs and build BUM flooding trees
4	Ethernet Segment	Designated Forwarder (DF) election for multihoming
5	IP Prefix	Advertise IP prefixes for inter-subnet routing

Route Type 2 (MAC/IP Advertisement)

The workhorse route type. Each learned MAC (and optionally its IP binding) is advertised via BGP:

[Diagram: see online version]

Field	Size	Description
Route Distinguisher (RD)	8 bytes	Makes the route unique per PE (Type:Value format)
Ethernet Segment Identifier (ESI)	10 bytes	Identifies the multihomed Ethernet segment (0 = single-homed)
Ethernet Tag ID	4 bytes	VLAN or service instance identifier (0 in VLAN-based mode)
MAC Address Length	1 byte	Always 48 (bits) for Ethernet
MAC Address	6 bytes	The learned MAC address
IP Address Length	1 byte	0 (MAC only), 32 (IPv4), or 128 (IPv6)
IP Address	0, 4, or 16 bytes	Host IP for ARP/ND suppression and IRB
MPLS Label 1	3 bytes	VNI/label for Layer 2 (maps to VXLAN VNI)
MPLS Label 2	3 bytes	VNI/label for Layer 3 (optional, for asymmetric IRB)

Route Type 1 (Ethernet Auto-Discovery)

Used for multihoming -- advertised per Ethernet Segment (per-ES) or per EVI (per-VLAN):

[Diagram: see online version]

Function	Scope	Description
Aliasing	Per-EVI	Allows remote PEs to load-balance to all active PEs on a segment
Fast convergence	Per-ES	Mass withdraw all MACs when a link fails
Split horizon	Per-ES	Prevents BUM loops in multihoming

Route Type 3 (Inclusive Multicast)

Advertised by each PE to indicate participation in a given broadcast domain. Used to build ingress replication lists or multicast trees for BUM traffic:

[Diagram: see online version]

Route Type 5 (IP Prefix)

Advertises IP prefixes for inter-subnet (Layer 3) routing across the EVPN fabric:

[Diagram: see online version]

MAC Learning via BGP

In traditional bridging, MACs are learned from the data plane (flood-and-learn). EVPN moves MAC learning to the control plane:

[Diagram: see online version]

BUM Traffic Handling

Broadcast, Unknown unicast, and Multicast traffic is handled via ingress replication (head-end replication) or multicast underlay:

[Diagram: see online version]

Multihoming

EVPN supports a host or CE device being connected to multiple PEs via the same Ethernet Segment:

[Diagram: see online version]

Multihoming Modes

Mode	Description
All-Active	All PEs forward traffic simultaneously -- full load balancing
Single-Active	Only one PE (the DF) forwards -- standby takes over on failure

Designated Forwarder (DF) Election

Route Type 4 (Ethernet Segment route) is exchanged between PEs sharing an ES to elect the Designated Forwarder for each VLAN:

Step	Description
1	Each PE advertises RT-4 with its ESI and Originating Router IP
2	All PEs on the segment discover each other
3	A deterministic algorithm assigns each VLAN to one PE as DF
4	Only the DF forwards BUM traffic for that VLAN (prevents duplicates)

Ethernet Segment Identifier (ESI)

Byte	Description
Type (1 byte)	0x00 = manual, 0x01 = LACP, 0x02 = STP, 0x03 = MAC-based, 0x05 = router ID

Value (9 bytes)	Unique identifier derived from the type-specific method

An ESI of all zeros (Type 0, Value 0) means the CE is single-homed.

ARP/ND Suppression

EVPN PEs cache MAC-IP bindings from RT-2 routes. When a host sends an ARP Request or IPv6 Neighbor Solicitation, the local PE can reply directly (proxy ARP/ND) without flooding:

Benefit	Description
Reduced flooding	ARP broadcasts do not cross the fabric
Lower latency	Local PE responds immediately from its cache
Fabric efficiency	Saves bandwidth, especially in large-scale fabrics

EVPN with VXLAN (RFC 8365)

The most common deployment model for data center fabrics:

Component	Mapping
EVPN Instance (EVI)	One per tenant or per VLAN
VNI	VXLAN Network Identifier (L2 VNI per bridge domain)
VTEP	VXLAN Tunnel Endpoint (the PE)
RT-2 MPLS Label field	Carries the VXLAN VNI value
UDP port	4789 (standard VXLAN)
Route Target	Controls route distribution between VTEPs

Encapsulation

[Diagram: see online version]

EVPN vs VPLS

Feature	EVPN	VPLS
MAC learning	Control plane (BGP)	Data plane (flood-and-learn)
BUM handling	Ingress replication or multicast	Full mesh flooding
All-active multihoming	Yes	No (only single-active)
MAC mobility	Built-in (sequence numbers)	Not supported
ARP suppression	Yes	No
Integrated L3 (IRB)	Yes (RT-5, symmetric/asymmetric)	Separate

Scalability	Superior (reduced flooding)	Limited by BUM flooding
Standards	RFC 7432	RFC 4761 / RFC 4762

Standards

Document	Title
RFC 7432	BGP MPLS-Based Ethernet VPN
RFC 8365	A Network Virtualization Overlay Solution Using EVPN
RFC 9135	Integrated Routing and Bridging in EVPN
RFC 9136	IP Prefix Advertisement in EVPN
RFC 7209	Requirements for Ethernet VPN
RFC 8584	Framework for EVPN Designated Forwarder Election

See Also

- BGP -- the control-plane protocol that carries EVPN routes
- VXLAN -- the most common data-plane encapsulation for EVPN
- Geneve -- alternative overlay encapsulation
- L2TP -- older Layer 2 tunneling
- VLAN 802.1Q -- traditional Layer 2 segmentation

Geneve -- Generic Network Virtualization Encapsulation

Standard: RFC 8926 | **Layer:** Tunneling (Layer 2 over Layer 3) | **Wireshark filter:** geneve

Geneve is a network virtualization overlay protocol designed to be the single flexible encapsulation format that unifies and replaces VXLAN, NVGRE, and STT. Like VXLAN, it encapsulates Layer 2 frames in UDP/IP, but adds a variable-length options field that makes it extensally extensible without protocol revisions. Geneve is the default overlay in Open vSwitch (OVS), AWS VPC networking, and several SDN platforms. It is designed to be "future-proof" -- new features are added via TLV options, not new protocol versions.

Header

[Diagram: see online version]

Field	Size	Description
Version	2 bits	Always 0
Opt Len	6 bits	Length of options in 4-byte units

O (OAM)	1 bit	1 = this is an OAM packet (not data)
C (Critical)	1 bit	1 = contains critical options the receiver must understand
Protocol Type	16 bits	EtherType of the inner payload (0x6558 = Ethernet)
VNI	24 bits	Virtual Network Identifier (same as VXLAN VNI)
Options	Variable	Zero or more TLV options

Option TLV

[Diagram: see online version]

Field	Size	Description
Option Class	16 bits	Namespace for the option (vendor or standard)
Type	5 bits	Option identifier within the class
Length	5 bits	Option data length in 4-byte units
Data	Variable	Option-specific data

Geneve vs VXLAN

Feature	VXLAN	Geneve
Header size	Fixed 8 bytes	8 bytes + variable options
Extensibility	None (flags only)	TLV options (unlimited)
VNI	24 bits	24 bits
OAM flag	No	Yes
Critical option flag	No	Yes
Protocol type	Implicit (always Ethernet)	Explicit (can carry non-Ethernet)
Inner payload	Ethernet only	Ethernet, IPv4, IPv6 (via Protocol Type)
UDP port	4789	6081
Adoption	Widespread	Growing (AWS, OVS, Linux kernel)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 8926	Geneve: Generic Network Virtualization Encapsulation

See Also

- VXLAN -- simpler overlay (Geneve is its extensible successor)
- GRE -- generic tunneling (no VNI)
- L2TP -- Layer 2 tunneling for VPN
- UDP

GTP -- GPRS Tunneling Protocol

Standard: 3GPP TS 29.060 (GTPv1) / 3GPP TS 29.274 (GTPv2) | **Layer:** Tunneling (Layer 2/3) | **Wireshark filter:** `gtp` or `gtpv2`

GTP is the tunneling protocol used in mobile networks (3G, 4G LTE, 5G) to carry user data and signaling between core network elements. It encapsulates subscriber IP packets inside UDP/IP tunnels, identified by Tunnel Endpoint Identifiers (TEIDs), allowing mobile devices to roam between cell sites while maintaining their IP session. GTP has two variants: GTP-C (control plane -- session management) and GTP-U (user plane -- data tunneling).

GTP Variants

Variant	Purpose	Transport	Description
GTP-C (v1)	Control	UDP 2123	Session management in 3G (SGSN<->GGSN)
GTP-C (v2)	Control	UDP 2123	Session management in 4G/5G (MME<->SGW<->PGW)
GTP-U	User data	UDP 2152	Tunnel subscriber IP packets
GTP' (prime)	Charging	TCP/UDP 3386	Charging data records to billing

GTP-U Header

[Diagram: see online version]

Key Fields

Field	Size	Description

Version	3 bits	GTP version (1 for GTPv1-U)
PT	1 bit	Protocol Type -- 1 = GTP, 0 = GTP' (charging)
E	1 bit	Extension header flag
S	1 bit	Sequence number present
PN	1 bit	N-PDU number present
Message Type	8 bits	0xFF = T-PDU (tunneled user data) for GTP-U
Length	16 bits	Payload length (excluding first 8 bytes of header)
TEID	32 bits	Tunnel Endpoint Identifier -- uniquely identifies the tunnel

TEID

The TEID is the key concept in GTP -- it identifies a specific bearer/tunnel at the receiving endpoint. Each side of a tunnel has its own TEID, negotiated during session setup. A single subscriber may have multiple TEIDs for different bearers (e.g., default bearer + dedicated QoS bearer).

GTPv2-C Header

[Diagram: see online version]

GTPv2-C Message Types

Type	Name	Description
1	Echo Request	Path keepalive
2	Echo Response	Path keepalive reply
32	Create Session Request	Establish a new PDN connection (attach/handover)
33	Create Session Response	Response with TEIDs, IP address, QoS
34	Modify Bearer Request	Update tunnel after handover
35	Modify Bearer Response	Handover update confirmed
36	Delete Session Request	Tear down a PDN connection (detach)
37	Delete Session Response	Session removed
170	Release Access Bearers Request	Suspend bearers (idle mode)
171	Release Access Bearers Response	Bearers suspended

4G LTE Architecture with GTP

[Diagram: see online version]

Interface	Protocol	Between	Purpose
S1-U	GTP-U	eNodeB <-> S-GW	User data (bearer traffic)
S5/S8	GTP-U + GTP-C	S-GW <-> P-GW	User data + session control
S11	GTP-C (v2)	MME <-> S-GW	Session management, bearer setup

LTE Attach / Session Establishment

[Diagram: see online version]

Handover (S1-based)

During handover between cell towers, the MME sends a Modify Bearer Request to update the GTP-U tunnel endpoint from the old eNodeB to the new one -- the subscriber's IP session and TEID remain unchanged.

5G and GTP

5G uses a split user/control architecture:

5G Element	Replaces	GTP Interface
gNB (5G base station)	eNodeB	N3 (GTP-U)
UPF (User Plane Function)	S-GW + P-GW user plane	N3/N9 (GTP-U)
SMF (Session Management)	MME + P-GW control plane	N4 (PFCP, not GTP)

GTP-U remains the user plane tunnel in 5G. The control plane replaced GTP-C with PFCP (RFC 8805) between SMF and UPF.

Encapsulation

[Diagram: see online version]

Standards

Document	Title
3GPP TS 29.060	GTPv1 (3G GPRS/UMTS)
3GPP TS 29.274	GTPv2-C (4G LTE / EPC)
3GPP TS 29.281	GTP-U (user plane)
3GPP TS 23.401	EPS (LTE) architecture
3GPP TS 23.501	5G system architecture

See Also

- IPsec -- sometimes used to encrypt GTP tunnels between operators
- L2TP -- alternative Layer 2 tunneling
- VXLAN -- data center tunneling
- UDP -- GTP transport
- SS7 -- legacy mobile signaling (GTP replaced GPRS tunneling over SS7)

L2TP -- Layer 2 Tunneling Protocol

Standard: RFC 2661 (L2TPv2) / RFC 3931 (L2TPv3) | **Layer:** Data Link / Tunneling (Layer 2) | **Wireshark filter:** `l2tp`

L2TP tunnels Layer 2 (PPP) frames across an IP network, creating a virtual point-to-point connection between two endpoints. It was developed by combining Cisco's L2F and Microsoft's PPTP. L2TP itself provides **no encryption** -- it is almost always paired with IPsec (L2TP/IPsec) for secure VPN connections. L2TPv2 tunnels PPP; L2TPv3 extends this to tunnel Ethernet, VLAN, Frame Relay, and other Layer 2 protocols.

L2TPv2 Header

[Diagram: see online version]

Key Fields

Field	Size	Description
T (Type)	1 bit	0 = data message, 1 = control message
L (Length)	1 bit	1 = Length field present (mandatory for control)
S (Sequence)	1 bit	1 = Ns/Nr fields present (mandatory for control)
O (Offset)	1 bit	1 = Offset field present
P (Priority)	1 bit	1 = priority processing for this control message
Version	4 bits	Always 2 for L2TPv2
Length	16 bits	Total message length (optional for data)
Tunnel ID	16 bits	Identifies the tunnel (locally significant)
Session ID	16 bits	Identifies the session within the tunnel

Ns	16 bits	Sequence number of this message (control)
Nr	16 bits	Expected next sequence number (control)
Payload	Variable	PPP frame (data messages) or AVPs (control messages)

Tunnel Establishment

[Diagram: see online version]

Control Message Types

Type	Name	Description
1	SCCRQ	Start tunnel (request)
2	SCCRP	Start tunnel (reply)
3	SCCCN	Start tunnel (connected)
4	StopCCN	Close the tunnel
6	HELLO	Tunnel keepalive
10	ICRQ	Incoming call (request)
11	ICRP	Incoming call (reply)
12	ICCN	Incoming call (connected)
14	CDN	Call Disconnect Notify
7	OCRQ	Outgoing call (request)
8	OCRP	Outgoing call (reply)
9	OCCN	Outgoing call (connected)
20	SLI	Set-Link-Info (PPP negotiation update)

L2TP Roles

Role	Name	Description
LAC	L2TP Access Concentrator	Near the user -- accepts PPP connections and tunnels them to the LNS
LNS	L2TP Network Server	Remote end -- terminates PPP sessions, assigns IP addresses

Voluntary vs Compulsory Tunneling

Mode	Description
Compulsory	ISP's NAS (LAC) tunnels PPP to the enterprise LNS -- user is unaware
Voluntary	User's device acts as the LAC -- software L2TP client on the endpoint

L2TP/IPsec

L2TP alone is unencrypted. The standard VPN combination wraps L2TP in IPsec:

[Diagram: see online version]

Layer	Protocol
Outer IP	IP endpoints of the tunnel
IPsec ESP	Encryption + authentication
UDP 1701	L2TP transport
L2TP	Tunnel/session framing
PPP	Point-to-point framing
Inner IP	Original user traffic

L2TPv3

L2TPv3 (RFC 3931) extends L2TP beyond PPP:

Feature	L2TPv2	L2TPv3
Payload	PPP only	Ethernet, VLAN, FR, ATM, HDLC, PPP
Transport	UDP	UDP or directly over IP (protocol 115)
Tunnel/Session ID	16-bit	32-bit
Cookies	No	Yes (for spoofing protection)
Use case	Remote access VPN	L2 pseudowires, carrier Ethernet

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 2661	Layer Two Tunneling Protocol (L2TPv2)

RFC 3931	Layer Two Tunneling Protocol Version 3 (L2TPv3)
RFC 3193	Securing L2TP using IPsec

See Also

- PPP -- the Layer 2 protocol L2TPv2 tunnels
- IPsec -- provides encryption for L2TP/IPsec VPN
- GRE -- alternative tunneling protocol
- VXLAN -- modern L2 overlay tunneling

OpenVPN

Standard: OpenVPN Protocol | **Layer:** Application / VPN (Layer 3 or Layer 2) | **Wireshark filter:** openvpn

OpenVPN is the most widely deployed open-source VPN protocol. It uses SSL/TLS for key exchange and authentication, then encrypts data channel traffic with a symmetric cipher (AES-256-GCM by default). OpenVPN operates in two modes: TUN (Layer 3, routing IP packets) and TAP (Layer 2, bridging Ethernet frames). It runs over UDP (preferred, port 1194) or TCP (port 443 for firewall bypass) and is available on all major platforms. OpenVPN uses a split-channel design: a reliable TLS-based control channel for handshake and key negotiation, and a fast data channel for encrypted tunnel traffic.

Packet Structure

All OpenVPN packets begin with a 1-byte opcode/key_id field:

[Diagram: see online version]

Opcodes

Value	Name	Description
1	P_CONTROL_HARD_RESET_CLIENT_V1	Initial client handshake (deprecated)
2	P_CONTROL_HARD_RESET_SERVER_V1	Initial server handshake (deprecated)
3	P_CONTROL_SOFT_RESET_V1	Renegotiate keys on existing session
4	P_CONTROL_V1	Control channel packet (carries TLS records)
5	P_ACK_V1	Acknowledgment for control channel reliability

6	P_DATA_V1	Data channel packet (encrypted tunnel traffic)
7	P_CONTROL_HARD_RESET_CLIENT_V2	Initial client handshake (current)
8	P_CONTROL_HARD_RESET_SERVER_V2	Initial server handshake (current)
9	P_DATA_V2	Data channel packet with peer-id (current)
10	P_CONTROL_HARD_RESET_CLIENT_V3	Client handshake with tls-crypt-v2 metadata

Control Channel Packet (P_CONTROL_V1)

[Diagram: see online version]

Data Channel Packet (P_DATA_V2)

[Diagram: see online version]

Key Fields

Field	Size	Description
Opcode	5 bits	Packet type (control, data, reset, ack)
Key ID	3 bits	Identifies which key set is in use (0-7, allows renegotiation overlap)
Session ID	8 bytes	Unique session identifier per endpoint
HMAC	20 bytes	HMAC-SHA1 authentication (when tls-auth is enabled)
Packet ID	4 bytes	Sequence number for replay protection and reliability
Peer ID	3 bytes	Server-assigned peer identifier (P_DATA_V2 only)
Encrypted Payload	Variable	AES-256-GCM (or other cipher) encrypted tunnel data
AEAD Tag	16 bytes	GCM authentication tag (in AEAD mode)

Connection Establishment

[Diagram: see online version]

Data Flow

[Diagram: see online version]

Control Channel Protection

Option	Description
tls-auth	Pre-shared HMAC key authenticates control packets; prevents DoS and port scanning
tls-crypt	Encrypts and authenticates the entire control channel; hides TLS handshake from DPI
tls-crypt-v2	Per-client tls-crypt keys wrapped with server key; scalable tls-crypt

TUN vs TAP Modes

Feature	TUN (Layer 3)	TAP (Layer 2)
Tunnels	IP packets	Ethernet frames
Use case	Routed VPN (most common)	Bridged VPN (LAN extension)
Broadcast	No	Yes (ARP, DHCP, NetBIOS)
Protocols	IPv4, IPv6 only	Any Ethernet protocol
Overhead	Lower	Higher (Ethernet header)
OS support	All platforms	Limited on some mobile platforms

OpenVPN vs WireGuard vs IPsec

Feature	OpenVPN	WireGuard	IPsec (IKEv2)
Codebase	~100,000 lines	~4,000 lines	400,000+ lines
Cipher negotiation	Yes (configurable)	None (fixed)	Yes (extensive)
Transport	UDP or TCP	UDP only	ESP (IP 50) or UDP
Firewall bypass	TCP 443 (looks like HTTPS)	UDP only	NAT-T (UDP 4500)
Architecture	Userspace	Kernel (Linux 5.6+)	Kernel (xfrm)
Performance	Good (userspace overhead)	Excellent	Good
Layer 2 support	Yes (TAP mode)	No (Layer 3 only)	L2TP/IPsec
Certificate PKI	Full X.509 PKI	Simple public keys	Full X.509 PKI
NAT traversal	Built-in	Built-in	MOBIKE
Default port	UDP 1194	UDP 51820	UDP 500/4500
DPI resistance	tls-crypt hides handshake	Identifiable	Identifiable

Encapsulation

[Diagram: see online version]

Standards

Document	Title
OpenVPN Protocol	OpenVPN protocol specification
OpenVPN Source	Reference implementation
RFC 5246	TLS 1.2 (used by OpenVPN control channel)

See Also

- WireGuard -- modern VPN with fixed cryptography and kernel integration
- L2TP -- Layer 2 tunneling, often paired with IPsec
- TLS -- OpenVPN control channel runs TLS inside its own reliability layer
- GRE -- unencrypted tunneling

SOCKS5 -- Socket Secure Protocol Version 5

Standard: RFC 1928 | **Layer:** Session (Layer 5) | **Wireshark filter:** `socks`

SOCKS is a general-purpose proxy protocol that relays TCP and UDP traffic through an intermediary server on behalf of a client. Unlike HTTP proxies, SOCKS operates below the application layer and is protocol-agnostic -- it can tunnel any TCP or UDP traffic without understanding the payload. SOCKS5 (RFC 1928) added authentication, IPv6, and UDP support over the earlier SOCKS4. The default port is **1080/tcp**. SOCKS5 is widely used for SSH dynamic port forwarding (`ssh -D`), Tor anonymity circuits, and corporate proxy gateways.

Greeting (Method Negotiation)

The client opens a TCP connection to the SOCKS server and sends a greeting listing supported authentication methods. The server selects one.

Client Greeting

[Diagram: see online version]

Server Choice

[Diagram: see online version]

Field	Size	Description
Version	1 byte	SOCKS version: `0x05`
NMethods	1 byte	Number of authentication methods the client supports
Methods	1-255 bytes	List of supported method identifiers
Method (reply)	1 byte	Server's chosen method (or `0xFF` to reject)

Authentication Methods

Value	Method	Description
`0x00`	No Authentication	No auth required
`0x01`	GSSAPI	GSS-API authentication (RFC 1961)
`0x02`	Username/Password	Username/password sub-negotiation (RFC 1929)
`0x03`-`0x7F`	IANA Assigned	Reserved for IANA-assigned methods
`0x80`-`0xFE`	Private Methods	Reserved for private use
`0xFF`	No Acceptable Methods	Server rejects all offered methods

Username/Password Authentication (RFC 1929)

If the server selects method `0x02`, a sub-negotiation occurs:

Auth Request

[Diagram: see online version]

Auth Response

[Diagram: see online version]

Field	Size	Description
Version	1 byte	Sub-negotiation version: `0x01`
ULen	1 byte	Username length
Username	1-255 bytes	Username string
PLen	1 byte	Password length
Password	1-255 bytes	Password string
Status	1 byte	`0x00` = success, any other = failure

CONNECT Flow

[Diagram: see online version]

Request

After authentication succeeds, the client sends a connection request:

[Diagram: see online version]

Field	Size	Description
Version	1 byte	`0x05`
CMD	1 byte	Command code
Reserved	1 byte	`0x00`
ATYP	1 byte	Address type of the destination
DST.ADDR	Variable	Destination address
DST.PORT	2 bytes	Destination port (network byte order)

Commands (CMD)

Value	Command	Description
`0x01`	CONNECT	Establish a TCP connection to the destination
`0x02`	BIND	Listen for an inbound TCP connection (for FTP-like protocols)
`0x03`	UDP ASSOCIATE	Establish a UDP relay

Address Types (ATYP)

Value	Type	Address Format
`0x01`	IPv4	4 bytes (e.g., `C0 A8 01 01` = 192.168.1.1)
`0x03`	Domain Name	1 byte length + domain string (e.g., `0B 65 78 61 6D 70 6C 65 2E 63 6F 6D` = example.com)
`0x04`	IPv6	16 bytes

Reply

[Diagram: see online version]

Field	Size	Description
Version	1 byte	`0x05`
REP	1 byte	Reply status code
Reserved	1 byte	`0x00`
ATYP	1 byte	Address type of the bound address
BND.ADDR	Variable	Server-bound address
BND.PORT	2 bytes	Server-bound port

Reply Codes (REP)

Value	Status	Description
`0x00`	Succeeded	Request granted
`0x01`	General failure	General SOCKS server failure
`0x02`	Not allowed	Connection not allowed by ruleset
`0x03`	Network unreachable	Network unreachable
`0x04`	Host unreachable	Host unreachable
`0x05`	Connection refused	Connection refused by destination
`0x06`	TTL expired	TTL expired
`0x07`	Command not supported	Command not supported
`0x08`	Address type not supported	Address type not supported

UDP ASSOCIATE

For UDP relay, the client sends a `UDP ASSOCIATE` request over TCP. The proxy replies with a UDP relay address/port. The client then sends UDP datagrams encapsulated with a SOCKS5 UDP header to the relay:

[Diagram: see online version]

UDP Request Header

[Diagram: see online version]

Field	Size	Description
Reserved	2 bytes	`0x0000`
FRAG	1 byte	Fragment number (0 = standalone, no fragmentation)
ATYP	1 byte	Address type
DST.ADDR	Variable	Target address
DST.PORT	2 bytes	Target port

Data	Variable	UDP payload

SOCKS4 vs SOCKS4a vs SOCKS5

Feature	SOCKS4	SOCKS4a	SOCKS5
RFC	None (NEC)	None (NEC)	RFC 1928
Authentication	User ID only	User ID only	Pluggable (none, user/pass, GSSAPI, ...)
Address types	IPv4 only	IPv4 + domain names	IPv4, IPv6, domain names
UDP support	No	No	Yes (UDP ASSOCIATE)
BIND command	Yes	Yes	Yes
Domain resolution	Client-side	Server-side (proxy resolves)	Server-side
IPv6	No	No	Yes

SOCKS4a extended SOCKS4 by allowing the client to send a domain name instead of an IP, letting the proxy perform DNS resolution. SOCKS5 formalized and extended this with a full address type system.

Common Use Cases

Use Case	Description
SSH dynamic tunneling	`ssh -D 1080 user@host` creates a local SOCKS5 proxy
Tor	Tor exposes a SOCKS5 interface (default 9050) for application tunneling
Corporate proxies	Route application traffic through monitored gateways
Browser proxy	Firefox, Chrome support SOCKS5 proxy configuration
Application tunneling	`tsocks`, `proxychains` force applications through SOCKS

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 1928	SOCKS Protocol Version 5
RFC 1929	Username/Password Authentication for SOCKS V5
RFC 1961	GSS-API Authentication Method for SOCKS V5

See Also

- HTTP -- HTTP CONNECT method provides application-layer proxying
- SSH -- `ssh -D` creates a SOCKS5 proxy via SSH tunnel
- WireGuard -- VPN-level tunneling alternative
- TLS -- SOCKS does not encrypt; often combined with TLS

Tor -- The Onion Router

Standard: Tor Protocol Specification (tor-spec.txt) | **Layer:** Application / Overlay (Layer 7) | **Wireshark filter:** `ssl || tls`

Tor is an anonymous overlay network protocol that routes traffic through a series of encrypted relays to conceal a user's identity and location. It uses a cell-based protocol with fixed 514-byte cells transported inside TLS connections between relays. Clients build three-hop circuits (Guard, Middle, Exit) using telescoping encryption --- each relay peels one layer of AES-CTR encryption, so no single relay knows both the source and destination. Tor powers the Tor Browser, onion services (.onion), and censorship circumvention tools. Over 6,000 volunteer relays carry traffic for millions of daily users.

Cell Format

Every Tor cell is exactly 514 bytes (fixed-size to resist traffic analysis):

[Diagram: see online version]

For link protocol versions below 4, the Circuit ID is 2 bytes (making cells 512 bytes).

Key Fields

Field	Size	Description
Circuit ID	4 bytes	Identifies the circuit on this link (2 bytes for link protocol < 4)
Command	1 byte	Cell command type (see table below)
Payload	509 bytes	Cell content (padded to fixed size)

Cell Commands

Command	Value	Description
PADDING	0	Keepalive / traffic padding (ignored by receiver)

CREATE	1	Create a circuit (TAP handshake)
CREATED	2	Acknowledge circuit creation
RELAY	3	Encrypted relay cell (carries end-to-end data)
DESTROY	4	Tear down a circuit
CREATE_FAST	5	Create circuit with fast handshake (one-hop, non-PFS)
CREATED_FAST	6	Acknowledge fast circuit creation
VERSIONS	7	Link protocol version negotiation
NETINFO	8	Exchange IP address and timestamp info
RELAY_EARLY	9	Relay cell during circuit extension (limited to 8 per circuit)
CREATE2	10	Create circuit (ntor handshake, modern)
CREATED2	11	Acknowledge ntor circuit creation
PADDING_NEGOTIATE	12	Negotiate padding parameters
VPADDING	128	Variable-length padding cell
CERTS	129	TLS certificate exchange
AUTH_CHALLENGE	130	Authentication challenge from responder
AUTHENTICATE	131	Authentication response from initiator
AUTHORIZE	132	Reserved for future authorization

Commands 128+ are variable-length cells (length field replaces fixed 509-byte payload).

Relay Cell Format

RELAY and RELAY_EARLY cells carry an inner relay header within the 509-byte payload:

[Diagram: see online version]

Field	Size	Description
Relay Command	1 byte	Relay sub-command (see table below)
Recognized	2 bytes	Set to 0 when cell is for this hop (else encrypted garbage)
Stream ID	2 bytes	Identifies the TCP stream within the circuit
Digest	4 bytes	Running SHA-1 digest for integrity verification
Length	2 bytes	Length of the data portion
Data	498 bytes	Relay payload (padded to fill cell)

Relay Commands

Command	Value	Description
RELAY_BEGIN	1	Open a TCP stream to a destination (from client via exit)
RELAY_DATA	2	Carry TCP stream data
RELAY_END	3	Close a TCP stream
RELAY_CONNECTED	4	Confirm stream connection succeeded
RELAY_SENDME	5	Flow control acknowledgment (window-based)
RELAY_EXTEND	6	Extend circuit by one hop (TAP handshake)
RELAY_EXTENDED	7	Confirm circuit extension
RELAY_TRUNCATE	8	Tear down part of a circuit
RELAY_TRUNCATED	9	Confirm partial circuit teardown
RELAY_DROP	10	Padding cell (ignored at destination)
RELAY_RESOLVE	11	DNS resolve request through exit
RELAY_RESOLVED	12	DNS resolve response
RELAY_BEGIN_DIR	13	Open a stream to a relay's directory port
RELAY_EXTEND2	14	Extend circuit by one hop (ntor handshake, modern)
RELAY_EXTENDED2	15	Confirm ntor circuit extension

Circuit Building

Tor clients build circuits using telescoping creation --- each hop is added one at a time, so each relay only knows its immediate neighbors:

[Diagram: see online version]

Onion Encryption

Each hop shares a unique AES-128-CTR key with the client. When the client sends data:

Direction	Encryption Layers
Client sends	Encrypt with Exit key, then Middle key, then Guard key
Guard receives	Decrypt Guard layer, forward to Middle
Middle receives	Decrypt Middle layer, forward to Exit
Exit receives	Decrypt Exit layer, send plaintext to destination

No single relay can observe both the source IP and the destination. The Guard knows who you are but not where you are going. The Exit knows where traffic is going but not who sent it.

Hidden Services (Onion Services v3)

Onion services (.onion addresses) allow servers to receive connections without revealing their IP address. v3 uses Ed25519 keys and produces 56-character addresses.

[Diagram: see online version]

v3 Onion Address Structure

The 56-character .onion address encodes:

Component	Size	Description
Public Key	32 bytes	Ed25519 public key of the hidden service
Checksum	2 bytes	SHA-3 checksum
Version	1 byte	Always 0x03 for v3

Directory System

Tor relies on a set of trusted directory authorities to distribute relay information:

Component	Description
Directory Authorities	9 hardcoded trusted servers that vote on network consensus
Consensus Document	Signed list of all relays, updated hourly
Relay Descriptors	Self-published relay capabilities, keys, policies
Bandwidth Authorities	Measure relay speeds for load balancing
Bridge Authority	Maintains list of unpublished bridge relays

Entry Guards and Bridges

Concept	Description
Entry Guard	Persistent first hop (rotated infrequently) to limit exposure to malicious relays
Bridge Relay	Unpublished relay used to circumvent censorship when direct Tor access is blocked
Pluggable Transports	Obfuscation layers (obfs4, Snowflake, meek) that disguise Tor traffic as normal HTTPS

Tor vs VPN

Feature	Tor	VPN
Anonymity model	Distributed trust across 3+ relays	Single trust point (VPN provider)
Encryption layers	3 (one per hop)	1 (client to VPN server)
Knows your IP	Guard relay only	VPN provider
Knows your destination	Exit relay only	VPN provider
Speed	Slower (multiple hops, volunteer relays)	Faster (direct, commercial infrastructure)
Latency	Higher (3+ hops across the globe)	Lower (single hop)
Hidden services	Yes (.onion)	No
Protocol	Cell-based overlay over TLS	Tunnel (WireGuard, OpenVPN, IPsec)
Censorship resistance	Bridges, pluggable transports	Server IP can be blocked
Typical use	Anonymity, censorship circumvention	Privacy, geo-unblocking, corporate access

Encapsulation

[Diagram: see online version]

Standards

Document	Title
tor-spec.txt	Tor Protocol Specification
dir-spec.txt	Tor Directory Protocol Specification
rend-spec-v3.txt	Tor Rendezvous (Hidden Service) Specification v3
pt-spec.txt	Pluggable Transport Specification

See Also

- TLS --- outer transport encryption for Tor links
- WireGuard --- modern VPN (different threat model)
- L2TP --- tunneling protocol (no anonymity)
- GRE --- simple tunneling alternative

VXLAN -- Virtual Extensible LAN

Standard: RFC 7348 | **Layer:** Data Link / Tunneling (Layer 2 over Layer 3) | **Wireshark filter:** `vxlan`

VXLAN is an overlay tunneling protocol that encapsulates Layer 2 Ethernet frames within UDP/IP packets, allowing Layer 2 networks to be extended across Layer 3 boundaries. It was designed to solve the scalability limitations of VLANs (4094 IDs) in large data centers by providing a 24-bit segment ID space (over 16 million virtual networks). VXLAN is the dominant overlay protocol in modern data center fabrics and cloud networking, used by VMware NSX, Linux bridges, Cisco ACI, and most cloud providers.

Packet Format

[Diagram: see online version]

The 8-byte VXLAN header encapsulates a complete inner Ethernet frame:

Full Encapsulation Stack

[Diagram: see online version]

Key Fields

Field	Size	Description
Flags	8 bits	Bit 3 (I flag) must be 1; indicates valid VNI
Reserved	24 bits	Must be zero
VNI	24 bits	VXLAN Network Identifier (0 - 16,777,215)
Reserved	8 bits	Must be zero

Flags Detail

[Diagram: see online version]

Only the **I flag** (bit 3) is defined. When set to 1, the VNI field is valid. All other bits are reserved and set to 0.

Outer UDP Header

Field	Value	Description
Source Port	Hash-based	Hash of inner frame headers (for ECMP load balancing)
Destination Port	4789	IANA-assigned VXLAN port
Length	Variable	UDP length including VXLAN header and inner frame

Checksum	0 or computed	Often set to 0 (inner frame has its own FCS)

The source port is derived from a hash of the inner frame's headers (src/dst MAC, IP, L4 ports), ensuring that flows between the same endpoints take the same path while different flows are distributed across ECMP paths.

How VXLAN Works

[Diagram: see online version]

Term	Description
VTEP	VXLAN Tunnel Endpoint -- encapsulates/decapsulates VXLAN
VNI	VXLAN Network Identifier -- isolates virtual networks (like VLAN ID but 24-bit)
Underlay	Physical IP network connecting VTEPs
Overlay	Virtual L2 network carried inside VXLAN

Traffic Flow

[Diagram: see online version]

MAC Learning

VTEPs learn remote MAC-to-VTEP mappings through:

Method	Description
Data plane learning	Inspect source MAC of received VXLAN packets (flood-and-learn)
Multicast	Unknown unicast/broadcast/multicast flooded via IP multicast group per VNI
EVPN (BGP)	Control plane distributes MAC/IP->VTEP mappings (no flooding needed)
Static	Manual VTEP-MAC configuration

EVPN (RFC 7432) is the modern preferred approach -- it eliminates flooding and provides efficient ARP suppression.

VXLAN vs VLAN

Feature	VLAN (802.1Q)	VXLAN
ID space	12 bits (4,094)	24 bits (16,777,216)
Scope	Single L2 domain	Across L3 boundaries
Encapsulation	4-byte tag in Ethernet header	Full UDP/IP encapsulation

Scalability	Limited by STP, MAC table size	Data center scale
Multi-tenancy	Limited	Native (millions of segments)
MTU overhead	4 bytes	50 bytes (requires jumbo frames)

MTU Considerations

VXLAN adds 50 bytes of overhead:

Component	Size
Outer Ethernet	14 bytes
Outer IP	20 bytes
Outer UDP	8 bytes
VXLAN header	8 bytes
Total overhead	**50 bytes**

With a standard 1500-byte MTU, the inner frame MTU is 1450 bytes. Most data center fabrics use **jumbo frames (9000+ byte MTU)** on the underlay to avoid fragmentation.

VXLAN-GPE (Generic Protocol Extension)

RFC 8926 extends VXLAN with a Next Protocol field, allowing encapsulation of non-Ethernet payloads (IPv4, IPv6, NSH).

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 7348	Virtual eXtensible Local Area Network (VXLAN)
RFC 8365	A Network Virtualization Overlay Solution Using EVPN
RFC 7432	BGP MPLS-Based Ethernet VPN (EVPN)
RFC 8926	Geneve -- Generic Network Virtualization Encapsulation

See Also

- Ethernet -- the inner frames VXLAN encapsulates
- UDP -- VXLAN transport
- GRE -- alternative tunneling protocol

- L2TP -- another L2 tunneling protocol
- MPLS -- carrier-grade L2/L3 VPN alternative
- BGP -- EVPN control plane for VXLAN

VoIP & Real-Time

Voice over IP, real-time communication, and session control protocols.

16 protocols in this section

DTLS -- Datagram Transport Layer Security

Standard: RFC 9147 (DTLS 1.3) / RFC 6347 (DTLS 1.2) | **Layer:** Presentation / Security (Layer 6) | **Wireshark filter:** `dtls`

DTLS adapts TLS for unreliable datagram transport (UDP). It provides the same encryption, authentication, and integrity guarantees as TLS but handles packet loss, reordering, and duplication that are inherent to UDP. DTLS is critical for WebRTC (DTLS-SRTP key exchange), VPN protocols (OpenConnect, Cisco AnyConnect), CoAP (IoT), and any application needing encrypted UDP.

Record

[Diagram: see online version]

Field	Size	Description
Content Type	8 bits	Same as TLS (22=Handshake, 23=Application Data, 21=Alert, 25=ACK in 1.3)
Protocol Version	16 bits	0xFEFD = DTLS 1.2, 0xFEFC = DTLS 1.3 (inverted from TLS)
Epoch	16 bits	Increments on each key change (cipher state)
Sequence Number	48 bits	Per-epoch sequence (anti-replay)
Length	16 bits	Fragment length
Fragment	Variable	Encrypted payload

Key Differences from TLS

Feature	TLS	DTLS
Transport	TCP (reliable, ordered)	UDP (unreliable, unordered)
Record numbering	Implicit (TCP ordering)	Explicit epoch + sequence number
Retransmission	TCP handles it	DTLS handshake has its own retransmit timers
Fragmentation	TCP handles it	DTLS fragments handshake messages
Replay protection	TCP ordering	Sliding window on sequence numbers
Connection ID	Not in TLS 1.3	Optional (RFC 9146) -- survives NAT rebinding

Handshake

DTLS adds retransmission, fragmentation, and cookie exchange to the TLS handshake:

DTLS 1.2 Handshake

[Diagram: see online version]

The HelloVerifyRequest/cookie exchange is a DoS mitigation -- the server doesn't allocate state until the client proves it can receive at its claimed address.

DTLS 1.3 Handshake

[Diagram: see online version]

Handshake Message Fragmentation

DTLS handshake messages include additional fields for fragmentation:

[Diagram: see online version]

Field	Description
Message Sequence	Orders handshake messages (survives reordering)
Fragment Offset	Byte offset of this fragment within the full message
Fragment Length	Length of this fragment

DTLS-SRTP (WebRTC)

In WebRTC, DTLS provides key exchange for SRTP media encryption:

[Diagram: see online version]

The DTLS fingerprint is exchanged in SDP (`a=fingerprint:sha-256 ...`) and verified during the handshake -- this provides end-to-end authentication without a certificate authority.

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 9147	DTLS 1.3
RFC 6347	DTLS 1.2
RFC 5764	DTLS-SRTP (WebRTC key exchange)

RFC 9146	DTLS Connection ID

See Also

- TLS -- TCP equivalent
- WebRTC -- primary consumer of DTLS-SRTP
- SRTP -- media encryption keyed by DTLS
- UDP

H.323 -- Packet-Based Multimedia Communications

Standard: ITU-T H.323 | **Layer:** Application (Layer 7) | **Wireshark filter:** h225 or h245

H.323 is an ITU-T umbrella standard for real-time audio, video, and data communication over packet networks. It was the dominant VoIP standard before SIP and is still widely deployed in video conferencing (Polycom/Poly, Cisco, legacy systems), carrier interworking, and some PBX systems. H.323 is actually a suite of protocols -- H.225.0 for call signaling and registration (RAS), H.245 for media capability negotiation, and RTP/RTCP for media transport.

Protocol Suite

[Diagram: see online version]

Components

Component	Description
Terminal	Endpoint (phone, video codec, softphone)
Gatekeeper	Call admission, address resolution, bandwidth control (optional)
Gateway	Bridges H.323 to PSTN, SIP, or other networks
MCU	Multipoint Control Unit -- manages multi-party conferences

Call Signaling (H.225.0 / Q.931)

H.225.0 uses Q.931-style messages (ASN.1/PER encoded) over TCP port 1720:

Messages

Message	Description

Setup	Initiate a call
Call Proceeding	Call is being routed
Alerting	Destination is ringing
Connect	Call answered
Release Complete	Call terminated
Facility	Supplementary services, redirect

Basic Call Flow

[Diagram: see online version]

RAS (Registration, Admission, Status)

RAS uses UDP port 1719 between terminals and the gatekeeper:

Message	Abbreviation	Description
Gatekeeper Request	GRQ	Terminal discovers a gatekeeper
Gatekeeper Confirm	GCF	Gatekeeper responds to discovery
Registration Request	RRQ	Terminal registers with gatekeeper
Registration Confirm	RCF	Registration accepted
Admission Request	ARQ	Request permission to make/receive a call
Admission Confirm	ACF	Call admitted (bandwidth allocated)
Admission Reject	ARJ	Call denied (bandwidth exceeded, policy)
Disengage Request	DRQ	Call is ending
Disengage Confirm	DCF	Acknowledged
Bandwidth Request	BRQ	Request bandwidth change mid-call
Location Request	LRQ	Resolve an H.323 address
Location Confirm	LCF	Address resolved

H.245 (Control Channel)

H.245 negotiates media capabilities and opens logical channels for RTP:

Message	Description
Terminal Capability Set (TCS)	Exchange supported codecs and capabilities
TCS Ack	Acknowledge capabilities

Open Logical Channel (OLC)	Open an RTP stream (codec, port, direction)
OLC Ack	Accept the channel (with receive port)
Close Logical Channel (CLC)	Close an RTP stream
Master Slave Determination (MSD)	Determine who controls conference parameters

H.323 vs SIP

Feature	H.323	SIP
Origin	ITU-T (telecom)	IETF (Internet)
Encoding	ASN.1 PER (binary)	UTF-8 text
Signaling	H.225.0 (TCP 1720)	SIP (UDP/TCP 5060)
Media negotiation	H.245 (separate TCP channel)	SDP (inline in SIP)
Gatekeeper	Optional (address resolution, admission)	Proxy/Registrar
Complexity	High (multiple protocols, ASN.1)	Lower (single text protocol + SDP)
NAT traversal	Difficult (multiple TCP/UDP ports)	Difficult (but ICE/STUN/TURN help)
Video conferencing	Strong (MCU, multi-party native)	Weaker (conference focus model)
Current adoption	Legacy, declining	Dominant

Encapsulation

[Diagram: see online version]

Standards

Document	Title
ITU-T H.323	Packet-based multimedia communications systems
ITU-T H.225.0	Call signalling protocols and media stream packetization
ITU-T H.245	Control protocol for multimedia communication

See Also

- SIP -- the protocol that largely replaced H.323
- RTP -- media transport used by H.323
- RTCP -- media quality feedback
- ISDN -- H.225.0 is based on Q.931 (ISDN signaling)
- TCP

IAX -- Inter-Asterisk eXchange

Standard: RFC 5765 (IAX2) | **Layer:** Application (Layer 7) | **Wireshark filter:** `iax2`

IAX (Inter-Asterisk eXchange, version 2) is a VoIP protocol developed by Digium for the Asterisk PBX. Its key advantage is that it multiplexes signaling and media on a single UDP port (4569), making it far simpler to traverse NATs and firewalls than SIP+RTP (which needs multiple ports). IAX uses a compact binary format, supports trunking (multiple calls on one connection with reduced overhead), and handles registration, call setup, and media in a single protocol.

Frame Types

Full Frame (signaling and first media)

[Diagram: see online version]

Mini Frame (subsequent media -- compact)

[Diagram: see online version]

Mini frames are only 4 bytes of overhead -- significantly less than RTP (12 bytes).

Key Fields

Field	Size	Description
F bit	1 bit	1 = Full frame, 0 = Mini frame
Source Call Number	15 bits	Identifies the call at the sender
R bit	1 bit	Retransmission flag
Dest Call Number	15 bits	Identifies the call at the receiver
Timestamp	32 bits (full) / 16 bits (mini)	Milliseconds
OSeqno	8 bits	Outgoing sequence number (reliability)
ISeqno	8 bits	Incoming sequence number (acknowledgment)
Frame Type	8 bits	Category of frame
Subclass	8 bits	Specific message within the type

Frame Types

Type	Name	Description

0x01	DTMF Begin	Start of a DTMF digit
0x02	Voice	Audio data
0x03	Video	Video data
0x04	Control	Call control (hangup, ringing, answer, etc.)
0x05	Null	Ignored (padding)
0x06	IAX	IAX signaling (call setup, registration, etc.)
0x07	Text	Text message
0x08	Image	Image data
0x09	HTML	HTML content
0x0A	Comfort Noise	Comfort noise generation

IAX Subclass (Frame Type 0x06)

Subclass	Name	Description
0x01	NEW	Initiate a new call
0x02	PING	Keepalive
0x03	PONG	Ping response
0x04	ACK	Acknowledge a full frame
0x05	HANGUP	Terminate the call
0x06	REJECT	Reject a call (with reason)
0x07	ACCEPT	Accept a call
0x08	AUTHREQ	Authentication challenge
0x09	AUTHREP	Authentication response
0x0A	INVAL	Invalid (destroy call reference)
0x0D	REGREQ	Registration request
0x0E	REGAUTH	Registration auth challenge
0x0F	REGACK	Registration accepted
0x10	REGREJ	Registration rejected
0x11	REGREL	Registration release
0x12	VNAK	Voice negative acknowledgment (resend)

Control Subclass (Frame Type 0x04)

Subclass	Name	Description
0x01	Hangup	Call terminated
0x02	Ring	Ring indication
0x03	Ringing	Destination is ringing
0x04	Answer	Call answered
0x05	Busy	Busy signal
0x06	Congestion	Congestion / fast busy
0x0E	Progress	Call progress (in-band audio available)
0x0F	Proceeding	Call is being routed
0x10	Hold	Put on hold
0x11	Unhold	Taken off hold

Call Flow

[Diagram: see online version]

Information Elements (IEs)

IAX messages carry data as TLV-style Information Elements:

IE	Name	Description
0x01	Called Number	Destination number
0x02	Calling Number	Caller ID number
0x03	Calling ANI	ANI (billing number)
0x04	Calling Name	Caller ID name
0x06	Username	Authentication username
0x07	Password	Authentication password
0x09	Codec Prefs	Preferred codec list
0x0E	Challenge	MD5 auth challenge string
0x0F	MD5 Result	MD5 auth response
0x10	RSA Result	RSA auth response
0x14	Cause Code	Hangup/reject reason (Q.931 cause codes)
0x2E	Format	Agreed codec for this call

Trunking

IAX trunking multiplexes audio from multiple calls in a single UDP packet, reducing per-call overhead:

[Diagram: see online version]

Trunking is used between Asterisk servers carrying many simultaneous calls -- it can reduce bandwidth by 2-3x compared to individual RTP streams.

IAX vs SIP

Feature	IAX2	SIP + RTP
Ports	1 UDP port (4569)	SIP port + 2 RTP ports per call
NAT traversal	Easy (single port)	Complex (STUN/TURN/ICE)
Media path	Through same UDP flow	Separate RTP streams
Trunking	Built-in	Not available
Codec negotiation	In-band	SDP
Standard	RFC 5456 (informational)	RFC 3261 (standards track)
Interoperability	Asterisk ecosystem	Universal
Overhead per call	4 bytes (mini frame)	12 bytes (RTP) + 40 bytes (IP/UDP)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 5456	IAX: Inter-Asterisk eXchange Version 2

See Also

- SIP -- dominant VoIP signaling (IAX is the Asterisk alternative)
- RTP -- separate media transport (IAX combines signaling + media)
- UDP

ICE -- Interactive Connectivity Establishment

Standard: RFC 8445 | **Layer:** Application (Layer 7) | **Wireshark filter:** `stun` (ICE uses STUN messages for connectivity checks)

ICE is a framework for NAT traversal that finds the best network path between two peers. It systematically gathers candidate addresses (local, NAT-mapped, and relayed), exchanges them via signaling, tests connectivity between all candidate pairs using STUN, and selects the best working pair. ICE is the NAT traversal mechanism used by WebRTC and SIP (with ICE support). It orchestrates STUN and TURN rather than defining its own wire format.

How ICE Works

[Diagram: see online version]

Candidate Types

ICE gathers three types of candidates, in order of preference:

Type	Abbreviation	Source	Priority
Host	host	Local network interface addresses	Highest
Server Reflexive	srflx	Public address discovered via STUN Binding	Medium
Relay	relay	Allocated address on a TURN server	Lowest

Additionally, during connectivity checks:

Type	Abbreviation	Source
Peer Reflexive	prflx	Discovered when a check arrives from an unexpected address

Candidate Gathering

[Diagram: see online version]

Candidate Exchange

Candidates are exchanged between peers via SDP in the signaling channel (SIP or WebRTC's offer/answer):

```
a=candidate:1 1 udp 2130706431 192.168.1.5 54321 typ host
a=candidate:2 1 udp 1694498815 203.0.113.50 12345 typ srflx raddr 192.168.1.5 rport 54321
a=candidate:3 1 udp 16777215 198.51.100.10 49152 typ relay raddr 203.0.113.50 rport 12345
```

SDP Field	Description
foundation	Identifier for the candidate base

component-id	1 = RTP, 2 = RTCP
transport	`udp` or `tcp`
priority	32-bit priority value
connection-address	Candidate IP address
port	Candidate port
typ	`host`, `srflx`, `prflx`, or `relay`
raddr/rport	Related address (the base address for srflx/relay)

Priority Calculation

```
priority = (2^24 x type_preference) + (2^8 x local_preference) + (2^0 x (256 - component_id))
```

Type	Typical Preference
Host	126
Peer Reflexive	110
Server Reflexive	100
Relay	0

Connectivity Checks

ICE forms candidate pairs (one local + one remote) and tests them with STUN Binding requests:

[Diagram: see online version]

Pair States

State	Description
Frozen	Waiting -- not yet checked
Waiting	Ready to be checked
In-Progress	STUN check sent, awaiting response
Succeeded	Check completed successfully
Failed	Check failed (timeout or error)

Check List Ordering

Pairs are checked in priority order. The pair priority for a controlling/controlled agent is:

```
pair_priority = 2^32 x min(G, D) + 2 x max(G, D) + (G &gt; D ? 1 : 0)
```

Where G = controlling agent's candidate priority, D = controlled agent's candidate priority.

ICE Roles

Role	Description
Controlling	Makes final nomination decisions; typically the offerer
Controlled	Follows the controlling agent's nominations

The controlling agent selects the best pair and nominates it by setting the `USE-CANDIDATE` attribute in a STUN check.

Nomination

Regular Nomination

The controlling agent performs checks first, then repeats the check with `USE-CANDIDATE` on the chosen pair.

Aggressive Nomination

The controlling agent sets `USE-CANDIDATE` on every check. The first successful pair is nominated. Faster but may select a suboptimal path.

Trickle ICE

RFC 8838 allows candidates to be sent incrementally as they are gathered, rather than waiting for all candidates before starting. This reduces setup latency significantly -- connectivity checks can begin while gathering is still in progress.

[Diagram: see online version]

ICE Restart

Either peer can trigger an ICE restart by generating new ICE credentials (ufrag/pwd) in a new offer. This re-gathers candidates and re-runs connectivity checks -- used when the network changes (e.g., Wi-Fi to cellular).

Encapsulation

ICE itself has no wire format -- it uses STUN Binding requests for connectivity checks and TURN for relay allocation. The ICE parameters (candidates, credentials) are exchanged via SDP in the signaling protocol (SIP or WebRTC).

[Diagram: see online version]

Standards

Document	Title
RFC 8445	Interactive Connectivity Establishment (ICE)
RFC 8838	Trickle ICE
RFC 8839	ICE Support in SDP
RFC 8489	STUN -- used for connectivity checks
RFC 8656	TURN -- relay fallback
RFC 5245	ICE (previous version)
RFC 6544	TCP Candidates with ICE

See Also

- STUN -- used for candidate gathering and connectivity checks
- TURN -- relay fallback when direct connectivity fails
- WebRTC -- primary consumer of ICE
- SIP -- ICE can be used with SIP signaling
- UDP

MGCP / H.248 -- Media Gateway Control

Standard: RFC 3435 (MGCP) / ITU-T H.248 / RFC 5125 (Megaco) | **Layer:** Application (Layer 7) | **Wireshark filter:** `mgcp` or `megaco`

MGCP and H.248 (Megaco) are call control protocols that separate the call intelligence (Call Agent / Media Gateway Controller) from the media handling (Media Gateway). Unlike SIP and H.323 where endpoints are intelligent, MGCP/H.248 gateways are "dumb" -- the central controller tells them exactly what to do: create connections, play tones, detect digits, send/receive RTP. This architecture is used by carriers for PSTN-to-VoIP trunking, class 5 softswitch deployments, and cable telephony (PacketCable).

Architecture

[Diagram: see online version]

The Call Agent handles all signaling logic (SIP/SS7 interworking, routing, billing). The gateways only manage media under instruction.

MGCP Commands

MGCP uses a text-based, command/response protocol over UDP:

Command	Direction	Description
CRCX	Agent -> GW	Create Connection -- set up an RTP stream
MDCX	Agent -> GW	Modify Connection -- change RTP parameters
DLCX	Agent -> GW	Delete Connection -- tear down an RTP stream
RQNT	Agent -> GW	Request Notification -- ask GW to detect events (digits, hook)
NTFY	GW -> Agent	Notify -- report a detected event (off-hook, digit, fax tone)
AUEP	Agent -> GW	Audit Endpoint -- query endpoint state
AUCX	Agent -> GW	Audit Connection -- query connection state
RSIP	GW -> Agent	Restart In Progress -- GW is restarting

MGCP Message Format

```
CRCX 1204 aaln/1@gw1.example.com MGCP 1.0
C: A3C47F21456789F0
L: p:20, a:PCMU
M: recvonly

v=0
c=IN IP4 0.0.0.0
m=audio 0 RTP/AVP 0
```

Line	Description
Verb + Transaction ID + Endpoint + Version	Command header
C:	Call-ID (correlates connections in the same call)
L:	Local connection options (packetization=20ms, codec=PCMU)
M:	Mode (sendrecv, recvonly, sendonly, inactive, confrnce)
SDP body	RTP session description

Response Format

```
200 1204 OK
I: FDE234C8

v=0
c=IN IP4 203.0.113.10
m=audio 36700 RTP/AVP 0
```

Call Flow Example (Incoming PSTN Call)

[Diagram: see online version]

MGCP Events

Event Package	Events	Description
L (Line)	hd (off-hook), hu (on-hook), hf (flash-hook)	Analog line events
D (DTMF)	0-9, *, #, A-D	DTMF digit detection
R (RTP)	oc (on completion), of (on failure)	RTP events
G (Generic)	ft (fax tone), mt (modem tone)	Signal detection
IT	--	Inactivity timer

Digit Map

The Call Agent can send a digit map pattern for the gateway to collect digits locally:

```
RQNT 1207 aaln/1@gw1.example.com MGCP 1.0
R: D/[0-9#*T](D)
D: (0T|00T|[2-9]xxxxxx|1[2-9]xxxxxxxxx|011x.T)
```

This tells the gateway: collect digits matching North American dialing patterns, then notify the Call Agent with the complete number.

H.248 / Megaco

H.248 (Megaco) is the ITU-T/IETF evolution of MGCP with a more powerful, ASN.1-based model:

Feature	MGCP	H.248 / Megaco
Encoding	Text (UDP)	Text or Binary (ASN.1 BER) over UDP/TCP/SCTP
Model	Endpoints + Connections	Terminations + Contexts
Scalability	Moderate	Higher (transaction model)
Standard body	IETF (RFC 3435)	ITU-T + IETF joint
Carrier adoption	Cable telephony (PacketCable)	Carrier trunking, IMS

H.248 Commands

Command	Description
Add	Add a termination to a context
Modify	Modify a termination's properties
Subtract	Remove a termination from a context

Move	Move a termination between contexts
AuditValue	Query termination properties
AuditCapabilities	Query termination capabilities
Notify	Report an event
ServiceChange	Register/deregister a termination

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 3435	MGCP Version 1.0
ITU-T H.248.1	Gateway Control Protocol Version 3
RFC 5125	Reclassification of Megaco/H.248 RFCs
PacketCable NCS	Network-Based Call Signaling (MGCP variant for cable)

See Also

- SIP -- peer-to-peer alternative (SIP endpoints are intelligent)
- H.323 -- older intelligent-endpoint VoIP suite
- RTP -- media transport controlled by MGCP/H.248
- SS7 -- PSTN signaling the gateway bridges
- ISDN -- digital trunk signaling

RTCP -- RTP Control Protocol

Standard: RFC 3550 | **Layer:** Application (Layer 7) | **Wireshark filter:** `rtcp`

RTCP is the companion control protocol to RTP. While RTP carries media data, RTCP provides out-of-band statistics and control information for an RTP session. It carries sender and receiver reports with quality metrics (packet loss, jitter, round-trip time), participant identification (CNAME), and feedback messages. RTCP packets are sent periodically at a rate that scales with session size to limit bandwidth overhead (typically 5% of the media bandwidth).

Common Header

Every RTCP packet begins with a common 4-byte header:

[Diagram: see online version]

Key Fields

Field	Size	Description
V (Version)	2 bits	RTCP version; always 2
P (Padding)	1 bit	1 = padding bytes at end of packet
RC/SC/FMT	5 bits	Report Count, Source Count, or Feedback Message Type (depends on packet type)
Packet Type	8 bits	RTCP packet type
Length	16 bits	Packet length in 32-bit words minus one

RTCP packets are typically sent as compound packets -- multiple RTCP packets concatenated in a single UDP datagram.

Packet Types

PT	Name	Description
200	SR (Sender Report)	Sender statistics + reception quality
201	RR (Receiver Report)	Reception quality from non-senders
202	SDES (Source Description)	Participant metadata (CNAME, name, email, etc.)
203	BYE	Participant is leaving the session
204	APP	Application-defined
205	RTPFB (Transport Feedback)	Transport-layer feedback (NACK, TMMBR, etc.)
206	PSFB (Payload-Specific Feedback)	Payload-specific feedback (PLI, FIR, REMB, etc.)
207	XR (Extended Report)	Extended statistics (RFC 3611)

Field Details

Sender Report (PT 200)

[Diagram: see online version]

The sender info block contains:

Field	Size	Description
NTP Timestamp	64 bits	Wall-clock time when this report was sent
RTP Timestamp	32 bits	Corresponding RTP timestamp (for NTP-to-RTP mapping)
Sender Packet Count	32 bits	Total RTP packets sent since session start
Sender Octet Count	32 bits	Total payload bytes sent since session start

Report Block (in SR and RR)

Each report block describes reception quality from one source:

[Diagram: see online version]

Field	Size	Description
SSRC	32 bits	Source being reported on
Fraction Lost	8 bits	Fraction of packets lost since last report (0-255 = 0-100%)
Cumulative Lost	24 bits	Total packets lost since session start
Ext. Highest Seq	32 bits	Highest sequence number received (with rollover count)
Interarrival Jitter	32 bits	Statistical variance of RTP packet inter-arrival time
LSR	32 bits	Middle 32 bits of NTP timestamp from last SR received
DLSR	32 bits	Delay between receiving last SR and sending this report (1/65536 sec units)

Round-trip time can be computed as: **RTT = now - LSR - DLSR**

Source Description (SDES, PT 202)

Type	Name	Description
1	CNAME	Canonical name -- persistent identifier across SSRC changes
2	NAME	User's real name
3	EMAIL	User's email address
4	PHONE	User's phone number
5	LOC	Geographic location
6	TOOL	Application or tool name

7	NOTE	Transient note about the source

CNAME is mandatory and must be unique within a session. It is typically `user@host` or a random string.

Transport-Layer Feedback (PT 205)

FMT	Name	Description
1	NACK	Negative acknowledgment -- requests retransmission
3	TMMBR	Temporary Maximum Media Stream Bitrate Request
4	TMMBN	Temporary Maximum Media Stream Bitrate Notification
15	Transport-CC	Transport-wide congestion control feedback (RFC 8888)

Payload-Specific Feedback (PT 206)

FMT	Name	Description
1	PLI	Picture Loss Indication -- requests a keyframe
2	SLI	Slice Loss Indication
4	FIR	Full Intra Request -- forces a keyframe
15	REMB	Receiver Estimated Maximum Bitrate (draft, widely used)

Compound Packet Structure

A typical compound RTCP packet:

[Diagram: see online version]

RFC 3550 requires that compound packets begin with SR or RR and always include SDES with at least a CNAME item.

Encapsulation

[Diagram: see online version]

Conventionally, RTCP uses the RTP port + 1. When multiplexed (RFC 5761), RTP and RTCP share the same port (standard in WebRTC).

Standards

Document	Title
RFC 3550	RTP / RTCP specification
RFC 3611	RTP Control Protocol Extended Reports (XR)
RFC 4585	Extended RTP Profile for RTCP-Based Feedback (AVPF)
RFC 5104	Codec Control Messages (FIR, TMMBR, etc.)
RFC 5761	Multiplexing RTP and RTCP on a Single Port
RFC 3711	Secure RTCP (SRTCP)
RFC 8888	RTP Control Protocol (RTCP) Feedback for Congestion Control

See Also

- RTP -- the media transport protocol RTCP accompanies
- SIP -- signaling protocol for RTP/RTCP sessions
- WebRTC -- uses RTCP feedback extensively for adaptive bitrate
- UDP

RTP -- Real-time Transport Protocol

Standard: RFC 3550 | **Layer:** Application (Layer 7) | **Wireshark filter:** `rtp`

RTP provides end-to-end delivery of real-time audio and video over IP networks. It carries the media payload and provides timestamping, sequence numbering, and payload type identification -- the mechanisms applications need to reconstruct a media stream, compensate for jitter, and detect packet loss. RTP is typically paired with RTCP, which provides feedback on quality of service. RTP does not guarantee delivery or provide congestion control; it relies on the application or companion protocols for that.

Header

[Diagram: see online version]

The fixed header is 12 bytes. Each CSRC entry adds 4 bytes.

Key Fields

Field	Size	Description
V (Version)	2 bits	RTP version; always 2
P (Padding)	1 bit	1 = padding bytes at end of payload

X (Extension)	1 bit	1 = extension header follows the fixed header
CC (CSRC Count)	4 bits	Number of CSRC identifiers following the header
M (Marker)	1 bit	Interpretation is profile-defined (e.g., first packet of a talkspurt)
PT (Payload Type)	7 bits	Identifies the codec/encoding of the payload
Sequence Number	16 bits	Increments by one per packet; detects loss and reordering
Timestamp	32 bits	Sampling instant of the first byte of the payload (clock rate depends on codec)
SSRC	32 bits	Synchronization Source -- uniquely identifies the stream sender
CSRC	32 bits each	Contributing Sources -- original SSRCs mixed into this stream

Field Details

Payload Types

Static payload type assignments (from RFC 3551):

PT	Encoding	Media	Clock Rate
0	PCMU (G.711 μ-law)	Audio	8000 Hz
3	GSM	Audio	8000 Hz
4	G723	Audio	8000 Hz
8	PCMA (G.711 A-law)	Audio	8000 Hz
9	G722	Audio	8000 Hz
18	G729	Audio	8000 Hz
26	JPEG	Video	90000 Hz
31	H261	Video	90000 Hz
32	MPV (MPEG Video)	Video	90000 Hz
33	MP2T (MPEG Transport)	Audio/Video	90000 Hz
34	H263	Video	90000 Hz
96-127	Dynamic	Any	Negotiated via SDP

Modern codecs (Opus, VP8, VP9, H.264, H.265, AV1) all use dynamic payload types (96-127), negotiated via SDP in SIP or WebRTC signaling.

Extension Header

When X=1, a header extension follows the CSRC list:

[Diagram: see online version]

RFC 8285 defines a general mechanism for carrying multiple short extensions (used heavily in WebRTC for audio levels, mid, rid, etc.).

Marker Bit

The meaning of M depends on the media profile:

Context	Meaning
Audio (RFC 3551)	First packet after a silence period (talkspurt)
Video (RFC 6184 H.264)	Last packet of a video frame
Generic	Profile-dependent

Sequence Number and Timestamp

- **Sequence Number** increments by 1 per packet. Receivers use it to detect loss and reorder packets.
- **Timestamp** reflects the media sampling clock, not wall-clock time. For audio at 8000 Hz, it increments by 160 per 20ms packet. For video at 90000 Hz, it reflects the capture time of the frame.

Encapsulation

[Diagram: see online version]

RTP has no assigned port; ports are negotiated via signaling (SIP/SDP or WebRTC). Even-numbered ports are conventionally used for RTP, odd for RTCP.

Standards

Document	Title
RFC 3550	RTP: A Transport Protocol for Real-Time Applications
RFC 3551	RTP Profile for Audio and Video Conferences with Minimal Control
RFC 3711	The Secure Real-time Transport Protocol (SRTP)
RFC 8285	A General Mechanism for RTP Header Extensions
RFC 5761	Multiplexing RTP and RTCP on a Single Port
RFC 7741	RTP Payload Format for VP8 Video
RFC 6184	RTP Payload Format for H.264 Video
RFC 7587	RTP Payload Format for Opus Audio

See Also

- RTCP -- companion control protocol for quality feedback
- SIP -- signaling protocol that sets up RTP sessions
- WebRTC -- browser-based real-time communication using RTP/SRTP
- UDP

RTSP -- Real Time Streaming Protocol

Standard: RFC 7826 | **Layer:** Application (Layer 7) | **Wireshark filter:** `rtsp`

RTSP is an application-level protocol for controlling the delivery of real-time streaming media. It acts as a "network remote control" -- RTSP does not carry the media itself but controls the RTP stream that does. It provides VCR-like commands: play, pause, seek, and teardown. RTSP is widely used in IP cameras (ONVIF), surveillance systems, and some streaming media servers (VLC, Wowza). It uses an HTTP-like text syntax but maintains session state.

Methods

Method	Description
OPTIONS	Query supported methods
DESCRIBE	Get media description (returns SDP)
SETUP	Establish transport parameters (RTP ports)
PLAY	Start media delivery
PAUSE	Pause media delivery
TEARDOWN	Stop session and free resources
GET_PARAMETER	Retrieve parameter values (keepalive)
SET_PARAMETER	Set parameter values
ANNOUNCE	Post media description to server (recording)
RECORD	Begin recording

Session Flow

[Diagram: see online version]

Response Codes

Code	Meaning

200	OK
301	Moved Permanently
401	Unauthorized
404	Not Found
453	Not Enough Bandwidth
454	Session Not Found
459	Aggregate Operation Not Allowed
461	Unsupported Transport

Key Headers

Header	Description
CSeq	Sequence number (matches requests to responses)
Session	Session identifier (assigned by server in SETUP response)
Transport	Transport specification (RTP/AVP, unicast/multicast, ports)
Range	Time range (npt=0-, npt=10-30, clock=...)
RTP-Info	RTP stream info (seq and timestamp for synchronization)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 7826	Real-Time Streaming Protocol Version 2.0
RFC 2326	RTSP 1.0 (original, widely deployed)

See Also

- RTP -- carries the media RTSP controls
- SDP -- describes the media session
- SIP -- alternative signaling for VoIP/video

SDP -- Session Description Protocol

Standard: RFC 8866 | **Layer:** Application (Layer 7) | **Wireshark filter:** `sdp`

SDP is a text-based format for describing multimedia session parameters -- what codecs to use, where to send media, and how to configure the connection. SDP is not a protocol per se (it has no request/response mechanism) but a description format carried by signaling protocols like SIP, RTSP, and WebRTC (via JSEP). It is used in every VoIP call, video conference, and WebRTC connection to negotiate media capabilities between participants.

Format

SDP is plain text, one attribute per line, in the form `<type>=<value>`:

```
v=0
o=alice 2890844526 2890844526 IN IP4 192.168.1.100
s=Session Title
c=IN IP4 192.168.1.100
t=0 0
m=audio 49170 RTP/AVP 0 8 97
a=rtpmap:97 opus/48000/2
a=sendrecv
m=video 51372 RTP/AVP 96
a=rtpmap:96 VP8/90000
a=sendrecv
```

Field Types

Session-Level

Type	Name	Required	Description
v=	Version	Yes	Protocol version (always 0)
o=	Origin	Yes	Session creator (username, session-id, version, network, address)
s=	Session Name	Yes	Session title (can be `-` for unnamed)
i=	Information	No	Session description text
u=	URI	No	URI for more information
e=	Email	No	Contact email
c=	Connection	Yes*	Connection address (`IN IP4 192.168.1.100` or `IN IP6 ::1`)
b=	Bandwidth	No	Bandwidth limit (`AS:512` kbps, `CT:1024`)
t=	Timing	Yes	Start and stop times (0 0 = permanent)
a=	Attribute	No	Session-level attributes

Media-Level

Type	Name	Description
m=	Media	Media type, port, protocol, format list
c=	Connection	Per-media connection address (overrides session level)
b=	Bandwidth	Per-media bandwidth
a=	Attribute	Media-level attributes (codecs, direction, etc.)

Media Line Format

```
m=&lt;media&gt; &lt;port&gt; &lt;proto&gt; &lt;fmt list&gt;
```

Field	Values	Example
media	audio, video, text, application	`audio`
port	Transport port number	`49170`
proto	Transport protocol	`RTP/AVP`, `UDP/TLS/RTP/SAVPF`, `UDP`
fmt	Payload type numbers (space-separated)	`0 8 97`

Common Protocol Values

Protocol	Description
RTP/AVP	RTP Audio/Video Profile (RFC 3551)
RTP/SAVP	Secure RTP (SRTP)
RTP/AVPF	RTP with feedback (RTCP-based)
UDP/TLS/RTP/SAVPF	WebRTC media (DTLS-SRTP with feedback)
UDP	Raw UDP
TCP	Raw TCP

Common Attributes

Attribute	Description
`a=rtpmap:<pt> <encoding>/<clock>[/<channels>]`	Map payload type to codec
`a=fmtp:<pt> <params>`	Format-specific parameters
`a=sendrecv`	Both directions

a=sendonly	Sender only (hold, one-way)
a=recvonly	Receiver only
a=inactive	No media in either direction
a=mid:<id>	Media identification (for BUNDLE)
a=group:BUNDLE 0 1 2	Multiplex media on one transport
a=ice-ufrag:<ufrag>	ICE credentials
a=ice-pwd:<pwd>	ICE password
a=fingerprint:sha-256 <hash>	DTLS certificate fingerprint
a=candidate:<...>	ICE candidate
a=setup:actpass	DTLS role (active, passive, or actpass)
a=rtcp-mux	Multiplex RTP and RTCP on one port
a=ssrc:<id> cname:<cname>	Synchronization source mapping
a=extmap:<id> <uri>	RTP header extension mapping
a=ptime:<ms>	Packet time (audio frame duration)
a=maxptime:<ms>	Maximum packet time

Offer/Answer Model (RFC 3264)

SDP is used in an offer/answer exchange:

[Diagram: see online version]

The answerer must include the same number of `m=` lines as the offer. Rejected media lines use port 0.

WebRTC SDP Example

```
v=0
o=- 4858location6 2 IN IP4 127.0.0.1
s=-
t=0 0
a=group:BUNDLE 0 1
a=ice-options:trickle
m=audio 9 UDP/TLS/RTP/SAVPF 111 0
c=IN IP4 0.0.0.0
a=mid:0
a=rtpmap:111 opus/48000/2
a=rtpmap:0 PCMU/8000
a=fmtp:111 minptime=10;useinbandfec=1
a=rtcp-mux
a=sendrecv
a=ice-ufrag:abcd
a=ice-pwd:efghijklmnopqrstuv
a=fingerprint:sha-256 AA:BB:CC:...
a=setup:actpass
a=candidate:1 1 udp 2130706431 192.168.1.5 54321 typ host
m=video 9 UDP/TLS/RTP/SAVPF 96 97
c=IN IP4 0.0.0.0
a=mid:1
a=rtpmap:96 VP8/90000
a=rtpmap:97 H264/90000
a=rtcp-mux
a=sendrecv
```

Standards

Document	Title
RFC 8866	SDP: Session Description Protocol
RFC 3264	An Offer/Answer Model with SDP
RFC 8829	JavaScript Session Establishment Protocol (JSEP)
RFC 5576	Source-Specific Media Attributes in SDP

See Also

- SIP -- carries SDP for VoIP call setup
- WebRTC -- uses SDP via JSEP
- RTP -- the media transport SDP describes
- RTSP -- uses SDP to describe streaming media
- ICE -- ICE candidates are carried in SDP `a=candidate` lines

SIMPLE -- SIP for Instant Messaging and Presence Leveraging Extensions

Standard: RFC 3428 (MESSAGE) / RFC 3856 (Presence) / RFC 4975 (MSRP) | **Layer:** Application (Layer 7) | **Wireshark filter:** `sip` (SIMPLE uses SIP methods)

SIMPLE is a set of SIP extensions that add instant messaging (IM) and presence to SIP-based communication systems. Rather than being a separate protocol, SIMPLE is a collection of RFCs that extend SIP with the MESSAGE method (page-mode IM), SUBSCRIBE/NOTIFY for presence (online/busy/away status), and MSRP for session-based messaging (chat sessions with typing indicators, file transfer, and message history). SIMPLE is the IM/presence standard in IMS (IP Multimedia Subsystem) used by mobile carriers, and was the basis of Microsoft's Office Communications Server (now Skype for Business/Teams) and many enterprise UC platforms.

Components

Component	RFCs	Description
Page-Mode Messaging	RFC 3428	SIP MESSAGE method -- fire-and-forget IM
Presence	RFC 3856, 3903	SUBSCRIBE/NOTIFY + PUBLISH for online status
Session-Mode Messaging	RFC 4975 (MSRP)	Persistent chat sessions with rich features
Presence Data Format	RFC 3863 (PIDF)	XML format for presence information
Watcher Info	RFC 3857	Who is watching my presence
Resource Lists	RFC 4826	Subscribe to multiple contacts in one request
Partial Presence	RFC 5263	Efficient partial updates to presence state

Page-Mode Messaging (SIP MESSAGE)

The simplest form -- a standalone SIP request carrying an IM:

```
MESSAGE sip:bob@example.com SIP/2.0
Via: SIP/2.0/TCP client.example.com;branch=z9hG4bK776
From: "Alice" &lt;sip:alice@example.com&gt;;tag=49583
To: "Bob" &lt;sip:bob@example.com&gt;
Call-ID: asd88asd77a@client.example.com
CSeq: 1 MESSAGE
Max-Forwards: 70
Content-Type: text/plain
Content-Length: 18

Hi Bob, are you free?
```

Response:

```
SIP/2.0 200 OK
```

MESSAGE Flow

[Diagram: see online version]

Page-mode is simple but has no delivery receipts, typing indicators, or message history. Each MESSAGE is a standalone SIP transaction.

Content Types

Content-Type	Description
text/plain	Plain text message
text/html	HTML formatted message
application/im-iscomposing+xml	"Is typing" indication (RFC 3994)
message/cpim	Common Presence and Instant Messaging format (wrapper)
multipart/mixed	Message with attachments

Presence

SIMPLE presence uses SIP's event framework (RFC 6665) with the `presence` event package:

Presence Flow

[Diagram: see online version]

PUBLISH (RFC 3903)

The presentity (user) publishes their status to the presence server:

```
PUBLISH sip:bob@example.com SIP/2.0
Event: presence
Content-Type: application/pidf+xml
Expires: 3600

&lt;?xml version="1.0" encoding="UTF-8"?&gt;
&lt;presence xmlns="urn:ietf:params:xml:ns:pidf"
  entity="sip:bob@example.com"&gt;
  &lt;tuple id="mobile"&gt;
    &lt;status&gt;
      &lt;basic&gt;open&lt;/basic&gt;
    &lt;/status&gt;
    &lt;note&gt;In a meeting until 3pm&lt;/note&gt;
  &lt;/tuple&gt;
&lt;/presence&gt;
```

PIDF (Presence Information Data Format -- RFC 3863)

Element	Description

`<presence>`	Root element, `entity` = presentity URI
`<tuple>`	One per device/endpoint, `id` attribute
`<status><basic>`	`open` (available) or `closed` (unavailable)
`<note>`	Human-readable status text
`<contact>`	Reachable URI for this tuple
`<timestamp>`	Last update time

RPID (Rich Presence -- RFC 4480)

Extends PIDF with detailed status:

Element	Values	Description
`<activities>`	on-the-phone, busy, away, holiday, in-transit, meeting, etc.	What the user is doing
`<mood>`	happy, sad, angry, nervous, etc.	Emotional state
`<place-type>`	office, home, airport, etc.	Location type
`<privacy>`	--	User wants privacy

Session-Mode Messaging (MSRP -- RFC 4975)

MSRP (Message Session Relay Protocol) provides a persistent chat session with richer features than page-mode:

MSRP vs PAGE-MODE

Feature	Page-Mode (MESSAGE)	Session-Mode (MSRP)
Session	None -- each message standalone	SIP INVITE establishes session
Typing indicators	Separate MESSAGE request	In-session indication
Delivery reports	No	Yes (REPORT mechanism)
Large messages	Limited by SIP message size	Chunked, unlimited
File transfer	No	Yes
Message history	No	Within session
Complexity	Very simple	Requires MSRP relay infrastructure

MSRP Session Setup

[Diagram: see online version]

MSRP Message

```
MSRP d93kswow SEND
To-Path: msrp://bob.example.com:8765/chat;tcp
From-Path: msrp://alice.example.com:7654/chat;tcp
Message-ID: 87652
Byte-Range: 1-25/25
Content-Type: text/plain

Hello, how are you doing?
-------d93kswow$
```

MSRP Methods

Method	Description
SEND	Send a message (or chunk of a message)
REPORT	Delivery/display report
AUTH	Authenticate with an MSRP relay (RFC 4976)

IMS Integration

In 3GPP IMS networks, SIMPLE provides the standard IM and presence service:

[Diagram: see online version]

SIMPLE vs XMPP

Feature	SIMPLE	XMPP
Base protocol	SIP extensions	Dedicated XML protocol
IM model	PAGE (MESSAGE) + SESSION (MSRP)	Persistent XML stream
Presence format	PIDF (XML)	Native XML stanzas
Federation	SIP routing (DNS SRV, SIP proxy)	Server-to-server (s2s, DNS SRV)
Multi-user chat	Conference focus (SIP)	MUC (XEP-0045)
Adoption	IMS/carriers, enterprise UC	Open source, consumer
Standards body	IETF	IETF + XSF

Encapsulation

[Diagram: see online version]

Standards

Document	Title

RFC 3428	SIP Extension for Instant Messaging (MESSAGE method)
RFC 3856	A Presence Event Package for SIP
RFC 3903	SIP Extension for Event State Publication (PUBLISH)
RFC 3863	Presence Information Data Format (PIDF)
RFC 4480	Rich Presence Extensions to PIDF (RPID)
RFC 4975	The Message Session Relay Protocol (MSRP)
RFC 4976	Relay Extensions for MSRP
RFC 3994	Indication of Message Composition ("is typing")
RFC 6665	SIP-Specific Event Notification (SUBSCRIBE/NOTIFY)

See Also

- SIP -- base protocol that SIMPLE extends
- XMPP -- alternative IM/presence protocol
- RTP -- media transport (SIP+SIMPLE combines voice/video + IM + presence)
- WebRTC -- browser-based alternative
- TCP

SIP -- Session Initiation Protocol

Standard: RFC 3261 | **Layer:** Application (Layer 7) | **Wireshark filter:** `sip`

SIP is a signaling protocol for initiating, modifying, and terminating multimedia sessions -- primarily voice and video calls (VoIP). It handles session setup, participant discovery, and capability negotiation, but does not carry the media itself (that's RTP/SRTP). SIP follows an HTTP-like request/response model with text-based messages. It is the dominant signaling protocol in VoIP, IMS (mobile telephony), and many unified communications systems.

Message Structure

SIP messages are text-based, similar to HTTP. Like HTTP, there is no fixed binary header -- messages use CRLF-delimited text lines.

Request

```
METHOD sip:user@domain SIP/2.0\r\n
Via: SIP/2.0/UDP client.example.com:5060;branch=z9hG4bK776asdhds\r\n
From: "Alice" &lt;sip:alice@example.com&gt;;tag=1928301774\r\n
To: "Bob" &lt;sip:bob@example.com&gt;\r\n
Call-ID: a84b4c76e66710@client.example.com\r\n
CSeq: 314159 INVITE\r\n
Contact: &lt;sip:alice@192.168.1.100:5060&gt;\r\n
Content-Type: application/sdp\r\n
Content-Length: 142\r\n
\r\n
[SDP body]
```

Response

```
SIP/2.0 200 OK\r\n
Via: SIP/2.0/UDP client.example.com:5060;branch=z9hG4bK776asdhds\r\n
From: "Alice" &lt;sip:alice@example.com&gt;;tag=1928301774\r\n
To: "Bob" &lt;sip:bob@example.com&gt;;tag=a6c85cf\r\n
Call-ID: a84b4c76e66710@client.example.com\r\n
CSeq: 314159 INVITE\r\n
Contact: &lt;sip:bob@192.168.1.200:5060&gt;\r\n
Content-Type: application/sdp\r\n
Content-Length: 131\r\n
\r\n
[SDP body]
```

Methods

Method	Description
INVITE	Initiate a session (call setup)
ACK	Confirm receipt of final response to INVITE
BYE	Terminate an established session
CANCEL	Cancel a pending INVITE
REGISTER	Register a user's contact address with a registrar
OPTIONS	Query capabilities of a server or user agent
SUBSCRIBE	Subscribe to event notifications (RFC 6665)
NOTIFY	Deliver an event notification (RFC 6665)
REFER	Ask recipient to issue a request (call transfer, RFC 3515)
MESSAGE	Instant messaging (RFC 3428)
INFO	Carry application-level information mid-dialog (RFC 6086)
UPDATE	Modify a session before it's established (RFC 3311)
PRACK	Provisional response acknowledgment (RFC 3262)

Response Codes

Range	Category	Examples
1xx	Provisional	100 Trying, 180 Ringing, 183 Session Progress
2xx	Success	200 OK, 202 Accepted
3xx	Redirection	301 Moved Permanently, 302 Moved Temporarily
4xx	Client Error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 408 Request Timeout, 486 Busy Here
5xx	Server Error	500 Internal Server Error, 503 Service Unavailable
6xx	Global Failure	600 Busy Everywhere, 603 Decline

Key Headers

Header	Description
Via	Records each hop; used to route responses back
From	Logical identity of the request originator (with tag)
To	Logical identity of the request target (with tag in responses)
Call-ID	Globally unique identifier for the dialog
CSeq	Command sequence number + method name
Contact	Direct reachable URI of the sender
Max-Forwards	Hop limit (like TTL), default 70
Record-Route / Route	Force requests through proxies
Authorization / WWW-Authenticate	Digest authentication (RFC 8760)
Supported / Require	Feature negotiation

Call Flow

Basic Call Setup and Teardown

[Diagram: see online version]

Registration

[Diagram: see online version]

SIP URI Schemes

Scheme	Description
`sip:user@domain`	Unencrypted SIP
`sips:user@domain`	SIP over TLS (mandatory encryption)
`tel:+15551234567`	Telephone number (RFC 3966)

SIP Architecture

Component	Role
User Agent Client (UAC)	Initiates requests
User Agent Server (UAS)	Responds to requests
Proxy Server	Routes requests on behalf of users
Registrar	Accepts REGISTER requests, maintains location database
Redirect Server	Returns alternative URIs (3xx responses)
Back-to-Back User Agent (B2BUA)	Acts as both UAC and UAS (session border controllers, PBXs)

Encapsulation

[Diagram: see online version]

SIP over UDP is most common. SIP over TLS (SIPS) provides hop-by-hop encryption. SIP over WebSocket enables browser-based VoIP.

Standards

Document	Title
RFC 3261	SIP: Session Initiation Protocol
RFC 3262	Reliability of Provisional Responses (PRACK)
RFC 3263	Locating SIP Servers (DNS SRV/NAPTR)
RFC 3264	An Offer/Answer Model with SDP
RFC 3515	The REFER Method (call transfer)
RFC 6665	SIP-Specific Event Notification (SUBSCRIBE/NOTIFY)
RFC 7118	The WebSocket Protocol as a Transport for SIP
RFC 8866	SDP: Session Description Protocol
RFC 3428	SIP Extension for Instant Messaging

See Also

- RTP -- carries the actual media streams set up by SIP
- RTCP -- quality feedback for RTP sessions
- WebRTC -- browser-based alternative (uses custom signaling instead of SIP)
- TLS -- used by SIPS for encrypted signaling
- DNS -- SRV/NAPTR records for SIP server discovery
- UDP
- TCP

Skinny -- SCCP -- Skinny Client Control Protocol

Standard: Cisco proprietary (publicly documented) | **Layer:** Application (Layer 7) | **Wireshark filter:** `skinny`

Skinny (SCCP) is Cisco's proprietary VoIP signaling protocol for communication between Cisco IP phones and Cisco Unified Communications Manager (CUCM, formerly CallManager). It follows a stimulus-model design where the phone is a "thin client" -- it reports button presses and hook events to the call server, which tells the phone exactly what to display and which RTP streams to open. Skinny has one of the largest installed bases of any VoIP endpoint protocol due to Cisco's dominance in enterprise IP telephony.

Message Format

[Diagram: see online version]

Field	Size	Description
Message Length	32 bits	Length of data following this field
Header Version	32 bits	Protocol version (0, 17, 22, etc.)
Message ID	32 bits	Identifies the message type
Message Body	Variable	Message-specific data

All fields are little-endian (unlike most network protocols).

Key Messages

Phone -> CUCM

ID	Name	Description
0x0000	KeepAliveMessage	Periodic keepalive
0x0001	RegisterMessage	Phone registers with CUCM
0x0004	KeypadButtonMessage	User pressed a digit (0-9, *, #)

0x0006	StimulusMessage	User pressed a button (line, speed dial, hold, transfer)
0x0007	OffHookMessage	Handset lifted / speaker activated
0x0008	OnHookMessage	Handset replaced / speaker deactivated
0x000B	SoftKeyEventMessage	User pressed a softkey (NewCall, EndCall, Hold, etc.)
0x0020	OpenReceiveChannelAck	Phone reports RTP port for receiving media
0x0022	MediaTransmissionFailure	RTP stream failed
0x0026	CapabilitiesResMessage	Phone reports supported codecs

CUCM -> Phone

ID	Name	Description
0x0081	RegisterAckMessage	Registration accepted
0x0082	StartToneMessage	Play a tone (dial tone, ringback, busy, etc.)
0x0083	StopToneMessage	Stop the current tone
0x0085	SetRingerMessage	Start/stop ringing
0x008A	SetLampMessage	Control LED indicators
0x008F	CallInfoMessage	Display caller ID, call state
0x0090	StartMediaTransmission	Begin sending RTP to a specified IP:port
0x0091	StopMediaTransmission	Stop sending RTP
0x0092	CallStateMessage	Update call state (on-hook, ring-out, connected, hold, etc.)
0x0105	OpenReceiveChannel	Prepare to receive RTP (phone opens a port)
0x0106	CloseReceiveChannel	Stop receiving RTP
0x0111	SelectSoftKeys	Control which softkeys are visible
0x0112	DisplayPromptStatus	Display text on the phone screen
0x0113	ClearPromptStatus	Clear the display

Call Flow

[Diagram: see online version]

Tones

Tone	Description

InsideDialTone	Internal dial tone
OutsideDialTone	External (PSTN) dial tone
AlertingTone	Ringback
BusyTone	Busy signal
ReorderTone	Fast busy / reorder
CallWaitingTone	Call waiting beep
ConfirmationTone	Confirmation beep
ZipZip	Transfer confirmation

Call States

State	Description
OnHook	Idle
OffHook	Handset up, awaiting digits
RingOut	Outbound ringing
RingIn	Inbound ringing
Connected	Active call
Hold	Call on hold
Proceed	Call proceeding
Busy	Called party busy

Skinny vs SIP

Feature	Skinny (SCCP)	SIP
Architecture	Stimulus (thin client)	Intelligent endpoint
Phone intelligence	Minimal -- server decides everything	Phone handles call logic
Protocol	Binary, proprietary	Text, open standard
Server dependency	Total -- phone is useless without CUCM	Partial -- phone can do basic calls standalone
Vendor	Cisco only	Multi-vendor
Media negotiation	Server-mediated	Endpoint-to-endpoint (SDP)
Codec decision	CUCM chooses	Endpoints negotiate

Encapsulation

[Diagram: see online version]

Standards

Skinny is Cisco proprietary but publicly documented:

Document	Title
Cisco SCCP	Skinny Client Control Protocol documentation
Wireshark Skinny	Wireshark dissector documentation

See Also

- SIP -- open standard alternative (Cisco phones also support SIP)
- RTP -- media transport between phones
- H.323 -- another VoIP signaling protocol
- MGCP -- similar thin-client model for gateways
- TCP

SRTP -- Secure Real-time Transport Protocol

Standard: RFC 3711 | **Layer:** Application (Layer 7) | **Wireshark filter:** `srtp`

SRTP provides encryption, authentication, and replay protection for RTP media streams. It encrypts the RTP payload while leaving the RTP header accessible for routing, and appends an authentication tag for integrity. SRTCP does the same for RTCP control messages. SRTP is mandatory in WebRTC and widely used in VoIP (SIP/RTP). Keys are typically negotiated via DTLS-SRTP (WebRTC) or SDES (SIP).

SRTP Packet

[Diagram: see online version]

Field	Encrypted	Authenticated	Description
RTP Header	No	Yes	Version, PT, SSRC, sequence, timestamp
Header Extensions	No (encrypted in WebRTC with RFC 6904)	Yes	RTP header extensions
Payload	**Yes**	Yes	Media data (audio/video)
MKI	No	No	Master Key Identifier (optional, for key rotation)
Authentication Tag	No	--	HMAC over header + payload

SRTCP Packet

[Diagram: see online version]

Field	Description
E flag	1 = this RTCP packet is encrypted; 0 = unencrypted
SRTCP Index	31-bit packet index for replay protection

Key Derivation

SRTP derives session keys from a single master key using a Key Derivation Function (KDF):

[Diagram: see online version]

Cipher Suites

Suite	Encryption	Auth	Tag Size
AES_CM_128_HMAC_SHA1_80	AES-128 Counter Mode	HMAC-SHA1	80 bits (default)
AES_CM_128_HMAC_SHA1_32	AES-128 Counter Mode	HMAC-SHA1	32 bits (compact)
AEAD_AES_128_GCM	AES-128 GCM	Implicit (AEAD)	128 bits
AEAD_AES_256_GCM	AES-256 GCM	Implicit (AEAD)	128 bits

Key Exchange Methods

Method	Used By	Description
DTLS-SRTP	WebRTC	Keys derived from DTLS handshake (RFC 5764) -- end-to-end
SDES	SIP	Keys exchanged in SDP `a=crypto:` attribute -- relies on signaling security
ZRTP	Some VoIP	Peer-to-peer key agreement in the media path (RFC 6189)
MIKEY	3GPP IMS	Multimedia Internet KEYing (RFC 3830)

DTLS-SRTP (WebRTC)

[Diagram: see online version]

SDES (SDP Security Descriptions)

```
a=crypto:1 AES_CM_128_HMAC_SHA1_80 inline:d0RmdmcmVCspeEc3QGZiNWpVLFJhQX1cfHAwJSoj
```

The `inline:` parameter carries a base64-encoded master key + salt. This method is insecure unless the SDP itself is encrypted (e.g., SIPS/TLS).

Replay Protection

SRTP maintains a replay list (sliding window) of received sequence numbers. The RTP sequence number (16-bit) is extended to a 48-bit index using a rollover counter (ROC) to handle wrap-around:

```
SRTP Index = ROC x 65536 + RTP Sequence Number
```

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 3711	The Secure Real-time Transport Protocol (SRTP)
RFC 5764	DTLS Extension to Establish Keys for SRTP
RFC 4568	SDP Security Descriptions (SDES)
RFC 7714	AES-GCM for SRTP
RFC 6904	Encryption of Header Extensions in SRTP

See Also

- RTP -- the unencrypted media transport SRTP secures
- RTCP -- control protocol secured by SRTCP
- DTLS -- key exchange for SRTP in WebRTC
- WebRTC -- mandates SRTP for all media
- SIP -- signaling that negotiates SRTP via SDP

STUN -- Session Traversal Utilities for NAT

Standard: RFC 8489 | **Layer:** Application (Layer 7) | **Wireshark filter:** `stun`

STUN is a protocol that allows a host behind a NAT to discover its public IP address and port mapping, and to determine the type of NAT it is behind. It works by having the client send a Binding request to a STUN server on the public Internet, which reflects back the client's observed transport address. STUN is a building block used by ICE for connectivity checks and by TURN for relay allocation. In WebRTC, STUN binding requests are also used directly between peers for ICE connectivity checks and consent freshness.

Message

[Diagram: see online version]

The header is 20 bytes. The two most significant bits of the first byte are always `00` (this distinguishes STUN from other protocols when multiplexed on the same port).

Key Fields

Field	Size	Description
Message Type	14 bits	Encodes the method and class (see below)
Message Length	16 bits	Payload length excluding the 20-byte header (must be multiple of 4)
Magic Cookie	32 bits	Fixed value `0x2112A442` -- identifies STUN and aids NAT detection
Transaction ID	96 bits	Unique identifier matching requests to responses

Field Details

Message Type Encoding

The 14-bit type field encodes both the method and class using an interleaved bit layout:

```
Bits:  M11 M10 M9 M8 M7 C1 M6 M5 M4 C0 M3 M2 M1 M0
```

Class	C1 C0	Description
Request	0 0	Client sends a request
Indication	0 1	No response expected (fire-and-forget)
Success Response	1 0	Server confirms success
Error Response	1 1	Server reports an error

Method	Value	Description
Binding	0x001	Discover reflexive transport address

Binding Request/Response Flow

[Diagram: see online version]

Attributes

Attributes follow the header as TLV (Type-Length-Value) entries, padded to 4-byte boundaries:

[Diagram: see online version]

Common Attributes

Type	Name	Description
0x0001	MAPPED-ADDRESS	Client's reflexive address (legacy, unobfuscated)
0x0006	USERNAME	Username for authentication (ICE uses `ufrag:ufrag`)
0x0008	MESSAGE-INTEGRITY	HMAC-SHA1 over the message (20 bytes)
0x0009	ERROR-CODE	Error class (hundreds) + number + reason phrase
0x000A	UNKNOWN-ATTRIBUTES	List of unrecognized mandatory attributes
0x0014	REALM	Authentication realm
0x0015	NONCE	Authentication nonce
0x001C	MESSAGE-INTEGRITY-SHA256	HMAC-SHA256 over the message (RFC 8489)
0x0020	XOR-MAPPED-ADDRESS	Client's reflexive address XORed with magic cookie (NAT-safe)
0x0024	PRIORITY	ICE candidate priority
0x0025	USE-CANDIDATE	ICE nomination flag
0x8022	SOFTWARE	Software version string
0x8028	FINGERPRINT	CRC-32 for message integrity (demultiplexing aid)
0x8029	ICE-CONTROLLED	ICE role
0x802A	ICE-CONTROLLING	ICE role

XOR-MAPPED-ADDRESS

The reflexive address is XORed with the magic cookie (and transaction ID for IPv6) to prevent NATs from rewriting the address inside the payload:

Field	Size	Description

Reserved	8 bits	Must be zero
Family	8 bits	0x01 = IPv4, 0x02 = IPv6
X-Port	16 bits	Port XORed with upper 16 bits of magic cookie
X-Address	32/128 bits	IP XORed with magic cookie (+ transaction ID for IPv6)

Error Codes

Code	Meaning
300	Try Alternate -- use a different server
400	Bad Request -- malformed message
401	Unauthenticated -- credentials required
420	Unknown Attribute -- mandatory attribute not understood
438	Stale Nonce -- nonce expired, retry
500	Server Error

NAT Type Detection

By sending Binding requests to different STUN server addresses/ports and comparing the reflexive addresses, a client can classify the NAT:

NAT Type	Behavior
Full Cone	Same external mapping regardless of destination
Address-Restricted Cone	External mapping reused only if destination IP matches
Port-Restricted Cone	External mapping reused only if destination IP:port matches
Symmetric	Different mapping for each destination IP:port

Symmetric NATs are the most challenging for peer-to-peer -- they often require a TURN relay.

Encapsulation

[Diagram: see online version]

In WebRTC, STUN messages are multiplexed on the same UDP port as DTLS and SRTP, distinguished by the first two bits being `00`.

Standards

Document	Title

RFC 8489	Session Traversal Utilities for NAT (STUN)
RFC 5389	STUN (previous version, widely referenced)
RFC 3489	STUN (original "classic" STUN, obsoleted)
RFC 7064	URI Scheme for STUN

See Also

- ICE -- uses STUN for connectivity checks
- TURN -- extends STUN with relay capabilities
- WebRTC -- primary modern consumer of STUN/ICE/TURN
- UDP

TURN -- Traversal Using Relays around NAT

Standard: RFC 8656 | **Layer:** Application (Layer 7) | **Wireshark filter:** `stun` (TURN uses STUN message format)

TURN is an extension of STUN that provides a relay server for media traffic when direct peer-to-peer connectivity fails. This happens when both peers are behind symmetric NATs or restrictive firewalls that block incoming UDP. The TURN server allocates a public transport address (the relay address) and forwards packets between the two peers. TURN is the fallback mechanism in ICE -- it guarantees connectivity at the cost of routing all media through a server, adding latency and server bandwidth.

How TURN Works

[Diagram: see online version]

Message Format

TURN reuses the STUN message format (20-byte header with magic cookie) and adds new methods and attributes:

[Diagram: see online version]

TURN Methods

Method	Value	Description
Allocate	0x003	Request a relay allocation
Refresh	0x004	Keep an allocation alive (or release with lifetime=0)

Send	0x006	Send data through relay (indication, no response)
Data	0x007	Receive data through relay (indication from server)
CreatePermission	0x008	Authorize a peer's IP to send through the relay
ChannelBind	0x009	Bind a channel number to a peer for efficient relay

Key Attributes

Type	Name	Description
0x000C	CHANNEL-NUMBER	Channel number for ChannelBind (0x4000-0x7FFF)
0x000D	LIFETIME	Allocation lifetime in seconds (default 600)
0x0012	XOR-PEER-ADDRESS	Peer's transport address (XOR-encoded)
0x0013	DATA	Application data being relayed
0x0016	XOR-RELAYED-ADDRESS	Allocated relay address (XOR-encoded)
0x0019	REQUESTED-TRANSPORT	Transport protocol for relay (17 = UDP)
0x0022	RESERVATION-TOKEN	Token for reserving a port pair (RTP/RTCP)

ChannelData Message

For efficiency, TURN provides ChannelData messages that bypass the full STUN header. After a ChannelBind, data is sent with a minimal 4-byte header:

[Diagram: see online version]

Field	Size	Description
Channel Number	16 bits	0x4000-0x7FFF (distinguishes from STUN by first two bits being `01`)
Length	16 bits	Length of application data
Data	Variable	The relayed payload (padded to 4 bytes over UDP)

ChannelData saves 36+ bytes of overhead per packet compared to Send/Data indications -- critical for high-frequency media streams.

Address Types in TURN/ICE

[Diagram: see online version]

Candidate Type	Source	Preference (typical)
Host	Local network interface	Highest
Server Reflexive (srflx)	STUN Binding response	Medium
Relay	TURN Allocate response	Lowest (but always works)

Authentication

TURN requires long-term credentials (unlike STUN Binding which can be unauthenticated):

Field	Description
USERNAME	Client's username
REALM	Server's authentication realm
NONCE	Server-provided nonce (time-limited)
MESSAGE-INTEGRITY	HMAC-SHA1 computed with `MD5(username:realm:password)`

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 8656	Traversal Using Relays around NAT (TURN)
RFC 5766	TURN (previous version)
RFC 6062	TURN Extensions for TCP Allocations
RFC 7065	URI Scheme for TURN
RFC 8489	STUN -- base protocol TURN extends

See Also

- STUN -- base protocol that TURN extends
- ICE -- orchestrates STUN and TURN for connectivity
- WebRTC -- primary consumer of TURN relays
- UDP

WebRTC -- Web Real-Time Communication

Standard: RFC 8825 | **Layer:** Application (Layer 7) | **Wireshark filter:** `stun || dtls || rtp || rtcp || sctp`

WebRTC is a framework for peer-to-peer real-time audio, video, and data communication directly between web browsers and native applications. Rather than being a single protocol, WebRTC is an architecture that composes several existing protocols: ICE for connectivity, DTLS for key exchange, SRTP/SRTCP for encrypted media, and SCTP-over-DTLS for data channels. WebRTC is built into all major browsers and requires no plugins.

Protocol Stack

[Diagram: see online version]

Connection Establishment

WebRTC uses an offer/answer model via SDP, exchanged over an application-defined signaling channel (WebSocket, HTTP, etc.):

[Diagram: see online version]

Component Protocols

ICE (Interactive Connectivity Establishment)

ICE finds the best network path between peers, handling NATs and firewalls:

Step	Description
Gathering	Collect candidate addresses: host (local IP), server-reflexive (STUN), relay (TURN)
Exchange	Candidates sent via signaling in SDP or as trickle-ICE updates
Connectivity checks	STUN binding requests between all candidate pairs
Nomination	Best working pair is selected

STUN Message

[Diagram: see online version]

Field	Size	Description
Type	16 bits	Message class (request/response) and method (Binding, etc.)

Message Length	16 bits	Payload length excluding 20-byte header
Magic Cookie	32 bits	Fixed value `0x2112A442`
Transaction ID	96 bits	Unique identifier matching requests to responses

DTLS (Datagram TLS)

DTLS adapts TLS for unreliable datagram transport (UDP). In WebRTC, DTLS serves two purposes:

1. **Key exchange** -- SRTP encryption keys are derived from the DTLS handshake (DTLS-SRTP, RFC 5764)
2. **Data channel encryption** -- SCTP runs inside the DTLS tunnel

The DTLS fingerprint (hash of the certificate) is exchanged in SDP and verified during the handshake, providing end-to-end authentication without a certificate authority.

SRTP (Secure RTP)

SRTP encrypts RTP media payloads while leaving the RTP header unencrypted (so middleboxes can route packets). WebRTC mandates:

- **AES-128-CM** or **AES-256-CM** for encryption
- **HMAC-SHA1** for authentication (10-byte tag)
- Keys derived from DTLS-SRTP key exchange

Data Channels (SCTP over DTLS)

[Diagram: see online version]

Data channels provide reliable or unreliable, ordered or unordered message delivery between peers. They use SCTP for stream multiplexing and optional reliability, tunneled through DTLS for encryption. The Data Channel Establishment Protocol (RFC 8832) negotiates channel parameters.

Mandatory Codecs

WebRTC requires browser support for:

Type	Codec	Standard
Audio	Opus	RFC 6716
Audio	G.711 (PCMU/PCMA)	RFC 3551
Video	VP8	RFC 6386
Video	H.264 Constrained Baseline	RFC 6184

Additional widely supported codecs: VP9, AV1, H.265.

SDP Offer/Answer

SDP (RFC 8866) describes the session. Key lines in a WebRTC SDP:

Line	Description
`m=audio 9 UDP/TLS/RTP/SAVPF 111`	Audio media line with Opus (PT 111)
`m=video 9 UDP/TLS/RTP/SAVPF 96`	Video media line with VP8 (PT 96)
`a=rtpmap:111 opus/48000/2`	Maps PT 111 to Opus codec
`a=fingerprint:sha-256 ...`	DTLS certificate fingerprint
`a=ice-ufrag:` / `a=ice-pwd:`	ICE credentials
`a=candidate:...`	ICE candidate addresses
`a=mid:0`	Media ID for bundle
`a=group:BUNDLE 0 1 2`	Multiplex all media on one transport

Encapsulation

[Diagram: see online version]

All WebRTC traffic is multiplexed on a single UDP port per peer connection. The packet type is identified by the first byte.

Standards

Document	Title
RFC 8825	Overview: Real-Time Communication in WEB-browsers
RFC 8445	Interactive Connectivity Establishment (ICE)
RFC 8489	Session Traversal Utilities for NAT (STUN)
RFC 8656	Traversal Using Relays around NAT (TURN)
RFC 5764	DTLS-SRTP -- key exchange for SRTP
RFC 8831	WebRTC Data Channels
RFC 8832	WebRTC Data Channel Establishment Protocol
RFC 8834	Media Transport and Use of RTP in WebRTC
RFC 8866	SDP: Session Description Protocol
RFC 8829	JavaScript Session Establishment Protocol (JSEP)

See Also

- RTP -- media transport carried inside SRTP
- RTCP -- quality feedback for adaptive bitrate
- SIP -- alternative signaling protocol (WebRTC uses custom signaling)
- TLS -- WebRTC uses DTLS, the datagram variant

- UDP

Web & API

Web application, API, and hypertext protocols.

16 protocols in this section

BitTorrent

Standard: BEP 3 | **Layer:** Application (Layer 7) | **Wireshark filter:** `bittorrent`

BitTorrent is a peer-to-peer file sharing protocol that distributes large files by splitting them into small pieces and downloading different pieces from multiple peers simultaneously. Instead of a single server bearing all load, every downloading client also uploads pieces it has already received, creating a decentralized swarm. A `.torrent` file (or magnet link) identifies the content via a SHA-1 hash (`info_hash`), and peers discover each other through trackers, DHT (Distributed Hash Table), or PEX (Peer Exchange). BitTorrent uses TCP for the peer wire protocol and HTTP/UDP for tracker communication.

Piece and Block Structure

Files are split into fixed-size **pieces** (typically 256 KB to 4 MB), each identified by a SHA-1 hash in the torrent metadata. Peers request **blocks** within pieces (typically 16 KB), which are the smallest transfer unit.

Concept	Size	Description
Piece	256 KB - 4 MB	Unit of integrity verification (SHA-1 hash per piece)
Block	16 KB (typical)	Unit of transfer (what `request` messages ask for)
Info Hash	20 bytes	SHA-1 of the bencoded `info` dictionary in the torrent metadata
Peer ID	20 bytes	Unique identifier for each client instance

Tracker Communication

Clients announce themselves to a tracker (HTTP or UDP) to discover peers in the swarm.

HTTP Tracker Announce

[Diagram: see online version]

Tracker Request Parameters

Parameter	Description
`info_hash`	20-byte SHA-1 hash of the `info` dictionary (URL-encoded)
`peer_id`	20-byte unique client identifier
`port`	Port the client is listening on (typically 6881-6889)
`uploaded`	Total bytes uploaded since `started` event
`downloaded`	Total bytes downloaded since `started` event

`left`	Bytes remaining to download
`event`	`started`, `completed`, `stopped`, or empty (periodic)
`compact`	`1` = request compact peer list (6 bytes per peer: 4 IP + 2 port)
`numwant`	Number of peers desired (default 50)

Tracker Response Fields

Field	Description
`interval`	Seconds between re-announces
`peers`	List of peers (compact: concatenated 6-byte entries, or list of dicts)
`complete`	Number of seeders
`incomplete`	Number of leechers

Peer Wire Protocol (TCP)

Handshake

After connecting to a peer via TCP, both sides exchange a handshake:

[Diagram: see online version]

Field	Size	Description
PStrLen	1 byte	Length of protocol string: `19`
Protocol String	19 bytes	`BitTorrent protocol`
Reserved	8 bytes	Extension flags (bit 20 = DHT, bit 44 = Extension Protocol BEP 10)
Info Hash	20 bytes	SHA-1 hash identifying the torrent
Peer ID	20 bytes	Client's unique identifier

Peer Wire Handshake and Piece Exchange

[Diagram: see online version]

Message Format

All messages after the handshake are length-prefixed:

[Diagram: see online version]

Message Types

ID	Name	Payload	Description
--	Keep-Alive	(none, length=0)	Sent to maintain connection (no message ID)
0	Choke	(none)	Sender will not upload to receiver
1	Unchoke	(none)	Sender will upload to receiver
2	Interested	(none)	Sender wants data the receiver has
3	Not Interested	(none)	Sender does not want data the receiver has
4	Have	piece index (4 bytes)	Sender has completed and verified this piece
5	Bitfield	bitfield (variable)	Bitmap of all pieces the sender has (sent once after handshake)
6	Request	index (4) + begin (4) + length (4)	Request a block within a piece
7	Piece	index (4) + begin (4) + block (variable)	Deliver a block of data
8	Cancel	index (4) + begin (4) + length (4)	Cancel a pending request
9	Port	port (2 bytes)	DHT listening port (BEP 5)

Choking Algorithm

BitTorrent uses a tit-for-tat incentive mechanism:

Concept	Description
Choked	Peer will not send data (default state)
Unchoked	Peer is willing to send data
Interested	Peer wants something the other has
Optimistic Unchoke	Periodically unchoke a random peer (every 30s) to discover faster peers
Regular Unchoke	Unchoke top 4 peers by upload rate (re-evaluated every 10s)

DHT (Distributed Hash Table) -- BEP 5

DHT enables trackerless peer discovery using a Kademlia-based distributed hash table over UDP.

Concept	Description
Node ID	160-bit identifier (same space as info_hash)

Distance	XOR metric between node IDs
Routing Table	K-buckets of known nodes
UDP Port	Same as peer wire port (or declared via `port` message)

DHT Queries (Krpc Protocol)

Query	Description
`ping`	Check if a node is alive
`find_node`	Find nodes closest to a target ID
`get_peers`	Find peers for a specific info_hash
`announce_peer`	Announce that this node is downloading a torrent

Magnet Links

Magnet links encode the info_hash so torrents can be found via DHT without a `.torrent` file:

```
magnet:?xt=urn:btih:{info_hash}&amp;dn={display_name}&amp;tr={tracker_url}
```

Parameter	Description
`xt`	Exact Topic -- `urn:btih:` + hex or base32 info_hash
`dn`	Display Name -- human-readable title
`tr`	Tracker URL (optional, can be repeated)

The client uses BEP 9 (Extension for Peers to Send Metadata Files) to fetch the full torrent metadata from peers in the swarm.

Extensions

PEX (Peer Exchange) -- BEP 11

Peers share their known peer lists with each other, reducing tracker dependency. Exchanged via the Extension Protocol (BEP 10) with message ID `ut_pex`.

uTP (Micro Transport Protocol) -- BEP 29

A UDP-based transport for BitTorrent that uses LEDBAT (Low Extra Delay Background Transport) congestion control, yielding to TCP traffic to avoid degrading other applications on the same connection.

Feature	TCP	uTP
Transport	TCP	UDP

Congestion control	Standard TCP (CUBIC, etc.)	LEDBAT (delay-based, low priority)
Impact on other traffic	Can saturate link	Yields to TCP traffic
NAT traversal	Standard	Better (UDP hole punching)

Key BEPs (BitTorrent Enhancement Proposals)

BEP	Title	Description
BEP 3	The BitTorrent Protocol	Base protocol specification
BEP 5	DHT Protocol	Kademlia-based trackerless peer discovery
BEP 9	Extension for Peers to Send Metadata	Enables magnet links (metadata transfer)
BEP 10	Extension Protocol	Extensible messaging framework
BEP 11	Peer Exchange (PEX)	Peers share known peer lists
BEP 15	UDP Tracker Protocol	Compact UDP tracker announce
BEP 29	uTP	UDP-based LEDBAT transport

Encapsulation

[Diagram: see online version]

Standards

Document	Title
BEP 3	The BitTorrent Protocol Specification
BEP 5	DHT Protocol
BEP 10	Extension Protocol
BEP Index	Index of BitTorrent Enhancement Proposals

See Also

- HTTP -- used for tracker communication
- UDP -- DHT and uTP transport
- TCP -- peer wire protocol transport

CalDAV -- Calendaring Extensions to WebDAV

Standard: RFC 4791 | **Layer:** Application (Layer 7) | **Wireshark filter:** `http` (CalDAV runs over HTTPS)

CalDAV is a calendar synchronization protocol built on top of WebDAV (HTTP extensions for distributed authoring). It allows clients to create, retrieve, update, and delete calendar events on a remote server using standard HTTP methods plus WebDAV extensions. Each calendar is a WebDAV collection, and each event is a resource stored as an iCalendar (`.ics`) object. CalDAV is the open standard behind calendar sync in Apple Calendar, Thunderbird, GNOME Calendar, and dozens of other clients, competing with proprietary sync protocols like Exchange ActiveSync and Google Calendar API.

Architecture

CalDAV organizes calendars as a hierarchy of WebDAV collections:

```
/calendars/                              &lt;- Calendar home (root)
  alice/                                 &lt;- User principal
    work/                                &lt;- Calendar collection
      meeting-abc123.ics                 &lt;- Event resource (iCalendar)
      standup-def456.ics                 &lt;- Event resource
    personal/                            &lt;- Another calendar collection
      birthday-ghi789.ics               &lt;- Event resource
```

HTTP Methods

CalDAV uses standard HTTP methods plus WebDAV extensions:

Method	Description
GET	Retrieve a single `.ics` event resource
PUT	Create or update an event (client provides the `.ics` body)
DELETE	Remove an event resource
MKCALENDAR	Create a new calendar collection
PROPFIND	Query properties of calendars or events (WebDAV)
PROPPATCH	Modify properties of a calendar (WebDAV)
REPORT	Execute a query -- calendar-query, calendar-multiget, free-busy-query

Event Creation Flow

[Diagram: see online version]

`If-None-Match: *` prevents overwriting an existing event. `If-Match` ensures no concurrent modification (optimistic locking via ETags).

Calendar Query (REPORT)

The calendar-query REPORT retrieves events matching filter criteria without fetching every resource individually:

[Diagram: see online version]

calendar-query Request Body

```
&lt;?xml version="1.0" encoding="utf-8" ?&gt;
&lt;C:calendar-query xmlns:D="DAV:" xmlns:C="urn:ietf:params:xml:ns:caldav"&gt;
  &lt;D:prop&gt;
    &lt;D:getetag/&gt;
    &lt;C:calendar-data/&gt;
  &lt;/D:prop&gt;
  &lt;C:filter&gt;
    &lt;C:comp-filter name="VCALENDAR"&gt;
      &lt;C:comp-filter name="VEVENT"&gt;
        &lt;C:time-range start="20240301T000000Z" end="20240401T000000Z"/&gt;
      &lt;/C:comp-filter&gt;
    &lt;/C:comp-filter&gt;
  &lt;/C:filter&gt;
&lt;/C:calendar-query&gt;
```

REPORT Types

Report	Description
calendar-query	Filter events by component type, date range, or property
calendar-multiget	Fetch specific events by their href/UID (bulk retrieval)
free-busy-query	Request free/busy information for a time range

CalDAV Properties

Properties are queried via PROPFIND and modified via PROPPATCH:

Property	Namespace	Description
calendar-home-set	CalDAV	URL of the user's calendar home collection
supported-calendar-component-set	CalDAV	Which components the calendar accepts (VEVENT, VTODO, etc.)
calendar-description	CalDAV	Human-readable calendar description
calendar-timezone	CalDAV	Default time zone for the calendar (VTIMEZONE)
max-resource-size	CalDAV	Maximum size of an individual iCalendar resource
supported-calendar-data	CalDAV	Supported iCalendar versions and formats
displayname	DAV	Calendar display name

getctag	CS (CalendarServer)	Calendar collection tag -- changes when any event changes
getetag	DAV	Entity tag for individual resources (concurrency control)
sync-token	DAV	Incremental sync token (RFC 6578)
calendar-color	Apple	Calendar color (e.g., `#FF5733FF`)
calendar-order	Apple	Sort order for calendar display

Synchronization

Clients need to detect changes efficiently without re-downloading all events:

Strategy	How It Works
CTag polling	Client stores `getctag` value; if it changes, something in the calendar changed -- then use PROPFIND to find changed ETags
ETag comparison	PROPFIND for all ETags, compare with local cache, GET only changed items
WebDAV Sync (RFC 6578)	Client sends `sync-token`, server returns only resources changed since that token

WebDAV Sync (RFC 6578) is the most efficient approach and is supported by all modern servers.

Service Discovery

CalDAV clients discover the server automatically using two mechanisms:

Method	Description
`.well-known` URI	Client requests `https://example.com/.well-known/caldav` and follows the redirect
DNS SRV records	`_caldavs._tcp.example.com` (TLS) or `_caldav._tcp.example.com` (plain)
DNS TXT record	`_caldavs._tcp.example.com` TXT `path=/calendars/`

Discovery defined in RFC 6764. The full discovery flow:

1. Look up DNS SRV record `_caldavs._tcp.example.com`
2. Or try `https://example.com/.well-known/caldav` (expect 301/302 redirect)
3. PROPFIND on redirect target to find `current-user-principal`
4. PROPFIND on principal to find `calendar-home-set`
5. PROPFIND on home set to enumerate calendar collections

Scheduling Extensions (RFC 6638)

When organizer and attendees share the same CalDAV server, scheduling can happen automatically:

Component	Description
schedule-outbox	Collection where the client submits scheduling messages (invitations, replies)
schedule-inbox	Collection where the server delivers incoming scheduling messages
Implicit scheduling	Server automatically sends iTIP REQUEST/REPLY/CANCEL when events are created/modified
schedule-tag	ETag-like value that changes only when scheduling-relevant properties change

With implicit scheduling, the client simply PUTs an event with ATTENDEEs, and the server handles iTIP delivery automatically.

Authentication

Method	Description
HTTP Basic over TLS	Username/password (most common for self-hosted)
HTTP Digest	Challenge-response (less common)
OAuth 2.0	Token-based (Google Calendar, Microsoft)
Client certificates	TLS mutual authentication

CalDAV vs Proprietary Sync

Feature	CalDAV	Exchange (EAS/EWS)	Google Calendar API
Protocol	HTTP/WebDAV + XML	HTTP + WBXML (EAS) / SOAP (EWS)	HTTP + JSON (REST)
Data format	iCalendar (text/calendar)	Proprietary XML/WBXML	JSON (proprietary schema)
Open standard	Yes (RFC 4791)	No (licensed)	No (proprietary)
Scheduling	iTIP via RFC 6638	Built-in	Built-in
Discovery	.well-known + SRV	Autodiscover	OAuth + hardcoded endpoints
Offline sync	ETag/sync-token based	Sync keys	Sync tokens
Server implementations	Many (open-source and commercial)	Microsoft Exchange only	Google only

Major Implementations

Implementation	Type	Notes

Apple iCloud Calendar	Server + Client	macOS/iOS native CalDAV support
Google Calendar	Server	CalDAV access alongside proprietary API
Nextcloud	Server	Open-source, self-hosted
Radicale	Server	Lightweight Python CalDAV/CardDAV server
Baikal	Server	Lightweight PHP CalDAV/CardDAV server
DAViCal	Server	PostgreSQL-backed CalDAV server
Cyrus IMAP	Server	Also provides CalDAV/CardDAV
Thunderbird	Client	Via built-in CalDAV support
GNOME Calendar	Client	Native CalDAV
DAVx5	Client	Android CalDAV/CardDAV sync adapter

Standards

Document	Title
RFC 4791	Calendaring Extensions to WebDAV (CalDAV)
RFC 6638	Scheduling Extensions to CalDAV
RFC 6764	Locating Services for Calendaring Extensions to WebDAV (CalDAV) and vCard Extensions to WebDAV (CardDAV)
RFC 4918	HTTP Extensions for Web Distributed Authoring and Versioning (WebDAV)
RFC 6578	Collection Synchronization for WebDAV
RFC 5545	iCalendar (the data format CalDAV carries)

See Also

- iCalendar -- the data format CalDAV stores and transports
- CardDAV -- sister protocol for contact sync (also WebDAV-based)
- HTTP -- the transport protocol underneath CalDAV
- WebSocket -- some CalDAV servers support push notifications via WebSocket
- JMAP -- modern alternative with JSCalendar support (RFC 8984)

CardDAV -- vCard Extensions to WebDAV

Standard: RFC 6352 | **Layer:** Application (Layer 7) | **Wireshark filter:** `http` (CardDAV runs over HTTPS)

CardDAV is a contact synchronization protocol built on top of WebDAV (HTTP extensions for distributed authoring). It allows clients to create, retrieve, update, and delete contacts on a remote server using standard HTTP methods plus WebDAV extensions. Each address book is a WebDAV collection, and each contact is a resource stored as a vCard (`.vcf`) object. CardDAV is the open standard behind contact sync in Apple Contacts, Thunderbird, GNOME Contacts, and Android (via DAVx5), serving as the contact counterpart to CalDAV for calendars.

Architecture

CardDAV organizes contacts as a hierarchy of WebDAV collections:

```
/addressbooks/                          &lt;- Address book home (root)
  alice/                                &lt;- User principal
    contacts/                           &lt;- Address book collection
      grace-abc123.vcf                  &lt;- Contact resource (vCard)
      alan-def456.vcf                   &lt;- Contact resource
    coworkers/                          &lt;- Another address book collection
      bob-ghi789.vcf                    &lt;- Contact resource
```

HTTP Methods

CardDAV uses standard HTTP methods plus WebDAV extensions:

Method	Description
GET	Retrieve a single `.vcf` contact resource
PUT	Create or update a contact (client provides the vCard body)
DELETE	Remove a contact resource
PROPFIND	Query properties of address books or contacts (WebDAV)
PROPPATCH	Modify properties of an address book (WebDAV)
REPORT	Execute a query -- addressbook-query, addressbook-multiget

Unlike CalDAV, CardDAV has no `MKCOL`-equivalent extension (address books are created via standard WebDAV `MKCOL` with the appropriate resource type).

Contact Creation Flow

[Diagram: see online version]

`If-None-Match: *` prevents overwriting an existing contact. `If-Match` ensures no concurrent modification (optimistic locking via ETags).

Address Book Query (REPORT)

The addressbook-query REPORT retrieves contacts matching filter criteria without downloading every resource:

[Diagram: see online version]

addressbook-query Request Body

```
&lt;?xml version="1.0" encoding="utf-8" ?&gt;
&lt;C:addressbook-query xmlns:D="DAV:" xmlns:C="urn:ietf:params:xml:ns:carddav"&gt;
  &lt;D:prop&gt;
    &lt;D:getetag/&gt;
    &lt;C:address-data/&gt;
  &lt;/D:prop&gt;
  &lt;C:filter&gt;
    &lt;C:prop-filter name="EMAIL"&gt;
      &lt;C:text-match collation="i;unicode-casemap" match-type="contains"&gt;
        @example.com
      &lt;/C:text-match&gt;
    &lt;/C:prop-filter&gt;
  &lt;/C:filter&gt;
&lt;/C:addressbook-query&gt;
```

REPORT Types

Report	Description
addressbook-query	Filter contacts by vCard property values (name, email, phone, etc.)
addressbook-multiget	Fetch specific contacts by their href/UID (bulk retrieval)

Filter Match Types

Match Type	Description
equals	Exact match
contains	Substring match
starts-with	Prefix match
ends-with	Suffix match

All text matches support the `i;unicode-casemap` collation for case-insensitive comparison.

CardDAV Properties

Properties are queried via PROPFIND and modified via PROPPATCH:

Property	Namespace	Description
addressbook-home-set	CardDAV	URL of the user's address book home collection
supported-address-data	CardDAV	Supported vCard versions (3.0, 4.0) and content types
max-resource-size	CardDAV	Maximum size of an individual vCard resource

displayname	DAV	Address book display name
getctag	CS (CalendarServer)	Collection tag -- changes when any contact changes
getetag	DAV	Entity tag for individual resources (concurrency control)
sync-token	DAV	Incremental sync token (RFC 6578)
resourcetype	DAV	Includes `addressbook` to identify address book collections

Synchronization

Clients need to detect changes efficiently without re-downloading all contacts:

Strategy	How It Works
CTag polling	Client stores `getctag`; if it changes, re-enumerate ETags via PROPFIND
ETag comparison	PROPFIND for all ETags, compare with local cache, GET only changed items
WebDAV Sync (RFC 6578)	Client sends `sync-token`, server returns only resources changed since that token

WebDAV Sync (RFC 6578) is the most efficient approach. A typical sync cycle:

1. Client sends `sync-collection` REPORT with stored `sync-token`
2. Server returns new/modified/deleted hrefs plus a new `sync-token`
3. Client uses `addressbook-multiget` to fetch changed contacts in bulk
4. Client stores the new `sync-token` for the next cycle

Service Discovery

CardDAV clients discover the server automatically using the same mechanisms as CalDAV:

Method	Description
`.well-known` URI	Client requests `https://example.com/.well-known/carddav` and follows the redirect
DNS SRV records	`_carddavs._tcp.example.com` (TLS) or `_carddav._tcp.example.com` (plain)
DNS TXT record	`_carddavs._tcp.example.com` TXT `path=/addressbooks/`

Discovery defined in RFC 6764. The full discovery flow:

1. Look up DNS SRV record `_carddavs._tcp.example.com`
2. Or try `https://example.com/.well-known/carddav` (expect 301/302 redirect)
3. PROPFIND on redirect target to find `current-user-principal`

4. PROPFIND on principal to find `addressbook-home-set`
5. PROPFIND on home set to enumerate address book collections

Authentication

Method	Description
HTTP Basic over TLS	Username/password (most common for self-hosted)
HTTP Digest	Challenge-response (less common)
OAuth 2.0	Token-based (Google Contacts, Microsoft)
Client certificates	TLS mutual authentication

CardDAV vs LDAP vs Proprietary Sync

Feature	CardDAV	LDAP	Google People API	Exchange (EAS)
Protocol	HTTP/WebDAV + XML	Binary over TCP	HTTP + JSON (REST)	HTTP + WBXML
Data format	vCard (text/vcard)	Directory entries (ASN.1/BER)	JSON (proprietary schema)	Proprietary WBXML
Open standard	Yes (RFC 6352)	Yes (RFC 4511)	No (proprietary)	No (licensed)
Primary use	Contact sync (personal)	Directory lookup (enterprise)	Contact sync (Google)	Contact sync (Microsoft)
Bidirectional sync	Yes	Limited (mostly read)	Yes	Yes
Search/filter	addressbook-query REPORT	LDAP search filters	API query parameters	Sync commands
Discovery	.well-known + SRV	DNS SRV (_ldap._tcp)	OAuth + hardcoded endpoints	Autodiscover
Offline sync	ETag/sync-token based	Not designed for offline	Sync tokens	Sync keys
Photo support	vCard PHOTO property	jpegPhoto attribute	API field	EAS field

Major Implementations

Implementation	Type	Notes
Apple iCloud Contacts	Server + Client	macOS/iOS native CardDAV support
Google Contacts	Server	CardDAV access alongside People API
Nextcloud	Server	Open-source, self-hosted
Radicale	Server	Lightweight Python CalDAV/CardDAV server

Baikal	Server	Lightweight PHP CalDAV/CardDAV server
Cyrus IMAP	Server	Also provides CalDAV/CardDAV
Thunderbird	Client	Via built-in CardDAV support
GNOME Contacts	Client	Native CardDAV via GNOME Online Accounts
DAVx5	Client	Android CalDAV/CardDAV sync adapter
Evolution	Client	GNOME PIM client with CardDAV support

Standards

Document	Title
RFC 6352	CardDAV: vCard Extensions to Web Distributed Authoring and Versioning (WebDAV)
RFC 6764	Locating Services for CalDAV and CardDAV
RFC 4918	HTTP Extensions for Web Distributed Authoring and Versioning (WebDAV)
RFC 6578	Collection Synchronization for WebDAV
RFC 6350	vCard Format Specification (the data format CardDAV carries)

See Also

- vCard -- the data format CardDAV stores and transports
- CalDAV -- sister protocol for calendar sync (also WebDAV-based)
- LDAP -- directory protocol for enterprise contact lookup
- HTTP -- the transport protocol underneath CardDAV
- SCIM -- identity provisioning protocol (user management, not contact sync)

CoAP -- Constrained Application Protocol

Standard: RFC 7252 | **Layer:** Application (Layer 7) | **Wireshark filter:** `coap`

CoAP is a lightweight RESTful protocol designed for constrained IoT devices and low-power, lossy networks (6LoWPAN, Thread, NB-IoT). It follows the same REST model as HTTP (GET, POST, PUT, DELETE on URIs) but uses a compact binary format over UDP with a 4-byte fixed header, built-in retransmission, multicast support, and resource observation (server-push). CoAP is the "HTTP of IoT" -- it's the standard application protocol for devices too small to run a full TCP/HTTP stack.

Header

[Diagram: see online version]

The fixed header is only 4 bytes -- the smallest of any RESTful protocol.

Key Fields

Field	Size	Description
Version	2 bits	Always 1
Type	2 bits	CON, NON, ACK, RST
Token Length	4 bits	Length of the Token field (0-8)
Code	8 bits	Method (request) or response code
Message ID	16 bits	Deduplication and matching for CON/ACK
Token	0-8 bytes	Matches responses to requests (chosen by client)
Options	Variable	URI-Path, Content-Format, Observe, etc.
Payload Marker	8 bits	0xFF separates options from payload
Payload	Variable	Request/response body

Message Types

Type	Value	Description
CON	0	Confirmable -- requires ACK (reliable)
NON	1	Non-confirmable -- fire-and-forget (unreliable)
ACK	2	Acknowledgment of a CON
RST	3	Reset -- indicates a received message was rejected

Reliable vs Unreliable

[Diagram: see online version]

Method Codes (Request)

Code	Name	HTTP Equivalent
0.01	GET	GET

0.02	POST	POST
0.03	PUT	PUT
0.04	DELETE	DELETE
0.05	FETCH	FETCH (RFC 8132)
0.06	PATCH	PATCH (RFC 8132)
0.07	iPATCH	Idempotent PATCH (RFC 8132)

Response Codes

Code	Name	HTTP Equivalent
2.01	Created	201
2.02	Deleted	204
2.03	Valid	304 (ETag match)
2.04	Changed	204
2.05	Content	200
4.00	Bad Request	400
4.01	Unauthorized	401
4.03	Forbidden	403
4.04	Not Found	404
4.05	Method Not Allowed	405
5.00	Internal Server Error	500
5.03	Service Unavailable	503

Common Options

Number	Name	Description
1	If-Match	Conditional (ETag match)
3	Uri-Host	Hostname (virtual hosting)
4	ETag	Entity tag for caching
6	Observe	Register for notifications (0=register, 1=deregister)
7	Uri-Port	Port number
8	Location-Path	Response location
11	Uri-Path	Path segment (repeatable: /a/b/c = three options)
12	Content-Format	Media type of payload

14	Max-Age	Cache freshness in seconds (default 60)
15	Uri-Query	Query parameter (repeatable)
17	Accept	Accepted content formats
35	Proxy-Uri	URI for proxy requests
39	Proxy-Scheme	Scheme for proxy
60	Size1	Request body size hint

Content Formats

Value	Media Type
0	text/plain; charset=utf-8
40	application/link-format
41	application/xml
42	application/octet-stream
47	application/exi
50	application/json
60	application/cbor
11542	application/vnd.oma.lwm2m+tlv

Observe (RFC 7641)

Observe provides server push -- the client registers interest in a resource, and the server sends notifications when the value changes:

[Diagram: see online version]

Resource Discovery (/.well-known/core)

CoAP devices advertise their resources in CoRE Link Format (RFC 6690):

```
GET coap://sensor.local/.well-known/core

&lt;/temperature&gt;;rt="temperature";if="sensor";ct=0,
&lt;/humidity&gt;;rt="humidity";if="sensor";ct=0,
&lt;/led&gt;;rt="actuator";if="switch"
```

CoAP vs HTTP

Feature	CoAP	HTTP
Transport	UDP (default), TCP, WebSocket	TCP

Header	4 bytes (binary)	~100+ bytes (text)
Methods	GET, POST, PUT, DELETE, FETCH, PATCH	GET, POST, PUT, DELETE, PATCH, etc.
Multicast	Yes (group requests)	No
Observe (push)	Built-in	SSE or WebSocket
Security	DTLS (coaps://)	TLS (https://)
Proxy	CoAP-to-HTTP proxies standardized	N/A
Target	Constrained devices (8-bit MCU, 10KB RAM)	General computing

Security (DTLS)

CoAP over DTLS is indicated by the `coaps://` scheme:

Security Mode	Description
NoSec	No security (coap://)
PreSharedKey	DTLS with pre-shared keys (constrained devices)
RawPublicKey	DTLS with raw public keys (no CA)
Certificate	DTLS with X.509 certificates
OSCORE	Object Security (RFC 8613) -- end-to-end through proxies

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 7252	The Constrained Application Protocol (CoAP)
RFC 7641	Observing Resources in CoAP
RFC 7959	Block-Wise Transfers in CoAP
RFC 6690	CoRE Link Format (resource discovery)
RFC 8323	CoAP over TCP, TLS, and WebSocket
RFC 8613	OSCORE (Object Security for CoAP)
RFC 8132	FETCH and PATCH methods for CoAP

See Also

- MQTT -- alternative IoT protocol (broker-based pub/sub)
- HTTP -- the protocol CoAP is modeled after
- DTLS -- security layer for CoAP (coaps://)
- UDP -- primary CoAP transport
- Zigbee -- constrained wireless network CoAP runs on

GraphQL

Standard: GraphQL Specification | **Layer:** Application (Layer 7) | **Wireshark filter:** `http`

GraphQL is a query language and runtime for APIs, developed by Facebook and open-sourced in 2015. Unlike REST, where the server defines fixed endpoints returning fixed data shapes, GraphQL lets clients specify exactly what data they need in a single request. A GraphQL service exposes a strongly typed schema; clients send queries, mutations, or subscriptions against that schema. The transport is typically HTTP POST with a JSON body, though subscriptions commonly use WebSocket. The GraphQL Foundation (under the Linux Foundation) maintains the specification.

HTTP Transport Format

GraphQL requests are typically sent as HTTP POST with `Content-Type: application/json`:

[Diagram: see online version]

Request JSON Body

Field	Type	Required	Description
query	String	Yes	The GraphQL document (query, mutation, or subscription)
variables	Object	No	Variable values for parameterized queries
operationName	String	No	Name of the operation to execute (when document contains multiple)

Response JSON Body

Field	Type	Present	Description
data	Object	On success	The result of the executed operation, shaped to match the query

errors	Array	On error	List of error objects with `message`, `locations`, `path`, `extensions`
extensions	Object	Optional	Implementation-specific metadata (tracing, caching hints)

Operation Types

Type	Keyword	Description
Query	`query`	Read-only fetch (analogous to GET)
Mutation	`mutation`	Write then fetch (analogous to POST/PUT/DELETE)
Subscription	`subscription`	Long-lived connection pushing data on events

Query Example

```
query GetUser($id: ID!) {
  user(id: $id) {
    name
    email
    posts(first: 5) {
      title
      createdAt
    }
  }
}
```

Variables: `{ "id": "42" }`

Type System

GraphQL schemas are built from these type kinds:

Kind	Description	Example
Scalar	Primitive leaf values	`Int`, `Float`, `String`, `Boolean`, `ID`, custom scalars
Object	Named type with fields	`type User { name: String! }`
Interface	Abstract type with required fields	`interface Node { id: ID! }`
Union	One of several object types	`union SearchResult = User \
Enum	Fixed set of allowed values	`enum Status { ACTIVE, INACTIVE }`
Input Object	Object type for arguments	`input CreateUserInput { name: String! }`

List	Ordered collection	`[String]`
Non-Null	Value guaranteed present	`String!`

Key Language Features

Feature	Description
Selection Sets	Nested field selections that shape the response
Arguments	Parameters on fields (`user(id: "42")`)
Variables	Parameterized values (`$id: ID!`) passed separately from the query
Fragments	Reusable field selections (`fragment UserFields on User { ... }`)
Directives	Annotations modifying execution (`@include(if: $flag)`, `@skip`, `@deprecated`)
Aliases	Rename fields in the response (`admin: user(role: ADMIN) { name }`)
Inline Fragments	Type-conditional selections (`... on Post { title }`)

Introspection

GraphQL services must support introspection queries, allowing clients and tools to discover the schema at runtime:

Query	Description
`__schema`	Root introspection entry -- returns types, directives, queryType, mutationType
`__type(name: "User")`	Details of a specific named type -- fields, interfaces, enum values
`__typename`	Meta-field returning the concrete type name of any object

Query Flow

[Diagram: see online version]

Error Response

[Diagram: see online version]

Subscriptions over WebSocket

GraphQL subscriptions use a persistent WebSocket connection. The `graphql-ws` protocol (newer) and `subscriptions-transport-ws` (legacy) define the message exchange:

graphql-ws Sub-protocol

Message Type	Direction	Description
`connection_init`	Client -> Server	Initialize connection (optionally with auth payload)
`connection_ack`	Server -> Client	Acknowledge connection
`subscribe`	Client -> Server	Start a subscription (includes query and variables)
`next`	Server -> Client	Subscription event data
`error`	Server -> Client	Subscription error
`complete`	Either	Subscription ended
`ping` / `pong`	Either	Keep-alive

[Diagram: see online version]

Batched Queries

Some GraphQL servers accept an array of operations in a single HTTP request:

```
[
  {"query": "{ user(id: 1) { name } }"},
  {"query": "{ user(id: 2) { name } }"}
]
```

The server returns an array of responses in the same order. This reduces HTTP round trips but is not part of the official specification -- it is a common server extension.

GraphQL vs REST

Feature	GraphQL	REST
Endpoint	Single endpoint (`/graphql`)	Multiple endpoints (`/users`, `/posts`)
Data fetching	Client specifies exact fields needed	Server defines fixed response shape
Over-fetching	Eliminated -- client requests only what it needs	Common -- endpoints return full resources
Under-fetching	Eliminated -- nested data in one query	Common -- requires multiple round trips
Versioning	Schema evolution with `@deprecated`	URL versioning (`/v1/`, `/v2/`)
Caching	More complex (POST body, persisted queries)	Simple (HTTP GET caching, ETags)
Type system	Built-in, strongly typed schema	External (OpenAPI/Swagger)
Real-time	Subscriptions (WebSocket)	SSE, polling, or WebSocket (ad hoc)
File uploads	Not in spec (multipart extension)	Native multipart/form-data

Tooling	Introspection enables auto-generated docs, IDEs	Depends on OpenAPI adoption
Error handling	Always HTTP 200, errors in response body	HTTP status codes (4xx, 5xx)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
GraphQL Specification	GraphQL Language and Type System Specification
graphql-ws Protocol	GraphQL over WebSocket Protocol
GraphQL over HTTP	GraphQL over HTTP Specification (working draft)
GraphQL Multipart Request	GraphQL multipart request spec (file uploads)

See Also

- HTTP -- primary transport for GraphQL
- gRPC -- alternative RPC framework (binary, schema-driven)
- WebSocket -- transport for GraphQL subscriptions
- CoAP -- constrained application protocol (different domain, similar request-response model)

gRPC -- gRPC Remote Procedure Call

Standard: gRPC Specification (grpc.io) | **Layer:** Application (Layer 7) | **Wireshark filter:** `http2` (gRPC runs over HTTP/2)

gRPC is a high-performance, open-source RPC framework developed by Google that uses HTTP/2 for transport and Protocol Buffers (protobuf) for serialization. It provides strongly-typed service definitions, bidirectional streaming, flow control, and automatic code generation for 12+ languages. gRPC is the dominant protocol for microservice-to-microservice communication, Kubernetes APIs, mobile backends, and any scenario requiring efficient, typed inter-service calls.

How gRPC Works

gRPC maps RPC methods to HTTP/2 streams:

[Diagram: see online version]

A `.proto` file defines the service:

```
service Greeter {
  rpc SayHello (HelloRequest) returns (HelloReply);
  rpc Chat (stream ChatMessage) returns (stream ChatMessage);
}
```

The protobuf compiler generates client stubs and server interfaces in the target language.

Wire Format

gRPC messages are carried as HTTP/2 DATA frames with a 5-byte Length-Prefixed Message framing:

[Diagram: see online version]

Field	Size	Description
Compressed Flag	1 byte	0 = uncompressed, 1 = compressed (per Message-Encoding)
Message Length	4 bytes	Length of the protobuf-encoded message
Message	Variable	Serialized Protocol Buffers payload

HTTP/2 Mapping

Request

HTTP/2 Header	Value	Description
`:method`	POST	Always POST
`:path`	/package.Service/Method	Fully qualified method path
`:scheme`	http or https	Transport scheme
`content-type`	application/grpc	Required
`te`	trailers	Required for gRPC
`grpc-encoding`	identity, gzip, deflate, snappy	Compression
`grpc-timeout`	1S, 500m, 100u	Timeout (seconds, milliseconds, microseconds)

Response

HTTP/2 Header	Value	Description
`:status`	200	HTTP status (always 200 for gRPC)

`content-type`	application/grpc	Required
`grpc-status`	0-16	gRPC status code (in trailers)
`grpc-message`	string	Error message (in trailers)

RPC Types

Type	Request	Response	Use Case
Unary	Single message	Single message	Standard request-response
Server Streaming	Single message	Stream of messages	Fetching paginated results, event feeds
Client Streaming	Stream of messages	Single message	Uploading data, batch operations
Bidirectional Streaming	Stream	Stream	Chat, real-time collaboration

Unary RPC Flow

[Diagram: see online version]

Bidirectional Streaming

[Diagram: see online version]

Status Codes

Code	Name	Description
0	OK	Success
1	CANCELLED	Operation cancelled by client
2	UNKNOWN	Unknown error
3	INVALID_ARGUMENT	Bad request data
4	DEADLINE_EXCEEDED	Timeout
5	NOT_FOUND	Resource not found
7	PERMISSION_DENIED	Unauthorized
8	RESOURCE_EXHAUSTED	Rate limited or quota exceeded
12	UNIMPLEMENTED	Method not implemented
13	INTERNAL	Server error
14	UNAVAILABLE	Service temporarily unavailable (retry)
16	UNAUTHENTICATED	Missing or invalid credentials

gRPC vs REST

Feature	gRPC	REST (HTTP/JSON)
Serialization	Protocol Buffers (binary)	JSON (text)
Transport	HTTP/2 (multiplexed)	HTTP/1.1 or HTTP/2
Code generation	Built-in (protoc)	Optional (OpenAPI)
Streaming	Native (4 types)	SSE or WebSocket
Schema	.proto file (strongly typed)	OpenAPI/Swagger (optional)
Browser support	Via grpc-web proxy	Native
Payload size	Compact (binary)	Larger (text)
Latency	Lower	Higher

Encapsulation

[Diagram: see online version]

Standards

Document	Title
gRPC Core Spec	gRPC over HTTP/2 specification
Protocol Buffers	Proto3 language guide and encoding
grpc-web	gRPC for browser clients

See Also

- HTTP -- gRPC runs over HTTP/2
- TLS -- encrypts gRPC connections
- TCP
- MQTT -- alternative for IoT messaging

HLS / MPEG-DASH -- Adaptive Bitrate Streaming

Standard: RFC 8216 (HLS) / ISO 23009-1 (DASH) | **Layer:** Application (Layer 7) | **Wireshark filter:** `http` (HLS and DASH run over HTTP)

HLS (HTTP Live Streaming) and MPEG-DASH (Dynamic Adaptive Streaming over HTTP) are the two dominant protocols for video streaming over the Internet. Both work by dividing media into small segments (typically 2-10

seconds) served over standard HTTP/HTTPS, with a manifest file describing available quality levels. The client adaptively selects segment quality based on network conditions. HLS was created by Apple; DASH is the ISO standard. Together they power Netflix, YouTube, Disney+, live sports, and virtually all modern video delivery.

How Adaptive Streaming Works

[Diagram: see online version]

Playback Flow

[Diagram: see online version]

HLS (HTTP Live Streaming)

Master Playlist (Multi-Variant)

```
#EXTM3U
#EXT-X-STREAM-INF:BANDWIDTH=800000,RESOLUTION=640x360
360p/playlist.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=2400000,RESOLUTION=1280x720
720p/playlist.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=5000000,RESOLUTION=1920x1080
1080p/playlist.m3u8
```

Media Playlist

```
#EXTM3U
#EXT-X-VERSION:7
#EXT-X-TARGETDURATION:6
#EXT-X-MEDIA-SEQUENCE:1
#EXTINF:6.006,
segment001.ts
#EXTINF:6.006,
segment002.ts
#EXTINF:5.338,
segment003.ts
```

Key HLS Tags

Tag	Description
#EXTM3U	Playlist header
#EXT-X-STREAM-INF	Variant stream (bandwidth, resolution, codecs)
#EXTINF	Segment duration
#EXT-X-TARGETDURATION	Maximum segment duration
#EXT-X-MEDIA-SEQUENCE	Sequence number of first segment
#EXT-X-ENDLIST	VOD playlist end (not present for live)

#EXT-X-KEY	Encryption key (AES-128 or SAMPLE-AES)
#EXT-X-MAP	Initialization segment (fMP4)
#EXT-X-DATERANGE	Date-range metadata (ad markers, program info)
#EXT-X-PART	LL-HLS partial segment
#EXT-X-PRELOAD-HINT	LL-HLS preload hint

Container Formats

Format	Extension	Description
MPEG-TS	.ts	Original HLS container (H.264 + AAC)
fMP4	.m4s + init.mp4	Fragmented MP4 (HLS + DASH compatible, preferred for HEVC/AV1)

MPEG-DASH

MPD (Media Presentation Description)

DASH uses an XML manifest:

```
&lt;MPD type="dynamic" minBufferTime="PT2S" availabilityStartTime="2026-03-21T10:00:00Z"&gt;
  &lt;Period&gt;
    &lt;AdaptationSet mimeType="video/mp4" contentType="video"&gt;
      &lt;Representation id="720p" bandwidth="2400000" width="1280" height="720"&gt;
        &lt;SegmentTemplate media="seg_$Number$.m4s" initialization="init.mp4"
          timescale="90000" duration="540000"/&gt;
      &lt;/Representation&gt;
      &lt;Representation id="1080p" bandwidth="5000000" width="1920" height="1080"&gt;
        &lt;SegmentTemplate media="seg_$Number$.m4s" initialization="init.mp4"
          timescale="90000" duration="540000"/&gt;
      &lt;/Representation&gt;
    &lt;/AdaptationSet&gt;
  &lt;/Period&gt;
&lt;/MPD&gt;
```

Key DASH Concepts

Concept	Description
MPD	Media Presentation Description (XML manifest)
Period	Time period (can change available streams, e.g., ad break)
AdaptationSet	Group of interchangeable Representations (same content, different quality)
Representation	Specific quality level (bandwidth, resolution, codec)
Segment	Individual media chunk
SegmentTemplate	URL pattern for segments

HLS vs DASH

Feature	HLS	MPEG-DASH
Manifest	M3U8 (text)	MPD (XML)
Container	TS or fMP4	fMP4 (or WebM)
DRM	FairPlay (Apple)	Widevine, PlayReady, ClearKey
Browser support	Safari native; MSE elsewhere	MSE (all browsers except Safari TS)
Low latency	LL-HLS (Apple)	CMAF + chunked transfer
Segment duration	Typically 6s (2s for LL-HLS)	Typically 2-6s
Live	Yes	Yes
Standard body	IETF (RFC 8216)	ISO (23009-1)

Low-Latency Streaming

Protocol	Latency	Method
Traditional HLS	15-30s	6s segments, 3-segment buffer
LL-HLS	2-4s	Partial segments, preload hints, blocking playlist reload
Low-Latency DASH	2-4s	CMAF chunks, chunked transfer encoding
WebRTC	<1s	Peer-to-peer RTP (not HTTP-based)

Encapsulation

[Diagram: see online version]

Both protocols use standard HTTP/HTTPS -- no special ports or transport.

Standards

Document	Title
RFC 8216	HTTP Live Streaming
RFC 8216bis	HLS 2nd Edition (LL-HLS, fMP4)
ISO 23009-1	MPEG-DASH
CMAF (ISO 23000-19)	Common Media Application Format

See Also

- HTTP -- transport for both HLS and DASH
- TLS -- HTTPS encryption for streaming
- RTP -- alternative real-time media transport (lower latency)
- WebRTC -- sub-second latency alternative

HTTP -- Hypertext Transfer Protocol

Standard: RFC 9110 | **Layer:** Application (Layer 7) | **Wireshark filter:** `http` or `http2`

HTTP is the protocol of the World Wide Web, used for transferring hypertext documents, APIs, and any resource identified by a URL. It follows a request-response model where a client sends a request (method + URL + headers) and a server returns a response (status code + headers + body). HTTP is a text-based protocol in versions 1.0 and 1.1, and binary-framed in HTTP/2 and HTTP/3.

Message Structure

HTTP/1.1 uses a text-based format. Unlike lower-layer protocols, HTTP does not have a fixed-width binary header -- it uses CRLF-delimited text lines.

Request

```
METHOD /path HTTP/1.1\r\n
Header-Name: Header-Value\r\n
Header-Name: Header-Value\r\n
\r\n
[optional body]
```

Response

```
HTTP/1.1 STATUS-CODE Reason-Phrase\r\n
Header-Name: Header-Value\r\n
Header-Name: Header-Value\r\n
\r\n
[optional body]
```

HTTP/2 Frame

HTTP/2 (RFC 9113) uses a binary framing layer:

[Diagram: see online version]

Field	Size	Description

Length	24 bits	Payload length (not including the 9-byte header)
Type	8 bits	Frame type
Flags	8 bits	Type-specific flags
R	1 bit	Reserved, must be zero
Stream ID	31 bits	Identifies the stream (0 for connection-level frames)

HTTP/2 Frame Types

Type	Name	Description
0	DATA	Carries request/response body
1	HEADERS	Carries HTTP headers (compressed with HPACK)
2	PRIORITY	Stream priority (deprecated in RFC 9113)
3	RST_STREAM	Abort a stream
4	SETTINGS	Connection configuration parameters
5	PUSH_PROMISE	Server push notification
6	PING	Keepalive / round-trip time measurement
7	GOAWAY	Graceful connection shutdown
8	WINDOW_UPDATE	Flow control window adjustment
9	CONTINUATION	Continues a HEADERS or PUSH_PROMISE frame

Key Fields

Request Methods

Method	Description
GET	Retrieve a resource
HEAD	Like GET but response has no body
POST	Submit data to a resource
PUT	Replace a resource
DELETE	Remove a resource
PATCH	Partial modification of a resource
OPTIONS	Describe communication options
CONNECT	Establish a tunnel (used for HTTPS proxying)

TRACE	Loop-back test

Response Status Codes

Range	Category	Examples
1xx	Informational	100 Continue, 101 Switching Protocols
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 304 Not Modified
4xx	Client Error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Server Error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

Common Headers

Header	Direction	Description
Host	Request	Target hostname (required in HTTP/1.1)
Content-Type	Both	MIME type of the body
Content-Length	Both	Size of the body in bytes
Authorization	Request	Authentication credentials
User-Agent	Request	Client software identifier
Accept	Request	Acceptable response media types
Cache-Control	Both	Caching directives
Set-Cookie	Response	Set a client cookie
Cookie	Request	Send stored cookies
Location	Response	Redirect target URL

Version Comparison

Feature	HTTP/1.0	HTTP/1.1	HTTP/2	HTTP/3
Connections	One request per connection	Persistent, pipelined	Multiplexed streams	Multiplexed (QUIC)
Header format	Text	Text	Binary (HPACK)	Binary (QPACK)
Transport	TCP	TCP	TCP + TLS	QUIC (UDP)
Server push	No	No	Yes	Yes
Head-of-line blocking	Per connection	Per connection	Per connection (TCP)	Eliminated

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 9110	HTTP Semantics
RFC 9112	HTTP/1.1
RFC 9113	HTTP/2
RFC 9114	HTTP/3
RFC 6265	HTTP State Management Mechanism (Cookies)
RFC 7540	HTTP/2 (original, superseded by RFC 9113)

See Also

- TCP
- TLS
- DNS -- resolves hostnames before HTTP connections

IPP -- Internet Printing Protocol

Standard: RFC 8011 | **Layer:** Application (Layer 7) | **Wireshark filter:** `ipp`

IPP is the standard protocol for printing over HTTP/HTTPS. An IPP client sends print jobs, queries printer capabilities, and manages job queues by POSTing binary-encoded IPP requests to a printer's URI. IPP replaced the venerable LPD/LPR protocol and the raw-socket port 9100 approach by providing rich attribute-based job control, encryption via TLS, and authentication -- all tunneled through standard HTTP. IPP is the foundation of driverless printing: Apple AirPrint, Mopria (Android), and IPP Everywhere all rely on IPP 2.0 with DNS-SD discovery, eliminating the need for vendor-specific printer drivers.

IPP Request Encoding

IPP operations are carried as HTTP POST requests with Content-Type `application/ipp`. The binary encoding:

[Diagram: see online version]

Key Fields

Field	Size	Description
Version Major	1 byte	IPP major version (2 for IPP 2.0)
Version Minor	1 byte	IPP minor version (0 for IPP 2.0)
Operation ID	2 bytes	Operation code (request) or status code (response)
Request ID	4 bytes	Matches requests to responses (client-chosen)
Attribute Group Tag	1 byte	Delimiter for attribute groups (operation, job, printer, etc.)
Attributes	Variable	Tag-length-value encoded name/value pairs
End-of-Attributes Tag	1 byte	0x03 marks the end of attribute sections
Document Data	Variable	Print data (PDF, PWG Raster, etc.) -- only in Print-Job/Send-Document

Attribute Group Tags

Tag	Value	Description
operation-attributes-tag	0x01	Operation-level attributes (URI, charset, language)
job-attributes-tag	0x02	Job-level attributes (copies, sides, media)
end-of-attributes-tag	0x03	Marks the end of all attribute groups
printer-attributes-tag	0x04	Printer-level attributes (capabilities, state)
unsupported-attributes-tag	0x05	Attributes not supported by the printer

Operations

Operation	Code	Description
Print-Job	0x0002	Submit a complete print job (attributes + document data)
Print-URI	0x0003	Print a document referenced by URI
Validate-Job	0x0004	Check if a job would be accepted (no actual printing)
Create-Job	0x0005	Create a job with attributes only (document sent via Send-Document)
Send-Document	0x0006	Send document data for a previously created job
Send-URI	0x0007	Add a URI-referenced document to a job

Cancel-Job	0x0008	Cancel a pending or processing job
Get-Job-Attributes	0x0009	Query attributes of a specific job
Get-Jobs	0x000A	List jobs on a printer
Get-Printer-Attributes	0x000B	Query printer capabilities and state
Hold-Job	0x000C	Hold a job in the queue
Release-Job	0x000D	Release a held job for printing
Restart-Job	0x000E	Restart a completed/canceled job

Common Attributes

Operation Attributes

Attribute	Type	Description
attributes-charset	charset	Always "utf-8"
attributes-natural-language	naturalLanguage	e.g., "en-us"
printer-uri	uri	Target printer URI (e.g., `ipp://printer.local:631/ipp/print`)
requesting-user-name	name	Username of the requesting user
job-name	name	Human-readable job name
document-format	mimeMediaType	MIME type of document (e.g., "application/pdf")

Job Template Attributes

Attribute	Type	Description
copies	integer	Number of copies
sides	keyword	"one-sided", "two-sided-long-edge", "two-sided-short-edge"
media	keyword	Paper size (e.g., "iso_a4_210x297mm", "na_letter_8.5x11in")
print-quality	enum	3=draft, 4=normal, 5=high
orientation-requested	enum	3=portrait, 4=landscape, 5=reverse-landscape, 6=reverse-portrait
print-color-mode	keyword	"auto", "monochrome", "color"
page-ranges	rangeOfInteger	Pages to print (e.g., 1-5)
finishings	enum	3=none, 4=staple, 5=punch, 6=cover, 7=bind
number-up	integer	Pages per sheet (1, 2, 4, 6, 9, 16)

Job States

State	Value	Description
pending	3	Queued, waiting to be processed
pending-held	4	Held in queue (manual release required)
processing	5	Currently being printed
processing-stopped	6	Processing paused (e.g., out of paper)
canceled	7	Job was canceled
aborted	8	Job aborted due to error
completed	9	Job finished successfully

Print-Job Flow

[Diagram: see online version]

Get-Printer-Attributes Flow

[Diagram: see online version]

IPP Everywhere (Driverless Printing)

IPP Everywhere eliminates printer drivers by combining:

Component	Description
DNS-SD (mDNS)	Discover printers on the local network (`_ipp._tcp` / `_ipps._tcp`)
IPP 2.0	Standard operations with required attributes
Document formats	PDF, PWG Raster, Apple Raster (URF), or JPEG
TLS (IPPS)	Encrypted printing via `ipps://` URI scheme

Discovery flow:

[Diagram: see online version]

IPP vs LPD/LPR vs Raw 9100

Feature	IPP	LPD/LPR (RFC 1179)	Raw 9100
Transport	HTTP/HTTPS (port 631)	TCP port 515	TCP port 9100
Encryption	TLS (IPPS)	None	None

Authentication	HTTP auth, TLS certs	None	None
Job attributes	Rich (copies, duplex, media, quality)	Minimal (class, name)	None
Job management	Cancel, Hold, Release, Get status	Limited	None
Printer discovery	DNS-SD / mDNS	Manual configuration	Manual configuration
Bidirectional	Yes (query status, capabilities)	Minimal	SNMP for status (separate)
Driverless	Yes (IPP Everywhere)	No	No
Protocol age	1999 (current: 2017)	1990	1990s

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 8011	Internet Printing Protocol/1.1: Model and Semantics
RFC 8010	Internet Printing Protocol/1.1: Encoding and Transport
PWG 5100.12	IPP 2.0, 2.1, and 2.2
PWG 5100.14	IPP Everywhere
RFC 6763	DNS-SD for printer discovery

See Also

- HTTP -- transport layer for IPP
- mDNS -- printer discovery via DNS-SD
- SNMP -- printer status monitoring (legacy)

LwM2M -- Lightweight Machine-to-Machine

Standard: OMA LwM2M v1.2 | **Layer:** Application (Layer 7) | **Wireshark filter:** `lwm2m` or `coap`

LwM2M is an IoT device management and service enablement protocol developed by OMA SpecWorks. It provides a standardized way to remotely manage constrained devices -- bootstrapping, registration, firmware updates, configuration, and telemetry collection. LwM2M is built on top of CoAP and uses a well-defined Object/Instance/Resource data model, making it far more structured than raw MQTT or HTTP for device

management. It is used in smart metering, asset tracking, industrial sensors, and cellular IoT (NB-IoT/LTE-M) deployments.

Protocol Stack

[Diagram: see online version]

CoAP Message with LwM2M Payload

LwM2M rides on CoAP. Each LwM2M operation maps to a CoAP method (GET, PUT, POST, DELETE) targeting an Object/Instance/Resource URI:

[Diagram: see online version]

Interfaces

LwM2M defines four interfaces between the LwM2M Client (device) and LwM2M Server:

Interface	Direction	Description
Bootstrap	Server -> Client	Initial provisioning of server URIs, security credentials, and access control
Registration	Client -> Server	Client registers its objects/instances; periodic updates and deregistration
Device Management & Service Enablement	Server -> Client	Read, Write, Execute, Create, Delete, Discover operations on resources
Information Reporting	Client -> Server	Observe/Notify for asynchronous telemetry and event reporting

Object / Instance / Resource Model

LwM2M organizes all device data into a three-level hierarchy addressed as `/{Object ID}/{Instance ID}/{Resource ID}`:

```
/3/0/0    -&gt; Device Manufacturer (Object 3, Instance 0, Resource 0)
/3/0/9    -&gt; Battery Level
/5/0/1    -&gt; Firmware Update URI
/6/0/0    -&gt; Latitude
```

Level	Description	Example
Object	A class of functionality (e.g., Device, Firmware Update)	`/3`
Instance	A specific instance of an object (multi-instance supported)	`/3/0`
Resource	An individual data point, action, or configuration	`/3/0/9` (Battery Level)

Standard Objects

Object ID	Name	Description
0	Security	LwM2M Server URI, security mode, keys/certs
1	Server	Registration lifetime, binding mode, notification storing
2	Access Control	Per-object ACLs for multi-server deployments
3	Device	Manufacturer, model, serial, battery, memory, error codes
4	Connectivity Monitoring	Radio signal, link quality, IP addresses, cell ID
5	Firmware Update	Firmware URI, state machine, update result
6	Location	Latitude, longitude, altitude, velocity, timestamp

OMA maintains a public registry of 300+ defined objects at OMA LwM2M Object Registry.

Operations

Operation	CoAP Method	Direction	Description
Read	GET	Server -> Client	Read resource or object instance value
Discover	GET (Accept: application/link-format)	Server -> Client	List available resources and their attributes
Write	PUT / POST	Server -> Client	Set resource value(s)
Write-Attributes	PUT (on attributes)	Server -> Client	Set observation parameters (pmin, pmax, gt, lt, st)
Execute	POST	Server -> Client	Trigger an action (e.g., reboot, firmware update)
Create	POST	Server -> Client	Create a new object instance
Delete	DELETE	Server -> Client	Delete an object instance
Observe	GET + Observe option	Server -> Client	Subscribe to resource changes
Notify	Response (2.05)	Client -> Server	Asynchronous value update for observed resource

Data Formats

Format	Content-Format ID	Description
Plain Text	0	Single resource value as UTF-8 string
Opaque	42	Single resource value as raw bytes
TLV	11542	Type-Length-Value binary encoding (compact, default in v1.0)
JSON	11543	LwM2M JSON (human-readable)
SenML JSON	110	IETF Sensor Measurement Lists in JSON
SenML CBOR	112	IETF Sensor Measurement Lists in CBOR (compact)
CBOR	11544	LwM2M CBOR encoding (v1.1+)

Bootstrap Flow

[Diagram: see online version]

Registration Flow

[Diagram: see online version]

Firmware Update Flow

[Diagram: see online version]

Firmware Update State Machine

State	Value	Description
Idle	0	No firmware update in progress
Downloading	1	Firmware package is being downloaded
Downloaded	2	Firmware package downloaded, ready to update
Updating	3	Firmware update is in progress

Security

LwM2M mandates transport security via DTLS (over UDP) or TLS (over TCP):

Security Mode	Value	Description
PSK	0	Pre-Shared Key -- lightweight, common in constrained devices

RPK	1	Raw Public Key -- ECDSA key pair without full certificate
Certificate	2	X.509 certificate-based authentication
NoSec	3	No security (testing/development only)

Binding Modes

Mode	Transport	Description
U	UDP	Default CoAP over UDP
T	TCP	CoAP over TCP (persistent connections)
S	SMS	CoAP over SMS (cellular IoT)
N	Non-IP	NIDD -- Non-IP Data Delivery (NB-IoT)
Q	Queue Mode	Server queues requests while device sleeps

LwM2M vs MQTT vs HTTP for IoT

Feature	LwM2M	MQTT	HTTP
Data model	Standardized (Object/Instance/Resource)	Application-defined topics	Application-defined endpoints
Device management	Built-in (bootstrap, firmware, config)	Not included	Not included
Transport	CoAP (UDP/TCP)	TCP	TCP
Header overhead	4 bytes (CoAP)	2 bytes minimum	~200+ bytes
Observe/Push	CoAP Observe (server-initiated)	Pub/Sub (broker-mediated)	Polling or WebSocket
Offline queuing	Queue mode (Q binding)	QoS 1/2 + persistent sessions	Not built-in
Firmware update	Standardized (Object 5)	Custom implementation	Custom implementation
Constrained devices	Designed for (Class 1-2 devices)	Suitable	Heavy for constrained
Standardization	OMA SpecWorks	OASIS	IETF

Encapsulation

[Diagram: see online version]

Standards

Document	Title
OMA LwM2M v1.2	Lightweight Machine-to-Machine Technical Specification (Core + Transport)
OMA LwM2M v1.1.1	LwM2M v1.1.1 (widely deployed)
OMA LwM2M Object Registry	Public registry of LwM2M object definitions
RFC 7252	CoAP -- underlying transport protocol
RFC 8132	PATCH and FETCH methods for CoAP

See Also

- MQTT -- pub/sub IoT messaging (no device management)
- HTTP -- heavyweight alternative for IoT APIs
- mDNS -- local service discovery

PROXY Protocol

Standard: HAProxy PROXY Protocol v1/v2 | **Layer:** Between Transport and Application (Layer 4.5) | **Wireshark filter:** `proxy` (v2)

The PROXY protocol preserves original client connection information (source IP, destination IP, ports) when traffic passes through load balancers and reverse proxies. Without it, the backend server sees only the proxy's IP address. Developed by HAProxy, it has become a de facto standard supported by nginx, AWS ELB/NLB, Cloudflare, Envoy, Traefik, and most modern proxies. The PROXY protocol header is sent exactly once, at the very beginning of a connection, before any application data -- making it protocol-agnostic (works with HTTP, TLS, SMTP, MySQL, or any TCP/UDP stream).

How It Works

[Diagram: see online version]

Multi-Proxy Chain

[Diagram: see online version]

Version 1 (Text)

Version 1 is a human-readable ASCII text header terminated by `\r\n`:

```
PROXY TCP4 198.51.100.10 203.0.113.1 56324 443\r\n
```

[Diagram: see online version]

v1 Format

```
"PROXY" SP PROTO SP SRC_IP SP DST_IP SP SRC_PORT SP DST_PORT "\r\n"
```

Field	Description	Example
PROXY	Fixed signature	`PROXY`
PROTO	`TCP4`, `TCP6`, or `UNKNOWN`	`TCP4`
SRC_IP	Original client IP address	`198.51.100.10`
DST_IP	Original destination IP (proxy's incoming address)	`203.0.113.1`
SRC_PORT	Original client source port	`56324`
DST_PORT	Original destination port	`443`
`\r\n`	Line terminator	CRLF

Maximum line length: 107 bytes (including CRLF). The `UNKNOWN` protocol is used when the proxy cannot determine the original addresses.

Version 2 (Binary)

Version 2 uses a fixed binary header for faster parsing and additional features:

[Diagram: see online version]

Signature

The 12-byte binary signature ensures the header is not confused with application data:

```
\x0D\x0A\x0D\x0A\x00\x0D\x0A\x51\x55\x49\x54\x0A
```

This was specifically chosen to be invalid in HTTP, SMTP, and most text protocols.

Key Fields

Field	Size	Description
Signature	12 bytes	Fixed magic bytes (see above)
Version	4 bits	Always 0x2 for PROXY protocol v2
Command	4 bits	0x0 = LOCAL, 0x1 = PROXY
Address Family	4 bits	0x0 = AF_UNSPEC, 0x1 = AF_INET, 0x2 = AF_INET6, 0x3 = AF_UNIX

Transport Protocol	4 bits	0x0 = UNSPEC, 0x1 = STREAM (TCP), 0x2 = DGRAM (UDP)
Address Length	2 bytes	Length of the address block that follows
Addresses	Variable	Source/Destination addresses and ports

Commands

Command	Value	Description
LOCAL	0x0	Health check or proxy-to-proxy connection -- no client info
PROXY	0x1	Proxied connection -- address block contains original client info

Address Families

Family	Value	Address Block Size	Contents
AF_UNSPEC	0x0	0 bytes	No address information
AF_INET	0x1	12 bytes	src_addr (4) + dst_addr (4) + src_port (2) + dst_port (2)
AF_INET6	0x2	36 bytes	src_addr (16) + dst_addr (16) + src_port (2) + dst_port (2)
AF_UNIX	0x3	216 bytes	src_addr (108) + dst_addr (108)

IPv4 Address Block

[Diagram: see online version]

IPv6 Address Block

[Diagram: see online version]

TLV Extensions (v2)

After the address block, v2 supports optional Type-Length-Value extensions:

[Diagram: see online version]

Type	Value	Name	Description
PP2_TYPE_ALPN	0x01	ALPN	Application-Layer Protocol Negotiation (e.g., "h2", "http/1.1")

PP2_TYPE_AUTHORITY	0x02	Authority	SNI hostname from TLS ClientHello
PP2_TYPE_CRC32C	0x03	CRC32C	CRC32C checksum of the entire PROXY header
PP2_TYPE_NOOP	0x04	No-op	Padding (ignored by receiver)
PP2_TYPE_UNIQUE_ID	0x05	Unique ID	Connection-unique identifier for tracing
PP2_TYPE_SSL	0x20	SSL	TLS version, cipher, client cert info (sub-TLVs)
PP2_TYPE_NETNS	0x30	Network Namespace	Linux network namespace name
PP2_TYPE_AWS	0xEA	AWS VPC	AWS VPC endpoint ID (vendor extension)

SSL Sub-TLVs (Type 0x20)

Sub-Type	Value	Description
PP2_SUBTYPE_SSL_VERSION	0x21	TLS version string (e.g., "TLSv1.3")
PP2_SUBTYPE_SSL_CN	0x22	Client certificate Common Name
PP2_SUBTYPE_SSL_CIPHER	0x23	TLS cipher suite name
PP2_SUBTYPE_SSL_SIG_ALG	0x24	Signature algorithm
PP2_SUBTYPE_SSL_KEY_ALG	0x25	Key algorithm

Supported Software

Software	v1	v2	Sends	Receives
HAProxy	Yes	Yes	Yes	Yes
nginx	Yes	Yes	Yes	Yes
AWS ELB (Classic)	Yes	No	Yes	N/A
AWS NLB	Yes	Yes	Yes	N/A
AWS ALB	No	No	No	N/A
Cloudflare	Yes	Yes	Yes	N/A
Envoy Proxy	Yes	Yes	Yes	Yes
Traefik	Yes	Yes	Yes	Yes
Go net/http	No	No	No	With library
Apache httpd	No	No	No	With mod_remoteip
Postfix	Yes	Yes	No	Yes
MySQL Proxy	No	No	No	Yes (native)

Configuration Examples

HAProxy (sending PROXY protocol)

```
backend web_servers
    server srv1 10.0.0.1:80 send-proxy-v2
```

nginx (receiving PROXY protocol)

```
server {
    listen 80 proxy_protocol;
    set_real_ip_from 10.0.0.0/8;
    real_ip_header proxy_protocol;
}
```

v1 vs v2

Feature	v1 (Text)	v2 (Binary)
Format	ASCII text line	Binary with 12-byte signature
Parsing speed	String splitting	Direct memory read
Max header size	107 bytes	16 + address length + TLVs
Extensions (TLV)	No	Yes (ALPN, SNI, SSL, AWS, etc.)
AF_UNIX	No	Yes
UDP support	No	Yes (DGRAM)
Human-readable	Yes (debug-friendly)	No (requires decoder)
Health check signal	UNKNOWN protocol	LOCAL command

Encapsulation

[Diagram: see online version]

Standards

Document	Title
PROXY Protocol Spec	The PROXY protocol Versions 1 & 2 (HAProxy)

See Also

- HTTP -- most common protocol carried behind PROXY protocol

Raft Consensus Protocol

Standard: In Search of an Understandable Consensus Algorithm (Ongaro & Ousterhout, 2014) | **Layer:** Application (Layer 7) | **Wireshark filter:** N/A (application-level, typically gRPC or custom TCP)

Raft is a consensus algorithm for managing a replicated log across a cluster of servers. It was designed as an understandable alternative to Paxos, decomposing consensus into three sub-problems: leader election, log replication, and safety. Raft guarantees that all nodes in a cluster agree on the same sequence of state machine commands, even in the presence of failures. It is the consensus engine behind etcd, Consul, CockroachDB, TiKV, and many other distributed systems.

Node Roles

Every server in a Raft cluster is in one of three states at any time:

Role	Description
Leader	Handles all client requests, replicates log entries to followers, sends heartbeats
Follower	Passive -- responds to RPCs from leader and candidates
Candidate	Transitional state during leader election; requests votes from peers

[Diagram: see online version]

RequestVote RPC

Used by candidates to gather votes during an election.

[Diagram: see online version]

RequestVote Response

[Diagram: see online version]

AppendEntries RPC

Used by the leader for log replication and as a heartbeat (with empty entries).

[Diagram: see online version]

AppendEntries Response

[Diagram: see online version]

InstallSnapshot RPC

Used by the leader to send a snapshot to followers that are too far behind.

[Diagram: see online version]

Key Fields

Field	RPC	Description
term	All	Monotonically increasing logical clock; identifies the election epoch
candidateId	RequestVote	ID of the candidate requesting the vote
lastLogIndex	RequestVote	Index of the candidate's last log entry
lastLogTerm	RequestVote	Term of the candidate's last log entry
voteGranted	RequestVote Response	True if the follower granted its vote
leaderId	AppendEntries	ID of the current leader (so followers can redirect clients)
prevLogIndex	AppendEntries	Index of log entry immediately preceding new ones
prevLogTerm	AppendEntries	Term of the prevLogIndex entry
entries[]	AppendEntries	Log entries to replicate (empty for heartbeat)
leaderCommit	AppendEntries	Leader's commit index
success	AppendEntries Response	True if follower's log matched prevLogIndex/prevLogTerm

Leader Election

[Diagram: see online version]

Election rules:

- Each server votes for at most one candidate per term (first-come-first-served)
- Candidate must have a log at least as up-to-date as the voter's
- Randomized election timeouts (typically 150-300ms) prevent split votes
- A candidate that receives a majority of votes becomes leader

Log Replication

[Diagram: see online version]

Replication rules:

- Leader appends the command to its local log
- Leader sends AppendEntries to all followers in parallel
- Entry is committed once a majority of servers have written it
- Committed entries are applied to the state machine in log order
- If a follower's log is inconsistent, the leader decrements nextIndex and retries

Leader Failure and Recovery

[Diagram: see online version]

Safety Properties

Property	Guarantee
Election Safety	At most one leader per term
Leader Append-Only	Leader never overwrites or deletes log entries; it only appends
Log Matching	If two logs contain an entry with the same index and term, all preceding entries are identical
Leader Completeness	If an entry is committed in a given term, it will be present in the logs of all leaders for higher terms
State Machine Safety	If a server applies a log entry at a given index, no other server applies a different entry at that index

Membership Changes

Raft supports dynamic cluster membership changes without downtime:

Approach	Description
Single-server changes	Add or remove one server at a time; safe because any majority of the old and new configurations overlap
Joint consensus	Two-phase approach for arbitrary changes; the cluster transitions through a joint configuration

Log Compaction

As the log grows, Raft uses snapshots to compact it:

Concept	Description
Snapshot	Captures the entire state machine state at a given log index
lastIncludedIndex	The last log entry included in the snapshot
lastIncludedTerm	The term of that entry

InstallSnapshot RPC	Leader sends snapshot to slow followers instead of replaying the entire log

Raft vs Paxos

Feature	Raft	Paxos
Design goal	Understandability	Theoretical elegance
Leader	Strong leader required	Optional (Multi-Paxos uses leader)
Election	Randomized timeouts, single round	Complex, multi-round possible
Log replication	Leader-driven, sequential	Proposer-driven, out-of-order possible
Membership changes	Built-in (joint consensus or single-server)	Not specified in basic Paxos
Implementations	etcd, Consul, CockroachDB, TiKV	Chubby, Spanner (modified)
Specification	Single paper, clear rules	Multiple papers, many variants
Understandability	High (designed for clarity)	Low (notoriously difficult)

Encapsulation

[Diagram: see online version]

Raft RPCs are typically carried over gRPC (etcd, TiKV) or custom TCP protocols. The wire format varies by implementation since Raft defines the algorithm, not a wire protocol.

Standards

Document	Title
Raft Paper	In Search of an Understandable Consensus Algorithm (Ongaro & Ousterhout, 2014)
Raft Dissertation	Consensus: Bridging Theory and Practice (Ongaro, 2014)
raft.github.io	Raft Consensus Algorithm -- resources and implementations

See Also

- gRPC -- common transport for Raft implementations (etcd, TiKV)
- HTTP -- client-facing API for Raft-backed services

S3 API -- Amazon Simple Storage Service

Standard: Amazon S3 API Reference (de facto standard) | **Layer:** Application (Layer 7) | **Wireshark filter:** `http` or `http2`

The Amazon S3 REST API is the de facto standard for object storage. It provides a simple HTTP-based interface for storing and retrieving any amount of data as objects in buckets. Beyond AWS, the S3 API has been adopted as an industry standard -- MinIO, Ceph, Wasabi, Backblaze B2, and Cloudflare R2 all implement S3-compatible APIs. Objects are addressed by bucket name and key (path), accessed via standard HTTP methods, and authenticated using AWS Signature Version 4 (HMAC-SHA256). The S3 API supports multipart uploads for large objects, conditional requests, lifecycle policies, versioning, and server-side encryption.

Request Structure

S3 requests are standard HTTP requests with AWS-specific headers:

[Diagram: see online version]

URL Styles

Style	Format	Example
Virtual-hosted (preferred)	`https://bucket.s3.region.amazonaws.com/key`	`https://photos.s3.us-east-1.amazonaws.com/2026/sunset.jpg`
Path-style (legacy)	`https://s3.region.amazonaws.com/bucket/key`	`https://s3.us-east-1.amazonaws.com/photos/2026/sunset.jpg`
S3:// URI	`s3://bucket/key`	`s3://photos/2026/sunset.jpg`

Key Operations

Operation	Method	Path	Description
PutObject	PUT	/{key}	Upload an object
GetObject	GET	/{key}	Download an object
HeadObject	HEAD	/{key}	Get object metadata without body
DeleteObject	DELETE	/{key}	Delete an object
CopyObject	PUT	/{key} (x-amz-copy-source header)	Copy an object server-side
ListObjectsV2	GET	/?list-type=2	List objects in a bucket
CreateBucket	PUT	/ (on bucket host)	Create a new bucket
DeleteBucket	DELETE	/ (on bucket host)	Delete an empty bucket
HeadBucket	HEAD	/ (on bucket host)	Check bucket existence and access
CreateMultipartUpload	POST	/{key}?uploads	Initiate a multipart upload

UploadPart	PUT	/{key}?partNumber=N&uploadId=X	Upload one part
CompleteMultipartUpload	POST	/{key}?uploadId=X	Finalize the multipart upload
AbortMultipartUpload	DELETE	/{key}?uploadId=X	Cancel and clean up parts
GetBucketVersioning	GET	/?versioning	Check versioning status
PutBucketVersioning	PUT	/?versioning	Enable/suspend versioning
SelectObjectContent	POST	/{key}?select&select-type=2	Run SQL query on object (S3 Select)

Authentication: AWS Signature V4

All S3 requests are authenticated using HMAC-SHA256 signing:

Step	Description
1. Canonical Request	HTTP method + URI + query string + headers + payload hash
2. String to Sign	Algorithm + timestamp + credential scope + hash of canonical request
3. Signing Key	HMAC chain: secret key -> date -> region -> service -> "aws4_request"
4. Signature	HMAC-SHA256(signing key, string to sign)

Authorization Header

```
Authorization: AWS4-HMAC-SHA256
  Credential=AKID/20260321/us-east-1/s3/aws4_request,
  SignedHeaders=host;x-amz-content-sha256;x-amz-date,
  Signature=abcdef1234567890...
```

Presigned URLs

For sharing temporary access without credentials:

```
https://bucket.s3.us-east-1.amazonaws.com/key
  ?X-Amz-Algorithm=AWS4-HMAC-SHA256
  &amp;X-Amz-Credential=AKID/20260321/us-east-1/s3/aws4_request
  &amp;X-Amz-Date=20260321T000000Z
  &amp;X-Amz-Expires=3600
  &amp;X-Amz-Signature=abcdef...
```

PutObject Flow

[Diagram: see online version]

Multipart Upload Flow

For objects larger than 5 GB (up to 5 TB total, 10,000 parts max):

[Diagram: see online version]

Multipart Limits

Constraint	Value
Minimum part size	5 MB (except last part)
Maximum part size	5 GB
Maximum parts	10,000
Maximum object size	5 TB
Recommended threshold	Use multipart for objects > 100 MB

Conditional Requests

Header	Description
If-Match	Only proceed if ETag matches (prevents overwrite conflicts)
If-None-Match	Only proceed if ETag does not match (skip if unchanged)
If-Modified-Since	Return object only if modified after this date
If-Unmodified-Since	Return object only if not modified after this date
Range	Retrieve a byte range (e.g., `bytes=0-1023`)

Storage Classes

Class	Description	Retrieval
STANDARD	Frequently accessed data	Instant
STANDARD_IA	Infrequent access, lower storage cost	Instant
ONEZONE_IA	Single-AZ infrequent access	Instant
INTELLIGENT_TIERING	Auto-moves between tiers based on access patterns	Instant
GLACIER_IR	Archive with instant retrieval	Instant
GLACIER_FLEXIBLE	Low-cost archive	Minutes to hours
DEEP_ARCHIVE	Lowest-cost long-term archive	12-48 hours

Common Response Headers

Header	Description
ETag	MD5 hash of the object (or multipart composite hash)
x-amz-version-id	Object version (when versioning is enabled)
x-amz-request-id	Unique request identifier for debugging
x-amz-storage-class	Storage class of the object
x-amz-server-side-encryption	Encryption algorithm (AES256, aws:kms)
Last-Modified	Timestamp of last object modification
Content-Length	Size of the object in bytes

S3 Select

Query structured data (CSV, JSON, Parquet) directly in S3 without downloading the entire object:

```
POST /{key}?select&amp;select-type=2

SELECT s.name, s.age FROM S3Object s WHERE s.age &gt; 30
```

Format	Support
CSV	Full SQL support
JSON	Full SQL support
Parquet	Columnar pushdown

S3-Compatible Services

Service	Provider	Notes
MinIO	Open source	Self-hosted, full S3 API compatibility
Ceph (RGW)	Open source	Distributed storage with S3 gateway
Wasabi	Wasabi	S3-compatible cloud, no egress fees
Backblaze B2	Backblaze	S3-compatible API layer
Cloudflare R2	Cloudflare	S3-compatible, no egress fees
DigitalOcean Spaces	DigitalOcean	S3-compatible object storage
Google Cloud Storage	Google	S3-compatible interoperability API

Encapsulation

[Diagram: see online version]

Standards

Document	Title
S3 API Reference	Amazon S3 REST API specification
Signature V4	AWS Signature Version 4 signing process
S3 Developer Guide	Amazon S3 User Guide

See Also

- HTTP -- S3 API transport
- TLS -- encrypts all S3 traffic
- REST -- S3 follows RESTful design principles

Server-Sent Events -- SSE

Standard: WHATWG HTML Living Standard -- Server-Sent Events | **Layer:** Application (Layer 7) | **Wireshark filter:** `http`

Server-Sent Events (SSE) is a lightweight, unidirectional streaming protocol that allows a server to push events to a client over a single long-lived HTTP connection. The client opens a standard HTTP request, and the server responds with `Content-Type: text/event-stream`, sending a stream of text-formatted events. SSE is built on top of standard HTTP (no protocol upgrade needed) and includes automatic reconnection and event ID tracking. Browsers expose SSE through the `EventSource` API. SSE is commonly used for live feeds, notifications, real-time dashboards, and AI/LLM token streaming.

Event Stream Format

The response body is a stream of UTF-8 text blocks, where each event is a set of `field:value` lines terminated by a blank line (`\n\n`):

[Diagram: see online version]

Event Fields

Each line in the stream has the format `field:value\n`. A blank line (`\n\n`) dispatches the accumulated event:

Field	Description
`data`	Event payload. Multiple `data:` lines are joined with newlines in the dispatched event
`event`	Event type name. If set, dispatches a named event; otherwise dispatches a generic `message` event

`id`	Event ID. Sets the last event ID; sent back on reconnect via `Last-Event-ID` header
`retry`	Reconnection time in milliseconds. Overrides the client's default reconnection delay

Lines starting with `:` (colon) are comments, often used as keep-alive heartbeats.

Example Stream

```
: heartbeat

event: user-joined
data: {"user": "alice", "room": "general"}
id: 1001

data: Simple message without event type
id: 1002

event: score-update
data: {"home": 3, "away": 2}
id: 1003

retry: 5000
```

SSE Connection Flow

[Diagram: see online version]

Automatic Reconnection

If the connection drops, the `EventSource` API automatically reconnects. The client sends the last received event ID in the `Last-Event-ID` header so the server can resume the stream:

[Diagram: see online version]

EventSource Browser API

Property / Method	Description
`new EventSource(url, options)`	Open an SSE connection; `options.withCredentials` enables cookies
`.readyState`	`0` = CONNECTING, `1` = OPEN, `2` = CLOSED
`.url`	The URL being connected to
`.onopen`	Fires when connection is established
`.onmessage`	Fires on unnamed events (no `event:` field)
`.onerror`	Fires on connection error or server close
`.addEventListener(type, fn)`	Listen for named event types

`.close()`	Close the connection (no automatic reconnect)

HTTP Headers

Header	Direction	Description
`Accept: text/event-stream`	Request	Client indicates SSE support
`Content-Type: text/event-stream`	Response	Server indicates event stream
`Cache-Control: no-cache`	Response	Prevents caching of the stream
`Connection: keep-alive`	Response	Keeps the HTTP connection open
`Last-Event-ID`	Request	Sent on reconnection with the last received `id` value

SSE vs WebSocket vs Long Polling

Feature	SSE	WebSocket	Long Polling
Direction	Server-to-client only	Bidirectional	Server-to-client (simulated)
Protocol	Standard HTTP	Upgraded protocol (ws://)	Standard HTTP
Connection	Single long-lived HTTP response	Persistent upgraded connection	Repeated HTTP requests
Data format	UTF-8 text (event stream)	Text or binary frames	Any HTTP response body
Auto-reconnect	Built-in (EventSource API)	Manual (application must implement)	Manual (application logic)
Event IDs	Built-in (`id:` field)	Not built-in	Not built-in
Browser API	`EventSource`	`WebSocket`	`fetch` / `XMLHttpRequest`
HTTP/2 multiplexing	Yes (shares connection)	No (separate TCP connection)	Yes (each poll is HTTP)
Proxy/firewall compat	Excellent (standard HTTP)	Can be blocked (upgrade required)	Excellent (standard HTTP)
Max connections	~6 per domain (HTTP/1.1); no limit on HTTP/2	No browser limit	~6 per domain (HTTP/1.1)
Binary data	No (text only; base64 encode if needed)	Yes (native binary frames)	Yes
Overhead	Very low (plain text, no framing)	Low (2-14 byte frame header)	High (full HTTP headers per poll)

Encapsulation

[Diagram: see online version]

SSE works over standard HTTP. With HTTP/2, multiple SSE streams can be multiplexed over a single TCP connection, eliminating the browser connection limit.

Standards

Document	Title
HTML Living Standard -- SSE	Server-Sent Events (WHATWG specification)
W3C EventSource	Server-Sent Events (W3C, superseded by WHATWG)
RFC 9110 Section 15.5.14	HTTP Semantics -- 204 No Content (used to stop reconnection)

See Also

- HTTP -- underlying transport for SSE
- WebSocket -- bidirectional alternative
- WebTransport -- modern bidirectional streaming over HTTP/3
- gRPC -- server streaming RPCs serve a similar role

WebSocket

Standard: RFC 6455 | **Layer:** Application (Layer 7) | **Wireshark filter:** `websocket`

WebSocket provides full-duplex, bidirectional communication over a single TCP connection between a client (typically a browser) and server. Unlike HTTP's request-response model, WebSocket allows either side to send messages at any time after the initial handshake, with minimal framing overhead (as low as 2 bytes per message). It is widely used for real-time applications: chat, live dashboards, collaborative editing, gaming, financial tickers, and as a transport for other protocols (MQTT, SIP, AMQP).

Handshake (HTTP Upgrade)

WebSocket connections start as an HTTP/1.1 upgrade request:

```
GET /chat HTTP/1.1
Host: server.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

After the 101 response, the TCP connection switches from HTTP to WebSocket framing.

Frame

[Diagram: see online version]

Key Fields

Field	Size	Description
FIN	1 bit	1 = final fragment of a message
RSV1-3	1 bit each	Reserved for extensions (0 unless negotiated)
Opcode	4 bits	Frame type
MASK	1 bit	1 = payload is masked (required for client->server)
Payload Length	7 bits	0-125 = actual length; 126 = 16-bit extended; 127 = 64-bit extended
Extended Payload Length	0, 16, or 64 bits	Actual length when > 125 bytes
Masking Key	32 bits	XOR key for payload (client frames only)
Payload Data	Variable	Application data (XOR-masked if MASK=1)

Opcodes

Opcode	Name	Description
0x0	Continuation	Continues a fragmented message
0x1	Text	UTF-8 text message
0x2	Binary	Binary message
0x8	Close	Connection close (with optional status code + reason)
0x9	Ping	Keepalive request
0xA	Pong	Keepalive response

Close Status Codes

Code	Meaning
1000	Normal closure
1001	Going away (server shutdown, page navigated away)
1002	Protocol error
1003	Unsupported data type
1006	Abnormal closure (no close frame, connection lost)
1007	Invalid payload data
1008	Policy violation
1009	Message too big
1011	Server error

Message Fragmentation

Large messages can be split across multiple frames:

[Diagram: see online version]

Masking

All client-to-server frames must be masked with a 32-bit key. Each byte of the payload is XORed with the corresponding byte of the masking key (cycling through 4 bytes). This prevents cache poisoning attacks on intermediary proxies -- it is not encryption.

Extensions

Extension	Description
permessage-deflate	Compresses message payloads with DEFLATE (RFC 7692)

Extensions are negotiated during the handshake via `Sec-WebSocket-Extensions` headers.

Subprotocols

Applications can negotiate a subprotocol during the handshake:

Subprotocol	Description
mqtt	MQTT over WebSocket
sip	SIP over WebSocket
graphql-transport-ws	GraphQL subscriptions
wamp.2.json	Web Application Messaging Protocol

WebSocket vs Alternatives

Feature	WebSocket	HTTP Long Polling	Server-Sent Events
Direction	Bidirectional	Bidirectional (simulated)	Server -> client only
Overhead per message	2-14 bytes	Full HTTP headers	~few bytes
Connection	Persistent	Reconnects each poll	Persistent
Binary data	Yes	Yes	No (text only)
Browser support	Universal	Universal	Universal (no IE)

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 6455	The WebSocket Protocol
RFC 7692	Compression Extensions for WebSocket (permessage-deflate)
RFC 8441	Bootstrapping WebSockets with HTTP/2

See Also

- HTTP -- WebSocket begins as an HTTP upgrade
- TCP
- TLS -- encrypts wss:// connections
- MQTT -- commonly carried over WebSocket
- SIP -- browser VoIP via WebSocket

WebTransport

Standard: W3C WebTransport API / RFC 9297 | **Layer:** Application (Layer 7) | **Wireshark filter:** `quic` (WebTransport runs over QUIC/HTTP/3)

WebTransport is a browser API and protocol framework for low-latency, bidirectional client-server communication built on HTTP/3 and QUIC. Unlike WebSocket, which is limited to a single ordered byte stream over TCP, WebTransport provides multiple independent streams and unreliable datagrams over QUIC -- eliminating head-of-line blocking and enabling use cases like real-time gaming, live media, and cloud rendering where occasional packet loss is acceptable. It reuses the HTTP/3 connection establishment (including TLS 1.3 and certificate-based

authentication), avoiding the need for a separate protocol upgrade path.

Protocol Stack

[Diagram: see online version]

Transport Types

WebTransport exposes three transport primitives over a single HTTP/3 connection:

Transport	Reliability	Ordering	Multiplexing	Use Case
Bidirectional Stream	Reliable	Ordered (per-stream)	Yes	Request-response, RPC, file transfer
Unidirectional Stream	Reliable	Ordered (per-stream)	Yes	Server push, log streaming, media segments
Datagram	Unreliable	Unordered	N/A	Game state, live video frames, sensor data

Connection Establishment

WebTransport sessions are established by sending an extended HTTP/3 CONNECT request with the `:protocol` pseudo-header set to `webtransport`:

[Diagram: see online version]

HTTP/3 CONNECT Request

The extended CONNECT method (RFC 8441 adapted for HTTP/3) carries these pseudo-headers:

Pseudo-Header	Value	Description
`:method`	`CONNECT`	HTTP method
`:protocol`	`webtransport`	Extended CONNECT protocol
`:scheme`	`https`	Always HTTPS (TLS required)
`:authority`	`server.example.com`	Server hostname and port
`:path`	`/session`	Application-defined session URL

The server responds with HTTP status 200 to accept the session or 4xx/5xx to reject it.

QUIC DATAGRAM Frame

WebTransport datagrams use the QUIC DATAGRAM extension (RFC 9221):

[Diagram: see online version]

Field	Size	Description
Type	Variable	QUIC frame type 0x30 (no length) or 0x31 (with length)
Quarter Stream ID	Variable-length integer	Identifies the WebTransport session (CONNECT stream ID / 4)
Datagram Payload	Variable	Application data (max size limited by QUIC packet)

Datagrams are not retransmitted -- if the QUIC packet is lost, the datagram is gone. Maximum datagram size is constrained by the QUIC max_datagram_frame_size transport parameter (typically ~1200 bytes after overhead).

WebTransport Stream Frame

Streams opened within a WebTransport session carry a session ID prefix:

[Diagram: see online version]

Field	Description
QUIC Stream ID	QUIC-level stream identifier
Session ID	Variable-length integer identifying the WebTransport session
Stream Data	Application payload (reliable, ordered within this stream)

Browser API Overview (JavaScript)

```
// Establish a WebTransport session
const transport = new WebTransport("https://server.example.com/session");
await transport.ready;

// Bidirectional stream
const bidi = await transport.createBidirectionalStream();
const writer = bidi.writable.getWriter();
await writer.write(new Uint8Array([1, 2, 3]));

// Receive bidirectional streams
const reader = transport.incomingBidirectionalStreams.getReader();
const { value: stream } = await reader.read();

// Unreliable datagrams
const dgWriter = transport.datagrams.writable.getWriter();
await dgWriter.write(new Uint8Array([0x01, 0x02]));

const dgReader = transport.datagrams.readable.getReader();
const { value: datagram } = await dgReader.read();

// Close the session
transport.close({ closeCode: 0, reason: "done" });
```

WebTransport vs WebSocket

Feature	WebTransport	WebSocket
Transport	QUIC (UDP)	TCP
Multiplexing	Multiple independent streams	Single ordered stream
Head-of-line blocking	No (per-stream only)	Yes (one lost packet blocks all)
Unreliable delivery	Yes (datagrams)	No (TCP guarantees delivery)
Connection setup	QUIC 1-RTT (0-RTT on reconnect)	TCP + TLS + HTTP upgrade
Encryption	Always TLS 1.3 (via QUIC)	Optional (wss://)
Binary/text	Binary streams + datagrams	Text and binary frames
Server certificate	Required (CA-signed)	Required for wss://, optional for ws://
Browser support	Chrome, Edge, Firefox (partial)	Universal
Protocol overhead	QUIC framing	2-14 bytes per frame
Congestion control	QUIC built-in	TCP built-in

When to Use WebTransport

Scenario	Recommended
Chat, notifications, live feeds	WebSocket (universal support, simpler)
Real-time multiplayer gaming	WebTransport (datagrams, no HOL blocking)
Live video/audio streaming	WebTransport (unreliable delivery, low latency)
Cloud rendering / remote desktop	WebTransport (datagrams for frames, streams for control)
File upload with progress	WebTransport (multiple streams, no HOL blocking)
Legacy browser support required	WebSocket

Encapsulation

[Diagram: see online version]

Standards

Document	Title
RFC 9297	HTTP Datagrams and the Capsule Protocol
RFC 9298	Proxying UDP in HTTP (CONNECT-UDP)
draft-ietf-webtrans-http3	WebTransport over HTTP/3
W3C WebTransport API	Browser JavaScript API specification

RFC 9221	An Unreliable Datagram Extension to QUIC
RFC 9114	HTTP/3
RFC 9000	QUIC: A UDP-Based Multiplexed and Secure Transport

See Also

- WebSocket -- predecessor for bidirectional browser-server communication
- HTTP -- WebTransport sessions are established via HTTP/3 CONNECT
- gRPC -- RPC framework that can also run over HTTP/3

Wireless & Radio

Wireless LAN, PAN, and radio communication protocols.

19 protocols in this section

6LoWPAN -- IPv6 over Low-Power Wireless Personal Area Networks

Standard: RFC 4944 / RFC 6282 | **Layer:** Network Adaptation (between L2 and L3) | **Wireshark filter:** `6lowpan`

6LoWPAN is an adaptation layer that enables IPv6 packets to be carried over IEEE 802.15.4 networks, which have a maximum frame size of only 127 bytes. Since an IPv6 header alone is 40 bytes (and a full IPv6 + UDP header is 48 bytes), 6LoWPAN provides header compression that can squeeze a typical IPv6/UDP header from 48 bytes down to just 6-7 bytes. It also handles fragmentation for packets that exceed the 802.15.4 MTU. 6LoWPAN is the foundation of Thread and is used in Zigbee IP and various IoT deployments.

The Problem

Layer	Size	Problem
802.15.4 MAX frame	127 bytes	
802.15.4 MAC header	9-23 bytes	
Available for payload	**~104 bytes**	
IPv6 header	40 bytes	Half the space gone!
UDP header	8 bytes	
Available for data	**~56 bytes**	Very little room for application data

6LoWPAN solves this with header compression and fragmentation.

Dispatch Byte

Every 6LoWPAN frame begins with a dispatch byte (or two) that identifies the frame type:

[Diagram: see online version]

Dispatch Values

Pattern	Name	Description
00 xxxxxx	NALP	Not a LoWPAN frame (reserved)
01 000001	IPv6	Uncompressed IPv6 (full 40-byte header follows)
01 1xxxxx	LOWPAN_IPHC	Compressed IPv6 header (RFC 6282)
10 0xxxxx	MESH	Mesh addressing header (for multi-hop)
11 000xxx	FRAG1	First fragment
11 100xxx	FRAGN	Subsequent fragment

11 110000	LOWPAN_BC0	Broadcast header

IPHC Header Compression (RFC 6282)

IPHC compresses IPv6 and UDP headers by eliding fields that can be inferred from the 802.15.4 layer or from defaults:

IPHC Encoding (2 bytes)

[Diagram: see online version]

Compression Strategies

IPv6 Field	Uncompressed	Compressed	How
Version	4 bits	0 bits	Always 6 -- elided
Traffic Class + Flow Label	4 bytes	0-4 bytes	TF field controls
Payload Length	2 bytes	0 bytes	Computed from 802.15.4 frame length
Next Header	1 byte	0-1 bytes	NH=1: NHC follows; NH=0: inline
Hop Limit	1 byte	0-1 bytes	HLIM: 00=inline, 01=1, 10=64, 11=255
Source Address	16 bytes	0-16 bytes	SAM: derive from 802.15.4 src addr
Destination Address	16 bytes	0-16 bytes	DAM: derive from 802.15.4 dst addr

Address Compression (SAM/DAM)

SAM/DAM	Mode	Compressed Size	How
00	Full	16 bytes	Full 128-bit address inline
01	64-bit	8 bytes	Prefix from context, IID inline
10	16-bit	2 bytes	Prefix from context, IID from 16-bit short addr
11	Elided	0 bytes	Entire address derived from 802.15.4 src/dst

Example: Maximum Compression

A typical link-local UDP packet:

Field	IPv6+UDP	6LoWPAN IPHC	Savings

IPv6 header	40 bytes	2 bytes (IPHC)	38 bytes
UDP header	8 bytes	4 bytes (NHC + compressed ports)	4 bytes
Total header	**48 bytes**	**6 bytes**	**42 bytes saved (87%)**

This leaves ~98 bytes for application data in a single 802.15.4 frame vs ~56 bytes uncompressed.

NHC -- Next Header Compression (UDP)

[Diagram: see online version]

P	Source Port	Dest Port	Size
00	16 bits inline	16 bits inline	4 bytes
01	16 bits inline	8 bits (0xF0xx)	3 bytes
10	8 bits (0xF0xx)	16 bits inline	3 bytes
11	4 bits (0xF0Bx)	4 bits (0xF0Bx)	1 byte

Ports in the range 0xF0B0-0xF0BF can be compressed to 4 bits each.

Fragmentation

When an IPv6 packet exceeds the 802.15.4 MTU (~104 bytes after MAC), 6LoWPAN fragments it:

First Fragment (FRAG1)

[Diagram: see online version]

Subsequent Fragments (FRAGN)

[Diagram: see online version]

Field	Size	Description
Datagram Size	11 bits	Total unfragmented datagram size
Datagram Tag	16 bits	Unique ID matching fragments of the same datagram
Datagram Offset	8 bits	Offset in 8-byte multiples (FRAGN only)

Fragmentation Example

[Diagram: see online version]

Mesh Addressing Header

For multi-hop forwarding at the 6LoWPAN layer (before IPv6 routing):

[Diagram: see online version]

6LoWPAN Neighbor Discovery (RFC 6775)

Standard IPv6 NDP is too chatty for constrained networks. 6LoWPAN-ND optimizes it:

Feature	IPv6 NDP	6LoWPAN-ND
Address registration	DAD (multicast)	Registration with router (unicast)
Router discovery	Periodic RA broadcast	RA with 6LoWPAN context options
Neighbor unreachability	NUD probes	Registration lifetime
Multicast	Required	Minimized (unicast preferred)

Context-Based Compression

Border Routers distribute compression contexts -- shared IPv6 prefixes that all nodes know, allowing addresses to be compressed to 0-8 bytes:

```
Context 0: 2001:db8:1::/48  (global prefix)
Context 1: fd00::/64         (mesh-local prefix)
```

A device with address `2001:db8:1::1234` only needs to send the last 2 bytes (`0x1234`) -- the prefix is inferred from context 0.

6LoWPAN in Protocol Stacks

[Diagram: see online version]

Standards

Document	Title
RFC 4944	Transmission of IPv6 Packets over IEEE 802.15.4 Networks
RFC 6282	Compression Format for IPv6 over 802.15.4 (IPHC)
RFC 6775	6LoWPAN Neighbor Discovery
RFC 8025	IPv6 over 802.15.4 with IPHC (updates)
RFC 8138	6LoWPAN Routing Header for RPL

See Also

- Thread -- IPv6 mesh built on 6LoWPAN

- Zigbee -- uses same 802.15.4 radio (non-IP version)
- IPv6 -- the protocol 6LoWPAN adapts
- CoAP -- constrained REST protocol designed for 6LoWPAN networks
- ICMPv6 -- NDP optimized by 6LoWPAN-ND
- LoRaWAN -- alternative LPWAN (does not use IPv6/6LoWPAN)

IEEE 802.11 -- Wi-Fi

Standard: IEEE 802.11-2020 | **Layer:** Data Link / Physical (Layers 1-2) | **Wireshark filter:** `wlan`

IEEE 802.11 defines wireless local area networking (Wi-Fi) -- the most widely deployed wireless technology in the world. It specifies the MAC layer and multiple physical layers for wireless communication in the 2.4 GHz, 5 GHz, and 6 GHz bands. Wi-Fi supports infrastructure mode (clients connect to access points), ad-hoc mode (peer-to-peer), and mesh mode (802.11s). The Wi-Fi Alliance certifies interoperability and assigns marketing names (Wi-Fi 4, 5, 6, 6E, 7).

Frame Format

[Diagram: see online version]

Frame Control Field

[Diagram: see online version]

Field	Size	Description
Protocol Version	2 bits	Always 0
Type	2 bits	Frame category (Management, Control, Data)
Subtype	4 bits	Specific frame type within the category
To DS / From DS	1 bit each	Direction relative to the Distribution System
More Fragments	1 bit	More fragments follow
Retry	1 bit	This is a retransmission
Power Management	1 bit	Station entering power save
More Data	1 bit	AP has buffered frames for this station
Protected Frame	1 bit	Frame body is encrypted
Order	1 bit	Strict ordering requested

Frame Types

Management Frames (Type = 0)

Subtype	Name	Description
0	Association Request	Client requests to join the BSS
1	Association Response	AP accepts or rejects association
4	Probe Request	Client scans for networks
5	Probe Response	AP responds with network info
8	Beacon	AP periodically broadcasts (typically every 102.4 ms)
10	Disassociation	End association
11	Authentication	Pre-association authentication
12	Deauthentication	Force disconnect
13	Action	Various management actions (spectrum, BA, etc.)

Control Frames (Type = 1)

Subtype	Name	Description
8	Block Ack Request	Request block acknowledgment
9	Block Ack	Acknowledge multiple frames at once
11	RTS	Request to Send (collision avoidance)
12	CTS	Clear to Send
13	ACK	Acknowledge a single frame

Data Frames (Type = 2)

Subtype	Name	Description
0	Data	Standard data frame
4	Null	No data (power management signaling)
8	QoS Data	Data with QoS (802.11e)
12	QoS Null	QoS null (power management)

Address Fields

The meaning of the four address fields depends on To DS / From DS:

ToDS	FromDS	Addr 1 (RA)	Addr 2 (TA)	Addr 3	Addr 4
0	0	Destination	Source	BSSID	--

0	1	Destination	BSSID	Source	--
1	0	BSSID	Source	Destination	--
1	1	Receiver	Transmitter	Destination	Source

ToDS=1, FromDS=1 is used in WDS (Wireless Distribution System) and mesh.

Connection Flow

[Diagram: see online version]

Security

Standard	Protocol	Encryption	Key Management	Status
WEP	--	RC4 (broken)	Static keys	Deprecated
WPA	TKIP	RC4 (improved)	802.1X or PSK	Deprecated
WPA2	CCMP	AES-128-CCM	802.1X or PSK	Current
WPA3	CCMP/GCMP	AES-128/256	SAE or 802.1X	Current (recommended)

WPA3 Improvements

Feature	WPA2	WPA3
Key exchange	4-way handshake (PSK)	SAE (Dragonfly) -- resistant to offline dictionary attacks
Forward secrecy	No	Yes
Open networks	Unencrypted	OWE (Opportunistic Wireless Encryption)
Enterprise	Optional 192-bit	Mandatory 192-bit mode (CNSA suite)

PHY Standards (Wi-Fi Generations)

Generation	Amendment	Band	Max Rate	Channel Width	MIMO	Year
--	802.11a	5 GHz	54 Mbps	20 MHz	--	1999
--	802.11b	2.4 GHz	11 Mbps	22 MHz	--	1999
--	802.11g	2.4 GHz	54 Mbps	20 MHz	--	2003
Wi-Fi 4	802.11n	2.4/5 GHz	600 Mbps	40 MHz	4x4	2009
Wi-Fi 5	802.11ac	5 GHz	6.9 Gbps	160 MHz	8x8 MU-MIMO	2013
Wi-Fi 6	802.11ax	2.4/5 GHz	9.6 Gbps	160 MHz	8x8, OFDMA	2020

Wi-Fi 6E	802.11ax	6 GHz	9.6 Gbps	160 MHz	8x8, OFDMA	2021
Wi-Fi 7	802.11be	2.4/5/6 GHz	46 Gbps	320 MHz	16x16, MLO	2024

Channel Access (CSMA/CA)

Wi-Fi uses Carrier Sense Multiple Access with Collision Avoidance:

1. **Listen** -- check if the medium is idle for DIFS (DCF Interframe Space)
2. **Random backoff** -- wait a random number of slot times
3. **Transmit** -- send the frame
4. **ACK** -- receiver sends ACK after SIFS (Short Interframe Space)
5. **Retry** -- if no ACK, double the backoff window and retry

RTS/CTS (optional, for hidden node problem)

[Diagram: see online version]

Encapsulation

[Diagram: see online version]

Standards

Document	Title
IEEE 802.11-2020	Wireless LAN MAC and PHY Specifications
IEEE 802.11ax-2021	Wi-Fi 6 (High Efficiency WLAN)
IEEE 802.11be	Wi-Fi 7 (Extremely High Throughput)
Wi-Fi Alliance	Certification and marketing names

See Also

- Ethernet -- wired LAN counterpart
- 802.11s -- Wi-Fi mesh networking
- RADIUS -- authentication for WPA Enterprise
- ARP -- address resolution on Wi-Fi networks

IEEE 802.11s -- Wi-Fi Mesh Networking

Standard: IEEE 802.11-2020 (Section 14) | **Layer:** Data Link / Physical (Layer 2) | **Wireshark filter:** `wlan.mesh`

IEEE 802.11s defines a mesh networking extension for Wi-Fi, allowing wireless access points and stations to form a self-configuring, self-healing multi-hop mesh network at Layer 2. Unlike traditional Wi-Fi infrastructure (where all traffic routes through an AP), 802.11s mesh nodes forward frames for each other, extending coverage without wired backhaul to every node. Originally published as an amendment in 2011, it is now incorporated into the main IEEE 802.11-2020 standard.

Architecture

[Diagram: see online version]

Node Types

Type	Abbreviation	Description
Mesh Point	MP	Basic mesh node -- forwards mesh traffic
Mesh AP	MAP	Mesh point + access point -- serves regular Wi-Fi clients
Mesh Portal	MPP	Mesh point + bridge to external network (wired LAN, Internet)
Mesh STA	--	Any mesh-capable station

Mesh Frame Format

802.11s extends the standard Wi-Fi frame with a Mesh Control field and additional address fields for multi-hop forwarding:

[Diagram: see online version]

Mesh Control Field

[Diagram: see online version]

Field	Size	Description
Mesh Flags	8 bits	Address extension mode and flags
Mesh TTL	8 bits	Hop limit (decremented at each mesh point)
Mesh Sequence Number	32 bits	De-duplication and ordering
Address Extension	0-12 bytes	Additional addresses for proxied or 6-address frames

Address Usage

In standard Wi-Fi, four addresses suffice. Mesh forwarding needs up to six:

Address	Standard Wi-Fi	Mesh (multi-hop)
Address 1	Receiver	Next-hop receiver (RA)
Address 2	Transmitter	This node (TA)
Address 3	Destination or BSSID	Mesh destination (DA)
Address 4	Source (in WDS)	Mesh source (SA)
Address 5	--	Proxy destination (if proxied)
Address 6	--	Proxy source (if proxied)

Addresses 5 and 6 are carried in the Mesh Address Extension when a MAP proxies frames for associated non-mesh clients.

Routing (HWMP)

802.11s specifies **HWMP (Hybrid Wireless Mesh Protocol)** as the default path selection protocol. It combines:

1. **On-demand routing** (reactive, AODV-like) -- routes discovered when needed
2. **Proactive tree routing** -- optional root-based tree for traffic to/from a mesh portal

Path Discovery (On-Demand)

[Diagram: see online version]

HWMP Messages

Message	Name	Description
PREQ	Path Request	Broadcast to discover a route to a destination
PREP	Path Reply	Unicast reply with route back to the source
PERR	Path Error	Notification that a route has broken
RANN	Root Announcement	Periodic broadcast from the root mesh portal

Path Metric

802.11s uses the **Airtime Link Metric** by default:

```
cost = (O + Bt/r) x (1 / (1 - ef))
```

Variable	Description
O	Channel access overhead (constant)
Bt	Test frame length (8192 bits)

r	Data rate in Mbps
ef	Frame error rate

Lower metric = better path. This accounts for both link speed and reliability.

Peer Link Management

Mesh points establish peer links with neighbors through a three-way handshake:

[Diagram: see online version]

Mesh Configuration

Parameter	Description
Mesh ID	Network name (like SSID, up to 32 bytes)
Mesh Profile	Identifies the path selection and metric protocol
Mesh Formation Info	Number of peers, connected to portal, etc.
Mesh Capability	Forwarding, MDA, power save support

Mesh Discovery

Mesh points discover each other through **Mesh Beacon** frames that include the Mesh ID and configuration. Nodes with matching Mesh ID and compatible configuration can peer.

Security

802.11s uses **SAE (Simultaneous Authentication of Equals)** -- the same protocol used in WPA3 -- for peer authentication. This provides:

- Password-based authentication without transmitting the password
- Forward secrecy per session
- Resistance to offline dictionary attacks

After SAE, traffic is encrypted with **CCMP (AES-128-CCM)** or **GCMP** per peer link.

802.11s vs Other Mesh

Feature	802.11s	Zigbee Mesh	Z-Wave
Frequency	2.4/5/6 GHz	2.4 GHz	Sub-GHz
Data rate	Mbps-Gbps	250 kbps	100 kbps
Range per hop	50-100 m	10-30 m	30 m
Layer	L2 (MAC)	Full stack	Full stack

IP native	Yes (standard Wi-Fi)	No	No
Power	High (mains)	Low (battery)	Low (battery)
Use case	Coverage extension, backhaul	IoT sensors	Home automation

Standards

Document	Title
IEEE 802.11-2020	Wireless LAN MAC and PHY (includes mesh, Section 14)
IEEE 802.11s-2011	Mesh Networking amendment (now merged into 802.11-2020)

See Also

- Ethernet -- wired LAN that mesh portals bridge to
- Zigbee -- low-power mesh for IoT
- Z-Wave -- sub-GHz home automation mesh

ADS-B -- Automatic Dependent Surveillance-Broadcast

Standard: ICAO Annex 10 Vol IV / RTCA DO-260B | **Layer:** Physical / Data Link | **Wireshark filter:** N/A (radio protocol; decoded by dump1090, RTL-SDR tools)

ADS-B is a surveillance technology in which aircraft determine their position via GNSS (GPS) and periodically broadcast it along with identity, altitude, velocity, and other data. It is "Automatic" (no pilot or operator input required), "Dependent" (relies on onboard navigation), and "Surveillance-Broadcast" (transmits to anyone listening, not interrogated like radar). ADS-B Out has been mandatory in most controlled airspace worldwide since January 2020 (FAA in the US, EASA in Europe). Ground stations, other aircraft (ADS-B In), and services like FlightAware and Flightradar24 receive these broadcasts to provide real-time air traffic awareness.

ADS-B Variants

Variant	Frequency	Data Rate	Range	Region
1090ES (Extended Squitter)	1090 MHz	1 Mbps	~250 NM	Worldwide (ICAO standard)
UAT (Universal Access Transceiver)	978 MHz	1 Mbps	~150 NM	US only, below FL180

1090ES is the international standard and is required for all aircraft operating above FL180. UAT is a US-only alternative for general aviation at lower altitudes, also providing TIS-B (Traffic Information Service) and FIS-B (Flight Information Service) uplinks.

1090ES Message Structure (112 bits)

[Diagram: see online version]

Field	Size	Description
DF (Downlink Format)	5 bits	Message format; DF=17 for ADS-B, DF=18 for TIS-B/ADS-R
CA (Capability)	3 bits	Transponder capability (0-7)
ICAO Address	24 bits	Unique aircraft address (e.g., A0B2C3 for a US-registered aircraft)
ME (Message, Extended Squitter)	56 bits	ADS-B payload (type code + data)
PI (Parity/Interrogator ID)	24 bits	CRC-24 parity check overlaid with interrogator identifier

ME Field Type Codes

The first 5 bits of the ME field contain the Type Code (TC), which determines the message content:

TC	Message Type	Description
1-4	Aircraft Identification	Callsign / flight ID (8 characters)
5-8	Surface Position	Position and ground speed for aircraft on the ground
9-18	Airborne Position (Baro Alt)	Lat/lon with barometric altitude
19	Airborne Velocity	Speed, heading, vertical rate
20-22	Airborne Position (GNSS Alt)	Lat/lon with GNSS (geometric) altitude
23-27	Reserved	Future use
28	Aircraft Status	Emergency/priority status, TCAS/ACAS RA
29	Target State and Status	Selected altitude, barometric setting, heading
31	Aircraft Operation Status	ADS-B version, NIC/NACp, SIL, antenna offset

Aircraft Identification (TC 1-4)

The callsign is encoded as 8 characters using ICAO 6-bit character encoding (48 bits):

Character Code	Mapping
1-26	A-Z
32	Space

48-57	0-9

[Diagram: see online version]

Aircraft Category (TC 1-4)

TC	Category	Examples
1	Reserved	--
2	Surface emergency vehicle, service vehicle	
3	A0-A7: Light to heavy aircraft, rotorcraft	A5 = heavy (B747), A3 = large (B737)
4	A0-A7: No info to space vehicle	A1 = light, A2 = small

Airborne Position (TC 9-18)

[Diagram: see online version]

Field	Size	Description
SS	2 bits	Surveillance status (0 = no condition, 1 = permanent alert, 2 = temp alert, 3 = SPI)
SAF	1 bit	Single antenna flag
Altitude	12 bits	Barometric altitude (25-ft or 100-ft increments, Gillham coded)
T	1 bit	UTC time synchronization flag
F	1 bit	CPR format: 0 = even frame, 1 = odd frame
Lat-CPR	17 bits	Compact Position Reporting latitude
Lon-CPR	17 bits	Compact Position Reporting longitude

Compact Position Reporting (CPR)

CPR encodes latitude and longitude into 17 bits each by dividing the globe into zones. Decoding requires two messages (one even, one odd frame) received close together:

- **Global decoding**: uses both even and odd frames to compute an unambiguous position
- **Local decoding**: uses a single frame plus a known reference position (prior position or receiver location)

The CPR scheme achieves approximately 5-meter resolution with just 34 bits for a full position.

Airborne Velocity (TC 19)

Subtype	Description

1	Ground speed (east/north components, subsonic)
2	Ground speed (east/north components, supersonic)
3	Airspeed and heading (subsonic)
4	Airspeed and heading (supersonic)

Subtype 1 is by far the most common. It encodes east-west and north-south velocity components plus vertical rate and GNSS/baro altitude difference.

ADS-B Reception

[Diagram: see online version]

ADS-B Ecosystem

[Diagram: see online version]

Common Decoder Output Formats

Format	Description
SBS (BaseStation)	CSV text format, one line per message
Beast binary	Raw binary with timestamp, signal level, and raw message bytes
AVR	Hex-encoded raw message (*MSG format)
JSON	Structured aircraft state (position, altitude, speed, callsign)

Receiver Hardware

Device	Description
RTL-SDR (R820T2)	Low-cost USB SDR dongle (~$25), 1090 MHz capable
FlightAware Pro Stick	Filtered, amplified RTL-SDR optimized for 1090 MHz
Mode-S Beast	High-performance dedicated receiver
Radarcape	Commercial-grade receiver with GPS timing

Integrity and Accuracy Parameters

Parameter	Abbreviation	Description
Navigation Integrity Category	NIC	Position containment radius
Navigation Accuracy Category - Position	NACp	Estimated position uncertainty (95%)

Navigation Accuracy Category - Velocity	NACv	Estimated velocity uncertainty (95%)
Surveillance Integrity Level	SIL	Probability of exceeding NIC containment
Source Integrity Level	SIL Supplement	Per-hour or per-sample SIL basis

Standards

Document	Title
ICAO Annex 10 Vol IV	Surveillance and Collision Avoidance Systems
RTCA DO-260B	MOPS for 1090 MHz Extended Squitter ADS-B and TIS-B
EUROCAE ED-102A	European equivalent of DO-260B
RTCA DO-282B	MOPS for UAT ADS-B
FAA 14 CFR 91.225/227	US ADS-B Out equipment mandate

See Also

- NFC -- contactless technology using radio (different domain, similar broadcast concept)
- Zigbee -- another wireless broadcast protocol (IoT domain)
- Bluetooth -- short-range wireless for comparison

ANT / ANT+

Standard: ANT Message Protocol (thisisant.com) / ANT+ Device Profiles | **Layer:** Full stack (Physical through Application) | **Wireshark filter:** N/A (proprietary 2.4 GHz radio, not IP-based)

ANT is a low-power wireless sensor network protocol operating in the 2.4 GHz ISM band, designed by Dynastream Innovations (a Garmin subsidiary). It provides a simple, reliable channel-based communication model for personal area networks. ANT+ is an interoperability layer built on ANT that defines standardized device profiles for fitness, health, and sport applications -- heart rate monitors, power meters, speed/cadence sensors, and fitness equipment. ANT is embedded in Garmin devices, many smartphones, and fitness equipment worldwide.

ANT Message Structure

All ANT communication between a host processor and the ANT radio chip uses a serial message format:

[Diagram: see online version]

Message Fields

Field	Size	Description
Sync	1 byte	Always 0xA4 (indicates start of ANT message)
Message Length	1 byte	Number of bytes in the content field (0-9 for standard messages)
Message ID	1 byte	Identifies the message type (configuration, data, event)
Content	0-9 bytes	Channel number and/or data payload (varies by message type)
Checksum	1 byte	XOR of all preceding bytes (Sync through last content byte)

ANT Message IDs

Configuration Messages (Host to ANT)

Message ID	Name	Description
0x42	Assign Channel	Assign a channel type (bidirectional, shared, etc.)
0x51	Set Channel ID	Set device number, device type, transmission type
0x43	Set Channel Period	Set message period (counts of 32768 Hz clock)
0x45	Set Channel RF Frequency	Set radio frequency offset from 2400 MHz
0x46	Set Channel Search Timeout	Set search timeout before giving up
0x47	Set Channel Low Priority Search Timeout	Extended low-priority search
0x44	Set Network Key	Set the 8-byte network key for a network
0x4B	Open Channel	Open a configured channel for communication
0x4C	Close Channel	Close an open channel
0x41	Unassign Channel	Release a channel assignment
0x4A	Request Message	Request a specific information message
0x6E	Set Search Waveform	Optimize search for specific applications

Data Messages

Message ID	Name	Description
0x4E	Broadcast Data	Send/receive 8-byte broadcast (unreliable, periodic)

0x4F	Acknowledged Data	Send/receive 8-byte acknowledged (reliable, single)
0x50	Burst Data	Send/receive 8-byte burst packet (reliable, high-throughput)
0x72	Advanced Burst Data	Extended burst with larger packets

Channel Event / Response Messages (ANT to Host)

Message ID	Name	Description
0x40	Channel Response / Event	Response to a command or channel event notification

Common Channel Events (via 0x40)

Event Code	Name	Description
0x00	RESPONSE_NO_ERROR	Command completed successfully
0x01	EVENT_RX_SEARCH_TIMEOUT	Search timeout -- no device found
0x02	EVENT_RX_FAIL	Expected message not received
0x03	EVENT_TX	Broadcast message transmitted successfully
0x05	EVENT_TRANSFER_TX_COMPLETED	Acknowledged/burst transfer completed
0x06	EVENT_TRANSFER_TX_FAILED	Acknowledged/burst transfer failed
0x07	EVENT_CHANNEL_CLOSED	Channel has been closed
0x0A	EVENT_RX_FAIL_GO_TO_SEARCH	Too many missed messages, returning to search

Information Messages

Message ID	Name	Description
0x52	Channel ID	Response: device number, device type, transmission type
0x54	ANT Version	ANT firmware version string
0x61	Capabilities	Supported features and number of channels
0x69	Serial Number	Device serial number

Channel Types

Type	Value	Description

Bidirectional Slave	0x00	Receives broadcast, can send acknowledged data back
Bidirectional Master	0x10	Transmits broadcast data, receives acknowledged responses
Shared Bidirectional Slave	0x20	Multiple slaves share one master's channel
Shared Bidirectional Master	0x30	Master communicates with multiple slaves
Slave Receive Only	0x40	Receive-only mode (no transmission capability)

Channel Communication

[Diagram: see online version]

Channel ID

Every ANT device is identified by a Channel ID used for pairing:

[Diagram: see online version]

Field	Size	Description
Device Number	16 bits	Unique device number (0 = wildcard for search)
Device Type	8 bits	Type of device (defines ANT+ profile)
Transmission Type	8 bits	Encoding of transmission characteristics

Setting any field to 0 during search acts as a wildcard, matching any value.

ANT+ Device Profiles

ANT+ defines standardized device profiles that ensure interoperability between manufacturers:

Common ANT+ Profiles

Device Type	Hex	Profile	Channel Period	RF Freq
120	0x78	Heart Rate	8070 (4.06 Hz)	57 (2457 MHz)
121	0x79	Speed & Cadence (combined)	8086 (4.05 Hz)	57
122	0x7A	Speed (standalone)	8118 (4.03 Hz)	57
123	0x7B	Cadence (standalone)	8102 (4.04 Hz)	57
11	0x0B	Bicycle Power	8182 (4.00 Hz)	57

17	0x11	Fitness Equipment	8192 (4.00 Hz)	57
25	0x19	Environment (temperature/humidity)	65535 (0.50 Hz)	57
119	0x77	Weight Scale	8192 (4.00 Hz)	57
18	0x12	Blood Pressure	8192 (4.00 Hz)	57
124	0x7C	Multi-Sport Speed & Distance (Footpod)	8134 (4.02 Hz)	57
12	0x0C	Geocache	8192 (4.00 Hz)	57
16	0x10	LEV (Light Electric Vehicle)	8192 (4.00 Hz)	57

Heart Rate Data Page 0 (Standard)

[Diagram: see online version]

Field	Size	Description
Page Number	8 bits	Data page identifier (0 = default)
Heart Beat Event Time	16 bits	Time of last heartbeat (1/1024 s resolution)
Heart Beat Count	8 bits	Rolling heartbeat counter
Computed Heart Rate	8 bits	Instantaneous heart rate in BPM (0-255)

Bicycle Power Data Page 0x10 (Standard Power Only)

[Diagram: see online version]

Common Data Pages (All Profiles)

Page	Name	Description
0x50 (80)	Manufacturer ID	Hardware manufacturer and model
0x51 (81)	Product Information	SW revision, serial number
0x52 (82)	Battery Status	Battery voltage, status, operating time

Radio Parameters

Parameter	Value
Frequency band	2.4 GHz ISM (2400-2524 MHz)
Number of channels	125 (1 MHz spacing)
Default ANT+ frequency	2457 MHz (offset 57)

Data rate	1 Mbps
Modulation	GFSK
Transmit power	0 dBm typical (configurable, up to +4 dBm)
Range	5-30 m typical (line of sight)
Message payload	8 bytes per message (broadcast/acknowledged)
Channel period	Application-dependent (typically 4 Hz for fitness)
Latency	~1 channel period (250 ms at 4 Hz)

Network Key

ANT channels use an 8-byte network key for authentication. Devices must share the same network key to communicate:

Network	Key	Description
ANT+	Managed by Garmin/Dynastream	Public ANT+ network key (available to ANT+ adopters)
ANT-FS	Application-specific	File transfer network
Custom	User-defined	Private application networks

The ANT+ network key provides a basic level of access control -- only devices programmed with the key can join the ANT+ network. It does not provide encryption.

ANT vs Bluetooth LE vs Zigbee

Feature	ANT / ANT+	Bluetooth LE	Zigbee
Frequency	2.4 GHz	2.4 GHz	2.4 GHz
Data rate	1 Mbps	1-2 Mbps	250 kbps
Payload	8 bytes	20-244 bytes	~100 bytes
Topology	Star, mesh, tree	Point-to-point, star	Mesh, star, tree
Power	Ultra-low (coin cell years)	Very low	Low
Range	5-30 m	10-100 m	10-30 m
Coexistence	Adaptive frequency hopping	Adaptive frequency hopping	DSSS, channel selection
Primary use	Sports/fitness sensors	General IoT, audio, fitness	Home automation, industrial
Smartphone support	Limited (some Android, Garmin)	Universal	Rare (hub required)
Bidirectional	Yes	Yes	Yes
Multi-device	Yes (concurrent channels)	Limited connections	Up to 65,000 nodes

Encapsulation

[Diagram: see online version]

ANT uses a proprietary radio protocol. USB ANT sticks (e.g., Garmin ANT+ USB stick) bridge ANT radio to host computers via USB serial. The host communicates with the ANT chip using the serial message format described above.

Standards

Document	Title
ANT Message Protocol	ANT Message Protocol and Usage specification
ANT+ Device Profiles	Managed device profiles for interoperability
ANT-FS	ANT File Share protocol for bulk data transfer
D52 Module Datasheet	ANT SoC / module reference (nRF52-based)

See Also

- Bluetooth / BLE -- competing low-power wireless (broader ecosystem)
- Zigbee -- mesh wireless for home automation and industrial

BLE -- Bluetooth Low Energy

Standard: Bluetooth Core Specification (bluetooth.com) | **Layer:** Full stack (Physical through Application) | **Wireshark filter:** `btle` or `btatt` or `btl2cap`

BLE (Bluetooth Low Energy, originally "Bluetooth Smart") is a wireless protocol optimized for short bursts of data from battery-powered devices. Introduced in Bluetooth 4.0 (2010), BLE shares the 2.4 GHz band with Classic Bluetooth but uses a completely different stack -- different PHY, different link layer, and the GATT/ATT attribute model that defines how all BLE data is exposed. This page covers the details embedded developers work with daily: GATT attributes, advertising data format, connection parameters, and the security model.

Protocol Stack

[Diagram: see online version]

GATT (Generic Attribute Profile)

GATT defines how BLE devices expose data as a hierarchy of **Services**, **Characteristics**, and **Descriptors** -- all stored as **Attributes** in an attribute table.

Attribute Table

Every piece of data on a BLE device is an attribute:

[Diagram: see online version]

Field	Size	Description
Handle	16 bits	Unique index in the attribute table (0x0001-0xFFFF)
UUID	16 or 128 bits	Attribute type identifier
Permissions	Flags	Read, write, encrypt, authenticate
Value	Variable	The attribute data (up to 512 bytes, negotiable via MTU)

GATT Hierarchy

[Diagram: see online version]

Service Declaration (Attribute)

Handle	UUID	Value
0x0001	0x2800 (Primary Service)	0x180D (Heart Rate Service UUID)

Characteristic Declaration (Attribute)

Handle	UUID	Value
0x0002	0x2803 (Characteristic)	Properties + Value Handle + Char UUID

Characteristic Properties (1 byte)

[Diagram: see online version]

Bit	Property	Description
0	Broadcast	Value can be broadcast in advertising
1	Read	Value can be read
2	Write Without Response	Value can be written without acknowledgment
3	Write	Value can be written with acknowledgment
4	Notify	Server can send value without acknowledgment

5	Indicate	Server can send value with acknowledgment
6	Authenticated Signed Writes	Signed write support
7	Extended Properties	Additional properties in Extended Properties descriptor

CCCD (Client Characteristic Configuration Descriptor -- 0x2902)

The most important descriptor -- the client writes to it to enable notifications or indications:

Value	Meaning
0x0000	Notifications and indications disabled
0x0001	Notifications enabled
0x0002	Indications enabled
0x0003	Both enabled

Common Standard UUIDs

Services

UUID	Service
0x1800	Generic Access
0x1801	Generic Attribute
0x180A	Device Information
0x180D	Heart Rate
0x180F	Battery Service
0x1810	Blood Pressure
0x1816	Cycling Speed and Cadence
0x181A	Environmental Sensing
0x181C	User Data
0x1822	Pulse Oximeter

Characteristics

UUID	Characteristic
0x2A00	Device Name
0x2A01	Appearance
0x2A04	Peripheral Preferred Connection Parameters
0x2A19	Battery Level

0x2A23	System ID
0x2A24	Model Number String
0x2A25	Serial Number String
0x2A26	Firmware Revision
0x2A28	Software Revision
0x2A29	Manufacturer Name
0x2A37	Heart Rate Measurement
0x2A6E	Temperature
0x2A6F	Humidity

Descriptors

UUID	Descriptor
0x2900	Characteristic Extended Properties
0x2901	Characteristic User Description
0x2902	Client Characteristic Configuration (CCCD)
0x2903	Server Characteristic Configuration
0x2904	Characteristic Presentation Format

Custom UUIDs (128-bit)

Vendor-specific services use full 128-bit UUIDs. The Bluetooth Base UUID is:

```
XXXXXXXX-0000-1000-8000-00805F9B34FB
```

Standard 16-bit UUIDs are shorthand: `0x180D` -> `0000180D-0000-1000-8000-00805F9B34FB`

Custom services use random 128-bit UUIDs, e.g., Nordic UART Service: `6E400001-B5A3-F393-E0A9-E50E24DCCA9E`

ATT Protocol Operations

Opcode	Name	Direction	Description
0x02	Exchange MTU Request	C->S	Negotiate ATT MTU (default 23, max 517)
0x03	Exchange MTU Response	S->C	Server's MTU
0x04	Find Information Request	C->S	Discover attribute handles and UUIDs
0x08	Read By Type Request	C->S	Read attributes by UUID (service/char discovery)

0x0A	Read Request	C->S	Read an attribute value by handle
0x0B	Read Response	S->C	Return attribute value
0x12	Write Request	C->S	Write with acknowledgment
0x13	Write Response	S->C	Write acknowledged
0x1B	Handle Value Notification	S->C	Server pushes value (no ACK)
0x1D	Handle Value Indication	S->C	Server pushes value (ACK required)
0x1E	Handle Value Confirmation	C->S	ACK for indication
0x52	Write Command	C->S	Write without acknowledgment

ATT MTU and Data Length

Parameter	Default	Maximum	Effect
ATT MTU	23 bytes	517 bytes	Max attribute value per ATT operation
ATT payload	20 bytes	514 bytes	MTU minus 3-byte ATT header
LE Data Length	27 bytes	251 bytes	Max Link Layer payload (BLE 4.2+)
Throughput (1M PHY)	~2.5 KB/s	~65 KB/s	Larger MTU + DLE dramatically improve speed

Advertising

Advertising PDU Format

[Diagram: see online version]

Advertising Data (AD) Structures

AdvData contains one or more AD structures, each with Length + Type + Data:

[Diagram: see online version]

Common AD Types

Type	Name	Description
0x01	Flags	Discoverable mode, BR/EDR support
0x02	Incomplete 16-bit UUIDs	Partial list of service UUIDs

0x03	Complete 16-bit UUIDs	Full list of service UUIDs
0x06	Incomplete 128-bit UUIDs	Partial list of custom service UUIDs
0x07	Complete 128-bit UUIDs	Full list of custom service UUIDs
0x08	Shortened Local Name	Device name (truncated)
0x09	Complete Local Name	Full device name
0x0A	TX Power Level	Transmit power in dBm
0xFF	Manufacturer Specific Data	Vendor data (first 2 bytes = company ID)
0x16	Service Data (16-bit UUID)	UUID + data payload
0x20	Service Data (32-bit UUID)	UUID + data payload
0x21	Service Data (128-bit UUID)	UUID + data payload
0x14	List of 16-bit Solicitation UUIDs	Services the peripheral wants to use

Flags (AD Type 0x01)

[Diagram: see online version]

Typical BLE-only device: `0x06` (General Discoverable + BR/EDR Not Supported)

Advertising Channels

Channel	Frequency	Purpose
37	2402 MHz	Advertising
38	2426 MHz	Advertising
39	2480 MHz	Advertising
0-36	2404-2478 MHz	Data channels (adaptive frequency hopping)

Advertising Types

PDU Type	Name	Connectable	Scannable	Directed
ADV_IND	Connectable Undirected	Yes	Yes	No
ADV_DIRECT_IND	Connectable Directed	Yes	No	Yes
ADV_NONCONN_IND	Non-connectable	No	No	No
ADV_SCAN_IND	Scannable Undirected	No	Yes	No

Extended Advertising (BLE 5.0+)

Feature	Legacy	Extended

Adv data size	31 bytes	254 bytes (chained up to ~1650)
Channels	37, 38, 39 only	Adv on 37-39, data on any channel
Coded PHY	No	Yes (long range)
Multiple adv sets	No	Yes (concurrent)
Periodic advertising	No	Yes (synchronized broadcasts)

Connection Parameters

After `CONNECT_IND`, the link layer uses these parameters:

Parameter	Range	Default	Description
Connection Interval	7.5ms - 4s	30-50ms typical	Time between connection events
Slave Latency	0-499	0	Number of connection events the peripheral can skip
Supervision Timeout	100ms - 32s	4-6s	Time before declaring connection lost
CE Length	0 - conn interval	--	Minimum/maximum connection event length

Throughput Estimation

```
Throughput ≈ (ATT_payload x packets_per_interval) / connection_interval
```

Scenario	Interval	MTU	PHY	Approx Throughput
Conservative	30ms	23	1M	~5 KB/s
Typical	15ms	247	1M	~65 KB/s
Maximum	7.5ms	247	2M	~170 KB/s

Connection Parameter Update

[Diagram: see online version]

Security (SMP -- Security Manager Protocol)

Pairing Methods

Method	MITM Protection	User Interaction	Use Case
Just Works	No	None	Headsets, mice (no display/keyboard)

Passkey Entry	Yes	Enter 6-digit code	Keyboard devices
Numeric Comparison	Yes	Confirm 6-digit number on both	Phones, watches (BLE 4.2+)
Out of Band (OOB)	Yes	NFC tap, QR code scan	High security, easy pairing

Pairing Flow (LE Secure Connections -- BLE 4.2+)

[Diagram: see online version]

Security Keys

Key	Name	Purpose
LTK	Long Term Key	Encryption key (stored for bonding)
IRK	Identity Resolving Key	Resolve random addresses to identity
CSRK	Connection Signature Resolving Key	Verify signed data
EDIV + Rand	Encrypted Diversifier + Random	Identify the LTK during reconnection (Legacy only)

Address Types

Type	Format	Description
Public	IEEE-assigned	Fixed, globally unique (like a MAC)
Random Static	11:xx:xx:xx:xx:xx	Fixed per boot (or permanent), locally assigned
Random Private Resolvable	01:xx:xx:xx:xx:xx	Changes periodically, resolvable with IRK
Random Private Non-Resolvable	00:xx:xx:xx:xx:xx	Changes periodically, not resolvable (rare)

Resolvable Private Addresses (RPAs) are the default in modern BLE -- they rotate every ~15 minutes to prevent tracking, but bonded devices can resolve them using the stored IRK.

Security Levels

Level	Mode	Description
1	No Security	No encryption, no authentication
2	Unauthenticated Encryption	Encrypted (Just Works pairing)
3	Authenticated Encryption	Encrypted + MITM protection (Passkey/Numeric Comparison)
4	LE Secure Connections + Authenticated	ECDH-based + MITM protection (highest)

PHY Options (BLE 5.0+)

PHY	Data Rate	Range	Use Case
LE 1M	1 Mbps	~100m	Default, backward compatible
LE 2M	2 Mbps	~80m	Higher throughput, lower power per bit
LE Coded (S=2)	500 kbps	~200m	Extended range
LE Coded (S=8)	125 kbps	~400m	Maximum range

Common BLE Profiles

Profile	Description	Example Devices
HRP (Heart Rate)	Heart rate monitoring	Fitness bands, chest straps
BLP (Blood Pressure)	Blood pressure measurement	BP monitors
GLP (Glucose)	Glucose readings	Glucose meters
HOGP (HID over GATT)	Keyboards, mice, gamepads	BLE peripherals
PXP (Proximity)	Proximity detection	Key finders, asset tags
FMP (Find Me)	Alert when out of range	Tile-like trackers
ESP (Environmental Sensing)	Temperature, humidity, pressure	Weather stations
CGM (Continuous Glucose)	Continuous glucose monitoring	CGM devices

Encapsulation

[Diagram: see online version]

Standards

Document	Title
Bluetooth Core 6.0	Core Specification (Vol 1-6 cover BLE)
Bluetooth GATT Specs	Services, Characteristics, Descriptors
Bluetooth Assigned Numbers	UUIDs, company IDs, AD types
Bluetooth Mesh	BLE Mesh Protocol

See Also

- Bluetooth -- overview covering both Classic and BLE
- Zigbee -- alternative low-power wireless mesh

- NFC -- very short range, sometimes used for BLE OOB pairing
- I2C -- BLE sensor modules often connect to MCUs via I2C/SPI

Bluetooth / BLE -- Bluetooth Low Energy

Standard: Bluetooth Core Specification (bluetooth.com) | **Layer:** Full stack (Physical through Application) | **Wireshark filter:** `bthci_evt` or `btle` or `btatt`

Bluetooth is a short-range wireless technology for exchanging data between devices in the 2.4 GHz ISM band. Bluetooth Classic (BR/EDR) provides continuous streaming connections for audio (headphones, speakers) and serial data. Bluetooth Low Energy (BLE), introduced in Bluetooth 4.0 (2010), is optimized for intermittent data transfer from battery-powered devices -- fitness trackers, sensors, beacons, smart home, and medical devices. Modern Bluetooth 5.x supports both Classic and BLE, with recent versions adding mesh networking and LE Audio.

Protocol Stack

[Diagram: see online version]

BLE Link Layer Packet

[Diagram: see online version]

PDU Header

[Diagram: see online version]

Key Fields (BLE)

Field	Size	Description
Preamble	1-2 bytes	Synchronization (0xAA or 0x55 pattern)
Access Address	4 bytes	0x8E89BED6 for advertising; random for data channels
PDU Type	4 bits	Advertising or data PDU type
TxAdd/RxAdd	1 bit each	Address type: 0 = public, 1 = random
Length	8 bits	Payload length
CRC	3 bytes	CRC-24

BLE Advertising PDU Types

Type	Name	Description
0	ADV_IND	Connectable undirected advertising
1	ADV_DIRECT_IND	Connectable directed advertising
2	ADV_NONCONN_IND	Non-connectable (beacon)
3	SCAN_REQ	Active scan request
4	SCAN_RSP	Scan response (additional data)
5	CONNECT_IND	Connection request
6	ADV_SCAN_IND	Scannable undirected advertising

BLE Connection

[Diagram: see online version]

GATT (Generic Attribute Profile)

GATT defines how BLE devices expose data as a hierarchy of services and characteristics:

[Diagram: see online version]

Common GATT Services

UUID	Service	Description
0x1800	Generic Access	Device name, appearance
0x1801	Generic Attribute	Service changed indication
0x180A	Device Information	Manufacturer, model, firmware version
0x180D	Heart Rate	Heart rate measurement
0x180F	Battery Service	Battery level percentage
0x1810	Blood Pressure	Blood pressure measurement
0x181A	Environmental Sensing	Temperature, humidity, pressure
0x181C	User Data	User profile data

Radio Parameters

Parameter	Classic (BR/EDR)	BLE
Frequency	2.4 GHz ISM	2.4 GHz ISM
Channels	79 x 1 MHz	40 x 2 MHz (3 advertising, 37 data)
Data rate	1 Mbps (BR), 2-3 Mbps (EDR)	1 Mbps (LE 1M), 2 Mbps (LE 2M)

Range	~10-100 m	~10-100 m (400 m with LE Coded)
Hopping	FHSS, 1600 hops/sec	Adaptive frequency hopping
Transmit power	0-20 dBm (class dependent)	-20 to +20 dBm
Connection interval	--	7.5 ms - 4 sec
Latency	~100 ms	~6 ms (BLE 5.0)

Bluetooth Versions

Version	Year	Key Features
1.0	1999	Initial release
2.0 + EDR	2004	Enhanced Data Rate (3 Mbps)
3.0 + HS	2009	High Speed (Wi-Fi for bulk transfer)
4.0	2010	Bluetooth Low Energy (BLE) introduced
4.2	2014	Larger packets, LE Data Length Extension, IPv6 (6LoWPAN)
5.0	2016	2x speed, 4x range, advertising extensions
5.1	2019	Direction finding (AoA/AoD)
5.2	2020	LE Audio (LC3 codec), Isochronous Channels
5.3	2021	Connection subrating, periodic advertising enhancement
5.4	2023	PAwR (Periodic Advertising with Responses)
6.0	2024	Channel sounding (cm-level ranging)

Security

Feature	Classic	BLE
Pairing	PIN, Numeric Comparison, Passkey	Just Works, Passkey, Numeric Comparison, OOB
Encryption	E0 (legacy), AES-CCM (Secure Connections)	AES-CCM
Authentication	HMAC-SHA-256	AES-CMAC
MITM protection	Numeric Comparison, OOB	Numeric Comparison, Passkey, OOB
LE Secure Connections	--	ECDH P-256 key exchange (BLE 4.2+)

Standards

Document	Title
Bluetooth Core Spec 6.0	Bluetooth Core Specification
Bluetooth GATT	GATT Specifications (services, characteristics)
Bluetooth Mesh	Bluetooth Mesh Protocol

See Also

- NFC -- very short range, used for pairing and payments
- Zigbee -- alternative low-power wireless (mesh-focused)
- Z-Wave -- alternative home automation wireless
- 802.11 (Wi-Fi) -- higher-bandwidth wireless LAN

DMR -- Digital Mobile Radio

Standard: ETSI TS 102 361 | **Layer:** Full stack (Physical through Application) | **Wireshark filter:** `dmr` (with DSD+/op25/SDR decoders)

DMR is an open digital two-way radio standard developed by ETSI for professional mobile radio (PMR) and amateur radio use. Its key innovation is TDMA (Time Division Multiple Access) -- splitting a single 12.5 kHz channel into two independent time slots, effectively doubling the capacity of a frequency allocation. DMR uses 4FSK modulation and the AMBE+2 voice codec, delivering clear digital audio with better spectral efficiency than analog FM. DMR has seen massive adoption in commercial, industrial, and amateur radio through manufacturers like Motorola, Hytera, and Tait, and through global amateur networks like Brandmeister and DMR-MARC.

DMR Tiers

Tier	Name	Description	Licensing
Tier I	Unlicensed	Low power (0.5 W), FDMA, 446 MHz (Europe)	License-free (dPMR446)
Tier II	Conventional	Licensed, repeater-based, TDMA 2-slot	Licensed PMR / amateur
Tier III	Trunked	Licensed, multi-site trunked with dynamic channel assignment	Licensed PMR

Tier II is by far the most widely deployed for both commercial and amateur radio use.

TDMA Frame Structure

DMR divides a 12.5 kHz channel into two time slots of 30 ms each, yielding a 60 ms TDMA frame:

[Diagram: see online version]

Each time slot carries an independent conversation, effectively giving two logical channels per physical frequency.

Burst Structure (Per Time Slot)

Each 30 ms time slot contains a single burst of 264 bits at 4800 symbols/s (9600 bps):

[Diagram: see online version]

Field	Size	Description
CACH	24 bits	Common Announcement Channel -- short LC fragments, slot type
Payload 1	108 bits	First half of voice or data payload
SYNC / EMB	48 bits	Synchronization pattern (voice/data/RC sync) or Embedded signaling
Payload 2	108 bits	Second half of voice or data payload

CACH (Common Announcement Channel)

The 24-bit CACH is present in every burst and carries:

- TACT (Trunk Announcement) bits for Tier III
- Short LC fragments (assembled from 4 consecutive bursts to form a 12-byte Short LC)
- The Short LC contains source ID, destination ID, and group/private call flag

SYNC Patterns

Pattern	Description
BS Voice	Base station voice sync
BS Data	Base station data sync
MS Voice	Mobile station voice sync
MS Data	Mobile station data sync
RC Sync	Reverse channel sync
Direct Voice TS1/TS2	Direct mode (simplex) voice
Direct Data TS1/TS2	Direct mode data

Link Control (LC)

Full Link Control is 96 bits total (72 bits of LC data + 24 bits of CRC) and is embedded across multiple voice bursts using the EMB field. It carries:

Field	Description
Protect Flag	0 = clear, 1 = encrypted
FLCO	Full LC Opcode (group voice, unit-to-unit voice, GPS, talker alias)
Feature Set ID	0 = standard, others vendor-specific
Service Options	Emergency, privacy, broadcast
Destination ID	24-bit talkgroup or radio ID
Source ID	24-bit originating radio ID

Data Types

Type	Description
Voice	AMBE+2 encoded voice frames (3 frames per superframe)
Voice + Embedded LC	Voice with Link Control in EMB fields
PI Header	Privacy (encryption) indicator header
Data Header	Start of a data transmission
Rate 1/2 Data	FEC-protected data (confirmed or unconfirmed)
Rate 3/4 Data	Less FEC, higher throughput
Rate 1 Data	No FEC, maximum throughput
CSBK	Control Signaling Block -- standalone signaling (preamble, aloha, etc.)
MBC	Multi-Block Control -- extended signaling
Idle	Idle burst (no traffic)

Voice Codec

Parameter	Value
Codec	AMBE+2 (Digital Voice Systems Inc.)
Bit rate	3600 bps (voice) + 2400 bps (FEC) = 6000 bps per time slot
Frame duration	60 ms (3 voice frames of 20 ms each per superframe)
Audio bandwidth	~300-3400 Hz

Modulation and RF

Parameter	Value
Modulation	4FSK (4-level Frequency Shift Keying)

Symbol rate	4800 symbols/s
Bit rate	9600 bps (2 bits per symbol)
Channel spacing	12.5 kHz
Deviation	+/- 1944 Hz (outer), +/- 648 Hz (inner)
Frequency bands	VHF (136-174 MHz), UHF (403-527 MHz)

Color Codes

Color Codes (0-15) serve a function similar to CTCSS/DCS tones in analog radio -- they prevent a radio from receiving traffic from an unintended repeater on the same frequency. Both the radio and repeater must be set to the same Color Code for communication. The Color Code is carried in the CACH and EMB fields.

Addressing

Address	Range	Description
Radio ID	1 - 16,776,415	Individual radio identifier (24-bit)
Talkgroup	1 - 16,776,415	Group call identifier (24-bit)
All Call	16,777,215	Broadcast to all radios

Group Call Flow

[Diagram: see online version]

IP Site Connect (IPSC)

IPSC links multiple DMR repeaters over IP networks, extending talkgroup coverage across sites:

[Diagram: see online version]

Amateur DMR Networks

Network	Description
Brandmeister	Largest global amateur DMR network, supports hotspots, multi-protocol
DMR-MARC	DMR-Mobile Amateur Radio Club, curated talkgroup system
TGIF	Community-driven, simplified talkgroup structure
FreeDMR	Open-source alternative network

Hotspots

Low-power personal DMR gateways (MMDVM-based) connect a single radio to DMR networks via the internet, enabling global reach from home:

Radio -> Hotspot (MMDVM + Pi-Star/WPSD) -> Internet -> Brandmeister -> World

Encryption

Type	Description
Basic Privacy	XOR-based scrambling (40-bit key), easily broken
Enhanced Privacy	AES-128 or AES-256 encryption (Tier II/III)
Hytera End-to-End	Proprietary E2E encryption (vendor-specific)

Note: Amateur radio use of DMR prohibits encryption in most jurisdictions.

Standards

Document	Title
ETSI TS 102 361-1	Air Interface (AI) protocol
ETSI TS 102 361-2	Voice and Generic Services and Facilities
ETSI TS 102 361-3	Data Protocol
ETSI TS 102 361-4	Trunking Protocol (Tier III)

See Also

- TETRA -- European emergency services digital radio (4-slot TDMA)
- Bluetooth -- short-range wireless for comparison
- Zigbee -- low-power wireless mesh

DVB-S / DVB-S2 / DVB-S2X -- Digital Video Broadcasting - Satellite

Standard: ETSI EN 302 307 (DVB-S2) / ETSI EN 302 307-2 (DVB-S2X) | **Layer:** Physical / Data Link | **Wireshark filter:** `dvb-s2` or `mp2t` (MPEG Transport Stream)

DVB-S, DVB-S2, and DVB-S2X are the successive generations of the Digital Video Broadcasting standard for satellite transmission. DVB-S (1994) established digital satellite TV using QPSK modulation. DVB-S2 (2005) introduced Adaptive Coding and Modulation (ACM) and LDPC/BCH forward error correction, roughly doubling spectral efficiency. DVB-S2X (2014) further extended the standard with finer granularity MODCODs, higher-order modulation up to 256APSK, and smaller roll-off factors. These standards carry the majority of the world's satellite

television (Sky, DirecTV, Freesat, Canal+), satellite internet (VSAT), contribution/distribution feeds, and professional video links.

DVB-S / DVB-S2 / DVB-S2X Comparison

Feature	DVB-S	DVB-S2	DVB-S2X
Year	1994	2005	2014
Modulation	QPSK	QPSK, 8PSK, 16APSK, 32APSK	+ 64APSK, 128APSK, 256APSK
FEC	Convolutional + Reed-Solomon	LDPC + BCH	LDPC + BCH (finer rates)
Roll-off factors	0.35	0.35, 0.25, 0.20	+ 0.15, 0.10, 0.05
Spectral efficiency	~2 bps/Hz max	~3-4 bps/Hz	~5 bps/Hz
ACM support	No (CCM only)	Yes (ACM/VCM)	Yes (enhanced)
MODCODs	1 (QPSK + rate)	28	116+
Transport	MPEG-TS only	MPEG-TS or GSE	MPEG-TS, GSE, or super-frame
Gain over DVB-S	Baseline	~30% capacity	~5-20% over DVB-S2

System Architecture

[Diagram: see online version]

Baseband Frame (BBFRAME)

The BBFRAME is the fundamental data unit in DVB-S2. The modulator receives BBFRAMEs, applies FEC coding, modulation, and physical layer framing.

[Diagram: see online version]

Field	Size	Description
MATYPE	2 bytes	Mode adaptation type: TS/GS input, SIS/MIS, CCM/ACM/VCM, ISSYI, NPD, roll-off
UPL	16 bits	User Packet Length (188 x 8 = 1504 for MPEG-TS)
DFL	16 bits	Data Field Length in bits
SYNC	8 bits	User packet sync byte (0x47 for MPEG-TS)
SYNCD	16 bits	Distance in bits from start of BBFRAME data to first UP sync
CRC-8	8 bits	CRC of the BBHEADER (10 bytes)

Data Field	Variable	User packets (MPEG-TS or generic stream)

MATYPE Field

Bits	Field	Description
0-1	TS/GS	11 = Transport Stream, 00 = Generic Packetized, 01 = Generic Continuous, 10 = Reserved
2	SIS/MIS	1 = Single Input Stream, 0 = Multiple Input Streams
3	CCM/ACM	1 = Constant Coding and Modulation, 0 = Adaptive
4	ISSYI	Input Stream Synchronization Indicator
5	NPD	Null Packet Deletion (efficiency for sparse TS)
6-7	Roll-off	00 = 0.35, 01 = 0.25, 10 = 0.20 (DVB-S2X adds 0.15, 0.10, 0.05)

Physical Layer Frame (PLFRAME)

[Diagram: see online version]

Field	Size	Description
PLHEADER	90 symbols	SOF (26 symbols) + PLSCODE (64 symbols)
XFECFRAME	Variable	FEC-encoded data (16200 or 64800 bits, depending on frame size)
Pilot blocks	36 symbols	Inserted every 16 slots (optional, for synchronization)

PLSCODE

The 64-symbol PLSCODE encodes the MODCOD (5 bits for modulation + FEC rate) and frame type (normal or short), enabling the receiver to identify how to demodulate each frame -- essential for ACM operation where every frame can use a different MODCOD.

FEC (Forward Error Correction)

DVB-S

Stage	Code	Description

Inner	Convolutional	Rates 1/2, 2/3, 3/4, 5/6, 7/8
Outer	Reed-Solomon (204,188)	Corrects up to 8 byte errors per packet

DVB-S2 / DVB-S2X

Stage	Code	Block Size	Description
Inner	LDPC	64800 (normal) or 16200 (short) bits	Low-Density Parity-Check: near-Shannon-limit performance
Outer	BCH	Variable	Bose-Chaudhuri-Hocquenghem: catches residual errors

LDPC + BCH operates within 0.7-1.0 dB of the Shannon limit, a significant improvement over convolutional + Reed-Solomon.

MODCOD (Modulation and Coding)

Each MODCOD defines a combination of modulation scheme and FEC code rate. The receiver reports its current signal-to-noise ratio (SNR), and the modulator selects the best MODCOD for the conditions.

DVB-S2 MODCODs (Selection)

Modulation	Code Rates	Approx. Spectral Efficiency	Min Es/N0
QPSK	1/4, 1/3, 2/5, 1/2, 3/5, 2/3, 3/4, 4/5, 5/6, 8/9, 9/10	0.49 - 1.79 bps/Hz	-2.35 to 6.42 dB
8PSK	3/5, 2/3, 3/4, 5/6, 8/9, 9/10	1.78 - 2.68 bps/Hz	5.50 to 10.69 dB
16APSK	2/3, 3/4, 4/5, 5/6, 8/9, 9/10	2.64 - 3.57 bps/Hz	8.97 to 13.13 dB
32APSK	3/4, 4/5, 5/6, 8/9, 9/10	3.70 - 4.45 bps/Hz	12.73 to 16.05 dB

DVB-S2X Additional Modulations

DVB-S2X adds 64APSK, 128APSK, and 256APSK for very high SNR links (e.g., spot beams, short hops), as well as very low SNR MODCODs (QPSK 2/9, 13/45) for mobile and low-margin links.

Adaptive Coding and Modulation (ACM)

[Diagram: see online version]

ACM allows the system to continuously optimize throughput per receiver. During clear sky, high-order modulation maximizes capacity; during rain fade, the system falls back to robust lower-order modulation, maintaining the link without outage.

Coding Modes

Mode	Abbreviation	Description
Constant Coding and Modulation	CCM	Single MODCOD for all frames (broadcast)
Variable Coding and Modulation	VCM	Different MODCODs per stream, fixed per stream
Adaptive Coding and Modulation	ACM	MODCOD adapts per-frame based on receiver feedback

Transport Mechanisms

MPEG Transport Stream (TS)

Parameter	Value
Packet size	188 bytes (fixed)
Sync byte	0x47
PID	13-bit Packet Identifier (routes to decoder)
Payload	PES (video/audio) or PSI/SI (tables)
Null packets	PID 0x1FFF (padding for CBR)

MPEG-TS is the traditional transport, designed for broadcast TV. Each 188-byte packet carries a fragment of an elementary stream (video, audio, subtitles) identified by its PID.

Generic Stream Encapsulation (GSE)

GSE encapsulates IP packets directly into DVB-S2 baseband frames, avoiding the overhead of MPEG-TS for data services:

Feature	MPEG-TS	GSE
Overhead	~5% (headers + null packets)	~1-3%
IP encapsulation	IP -> ULE -> TS -> BBFRAME	IP -> GSE -> BBFRAME
Fragmentation	Fixed 188-byte packets	Variable-length PDUs, direct fragmentation
Best for	Broadcast TV	Satellite internet, unicast IP data

[Diagram: see online version]

Typical Deployment Parameters

Application	Standard	Frequency Band	Transponder BW	Typical Bit Rate
Satellite TV (DTH)	DVB-S2	Ku-band (10.7-12.75 GHz)	36 MHz	40-60 Mbps

HD/UHD broadcast	DVB-S2X	Ku/Ka-band	36 MHz	50-80 Mbps
VSAT (internet)	DVB-S2/S2X	Ku/Ka-band	Variable	1-50 Mbps per beam
Contribution feed	DVB-S2	C-band (3.4-4.2 GHz)	36 MHz	40-60 Mbps
SNG (Satellite News Gathering)	DVB-S2	Ku-band	9-18 MHz	5-20 Mbps
Maritime/Aero	DVB-S2X	Ku/Ka-band	Shared	1-10 Mbps

Encapsulation

[Diagram: see online version]

Standards

Document	Title
ETSI EN 300 421	DVB-S (first generation satellite)
ETSI EN 302 307-1	DVB-S2 second generation framing, coding, and modulation
ETSI EN 302 307-2	DVB-S2X extensions
ETSI TS 102 606	Generic Stream Encapsulation (GSE)
ISO/IEC 13818-1	MPEG-2 Transport Stream
ETSI EN 301 545	DVB-RCS2 (return channel via satellite)

See Also

- Ethernet -- terrestrial equivalent data link framing
- RTP -- real-time media transport (often carried over DVB-S2)

EnOcean

Standard: ISO/IEC 14543-3-10 / EnOcean Alliance | **Layer:** Full stack (Physical through Application) | **Wireshark filter:** N/A (proprietary radio; ESP3 serial protocol can be captured via serial port sniffers)

EnOcean is an energy-harvesting wireless protocol designed for batteryless sensors, switches, and actuators. Devices harvest energy from their environment -- piezoelectric generators in switches, solar cells in sensors, Peltier elements from thermal gradients -- and transmit ultra-short radio telegrams (under 1 ms) at sub-1 GHz frequencies. This makes EnOcean uniquely maintenance-free: no batteries to replace, ever. It is widely used in building automation, smart homes, and industrial monitoring where wiring and battery maintenance are impractical.

Radio Parameters

Parameter	Value
Frequency (Europe)	868.3 MHz
Frequency (North America)	902 MHz
Frequency (Japan)	928 MHz
Modulation	ASK (Amplitude Shift Keying)
Data rate	125 kbps
Telegram duration	< 1 ms
Range (indoor)	~30 m
Range (outdoor)	~300 m
Transmit power	~10 mW (10 dBm)
Channel bandwidth	280 kHz

The sub-1 GHz frequencies provide better wall penetration and range than 2.4 GHz protocols (Zigbee, BLE, Thread), while the extremely short telegram duration minimizes energy consumption and collision probability.

EnOcean Radio Protocol 1 (ERP1) Telegram

The over-the-air telegram structure for ERP1:

[Diagram: see online version]

Telegram Fields

Field	Size	Description
Preamble	9 bits	Synchronization (alternating 1/0 pattern)
Start of Frame (SOF)	4 bits	Frame delimiter
R-ORG (Radio Organization)	8 bits	Telegram type identifier
Data	Variable	Payload (1-4 bytes depending on R-ORG)
Sender ID	32 bits	Unique transmitter address (factory-programmed)
Status	8 bits	Repeater count, T21/NU flags
CRC	8 bits	CRC-8 of the telegram

R-ORG Types (Telegram Types)

R-ORG	Name	Data Size	Description

0xF6	RPS (Repeated Switch)	1 byte	Rocker switches, push buttons
0xD5	1BS (1 Byte Sensor)	1 byte	Single-contact sensors (door/window)
0xA5	4BS (4 Byte Sensor)	4 bytes	Temperature, humidity, illumination, occupancy
0xD2	VLD (Variable Length Data)	1-14 bytes	Bidirectional, multi-function actuators/sensors
0xD4	UTE (Universal Teach-In)	Variable	Teach-in for bidirectional devices

EnOcean Serial Protocol 3 (ESP3)

Gateways and USB dongles communicate with host systems using ESP3 over a serial interface (typically UART at 57600 baud):

[Diagram: see online version]

ESP3 Fields

Field	Size	Description
Sync Byte	1 byte	Always 0x55 -- frame synchronization
Data Length	2 bytes	Length of the data field
Optional Length	1 byte	Length of optional data field
Packet Type	1 byte	Type of ESP3 packet
CRC8 Header	1 byte	CRC-8 of header (Data Length + Optional Length + Packet Type)
Data	Variable	Payload (content depends on packet type)
Optional Data	Variable	Additional data (SubTelNum, RSSI, etc.)
CRC8 Data	1 byte	CRC-8 of Data + Optional Data

ESP3 Packet Types

Type	Value	Name	Description
0x01	1	Radio Telegram	ERP1 radio telegram received or to transmit
0x02	2	Response	Response to a command
0x03	3	Radio Sub Telegram	ERP2 radio telegram
0x04	4	Event	Asynchronous event from the transceiver

0x05	5	Common Command	Configuration and management commands
0x06	6	Smart Ack Command	Smart Acknowledge (mailbox) commands
0x07	7	Remote Man Command	Remote management
0x09	9	Radio Message	Extended radio telegram
0x0A	10	Radio ERP2	ERP2 protocol telegram

ESP3 Response Codes

Code	Meaning
0x00	OK -- command executed successfully
0x01	Error -- general failure
0x02	Not supported
0x03	Wrong parameter
0x04	Operation denied
0x05	Lock set (device locked)
0x82	Buffer too small
0x83	No free buffer

EnOcean Equipment Profiles (EEPs)

EEPs define the data encoding for each device type. The format is `R-ORG-Function-Type`:

EEP	Name	Description
F6-02-01	Rocker Switch, 2 Rocker	Light switch (2-button, EU style)
F6-02-02	Rocker Switch, 2 Rocker	Light switch (2-button, US style)
F6-10-00	Window Handle	Handle position (open/closed/tilted)
D5-00-01	Single Input Contact	Door/window sensor (open/closed)
A5-02-05	Temperature Sensor	Range 0-40 C, 8-bit resolution
A5-04-01	Temp + Humidity	Temperature and relative humidity
A5-06-01	Light Sensor	Illumination 300-60,000 lux
A5-07-03	Occupancy Sensor	PIR with illumination and supply voltage
A5-08-01	Light/Temp/Occupancy	Multi-function room sensor
D2-01-01	Electronic Switch	Bidirectional relay actuator
D2-05-00	Blind Control	Bidirectional blinds/shutter actuator

Energy Harvesting Sources

Source	Mechanism	Typical Application
Piezoelectric	Mechanical press generates voltage spike	Rocker switches, push buttons
Solar (photovoltaic)	Indoor light charges buffer capacitor	Temperature, humidity, illumination sensors
Thermal (Peltier)	Temperature differential generates current	Radiator valve sensors, pipe monitors
Kinetic	Motion/vibration harvesting	Machine-mounted sensors

Energy Budget

Component	Typical Energy
Single switch press (piezo)	~120 uJ harvested
Telegram transmission	~50 uJ consumed
Telegrams per press	2-3 (redundant transmissions)
Solar sensor cycle	Measure + transmit every 100 seconds
Buffer capacitor	Stores energy for operation in darkness (hours to days)

Communication Model

[Diagram: see online version]

EnOcean uses a unidirectional model for most sensors (transmit-only, no acknowledgment) to minimize energy consumption. Bidirectional communication (VLD profiles, D2-xx) is available for actuators that have mains power.

Repeaters

Level	Description
No repeater	Direct sensor-to-gateway (30 m indoor range)
1-hop repeater	Extends range to ~60 m indoor
2-hop maximum	Maximum two repeater hops per telegram (status byte tracks count)

Teach-In Process

Devices are paired with receivers through a teach-in process:

Method	Description

Push-button teach-in	Press the learn button on the sensor; gateway captures Sender ID and EEP
UTE (Universal Teach-In)	D4 telegram with full EEP information, bidirectional confirmation
Smart Ack	Mailbox-based teach-in for energy-constrained bidirectional devices
Manual	Enter Sender ID and EEP into the gateway configuration

EnOcean vs Other Wireless Protocols

Feature	EnOcean	Zigbee	Z-Wave	BLE	Thread
Frequency	Sub-1 GHz	2.4 GHz	Sub-1 GHz	2.4 GHz	2.4 GHz
Battery	None (energy harvesting)	Yes (years)	Yes (years)	Yes (months-years)	Yes (years)
Data rate	125 kbps	250 kbps	100 kbps	1 Mbps	250 kbps
Range (indoor)	30 m	10-30 m	30 m	10 m	10-30 m
Mesh	No (repeaters only)	Yes	Yes	No (scatternet)	Yes
Telegram size	Tiny (< 1 ms)	Medium	Medium	Variable	Medium
IP-based	No	No	No	No	Yes (IPv6)
Bidirectional	Optional (VLD)	Yes	Yes	Yes	Yes
Best for	Maintenance-free sensors/switches	Complex automation	Home automation	Wearables, beacons	IP-connected IoT

Standards

Document	Title
ISO/IEC 14543-3-10	EnOcean Radio Protocol
ISO/IEC 14543-3-11	EnOcean Radio Protocol (2.4 GHz)
EnOcean Alliance	EnOcean Equipment Profiles (EEP), ESP3 specification
EnOcean EEP Specification	Equipment Profile definitions

See Also

- Zigbee -- 2.4 GHz mesh protocol for home/building automation
- Z-Wave -- sub-GHz mesh protocol for smart home
- BLE -- Bluetooth Low Energy for short-range wireless
- Thread -- IPv6-based mesh for IoT
- KNX -- wired (and wireless) building automation standard

LoRa PHY -- Long Range Physical Layer

Standard: Semtech LoRa Documentation / LoRa Alliance | **Layer:** Physical (Layer 1) | **Wireshark filter:** N/A (radio layer -- LoRaWAN decoders exist: `lorawan`)

LoRa is a proprietary Chirp Spread Spectrum (CSS) radio modulation technique developed by Semtech for long-range, low-power wireless communication. It operates in unlicensed sub-GHz ISM bands and achieves links of 5-15 km (rural) or 2-5 km (urban) with milliwatt transmit power. LoRa is strictly the **physical layer** -- the radio modulation that encodes bits into chirp signals. LoRaWAN is the separate network protocol (MAC layer, device classes, join procedures) built on top of LoRa PHY. This page covers the radio modulation, framing, spreading factors, and link budget.

LoRa PHY vs LoRaWAN

[Diagram: see online version]

Aspect	LoRa PHY	LoRaWAN
Scope	Radio modulation	Network protocol
Defines	Chirp encoding, spreading factor, bandwidth, coding rate	Device classes, join, ADR, MAC commands, encryption
Analogy	Like Wi-Fi's OFDM modulation	Like the 802.11 MAC + IP stack on top
Proprietary	Yes (Semtech patent)	Open specification (LoRa Alliance)
Point-to-point	Yes (can be used standalone)	Star-of-stars topology with gateways

Chirp Spread Spectrum (CSS)

LoRa encodes data by sweeping the carrier frequency across the entire bandwidth -- an **up-chirp** sweeps from the lowest to the highest frequency, and a **down-chirp** sweeps from highest to lowest. Each symbol is a chirp whose starting frequency encodes the data value. With spreading factor SF, each symbol carries SF bits and the chirp has 2^SF frequency steps.

Property	Description
Up-chirp	Frequency increases linearly over the symbol period
Down-chirp	Frequency decreases linearly (used in preamble sync)
Symbol value	Encoded as the initial frequency offset of the chirp
Chips per symbol	2^SF (e.g., SF7 = 128 chips, SF12 = 4096 chips)
Processing gain	Higher SF = more chips = more noise resilience = longer range

Orthogonality	Different SFs are quasi-orthogonal (can coexist on the same channel)

LoRa PHY Frame

[Diagram: see online version]

Key Fields

Field	Size	Description
Preamble	6-65535 symbols (default 8)	Sequence of unmodulated up-chirps for receiver detection and synchronization
Sync Word	2 symbols	Network identifier -- distinguishes overlapping networks (0x12 = private, 0x34 = LoRaWAN)
SFD (Start Frame Delimiter)	2.25 symbols	Down-chirps marking the end of preamble and start of data
Header	20 bits + CR 4/8 FEC	Payload length, coding rate, CRC presence (explicit mode only)
Payload	1-255 bytes	Application data, encoded with the configured coding rate
Payload CRC	16 bits	CRC-16 over the payload (optional, indicated in header)

Explicit vs Implicit Header Mode

Mode	Header Present	Use Case
Explicit	Yes -- payload length, CR, CRC flag are transmitted in the header	Default mode; receiver does not need to know parameters in advance
Implicit	No -- all parameters must be pre-configured on both sides	Saves airtime; used when payload size and coding rate are fixed (e.g., beacons)

Header Fields (Explicit Mode)

[Diagram: see online version]

Field	Size	Description
Payload Length	8 bits	Number of payload bytes (1-255)
Coding Rate	3 bits	FEC coding rate used for payload
CRC Present	1 bit	Whether the payload CRC-16 is appended
Header CRC	4 bits	CRC protecting the header itself

The header is always transmitted at CR 4/8 (maximum redundancy) regardless of the payload coding rate, ensuring reliable decoding of frame parameters.

Spreading Factor (SF)

The spreading factor determines how many chips encode each data bit. Higher SF increases range and noise immunity at the cost of data rate and airtime.

SF	Chips/Symbol	Bits/Symbol	Sensitivity (125 kHz)	Approx Range (rural)	Relative Airtime
SF7	128	7	-123 dBm	2-5 km	1x (baseline)
SF8	256	8	-126 dBm	4-8 km	2x
SF9	512	9	-129 dBm	5-10 km	4x
SF10	1024	10	-132 dBm	8-13 km	8x
SF11	2048	11	-134.5 dBm	10-15 km	16x
SF12	4096	12	-137 dBm	15+ km	32x

Each step up in SF roughly doubles the airtime and halves the data rate, but gains ~2.5 dB of link budget.

Bandwidth (BW)

Bandwidth	Symbol Rate	Use Case
125 kHz	Standard	Default for most LoRa/LoRaWAN deployments; best sensitivity
250 kHz	2x of 125 kHz	Higher data rate, lower sensitivity (+3 dB noise floor)
500 kHz	4x of 125 kHz	Highest data rate, lowest sensitivity; used in US915 uplink

Symbol rate = BW / 2^SF (symbols per second)

Coding Rate (CR) -- Forward Error Correction

LoRa uses a cyclic error coding scheme for forward error correction:

Coding Rate	Notation	Overhead	Error Correction Capability
4/5	CR1	25%	Low (corrects fewer errors)
4/6	CR2	50%	Medium
4/7	CR3	75%	Medium-High
4/8	CR4	100%	High (corrects most errors, doubles airtime)

The notation "4/n" means 4 data bits are encoded into n bits total. CR 4/5 is the default for most applications; CR 4/8 is used in noisy environments or for the header.

Data Rate Table

Nominal bit rates for common SF/BW combinations (CR 4/5):

SF	BW 125 kHz	BW 250 kHz	BW 500 kHz
SF7	5,469 bps	10,938 bps	21,875 bps
SF8	3,125 bps	6,250 bps	12,500 bps
SF9	1,758 bps	3,516 bps	7,031 bps
SF10	977 bps	1,953 bps	3,906 bps
SF11	537 bps	1,074 bps	2,148 bps
SF12	293 bps	586 bps	1,172 bps

Formula: Bit Rate = SF x (BW / 2^SF) x (4 / (4 + CR))

Link Budget and Receiver Sensitivity

Parameter	Typical Value
Transmit power	+14 dBm (EU) to +20 dBm (US)
Antenna gain (node)	+2 dBi
Antenna gain (gateway)	+6 dBi
Cable/connector loss	-2 dB
Link budget (SF12, 125 kHz)	~157 dB
Link budget (SF7, 125 kHz)	~143 dB

Receiver Sensitivity by SF (125 kHz BW)

SF	Sensitivity	SNR Demodulation Limit
SF7	-123 dBm	-6 dB
SF8	-126 dBm	-9 dB
SF9	-129 dBm	-12 dB
SF10	-132 dBm	-15 dB
SF11	-134.5 dBm	-17.5 dB
SF12	-137 dBm	-20 dB

LoRa can demodulate signals well below the noise floor (negative SNR), which is the key advantage of CSS modulation.

Frequency Bands

Region	Band	Channels	Max EIRP	Duty Cycle
EU868	863-870 MHz	8+ (125 kHz)	+14 dBm	0.1%-1% (sub-band dependent)
US915	902-928 MHz	64 up (125 kHz) + 8 up (500 kHz) + 8 down	+30 dBm	No duty cycle (FCC dwell time)
AU915	915-928 MHz	Same as US915	+30 dBm	No duty cycle
AS923	920-923 MHz	8+ (varies by country)	+16 dBm	Varies
IN865	865-867 MHz	3 default	+30 dBm	No duty cycle
CN470	470-510 MHz	96 up / 48 down	+19.15 dBm	No duty cycle
EU433	433.05-434.79 MHz	3	+12.15 dBm	10%

All bands are ISM (Industrial, Scientific, Medical) -- no license required, but regional regulations on power, duty cycle, and channel access apply.

Time on Air Calculation

The time a LoRa packet occupies the channel is critical for duty cycle compliance and battery life:

```
T_symbol = 2^SF / BW (seconds)
T_preamble = (n_preamble + 4.25) x T_symbol

payloadSymbNb = 8 + max(ceil((8*PL - 4*SF + 28 + 16*CRC - 20*IH) / (4*(SF-2*DE))) * (CR+4), 0)

T_payload = payloadSymbNb x T_symbol
T_packet = T_preamble + T_payload
```

Where: PL = payload bytes, CRC = 1 if CRC enabled, IH = 1 if implicit header, DE = 1 if low data rate optimization enabled (mandatory for SF11/SF12 at 125 kHz).

Example Time on Air

Configuration	10-byte payload	50-byte payload
SF7 / 125 kHz / CR4/5	41 ms	72 ms
SF9 / 125 kHz / CR4/5	165 ms	329 ms
SF12 / 125 kHz / CR4/5	1,318 ms	2,466 ms

LoRa vs FSK

Semtech transceivers (SX1276, SX1262) support both LoRa (CSS) and traditional FSK modulation:

Feature	LoRa (CSS)	FSK (GFSK)
Modulation	Chirp Spread Spectrum	Gaussian Frequency Shift Keying
Range	15+ km	5-10 km
Sensitivity (at equivalent data rate)	-137 dBm (SF12)	-120 dBm
Sub-noise-floor demodulation	Yes (negative SNR)	No (requires positive SNR)
Max data rate	~21.9 kbps (SF7/500 kHz)	~300 kbps
Interference resilience	High (spread spectrum)	Lower
Proprietary	Yes (Semtech patent)	Open standard
Use case	Long range, low data rate	Higher data rate, shorter range

Typical Deployment

[Diagram: see online version]

Encapsulation

[Diagram: see online version]

Standards

Document	Title
Semtech SX1276 Datasheet	LoRa transceiver reference (SX1276/77/78/79)
Semtech SX1262 Datasheet	Next-gen LoRa transceiver (SX1261/62)
Semtech AN1200.22	LoRa Modulation Basics (application note)
LoRa Alliance	LoRaWAN specification and regional parameters
ETSI EN 300 220	Short Range Devices -- EU 868 MHz regulation
FCC Part 15	US 915 MHz ISM band regulation

See Also

- Zigbee -- alternative low-power wireless (2.4 GHz, mesh topology)
- BLE -- Bluetooth Low Energy (short range, 2.4 GHz)
- NFC -- near-field communication (very short range)
- MQTT -- lightweight messaging protocol commonly used with LoRa/LoRaWAN

LoRaWAN -- Long Range Wide Area Network

Standard: LoRaWAN Specification (lora-alliance.org) | **Layer:** Data Link / Network (Layers 2-3) | **Wireshark filter:** `lorawan`

LoRaWAN is a low-power wide-area network (LPWAN) protocol for connecting battery-powered IoT devices over distances of 2-15 km in urban areas and up to 50+ km in rural/line-of-sight conditions. It uses LoRa modulation (chirp spread spectrum) in sub-GHz ISM bands for extreme range at very low data rates (0.3-50 kbps). LoRaWAN is designed for sensors that send small amounts of data infrequently -- smart meters, environmental monitoring, asset tracking, agriculture, and smart city applications.

Architecture

[Diagram: see online version]

Component	Description
End Device	Sensor/actuator with LoRa radio (battery-powered)
Gateway	Receives LoRa packets, forwards to Network Server over IP
Network Server	Deduplication, routing, MAC commands, device management
Application Server	Application logic, data processing
Join Server	Manages device keys and join procedures (OTAA)

LoRaWAN is **star-of-stars** -- end devices communicate with any gateway in range, and all gateways forward to the central Network Server. There is no mesh routing.

PHY Frame (LoRa Modulation)

[Diagram: see online version]

LoRa Radio Parameters

Parameter	Value
Modulation	CSS (Chirp Spread Spectrum)
Frequency (EU)	868 MHz (EU868)
Frequency (US)	915 MHz (US915)
Frequency (others)	AS923, AU915, IN865, KR920, etc.
Bandwidth	125 kHz, 250 kHz, or 500 kHz
Spreading Factor	SF7-SF12 (higher = longer range, lower rate)
Data rate	0.3 kbps (SF12) to 50 kbps (SF7)
Range (urban)	2-5 km
Range (rural)	10-15 km

Range (LOS)	50+ km (world records >800 km)
Transmit power	14-27 dBm (region dependent)
Sensitivity	-137 dBm (SF12, 125 kHz)

Spreading Factor Trade-off

SF	Bit Rate (125 kHz BW)	Range	Air Time (10 bytes)	Battery Impact
7	5,470 bps	Shortest	36 ms	Best
8	3,125 bps	↓	72 ms	↓
9	1,760 bps	↓	123 ms	↓
10	980 bps	↓	247 ms	↓
11	440 bps	↓	495 ms	↓
12	250 bps	Longest	991 ms	Worst

The Network Server uses ADR (Adaptive Data Rate) to automatically select the optimal SF for each device.

MAC Frame

[Diagram: see online version]

MHDR (MAC Header)

[Diagram: see online version]

Message Types (MType)

MType	Name	Direction	Description
000	Join Request	Up	OTAA join request
001	Join Accept	Down	OTAA join response
010	Unconfirmed Data Up	Up	Sensor data (no ack required)
011	Unconfirmed Data Down	Down	Command (no ack required)
100	Confirmed Data Up	Up	Sensor data (ack required)
101	Confirmed Data Down	Down	Command (ack required)
110	Rejoin Request	Up	Re-join for key refresh

Data Frame (MAC Payload)

[Diagram: see online version]

Field	Size	Description
DevAddr	4 bytes	Device address (network-assigned)
FCtrl	1 byte	ADR, ACK, FPending, FOptsLen
FCnt	2 bytes	Frame counter (anti-replay)
FOpts	0-15 bytes	MAC commands (piggy-backed on data)
FPort	1 byte	Application port (0 = MAC commands, 1-223 = app)
FRMPayload	Variable	Encrypted application data

Device Classes

Class	Receive Windows	Latency	Power	Use Case
A (All)	2 short windows after each uplink	High (must wait for uplink)	Lowest	Sensors, meters
B (Beacon)	Scheduled ping slots (beacon-synchronized)	Medium (~128s max)	Medium	Actuators needing periodic downlink
C (Continuous)	Always listening (except when transmitting)	Low (~seconds)	Highest	Mains-powered actuators, streetlights

Class A Receive Windows

[Diagram: see online version]

Security

LoRaWAN uses AES-128 encryption with two separate keys:

Key	Name	Encrypts	Purpose
NwkSKey	Network Session Key	MAC commands, MIC computation	Network integrity and routing
AppSKey	Application Session Key	FRMPayload	End-to-end application data encryption

The Network Server sees MAC commands but **cannot read application data**. The Application Server can read data but cannot forge MAC commands.

Activation Methods

Method	Description
OTAA (Over-The-Air Activation)	Device sends JoinRequest with DevEUI + AppKey; server returns session keys. **Recommended.**

ABP (Activation By Personalization)	Keys pre-provisioned on device. No join procedure. Less secure.

OTAA Join Flow

[Diagram: see online version]

Duty Cycle Limitations (EU868)

European regulations limit air time:

Sub-band	Frequency	Max Duty Cycle	Effective
g (default)	868.1-868.5 MHz	1%	~36s/hour
g1	868.7-869.2 MHz	0.1%	~3.6s/hour
g2	869.4-869.65 MHz	10%	~360s/hour

This limits how often a device can transmit -- typically every few minutes to once per hour depending on payload size and SF.

LoRaWAN vs Other LPWAN

Feature	LoRaWAN	Sigfox	NB-IoT	LTE-M
Spectrum	Unlicensed ISM	Unlicensed ISM	Licensed (cellular)	Licensed (cellular)
Range	2-15 km	3-50 km	1-10 km	1-10 km
Data rate	0.3-50 kbps	100-600 bps	250 kbps	1 Mbps
Payload	51-222 bytes	12 bytes up, 8 down	1600 bytes	1600 bytes
Power	Very low	Very low	Low	Low-medium
Bidirectional	Yes (class A/B/C)	Limited (4 DL/day)	Yes	Yes
Private network	Yes (own gateways)	No (operator only)	No (operator only)	No
Cost per device	Low (commodity radio)	Low	Medium (cellular modem)	Medium

Standards

Document	Title
LoRaWAN 1.0.4	LoRaWAN L2 specification
LoRaWAN 1.1	LoRaWAN 1.1 (roaming, key management improvements)
Regional Parameters	Regional radio parameters (EU868, US915, etc.)
Semtech LoRa	LoRa PHY modulation (proprietary to Semtech)

See Also

- 6LoWPAN -- IPv6 adaptation for constrained networks (Thread uses it, LoRaWAN does not)
- Zigbee -- short-range mesh alternative
- Thread -- IP-based mesh for smart home
- MQTT -- common application protocol for LoRaWAN payloads
- CoAP -- lightweight REST for constrained devices

Matter

Standard: CSA Matter Specification | **Layer:** Application (Layer 7, runs over IP) | **Wireshark filter:** `matter` (limited; Matter traffic is encrypted)

Matter (formerly Project CHIP -- Connected Home over IP) is an open-source, royalty-free smart home interoperability standard developed by the Connectivity Standards Alliance (CSA, formerly Zigbee Alliance) with backing from Apple, Google, Amazon, and Samsung. Matter provides a unified application layer that runs over existing IP networks -- Wi-Fi, Thread (802.15.4), and Ethernet -- with BLE used for device commissioning. Any Matter-certified device from any manufacturer works with any Matter-certified controller, solving the fragmentation problem that plagued Zigbee, Z-Wave, and proprietary ecosystems.

Protocol Stack

[Diagram: see online version]

Matter Message Frame

All Matter messages share a common framing structure. Messages are encrypted after session establishment.

Message Header

[Diagram: see online version]

Protocol Header (inside encrypted payload)

[Diagram: see online version]

Key Fields

Field	Size	Description

Flags	8 bits	Version (4 bits), source/dest node ID presence flags
Session ID	16 bits	Identifies the secure session (CASE or PASE)
Security Flags	8 bits	Session type, privacy flag, message extensions
Message Counter	32 bits	Anti-replay counter, incremented per message
Source Node ID	0 or 64 bits	Sender's operational node ID (omitted in group messages)
Dest Node ID	0 or 64 bits	Recipient's node ID or group ID
Exchange Flags	8 bits	Initiator flag, acknowledgment requested, reliability flags
Protocol Opcode	8 bits	Operation within the protocol (Read, Write, Report, etc.)
Exchange ID	16 bits	Correlates request-response pairs
Protocol ID	16 bits	Identifies the protocol (Interaction Model, BDX, etc.)

Message Security Flags

Bit	Name	Description
0-1	Session Type	0 = Unicast, 1 = Group
2	Privacy	Message header is encrypted (Thread privacy)
3-7	Reserved	Must be zero

Interaction Model

The Interaction Model defines how controllers (clients) communicate with devices (servers):

Action	Direction	Description
Read	Client -> Server	Read one or more attribute values
Write	Client -> Server	Set attribute values on the device
Subscribe	Client -> Server	Register for periodic attribute reports
Invoke	Client -> Server	Execute a command (e.g., toggle, lock)
Report	Server -> Client	Deliver attribute data (response to Read or subscription update)

Interaction Flow -- Subscribe

[Diagram: see online version]

Interaction Flow -- Invoke (Command)

[Diagram: see online version]

Data Model -- Clusters

Matter uses a cluster-based data model inherited from Zigbee's ZCL. Each device type is composed of required and optional clusters:

Common Clusters

Cluster	ID	Description
On/Off	0x0006	Binary switch (on, off, toggle)
Level Control	0x0008	Brightness, volume, or other continuous control
Color Control	0x0300	Hue, saturation, color temperature
Temperature Measurement	0x0402	Temperature sensor readings
Humidity Measurement	0x0405	Relative humidity
Occupancy Sensing	0x0406	Motion / presence detection
Door Lock	0x0101	Lock, unlock, set PIN codes
Thermostat	0x0201	HVAC setpoint and mode control
Fan Control	0x0202	Fan speed and mode
Window Covering	0x0102	Blinds, shades, shutters
Descriptor	0x001D	Lists endpoints, device types, clusters on a node
Basic Information	0x0028	Vendor, product, serial number, software version
OTA Software Update	0x002A	Over-the-air firmware updates
Network Commissioning	0x0031	Wi-Fi/Thread network joining

Device Types

Device Type	ID	Required Clusters
On/Off Light	0x0100	On/Off, Level Control
Dimmable Light	0x0101	On/Off, Level Control
Color Temperature Light	0x010C	On/Off, Level Control, Color Control
Contact Sensor	0x0015	Boolean State

Temperature Sensor	0x0302	Temperature Measurement
Door Lock	0x000A	Door Lock
Thermostat	0x0301	Thermostat
Window Covering	0x0202	Window Covering

Commissioning

Commissioning is the process of adding a new device to a Matter fabric. BLE is used for the initial secure channel when the device has no network:

[Diagram: see online version]

Setup Payload

Devices carry a setup code (printed or in QR code) containing:

Field	Size	Description
Version	3 bits	Payload version
Vendor ID	16 bits	Manufacturer identifier
Product ID	16 bits	Product identifier
Custom Flow	2 bits	Commissioning flow (0 = standard, 1 = user-intent, 2 = custom)
Discovery Capabilities	8 bits	BLE, Wi-Fi SoftAP, On-network
Discriminator	12 bits	Distinguishes devices of the same type
Passcode	27 bits	SPAKE2+ PIN (000000-99999999, excluding reserved values)

Security

Mechanism	Protocol	Description
PASE	SPAKE2+	Passcode-based session during commissioning (setup code)
CASE	Sigma (3-pass)	Certificate-based session for operational communication
Encryption	AES-128-CCM	All messages encrypted after session establishment
Message Integrity	AES-128-CCM	Authentication tag on every message
Anti-Replay	Message Counter	Monotonically increasing counter per session
Node Certificates	X.509 (NOC)	Each node receives a Node Operational Certificate from the fabric CA

Group Security	Group Key (epoch keys)	Multicast messages use shared symmetric keys

CASE Session Establishment

[Diagram: see online version]

Multi-Admin (Multi-Fabric)

A single Matter device can be commissioned into multiple fabrics simultaneously -- for example, both Apple Home and Google Home can control the same light. Each fabric has its own certificate authority and operational credentials.

Network Topology

[Diagram: see online version]

Matter vs Zigbee vs Z-Wave

Feature	Matter	Zigbee	Z-Wave
Transport	IP (Wi-Fi, Thread, Ethernet)	IEEE 802.15.4	ITU-T G.9959 (sub-GHz)
Network layer	IPv6	Zigbee NWK	Z-Wave NWK
Application model	Clusters (ZCL-derived)	ZCL clusters	Z-Wave command classes
Encryption	AES-128-CCM per session	AES-128 network/link key	S2 (AES-128-CCM)
Max nodes	Fabric-dependent (1000s)	65,000	232
Multi-vendor	Yes (by design)	Partial	Partial
Multi-ecosystem	Yes (multi-admin)	No	No
Commissioning	BLE + QR code	Install codes	SmartStart QR
IP-based	Yes	No	No
Mesh support	Via Thread	Built-in	Built-in

Standards

Document	Title
Matter 1.0 Specification	Core Specification (CSA members, public summary available)
Matter 1.3 Specification	Latest release with energy management, EV charging
connectedhomeip (GitHub)	Open-source Matter SDK
Thread 1.3	IPv6 mesh networking for Matter
RFC 9202	DTLS/TLS Profiles for the Internet of Things

See Also

- Zigbee -- predecessor, Matter's cluster model is derived from Zigbee ZCL
- BLE -- used for Matter device commissioning
- NFC -- alternative commissioning path (NFC tag with setup payload)
- Bluetooth -- BLE transport for commissioning phase

NFC -- Near Field Communication

Standard: ISO/IEC 18092 (NFCIP-1) / NFC Forum Specifications | **Layer:** Physical / Data Link | **Wireshark filter:** N/A (specialized NFC capture hardware)

NFC is a set of short-range wireless protocols operating at 13.56 MHz for communication between devices within approximately 4 cm. It evolved from RFID technology and enables contactless payments (Apple Pay, Google Pay), transit cards, access badges, Bluetooth/Wi-Fi pairing, and data exchange between phones. NFC supports three modes: reader/writer (tag reading), peer-to-peer (device-to-device), and card emulation (phone acts as a contactless card).

Operating Modes

Mode	Description	Typical Use
Reader/Writer	Active device reads/writes a passive NFC tag	Reading product info, smart posters, Amiibo
Peer-to-Peer	Two active devices exchange data	Android Beam (deprecated), handover to Bluetooth/Wi-Fi
Card Emulation	Device emulates a contactless smart card	Mobile payments, transit cards, access badges

Radio Parameters

Parameter	Value
Frequency	13.56 MHz (ISM band)
Range	<= 4 cm (practical), 10 cm (theoretical)
Data rates	106, 212, 424, 848 kbps
Modulation	ASK (Type A), Load modulation (Type B, Type F)
Coupling	Inductive (magnetic field)
Power	Passive tags powered by the reader's RF field
Duplex	Half-duplex

NFC Communication Types

Type	Standard	Modulation	Rate	Origin
NFC-A	ISO 14443A	Modified Miller / Manchester	106 kbps	Philips (NXP) MIFARE
NFC-B	ISO 14443B	NRZ / BPSK	106 kbps	Various manufacturers
NFC-F	JIS X 6319-4 (FeliCa)	Manchester	212/424 kbps	Sony FeliCa
NFC-V	ISO 15693	ASK / FSK	26.48 kbps	Vicinity cards

NFC-A (ISO 14443A) Anti-Collision

[Diagram: see online version]

NFC Tag Types (NFC Forum)

Type	Standard	Memory	Data Rate	Examples
Type 1	ISO 14443A (Topaz)	96-2048 bytes	106 kbps	Broadcom Topaz
Type 2	ISO 14443A	48-2048 bytes	106 kbps	NXP NTAG213/215/216
Type 3	JIS X 6319-4 (FeliCa)	Variable	212/424 kbps	Sony FeliCa Lite
Type 4	ISO 14443A/B	Up to 32 KB	106-424 kbps	NXP DESFire
Type 5	ISO 15693	Variable	26.48 kbps	NXP ICODE SLIX

NDEF (NFC Data Exchange Format)

NDEF is the standard message format for NFC data exchange. An NDEF message contains one or more records:

NDEF Record

[Diagram: see online version]

Field	Description
MB	Message Begin -- first record in the message
ME	Message End -- last record in the message
CF	Chunk Flag -- payload is chunked
SR	Short Record -- payload length is 1 byte (<= 255)
IL	ID Length present
TNF	Type Name Format (3 bits)

Type Name Format (TNF)

TNF	Name	Description
0x00	Empty	Empty record
0x01	NFC Forum well-known type	URI, Text, Smart Poster, etc.
0x02	Media type (RFC 2046)	MIME type (e.g., `image/png`)
0x03	Absolute URI	URI as the type
0x04	NFC Forum external type	Custom types (`example.com:myapp`)
0x05	Unknown	Payload type unknown

Common Well-Known Types

Type	Record Type	Description
`U`	URI	URL -- web address, phone number, email
`T`	Text	Human-readable text with language code
`Sp`	Smart Poster	URI + title + icon + action
`Hc`	Handover Carrier	Bluetooth/Wi-Fi handover parameters
`Hr`	Handover Request	Request for connection handover
`Hs`	Handover Select	Selected handover method
`ac`	Alternative Carrier	Carrier description for handover

URI Identifier Codes

The URI record compresses common prefixes:

Code	Prefix
0x01	`http://www.`
0x02	`https://www.`
0x03	`http://`
0x04	`https://`
0x05	`tel:`
0x06	`mailto:`

Card Emulation (Payments)

Mobile payments use Host Card Emulation (HCE) or Secure Element (SE):

[Diagram: see online version]

Payment cards use ISO 7816-4 APDU commands over the contactless interface, following EMV contactless specifications.

Standards

Document	Title
ISO/IEC 18092	NFC Interface and Protocol (NFCIP-1)
ISO/IEC 14443	Contactless Smart Cards (Type A and B)
ISO/IEC 15693	Vicinity Cards
NFC Forum NDEF	NFC Data Exchange Format
NFC Forum Tag Types	Tag Type Technical Specifications
EMV Contactless	EMV Contactless payment specifications

See Also

- Bluetooth -- NFC often used to initiate Bluetooth pairing
- I2C -- NFC controller ICs connect to host MCU via I2C/SPI

TETRA -- Terrestrial Trunked Radio

Standard: ETSI EN 300 392 series | **Layer:** Full stack (Physical through Application) | **Wireshark filter:** `tetra`

TETRA is a digital trunked radio standard developed by ETSI, designed primarily for public safety and emergency services (police, fire, ambulance), as well as transport, utilities, and military. It provides group calls, individual calls, direct mode operation (without infrastructure), emergency pre-emptive calling, short data messaging, and encryption -- capabilities essential for mission-critical communications. TETRA uses TDMA with 4 time slots per 25 kHz carrier, and is deployed in over 130 countries. It is the dominant PMR technology for European emergency services and is widely used in Asia, the Middle East, and Africa.

TDMA Frame Structure

TETRA divides a 25 kHz carrier into 4 time slots, each 14.167 ms, within a 56.67 ms TDMA frame:

[Diagram: see online version]

Multiframe Structure

Level	Composition	Duration

Time Slot	1 of 4 per frame	14.167 ms
TDMA Frame	4 time slots	56.67 ms
Multiframe	18 TDMA frames	1.02 s
Hyperframe	60 multiframes	61.2 s

Frame 18 of each multiframe is used for control signaling (AACH, STCH, SCH), while frames 1-17 carry traffic and control.

Modulation and RF

Parameter	Value
Modulation	pi/4-DQPSK
Symbol rate	18,000 symbols/s
Bit rate	36,000 bps per carrier
Bit rate per slot	9,000 bps per time slot (gross)
Channel spacing	25 kHz
Frequency bands	380-400 MHz (emergency), 410-430 MHz, 450-470 MHz, 870-876 / 915-921 MHz

Burst Structure

Each time slot contains a burst of 510 bits:

[Diagram: see online version]

Field	Size	Description
Tail + Ramp-up	30 bits	Power ramp-up and known bits for synchronization
Block 1	216 bits	First half of coded data (speech, data, or signaling)
Training Sequence	34 bits	Channel estimation and synchronization
Block 2	216 bits	Second half of coded data
Tail + Guard	12 bits	Ramp-down and guard time between slots

Logical Channels

Traffic Channels

Channel	Description

TCH/S	Speech traffic channel (full rate, 7.2 kbps net)
TCH/7.2	Unprotected data channel (7.2 kbps)
TCH/4.8	Low-protection data channel (4.8 kbps net)
TCH/2.4	High-protection data channel (2.4 kbps net)
TCH/S+D	Speech + low-rate data multiplexed

Control Channels

Channel	Description
BCCH	Broadcast Control Channel -- system information, frequencies, LA
BNCH	Broadcast Network Channel -- neighbor cell information
SCH/F	Signaling Channel Full -- call setup, registration, mobility
SCH/H	Signaling Channel Half -- shared signaling
STCH	Stealing Channel -- urgent signaling stealing a traffic slot
AACH	Access Assignment Channel -- slot usage indicator (per frame)
BSCH	Broadcast Synchronization Channel -- time sync, color code

Voice Codec

Parameter	Value
Codec	ACELP (Algebraic Code-Excited Linear Prediction)
Bit rate	4,567 bps (net speech)
FEC overhead	Results in 7,200 bps gross
Frame duration	30 ms (speech), 4 frames per 120 ms
Audio bandwidth	~300-3400 Hz

TETRA speech quality is generally rated as equivalent to or slightly below GSM, but is optimized for intelligibility in noisy environments.

Trunking Architecture

TETRA uses automatic trunking managed by the SwMI (Switching and Management Infrastructure):

[Diagram: see online version]

The SwMI automatically assigns free channels on demand -- users do not select frequencies manually. This is the core trunking advantage: a pool of channels serves a large number of users efficiently.

Services

Call Types

Service	Description
Group Call	One-to-many push-to-talk within a talkgroup
Individual Call	Private one-to-one call (half or full duplex)
Broadcast Call	One-to-all announcement (no response)
Emergency Call	Pre-emptive priority call -- overrides all other traffic
Ambience Listening	Covert remote activation of microphone
Include Call	Adding a party to an ongoing group call

Data Services

Service	Description
SDS (Short Data Service)	Up to 2047 bits per message (similar to SMS)
SDS-TL	Transport layer SDS with delivery confirmation
Status Messages	Pre-coded 16-bit status values (e.g., "on scene", "available")
IP Packet Data	Multi-slot packet data for IP connectivity
WAP	Wireless application browsing (legacy)

Group Call Setup

[Diagram: see online version]

Emergency Call

[Diagram: see online version]

Emergency calls have the highest priority (level 15) and will pre-empt any ongoing non-emergency call. Pressing the emergency button also typically enables GPS reporting and ambient listening.

Direct Mode Operation (DMO)

DMO allows TETRA radios to communicate directly without infrastructure, essential when out of coverage:

Feature	Description
MS-MS Direct	Radio to radio on a pre-assigned DMO frequency
DMO Repeater	A radio acting as a simplex repeater to extend range

DMO Gateway	A radio bridging DMO and trunked mode (infrastructure relay)
Communication range	Typically 1-5 km (handheld), up to 15 km (mobile/vehicular)

Encryption

Algorithm	Designation	Description
TEA1	TETRA Encryption Algorithm 1	Export-grade, for commercial use
TEA2	TETRA Encryption Algorithm 2	European public safety (restricted distribution)
TEA3	TETRA Encryption Algorithm 3	Non-European public safety
TEA4-TEA7	Reserved	Future algorithms
E2EE	End-to-End Encryption	Additional layer over air interface encryption

Note: TEA1 was found to have a deliberate weakness (reduced key space) that was disclosed in 2023. TEA2 remains considered secure for its intended use.

Security Layers

Layer	Function
Authentication	Mutual authentication between MS and SwMI (challenge-response)
Air Interface Encryption	TEA1/2/3 encrypt traffic on the radio link
End-to-End Encryption	Optional AES-256 or national algorithms from terminal to terminal
Enable/Disable	Remote kill/stun of stolen radios
OTAR	Over-the-Air Rekeying for key management

TETRA Enhanced Data Service (TEDS)

TEDS extends TETRA with wider bandwidth channels and higher-order modulation:

Channel Width	Modulation Options	Peak Data Rate
25 kHz	pi/4-DQPSK, pi/8-D8PSK, 4-QAM, 16-QAM, 64-QAM	Up to 115 kbps
50 kHz	Same	Up to 230 kbps
100 kHz	Same	Up to 461 kbps
150 kHz	Same	Up to 691 kbps

ISSI (Inter-System Interface)

ISSI connects separate TETRA networks (e.g., police and fire departments on different systems) to enable cross-network group calls and individual calls. It uses IP-based signaling and voice transport between SwMI nodes.

Standards

Document	Title
ETSI EN 300 392-1	General network design
ETSI EN 300 392-2	Air Interface (AI)
ETSI EN 300 392-3	Interworking at the ISI
ETSI EN 300 392-7	Security
ETSI EN 300 392-12	Supplementary services
ETSI EN 300 396-3	Direct Mode Operation (DMO)
ETSI TS 100 392-18	Air interface optimized codec (ACELP)
ETSI EN 300 394-1	TETRA Enhanced Data Service (TEDS)

See Also

- DMR -- 2-slot TDMA digital radio (simpler, lower cost)
- Bluetooth -- short-range wireless for comparison
- GSM -- cellular network architecture comparison

Thread

Standard: Thread Specification (threadgroup.org) | **Layer:** Network / Mesh (Layers 2-3) | **Wireshark filter:** `wpan` or `6lowpan` or `mle`

Thread is an IPv6-based mesh networking protocol for IoT and smart home devices. Built on IEEE 802.15.4 (same radio as Zigbee) with 6LoWPAN for IPv6 adaptation, Thread provides a reliable, self-healing mesh network where every device has a standard IPv6 address -- no proprietary gateways or application-layer translation needed. Thread is the mesh transport for **Matter** (the unified smart home standard backed by Apple, Google, Amazon, and Samsung).

Protocol Stack

[Diagram: see online version]

Unlike Zigbee (which uses a proprietary network layer), Thread uses standard IPv6/UDP end-to-end. Any IPv6 application protocol (CoAP, DTLS, DNS, mDNS) works natively.

Key Concepts

Concept	Description
IP-native	Every Thread device has a routable IPv6 address -- no gateway translation
Mesh	Self-healing mesh; devices route for each other automatically
No single point of failure	Leader role migrates; any router can become leader
Border Router	Bridges Thread mesh to Wi-Fi/Ethernet (home network + Internet)
Low power	Sleepy End Devices wake periodically; battery life measured in years
Matter	The smart home standard that runs over Thread (and Wi-Fi)

Device Roles

Role	Description	Power	Routes
Leader	Manages router IDs, partition, network data	Mains	Yes
Router	Forwards messages, keeps neighbor table	Mains	Yes
REED (Router-Eligible End Device)	Can become a router if needed	Mains	No (until promoted)
SED (Sleepy End Device)	Sleeps, polls parent for messages	Battery	No
SSED (Synchronized Sleepy)	Tighter sync with parent for lower latency	Battery	No
MED (Minimal End Device)	Always-on end device, doesn't route	Mains	No
Border Router	Connects Thread mesh to external IP networks	Mains	Yes + bridge

Network Topology

[Diagram: see online version]

The mesh self-heals -- if R1 fails, R3 finds an alternate route through R2 or Leader.

Radio Parameters

Parameter	Value

Radio	IEEE 802.15.4
Frequency	2.4 GHz (channels 11-26)
Data rate	250 kbps
Modulation	O-QPSK with DSSS
Range	10-30m indoor per hop
Max hops	~32 (practical mesh depth)
Max devices	250+ routers, thousands of end devices (per partition)
Channel	Single channel per network

Addressing

Thread devices have multiple IPv6 addresses:

Address Type	Scope	Example	Description
Link-Local	Link	`fe80::...`	Auto-generated from MAC, single-hop only
Mesh-Local (RLOC)	Mesh	`fd00::ff:fe00:RLOC16`	Routing Locator -- changes with topology
Mesh-Local (ML-EID)	Mesh	`fd00::random`	Stable mesh-local endpoint ID
Global Unicast	Global	`2001:db8::...`	Internet-routable (via Border Router, SLAAC or DHCPv6)

RLOC16 (Routing Locator)

[Diagram: see online version]

The RLOC16 encodes the device's position in the mesh hierarchy -- which router it's attached to and its child index.

Mesh Link Establishment (MLE)

MLE is Thread's protocol for forming and maintaining mesh links:

Message	Description
Link Request	Request to form a link with a neighbor
Link Accept	Accept a link request
Link Accept and Request	Accept + request in one message
Advertisement	Periodic broadcast of routing info
Parent Request	End device looking for a parent router
Parent Response	Router offering to be a parent

Child ID Request	End device requesting attachment
Child ID Response	Router assigns a child ID
Data Request	Sleepy device polls for pending messages
Data Response	Router delivers buffered messages

Joining a Thread Network

[Diagram: see online version]

Security

Layer	Mechanism
802.15.4 MAC	AES-128-CCM (per-link encryption, network-wide key)
MLE	AES-128-CCM (mesh key)
Application	DTLS 1.2 (per-session encryption, e.g., for Matter/CoAP)
Commissioning	DTLS with JPAKE (password-authenticated key exchange)

Thread Network Key

All devices in a Thread network share a **Network Master Key** (128-bit AES). This key encrypts all 802.15.4 frames at the MAC layer. It is distributed during commissioning.

Thread vs Zigbee vs Z-Wave vs Wi-Fi

Feature	Thread	Zigbee	Z-Wave	Wi-Fi
IP-based	**Yes (IPv6 native)**	No (proprietary NWK)	No	Yes
Mesh	Yes	Yes	Yes (4 hops)	802.11s only
Radio	802.15.4 (2.4 GHz)	802.15.4 (2.4 GHz)	Sub-GHz	2.4/5/6 GHz
Data rate	250 kbps	250 kbps	100 kbps	Mbps-Gbps
Power	Low (battery years)	Low	Low	High (mains)
Application layer	Any (CoAP, Matter)	Zigbee Cluster Library	Z-Wave command classes	Any
Interop standard	**Matter**	Via bridge	Via bridge	**Matter**
Gateway needed	Border Router (for Internet)	Zigbee gateway (for IP)	Z-Wave hub (for IP)	Wi-Fi router

Thread and Matter

Matter is the smart home application protocol. It can run over **Thread** (for low-power battery devices) or **Wi-Fi** (for high-bandwidth devices):

[Diagram: see online version]

Standards

Document	Title
Thread 1.3 Specification	Thread Networking Protocol
IEEE 802.15.4	Low-Rate Wireless PANs
RFC 4944	6LoWPAN (IPv6 over 802.15.4)
RFC 6775	6LoWPAN Neighbor Discovery
Matter Specification	Matter application layer

See Also

- 6LoWPAN -- IPv6 adaptation layer Thread uses
- Zigbee -- alternative 802.15.4 mesh (non-IP)
- Z-Wave -- alternative home automation mesh (sub-GHz)
- CoAP -- constrained REST protocol used over Thread
- BLE -- Bluetooth Low Energy (Matter also uses BLE for commissioning)
- 802.11 (Wi-Fi) -- Matter's other transport

UWB -- Ultra-Wideband

Standard: IEEE 802.15.4z-2020 / FiRa Consortium | **Layer:** Physical / Data Link | **Wireshark filter:** `wpan` (IEEE 802.15.4 dissector; UWB-specific fields require specialized capture)

Ultra-Wideband (UWB) is a short-range radio technology that uses very wide bandwidth pulses (>500 MHz) to achieve centimeter-level positioning accuracy and secure ranging between devices. Unlike Bluetooth or Wi-Fi, UWB's primary purpose is not data transfer but precise distance measurement -- enabled by sub-nanosecond pulse timing. IEEE 802.15.4z (2020) added enhanced ranging with anti-spoofing protections (Scrambled Timestamp Sequence), making UWB suitable for security-sensitive applications: digital car keys, access control, asset tracking (Apple AirTag, Samsung SmartTag+), and indoor positioning systems.

Radio Parameters

Parameter	Value
Frequency bands	3.1-4.8 GHz (low band), 6.0-10.6 GHz (high band)

Common channels	Channel 5 (6.5 GHz), Channel 9 (7.9 GHz)
Bandwidth	>500 MHz per channel
Data rates	110 kbps, 850 kbps, 6.8 Mbps, 27.2 Mbps
Modulation	BPM-BPSK (Burst Position Modulation with BPSK)
Pulse duration	~2 ns
Transmit power	-41.3 dBm/MHz (FCC limit, very low)
Range	Typically 10-30 m (indoor), up to 200 m (line of sight)
Ranging accuracy	5-30 cm (typical), <10 cm (ideal conditions)
Time resolution	~65 ps (enables cm-level distance calculation)

PHY Frame Structure (HRP UWB)

The IEEE 802.15.4z HRP (High Rate Pulse) PHY frame:

[Diagram: see online version]

SHR (Synchronous Header)

[Diagram: see online version]

PHR (PHY Header)

[Diagram: see online version]

Key Fields

Field	Size	Description
SYNC Preamble	16-4096 symbols	Preamble code for receiver synchronization and channel estimation
SFD	8-64 symbols	Start-of-Frame Delimiter marks end of preamble
Data Rate	2 bits	00=110 kbps, 01=850 kbps, 10=6.8 Mbps, 11=27.2 Mbps
Frame Length	11 bits	Payload length in octets
Ranging	1 bit	1 = frame is used for ranging (timestamp captured)
SECDED	5 bits	Single Error Correct, Double Error Detect parity over PHR
Payload	Variable	MAC frame data (IEEE 802.15.4 MAC)
STS	Variable	Scrambled Timestamp Sequence (802.15.4z security)

PHY Modes

IEEE 802.15.4z defines two PHY modes:

Mode	Pulse Rate	Data Rate	Range	Use Case
HRP (High Rate Pulse)	15.6 / 62.4 / 124.8 MHz	850 kbps - 27.2 Mbps	10-100 m	Smartphones, car keys, tracking tags
LRP (Low Rate Pulse)	1-4 MHz	~50-250 kbps	Up to 200 m	Access control readers, industrial

HRP vs LRP

Feature	HRP	LRP
Peak pulse rate	Up to 124.8 MHz	1-4 MHz
Preamble codes	TERNARY (31/127 length)	Binary
Complexity	Higher (smartphone SoC)	Lower (simple readers)
FiRa support	Primary mode	Supplementary
Power consumption	Higher	Lower
Common devices	iPhone, Galaxy, AirTag	Door locks, access readers

Ranging Methods

Two-Way Ranging (TWR)

TWR measures round-trip time of flight between two devices to calculate distance:

[Diagram: see online version]

Distance = speed of light * (round-trip time - processing delay) / 2. At ~3.3 ns per meter, sub-nanosecond timing resolution yields centimeter-level accuracy.

Double-Sided TWR (DS-TWR)

DS-TWR adds a third exchange to cancel out clock drift errors:

[Diagram: see online version]

Time Difference of Arrival (TDoA)

TDoA uses multiple synchronized anchors -- the tag only transmits, anchors compute position from arrival time differences:

[Diagram: see online version]

TDoA is more scalable than TWR for large deployments (warehouses, stadiums) because the tag does not need to participate in two-way exchanges with each anchor.

Scrambled Timestamp Sequence (STS)

STS is the key security addition in IEEE 802.15.4z. It prevents relay attacks and distance spoofing:

Feature	Description
Purpose	Secure the time-of-arrival measurement against manipulation
Mechanism	Cryptographically generated pseudo-random pulse sequence appended to the frame
Key derivation	STS key derived from a shared secret between initiator and responder
Verification	Receiver correlates incoming STS against expected sequence
Attack prevention	Relay attacks, early-detect / late-commit, distance reduction

STS Modes (802.15.4z)

Mode	STS Position	Description
SP0	No STS	Legacy mode, no secure ranging
SP1	After payload	STS appended after the MAC payload
SP2	Before payload	STS placed between SFD and payload
SP3	No payload	STS only (ranging-only frames, no data)

Frame with STS (SP1)

[Diagram: see online version]

FiRa Consortium Profiles

The FiRa Consortium defines interoperability profiles on top of IEEE 802.15.4z:

Profile	Description	Example Applications
PACS (Physical Access Control)	Secure door/gate access	Office badges, hotel rooms
Peer-to-Peer Ranging	Device-to-device distance	Find My (AirTag), nearby sharing
Device Tracking	Tag-to-infrastructure ranging	Warehouse inventory, luggage tracking
Secure Ranging	High-security distance bounding	Digital car keys (CCC), payment terminals

FiRa Ranging Session

[Diagram: see online version]

Use Cases

Application	Method	Accuracy	Devices
Apple AirTag / Find My	TWR + TDoA	~10 cm	iPhone 11+, AirTag
Digital car keys (CCC)	DS-TWR + STS	~10 cm	BMW, Hyundai, NXP
Indoor positioning (RTLS)	TDoA	10-30 cm	Warehouses, hospitals
Smart home access	TWR + STS	~10 cm	Door locks, lights
AR/VR spatial tracking	TWR	~5 cm	Headsets, controllers
Contactless payments	Secure ranging	~30 cm	Payment terminals
Robotic navigation	TWR / TDoA	~5 cm	Autonomous vehicles, drones

UWB vs Bluetooth vs NFC

Feature	UWB	Bluetooth (BLE)	NFC
Primary purpose	Ranging / positioning	Data transfer, audio	Tap interaction
Range	10-200 m	10-100 m	<4 cm
Accuracy	5-30 cm	1-3 m (RSSI), ~50 cm (BLE 6.0 CS)	N/A
Data rate	6.8-27.2 Mbps	1-2 Mbps	106-424 kbps
Anti-relay	STS (802.15.4z)	Limited	Physical proximity
Power	Moderate	Low	Passive (tags)
Frequency	3.1-10.6 GHz	2.4 GHz	13.56 MHz
Spatial awareness	Yes (angle + distance)	Limited (BLE 5.1 AoA)	No

Encapsulation

[Diagram: see online version]

Standards

Document	Title
IEEE 802.15.4z-2020	Enhanced HRP and LRP UWB PHY
IEEE 802.15.4-2020	Low-Rate Wireless PAN (base MAC/PHY)

FiRa Consortium	UWB interoperability profiles and certifications
Car Connectivity Consortium (CCC)	Digital Key 3.0 (UWB for car access)
Apple Nearby Interaction	iOS UWB framework (U1/U2 chip)

See Also

- BLE -- often used alongside UWB for session setup and fallback
- Bluetooth -- Bluetooth 6.0 Channel Sounding competes with UWB for ranging
- NFC -- alternative for very-short-range secure interaction
- Zigbee -- shares IEEE 802.15.4 MAC layer heritage

Zigbee

Standard: Zigbee 3.0 (CSA) / IEEE 802.15.4 | **Layer:** Full stack (Physical through Application) | **Wireshark filter:** `zbee_nwk` or `wpan`

Zigbee is a low-power, low-data-rate wireless mesh networking protocol built on the IEEE 802.15.4 physical and MAC layers. It is designed for IoT and home automation -- smart lighting (Philips Hue), sensors, switches, and industrial monitoring. Zigbee supports self-healing mesh networks with up to 65,000 nodes, AES-128 encryption, and very low power consumption (battery-powered devices can last years). Zigbee 3.0 unified earlier profiles (Home Automation, Light Link, etc.) into a single standard.

Protocol Stack

[Diagram: see online version]

IEEE 802.15.4 Frame (MAC Layer)

[Diagram: see online version]

Key Fields (MAC)

Field	Size	Description
Frame Control	16 bits	Frame type, security, addressing mode, PAN compression
Sequence Number	8 bits	Frame sequence for acknowledgment matching
Dest PAN ID	16 bits	Destination Personal Area Network ID

Dest Address	16 or 64 bits	Short (network) or extended (IEEE/MAC) address
Source PAN ID	16 bits	Source PAN ID (omitted with PAN compression)
Source Address	16 or 64 bits	Short or extended address
Payload	Variable	NWK + APS + application data
FCS	16 bits	CRC-16-CCITT

Radio Parameters

Parameter	Value
Frequency	2.4 GHz (worldwide), 915 MHz (NA), 868 MHz (EU)
Channels	16 channels at 2.4 GHz (channels 11-26)
Data rate	250 kbps (2.4 GHz)
Modulation	O-QPSK with DSSS
Range (indoor)	10-30 m
Range (outdoor)	100+ m
Max payload (MAC)	127 bytes total frame
Transmit power	Typically 0-20 dBm

Network Layer (NWK)

[Diagram: see online version]

Device Types

Type	Role	Power	Can Route
Coordinator	Forms the network, trust center	Mains	Yes
Router	Extends the mesh, relays packets	Mains	Yes
End Device	Leaf node, can sleep	Battery	No

Network Topology

[Diagram: see online version]

Zigbee supports star, tree, and mesh topologies. The mesh self-heals -- if a router fails, alternative routes are discovered.

Routing

Method	Description
AODV (Ad-hoc On-demand Distance Vector)	Default mesh routing -- routes discovered when needed
Many-to-One	Optimized for sensor networks reporting to a collector
Source Routing	Route specified by the sender (for efficiency after discovery)
Tree Routing	Hierarchical routing based on network address assignment

Application Layer (ZCL)

The Zigbee Cluster Library defines standardized clusters (functional groups):

Common Clusters

Cluster	ID	Description
On/Off	0x0006	Binary switch control
Level Control	0x0008	Dimming, volume, etc.
Color Control	0x0300	Hue, saturation, color temperature
Temperature Measurement	0x0402	Temperature sensor
Humidity Measurement	0x0405	Humidity sensor
Occupancy Sensing	0x0406	Motion/presence detection
IAS Zone	0x0500	Intrusion Alarm System (door/window, smoke)
Door Lock	0x0101	Lock/unlock control
Thermostat	0x0201	HVAC control
Electrical Measurement	0x0B04	Power monitoring
OTA Upgrade	0x0019	Over-the-air firmware update

ZCL Frame

[Diagram: see online version]

Frame Types

Type	Description
Global	Standard commands (read/write attribute, report, discover)
Cluster-specific	Commands defined by each cluster (on, off, move, step)

Security

Layer	Mechanism
Network	AES-128-CCM encryption with network key (all devices share)
Application	AES-128-CCM with link key (per device pair)
Trust Center	Coordinator manages key distribution and device admission
Install Codes	Pre-shared key derived from device-specific code (Zigbee 3.0)

Security Levels

Level	Description
No Security	Unencrypted (not recommended)
Network Key	All devices share one key -- protects against external attackers
Link Key	Per-pair keys -- protects against compromised network devices

Zigbee vs Z-Wave vs Thread

Feature	Zigbee	Z-Wave	Thread
Frequency	2.4 GHz	Sub-GHz	2.4 GHz
Data rate	250 kbps	100 kbps	250 kbps
Range	10-30 m	30 m	10-30 m
Max nodes	65,000	232	250+
Mesh	Yes	Yes (4 hops)	Yes
IP-based	No	No	Yes (IPv6/6LoWPAN)
Interoperability	Via Matter bridge	Via Matter bridge	Native Matter

Standards

Document	Title
IEEE 802.15.4-2020	Low-Rate Wireless PANs (PHY and MAC)
Zigbee 3.0	Zigbee PRO stack and Zigbee Cluster Library
Zigbee Cluster Library (ZCL)	Application-layer cluster definitions
Zigbee Green Power	Ultra-low-power / energy-harvesting devices

See Also

- Z-Wave -- alternative home automation mesh (sub-GHz)
- 802.11s -- Wi-Fi mesh networking
- CAN -- wired alternative for industrial sensing

Z-Wave

Standard: ITU-T G.9959 / Z-Wave Specification | **Layer:** Full stack (Physical through Application) | **Wireshark filter:** `zwave`

Z-Wave is a low-power wireless mesh networking protocol designed for home automation. It operates in the sub-GHz ISM band (908.42 MHz in North America, 868.42 MHz in Europe), which provides better range and wall penetration than 2.4 GHz alternatives. Z-Wave supports up to 232 devices per network with a mandatory security layer (S2 framework). It is widely used for smart locks, sensors, switches, thermostats, and other IoT devices. Since 2020, the specification has been open through the Z-Wave Alliance.

Protocol Stack

[Diagram: see online version]

Frame (MAC Layer)

[Diagram: see online version]

Key Fields

Field	Size	Description
Home ID	32 bits	Identifies the Z-Wave network (set during inclusion)
Source Node ID	8 bits	Sender's node ID (1-232)
Frame Control	8 bits	Frame type, routed flag, ACK request, speed
Length	8 bits	Remaining bytes in the frame
Destination Node ID	8 bits	Recipient node ID (0xFF = broadcast)
Command Class	8 bits	Functional category (see below)
Command	8 bits	Specific operation within the class
Parameters	Variable	Command-specific data
Checksum	8 bits	XOR of all preceding bytes

Field Details

Radio Parameters

Parameter	Z-Wave	Z-Wave Long Range
Frequency (NA)	908.42 MHz	912 MHz
Frequency (EU)	868.42 MHz	868.4 MHz
Data rates	9.6 / 40 / 100 kbps	100 kbps
Modulation	FSK	DSSS-OQPSK
Range (indoor)	~30 m	~200 m
Range (outdoor)	~100 m	~1.6 km
Max nodes	232	4000 (with Long Range)
Mesh hops	Up to 4	Direct (no mesh)

Frame Types

Type	Description
Singlecast	Direct message to one node
Multicast	Message to a group of nodes
Broadcast	Message to all nodes (Node ID 0xFF)
Routed	Multi-hop message through repeater nodes
ACK	Acknowledgment of received frame

Common Command Classes

Class	ID	Description
Basic	0x20	Simple on/off/level control
Switch Binary	0x25	On/off switch
Switch Multilevel	0x26	Dimmable switch (0-99%)
Sensor Binary	0x30	Binary sensor (open/closed, motion)
Sensor Multilevel	0x31	Measured values (temperature, humidity, power)
Meter	0x32	Accumulated measurements (energy, water)
Thermostat Mode	0x40	HVAC mode (heat, cool, auto)
Thermostat Setpoint	0x43	Temperature setpoints
Door Lock	0x62	Lock/unlock control

Alarm/Notification	0x71	Alerts (smoke, CO, intrusion, water leak)
Battery	0x80	Battery level reporting
Wake Up	0x84	Sleepy device scheduling
Configuration	0x70	Device-specific parameters
Association	0x85	Direct device-to-device linking
Security 2	0x9F	S2 encryption and authentication

Security

Framework	Description
S0 (legacy)	AES-128 encryption, single network key
S2	AES-128-CCM, per-class keys, ECDH key exchange
S2 Access Control	Highest security (door locks, garage doors)
S2 Authenticated	Intermediate security (sensors, switches)
S2 Unauthenticated	Encrypted but no authentication pin

Mesh Routing

Z-Wave uses source routing -- the controller computes the route and includes it in the frame header:

[Diagram: see online version]

Maximum 4 hops per route. Explorer frames can discover new routes when existing ones fail.

Standards

Document	Title
ITU-T G.9959	Short range narrow-band digital radiocommunication
Z-Wave Specification	Z-Wave Alliance public specification
Z-Wave Long Range	Extended range specification (800 series)

See Also

- Zigbee -- alternative home automation mesh protocol (2.4 GHz)
- 802.11s -- Wi-Fi mesh networking